# Java Handbook — Vol 2

I/O, concurrency, JVM internals, modern Java 17–25, testing and tooling, coding interviews, cheat sheet, question bank, multithreading interview problems (Ch 13–22)

10 parts · one place to study: DesignGurus courses + classic books + the web, combined, beginner to intermediate

what it is · problem · how it works · trade-offs · real world · interview · self-check · TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

# Contents

1. **Chapter 13 — I/O and NIO**
    - 13.1 The big picture: `java.io` vs NIO.2
    - 13.2 `Path`: naming files and directories
    - 13.3 `Files`: managing files and directories
    - 13.4 Reading and writing whole files
    - 13.5 Byte streams and character streams
    - 13.6 Buffering
    - 13.7 Charsets and encodings
    - 13.8 Console and standard streams
    - 13.9 Binary data, channels and buffers
    - 13.10 Serialization (and why to avoid it)
    - 13.11 Properties files
    - 13.12 Working with JSON
    - 13.13 Best practices and common mistakes (checklist)
    - Chapter TL;DR
2. **Chapter 14 — Concurrency I: Threads, Locks and the `java.util.concurrent` Toolkit**
    - 14.1 Concurrency basics
    - 14.2 Creating and starting threads
    - 14.3 Thread lifecycle
    - 14.4 Interruption: cooperative cancellation
    - 14.5 Race conditions
    - 14.6 `synchronized` and intrinsic locks
    - 14.7 `volatile` and visibility
    - 14.8 `wait` / `notify`: guarded blocks
    - 14.9 Explicit locks: `ReentrantLock`, `Condition`, `ReadWriteLock`, `StampedLock`
    - 14.10 Atomic variables
    - 14.11 Executors and thread pools
    - 14.12 Concurrent collections
    - 14.13 Synchronizers: `CountDownLatch`, `CyclicBarrier`, `Semaphore`, `Phaser`
    - 14.14 Designing thread-safe classes
    - Chapter TL;DR
3. **Chapter 15 — Concurrency II: Modern Concurrency**
    - 15.1 `Future` and its limits
    - 15.2 `CompletableFuture`
    - 15.3 Fork/Join
    - 15.4 Virtual threads
    - 15.5 Pinning and JEP 491
    - 15.6 `ThreadLocal` and scoped values
    - 15.7 Structured concurrency (preview)
    - 15.8 The Java Memory Model and happens-before
    - 15.9 Liveness: deadlock, livelock, starvation
    - 15.10 Best practices and choosing a concurrency model
    - Chapter TL;DR
4. **Chapter 16 — JVM Internals and Performance**
    - 16.1 The JVM's architecture
    - 16.2 Class loading in depth
    - 16.3 Runtime memory areas
    - 16.4 Object layout basics
    - 16.5 Garbage-collection algorithms
    - 16.6 Collectors compared on a real workload
    - 16.7 GC tuning basics
    - 16.8 JIT compilation, inlining and escape analysis
    - 16.9 Benchmarking: why naive timing lies, and JMH
    - 16.10 Profiling and diagnostic tools
    - 16.11 Memory leaks
    - 16.12 Modules (JPMS) basics
    - 16.13 Operating-system fundamentals for engineers
    - Chapter TL;DR
5. **Chapter 17 — Modern Java: 17 to 25**
    - 17.1 Reading a feature's status
    - 17.2 The version table
    - 17.3 The Java 17 baseline
    - 17.4 Java 18
    - 17.5 Java 19 and 20: the preview years
    - 17.6 Java 21 LTS
    - 17.7 Java 22
    - 17.8 Java 23
    - 17.9 Java 24
    - 17.10 Java 25 LTS
    - 17.11 Still preview or incubating in Java 25 — and what came next
    - 17.12 Upgrading 17 → 21 → 25: removals and deprecations
    - 17.13 In the interview
    - Chapter TL;DR
6. **Chapter 18 — Testing, Build and Tooling**
    - 18.1 The shape of a real Java project
    - 18.2 Maven
    - 18.3 Gradle
    - 18.4 JUnit: your first automated tests
    - 18.5 Parameterized, repeated and dynamic tests
    - 18.6 Test doubles and Mockito
    - 18.7 AssertJ fluent assertions
    - 18.8 Reading a failing test report
    - 18.9 Test design
    - 18.10 Logging
    - 18.11 JDBC basics
    - 18.12 Packaging: JAR files
    - 18.13 Other everyday tools
    - 18.14 Where next: Spring Boot and friends
    - Chapter TL;DR
7. **Chapter 19 — Java for Coding Interviews**
8. **Part 1 — The Java DSA toolkit**

  - 19.1 Testing your solutions: `assert` is off, use a helper
  - 19.2 Arrays, strings and char arithmetic
  - 19.3 `HashMap` and `HashSet` idioms
  - 19.4 `ArrayDeque`, `PriorityQueue` and `TreeMap`
  - 19.5 Sorting with comparators
  - 19.6 Numeric and equality pitfalls
  - 19.7 Complexity cheat table

9. **Part 2 — Twenty patterns**
  - 19.8 Two pointers
  - 19.9 Sliding window
  - 19.10 Hashing
  - 19.11 Prefix sums
  - 19.12 Stacks and monotonic stacks
  - 19.13 Heaps and top-k
  - 19.14 Binary search (and binary search on the answer)
  - 19.15 Intervals
  - 19.16 BFS and DFS: trees, grids and graphs
  - 19.17 Topological sort
  - 19.18 Union-find (disjoint set union)
  - 19.19 Dynamic programming
  - 19.20 Backtracking
  - 19.21 Tries
  - 19.22 Linked lists and fast/slow pointers
  - 19.23 Trees: recursion templates, LCA and serialisation
  - 19.24 Greedy
  - 19.25 Sort and search: quicksort, quickselect, Dutch flag, merge sort on a list
  - 19.26 Bit manipulation
  - 19.27 Math and geometry
  - 19.28 Talking through a solution in Java
10. **Part 3 — Java interview questions**
  - 19.29 Java interview Q&A
  - Chapter TL;DR
11. **Chapter 20 — Cheat Sheets and Glossary**
  - 20.1 Syntax, types and primitive ranges
  - 20.2 Operators and precedence
  - 20.3 Control flow, `switch` and patterns
  - 20.4 Methods and arrays
  - 20.5 OOP: classes, interfaces, records, enums and sealed types
  - 20.6 Core APIs: String, numbers, dates
  - 20.7 Exceptions
  - 20.8 Generics and PECS
  - 20.9 Collections: big-O and which to choose
  - 20.10 Functional interfaces, lambdas and `Optional`
  - 20.11 Streams and collectors
  - 20.12 I/O and NIO.2
  - 20.13 Concurrency
  - 20.14 The JVM: memory, GC, flags and tools
  - 20.15 Modern Java by version: 17 to 25
  - 20.16 Testing, build and tooling commands
  - 20.17 Interview question index
  - 20.18 Where next
  - 20.19 Glossary (A–Z)
  - Chapter TL;DR
12. **Chapter 21 — Interview Question Bank**
  - How to use this bank
  - 1. Platform and language basics
  - 2. OOP: classes, inheritance and polymorphism
  - 3. Strings
  - 4. equals, hashCode, immutability and cloning
  - 5. Exceptions
  - 6. Generics
  - 7. Collections
  - 8. Lambdas, functional interfaces and Optional
  - 9. Streams
  - 10. Modern Java (9 → 25)
  - 11. Concurrency
  - 12. JVM, memory and garbage collection
  - 13. I/O, serialization, reflection and annotations
  - 14. Testing and tooling
  - 15. Scenario and senior-level questions
  - 16. "What does this print?" output puzzles
  - 17. Coding tasks asked in Java rounds
  - 18. DSA problems in Java rounds
  - 19. Multithreading problems (Chapter 22)
  - Sources of the question lists
13. **Chapter 22 — Multithreading Interview Problems**
  - 22.1 Blocking queue / bounded buffer — four ways
  - 22.2 Implement a counting semaphore
  - 22.3 Implement a read-write lock
  - 22.4 Implement a reusable barrier
  - 22.5 Thread-safe deferred callback executor
  - 22.6 Asynchronous to synchronous
  - 22.7 Ordered printing (three threads print in order)
  - 22.8 Token-bucket rate limiter — blocking and background-refill versions
  - 22.9 Unisex bathroom
  - 22.10 Uber ride (riders of two parties, safe car combinations)
  - 22.11 Sleeping barber
  - 22.12 Multithreaded merge sort with plain threads (and why Fork/Join is better)
  - 22.13 Problems covered elsewhere in this book and in the LLD book

# Chapter 13 — I/O and NIO

Almost every program reads or writes something outside itself: config files, CSV exports, logs, uploads, the keyboard, the terminal. **I/O** (input/output) moves data across that boundary. Java has two generations of file APIs, met side by side in real code: the original **`java.io`** package (Java 1.0, *streams* of bytes and characters) and **NIO.2** (`java.nio.file`, Java 7, `Path` + the `Files` utility class). This chapter covers both, which to reach for, and the production trip-ups: buffering, character encodings, serialization and configuration files. It leans on try-with-resources and `IOException` (Chapter 8) and lazy streams (Chapter 12 — `Files.lines` and `Files.walk` return `Stream`s).

All runnable code is in `code/` (package `ch13.*`), compiled with `javac --release 25` and run on JDK 26. Every program works inside a `Files.createTempDirectory` directory and deletes it at the end. Outputs are pasted from real runs.

> **Exam depth.** `File` vs `Path` method-name differences, `resolve`/`relativize` puzzles, `Files` exception rules, stream class tables and `Console` details: OCP Chapter 14 — I/O. This chapter stays at "use it correctly at work" depth and links there for traps.

---

## 13.1 The big picture: `java.io` vs NIO.2

### What it is

- **`java.io`** (Java 1.0) models I/O as **streams**: a one-way flow of data you read from (*input stream*) or write to (*output stream*), piece after piece. Files are the old `java.io.File` class. (Unrelated to Chapter 12's `java.util.stream.Stream` — same word, different idea.)
- **NIO** ("New I/O", Java 1.4, `java.nio`) added **buffers** (`ByteBuffer`) and **channels** (`FileChannel`, `SocketChannel`) — lower-level, block-oriented (data in chunks), and capable of **non-blocking** network I/O (a read/write returns immediately even if no data is ready, instead of making the thread wait).
- **NIO.2** (Java 7, `java.nio.file`) replaced `File` with the **`Path`** interface and the **`Files`** class of static helpers, plus file attributes, **symbolic links** (symlinks: small files pointing to another file or directory), directory walking and change watching.

### Why it matters

Both APIs are current and interoperate. Modern code uses `Path` + `Files` for everything about *files*, and `java.io` streams whenever data flows from something that isn't a file (network **sockets**, HTTP bodies, zip entries, in-memory arrays) — `Files.newInputStream(path)` even *returns* a `java.io.InputStream`.

| Need | Use | Since |
|---|---|---|
| Represent a file/directory location | `Path` (`Path.of(...)`) | 7 (`Path.of` 11) |
| Create, copy, move, delete, check, list | `Files.*` static methods | 7 |
| Read/write a small text file in one call | `Files.readString` / `Files.writeString` | 11 |
| Read a huge text file line by line | `Files.lines` (lazy `Stream`) or `Files.newBufferedReader` | 8 / 7 |
| Process bytes from any source (socket, zip, HTTP) | `InputStream` / `OutputStream` | 1.0 |
| Process text from any source | `Reader` / `Writer` (+ a charset) | 1.1 |
| Random access (jump to any position), big files, memory mapping | `FileChannel` + `ByteBuffer` | 1.4 |
| Legacy APIs that want a `File` | `path.toFile()` / `file.toPath()` | 7 |

### Why `File` was replaced

The old `File` class has real flaws that NIO.2 fixes:

1. **Silent failures.** `file.delete()`, `mkdir()` and `renameTo()` return `false` without saying *why* (missing? permission? not empty?). `Files.delete` throws a specific exception (`NoSuchFileException`, `DirectoryNotEmptyException`, `AccessDeniedException`).
2. **No symbolic-link support, poor metadata** (size, dates, owner). NIO.2 reads attributes (size, times, POSIX permissions — Unix read/write/execute flags for owner, group and others — owner) in one call.
3. **No efficient directory walking.** `File.listFiles()` loads everything into an array; `Files.walk`/`Files.list` stream lazily.

### TL;DR

Use `Path` + `Files` for files and directories; use `InputStream`/`Reader` families for data flowing from anywhere; convert with `toFile()`/`toPath()` only at legacy boundaries.

---

## 13.2 `Path`: naming files and directories

### What it is

A `Path` is an **immutable** object representing a file-system location as a sequence of **name elements** (`reports`, `2025`, `q3`, `sales.csv`) plus an optional **root** (`/` on Unix/macOS, `C:\` on Windows). A `Path` is *just a name*: creating one never checks that the file exists or touches the disk — `Path.of("missing.txt")` doesn't throw.

Key terms:

- **Absolute path** — starts at a root (`/srv/app/logs`). **Relative path** — interpreted from the JVM process's **current working directory** (the folder the program was started from) (`reports/q3.csv`).
- **Separator** — `/` on Unix/macOS, `\` on Windows. `Path.of("a", "b")` inserts the right one; don't concatenate with `"/"` yourself.
- `.` means "this directory", `..` means "the parent directory".

### How it works

1. Create a path: `Path.of("reports", "2025", "sales.csv")` (Java 11) or the older, identical `Paths.get(...)` (Java 7).
2. Take it apart: `getFileName()`, `getParent()`, `getRoot()`, `getNameCount()`, `getName(i)`, `subpath(from, to)`.
3. Combine paths: `resolve` (append), `resolveSibling` (replace the last element), `relativize` (the route from one path to another).
4. Clean it: `normalize()` removes `.` and resolves `name/..` pairs *textually*; `toAbsolutePath()` prefixes the working directory; `toRealPath()` asks the file system (resolves symlinks and **requires the file to exist**).

`code/ch13/paths/PathBasics.java` (key parts):

```java
Path p = Path.of("reports", "2025", "q3", "sales.csv");
p.getFileName(); p.getParent(); p.getRoot(); p.getNameCount(); p.getName(1); p.subpath(1, 3);

Path base = Path.of("/srv/app");
base.resolve("logs/app.log");                 // append a relative path
base.resolve("/etc/hosts");                   // an absolute argument simply wins
base.resolve("config.yml").resolveSibling("secrets.yml");

Path.of("/srv/app/./logs/../data/./users.db").normalize();
Path from = Path.of("/srv/app/logs"), to = Path.of("/srv/app/data/users.db");
from.relativize(to);                          // how do I get from 'from' to 'to'?
to.startsWith("/srv/ap");                     // compares whole NAME elements, not characters
```

Output:

```
path          = reports/2025/q3/sales.csv
fileName      = sales.csv
parent        = reports/2025/q3
root          = null
nameCount     = 4
name(1)       = 2025
subpath(1,3)  = 2025/q3
isAbsolute    = false
resolve       = /srv/app/logs/app.log
resolve abs   = /etc/hosts
resolveSibling= /srv/app/secrets.yml
normalize     = /srv/app/data/users.db
relativize    = ../data/users.db
round trip    = /srv/app/data/users.db
startsWith    = true / string-wise 'app' = false
endsWith      = true
File <-> Path = reports/2025/q3/sales.csv -> true
Paths.get     = true
```

What to notice:

- `getRoot()` is `null` for a relative path, and `getName(0)` is the first element *after* the root.
- `subpath(1, 3)` is **half-open** (includes the start index, excludes the end) like `String.substring`: elements 1 and 2.
- `resolve` with an absolute argument returns the argument (`Path.of("/a/b").resolve("/c")` → `/c`) — a classic bug when a user-supplied "file name" starts with `/`.
- `relativize` requires both paths to be the same kind (both absolute or both relative), otherwise `IllegalArgumentException`; `Path.of("/a/b").relativize(Path.of("/a/c/d"))` → `../c/d`.
- `startsWith("/srv/ap")` is `false`: comparison is by name elements, not characters.

### Common mistakes

- Building paths by string concatenation (`dir + "/" + name`) — breaks on Windows and doubles separators. Use `resolve`.
- Assuming `normalize()` checks the disk. It is purely textual; `a/link/..` normalises to `a` even if `link` is a symlink pointing elsewhere. Use `toRealPath()` when it matters.
- **Path traversal** (an attack where `..` segments in input escape the allowed folder): `uploads.resolve(userInput)` with `userInput = ../../etc/passwd`. Guard with `Path target = uploads.resolve(name).normalize(); if (!target.startsWith(uploads)) throw ...` — exactly the check web servers do.

---

## 13.3 `Files`: managing files and directories

### What it is

`java.nio.file.Files` is a class of static methods that *do* touch the disk: test, create, copy, move, delete, read attributes, list and walk. Every method that can fail throws a specific subclass of the checked `IOException` (§8.3).

### How it works — the core operations

| Task | Method | Fails with (if…) |
|---|---|---|
| Does it exist? | `exists`, `notExists`, `isRegularFile`, `isDirectory`, `isReadable`, `isSymbolicLink` | never throws; returns `false` if unsure |
| Make a directory | `createDirectory(p)` / `createDirectories(p)` | `FileAlreadyExistsException` / parents made too, no error if present |
| Make an empty file | `createFile(p)` | `FileAlreadyExistsException` |
| Temporary file/dir | `createTempFile(prefix, suffix)`, `createTempDirectory(prefix)` | in `java.io.tmpdir` |
| Copy | `copy(src, dst, options...)` | `FileAlreadyExistsException` unless `REPLACE_EXISTING` |
| Move / rename | `move(src, dst, options...)` | same; add `ATOMIC_MOVE` for all-or-nothing (**atomic**: others see the old state or the new one, never half-done) within one file system |
| Delete | `delete(p)` / `deleteIfExists(p)` | `NoSuchFileException`, `DirectoryNotEmptyException` / returns `boolean` |
| Size, times, type | `size`, `getLastModifiedTime`, `readAttributes(p, BasicFileAttributes.class)` | `NoSuchFileException` |
| Same content? | `mismatch(a, b)` (Java 12) → `-1` if identical, else first differing byte index | |
| Same file? | `isSameFile(a, b)` — follows links | |
| One level listing | `list(dir)` → `Stream<Path>` (lazy, **close it**) | `NotDirectoryException` |
| Recursive walk | `walk(dir[, maxDepth])`, `find(dir, maxDepth, (path, attrs) -> …)` → `Stream<Path>` | |
| Fine-grained walk with callbacks | `walkFileTree(dir, FileVisitor)` | |

Details worth knowing:

- `copy` of a directory copies only the directory itself (empty), not its contents. Recursive copy = `walk` + `copy` each entry.
- `walk` visits **depth-first** (down one branch fully before the next), starts with the directory itself, and by default does **not** follow symbolic links (pass `FileVisitOption.FOLLOW_LINKS`).
- `readAttributes(..., BasicFileAttributes.class)` fetches size, times and type in **one** file-system call — much cheaper than calling `size`, `isDirectory` and `getLastModifiedTime` separately in a loop over thousands of files.

`code/ch13/files/FileOps.java` (key parts):

```
Path root = Files.createTempDirectory("java1315_fileops");
try {
    Path docs = Files.createDirectories(root.resolve("docs/2025"));   // makes parents too
    Path a = Files.writeString(docs.resolve("a.txt"), "alpha\n");
    Files.createDirectory(docs);                                       // -> FileAlreadyExistsException
    Path b = Files.copy(a, docs.resolve("b.txt"));
    Files.copy(a, b);                                                  // -> FileAlreadyExistsException
    Files.copy(a, b, StandardCopyOption.REPLACE_EXISTING);            // allowed
    Path moved = Files.move(b, archive.resolve("b-old.txt"));         // move = rename
    Files.mismatch(a, moved) == -1;                                    // identical content
    BasicFileAttributes attrs = Files.readAttributes(a, BasicFileAttributes.class);

    try (Stream<Path> s = Files.walk(root)) {                          // lazy + holds OS handles
        s.map(p -> root.relativize(p).toString()).sorted().forEach(...);
    }
    try (Stream<Path> s = Files.find(root, 10,
            (p, at) -> at.isRegularFile() && p.toString().endsWith(".md"))) { ... }
    Files.delete(docs);                                                // -> DirectoryNotEmptyException
} finally {
    deleteRecursively(root);
}

static void deleteRecursively(Path root) throws IOException {
    try (Stream<Path> s = Files.walk(root)) {
        List<Path> all = s.sorted(Comparator.reverseOrder()).toList();  // children before parents
        for (Path p : all) Files.delete(p);
    }
}
```

Output:

```
exists a.txt?        true
isDirectory docs?    true
size a.txt           6 bytes
createDirectory again -> FileAlreadyExistsException
copy onto existing   -> FileAlreadyExistsException
after move b exists? false, moved exists? true
same content a/moved true
attrs: regular=true dir=false size=6 created<=modified true
-- Files.list(root) (one level, sorted):
   archive
   docs
   readme.md
-- Files.walk(root) (recursive, sorted):
   ''
   'archive'
   'archive/b-old.txt'
   'docs'
   'docs/2025'
   'docs/2025/a.txt'
   'docs/2025/notes.md'
   'readme.md'
-- Files.find *.md:
   docs/2025/notes.md
   readme.md
delete non-empty dir -> DirectoryNotEmptyException
deleteIfExists(missing) = false
read missing file    -> NoSuchFileException
temp dir removed?    true
```

- The empty string `''` in the walk output is the root itself (`root.relativize(root)`), proving that `walk` includes its starting point.
- We **sort** listing output because the OS order is unspecified; never rely on it (in tests either).
- `deleteRecursively` sorts paths in reverse so `docs/2025/a.txt` comes before `docs/2025`, which comes before `docs` — a directory can only be deleted once it's empty. (`walkFileTree` with a `postVisitDirectory` callback is the other standard way.)

### Watching a directory for changes

`WatchService` (Java 7) notifies a program when files in a directory are created, modified or deleted — how IDEs and dev servers notice edits:

```
// (not run: event delivery timing is OS-dependent; macOS uses polling with a delay of seconds)
try (WatchService ws = FileSystems.getDefault().newWatchService()) {
    dir.register(ws, StandardWatchEventKinds.ENTRY_CREATE, StandardWatchEventKinds.ENTRY_MODIFY);
    WatchKey key = ws.take();                         // blocks until something happens
    for (WatchEvent<?> e : key.pollEvents()) System.out.println(e.kind() + " " + e.context());
    key.reset();                                      // re-arm, or no more events arrive
}
```

It watches a single directory (not subdirectories), and events can be coalesced (merged) or overflow (`OVERFLOW` kind), so treat events as hints and re-scan when in doubt.

### Common mistakes

- **Not closing** `Files.list` / `walk` / `find` / `lines`. They hold an open directory or **file handle** (an OS token for an open file; each process gets a limited number); a long-running server leaking them eventually hits "Too many open files". Always use try-with-resources.
- **Check-then-act**: `if (!Files.exists(p)) Files.createFile(p);` — another process can create it in between (a **race condition**). Call `createFile` and catch `FileAlreadyExistsException` instead (or use `CREATE_NEW`, §13.4).
- Using `Files.exists` as proof: it returns `false` both for "doesn't exist" and "can't tell" (no permission). `Files.notExists` is not simply its negation — both can be `false`.
- Hard-coding `/tmp` — use `Files.createTempDirectory` (portable, unique name).

### Real-world examples

Log rotation (rename the current log and start a fresh one; `move` with `ATOMIC_MOVE` so readers never see half-written files), build tools walking source trees (`walk` + `find`), and "write to a temp file, then atomically move over the real file" — the standard trick for crash-safe config and cache updates.

### In the interview

> **30-second answer:** "I use `Path.of` and the `Files` class: `createDirectories`, `copy` / `move` with explicit `StandardCopyOption`s, `delete` which throws a descriptive exception, and `walk` / `list` / `find` inside try-with-resources because they hold OS handles. For crash-safe updates I write to a temp file in the same directory and `move` it with `ATOMIC_MOVE`."

- *Why prefer `Files.delete` over `File.delete`?* → It throws a specific exception explaining the failure; `File.delete` just returns `false`.
- *How do you delete a directory tree?* → `walk`, sort in reverse order, delete each; or `walkFileTree` deleting in `postVisitDirectory`.
- *Is `walk` depth-first or breadth-first?* → Depth-first; it includes the start directory.

---

## 13.4 Reading and writing whole files

### What it is

For everyday text files, `Files` has one-line methods. They default to **UTF-8** (§13.7; true for `Files` methods even before Java 18) and open/close the file for you.

| Method | Reads/writes | Memory | Since |
|---|---|---|---|
| `readString(p[, cs])` / `writeString(p, text[, cs], opts...)` | whole file as one `String` | whole file | 11 |
| `readAllLines(p)` / `write(p, lines, opts...)` | `List<String>` of lines | whole file | 7 |
| `readAllBytes(p)` / `write(p, bytes, opts...)` | `byte[]` | whole file | 7 |
| `lines(p)` | lazy `Stream<String>` — **must be closed** | one line at a time | 8 |
| `newBufferedReader(p)` / `newBufferedWriter(p, opts...)` | `BufferedReader` / `BufferedWriter` | small buffer | 7 |
| `newInputStream(p)` / `newOutputStream(p, opts...)` | raw bytes | small buffer | 7 |

**Open options** (`StandardOpenOption`) control what "write" means. The default for the write methods is `CREATE`, `TRUNCATE_EXISTING`, `WRITE` — create if missing, **otherwise wipe it first** (*truncate* to zero length). Others: `APPEND`, `CREATE_NEW` (fail if it exists — an atomic "create only if absent"), `READ`, `SYNC` / `DSYNC` (force every write to the storage device — `DSYNC` syncs only the contents, `SYNC` also the metadata), `DELETE_ON_CLOSE`.

`code/ch13/files/ReadWrite.java` (key parts):

```
Files.writeString(f, "buy milk\n");                              // create or truncate
Files.writeString(f, "call bob\n", StandardOpenOption.APPEND);
Files.write(f, List.of("fix bug #42", "ship v2"), StandardOpenOption.APPEND);
Files.readString(f);
List<String> lines = Files.readAllLines(f);
try (Stream<String> s = Files.lines(f)) {                           // lazy; MUST be closed
    long withB = s.filter(l -> l.contains("b")).count();
}
Files.writeString(f, "x", StandardOpenOption.CREATE_NEW);          // FileAlreadyExistsException

try (BufferedWriter w = Files.newBufferedWriter(csv, StandardCharsets.UTF_8)) {
    w.write("name,score"); w.newLine();
    w.write("ana,91");     w.newLine();
    w.write("raj,78");     w.newLine();
}                                                                  // close() flushes the buffer
try (BufferedReader r = Files.newBufferedReader(csv)) {
    r.readLine();                                                  // skip header
    String line;
    while ((line = r.readLine()) != null) total += Integer.parseInt(line.split(",")[1]);
}
```

Output:

```
readString:
buy milk
call bob
fix bug #42
ship v2
readAllLines size = 4 -> [buy milk, call bob, fix bug #42, ship v2]
lines containing 'b' = 3
bytes = 38
CREATE_NEW on existing -> FileAlreadyExistsException
total score = 169
```

- `Files.write(path, lines)` writes each line followed by the platform line separator (`\n` here). `readAllLines` strips line terminators.
- `readLine()` returns `null` at end of file — the classic `while ((line = r.readLine()) != null)` idiom.

### Trade-offs

| Style | Pros | Cons |
|---|---|---|
| `readString` / `readAllLines` | shortest code | loads the whole file; a 4 GB log → `OutOfMemoryError` |
| `lines()` | constant memory, stream operations | must remember try-with-resources; `UncheckedIOException` if a read fails mid-stream |
| `BufferedReader` loop | constant memory, full control, easy early exit | more code |

Rule of thumb: whole-file methods for files known to be small (configs, test fixtures, < a few MB); streaming for anything user-supplied or unbounded.

### Common mistakes

- Forgetting that the default write **truncates** — appending needs `APPEND`.
- Writing a CSV "by hand" with `split(",")` for real data: quoted fields containing commas break it. Use a CSV library (Apache Commons CSV, OpenCSV) for anything external.
- Writing important files in place. If the program crashes mid-write the file is corrupt. Write a temp file in the same directory, then `Files.move(tmp, target, ATOMIC_MOVE, REPLACE_EXISTING)`.

### TL;DR

`readString`/`writeString` for small text, `lines`/`newBufferedReader` for big text, and remember: writes truncate unless you say `APPEND`.

---

## 13.5 Byte streams and character streams

### What it is

In `java.io` all I/O is built from four abstract classes:

| | Input (read) | Output (write) | Unit |
|---|---|---|---|
| **Byte streams** | `InputStream` | `OutputStream` | `byte` (8-bit raw data: images, zip, protobuf, anything) |
| **Character streams** | `Reader` | `Writer` | `char` (16-bit UTF-16 text unit; see §13.7) |

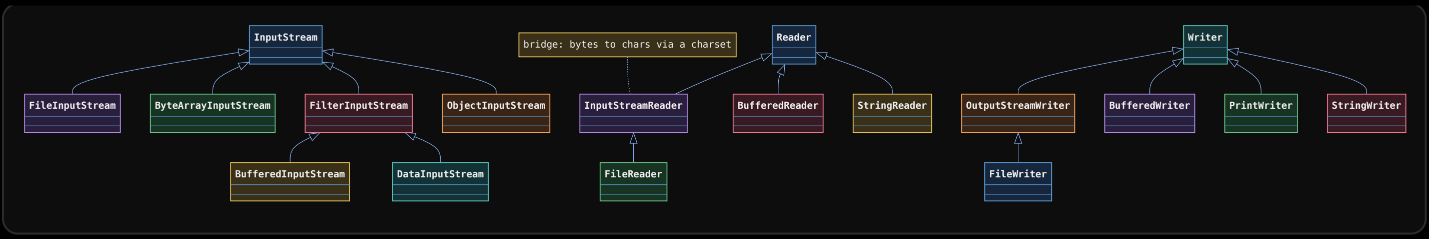
Byte streams move raw bytes. Character streams move text, and at the point where text meets bytes (a file, a socket) something must **encode** chars to bytes or **decode** bytes to chars using a **charset** (§13.7). The **bridge** classes are `InputStreamReader` (bytes → chars) and `OutputStreamWriter` (chars → bytes).

### How it works: the decorator pattern

Concrete classes come in two kinds:

1. **Source/sink streams** that actually connect to something: `FileInputStream`, `ByteArrayInputStream`, a socket's stream, `FileReader`, `StringReader`, `StringWriter`.
2. **Filter (wrapper) streams** that take another stream in their constructor and add a feature: buffering (`BufferedInputStream`, `BufferedReader`), primitives (`DataInputStream`), objects (`ObjectInputStream`), compression (`GZIPInputStream`), formatting (`PrintStream`, `PrintWriter`).

You stack wrappers like nested boxes. Each `read`/`write` passes through the layers, and **closing the outermost wrapper closes the whole chain** (e.g. a `BufferedReader` over an `InputStreamReader`). This is the **Decorator** pattern (wrap an object in another of the same type that adds behaviour, LLD Chapter 9).


InputStream
bridge: bytes to chars via a charset
Reader
Writer
FileInputStream
ByteArrayInputStream
FilterInputStream
ObjectInputStream
InputStreamReader
BufferedReader
StringReader
OutputStreamWriter
BufferedWriter
PrintWriter
StringWriter
BufferedInputStream
DataInputStream
FileReader
FileWriter

(The output side mirrors this: `OutputStream` → `FileOutputStream`, `ByteArrayOutputStream`, `BufferedOutputStream`, `DataOutputStream`, `ObjectOutputStream`, `PrintStream`.)

The read contracts, which every subclass follows:

- `int read()` returns the next byte (0–255) or char (0–65535) **as an** `int`, or `-1` **at end of stream**. It returns `int` rather than `byte` precisely so `-1` can't be confused with a real value.
- `int read(byte[] buf)` fills *up to* `buf.length` bytes and returns **how many it actually read** (possibly fewer!) or `-1`. Always write `buf, 0, n`, never the whole array.
- Reads **block**: the thread waits until data arrives, the stream ends, or an error occurs.

`code/ch13/streams/ByteAndCharStreams.java` (key parts):

```java
// 1) the classic copy loop
try (InputStream in = new FileInputStream(src.toFile());
     OutputStream out = new FileOutputStream(dst.toFile())) {
    byte[] buf = new byte[4];                   // tiny, to make the loop visible
    int n;
    while ((n = in.read(buf)) != -1) out.write(buf, 0, n);
}
// 2) Java 9: let the library do the loop
in.transferTo(out);
// 3) characters: specify the charset at the byte boundary
try (Writer w = new OutputStreamWriter(new FileOutputStream(text.toFile()), StandardCharsets.UTF_8)) {
    w.write("naïve café ☕");
}
// 4) decorators: text -> buffered -> encoded -> gzipped -> file
try (PrintWriter pw = new PrintWriter(new BufferedWriter(new OutputStreamWriter(
        new GZIPOutputStream(new FileOutputStream(gz.toFile())), StandardCharsets.UTF_8)))) {
    for (int i = 1; i <= 1000; i++) pw.printf("line %d: all work and no play%n", i);
}
// 5) in memory
ByteArrayOutputStream bos = new ByteArrayOutputStream();
new StringReader("abc").transferTo(sw);         // Reader.transferTo: Java 10
```

Output:

```
copied with 4 read() calls -> Hello, bytes!
transferTo copied 13 bytes
chars written = 12, bytes on disk = 16
chars read back = 12
gzip file 2453 bytes holds 1000 lines
ByteArrayOutputStream -> captured
StringReader -> StringWriter: abc
```

- 13 bytes through a 4-byte buffer = 4 reads (4+4+4+1); the last `write(buf, 0, 1)` writes only one byte.
- 12 characters became 16 bytes: `ï` and `é` take 2 bytes in UTF-8, `☕` takes 3.
- ~30 KB of repetitive text compressed to about 2.4 KB — and the code reading it back just adds `GZIPInputStream` to the chain.

### Useful extras (Java 9–25)

| Method | What it does | Since |
|---|---|---|
| `InputStream.readAllBytes()`, `readNBytes(n)` | read everything / exactly up to n bytes | 9 / 11 |
| `InputStream.transferTo(out)`, `Reader.transferTo(w)` | copy to the end | 9 / 10 |
| `InputStream.nullInputStream()`, `OutputStream.nullOutputStream()` | empty source / discard sink (tests) | 11 |
| `Reader.of(CharSequence)` | a `Reader` over a string | 24 |
| `Reader.readAllAsString()`, `readAllLines()` | read the rest of a reader | 25 |

`mark`/`reset` (remember a position and go back) and `skip` also exist; only some streams support `mark` (check `markSupported()`) — details in OCP Chapter 14 §8a.

### Common mistakes

- `FileReader`/`FileWriter` **without a charset** on Java 17 and earlier used the platform default (often Windows-1252 on Windows) — files written on one machine read wrongly on another. On Java 11+ pass the charset: `new FileReader(file, StandardCharsets.UTF_8)`.
- Ignoring the return value of `read(buf)` and writing the whole buffer → garbage at the end of the copy.
- Closing only the inner stream, or not closing at all — buffered data is lost and the file handle leaks. Declare the outermost wrapper in try-with-resources.
- Using `PrintWriter`/`PrintStream` for important output: they **swallow `IOException`s** (check `checkError()`), which is fine for the console but not for writing a file you need.

### In the interview

> **30-second answer:** "`java.io` has byte streams, `InputStream`/`OutputStream`, and character streams, `Reader`/`Writer`, bridged by `InputStreamReader`/`OutputStreamWriter` with an explicit charset. Features are added by wrapping — buffering, data types, compression — which is the Decorator pattern. `read` returns -1 at end of stream, and I always close the outermost wrapper with try-with-resources."

- *Why does `read()` return `int`?* → To fit all 256 byte values plus `-1` as the end marker.
- *Byte or character stream for a PNG?* → Byte stream; it isn't text.

---

## 13.6 Buffering

### What it is

A **buffer** is a block of memory (8 KiB by default for the `Buffered*` classes) that collects many small reads or writes into one large operation. Without it, each `write('x')` on a `FileOutputStream` becomes a **system call** — a request to the OS **kernel**, which costs far more than copying a byte in memory.

### How it works

1. `BufferedOutputStream.write(b)` puts the byte in its internal array.
2. When the array is full, it writes the whole block to the wrapped stream in one call.
3. `flush()` forces out whatever is buffered now; `close()` flushes then closes.
4. On input, `BufferedInputStream`/`BufferedReader` read a large block at once and hand out pieces from memory; `BufferedReader` also adds `readLine()` and `lines()`.

`code/ch13/streams/BufferingDemo.java` (key parts):

```
try (OutputStream out = new FileOutputStream(f1.toFile())) {
    for (int i = 0; i < 200_000; i++) out.write('x');          // one system call per byte
}
try (OutputStream out = new BufferedOutputStream(new FileOutputStream(f2.toFile()))) {
    for (int i = 0; i < 200_000; i++) out.write('x');          // one system call per 8 KiB
}
BufferedWriter w = new BufferedWriter(new FileWriter(lost.toFile()));
w.write("this sits in the buffer");
Files.size(lost);   // 0 !
w.flush();
Files.size(lost);   // 23
```

Output (timings are machine-dependent; the ratio is the lesson):

```
same size: true (200000 bytes)
unbuffered: 205 ms   [machine-dependent]
buffered:   2 ms   [machine-dependent]
buffered faster? true
before flush: 0 bytes on disk
after flush:  23 bytes on disk
```

Same 200,000 bytes, roughly 100× faster with a buffer on this machine. And until `flush()`/`close()`, written data lives only in the buffer: if the JVM crashes or you forget to close, it never reaches the file.

### When not to use it

- When you already read/write big arrays (`read(byte[8192])`, `Files.readAllBytes`, `transferTo`) — a buffer adds only a copy.
- The `Files` convenience methods and `Files.newBufferedReader/Writer` already buffer.
- Interactive **protocols** where each message must go out immediately — you can still buffer, but call `flush()` after each message.

### Common mistakes

- Forgetting `flush()` on a socket writer → the other side waits forever for a message sitting in your buffer.
- Wrapping twice (`new BufferedReader(Files.newBufferedReader(p))`) — harmless but pointless.
- `flush()` hands data to the OS, not necessarily to the physical disk. For **durability** (surviving a crash or power cut) use `FileChannel.force(true)` or the `SYNC`/`DSYNC` open options (databases do this).

### TL;DR

Always buffer small reads/writes; `close()` flushes; `flush()` ≠ on disk.

---

## 13.7 Charsets and encodings

### What it is

Files and networks carry **bytes**; Java `String`s hold **characters** (as UTF-16 `char` units). A **charset** (character encoding) is the rulebook that maps between them.

- **UTF-8** — variable length: 1 byte for ASCII (the basic 128 English letters, digits and symbols), 2 for most European accents, 3 for most other symbols (`€`, CJK), 4 for emoji. The web's and Java's default.
- **UTF-16** — 2 or 4 bytes per character; what `String` uses internally (conceptually).
- **ISO-8859-1** (Latin-1) — exactly 1 byte per character, only 256 characters.
- **US-ASCII** — 7-bit, 128 characters.

**Mojibake** is the garbled text you get when bytes are decoded with a different charset from the one used to encode them (`cafÃ©`).

### Why it matters: JEP 400 (Java 18)

Before Java 18, APIs without an explicit charset (`FileReader`, `new String(bytes)`, `String.getBytes()`, `Scanner(File)`) used the **platform default**, which depended on the OS and **locale** (language/region settings): UTF-8 on most Linux/macOS, Windows-1252 on many Windows machines. The same program could read a file correctly on a Mac and wrongly on a Windows server. Since **Java 18 (JEP 400; JEPs: §1.12) the default charset is UTF-8 everywhere**. (The console's own encoding still follows the terminal: see `System.getProperty("stdout.encoding")`.) Being explicit — `StandardCharsets.UTF_8` — is still best practice: it documents intent and stays correct if someone runs with `-Dfile.encoding=COMPAT` (a *system property*, §13.11).

`code/ch13/charsets/CharsetDemo.java` (key parts):

```
String s = "café €";
for (Charset cs : new Charset[]{UTF_8, UTF_16BE, ISO_8859_1, US_ASCII}) {
    byte[] b = s.getBytes(cs);                                    // encode
    System.out.printf("%-10s %2d bytes  %s%n", cs, b.length, hex.formatHex(b));
}
byte[] utf8 = "café".getBytes(UTF_8);
new String(utf8, ISO_8859_1);                                     // decode with the wrong charset
Files.writeString(latin, "café", ISO_8859_1);
Files.readString(latin);                                          // assumes UTF-8 -> throws
String e = "hi👋";  e.length(); e.codePoints().count();
```

Output:

```
default charset = UTF-8 (UTF-8 by default since Java 18, JEP 400)
UTF-8       9 bytes  63 61 66 c3 a9 20 e2 82 ac
UTF-16BE   12 bytes  00 63 00 61 00 66 00 e9 00 20 20 ac
ISO-8859-1  6 bytes  63 61 66 e9 20 3f
US-ASCII    6 bytes  63 61 66 3f 20 3f
decoded wrong  -> cafÃ©
decoded right  -> café
readString(latin1 file) -> MalformedInputException
with right charset -> café
length()=4 codePoints=3 utf8 bytes=6
```

What to notice:

- The same 6-character string is 9, 12 or 6 bytes depending on the charset.
- `3f` is `?`: `String.getBytes` **silently replaces** characters the charset can't represent (`€` in ISO-8859-1; `é` and `€` in ASCII). Data is lost without an error. (To get an error, use a `CharsetEncoder` with `CodingErrorAction.REPORT`.)
- `Files.readString`/`readAllLines` are **strict**: invalid UTF-8 throws `MalformedInputException` rather than guessing — better than silent corruption. `BufferedReader` built with `InputStreamReader` is lenient and substitutes `�` (U+FFFD).
- `"hi👋".length()` is 4: the emoji is one **code point** outside the Basic Multilingual Plane, stored as two `char`s (a *surrogate pair*). Use `codePoints()` to count what humans see as characters (Chapter 7).

### Common mistakes

- Decoding a byte array that was cut in the middle of a multi-byte character (e.g. splitting network data at a fixed size) — decode after reassembling, or use a `Reader` which handles the boundary.
- A **BOM** (byte-order mark, `EF BB BF`) at the start of files from some Windows tools: Java's UTF-8 decoder keeps it as the character `\uFEFF`, so the first header of a CSV might not equal `"name"`. Strip it explicitly.
- Using `Properties.load(InputStream)`, which assumes ISO-8859-1 (§13.11).

### In the interview

> **30-second answer:** "Strings are characters; files are bytes; a charset converts. Since Java 18 the default is UTF-8 on every platform, but I still pass `StandardCharsets.UTF_8` explicitly. Wrong charset

> gives mojibake; `getBytes` silently substitutes `?` for unmappable characters while `Files.readString` throws on malformed input."

- *How many bytes is `é` in UTF-8?* → 2.
- *Why can `String.length()` differ from the number of visible characters?* → It counts UTF-16 `char`s; characters outside the Basic Multilingual Plane (emoji) take two.

---

## 13.8 Console and standard streams

### What it is

Every process starts with three **standard streams** connected by the OS: **standard input** (`System.in`, an `InputStream` — usually the keyboard), **standard output** (`System.out`, a `PrintStream`) and **standard error** (`System.err`, a `PrintStream` for diagnostics and error messages). Shells can **redirect** each independently to files: `java App < input.txt > out.txt 2> errors.txt`.

Ways to read input:

| Tool | Best for | Notes |
|---|---|---|
| `Scanner(System.in)` | small interactive programs, exercises | parses tokens (`nextInt`, `nextDouble`, `nextLine`); regex-based, slow for big input |
| `BufferedReader(new InputStreamReader(System.in))` | large inputs (competitive programming, piped data; e.g. $10^6$ numbers) | line-based; parse with `split` / `Integer.parseInt` |
| `System.console()` | real terminals; passwords | `readPassword()` doesn't echo and returns `char[]` (can be wiped) |
| `IO.readln(prompt)` / `IO.println(x)` | beginner programs | `java.lang.IO`, **Java 25** (JEP 512), auto-imported — no `import` needed |

`code/ch13/console/ConsoleInput.java` simulates typed input with an in-memory stream so the output is repeatable:

```java
String typed = "Ada\n36\n3 4 5\n";
Scanner sc = new Scanner(new ByteArrayInputStream(typed.getBytes(UTF_8)));  // real code: new Scanner(System.in)
String name = sc.nextLine();
int age = sc.nextInt();
while (sc.hasNextInt()) sum += sc.nextInt();

BufferedReader br = new BufferedReader(new StringReader(typed));  // real code: new InputStreamReader(System.in)
String first = br.readLine();
int parsedAge = Integer.parseInt(br.readLine().trim());
for (String tok : br.readLine().split("\\s+")) total += Integer.parseInt(tok);

Scanner trap = new Scanner("42\nhello\n");
int n = trap.nextInt();
String rest = trap.nextLine();            // "" - the rest of the line after 42!

Console c = System.console();
c != null && c.isTerminal();              // isTerminal(): Java 22
IO.println("IO.println works without System.out");              // Java 25
```

Output (the stderr line may appear in a different position when both streams go to one terminal — they are separate streams):

```
Scanner: name=Ada age=36 sum=12
BufferedReader: name=Ada age=36 sum=12
trap: n=42 nextLine=[] then [hello]
stdout and stderr are separate streams (redirect with > and 2>)
this line went to stderr
System.console() available? false
IO.println works without System.out
```

- **The `nextInt`/`nextLine` trap:** `nextInt()` consumes `42` but leaves the newline; the following `nextLine()` returns the empty remainder of that line. Fix: call an extra `nextLine()` after `nextInt()`, or read whole lines and parse them.
- `System.console()` returns `null` when input or output is redirected (IDEs, pipes, CI servers) — always null-check it. Java 22–24 used a JLine-based console by default that could return a `Console` even then; Java 25 reverted to the old behaviour. `isTerminal()` (22) tells you whether a returned console is really interactive.

### Common mistakes

- Closing a `Scanner` wrapping `System.in` in a helper method — that closes `System.in` for the rest of the program.
- Using `System.out.println` for logging in applications — use a logging framework (Chapter 18).
- Reading passwords with `Scanner` into a `String` — use `Console.readPassword()`.

---

## 13.9 Binary data, channels and buffers

### Primitive values: `DataOutputStream` / `DataInputStream`

To store numbers compactly (not as text), `DataOutputStream` writes Java primitives (`int`, `double`, `boolean` …) in a fixed **big-endian** (most significant byte first) binary format; `DataInputStream` reads them back. **You must read in exactly the same order and types you wrote** — the file has no labels, so the reader can't ask what type comes next.

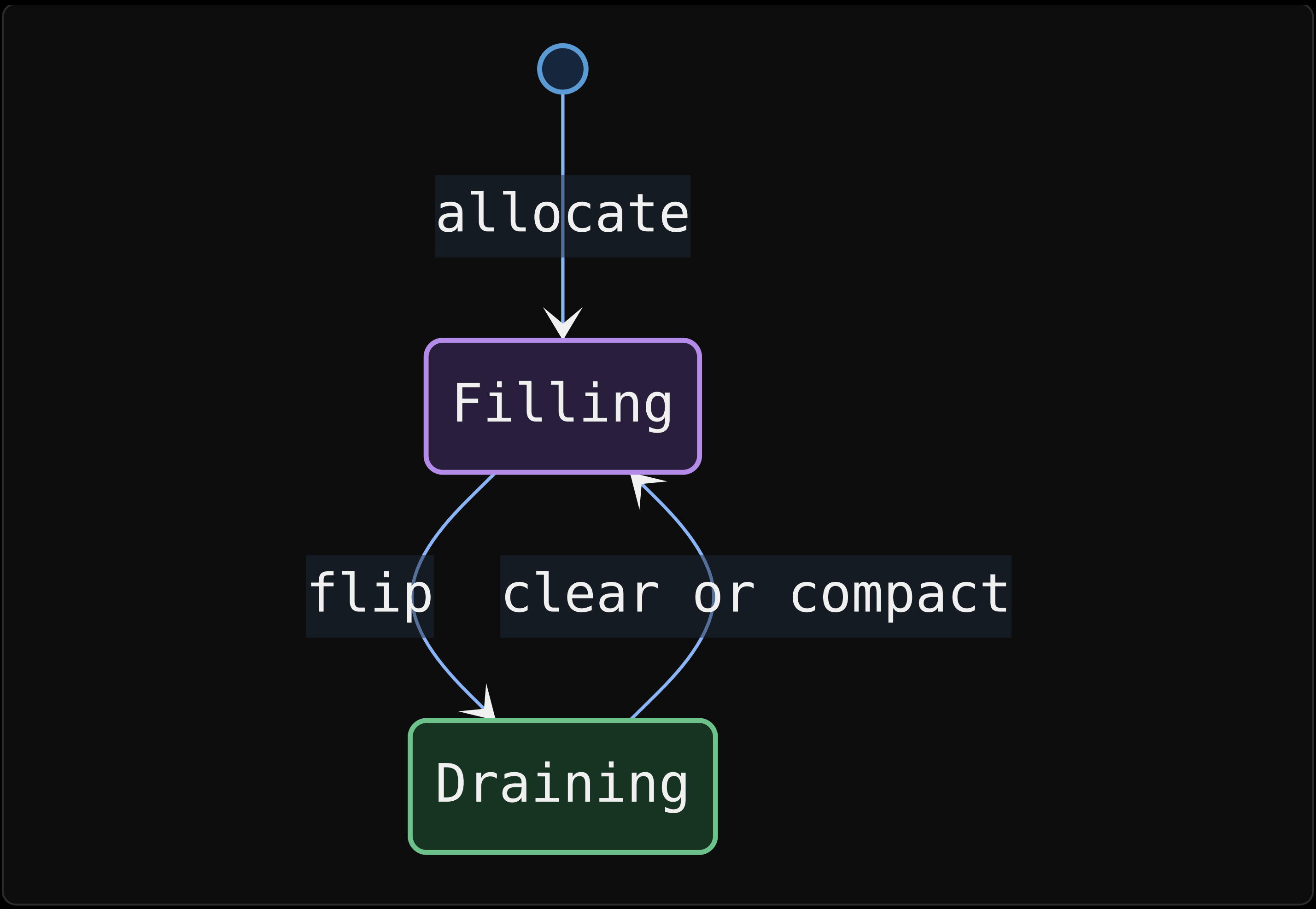
`code/ch13/binary/DataAndChannels.java` (key parts):

```java
try (DataOutputStream out = new DataOutputStream(new BufferedOutputStream(Files.newOutputStream(f)))) {
    out.writeInt(2025);  out.writeDouble(3.5);  out.writeBoolean(true);  out.writeUTF("ok");
}
try (DataInputStream in = new DataInputStream(new BufferedInputStream(Files.newInputStream(f)))) {
    in.readInt(); in.readDouble(); in.readBoolean(); in.readUTF();
}
```

### Channels and `ByteBuffer`

NIO models I/O differently from streams:

- A **channel** (`FileChannel`, `SocketChannel`) is a connection that can read *into* and write *from* a buffer, often in both directions, and supports **positioned** (random-access) reads.
- A `ByteBuffer` is a fixed-capacity block of bytes with three cursors: **position** (next index to use), **limit** (end of valid data), **capacity**. You *fill* it (channel read), call `flip()` to switch to *draining* (limit = position, position = 0), consume it, then `clear()` to fill again (position = 0, limit = capacity) — or `compact()`, which keeps any unread bytes at the front.



```
try (FileChannel ch = FileChannel.open(g, READ)) {
    ByteBuffer buf = ByteBuffer.allocate(8);
    while (ch.read(buf) != -1) {          // fill
        buf.flip();                       // switch to draining
        sb.append(UTF_8.decode(buf));
        buf.clear();                      // ready to fill again
    }
}
ch.read(word, 13);                        // positioned read at byte offset 13
```

Output:

```
file size = 17 bytes (4 + 8 + 1 + 2 + 2)
read back: 2025, 3.5, true, ok
channel wrote 20 bytes, position now 20
channel read: channels and buffers
bytes 13..15 = buf
```

(`writeUTF("ok")` = a 2-byte length prefix + 2 bytes. Decoding an 8-byte UTF-8 chunk independently is only safe here because the text is ASCII — with multi-byte characters use a `CharsetDecoder` or a `Reader`.)

### What else NIO offers (overview)

| Feature | What it's for |
|---|---|
| `FileChannel.map(...)` → `MappedByteBuffer` | **memory-mapped files**: the OS maps the file into memory; fast random access to huge files (databases, search indexes) |
| `FileChannel.transferTo/transferFrom` | **zero-copy** (the OS moves the data without copying it through your program's memory) file-to-socket transfer (web servers sending static files) |
| `FileChannel.lock()` | OS-level file locks between processes |
| `Selector` + non-blocking `SocketChannel` | one thread watching thousands of connections — the basis of Netty; far less needed now that virtual threads make blocking I/O cheap (Chapter 15) |
| `AsynchronousFileChannel` | callback / `Future`-based file I/O |
| `ByteBuffer.allocateDirect` | memory outside the heap for native (OS/C-level) I/O (the Foreign Function & Memory API, final in Java 22, is the modern route to off-heap memory) |
| `RandomAccessFile` (java.io) | the older way to seek and read/write at positions |

---

## 13.10 Serialization (and why to avoid it)

### What it is

**Java serialization** converts an **object graph** (an object and everything it references) into a byte stream with `ObjectOutputStream.writeObject`, and rebuilds it (*deserialization*) with `ObjectInputStream.readObject`. A class opts in by implementing the **marker interface** `java.io.Serializable` (no methods; it only tags the class).

Rules:

1. Every non-`transient`, non-`static` field is written; referenced objects must also be `Serializable`, or `NotSerializableException` is thrown.
2. **`transient`** fields are skipped and come back as default values (`null`, `0`, `false`).
3. On deserialization of a normal class, **its constructor does not run**. Java allocates the object and fills fields directly; only the no-arg constructor of the first *non-serializable* superclass runs. Validation in constructors is therefore bypassed.
4. **Records** are the exception: they are rebuilt **through the canonical constructor** (the record's main constructor taking every component in order), so validation runs (Java 16).
5. **`serialVersionUID`** is a version stamp. If you don't declare one, the JVM computes it from the class structure; almost any change (adding a method) changes it, and old data then fails with `InvalidClassException`. Declare it explicitly (`@Serial private static final long serialVersionUID = 1L;` — `@Serial` is Java 14) if you serialize at all.

`code/ch13/serial/SerializationDemo.java` (key parts):

```
static class Account implements Serializable {
    @Serial private static final long serialVersionUID = 1L;
    private final String owner;
    private final long balanceCents;
    private transient String sessionToken;                      // skipped
    Account(...) { ...; System.out.println("  constructor ran for " + owner); }
}
record Point(int x, int y) implements Serializable {
    Point { if (x < 0 || y < 0) throw new IllegalArgumentException("negative"); ... }
}

ObjectInputFilter allow = ObjectInputFilter.Config.createFilter(
        "ch13.serial.SerializationDemo$Account;ch13.serial.SerializationDemo$Point;!*");
in.setObjectInputFilter(allow);                                 // before readObject()
```

Output:

```
create:
  constructor ran for ana
serialized to 116 bytes
deserialize (note: NO constructor call for a normal class):
  Account[ana, 1250, token=null]  same object? false
record - create:
  canonical constructor ran for (1,2)
record - deserialize (the canonical constructor runs again):
  canonical constructor ran for (1,2)
  Point[x=1, y=2]
  canonical constructor ran for (1,2)
filtered, allowed class: Point[x=1, y=2]
filtered, ArrayList -> InvalidClassException: filter status: REJECTED
```

- The transient token came back `null`; the account is a new object with equal state.
- The record's canonical constructor ran on every deserialization (including the filtered one) — so an invalid byte stream can't create an invalid `Point`.
- The allow-list filter (`!*` = reject everything else) blocked an `ArrayList` before any of its code ran.

### Why to avoid it

*Effective Java* (Item 85) is blunt: there is no reason to use Java serialization in new systems.

1. **Security.** `readObject` can instantiate any serializable class on the **classpath** and run its deserialization methods. Attackers chain such "gadget" classes (harmless-looking classes whose side effects combine into an attack) to execute code — many real **remote-code-execution** (RCE) vulnerabilities in application servers and libraries came from deserializing untrusted bytes. Filters (`ObjectInputFilter`, Java 9, JEP 290; JVM-wide filter factories, Java 17, JEP 415; or the `jdk.serialFilter` system property) reduce the risk but must be configured correctly.
2. **Hidden constructor.** Deserialization bypasses constructors and invariants (except for records).
3. **Fragile versioning** and a format only Java can read.
4. **Freezes your implementation**: private fields become part of the public **wire format** (the byte layout sent or stored).

**Use instead:** JSON (§13.12) for human-readable data and APIs, Protocol Buffers/Avro (compact schema-based binary formats readable from many languages) for cross-language data, a database for persistence. If stuck with serialization (legacy RMI — Remote Method Invocation — some caches), serialize records, set an allow-list filter, and never deserialize untrusted data.

### In the interview

**30-second answer:** "Serialization turns an object graph into bytes via `Serializable`; `transient` fields are skipped, `serialVersionUID` guards compatibility, and normal classes are rebuilt without calling their constructor. I avoid it for new code because deserializing untrusted data can lead to remote code execution and it's brittle; I use JSON or protobuf. If I must, I use records and an `ObjectInputFilter` allow-list."

- *What happens to a `static` field?* → Not serialized; it belongs to the class.
- *Superclass not `Serializable`?* → Its no-arg constructor runs on deserialization and its fields are not restored (and it must have an accessible no-arg constructor).

## 13.11 Properties files

### What it is

A **properties file** is plain text of `key=value` lines — the traditional Java configuration format (Spring Boot's `application.properties`, `log4j2.properties`, `gradle.properties`). `java.util.Properties` loads and stores them.

Format rules: `#` or `!` start a comment; the separator can be `=`, `:` or whitespace, with spaces around it ignored; a trailing `\` continues the value on the next line (leading whitespace of the next line is dropped); values are **always strings**.

`code/ch13/props/PropertiesDemo.java` (key parts):

```
Files.writeString(file, """
        # database settings
        db.url = jdbc:postgresql://localhost:5432/shop
        db.pool.size=10
        ! bang also starts a comment
        greeting=Hello, \\
                 world
        city=Zürich
        """);
Properties p = new Properties();
try (Reader r = Files.newBufferedReader(file, StandardCharsets.UTF_8)) { p.load(r); }
p.getProperty("db.url");
Integer.parseInt(p.getProperty("db.pool.size"));
p.getProperty("timeout", "30");                  // default when absent
p.setProperty("timeout", "45");
try (Writer w = Files.newBufferedWriter(out, UTF_8)) { p.store(w, "saved by demo"); }
```

Output:

```
keys        = [city, db.pool.size, db.url, greeting]
db.url      = jdbc:postgresql://localhost:5432/shop
pool+1      = 11   (values are always Strings)
greeting    = Hello, world
city        = Zürich
timeout     = 30   (default used)
saved 5 key lines
system property java.version starts with 2? true
```

(`\\` in the Java text block produces a single `\` in the file.)

### Common mistakes

- `load(InputStream)` decodes as **ISO-8859-1**, so `Zürich` saved as UTF-8 becomes `ZÃ¼rich`. Use `load(Reader)` with UTF-8. (Resource bundles, `PropertyResourceBundle`, read UTF-8 since Java 9.)
- `Properties` extends `Hashtable<Object,Object>` (an old synchronised map; a historical design mistake); use only `getProperty`/`setProperty`/`stringPropertyNames`, never `put`/`get` with non-strings.
- Committing secrets (passwords, API keys) to version control in properties files — use **environment variables** or a **secret manager** (a service that stores and hands out credentials).
- **System properties** (`System.getProperty("user.home")`, set with `-Dkey=value`) are a separate, JVM-wide `Properties` object; **environment variables** come from `System.getenv("HOME")`.

For locale-specific messages with `ResourceBundle`, see OCP Chapter 11.

## 13.12 Working with JSON

### What it is

**JSON** (JavaScript Object Notation: nested `{"key": value}` objects and `[ … ]` lists in plain text) is the format of web APIs and most modern config/data files. As of Java 25/26 the JDK has **no public JSON API** — you use a library: **Jackson** (Spring Boot's default) or **Gson**; JSON-B/Yasson is the Jakarta EE standard. Jackson 3 (October 2025, default in Spring Boot 4) moved to `tools.jackson.*` packages and prefers building an immutable `JsonMapper` via `JsonMapper.builder()`; the calls below are otherwise the same.

### How it works (Jackson 2.x — not run: needs the `jackson-databind` dependency)

```java
// (Jackson - not run)
record Order(String id, List<String> items, double total) {}

ObjectMapper mapper = new ObjectMapper();                   // thread-safe once configured; reuse one
String json = mapper.writeValueAsString(new Order("A1", List.of("pen", "ink"), 9.5));
// {"id":"A1","items":["pen","ink"],"total":9.5}
Order back = mapper.readValue(json, Order.class);           // records supported since Jackson 2.12
JsonNode tree = mapper.readTree(json);                      // untyped tree when there's no class
tree.get("items").get(0).asText();                          // "pen"
List<Order> orders = mapper.readValue(arrayJson, new TypeReference<List<Order>>() {});  // generics survive erasure
```

- **Data binding** (JSON ↔ your classes; records fit perfectly) for known shapes; the **tree model** (`JsonNode`) for dynamic or partial data; **streaming** (`JsonParser`) for gigantic files.
- `TypeReference` works around **type erasure** (generic type arguments are unknown at run time; Chapter 9).
- Money: use `BigDecimal`, not `double` (Chapter 7).
- Adding the dependency with Maven/Gradle: Chapter 18.

### Common mistakes

Hand-building JSON with string concatenation (quotes and escapes break it; an **injection** risk — attacker text changing the structure), creating a new `ObjectMapper` per request (expensive — share one), and not deciding whether unknown fields should be ignored or fail (`FAIL_ON_UNKNOWN_PROPERTIES`).

---

## 13.13 Best practices and common mistakes (checklist)

1. **`Path` + `Files`** for files; `java.io.File` only for legacy APIs.
2. **try-with-resources** for every stream, reader, writer, channel, and for `Files.lines/list/walk/find`.
3. **Always name the charset** (`StandardCharsets.UTF_8`), even though it's the default since Java 18.
4. **Buffer** small reads/writes; remember `close()` flushes and `flush()` isn't a disk sync.
5. **Stream large inputs**; load whole files only when they're known to be small.
6. **Don't check-then-act** (`exists` then `create`); do the operation and handle `FileAlreadyExistsException`/`NoSuchFileException`.
7. **Write atomically**: temp file in the same directory + `move(..., ATOMIC_MOVE)`.
8. **Validate user-supplied paths**: `normalize()` then `startsWith(baseDir)`.
9. **Don't swallow `IOException`**: it carries the reason (disk full, permission denied). Wrap in `UncheckedIOException` inside lambdas (Chapter 8 §8.8).
10. **No Java serialization of untrusted data**; prefer JSON/protobuf; if unavoidable, records + filters.
11. **Tests**: use JUnit's `@TempDir` or `Files.createTempDirectory`, and in-memory streams (`ByteArrayInputStream`, `StringReader`) to test parsing without files.
12. **Blocking I/O is fine on virtual threads** — thousands of concurrent file/socket operations no longer need async APIs (Chapter 15 §15.4).

### In the interview (chapter-level)

- *How would you count lines in a 50 GB log file?* → `Files.lines(p)` in try-with-resources (or a `BufferedReader` loop) — constant memory; never `readAllLines`.

- *How do you make a config update crash-safe?* → Write to a temp file, `force` if durability matters, then `ATOMIC_MOVE` over the old file.
- *NIO vs IO?* → `java.io` = blocking streams; NIO = channels/buffers/selectors (non-blocking sockets, memory mapping); NIO.2 = `Path`/`Files` file-system API. Most file code today is NIO.2 + `java.io` readers.

---

## Chapter TL;DR

| Topic | Remember |
|---|---|
| APIs | `java.io` streams (1.0), NIO channels/buffers (1.4), NIO.2 `Path`/`Files` (7) — use NIO.2 for files |
| `Path` | Just a name; `resolve` appends (absolute arg wins), `relativize` gives the route, `normalize` is textual |
| `Files` ops | Specific exceptions; `copy`/`move` need `REPLACE_EXISTING`; delete trees children-first; close `list`/`walk`/`find`/`lines` |
| Whole files | `readString`/`writeString` (11) for small files; writes truncate unless `APPEND`; `CREATE_NEW` = atomic create |
| Streams | Byte (`InputStream`/`OutputStream`) vs char (`Reader`/`Writer`); bridge with a charset; decorators stack; `read` → `-1` at end |
| Buffering | ~100× fewer system calls; `close()` flushes; unflushed data is lost |
| Charsets | UTF-8 default since Java 18 (JEP 400); still pass it explicitly; wrong charset = mojibake; `getBytes` substitutes `?` |
| Console | `Scanner` (easy), `BufferedReader` (fast), `Console` (passwords, `isTerminal` 22), `IO` (25); beware `nextInt`+`nextLine` |
| Binary/NIO | `Data*Stream` for primitives in a fixed order; `ByteBuffer` fill → `flip` → drain → `clear`; memory mapping for huge files |
| Serialization | `Serializable`, `transient`, `serialVersionUID`; constructors skipped (records excepted); security risk — avoid, or filter |
| Properties | `load(Reader)` with UTF-8; values are strings; `getProperty(key, default)` |
| JSON | Needs a library (Jackson/Gson); bind to records |

**Where next:** doing many I/O operations at once — threads and thread pools in Chapter 14 — Concurrency I, then virtual threads, which make blocking I/O scale, in Chapter 15. Exam traps: OCP Chapter 14 — I/O. Python comparison (`open`, `pathlib`): Python Chapter 9.

# Chapter 14 — Concurrency I: Threads, Locks and the `java.util.concurrent` Toolkit

A web server handling thousands of requests, an app downloading images while the user scrolls, a **batch job** (processing a large pile of data in one go, no user waiting) using all 16 **cores**: all do several things **at the same time**. Java has supported this since 1.0 through **threads** (§14.1), and since Java 5 through the high-level `java.util.concurrent` package (designed by Doug Lea and the authors of *Java Concurrency in Practice*, "JCIP"). Concurrency is also where Java programs get their nastiest bugs — results that are wrong only sometimes, on some machines, under load.

This chapter builds the foundation: creating and controlling threads, *why* shared data goes wrong (race conditions and visibility), the tools that make it right (`synchronized`, `volatile`, locks, atomics), and the everyday toolkit (thread pools, concurrent collections, synchronizers). Chapter 15 continues with `CompletableFuture`, Fork/Join, **virtual threads**, structured concurrency, scoped values, the Java Memory Model (the rules for when one thread's writes become visible to another) and **liveness** problems — threads that stop making progress: **deadlock** (each waiting forever for a lock another holds), **livelock** (busily reacting to each other without progress) and **starvation** (a thread never getting its turn).

All runnable code is in `code/` (package `ch14.*`), compiled with `javac --release 25` and run on JDK 26 on a 14-core laptop. **Concurrency output is not always repeatable.** Lines marked `[nondeterministic]` change from run to run (that is the point being demonstrated); lines marked `[machine-dependent]` depend on your hardware. Everything else is forced into a fixed order with `join`, latches or barriers (§14.2, §14.13). The programs added in the numbered subsections (§14.1.1 and later, and the extensions of §14.7, §14.13 and §14.14) were compiled with `javac --release 21` and run on JDK 21 on a 4-core Linux machine, so their `[machine-dependent]` numbers come from that machine.

> **Related material — linked, not repeated.**
>
> - Exam-level API tables and trick questions (`Future` methods, `Executors` factories, `TimeUnit`, lock methods, parallel streams): OCP Chapter 13 — Concurrency.
> - Design-level concepts with diagrams and interview framing (mutex — a lock that lets one thread in at a time — semaphores, producer-consumer, reader-writer, thread-pool patterns): LLD Chapter 12 — Concurrency concepts.
> - Classic problems solved step by step (FooBar alternation, H2O, dining philosophers, blocking queue, TTL cache — entries expire after a *time to live* — rate limiter, which caps how many requests are allowed per second): LLD Chapter 13 — Concurrency problems.

---

## 14.1 Concurrency basics

### What it is

- A **program** is passive: instructions stored in files (`App.class`, a JAR, the `java` launcher binary). Nothing runs until the operating system loads it.
- A **process** is a running instance of a program, with its own **address space** (the private range of memory addresses it can use), its own open files and sockets, and an OS-assigned **process id** (PID); the OS isolates processes from each other. Starting `java App` twice from the same program gives two processes that share nothing: a `static` field changed in one is unchanged in the other. Each is a JVM (Java Virtual Machine, the program that runs your compiled Java code). Processes communicate only through the OS (pipes, sockets, files).
- A **thread** is an independent path of execution *inside* a process and the unit the OS **schedules** onto a core. Each thread has its own **call stack** (local variables, method frames) and **program counter** (the marker of which instruction runs next), but **all threads in a process share the same heap** (the memory area where all objects live) — the same objects and fields. A JVM is never single-threaded: even "hello world" runs alongside garbage-collector, JIT-compiler and reference-handler threads. Creating

and switching threads is far cheaper than doing the same for processes, because no address space has to be set up or swapped.
- **Concurrency** means several tasks are *in progress* during the same period (possibly taking turns on one CPU core). **Parallelism** means tasks literally run *at the same instant* on different cores. Concurrency is about structure; parallelism is about hardware. (Detailed comparison with diagrams: LLD §12.1–12.2.)

### Why it matters

1. **Responsiveness** — a UI (user interface) or server keeps answering while slow work (disk, network) happens elsewhere.
2. **Throughput** (work completed per second, §14.1.2) — while one request waits for a database, another can use the CPU.
3. **Using all cores** — CPU-heavy work (image processing, sorting big data) can be split across cores.

### The core problem

Because threads share the heap, two threads touching the same object can interfere. Three things can go wrong:

1. **Atomicity** — an operation that looks like one step (`count++`) is really several, and threads interleave in the middle (§14.5).
2. **Visibility** — a value written by one thread may not be seen by another, because of **CPU caches** (small, fast per-core copies of memory) and **compiler reordering** (the compiler or CPU shuffling independent instructions to run faster) (§14.7, and the Java Memory Model in Chapter 15).
3. **Ordering** — operations may appear to happen in a different order to another thread.

Everything in this chapter is a tool against one or more of these.

### Key terms

| Term | Meaning |
|---|---|
| **Thread-safe** | behaves correctly when used by several threads at once, with no extra coordination by callers |
| **Shared mutable state** | data that more than one thread can access *and* that can change — the source of all concurrency bugs |
| **Critical section** | code that touches shared mutable state and must not be executed by two threads at once |
| **Lock / monitor** | a mechanism that lets only one thread (or a controlled number) into a critical section |
| **Blocking** | a thread waiting (for a lock, I/O, a sleep) without doing work |
| **Context switch** | the OS pausing one thread and running another on the same core |

### 14.1.1 Preemptive vs cooperative multitasking

A core runs one thread at a time; **multitasking** shares it by saving one thread's registers and program counter and switching to another (a *context switch*). The two designs differ in **who decides when to switch**:

| | **Preemptive** | **Cooperative** |
|---|---|---|
| Who switches | the **scheduler** (the OS component that picks the next thread), using a hardware timer interrupt | the running task, voluntarily |
| When | at the end of a **time slice** (a few milliseconds) or when a higher-priority thread becomes ready — at *any* instruction | only at **yield points**: a blocking call, an `await`, an explicit `yield` |
| A task stuck in a loop | slows everyone a little; others still run | **freezes** every other task sharing that thread/core |
| Shared-data consequence | a thread can be interrupted between any two instructions → races (§14.5) | no switch between yield points, so code between them runs without interference (on one thread) |
| Examples | OS threads on Linux, Windows, macOS; Java **platform threads** | Windows 3.x, classic Mac OS; JavaScript's event loop; Python `asyncio`; Kotlin coroutines; Java **virtual threads** with respect to their carrier |

Platform threads are OS threads, so they are preemptively scheduled. **Virtual threads** (§15.4) leave their **carrier** (the platform thread running them) only when they block in a JDK operation or call `Thread.yield()`: JEP 444 says the scheduler "does not currently implement time sharing for virtual threads". The OS can still preempt the carrier, but that only pauses the virtual thread; no other virtual thread gets the carrier.

`code/ch14/threads/Multitasking.java` starts threads that spin until a shared deadline 300 ms away and count iterations; a count of 0 means the thread never got a CPU in time. Virtual threads share one carrier (`jdk.virtualThreadScheduler.parallelism=1`, set by the program at startup):

```java
Runnable spin = () -> {
    long count = 0;
    while (System.nanoTime() < deadline) {                 // pure CPU work, never blocks
        count++;
        if (yield && count % 1_000 == 0) Thread.yield();   // a voluntary yield point
    }
    loops.set(id, count);
};
```

Output:

```
platform threads that ran before the deadline: 12 of 12
virtual threads, 1 carrier, no yield:          1 of 3
virtual threads, 1 carrier, Thread.yield():    3 of 3
```

- 12 platform threads on 4 cores all progressed: the OS time-sliced them.
- The first virtual thread kept the only carrier past the deadline, so the other two never ran: cooperative scheduling.
- With a `yield()` every 1,000 iterations all three ran. Real virtual threads yield whenever they block on I/O, locks or queues — why they suit I/O-bound work, not long CPU loops.

**In the interview:** "Preemptive means the scheduler can switch threads at any instruction; cooperative means a task runs until it yields. Platform threads are preemptive. Virtual threads switch only when they block or yield, so a CPU-bound virtual thread can hog its carrier."

Sources: JEP 444 (Virtual Threads), `Thread.yield` Javadoc.

### 14.1.2 Sync vs async, I/O-bound vs CPU-bound, throughput vs latency

Four pairs of words that describe *how a task uses its thread*:

| Pair | Definition | Java example |
|---|---|---|
| **Synchronous** | the caller waits for the result before continuing; the result is the return value | `String s = client.send(req, ofString()).body();` |
| **Asynchronous** | the call returns at once and the result arrives later, through a `Future`, a callback, or a `CompletableFuture` stage | `client.sendAsync(req, ofString()).thenAccept(...)` (§15.2) |
| **Blocking** | the *thread* is parked until the operation can finish | `queue.take()`, `lock.lock()`, `InputStream.read()` |
| **Non-blocking** | returns immediately, even if it has to report "not ready" | `queue.poll()` → `null`, `lock.tryLock()` → `false`, NIO channels in non-blocking mode |
| **I/O-bound** | most of the time is spent waiting for disk, network or a database; the CPU sits idle | a REST call, a JDBC query |
| **CPU-bound** | the time is spent computing; adding waiting threads doesn't help, adding cores does | hashing, image resizing, sorting a large array |
| **Latency** | time for **one** operation, start to finish (report percentiles such as p50/p99 — the value 50%/99% of requests stay under) | "p99 checkout latency is 180 ms" |
| **Throughput** | operations **completed per unit of time** | "the service handles 4,000 requests/s" |

The pairs are independent: a synchronous call can be non-blocking (`poll()` in a loop), and an asynchronous API can block a thread elsewhere (`supplyAsync` runs a blocking call on a pool thread). Virtual threads give synchronous, blocking code the scalability of asynchronous code, because a blocked virtual thread holds no OS thread.

**Little's law** ties the last pair together for any stable system: **in-flight requests = throughput × latency** (L = λ × W). Two worked uses:

- *Sizing*: 2,000 requests/s at 50 ms each → 2,000 × 0.05 = **100 requests in flight**. If each holds a thread, you need ≥ 100 threads.
- *Ceiling*: a pool of 20 threads with 200 ms requests can complete at most 20 / 0.2 = **100 requests/s**, however fast the CPU is.

Throughput and latency trade against each other: batching and deep queues raise throughput but lengthen each wait, and near full utilisation waiting explodes. In the simplest queueing model (M/M/1: one server, random arrivals, 10 ms average service, so at most 100 requests/s) the average time in the system is 1 / (100 − arrival rate) s: 20 ms at 50 requests/s, 100 ms at 90/s, 1 s at 99/s. Hence servers run well below 100% busy and bounded queues reject work early (§14.11).

`code/ch14/threads/IoVsCpu.java` compares a task's CPU time (`ThreadMXBean.getCurrentThreadCpuTime()`) with its wall-clock time, then runs 16 copies on 1 thread and on 16 threads:

```java
long wall0 = System.nanoTime(), cpu0 = MX.getCurrentThreadCpuTime();
task.run();                                   // ioTask = sleep(100); cpuTask = 100M arithmetic steps
// ... CPU busy % = cpu / wall
long speedup = timeOn(1, tasks, task) / timeOn(tasks, tasks, task);
```

Output:

```
I/O-bound task: wall 100 ms, CPU 0 ms -> CPU busy 0%   [machine-dependent]
CPU-bound task: wall 80 ms, CPU 79 ms -> CPU busy 99%   [machine-dependent]
cores = 4, tasks = 16, pool = 16 threads   [machine-dependent]
I/O tasks: speed-up vs one thread = 15x   [machine-dependent]
CPU tasks: speed-up vs one thread = 4x   [machine-dependent]
```

Sixteen threads gave the waiting tasks a 15–16× speed-up (waits overlap for free; thread start-up costs the rest), while the CPU tasks stopped at 4×, the core count — the measured basis of the pool-sizing rule in §14.11.

Sources: `java.net.http.HttpClient`, `BlockingQueue`, `ThreadMXBean` Javadoc; JEP 444.

### 14.1.3 Limits of speed-up: Amdahl's law, Gustafson's law, Moore's law

**Amdahl's law** (Gene Amdahl, 1967): if a fraction **p** of a program's running time can be parallelised and the rest (1 − p) is serial, the best speed-up on **N** cores is

**$S(N) = 1 / ((1 - p) + p / N)$**, which can never exceed **$1 / (1 - p)$** however many cores you add.

| Parallel fraction p | 2 cores | 4 | 8 | 16 | 64 | 1,024 | limit |
|---|---|---|---|---|---|---|---|
| 50% | 1.33× | 1.60× | 1.78× | 1.88× | 1.97× | 2.00× | 2× |
| 90% | 1.82× | 3.08× | 4.71× | 6.40× | 8.77× | 9.91× | 10× |
| 95% | 1.90× | 3.48× | 5.93× | 9.14× | 15.42× | 19.64× | 20× |
| 99% | 1.98× | 3.88× | 7.48× | 13.91× | 39.26× | 91.18× | 100× |

Worked example: a 100 s job, 95 s parallelisable, takes 5 + 95/16 ≈ 10.9 s on 16 cores — 9.1×, not 16× — and never under 5 s on any number of cores. In Java the serial parts are code under a contended lock, single-threaded set-up and merge steps, and a counter every thread updates; shrinking them (lock striping, `LongAdder`, per-thread partial results) pays more than adding threads.

**Gustafson's law** (1988) is the optimistic counterpart: more cores usually solve *bigger* problems in the same time, the serial part staying roughly fixed. With s the serial fraction *measured on the N-core run*, the **scaled speed-up** is **$S = s + (1 - s) \times N$** (s = 5%, 16 cores → 15.25×). Amdahl asks "how much faster is this fixed job?", Gustafson "how much more work fits in the same time?".

**Moore's law** (Gordon Moore, 1965, revised 1975) is an *observation*: transistor counts double roughly every two years. For decades **Dennard scaling** (smaller transistors need proportionally less power) also raised clock speeds, so single-threaded code got faster for free. Around 2005 power and heat stopped that; clock rates levelled off at a few GHz and the growing transistor budget became **more cores** (Herb Sutter, "The Free Lunch Is Over", 2005). Programs now get faster on new hardware only by using several cores — within Amdahl's limit.

**In the interview:** "Amdahl: speed-up ≤ 1/(serial fraction), so 5% serial work caps you at 20× on any number of cores; shrink contention and serial steps before adding threads. Gustafson: if the problem grows with the machine, speed-up grows almost linearly. Moore's law still adds transistors, but since Dennard scaling ended they arrive as more cores, which is why concurrency matters."

Sources: G. Amdahl, AFIPS 1967; J. Gustafson, "Reevaluating Amdahl's Law", CACM 1988; G. Moore, *Electronics* 1965; H. Sutter, "The Free Lunch Is Over", 2005.

### Self-check

1. Two `java App` processes and two threads in one process both change a `static int`. Who sees whose change? → Threads share it (same heap); processes don't (separate address spaces).
2. Why can one CPU-bound virtual thread starve others on the same carrier? → Virtual threads are scheduled cooperatively: they give up the carrier only when they block or yield.
3. A service gets 500 requests/s and each takes 80 ms. How many are in flight on average? → Little's law: 500 × 0.08 = 40.
4. 10% of a job is serial. Best possible speed-up on 1,000 cores? → Below 1/0.1 = 10× (9.91×).

---

## 14.2 Creating and starting threads

### What it is

A `java.lang.Thread` object represents one thread. The *work* it does is a `Runnable` — an interface with one method, `void run()`. Since Java 21 there are two kinds of threads: **platform threads** (a thin wrapper over an **OS thread**, i.e. a thread created and scheduled by the operating system — this chapter) and **virtual threads** (lightweight threads managed by the JVM — Chapter 15 §15.4).

### How it works

1. Describe the work: subclass `Thread` and override `run()`, or (better) write a `Runnable` — often a lambda.
2. Create the thread: `new Thread(runnable, "name")`, or the **builder** (an object you configure step by step and then ask to create the thing) `Thread.ofPlatform().name("x").unstarted(runnable)` (Java 21).
3. Call **`start()`**: the JVM creates a new OS thread, which calls `run()`. `start()` returns immediately; the new thread runs concurrently.
4. Optionally **`join()`**: the calling thread waits until the other thread has finished.
5. The thread ends when `run()` returns (or throws). A thread can be started **only once**.

`code/ch14/threads/CreatingThreads.java` (key parts):

```
static class Greeter extends Thread {                        // 1) subclass Thread
    Greeter() { super("greeter"); }
    @Override public void run() { System.out.println("1 subclass  running in " + Thread.currentThread().getName());
}
}
static class Task implements Runnable { ... }                // 2) implement Runnable

Thread t1 = new Greeter();
Thread t2 = new Thread(new Task(), "worker-2");
Thread t3 = new Thread(() -> System.out.println(...), "worker-3");           // 3) lambda
Thread t4 = Thread.ofPlatform().name("worker-4").unstarted(() -> ...);       // 4) builder (Java 21)
for (Thread t : new Thread[]{t1, t2, t3, t4}) { t.start(); t.join(); }

Thread notStarted = new Thread(() -> ...);
notStarted.run();                                            // just a method call - runs in main!
t1.start();                                                  // second start -> IllegalThreadStateException

Thread a = new Thread(() -> System.out.println("   [order varies] A says hi"));
Thread b = new Thread(() -> System.out.println("   [order varies] B says hi"));
a.start(); b.start(); a.join(); b.join();

Thread daemon = new Thread(() -> { while (true) Thread.onSpinWait(); }, "background-spinner");
daemon.setDaemon(true);                                      // before start()
daemon.start();
```

Output:

```
main thread is called 'main'
1 subclass  running in greeter
2 Runnable  running in worker-2
3 lambda    running in worker-3
4 builder   running in worker-4
run() called directly executes in main
state after run(): NEW
start() twice -> IllegalThreadStateException
   [order varies] A says hi
   [order varies] B says hi
both finished (join guarantees this line comes last)
daemon? true, alive? true, id>0? true
main ends; the JVM exits even though the daemon still spins
```

What to notice:

- Calling **`run()` directly** does *not* create a thread: the code runs in `main`, and the `Thread` object stays `NEW`.
- A and B were started together, so either may print first — that's what `[order varies]` means. `join()` guarantees the line after it runs only when both are done.
- **Daemon threads** are background helpers (e.g. the **garbage collector**'s threads — the GC is the JVM component that automatically frees memory of objects no longer in use). The JVM exits when only daemon threads remain — so the infinite spinner didn't keep the program alive. Normal (non-daemon) threads *do* keep the JVM running.

### Useful `Thread` methods

| Method | Purpose |
|---|---|
| `Thread.currentThread()` | the thread running this code |
| `getName()` / `setName()` | names appear in logs and **thread dumps** (a snapshot listing every thread and what it is doing) — always name your threads |
| `threadId()` (19) | unique id (`getId()` is deprecated — still works but officially discouraged and may be removed) |
| `Thread.sleep(ms)` / `sleep(Duration)` (19) | pause the current thread; throws `InterruptedException` |
| `join()` / `join(ms)` / `join(Duration)` (19) | wait for another thread to finish |
| `isAlive()`, `getState()` | inspect |
| `setDaemon(true)` | background thread (set before `start`) |
| `setPriority(n)` | a *hint* (1–10) the OS may ignore — never rely on it for correctness |
| `setUncaughtExceptionHandler(h)` | what to do if `run()` throws (default: print **stack trace** — the chain of method calls that led to the error; the thread dies, others continue) |
| `Thread.onSpinWait()` (9) | hint inside **busy-wait** loops (loops that keep checking a condition instead of sleeping) |
| `Thread.yield()` | hint to let others run; rarely useful |

### Trade-offs: `extends Thread` vs `Runnable`

| `extends Thread` | `implements Runnable` / lambda |
|---|---|
| Uses up your single **superclass** (Java classes can extend only one parent class) | Class can extend something else |
| Couples *what* to do with *how* it runs | The same task can run in a `Thread`, a pool, a virtual thread |
| Rarely the right choice | Preferred |

### When not to use raw threads

In application code you should almost never call `new Thread(...)` directly: creating a platform thread is expensive (an OS thread with a 1–2 MB stack reservation by default, depending on the platform), and unmanaged threads are hard to shut down and monitor. Use an **`ExecutorService`** (§14.11) or, for I/O-heavy tasks on Java 21+, virtual threads (Chapter 15).

### Self-check

1. What's the difference between `t.start()` and `t.run()`? → `start` creates a new thread that calls `run`; calling `run` directly executes on the current thread.
2. Can you restart a finished thread? → No: `IllegalThreadStateException`. Create a new `Thread` (or reuse the `Runnable`).
3. Why doesn't a running daemon thread stop the JVM from exiting? → The JVM exits when all non-daemon threads have finished.

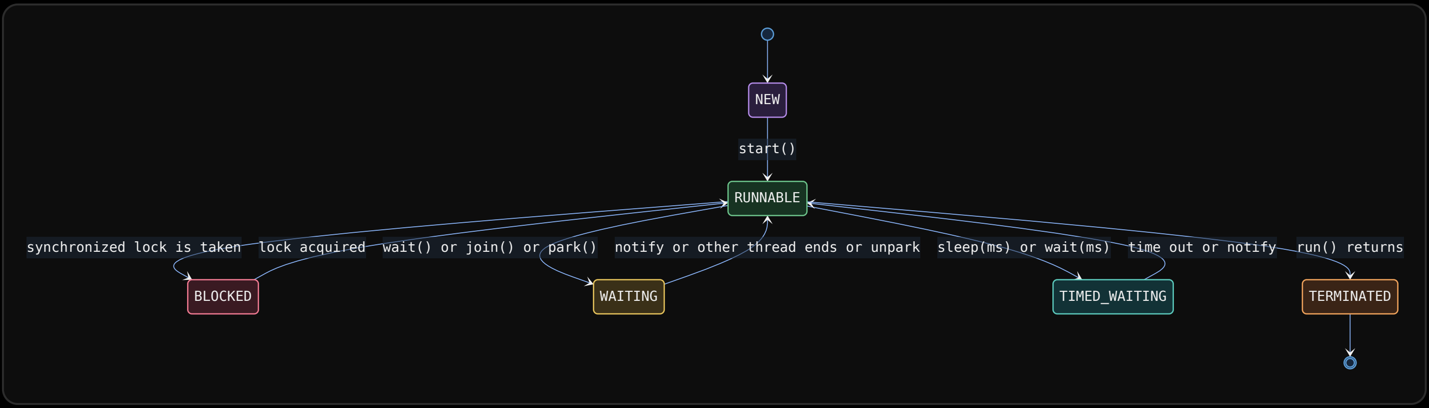
---

## 14.3 Thread lifecycle

### What it is

`Thread.getState()` returns one of six `Thread.State` values:

| State | Meaning | Typical cause |
|---|---|---|
| `NEW` | created, not yet started | `new Thread(...)` |
| `RUNNABLE` | running **or ready to run** (waiting for a CPU, or even blocked in OS-level I/O) | after `start()` |
| `BLOCKED` | waiting to **acquire a monitor lock** (`synchronized`) held by another thread | entering a `synchronized` block that's taken |
| `WAITING` | waiting indefinitely for another thread's action | `Object.wait()`, `join()`, `LockSupport.park()` (low-level "pause this thread until woken", §14.9.1) |
| `TIMED_WAITING` | waiting with a timeout | `sleep(ms)`, `wait(ms)`, `join(ms)`, `tryLock(timeout)` |
| `TERMINATED` | `run()` has finished (normally or by exception) | end of `run()` |



`code/ch14/threads/Lifecycle.java` drives one thread through every state, polling `getState()` until each is reached (so the output is deterministic):

```
Thread t = new Thread(() -> {
    while (keepSpinning) Thread.onSpinWait();    // RUNNABLE
    Thread.sleep(300);                           // TIMED_WAITING
    go.await();                                  // WAITING (a CountDownLatch parks the thread)
    synchronized (LOCK) { }                      // BLOCKED while main holds LOCK
});
```

Output:

```
after new:             NEW
while spinning:        RUNNABLE
during sleep(300):     TIMED_WAITING
during latch.await():  WAITING
waiting for a monitor: BLOCKED
after run() returns:   TERMINATED
```

Note the subtle split: waiting for a **`synchronized`** monitor is `BLOCKED`, but waiting for a **`ReentrantLock`** is `WAITING` (locks park the thread). A thread blocked reading a socket shows as `RUNNABLE` — the JVM doesn't distinguish OS-level I/O waits. Thread dumps (`jstack <pid>`, `jcmd <pid> Thread.print` — `pid` is the OS *process id*) show these states and are the first tool for diagnosing a hung program. More: LLD §12.3.

---

## 14.4 Interruption: cooperative cancellation

### What it is

Java has no safe way to forcibly kill a thread (`Thread.stop()` was deprecated because it could leave objects half-updated; since Java 20 it throws `UnsupportedOperationException`, and JDK 26 removed it entirely). Instead, one thread **requests** another to stop by calling `t.interrupt()`, which sets the target's **interrupt flag** (a per-thread true/false marker meaning "someone asked you to stop"). The target decides how and when to stop. Analogy: tapping a colleague on the shoulder, not pulling their chair away.

### How it works

1. `t.interrupt()` sets `t`'s interrupt flag.
2. If `t` is blocked in an **interruptible** method (`sleep`, `wait`, `join`, `BlockingQueue.take`, `Lock.lockInterruptibly`, `Future.get`…), that method wakes up and throws **`InterruptedException`** — and **clears the flag**.
3. If `t` is busy computing, nothing happens automatically; the code must check `Thread.currentThread().isInterrupted()` periodically.
4. `Thread.interrupted()` (static) returns the current thread's flag **and clears it**; `isInterrupted()` only reads it.

`code/ch14/threads/Interruption.java` (key parts):

```
try {
    Thread.sleep(60_000);
} catch (InterruptedException e) {
    // flag was cleared when the exception was thrown
    Thread.currentThread().interrupt();          // restore it so callers can see the request
}

while (!Thread.currentThread().isInterrupted()) loops++;     // busy code polls the flag

Thread.currentThread().interrupt();
Thread.interrupted();   // true  (and clears)
Thread.interrupted();   // false
```

Output:

```
sleeper: woke up early, flag now = false
sleeper: flag restored = true
worker: saw the interrupt and stopped cleanly (loops > 0: true)
interrupted() first call  = true
interrupted() second call = false
```

### Common mistakes

- **Swallowing** `InterruptedException`: `catch (InterruptedException e) {}`. The cancellation request is lost; `shutdownNow()` on a pool then can't stop the task. Either **rethrow** it (declare `throws InterruptedException`) or **restore the flag** with `Thread.currentThread().interrupt()` — JCIP's rule, and why every `catch` in this chapter's code does so.
- Using a plain `boolean` field as a stop flag without `volatile` (§14.7). Interruption is preferable because it also wakes blocked threads.
- Assuming interruption stops *non-interruptible* blocking such as classic `java.io` socket reads — closing the socket is the way to unblock those.

### In the interview

> **30-second answer:** "Interruption is cooperative cancellation: `interrupt()` sets a flag; blocking methods respond by throwing `InterruptedException` and clearing the flag; long computations should poll `isInterrupted()`. When I catch `InterruptedException` and can't rethrow it, I restore the flag with `Thread.currentThread().interrupt()`."

- *Why was `Thread.stop` removed?* → It released all locks mid-operation, exposing corrupted state.

---

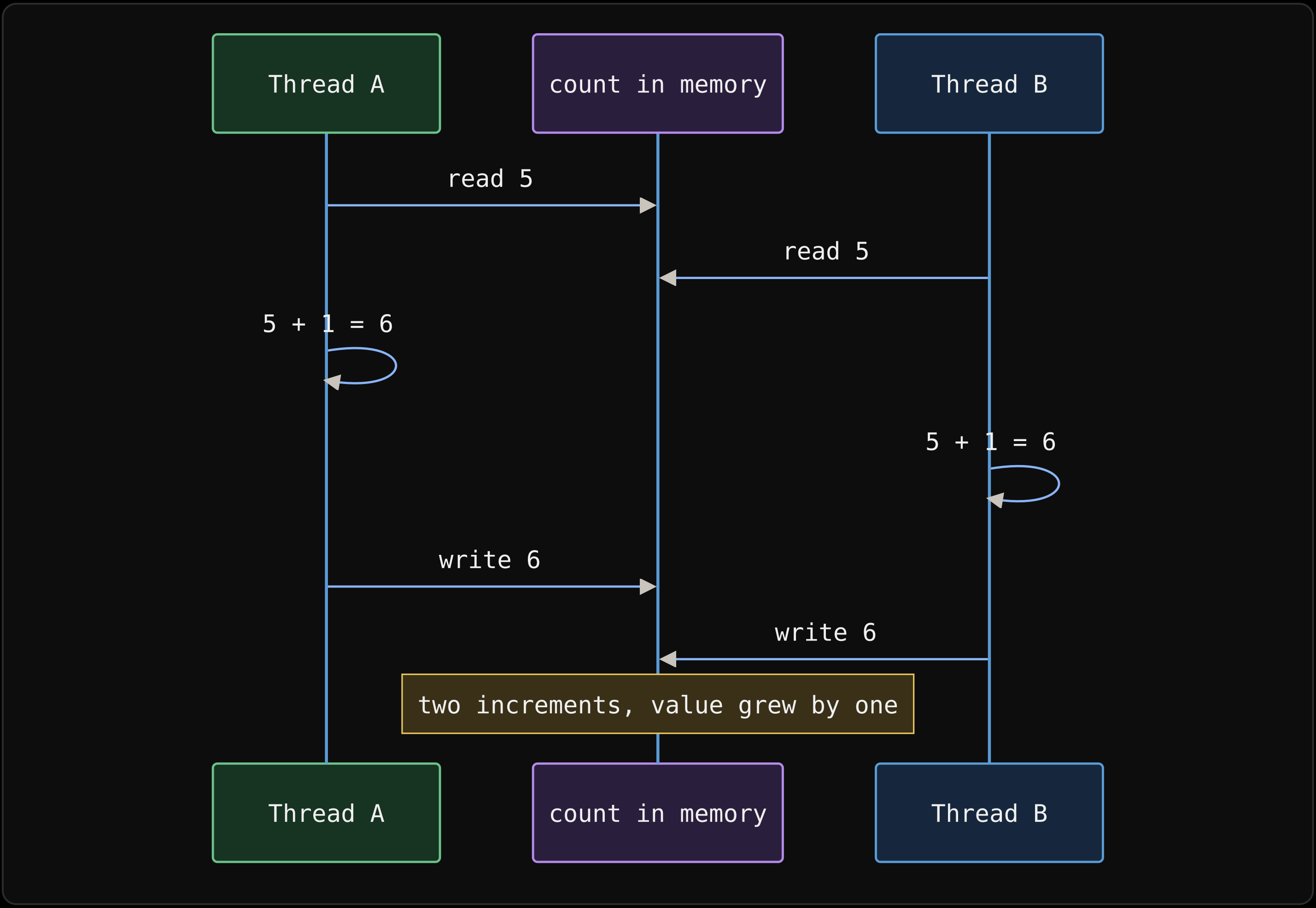
## 14.5 Race conditions

### What it is

A **race condition** is a bug where the result depends on the unlucky timing (**interleaving** — how the steps of different threads happen to get mixed together) of threads. The two classic shapes:

1. **Read-modify-write**: `count++` is really *read* `count` → *add* 1 → *write* back. If two threads both read 5, both write 6: one increment is **lost**.
2. **Check-then-act**: `if (seat.isFree()) seat.book(me);` — between the check and the act another thread can book the same seat. **Lazy initialisation** (creating an object only the first time it's needed: `if (instance == null) instance = new X();`) has the same shape.

A **data race**, more narrowly, is when two threads access the same variable without synchronisation and at least one writes. Data races also cause *visibility* problems (§14.7).



`code/ch14/race/RaceCondition.java` (key parts) — 4 threads each add 1,000,000:

```
run(() -> { for (int i = 0; i < PER_THREAD; i++) unsafeCount++; });
run(() -> { for (int i = 0; i < PER_THREAD; i++) synchronized (LOCK) { syncCount++; } });
run(() -> { for (int i = 0; i < PER_THREAD; i++) atomicCount.incrementAndGet(); });
run(() -> { for (int i = 0; i < PER_THREAD; i++) adder.increment(); });

void bookUnsafe(String who) {
    if (owner == null) {           // check ...
        sleepQuietly();            // (real work would widen the window like this)
        owner = who;               // ... then act
        successes++;
    }
}
synchronized void bookSafe(String who) { ...same body... }
```

Output (two runs; the first line differs every time):

```
unsynchronized count++ : 1,127,175   [nondeterministic - lost updates, changes every run]
synchronized           : 4,000,000
AtomicInteger          : 4,000,000
LongAdder              : 4,000,000
unsafe booking attempt 1: successful bookings = 2   [nondeterministic - 2 means double-booked]
unsafe booking attempt 2: successful bookings = 2   [nondeterministic - 2 means double-booked]
unsafe booking attempt 3: successful bookings = 2   [nondeterministic - 2 means double-booked]
synchronized booking: successful bookings = 1
```

```
unsynchronized count++ : 1,927,211   [nondeterministic - lost updates, changes every run]
...
```

Almost **three quarters of the increments vanished** in the first run, and a different number in the second. The seat was double-booked every time. Both fixed versions are exact.

### Why these bugs are nasty

- They may **never appear** in tests on a laptop and then appear under **production** load (production = the real system that real users rely on).
- Adding a `println` to debug them changes the timing and often hides them ("Heisenbugs").
- The JIT compiler (the JVM's *just-in-time* compiler, which turns hot code into machine code while the program runs) may keep a non-volatile field in a **register** (a tiny storage slot inside the CPU core) for a whole loop, making results even stranger than simple interleaving suggests.

### How to fix (preview of the rest of the chapter)

1. **Don't share** — confine data to one thread (local variables, `ThreadLocal` — a variable that holds a separate value per thread) — §14.14.
2. **Don't mutate** — share only immutable objects (records with immutable fields) — §14.14.
3. **Synchronise** every access to shared mutable state — `synchronized` (§14.6), locks (§14.9).
4. **Use atomic or concurrent building blocks** — `AtomicInteger`, `ConcurrentHashMap.merge` (§14.10, §14.12).

More examples and the "critical section" framing: LLD §12.5.

### Self-check

1. Why is `count++` not atomic? → It's three steps: read, add, write.
2. Name the other classic race shape. → Check-then-act (e.g. lazy initialisation, "if free then book").
3. Does making a field `volatile` fix `count++`? → No (§14.7).

---

## 14.6 `synchronized` and intrinsic locks

### What it is

Every Java object has a built-in lock called its **intrinsic lock** or **monitor**. A `synchronized` block or method acquires it on entry and releases it on exit — even if an exception is thrown. While one thread holds a monitor, other threads trying to enter a block synchronised on the *same object* wait (state `BLOCKED`).

`synchronized` gives two guarantees:

1. **Mutual exclusion (atomicity)**: only one thread at a time runs code guarded by that lock. (*Atomic* = happens as one indivisible step that other threads can't see half-done.)
2. **Visibility**: everything a thread wrote before releasing a lock is visible to the next thread that acquires the *same* lock (a *happens-before* edge — a guaranteed "this write is visible to that read" ordering — Chapter 15 §15.8).

### How it works

| Form | Lock used |
|---|---|
| `synchronized void m()` (instance method) | `this` |
| `static synchronized void m()` | the `Class` object (`Counter.class`) |
| `synchronized (obj) { ... }` | `obj` — any object you choose |

Intrinsic locks are **reentrant** (re-enterable by the thread that already holds them): a thread that holds a lock can enter another block guarded by the same lock (e.g. a synchronized method calling another synchronized method on `this`) without deadlocking itself. The JVM counts the entries.

`code/ch14/sync/SynchronizedBasics.java` (key parts):

```java
static class Counter {
    private int value;
    public synchronized void increment() { value++; }                  // locks this
    public void incrementBlock() { synchronized (this) { value++; } }  // same lock
    public synchronized int get() { return value; }                    // reads need it too
    public synchronized void addTwice() { increment(); increment(); }  // reentrant
    public static synchronized String describe() { ... }               // locks Counter.class
}
synchronized (c) { Thread.holdsLock(c); Thread.holdsLock(Counter.class); }

Object lockA = new Object(), lockB = new Object();       // different locks don't exclude each other
```

Output:

```
4 threads x 500,000 synchronized increments = 2000000
reentrant call worked, value = 2000002
static synchronized locks Counter.class
inside synchronized(c): holdsLock(c)=true holdsLock(Counter.class)=false
different locks -> both threads were inside at once (latch reached 0: true)
```

The last line proves the most common mistake: protection only works if **all** code touching the data uses **the same lock**. Here the two threads each held a different lock, so both were "inside" at once (the latch could only reach 0 because both got in).

### Guidelines

- **Guard every access** — reads too. An unsynchronised `get()` might return a **stale** (out-of-date) value (visibility).
- **Keep critical sections short**; never do slow I/O (input/output: disk, network) or call unknown code (**callbacks** and **listeners** — functions other code registered to be called when something happens) while holding a lock — that's how deadlocks and stalls happen.
- **Prefer a private final lock object** (`private final Object lock = new Object();`) over `synchronized(this)` in library code, so outside code can't grab your lock and interfere.
- **Never lock on** `String` literals, boxed integers (`Integer` cache!) or other shared/**interned** objects (objects the JVM reuses so one copy is shared program-wide), and not on a field that can be reassigned.
- Document which lock guards which fields with JCIP's `@GuardedBy("lock")` **annotation** (a `@` label on code for tools and readers; §14.14).
- Since Java 24 (JEP 491 — a JEP, *JDK Enhancement Proposal*, is the numbered design document for a Java feature), `synchronized` no longer "pins" (glues to one OS thread) virtual threads — details in Chapter 15 §15.5.

### Common mistakes

- Synchronising the writer but not the reader.
- Synchronising two methods on *different* objects and assuming they're mutually exclusive (static vs instance synchronized methods use different locks!).

- Thinking `Collections.synchronizedList(list)` makes *iteration* safe — you must still `synchronized (list) { for (...) }` (§14.12).

### In the interview

> **30-second answer:** "`synchronized` acquires the monitor of an object — `this` for instance methods, the `Class` for static ones, or any object for a block. It gives mutual exclusion and visibility, it's reentrant, and the lock is released automatically, even on exceptions. All accesses to a piece of shared state must use the same lock."

- *Is `synchronized` reentrant?* → Yes; the same thread can re-acquire a lock it holds.
- *Static and instance synchronized methods — do they block each other?* → No; different locks (`Class` object vs instance).

Deeper treatment with diagrams: LLD §12.7.

---

## 14.7 `volatile` and visibility

### What it is

Modern CPUs keep copies of memory in per-core caches, and both the compiler and the CPU reorder instructions for speed. So without synchronisation, a value written by one thread **may never become visible** to another thread, or become visible out of order. A field declared **`volatile`** fixes visibility:

- Every read of a volatile field sees the **most recent write** by any thread.
- Writes made *before* a volatile write become visible to a thread that later *reads* that volatile field (it creates a happens-before edge — Chapter 15 §15.8).
- `volatile long`/`double` reads and writes are atomic; plain ones are not guaranteed to be (see the table below).

`volatile` does **not** provide mutual exclusion: `volatileCounter++` is still read-modify-write.

**Atomic assignments (JLS 17.7).** A single read or write is *atomic* when another thread can never observe half of it. The JLS (the *Java Language Specification*, the official rulebook of the language) guarantees:

| Access | Atomic? |
|---|---|
| read or write of `boolean`, `byte`, `char`, `short`, `int`, `float` | yes |
| read or write of any **reference** (even on 64-bit JVMs) | yes |
| plain `long` / `double` | **not guaranteed**: the JVM may perform it as two 32-bit writes, so a reader can see 32 bits of an old value and 32 bits of a new one (**word tearing**) |
| `volatile long` / `volatile double`, or access under a lock, `AtomicLong`, a `VarHandle` | yes |
| `x++`, `x += 2`, `if (x == 0) x = 1` | **no**: these are compound operations (§14.5), whatever the type |

Two interview points. **Atomic is not visible**: without a happens-before edge an atomic `int` write can go unseen indefinitely; atomicity only rules out torn values. And the JLS *encourages* JVMs not to split 64-bit values (64-bit HotSpot writes a `long` in one instruction), so tearing is legal but rarely observable — you can't test your way to safety. `code/ch14/sync/Tearing.java` flips a field between `0L` and `-1L` 20 million times while another thread looks for any other value:

```
long v = (i & 1) == 0 ? A : B;  plain = v;              // writer
long v = plain;  if (v != A && v != B) torn[0]++;         // reader: a mix of the two halves
```

```
plain long   torn reads: 0   [machine-dependent: 64-bit JVMs write a long in one instruction; a 32-bit JVM may
tear]
volatile long torn reads: 0   (guaranteed 0 by JLS 17.7)
```

Rule: share a `long`/`double` across threads only as `volatile`, under a lock, or in an `AtomicLong`. Sources: JLS §17.7 "Non-Atomic Treatment of double and long", JLS §17.4.

`code/ch14/sync/VolatileFlag.java` (key parts):

```java
private static volatile boolean running = true;          // without volatile the worker MAY spin forever

Thread worker = new Thread(() -> { long spins = 0; while (running) spins++; ... });
worker.start();
Thread.sleep(200);
running = false;                                         // guaranteed to be seen

// 4 threads x 1,000,000 of volatileCounter++
```

Output:

```
worker saw running=false and stopped (spins > 0: true)
volatile count++ from 4 threads: 1,113,395 of 4,000,000   [nondeterministic - still loses updates]
```

(Why "may spin forever" without `volatile`: the JIT is allowed to read `running` once and turn the loop into `while (true)`. Whether it does depends on the JVM and timing, which is exactly why you can't test your way to safety — follow the rules instead.)

### When to use `volatile`

- A **status flag** written by one thread and read by others (`shutdownRequested`).
- **Publishing** an immutable object reference (`volatile Config current;` replaced wholesale).
- The double-checked-locking **idiom** (a standard code pattern; here, lazy initialisation that only locks the first time) (Chapter 15 §15.8).

### When not to use it

When the new value depends on the old (`count++`, `balance -= x`), or several fields must change together — use locks or atomics.

| | `volatile` | `synchronized` | `AtomicInteger` |
|---|---|---|---|
| Visibility | ✓ | ✓ | ✓ |
| Atomic compound update (`++`) | ✗ | ✓ | ✓ (single variable) |
| Multi-variable invariants | ✗ | ✓ | ✗ |
| Blocking | never | may block | never (retries) |

More: LLD §12.6.

## 14.8 `wait` / `notify`: guarded blocks

### What it is

Sometimes a thread must wait until a *condition* becomes true ("the queue is not empty"). Every object's monitor has a **wait set**:

- `obj.wait()` — releases `obj`'s lock and sleeps until notified (or interrupted, or a *spurious wakeup* — waking up for no reason, which the JVM is allowed to do); then re-acquires the lock before returning.
- `obj.notify()` — wakes **one** waiting thread (which one is unspecified). `obj.notifyAll()` wakes **all** of them.
- All three must be called **while holding `obj`'s lock**, or you get `IllegalMonitorStateException`.

### How it works — the guarded-block idiom

1. Acquire the lock (`synchronized`).
2. `while` the condition is false, `wait()`. (A `while`, not `if`: after waking, the condition may already be false again, or the wakeup may be spurious — §14.8.2.)
3. Do the action.
4. After changing state that others wait on, `notifyAll()`.

`code/ch14/sync/WaitNotify.java` — a two-slot buffer between a **producer** (a thread that creates work items) and a **consumer** (a thread that processes them):

```java
public synchronized void put(T item) throws InterruptedException {
    while (items.size() == capacity) wait();     // wait for "not full"
    items.add(item);
    notifyAll();                                 // wake consumers
}
public synchronized T take() throws InterruptedException {
    while (items.isEmpty()) wait();              // wait for "not empty"
    T item = items.remove();
    notifyAll();                                 // wake producers
    return item;
}
```

Output:

```
consumer received in order: 1 2 3 4 5 6 7 8 9 10
sum = 55 (1..10 = 55)
wait() without the lock -> IllegalMonitorStateException
```

The producer ends with a **poison pill** (`-1`), a special value meaning "no more items" — a common way to stop consumers.

### When not to use it

In real code, use `BlockingQueue` (§14.12), `CountDownLatch`/`Semaphore` (§14.13) or `Condition` (§14.9), which are easier to get right. Know `wait`/`notify` for interviews and legacy code. `notify()` (single) is only safe when all waiters wait for the same condition and any one can proceed; otherwise it can wake the wrong thread (§14.8.3) — `notifyAll()` is the safe default. Step-by-step coverage: LLD §12.13 and the blocking queue built two ways in LLD §13.8.

### Self-check

1. Why `while` and not `if` around `wait()`? → Spurious wakeups, and the condition may have changed before the thread re-acquired the lock.
2. Does `wait()` release the lock? → Yes (only that object's lock); `sleep()` does not release any lock.
3. Why can a `release()` before `acquire()` work, but a `notify()` before `wait()` not? → A semaphore stores permits; a condition variable has no memory.

### 14.8.1 Mutex vs semaphore vs monitor

Three classic primitives that are often confused:

| | **Mutex (mutual-exclusion lock)** | **Semaphore (Dijkstra, 1965)** | **Monitor (Brinch Hansen 1973, Hoare 1974)** |
|---|---|---|---|
| What it is | a lock: one thread inside at a time | a counter of **permits**: `acquire` takes one (waits at 0), `release` adds one | a mutex **plus** one or more **condition variables** (queues where threads wait for a condition), bundled with the data they protect |
| Owner | yes: only the holder may unlock | **none**: any thread may `release` | the mutex part has an owner |
| Memory of signals | — | **yes**: a `release` with no waiter is kept as a permit | **no**: a `notify`/`signal` with no waiter is lost (§14.8.3) |
| Re-entrant | Java's are (`synchronized`, `ReentrantLock`) | no: a thread asking for a second permit can block itself | yes in Java |
| Can admit N threads | no (exactly 1) | yes (N permits) | no (1 inside, others wait) |
| Java | `synchronized` lock, `ReentrantLock` | `java.util.concurrent.Semaphore` | every object (`synchronized` + `wait`/`notify`, one condition); `Lock` + `Condition` (many conditions) |
| Use for | protecting a critical section | limiting concurrency (10 DB connections), signalling between threads | "wait until a state is true" (buffer not empty / not full) |

A **binary semaphore** looks like a mutex but has no owner: another thread may release it, and an extra `release()` raises it to 2 permits. Each primitive can build the others — a monitor implements a semaphore with a counter and `while (count == 0) wait()` (§22.2), semaphores implement mutexes and condition waits — but monitors make the common case, waiting for a state change under a lock, hard to get wrong. `code/ch14/sync/MonitorPitfalls.java` part 1:

```
1 mutex locked twice by its owner, holdCount = 2
1 semaphore: same thread tries a 2nd permit -> false (acquire() here would block forever)
1 other thread unlock(): IllegalMonitorStateException (not the owner)
1 other thread release(): ok, permits = 1
1 extra release() -> permits = 2 (a 'binary' semaphore now admits 2)
```

Deeper design discussion: LLD §12.12. Sources: `Semaphore`, `ReentrantLock`, `Object` Javadoc.

### 14.8.2 Hoare vs Mesa monitors: why Java needs `while`

When a thread signals a condition, *who runs next*? The two classic answers:

| | **Hoare semantics (signal-and-wait)** | **Mesa semantics (signal-and-continue)** |
|---|---|---|
| On `signal` | the lock passes **directly** to one waiter, which runs at once; the signaller is suspended | the signaller **keeps** the lock and carries on; the waiter is only moved to the queue for the lock |
| When the waiter resumes | immediately, with the condition **guaranteed** true | later, after competing for the lock, so another thread may have made the condition false again |
| Correct waiting idiom | `if (!condition) wait();` | **`while (!condition) wait();`** |
| Cost | two extra context switches per signal | cheap; the signal is just a hint |
| Where | Hoare's 1974 paper, teaching languages | Xerox's Mesa language (Lampson & Redell, 1980), **Java**, POSIX threads (`pthread_cond_wait`), C#, Python |

Java monitors are Mesa-style: `notify`/`notifyAll` and `Condition.signal` only make waiters *eligible* to re-acquire the lock, and `Object.wait`/`Condition.await` may also return with no signal at all (a **spurious wakeup**, allowed by their Javadoc). In exchange, broadcast and timed waits are easy, since a woken thread always re-checks. Part 2: two consumers wait on an empty box, one item arrives, `notifyAll` wakes both:

```
synchronized int takeWithIf() throws InterruptedException {
    if (items.isEmpty()) wait();          // checked once only
    return items.removeFirst();           // the other consumer may have emptied it
}
synchronized int takeWithWhile() throws InterruptedException {
    while (items.isEmpty()) wait();       // re-check after every wakeup
    return items.removeFirst();
}
```

```
2 if   : consumers that got an item = 1, crashed with NoSuchElementException = 1
2 while: after 1 item, one consumer is back in wait() instead of failing
2 while: consumers that got an item = 2, crashed with NoSuchElementException = 0
```

Under Hoare semantics the `if` version would be correct. Under Java's Mesa semantics the second consumer wakes to an empty box. Sources: `Object.wait`, `Condition` Javadoc (spurious wakeups); JLS §17.2.

### 14.8.3 Missed signals

A **missed signal** (or *lost wakeup*) happens when a notification is sent while nobody is waiting yet: condition variables have **no memory**, so the later waiter sleeps forever. Three common ways it happens:

1. **Signal before wait**: the worker finishes and calls `notify()` before the main thread reaches `wait()`.
2. **Check outside the lock**: `if (!done) { /* signal lands here */ synchronized (lock) { lock.wait(); } }`. Java blocks the worst form by requiring the lock for `wait` (otherwise `IllegalMonitorStateException`), but checking the condition *before* taking the lock still opens the gap.
3. **`notify()` wakes the wrong thread**: producers and consumers share one wait set, and the single wakeup goes to a thread that can't proceed. That thread waits again, and the thread that could proceed is never woken. `notifyAll()` or separate `Condition`s avoid it.

The fix is always the same: **record the event in state guarded by the lock, and wait in a `while` loop on that state.** Alternatively, use a primitive that remembers: `Semaphore` permits, `CountDownLatch`, the `LockSupport` permit (§14.9.1) or a `BlockingQueue`. Demo part 3, where the signal is sent first every time:

```
synchronized (LOCK) { LOCK.wait(500); }                    // naive: nothing remembers the notify()
synchronized (LOCK) { while (!ready) LOCK.wait(500); }     // signaller did: ready = true; notifyAll()
sem.release(); ... sem.tryAcquire(500, MILLISECONDS);      // permit stored before anyone waits
```

```
3 naive wait: still waiting until the 500 ms timeout: true -> the notify() was lost
3 flag + while: returned at once: true
3 semaphore: permit was remembered: true
```

**In the interview:** "Java monitors are Mesa-style and allow spurious wakeups, so always wait in a `while`. Condition variables don't remember signals, so keep the condition in a guarded field; a semaphore or latch remembers releases."

---

## 14.9 Explicit locks: `ReentrantLock`, `Condition`, `ReadWriteLock`, `StampedLock`

### What it is

`java.util.concurrent.locks` (Java 5) provides lock *objects* with the same mutual-exclusion and visibility guarantees as `synchronized`, plus features `synchronized` lacks:

| Feature | `synchronized` | `ReentrantLock` |
|---|---|---|
| Released automatically | ✓ (end of block) | ✗ — **must `unlock()` in `finally`** |
| Try without waiting / with timeout | ✗ | `tryLock()`, `tryLock(time, unit)` |
| Interruptible wait for the lock | ✗ | `lockInterruptibly()` |
| Fairness (FIFO-ish order — first in, first out: longest waiter goes next) | ✗ | `new ReentrantLock(true)` |
| Multiple wait conditions | one wait set per object | many `Condition`s per lock |
| Lock/unlock in different methods | ✗ | ✓ (use with care) |
| Inspect (`isLocked`, `getHoldCount`, queued threads) | ✗ | ✓ |
| Syntax | concise, can't forget to unlock | verbose |

### How it works

```
lock.lock();
try {
    // critical section
} finally {
    lock.unlock();          // ALWAYS in finally, or an exception leaves it locked forever
}
```

`code/ch14/locks/LocksDemo.java` (key parts):

```
lock.lock(); try { lock.lock(); try { ... } finally { lock.unlock(); } } finally { lock.unlock(); }  // reentrant
boolean got = lock.tryLock(150, TimeUnit.MILLISECONDS);      // another thread holds it
lock.lockInterruptibly();                                     // in a waiter thread, then waiter.interrupt()

Condition ready = l.newCondition();
while (!flag[0]) ready.await();                               // like wait(); signalAll() like notifyAll()

ReadWriteLock rw = new ReentrantReadWriteLock();
rw.readLock().lock();   // 3 readers wait at a CyclicBarrier(3) while holding the read lock
rw.writeLock().lock();  // exclusive: a reader's tryLock fails
```

Output:

```
holding lock: isHeldByCurrentThread=true holdCount=1
re-entered: holdCount=2
tryLock(150ms) while another thread holds it -> false after >=150ms: true
waiter: interrupted while waiting for the lock
Condition: woke up, flag = true
ReadWriteLock: readers holding the read lock simultaneously = 3
writer holds write lock; can a reader get in now? false
```

The barrier trick proves read-sharing: a `CyclicBarrier(3)` only opens when three threads are waiting at it, and they were all holding the read lock at the time.

#### `ReadWriteLock` and `StampedLock`

- **`ReentrantReadWriteLock`**: many readers **or** one writer. Worth it when reads vastly outnumber writes and critical sections are not tiny (e.g. an in-memory catalogue refreshed hourly). For small, fast sections a plain lock or `ConcurrentHashMap` is usually faster.
- **`StampedLock`** (Java 8): adds **optimistic reads** — read without locking, then `validate(stamp)` to check that no write happened; fall back to a real read lock if one did. Very fast for read-mostly data, but **not reentrant** and doesn't support `Condition`s — for experts. Examples: LLD §12.10.

### When not to use explicit locks

If you don't need `tryLock`, timeouts, interruptibility, fairness or several conditions, **prefer `synchronized`**: shorter, impossible to forget to unlock, and (since Java 24) equally friendly to virtual threads. Fair locks have much lower throughput; use them only when starvation is a real problem.

### In the interview

> **30-second answer:** "`ReentrantLock` offers what `synchronized` doesn't: `tryLock` with timeouts to avoid deadlock, interruptible acquisition, optional fairness and multiple `Condition`s. The price is that I must unlock in `finally`. I default to `synchronized` and switch to `ReentrantLock` when I need those features; `ReadWriteLock` for read-heavy data, `StampedLock` for optimistic reads."

- *How does `tryLock` help against deadlock?* → A thread that can't get the second lock backs off, releases the first and retries, breaking the circular wait (Chapter 15 §15.9).

### 14.9.1 `LockSupport`: the park/unpark primitive

Every blocking class in `java.util.concurrent` (locks, `Condition`, latches, semaphores, `FutureTask`, blocking queues) finally suspends threads with **`LockSupport`**, most via `AbstractQueuedSynchronizer` (AQS), which keeps the queue of waiters. Each thread has one **permit**, available or not:

| Method | Effect |
|---|---|
| `park()` / `park(blocker)` | if the permit is available, consume it and return at once; otherwise block (state `WAITING`) |
| `parkNanos(ns)` / `parkUntil(deadlineMs)` | the same with a timeout (`TIMED_WAITING`) |
| `unpark(thread)` | make that thread's permit available; if it is parked, it wakes up |
| `getBlocker(thread)` | the object passed to `park(blocker)`; thread dumps print it ("parking to wait for …") |

Rules that follow from "one permit, not a counter":

- **`unpark` before `park` is remembered**, so park/unpark has no missed-signal problem (unlike `notify`, §14.8.3).
- **Permits don't accumulate**: two `unpark`s followed by two `park`s block on the second `park`.
- `park` also returns **when the thread is interrupted** (no exception; the flag stays set) and may return **for no reason** (spuriously). So callers always re-check their condition in a loop, and many first check `Thread.interrupted()`.
- Unlike `wait`, `park` needs no lock. When a virtual thread parks, it unmounts from its carrier, which is how JDK locks and queues free carriers.

`code/ch14/locks/ParkDemo.java` (key parts):

```java
LockSupport.unpark(Thread.currentThread()); LockSupport.park();            // 1 returns at once
LockSupport.unpark(t); LockSupport.unpark(t); LockSupport.parkNanos(1);   // 2 one permit only...
LockSupport.parkNanos(300_000_000L);                                      //   ...this one waits
Thread sleeper = new Thread(() -> LockSupport.park(blocker));             // 3 unpark from main
Thread victim  = new Thread(() -> { LockSupport.park(); flag[0] = Thread.currentThread().isInterrupted(); });
```

Output:

```
1 unpark then park: returned immediately: true
2 second park after two unparks waited for the timeout: true
3 sleeper state = WAITING, blocker recorded: true
3 after unpark: woke = true
4 park returned on interrupt, no exception, interrupt flag still set: true
```

Use `LockSupport` directly only when building your own synchronizer (or extend AQS); application code should use the higher-level classes. Source: `LockSupport` and `AbstractQueuedSynchronizer` Javadoc.

## 14.10 Atomic variables

### What it is

`java.util.concurrent.atomic` classes (`AtomicInteger`, `AtomicLong`, `AtomicBoolean`, `AtomicReference<V>`, `AtomicIntegerArray`, …) offer single-variable operations that are **atomic without locks**. They're built on the CPU's **compare-and-set (CAS)** instruction (a single hardware step that checks and updates a memory location together): "if the value is still `expected`, set it to `new` and return true; otherwise change nothing and return false". A thread that loses the race simply **retries**; no thread ever blocks.

Analogy: editing a shared document with version numbers — "save my change only if the document is still version 17; otherwise reload and try again."

`code/ch14/atomic/AtomicDemo.java` (key parts):

```java
AtomicInteger n = new AtomicInteger(10);
n.incrementAndGet();  n.getAndIncrement();  n.addAndGet(5);
n.compareAndSet(17, 100);                  // true: value was 17
n.compareAndSet(17, 200);                  // false: value is 100 now
n.updateAndGet(x -> x * 2);
n.accumulateAndGet(50, Math::max);

do { prev = hits.get(); next = prev + 1; } while (!hits.compareAndSet(prev, next));   // what updateAndGet does

AtomicReference<Range> range = new AtomicReference<>(new Range(0, 10));
range.updateAndGet(r -> new Range(r.low() + 1, r.high() + 1));     // 4 threads x 10,000

LongAdder requests = new LongAdder();       // 8 threads x 100,000 increments
```

Output:

```
incrementAndGet -> 11
getAndIncrement -> 11 (now 12)
addAndGet(5)    -> 17
CAS 17->100     -> true, value 100
CAS 17->200     -> false, value 100 (expected value was stale)
updateAndGet x2 -> 200
accumulateAndGet(max with 50) -> 200
hand-written CAS loop -> 1
AtomicReference after 40,000 shifts -> Range[low=40000, high=40010]
LongAdder sum -> 800000
```

- `AtomicReference` + an **immutable** (unchangeable once built) record lets you update several related values (low and high) atomically: build a new object, CAS it in. The **invariant** (a rule about the data that must always be true) `high - low == 10` always holds.
- The function passed to `updateAndGet` may run **more than once** (on retries), so it must have no side effects.
- **`LongAdder`** (Java 8) spreads increments over several internal cells to avoid CAS **contention** (many threads fighting over the same variable or lock); `sum()` adds them up. Much faster than `AtomicLong` for hot counters (**metrics** — numbers a service records about itself, like request counts), but `sum()` is not an atomic snapshot and there is no `compareAndSet`.

### Trade-offs

| | **Locks** | **Atomics** |
|---|---|---|
| Scope | any number of variables | one variable (or one reference to an immutable object) |
| Under low contention | slower | very fast |
| Under very high contention | threads sleep | many CAS retries burn CPU (`LongAdder` helps) |
| Deadlock possible? | yes | no |

CAS internals, the ABA problem (a value changes A→B→A, so CAS wrongly thinks nothing happened) and `AtomicStampedReference`: LLD §12.15.

### Self-check

1. What does `compareAndSet` return if the current value isn't the expected one? → `false`, and the value is unchanged.
2. When is `LongAdder` better than `AtomicLong`? → Many threads updating, rare reads.
3. Why must the `updateAndGet` lambda be side-effect free? → It may be retried.
4. How do you get a concurrent maximum without a lock? → `LongAccumulator(Long::max, Long.MIN_VALUE)` (or `accumulateAndGet(x, Math::max)` on an `AtomicLong`).

### 14.10.1 The rest of the atomics family

| Class | Adds | Typical use |
|---|---|---|
| `AtomicBoolean` | CAS on a flag | **run once**: `if (started.compareAndSet(false, true)) init();`; start/stop state |
| `AtomicIntegerArray`, `AtomicLongArray`, `AtomicReferenceArray<E>` | every **element** is atomic and volatile (a `volatile int[]` makes only the *array reference* volatile, not its elements) | histograms, per-slot counters, claiming slots with `compareAndSet(i, null, me)`, Bloom-filter bits |
| `AtomicStampedReference<V>` | a reference and an `int` **stamp** (version) compared and set together | fixing **ABA**: A→B→A changes the stamp, so a stale CAS fails |
| `AtomicMarkableReference<V>` | a reference and a `boolean` **mark** set together | **logical deletion** in lock-free linked lists: mark a node's `next` as "deleted" so a concurrent insert after it fails its CAS |
| `AtomicIntegerFieldUpdater`, `AtomicLongFieldUpdater`, `AtomicReferenceFieldUpdater` | atomic operations on a plain **`volatile`** **field** of existing objects, set up once (reflectively) in a `static final` | millions of small objects (nodes, connections) where an `AtomicInteger` each would waste memory. The field must be `volatile`, non-`static`, exactly `int`/`long` and accessible; atomicity holds only among updater operations |
| `VarHandle` (Java 9, `java.lang.invoke`) | the modern low-level equivalent of the updaters, for fields **and** array elements, plus weaker memory-order modes (`getAcquire`/`setRelease`/`getOpaque`) | library and JDK internals; prefer it to updaters in new low-level code |
| `LongAdder`, `DoubleAdder` | sum spread over per-thread cells (§14.10 above) | hot counters, totals. A `DoubleAdder` sum can differ in the last bits between runs because floating-point addition depends on order |
| `LongAccumulator(op, identity)`, `DoubleAccumulator` | generalise the adders to any **associative**, **commutative** operation; `accumulate(x)`, `get()`, `reset()` | concurrent **max/min**: `new LongAccumulator(Long::max, Long.MIN_VALUE)` |

Every reference CAS compares with `==` (identity), not `equals`. With boxed values such as `AtomicStampedReference<Integer>`, two equal `Integer`s outside the small-value cache are different objects, and the CAS fails. `code/ch14/atomic/AtomicFamily.java` (key parts):

```java
if (started.compareAndSet(false, true)) initRuns.incrementAndGet();            // 8 racing threads
histogram.incrementAndGet(i % 4);                                               // AtomicIntegerArray(4)
stamped.compareAndSet("A", "C", seenStamp, seenStamp + 1);                      // after A->B->A
link.attemptMark(next, true); link.compareAndSet(next, "node-new", false, false);
static final AtomicIntegerFieldUpdater<Connection> REFS =
        AtomicIntegerFieldUpdater.newUpdater(Connection.class, "refCount");     // volatile int refCount
REFS_VH = MethodHandles.lookup().findVarHandle(Connection.class, "refCount", int.class);
LongAccumulator maxLatency = new LongAccumulator(Long::max, Long.MIN_VALUE);
```

Output:

```
AtomicBoolean       : 8 racing threads, init ran 1 time(s)
AtomicIntegerArray  : [10000, 10000, 10000, 10000]
AtomicReferenceArray: 5 threads tried to claim seat 0, winners = 1
ABA                 : plain CAS A->C true, stamped CAS A->C false (stamp is now 2)
AtomicMarkableRef   : marked deleted true, later insert after it succeeds? false
IntegerFieldUpdater : refCount = 40000
RefFieldUpdater     : NEW->OPEN true, NEW->OPEN again false
VarHandle CAS       : 40000->0 true, refCount = 0
updater on non-volatile field -> IllegalArgumentException: Must be volatile type
LongAccumulator(max): 99999
DoubleAdder sum     : 1000.0, DoubleAccumulator(min): 0.5
```

Sources: `java.util.concurrent.atomic` package summary and class Javadoc; `VarHandle` Javadoc (JEP 193).

### 14.10.2 Progress guarantees: blocking, obstruction-free, lock-free, wait-free

"Lock-free" is a precise term about **progress**, not about whether a `Lock` object appears in the code:

| Guarantee | Definition | If one thread is paused mid-operation (preempted, page fault, GC) | Java examples |
|---|---|---|---|
| **Blocking** | one thread can stop others from progressing indefinitely | every thread needing its lock stops too | `synchronized`, `ReentrantLock`, `BlockingQueue.put`, and also a **spin lock** made from an `AtomicBoolean` (no `Lock` object, still blocking) |
| **Obstruction-free** | a thread finishes in a finite number of its own steps **if it runs alone** | others still progress, but contending threads can keep undoing each other (livelock) | rare in practice (some transactional-memory designs) |
| **Lock-free** | at any moment, **some** thread finishes in a finite number of steps (the system as a whole progresses); an individual thread may retry forever | others unaffected | CAS-retry loops: `AtomicInteger.updateAndGet`, `ConcurrentLinkedQueue` (Michael-Scott queue), the Treiber stack in LLD §12.15 |
| **Wait-free** | **every** thread finishes in a bounded number of its own steps, so no thread can starve | others unaffected | `volatile` reads/writes; `AtomicInteger.getAndAdd` where the CPU has an atomic add (on x86-64 the JIT emits a single `LOCK XADD`, JDK-7023898); wait-free queues exist but are complex |

Each row implies the ones above it (wait-free ⇒ lock-free ⇒ obstruction-free); the JDK's **non-blocking** (e.g. `ConcurrentLinkedQueue` Javadoc) means lock-free or better. Lock-free code can't deadlock or stall behind a descheduled lock holder, but it is **not automatically faster** — contended CAS retries burn CPU (hence `LongAdder`) — and multi-variable invariants still need locks.

**In the interview:** "Lock-free means *some* thread always makes progress even if others are paused; wait-free means *every* thread finishes in bounded steps. A CAS retry loop is lock-free but not wait-free, because one unlucky thread can keep losing. A spin lock uses no `Lock` class but is still blocking."

## 14.11 Executors and thread pools

### What it is

An `ExecutorService` separates **submitting tasks** from **running them**. You hand it `Runnable`s or `Callable`s; it runs them on a **thread pool** — a set of reusable worker threads fed from a **work queue**. Analogy: a taxi rank. Passengers (tasks) queue; a fixed number of taxis (threads) take the next passenger when free, instead of buying a new car for every trip.

- **`Runnable`** — `void run()`, can't return a value or throw checked exceptions.
- **`Callable<V>`** — `V call() throws Exception`, returns a result. (**Checked exceptions** are the ones the compiler forces you to catch or declare.)
- **`Future<V>`** — a handle to a pending result: `get()` (blocks), `get(timeout, unit)`, `isDone()`, `cancel(mayInterrupt)`, `isCancelled()`, and since Java 19 `state()`, `resultNow()`, `exceptionNow()`.

### Why it matters

1. **Reuse**: creating an OS thread per task costs time and memory; pools **amortise** it (spread the one-off cost over many tasks).
2. **Bounded resources** (with a fixed upper limit): a pool of 20 threads can't create 10,000 threads during a traffic spike.
3. **Lifecycle management**: one place to shut everything down, wait for completion, collect results and exceptions.

### How it works

1. Create a pool (factory methods below, or a `ThreadPoolExecutor` directly).
2. `execute(runnable)` (fire and forget) or `submit(callable)` → `Future`.
3. `invokeAll(tasks)` → all `Future`s when every task is done; `invokeAny(tasks)` → the first successful result, cancelling the rest.
4. Shut down: `shutdown()` (stop accepting, finish queued), `awaitTermination(timeout)` (wait), `shutdownNow()` (interrupt running tasks, return queued ones). Since **Java 19** `ExecutorService` is `AutoCloseable`: **`close()`** = shutdown + wait, so try-with-resources (the `try (X x = …) { }` form that closes `x` automatically) works.

`code/ch14/executors/ExecutorBasics.java` (key parts):

```java
try (ExecutorService pool = Executors.newFixedThreadPool(3)) {
    pool.execute(() -> ...);                                         // 6 tasks
    Future<Integer> f = pool.submit(() -> { Thread.sleep(100); return 6 * 7; });
    f.isDone(); f.get(); f.state(); f.resultNow();

    Future<Integer> bad = pool.submit(() -> Integer.parseInt("forty-two"));
    bad.get();                                                       // ExecutionException wrapping the cause

    Future<String> slow = pool.submit(() -> { Thread.sleep(10_000); return "late"; });
    slow.get(100, TimeUnit.MILLISECONDS);                            // TimeoutException
    slow.cancel(true);                                               // interrupts the sleeping task

    pool.invokeAll(List.of(() -> "a", () -> { Thread.sleep(50); return "b"; }, () -> "c"));
    pool.invokeAny(List.of(() -> { Thread.sleep(300); return "slow mirror"; },
                           () -> { Thread.sleep(10);  return "fast mirror"; }));
}                                                                    // close(): waits for all tasks

ex.shutdown(); ex.awaitTermination(5, TimeUnit.SECONDS); ex.submit(...);   // rejected

try (ScheduledExecutorService sched = Executors.newScheduledThreadPool(1)) {
    sched.scheduleAtFixedRate(() -> ..., 50, 100, TimeUnit.MILLISECONDS);
    sched.schedule(() -> "delayed hello", 50, TimeUnit.MILLISECONDS).get();
}
```

Output:

```
6 tasks ran on at most 3 threads: true
isDone right after submit? false
get() blocks, then returns 42
state = SUCCESS, resultNow = 42
get() -> ExecutionException caused by NumberFormatException
get(100ms) -> TimeoutException; cancel(true) = true, isCancelled = true
invokeAll results in order: abc
invokeAny -> fast mirror
pool closed (all tasks finished)
single-thread executor task
awaitTermination -> true, isTerminated = true
submit after shutdown -> RejectedExecutionException
tick 1 at ~50 ms   [timing approximate]
tick 2 at ~150 ms   [timing approximate]
tick 3 at ~250 ms   [timing approximate]
delayed hello
```

- An exception inside a task **doesn't crash anything**: it's stored in the `Future` and rethrown by `get()` as `ExecutionException` with the original as its *cause*. If you never call `get()`, a `submit`ted task's exception is silently lost — a frequent source of "the job just stopped working" mysteries. (`execute()`d tasks, by contrast, let the exception reach the thread's uncaught-exception handler, which prints it.)
- `invokeAll` returns results in **submission order**, not completion order.
- `scheduleAtFixedRate` runs at a fixed *rate* (start times 100 ms apart); `scheduleWithFixedDelay` waits a fixed *delay* after each run ends. If a periodic task throws, its future runs are silently cancelled (compared with `Timer` in §14.11.2).

### The `Executors` factory methods

| Factory | Pool | Queue | Watch out |
|---|---|---|---|
| `newFixedThreadPool(n)` | exactly n threads | **unbounded** `LinkedBlockingQueue` | a flood of tasks piles up in memory |
| `newCachedThreadPool()` | 0…∞ threads, idle ones die after 60 s | hand-off (`SynchronousQueue`) | a flood creates unbounded threads |
| `newSingleThreadExecutor()` | 1 thread | unbounded | tasks run sequentially in order |
| `newScheduledThreadPool(n)` | n threads | delay queue | periodic tasks die silently on exception |
| `newWorkStealingPool()` (8) | a `ForkJoinPool` | per-thread deques | for many small CPU tasks |
| `newVirtualThreadPerTaskExecutor()` (21) | a new virtual thread per task | none | not a pool at all — Chapter 15 |

### Configuring `ThreadPoolExecutor`

For production services, build the pool explicitly so every limit is a decision. The rules for a new task are:

1. Fewer than **core** threads running → start a new thread.
2. Otherwise → put it in the **queue**.
3. Queue full and fewer than **max** threads → start an extra thread.
4. Otherwise → call the **rejection policy**.

`code/ch14/executors/PoolConfig.java` (key parts) — core 2, max 3, queue capacity 2, 7 tasks that all block:

```java
ThreadPoolExecutor pool = new ThreadPoolExecutor(
        2, 3, 30, TimeUnit.SECONDS,
        new ArrayBlockingQueue<>(2),
        new ThreadPoolExecutor.AbortPolicy());
```

Output:

```
available processors: 14   [machine-dependent]
task 1 accepted  (threads=1, queued=0)
task 2 accepted  (threads=2, queued=0)
task 3 accepted  (threads=2, queued=1)
task 4 accepted  (threads=2, queued=2)
task 5 accepted  (threads=3, queued=2)
task 6 REJECTED  (threads=3, queued=2)
task 7 REJECTED  (threads=3, queued=2)
completed tasks = 5
third task ran in: main (the caller)
```

Note the surprise in step 3: the pool grows beyond the core size only when the queue is **full** — so with an unbounded queue, `max` is never used.

**Rejection policies**: `AbortPolicy` (default, throws `RejectedExecutionException`), **`CallerRunsPolicy`** (the submitting thread runs the task itself — natural **back-pressure** (a signal that pushes back on whoever sends work too fast, slowing them down) that slows the producer; shown in the last line), `DiscardPolicy` (silently drop), `DiscardOldestPolicy` (drop the oldest queued task).

**Sizing** (JCIP): for **CPU-bound** work (§14.1.2), about **$N_{cpu}$ (+1)** threads — more just adds context switches. For tasks that wait, **$N_{threads} \approx N_{cpu} \times (1 + \text{wait time} / \text{compute time})$**; e.g. 8 cores, tasks that wait 90 ms per 10 ms of CPU → ~80 threads. Then measure. For **I/O-bound** tasks on Java 21+, virtual threads usually remove the need for this calculation.

### Common mistakes

- Forgetting to shut down a pool → non-daemon worker threads keep the JVM alive forever.
- Unbounded queues/pools in servers → `OutOfMemoryError` (the JVM ran out of heap memory) under load instead of a clean rejection.
- Tasks that wait on other tasks **in the same bounded pool** → **thread-starvation deadlock** when all threads wait for queued tasks that can't run.
- Ignoring `Future`s from `submit` → exceptions disappear.
- Sharing one pool for fast and slow work → slow tasks hog all threads (use separate pools, the *bulkhead* pattern — named after a ship's watertight walls: a failure in one compartment can't sink the rest).

### In the interview

> **30-second answer:** "I use an `ExecutorService` rather than raw threads: tasks go in as `Runnable` / `Callable`, results and exceptions come back through `Future`, and I shut it down with `shutdown` / `awaitTermination` or try-with-resources on Java 19+. In production I configure `ThreadPoolExecutor` explicitly — bounded queue, a sized max, a rejection policy like `CallerRunsPolicy` for back-pressure, and named threads."

- *`shutdown` vs `shutdownNow`?* → Graceful (queued tasks still run) vs interrupt running tasks and return the queued ones.
- *Why is `newFixedThreadPool` risky?* → Its queue is unbounded.
- *`submit` vs `execute`?* → `submit` returns a `Future` and captures exceptions; `execute` returns nothing and exceptions go to the uncaught handler.

Design discussion: LLD §12.21 Thread pool pattern. Exam tables (`TimeUnit`, `ScheduledExecutorService` methods): OCP Chapter 13.

### 14.11.1 `CompletionService`: results in completion order

`invokeAll` and a loop of `future.get()` return results in **submission** order, so a slow first task holds up all the fast ones. **`ExecutorCompletionService`** wraps an executor and queues each `Future` **when it completes**:

| Method | Does |
|---|---|
| `submit(callable)` / `submit(runnable, result)` | run on the wrapped executor; returns the `Future` |
| `take()` | block until the next task completes and return its (done) `Future` |
| `poll()` / `poll(timeout, unit)` | the same without blocking / with a timeout (`null` if none) |

Call `take()` once per submission; a failed task arrives like any other and its `get()` throws `ExecutionException`. Uses: handling results as they arrive, and "first success wins" — take until one succeeds, then `cancel(true)` the rest (`invokeAny` does this when you don't need the others). `code/ch14/executors/CompletionAndTimers.java`, tasks that sleep 300, 100 and 200 ms:

```java
CompletionService<String> cs = new ExecutorCompletionService<>(pool);
for (Callable<String> j : jobs) cs.submit(j);
for (int i = 0; i < jobs.size(); i++) asReady.add(cs.take().get());
```

```
invokeAll         : [A(300ms), B(100ms), C(200ms)]
CompletionService : [B(100ms), C(200ms), A(300ms)]
```

On Java 8+, `CompletableFuture`s with `thenAccept` give the same "handle each as it finishes" behaviour without a queue (§15.2).

### 14.11.2 `java.util.Timer` vs `ScheduledThreadPoolExecutor`

`java.util.Timer` + `TimerTask` (Java 1.3) is the legacy scheduler. `ScheduledThreadPoolExecutor` (Java 5, what `Executors.newScheduledThreadPool` returns) replaces it; the `Timer` Javadoc itself calls it "effectively a more versatile replacement":

| | `Timer` | `ScheduledThreadPoolExecutor` |
|---|---|---|
| Threads | **one** background thread for all tasks; a slow task delays every other task | a configurable pool of core threads |
| A task throws | the Timer thread **dies**: all tasks are cancelled and every later `schedule` throws `IllegalStateException` | only that task stops; a periodic task's later runs are suppressed and its `ScheduledFuture.get()` throws `ExecutionException`; other tasks keep running |
| Clock | OpenJDK's implementation uses `System.currentTimeMillis()` (wall clock), so a clock change can shift schedules | `System.nanoTime()` (a monotonic clock that never jumps) |
| Task type | must subclass `TimerTask` | any `Runnable` or `Callable`, with a result through `ScheduledFuture` |
| Shutdown | `cancel()`; non-daemon by default (`new Timer(true)` for daemon) | `shutdown()` / `close()` like any executor |

Output of the second half of the same program (the Timer thread's stack trace also goes to stderr, as an uncaught exception):

```
Timer after a task threw -> IllegalStateException: Timer already cancelled.
STPE failing periodic task -> ExecutionException caused by IllegalStateException; isDone = true
STPE other periodic task kept running: true
STPE terminated after shutdown: true
```

Use a scheduled executor, and wrap periodic task bodies in a logging `try/catch`: even there, an exception silently ends that task's schedule. Sources: `ExecutorCompletionService`, `Timer`, `ScheduledThreadPoolExecutor` Javadoc.

## 14.12 Concurrent collections

### What it is

The ordinary collections of Chapter 10 (`ArrayList`, `HashMap`) are not thread-safe. There are two ways to share collections:

1. **Synchronised wrappers** — `Collections.synchronizedList/Map/Set(...)`, and the legacy `Vector`/`Hashtable`: every method takes one lock. Simple, but only one thread at a time, and **compound actions and iteration still need external locking**.
2. **Concurrent collections** (`java.util.concurrent`) — designed for concurrent access with **fine-grained locking** (many small locks, each guarding a small part) or **lock-free** algorithms (built on CAS, so no thread ever waits for a lock), plus **atomic compound operations** built in.

| Class | Use for | Notes |
|---|---|---|
| **`ConcurrentHashMap`** | shared maps, caches, counters | atomic `putIfAbsent`, `computeIfAbsent`, `compute`, `merge`; reads don't block; **no null keys/values**; weakly consistent iterators (they never throw but may or may not see changes made during iteration) |
| `ConcurrentSkipListMap` / `Set` | sorted concurrent map/set | concurrent `TreeMap`; O(log n) (time grows with the logarithm of size — very slowly) |
| **`CopyOnWriteArrayList`** / `Set` | read-mostly lists (listeners, config) | every write copies the array; iterators see a snapshot, never throw CME (`ConcurrentModificationException`) |
| **`ArrayBlockingQueue`** | bounded producer-consumer | `put` waits when full, `take` waits when empty |
| `LinkedBlockingQueue` | producer-consumer (optionally bounded) | unbounded by default |
| `PriorityBlockingQueue`, `DelayQueue` | priority / delayed tasks | unbounded |
| `SynchronousQueue` | direct hand-off (capacity 0) | used by cached thread pools |
| `LinkedTransferQueue` | producer can wait until consumed | |
| `ConcurrentLinkedQueue` / `Deque` (double-ended queue) | non-blocking queue | lock-free; `poll` returns `null` when empty |

`BlockingQueue` methods come in four flavours: throw (`add`/`remove`), special value (`offer`/`poll` → `false`/`null`), **block** (`put`/`take`), and **time out** (`offer(e, t, unit)`/`poll(t, unit)`).

`code/ch14/collections/ConcurrentCollections.java` (key parts):

```
counts.merge(w, 1, Integer::sum);                                   // atomic per key

Integer old = syncMap.get(w);                                       // each call is thread-safe,
syncMap.put(w, old == null ? 1 : old + 1);                          // the combination is NOT

groups.computeIfAbsent(w.substring(0, 1), k -> new CopyOnWriteArrayList<>()).add(w);
counts.put("x", null);                                              // NullPointerException

for (String s : plain) if (s.equals("a")) plain.add("d");           // ArrayList: CME
for (String s : cow)   { seen++; if (s.equals("a")) cow.add("d"); } // snapshot iteration

BlockingQueue<String> queue = new ArrayBlockingQueue<>(3);          // producer put()s, consumer take()s
```

Output:

```
ConcurrentHashMap merge : {and=80000, bat=40000, cat=40000, hat=40000, the=120000}
synchronizedMap get+put : {and=55878, bat=32934, cat=33358, hat=33388, the=76273}   [nondeterministic - lost updates; expected the=120000]
computeIfAbsent sizes   : {a=8, b=4, c=4, h=4, t=12}
ConcurrentHashMap rejects null values -> NullPointerException
ArrayList modify during for-each -> ConcurrentModificationException
CopyOnWriteArrayList: iterator saw 3 elements, list now [a, b, c, d]
BlockingQueue consumed  : [job-1, job-2, job-3, job-4, job-5]
offer on full queue     : offer(2) = false (returns false instead of blocking)
poll with timeout, empty: null
ConcurrentSkipListMap sorted descending? true
ConcurrentLinkedQueue size after 4 x 1000 offers = 4000
```

The second line is the key lesson: a thread-safe collection does **not** make *your sequence of calls* atomic. `get` then `put` is check-then-act; a third of the counts were lost. Use the atomic methods (`merge`, `compute`, `computeIfAbsent`, `putIfAbsent`) — or hold one lock around the whole sequence.

## Common mistakes

- Using `containsKey` + `put` (or `get` + `put`) on a `ConcurrentHashMap` — use `putIfAbsent`/`computeIfAbsent`/`merge`.
- Long or blocking work inside `computeIfAbsent`'s function — it runs while that key's **hash bin** (the internal slot that keys with the same hash land in) is locked, so other updates to keys in the same bin wait. Never modify the same map from inside the function.
- `size()` on a busy `ConcurrentHashMap` is an estimate at that moment; don't use it for logic.
- `CopyOnWriteArrayList` for write-heavy lists — each add copies the whole array: O(n).
- Iterating a `synchronizedList` without `synchronized (list) { ... }` → `ConcurrentModificationException` or worse.

## In the interview

> **30-second answer:** "For shared maps I use `ConcurrentHashMap` and its atomic methods — `merge`, `computeIfAbsent` — because separate `get` and `put` calls are a check-then-act race even on a thread-safe map. `CopyOnWriteArrayList` for read-mostly lists like listeners, `BlockingQueue` for producer-consumer, and the skip-list classes (a **skip list** is a sorted linked list with extra "express lane" links for fast search) when I need ordering."

- *How does `ConcurrentHashMap` scale?* → Since Java 8: CAS for inserting into empty bins and a lock on only the first node of a bin for updates; reads are lock-free. (Java 7 used segments.) A simplified build: LLD §13.7.
- *Why no nulls?* → `get` returning `null` must unambiguously mean "absent"; with concurrent updates you can't follow up with `containsKey` safely.

More in LLD §12.16 and §12.22 Producer-consumer.

## 14.13 Synchronizers: `CountDownLatch`, `CyclicBarrier`, `Semaphore`, `Phaser`

### What it is

**Synchronizers** are objects whose job is to coordinate the *flow* of threads based on their own state.

| Synchronizer | Idea | Reusable? | Typical use |
|---|---|---|---|
| **`CountDownLatch(n)`** | threads `await()` until `countDown()` has been called n times | no (one-shot) | "start all workers together"; "wait until N services are initialised" |
| **`CyclicBarrier(n, action)`** | n threads `await()` each other; when the last arrives, the optional action runs and all proceed | yes | iterative algorithms in phases (simulations) |
| **`Semaphore(permits)`** | `acquire()` takes a permit (waits if none), `release()` returns one | yes | limit concurrency: at most 10 DB (database) connections / API calls |
| **`Phaser`** (7) | like a barrier, but parties can register/deregister dynamically; numbered phases | yes | variable number of participants |
| `Exchanger<V>` | two threads swap objects at a meeting point | yes | double-buffering (one buffer is filled while the other is used, then they swap; rare) |

`code/ch14/synchronizers/Synchronizers.java` (key parts):

```java
CountDownLatch startGate = new CountDownLatch(1), finished = new CountDownLatch(n);
// workers: startGate.await(); work...; finished.countDown();
startGate.countDown();  finished.await();

CyclicBarrier barrier = new CyclicBarrier(3, () -> System.out.println("barrier: all 3 arrived, ..."));
// 3 threads: for (round 0..1) barrier.await();

Semaphore permits = new Semaphore(3);
permits.acquire();
try { /* at most 3 threads here */ } finally { permits.release(); }

Phaser phaser = new Phaser(1);  phaser.register();  phaser.arriveAndAwaitAdvance();
```

Output:

```
latch: work before opening the gate = 0
latch: work after all finished = 40, count = 0
barrier: all 3 arrived, phase 1 complete
barrier: all 3 arrived, phase 2 complete
semaphore: 20 tasks on 10 threads, max ever inside = 3 (limit 3)
phaser: main + 2 workers passed phase 0, current phase = 1
```

- `countDown()` goes in `finally` so a failing worker can't leave the waiter stuck forever. Prefer `await(timeout, unit)` in production.
- A `Semaphore` has no owner: any thread can `release()`, and releasing without acquiring *increases* the permits — always pair `acquire` with `release` in `finally`.
- If one thread waiting at a `CyclicBarrier` is interrupted or times out, the barrier becomes **broken** and all others get `BrokenBarrierException`.

**`Exchanger<V>` in depth.** `exchange(x)` blocks until a partner also calls it; each returns the other's object (the timed form throws `TimeoutException`), and everything done before the hand-over is visible to the partner. With more than two threads the pairing is arbitrary, so it fits two roles — classically **double buffering**: a filler and a drainer swap buffers when the filler's is full, so no buffer is ever shared and nothing is copied or locked.

`Phaser` **in depth.** A `Phaser` is a reusable barrier whose number of **parties** (registered participants) can change between phases:

| Method | Meaning |
|---|---|
| `new Phaser(n)`, `register()`, `bulkRegister(k)` | add parties (at any time) |
| `arriveAndAwaitAdvance()` | arrive and wait for the others (like `CyclicBarrier.await`) |
| `arrive()` | arrive **without** waiting (signal "my part of this phase is done") |
| `arriveAndDeregister()` | arrive and leave for good; later phases need one party fewer |
| `awaitAdvance(phase)` | wait for a phase to finish without being a party |
| `onAdvance(phase, registeredParties)` | override: runs once when a phase completes (like a barrier action); **return `true` to terminate** the phaser. The default terminates when no parties remain |
| `getPhase()`, `isTerminated()` | the phase number (wraps to 0 after `Integer.MAX_VALUE`; **negative** once terminated) |

A phaser holds at most 65,535 parties (**tiered** phasers — children registered with a parent — go beyond and cut contention). Unlike `CyclicBarrier` it has no "broken" state, and `arriveAndAwaitAdvance` ignores interrupts (`awaitAdvanceInterruptibly` doesn't). In the demo, three workers need 1, 2 and 4 phases and `onAdvance` stops after phase 3. `code/ch14/synchronizers/ExchangerPhaser.java` (key parts):

```java
if (buf.size() == 3) buf = exchanger.exchange(buf);         // filler: full buffer out, empty one back
buf = exchanger.exchange(buf); log.add("drained " + buf);   // drainer

Phaser phaser = new Phaser(1) {                             // main is a party
    @Override protected boolean onAdvance(int phase, int registeredParties) {
        return phase >= 3 || registeredParties == 0;        // true = terminate
    }
};
phaser.register();                                          // per worker; the worker loops:
if (r == rounds - 1) phaser.arriveAndDeregister(); else phaser.arriveAndAwaitAdvance();
while (!phaser.isTerminated()) phaser.arriveAndAwaitAdvance();   // main drives the phases
```

Output:

```
Exchanger: [drained [1, 2, 3], drained [4, 5, 6]]
Exchanger: no partner within 100 ms -> TimeoutException
  phase 0 done, parties still registered = 3
  phase 1 done, parties still registered = 2
  phase 2 done, parties still registered = 2
  phase 3 done, parties still registered = 1
Phaser: terminated = true, getPhase() = -2147483644 (negative once terminated)
Phaser: awaitAdvance on a terminated phaser returns at once: -2147483644
```

The registered count fell from 4 (main + 3 workers) as workers deregistered. Sources: `Exchanger`, `Phaser` Javadoc.

Worked examples: LLD §12.12 Semaphores, §12.14 Latch and barrier, and the rate limiter in LLD §13.11.

### Self-check

1. Latch or barrier for "main waits until 5 workers finish"? → `CountDownLatch(5)`.
2. How do you cap concurrent calls to an API at 10? → `Semaphore(10)`, acquire/release around each call.
3. Can a `CountDownLatch` be reset? → No; use `CyclicBarrier` or `Phaser` for reuse.

---

## 14.14 Designing thread-safe classes

*Java Concurrency in Practice* boils thread safety down to one sentence: **manage access to shared, mutable state.** You can remove "shared", remove "mutable", or manage the access:

| Strategy | How | Example |
|---|---|---|
| **Thread confinement** | only one thread ever touches the data | local variables; objects created and used inside one task; one thread owning a UI; `ThreadLocal` (Chapter 15) |
| **Immutability** | no state changes after construction; `final` fields; defensive copies (copying incoming/outgoing mutable data so outsiders can't change yours) | records with immutable components, `String`, `List.of(...)`, `java.time` — always thread-safe |
| **Encapsulated synchronisation** | private state, every access under the same private lock | the `Counter` in §14.6 |
| **Delegation** | keep state in thread-safe components | a `ConcurrentHashMap` field, an `AtomicLong` counter |

Guidelines:

1. **Prefer immutable objects**; make fields `final` unless they must change (Chapter 5).
2. **Don't let `this` escape** from a constructor (starting a thread or registering a listener in the constructor) — other threads could see a half-built object.
3. **Invariants involving several variables** need one lock around all of them (atomics per field aren't enough).
4. **Document** thread-safety guarantees in Javadoc (the `/** … */` API documentation comments) ("This class is thread-safe" / "not thread-safe").
5. **Prefer high-level utilities** (executors, concurrent collections, synchronizers) over `wait`/`notify` and raw threads — *Effective Java* Items 80–81.
6. **Test under load** and use tools: thread dumps, JFR (Java Flight Recorder, the JVM's built-in low-overhead profiler/recorder), and stress-testing harnesses such as OpenJDK's jcstress for low-level code — but remember tests can't prove absence of races; reasoning about happens-before can (Chapter 15).

**Concurrency annotations (guideline 4 in code).** *Java Concurrency in Practice* introduced four annotations that record thread-safety decisions where readers and tools can see them:

| Annotation | On | Promise |
|---|---|---|
| `@ThreadSafe` | class | correct under any concurrent use without extra synchronisation by callers |
| `@NotThreadSafe` | class | callers must confine it to one thread or lock around it |
| `@Immutable` | class | state never changes after construction (so it is also thread-safe) |
| `@GuardedBy("lock")` | field, method | access only while holding the named lock: `"this"`, a field name (`"lock"`), `"Foo.class"`, `"Outer.this"`, `"itself"` (the field's own object) or `"method()"` (the lock returned by a method) |

```
@ThreadSafe
public class Inventory {
    private final Object lock = new Object();
    @GuardedBy("lock") private final Map<String, Integer> stock = new HashMap<>();
    public int available(String sku) { synchronized (lock) { return stock.getOrDefault(sku, 0); } }
}
```

They are **not in the JDK**, and `javac` checks nothing. They come from a library — `net.jcip:jcip-annotations`, the dormant JSR 305 jar (`javax.annotation.concurrent`, never part of the JDK despite the name), or a checker's own versions — and static analysis enforces them: Error Prone (its `@GuardedBy`, `@ThreadSafe`, `@Immutable`, at compile time), the Checker Framework's Lock Checker, SpotBugs. Even unchecked, `@GuardedBy` tells the next maintainer which lock protects which field. Sources: JCIP Appendix A; Error Prone and Checker Framework documentation.

### Choosing the tool

| Need | Reach for |
|---|---|
| Run tasks in the background | `ExecutorService` (virtual threads for I/O-bound — Chapter 15) |
| Counter / accumulator | `AtomicInteger` / `AtomicLong` ; `LongAdder` if very hot |
| Stop flag, publish an immutable snapshot | `volatile` |
| Multi-field invariant | `synchronized` (or `ReentrantLock` if you need `tryLock`/timeouts) |
| Shared map / cache | `ConcurrentHashMap` + atomic methods |
| Producer-consumer | `BlockingQueue` |
| Wait for N things to finish | `CountDownLatch` (or `invokeAll`, or `CompletableFuture.allOf`) |
| Limit concurrent access | `Semaphore` |
| Read-mostly data with rare writes | `CopyOnWriteArrayList`, `ReadWriteLock`, or an immutable snapshot in a `volatile`/`AtomicReference` |

The LLD book's version of this table with interview framing: LLD §12.26; how concurrency appears in design interviews (seat booking, parking lots, inventory): LLD §12.25.

### In the interview (chapter-level)

- *How would you make a seat-booking method safe?* → Make check-and-book atomic: `synchronized` on the show (or a per-seat lock), or `ConcurrentHashMap.putIfAbsent(seatId, userId)` — and in a real system a database **constraint** (a rule like "seat id must be unique" that the database enforces) or `SELECT … FOR UPDATE` (an SQL query that locks the rows it reads until the transaction ends), because several servers run the code.
- *`synchronized` vs `volatile` vs `AtomicInteger`?* → Mutual exclusion + visibility / visibility only / lock-free atomic single-variable updates.
- *What is a thread-safe class?* → One that stays correct under any interleaving of calls from multiple threads without callers adding synchronisation.

## Chapter TL;DR

| Topic | Remember |
|---|---|
| Basics | Program (file) → process (own memory, PID) → thread (shared heap); platform threads preemptive, virtual threads cooperative; Little's law: in-flight = throughput × latency; Amdahl: speed-up ≤ 1/(serial fraction) |
| Threads | Share the heap, own their stack; `start()` (not `run()`) creates a thread; `join()` waits; daemon threads don't keep the JVM alive; `Thread.Builder` (21) |
| States | NEW, RUNNABLE, BLOCKED (monitor), WAITING, TIMED_WAITING, TERMINATED |
| Interruption | Cooperative; blocking calls throw `InterruptedException` and clear the flag; restore it or rethrow |
| Races | Read-modify-write and check-then-act; results vary run to run; tests can't prove absence |
| `synchronized` | Mutual exclusion + visibility; `this` / `Class` / chosen object; reentrant; same lock for every access |
| `volatile` | Visibility (and ordering) only; perfect for flags; `++` still races; plain `long`/`double` may tear (JLS 17.7) |
| `wait`/`notify` | Hold the lock, `while` loop (Mesa semantics + spurious wakeups), prefer `notifyAll`; signals aren't remembered — keep state in a field; mutex = owner, semaphore = counter, monitor = mutex + conditions |
| Locks | `ReentrantLock`: `tryLock`, timeouts, interruptible, fair, `Condition`s — unlock in `finally`; `ReadWriteLock`; `StampedLock`; all built on `LockSupport` park/unpark (one permit) |
| Atomics | CAS-based, lock-free, single variable; `AtomicReference` + immutable object; `LongAdder` for hot counters; arrays, stamped/markable refs, field updaters/`VarHandle`, accumulators; lock-free = someone progresses, wait-free = everyone does |
| Executors | Prefer to raw threads; `Future` + `ExecutionException`; shut down (or `close()`, 19); bounded queues + rejection policy in servers; size by CPU vs wait; `CompletionService` for completion order; scheduled pool, not `Timer` |
| Collections | `ConcurrentHashMap` atomic methods (`merge`, `computeIfAbsent`); COW lists for read-mostly; `BlockingQueue` for producer-consumer |
| Synchronizers | Latch (one-shot), barrier (cyclic), semaphore (permits), phaser (dynamic; `arriveAndDeregister`, `onAdvance` to terminate), exchanger (two-thread swap) |
| Design | Confinement, immutability, encapsulated locking, delegation; document with `@ThreadSafe`/`@GuardedBy` (library annotations, not JDK) |

**Where next:** composing asynchronous work with `CompletableFuture`, Fork/Join, **virtual threads**, structured concurrency, scoped values, the **Java Memory Model** and deadlock/livelock/starvation in Chapter 15 — Concurrency II. Practise with classic problems in LLD Chapter 13; exam traps in OCP Chapter 13. Python comparison (GIL, `threading`, `asyncio`): Python Chapter 16.

# Chapter 15 — Concurrency II: Modern Concurrency

Chapter 14 gave you threads, locks, atomics, thread pools and concurrent collections. This chapter covers how concurrent Java is written **today**: composing **asynchronous** results (results of work that finishes later, while the caller carries on) with `CompletableFuture`, splitting CPU work with Fork/Join, handling huge numbers of **blocking** tasks (tasks that spend time waiting, e.g. for a network reply) with **virtual threads** (Java 21), passing context safely with **scoped values** (final in Java 25), and grouping related tasks with **structured concurrency** (still a **preview** feature — shipped in the JDK for feedback but switched off unless you pass `--enable-preview`, and still allowed to change). It then goes underneath all of them — the **Java Memory Model** (the rules for when one thread's writes become visible to another) and *happens-before* (the ordering guarantee those rules are built on), which explain *why* the Chapter 14 tools work — and ends with the ways concurrent programs stop making progress: deadlock, livelock and starvation.

All runnable code is in `code/` (package `ch15.*`), compiled with `javac --release 25` and run on JDK 26 on a 14-core Apple-silicon laptop. As in Chapter 14, lines tagged `[nondeterministic]` vary between runs and `[machine-dependent]` lines depend on your hardware; timing numbers are rounded. The programs added in §15.6.1 and §15.10.1 were compiled with `javac --release 21` and run on JDK 21 on a 4-core Linux machine. The structured-concurrency example (§15.7) uses a **preview** API, so it lives only in this README and was run separately with `--enable-preview` on JDK 25 and JDK 26.

> **Related material — linked, not repeated.** Virtual threads, `Future`, parallel-stream and thread-problem exam traps: OCP Chapter 13 — Concurrency. Deadlock, livelock and starvation with more diagrams and the dining philosophers: LLD §12.17–12.19 and LLD §13.5. Fork/Join merge sort: LLD §13.10. Parallel streams: Chapter 12.

---

## 15.1 `Future` and its limits

### What it is

A **`Future<V>`** (Java 5, Chapter 14 §14.11) is a handle to a result that will exist later. You can `get()` it (blocking), poll `isDone()`, `cancel()` it, and since Java 19 read `state()` / `resultNow()` / `exceptionNow()` without blocking.

### The problem it doesn't solve

Real work is a *pipeline* (a chain of steps, each feeding the next): "fetch the user, **then** fetch their orders, **combined with** today's exchange rate, **falling back** to a **cached** value (a saved copy from earlier) on error, **giving up** after 2 seconds". With a plain `Future`:

1. The only way to use the result is **`get()`, which blocks** the calling thread — so "then do X" means parking a thread just to wait.
2. There's **no way to chain** a next step, combine two futures, or react to "whichever finishes first".
3. **Error handling** is a `try/catch (ExecutionException)` around each `get()`.
4. You can't **complete it yourself** (e.g. from a callback-based library — one that reports results by calling a function you gave it).

`CompletableFuture` (Java 8) fixes all four.

---

## 15.2 `CompletableFuture`

### What it is

**`CompletableFuture<T>`** implements both `Future<T>` and **`CompletionStage<T>`**, an interface of ~40 methods for saying *what should happen when this completes* — without blocking.

## How it works

1. **Start** async work: `supplyAsync(supplier[, executor])` (returns a value) or `runAsync(runnable[, executor])`. Or create an already-finished one with `completedFuture(v)` / `failedFuture(ex)` (9), or an empty one with `new CompletableFuture<>()` that you later `complete(v)` / `completeExceptionally(ex)` — a **promise** (a placeholder for a result that some code promises to fill in later).
2. **Attach stages**; each returns a *new* `CompletableFuture`, so calls chain:

| Method | Input → output | Stream analogy |
|---|---|---|
| `thenApply(fn)` | `T` → `U` | `map` |
| `thenAccept(consumer)` | `T` → nothing | `forEach` |
| `thenRun(runnable)` | nothing → nothing | |
| `thenCompose(fn)` | `T` → `CompletableFuture<U>`, flattened | `flatMap` |
| `thenCombine(other, bifn)` | `T` and `U` → `V` (both needed) | `zip` |
| `applyToEither(other, fn)` / `acceptEither` | whichever of two finishes first | |
| `allOf(cfs...)` | wait for all → `CompletableFuture<Void>` | |
| `anyOf(cfs...)` | first to finish → `CompletableFuture<Object>` | |

3. **Handle errors**: `exceptionally`, `handle`, `whenComplete`, `exceptionallyCompose` (12).
4. **Time out** (Java 9): `orTimeout(t, unit)` fails with `TimeoutException`; `completeOnTimeout(value, t, unit)` substitutes a default.
5. **Get the result at the edge** of your program: `join()` (unchecked `CompletionException` — *unchecked* means the compiler doesn't force you to catch it) or `get()` (checked `ExecutionException` — one you must catch or declare), or hand the `CompletableFuture` to a framework that accepts it (Spring MVC **controllers** — the classes in the Spring web framework that handle incoming HTTP requests — can return one).

**Which thread runs a stage?** A non-async stage (`thenApply`) runs on whichever thread completes the previous stage — or immediately on the *calling* thread if it's already complete. Every method has an **`…Async`** twin (`thenApplyAsync(fn[, executor])`) that submits the stage to an **executor** (an object that runs tasks on its threads, usually a thread pool — Chapter 14 §14.11). **Without an executor argument, async methods use `ForkJoinPool.commonPool()`** — shared by the whole JVM and sized for CPU work, so **always pass your own executor for blocking I/O** (or a virtual-thread executor, §15.4).

`code/ch15/cf/CompletableFutureBasics.java` (key parts; each `fetch…` sleeps to simulate a remote call):

```java
try (ExecutorService io = Executors.newFixedThreadPool(8)) {
    CompletableFuture<Integer> idF = CompletableFuture.supplyAsync(() -> fetchUserId("grace"), io);
    idF.thenApply(id -> "user-" + id).thenApply(String::toUpperCase).join();

    // thenApply with an async function nests; thenCompose flattens
    CompletableFuture<CompletableFuture<String>> nested = idF.thenApply(id -> supplyAsync(() -> fetchProfile(id),
io));
    CompletableFuture<String> flat = idF.thenCompose(id -> supplyAsync(() -> fetchProfile(id), io));

    price.thenCombine(taxRate, (p, t) -> p * (1 + t));

    List<CompletableFuture<Double>> quotes = List.of(
            supplyAsync(() -> fetchPrice("A", 300, 19.99), io),
            supplyAsync(() -> fetchPrice("B", 200, 17.49), io),
            supplyAsync(() -> fetchPrice("C", 250, 18.25), io));
    CompletableFuture<Void> all = CompletableFuture.allOf(quotes.toArray(CompletableFuture[]::new));
    double best = all.thenApply(v ->
quotes.stream().mapToDouble(CompletableFuture::join).min().orElseThrow()).join();

    CompletableFuture.anyOf(supplyAsync(() -> fetchPrice("slow", 300, 1.0), io),
                            supplyAsync(() -> fetchPrice("fast", 50, 2.0), io)).join();

    CompletableFuture<String> promise = new CompletableFuture<>();
    io.submit(() -> { sleep(50); promise.complete("completed by another thread"); });
}
```

Output:

```
1 submitted, done yet? false
2 thenApply chain -> USER-35
3 thenApply gives a nested future: CompletableFuture; thenCompose gives the value: profile#35
4 thenCombine -> total = 25.0
5 allOf best price = 17.49, took ~300 ms (sequential would be ~750)   [timing machine-dependent]
6 anyOf -> 2.0
7 thenAccept got cached
7 thenRun: nothing to receive, just runs after
8 manual complete -> completed by another thread
9 thenApplyAsync(fn, io) ran on our pool
```

- `allOf` returns `CompletableFuture<Void>` — it only signals "all done". Collect the values by calling `join()` on each original future inside the next stage; by then they're complete, so `join()` doesn't block.
- The three price lookups ran **in parallel**: total time ≈ the slowest (300 ms), not the sum (750 ms). This **fan-out/fan-in** (send several requests at once, then gather all the answers) is the most common real use (price comparison, aggregating **microservice** calls — a microservice being a small, separately deployed service that owns one business area).

### Error handling

When a stage throws, all later *transforming* stages are skipped and the exception travels down the chain until a handler deals with it — like an exception unwinding a **call stack** (the chain of method calls currently in progress).

| Method | Runs when | Can change the result? | Use for |
|---|---|---|---|
| `exceptionally(ex -> fallback)` | only on failure | yes — recovers | default value |
| `handle((val, ex) -> …)` | always | yes | map success and failure into one result |
| `whenComplete((val, ex) -> …)` | always | no — passes the original through | logging, metrics (counters and timings a service records about itself), cleanup |
| `exceptionallyCompose(ex -> otherCf)` (12) | only on failure | yes, asynchronously | retry / call a backup service |

`code/ch15/cf/CompletableFutureErrors.java` (key parts):

```java
CompletableFuture<Integer> failed = supplyAsync(() -> parse("x")).thenApply(n -> n * 2);  // thenApply skipped
failed.join();   // CompletionException(cause NumberFormatException)
failed.get();    // ExecutionException(cause NumberFormatException)

supplyAsync(() -> parse("x")).exceptionally(ex -> -1);
supplyAsync(() -> parse(in)).handle((val, ex) -> ex == null ? "ok " + val * 2 : "failed: " +
ex.getCause().getMessage());
supplyAsync(() -> parse("oops")).whenComplete((val, ex) -> log(...));

slow.orTimeout(100, MILLISECONDS).join();                         // TimeoutException
slow.completeOnTimeout("default", 100, MILLISECONDS).join();       // "default"
CompletableFuture.<String>failedFuture(new IllegalStateException("primary down"))
        .exceptionallyCompose(ex -> supplyAsync(() -> "from backup"));

CompletableFuture.runAsync(() -> { throw new IllegalStateException("nobody will ever see this"); });
```

Output:

```
1 join -> CompletionException, cause NumberFormatException
1 get  -> ExecutionException,  cause NumberFormatException
1 isCompletedExceptionally = true, state = FAILED
2 exceptionally sees CompletionException wrapping NumberFormatException
2 recovered value = -1
3 handle(21) -> ok 42
3 handle(oops) -> failed: For input string: "oops"
4 whenComplete logs: error
4 still failed afterwards? true
5 orTimeout -> TimeoutException
5 completeOnTimeout -> default
6 exceptionallyCompose -> from backup
7 fire-and-forget failure printed nothing (that's the danger)
```

- The exception a handler receives is often **wrapped in `CompletionException`** (line 2). Unwrap before checking its type: `Throwable cause = ex instanceof CompletionException ce ? ce.getCause() : ex;`.
- Line 7 is the biggest practical danger: a failed `CompletableFuture` that nobody joins or handles **fails silently**. Every chain should end in a `join`, a handler, or at least a `whenComplete` that logs.

## Trade-offs

| | `Future` + pool | `CompletableFuture` | **Virtual threads + plain blocking code** |
|---|---|---|---|
| Composition | none | rich | ordinary sequential code |
| Readability | simple | chains can get long; **stack traces** (the printed list of calls at an error) are fragmented | simplest |
| Threads blocked while waiting | yes | no (callbacks) | yes, but they're cheap |
| Best for | simple fire-and-get | fan-out/fan-in, async APIs, callbacks | most request/response I/O (Java 21+) |

## Common mistakes

- Blocking I/O in `supplyAsync` **without an executor** → the common pool (≈ one thread per core) is exhausted and every parallel stream in the JVM stalls.
- Calling `join()` right after `supplyAsync` everywhere → you've written slow synchronous code with extra steps.
- Using `thenApply` with a function returning a `CompletableFuture` → `CompletableFuture<CompletableFuture<T>>`; use `thenCompose`.
- Expecting `cancel(true)` to interrupt the running task: for `CompletableFuture` it just completes the future with `CancellationException`; the computation keeps running.
- Swallowing errors with `exceptionally(ex -> null)`.

## In the interview

**30-second answer:** "`CompletableFuture` is a composable, completable `Future`. I start work with `supplyAsync` on a dedicated executor, chain with `thenApply` / `thenCompose`, combine with `thenCombine` / `allOf`, recover with `exceptionally` / `handle`, and bound **latency** (how long a call takes, start to finish) with `orTimeout`. Non-async stages run on the completing thread; async variants default to the common pool, so for I/O I always pass an executor."

- *`thenApply` vs `thenCompose`?* → `map` vs `flatMap`: use `thenCompose` when the function itself returns a `CompletableFuture`.
- *`join` vs `get`?* → Same result; `join` throws unchecked `CompletionException`, `get` throws checked `ExecutionException` / `InterruptedException` and has a timeout overload.
- *How do you wait for a list of futures?* → `allOf(...).thenApply(v -> list.stream().map(CompletableFuture::join).toList())`.

### Self-check

1. What does `allOf` return? → `CompletableFuture<Void>`; read the individual futures for values.
2. Which handler cannot change the result? → `whenComplete`.
3. Which pool do `…Async` methods use without an executor? → `ForkJoinPool.commonPool()`.

## 15.3 Fork/Join

### What it is

The **Fork/Join framework** (Java 7) runs **divide-and-conquer** algorithms (split a problem into smaller copies of itself, solve those, combine the answers) in parallel: split a big problem into halves recursively until pieces are small, solve the pieces, and combine the answers. Its **`ForkJoinPool`** uses **work stealing**: each worker thread has its own double-ended queue (deque) of tasks; when a worker runs out, it **steals** tasks from the other end of a busy worker's deque, keeping all cores busy with little **contention** (threads competing for the same lock or data).

### How it works

1. Extend **`RecursiveTask<V>`** (returns a value) or **`RecursiveAction`** (no value) and implement `compute()`.
2. In `compute()`: if the problem is below a **threshold**, solve it directly (plain loop).
3. Otherwise split: `left.fork()` (push onto this worker's deque), compute the other half directly (`right.compute()`), then `left.join()` (wait for — or run — the forked half), and combine.
4. Start with `pool.invoke(task)`; `ForkJoinPool.commonPool()` has parallelism = cores − 1 by default.

`code/ch15/forkjoin/ForkJoinSum.java` (key parts):

```java
static class SumTask extends RecursiveTask<Long> {
    static final int THRESHOLD = 10_000;
    @Override protected Long compute() {
        if (to - from <= THRESHOLD) { long s = 0; for (int i = from; i < to; i++) s += data[i]; return s; }
        int mid = (from + to) >>> 1;
        SumTask left = new SumTask(data, from, mid), right = new SumTask(data, mid, to);
        left.fork();                   // others may steal it
        long r = right.compute();      // keep this thread busy
        long l = left.join();
        return l + r;
    }
}
long fj = ForkJoinPool.commonPool().invoke(new SumTask(data, 0, data.length));
long ps = Arrays.stream(data).parallel().sum();        // same framework underneath
```

Output (20 million longs):

```
common pool parallelism = 13   [machine-dependent: cores - 1]
round 1: correct=true  sequential 7 ms, fork/join 15 ms, parallel stream 28 ms   [timings machine-dependent]
round 2: correct=true  sequential 11 ms, fork/join 1 ms, parallel stream 0 ms   [timings machine-dependent]
round 3: correct=true  sequential 5 ms, fork/join 1 ms, parallel stream 0 ms   [timings machine-dependent]
tasks stolen between workers so far > 0: true
```

- Round 1 is *slower* in parallel: thread start-up and JIT compilation (the JVM's *just-in-time* compiler turning frequently used bytecode into machine code at run time) dominate. Parallelism pays off only once **warmed up** (after the JIT has optimised the hot code) and for enough data — measure with a proper **benchmark harness** (a tool that runs timing experiments correctly — warm-up, repetition, statistics; JMH, the Java Microbenchmark Harness, Chapter 16) before concluding anything.
- The order `fork` one half → `compute` the other → `join` matters: calling `left.fork(); right.fork(); left.join(); right.join();` wastes the current thread.

### When to use it / when not

- **Use** for **CPU-bound** (limited by processing, not by waiting), recursively splittable work on large in-memory data: sorting, searching, image processing, tree traversal. Most of the time, a **parallel**

**stream** (Chapter 12 §12.8) gives you the same engine with one method call.
- **Don't** do blocking I/O in Fork/Join tasks (or parallel streams): the common pool is shared by the whole JVM and has few threads. (`ForkJoinPool.ManagedBlocker` exists for special cases.) For I/O (input/output — disk, network) concurrency use virtual threads.
- **Don't** make the threshold too small (task overhead dominates) or too large (not enough tasks to balance). Thousands of elements per leaf is a typical starting point.

### Self-check

1. What is work stealing? → Idle workers take queued tasks from busy workers' deques.
2. `RecursiveTask` vs `RecursiveAction`? → Returns a value vs doesn't.
3. What else uses the common pool? → Parallel streams and `CompletableFuture` async methods without an executor.

---

## 15.4 Virtual threads

### What it is

A **virtual thread** (final in **Java 21, JEP 444** — a JEP, *JDK Enhancement Proposal*, is the numbered design document for a Java feature; previews in 19 and 20) is a `java.lang.Thread` that is **not tied to an operating-system thread**. The JVM schedules many virtual threads onto a small pool of OS threads called **carrier threads**. When a virtual thread performs a blocking operation (sleep, socket read, `BlockingQueue.take`, lock wait…), the JVM **unmounts** it — saves its stack to the **heap** (the JVM's memory area for objects) — and the carrier is free to run another virtual thread. When the operation completes, the virtual thread is **mounted** again on some carrier and continues.

| | **Platform thread** | **Virtual thread** |
|---|---|---|
| Backed by | one OS thread, 1:1 | carrier threads, M:N (many virtual threads share a few OS threads) |
| Creation cost | expensive (OS call, 1–2 MB stack reserved) | cheap (a small heap object; stack grows/shrinks on the heap) |
| How many | thousands | millions |
| Blocking | holds the OS thread | unmounts; carrier is released |
| Pool them? | yes | **no** — create one per task |
| Daemon / priority | configurable | always daemon (doesn't keep the JVM alive), normal priority |
| API | same `Thread` API | same `Thread` API (`isVirtual()`) |

### Why it matters

A server that handles each request on its own thread ("**thread-per-request**") is the easiest style to write and debug: plain sequential code, readable stack traces, `try/catch` works. But with platform threads, if each request spends 100 ms waiting on a database, a server with 200 threads can handle only ~2,000 requests per second, no matter how idle the CPU is. **Little's law** says concurrency = **throughput** (requests completed per second) × latency (both explained in System Design Fundamentals): 10,000 requests/s × 0.1 s = 1,000 requests in flight — more than a platform-thread pool comfortably holds. The previous fix was asynchronous/**reactive** code (a style where code is written as reactions to streams of events, never blocking — `CompletableFuture` chains, reactive libraries), which scales but is harder to write, read and debug. Virtual threads let simple blocking code scale like async code.

### How it works

1. Create one: `Thread.ofVirtual().name("vt-1").start(runnable)`, `Thread.startVirtualThread(runnable)`, or — most commonly — **`Executors.newVirtualThreadPerTaskExecutor()`**, which starts a new virtual thread for every submitted task.

2. The default **scheduler** (the component that decides which virtual thread runs on which carrier next) is a `ForkJoinPool` (FIFO — first in, first out — mode) whose parallelism equals the number of cores.
3. When the virtual thread blocks in a JDK operation that supports it (almost all of `java.io`, `java.net`, `java.util.concurrent`, `Thread.sleep`), its stack frames are copied to the heap and the carrier moves on.
4. When unblocked, it's rescheduled on any free carrier.

`code/ch15/virtual/VirtualThreadsDemo.java` (key parts):

```java
Thread v1 = Thread.ofVirtual().name("vt-1").start(() -> ...);
Thread v2 = Thread.startVirtualThread(() -> {});
Thread.currentThread().toString();       // inside a virtual thread: shows its carrier

static long time(ExecutorService executor) {
    try (executor) {                                   // close() waits for all tasks
        for (int i = 0; i < 10_000; i++) executor.submit(() -> { Thread.sleep(Duration.ofMillis(100)); return null;
});
    }
}
time(Executors.newFixedThreadPool(200));
time(Executors.newVirtualThreadPerTaskExecutor());

Semaphore dbPermits = new Semaphore(10);            // limit a resource, not the number of threads
```

Output:

```
1 hello from vt-1
1 isVirtual=true, daemon=true, main isVirtual=false
2 VirtualThread[#N]/runnable@ForkJoinPool-1-worker-K   [ids vary]
3 fixed pool of 200 platform threads: 5,202 ms  (theory: 10,000 / 200 x 100 ms = 5,000)   [machine-dependent]
3 one virtual thread per task:          126 ms  (theory: ~100 ms + overhead)             [machine-dependent]
4 carrier threads used: 14 <= cores (14)? true   [machine-dependent]
5 200 virtual threads, Semaphore(10): max concurrent 'DB calls' = 10
```

- **10,000 blocking tasks: ~5.2 s on a 200-thread pool vs ~0.13 s on virtual threads** — 40× faster here, because all 10,000 could wait at the same time. With a 10,000-thread platform pool you'd get similar speed at a much higher memory cost, and at 100,000 or 1,000,000 tasks platform threads run out while virtual threads keep going.
- Only **14 carrier threads** (the number of cores) ran all 10,000 virtual threads.
- Note that virtual threads don't make anything *faster* per task: each still took 100 ms. They raise **throughput** for tasks that **wait**.

### When they help

- Servers and clients with **many concurrent, mostly-waiting tasks**: HTTP handlers (code that answers web requests), calls to databases and other services, message consumers (code that processes items from a message queue). Spring Boot (3.2+; the most popular Java framework for building services) enables them for request handling with `spring.threads.virtual.enabled=true`; Tomcat, Jetty and Helidon (Java web servers / frameworks) support them.
- Replacing complex async code with straightforward blocking code.
- Concurrent mapping inside a stream: the `Gatherers.mapConcurrent(n, fn)` gatherer (Java 24) runs `fn` on virtual threads with at most `n` in flight (Chapter 12 §12.9).

### When they don't help (and pitfalls)

1. **CPU-bound work** — there are still only as many cores as there are. Use a platform pool sized to the cores, Fork/Join or parallel streams.
2. **Don't pool virtual threads.** A pool exists to limit an expensive resource; virtual threads aren't expensive. To limit access to a scarce resource (a database allowing 10 connections, a **rate-limited** API that accepts only so many calls per second), use a **`Semaphore`** (a counter of permits; Chapter 14 §14.13) or a bounded **connection pool** (a fixed set of reusable, already-open database connections), as in line 5.
3. **Heavy `ThreadLocal` caching**: code that caches expensive objects per thread (one parser per thread) was fine with 200 pooled threads, but with a million short-lived virtual threads you get a million

copies, each used once (§15.6).
4. **Pinning** on Java 21–23 when blocking inside `synchronized` (§15.5).
5. Your **downstream** systems (the services and databases your code calls) still have limits: 10,000 concurrent requests may overwhelm a database that could handle 200 at a time.

### Observing virtual threads

`jcmd <pid> Thread.dump_to_file -format=json dump.json` (`pid` = the OS process id of the JVM) writes a **thread dump** (a snapshot of every thread and what it's doing) that includes virtual threads (the older `jstack` output shows only platform threads). JDK Flight Recorder (JFR, the JVM's built-in low-overhead event recorder) has `jdk.VirtualThreadStart`/`End` (off by default), `jdk.VirtualThreadPinned` and `jdk.VirtualThreadSubmitFailed` events.

### In the interview

> **30-second answer:** "Virtual threads, final in Java 21, are JVM-managed threads **multiplexed** (many sharing a few) onto a few carrier threads. When one blocks on I/O it unmounts and frees the carrier, so millions of blocking tasks can be in flight and simple thread-per-request code scales like async code. They help I/O-bound, high-concurrency workloads, not CPU-bound ones; you don't pool them — you create one per task and limit scarce resources with semaphores."

- *Are virtual threads faster?* → Not per task; they give higher throughput for waiting tasks.
- *How many carrier threads?* → By default one per core.
- *What replaces a pool size limit?* → A `Semaphore` or a bounded resource pool (e.g. HikariCP, the most common Java database connection pool).
- *Does existing code need to change?* → Usually not: same `Thread` API; switch the executor.

### Self-check

1. What happens to a virtual thread's stack when it blocks? → It's copied to the heap; the carrier is released.
2. Why is `newFixedThreadPool(100)` of virtual threads a bad idea? → It defeats their purpose and still limits concurrency the wrong way; use one per task plus a semaphore.
3. Will a CPU-heavy image resize run faster on virtual threads? → No.

---

## 15.5 Pinning and JEP 491

### What it is

A virtual thread is **pinned** when it blocks but *cannot* unmount, so it holds its carrier thread hostage for the whole wait. If all carriers are pinned, no other virtual thread can run — throughput collapses back to "one task per carrier", and in the worst case the application deadlocks.

### What changed in Java 24

In Java 21–23, a virtual thread was pinned when it blocked **inside a** `synchronized` **block or method** (including `Object.wait()`), because the JVM's **monitor** (the built-in lock behind `synchronized`) implementation tracked lock ownership by carrier thread. The advice at the time was "replace `synchronized` with `ReentrantLock` around blocking operations". **JEP 491 (Java 24) re-implemented monitors so virtual threads can acquire, hold and release them independently of their carrier.** Blocking inside `synchronized` — and `Object.wait()` — now unmounts normally, so that advice no longer applies on Java 24+ (including Java 25 LTS — a *long-term support* release that receives updates for years).

`code/ch15/virtual/PinningDemo.java` uses only Java 21 APIs, so it was also compiled and run on JDK 21 for comparison:

```
try (ExecutorService ex = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 0; i < 1_000; i++) {
        Object lock = new Object();                 // one lock per task: no contention
        ex.submit(() -> { synchronized (lock) { Thread.sleep(50); } return null; });
    }
}
```

Output on JDK 26 (the runner):

```
Java 26: 1000 virtual threads sleeping 50 ms inside synchronized took 63 ms   [machine-dependent]
if every carrier were pinned: ~3600 ms with 14 carriers
```

Output of the same program on JDK 21 (Amazon Corretto 21.0.10 — Amazon's free build of the JDK — run by hand):

```
Java 21: 1000 virtual threads sleeping 50 ms inside synchronized took 3869 ms   [machine-dependent]
if every carrier were pinned: ~3600 ms with 14 carriers
```

On Java 21 every sleeping virtual thread pinned its carrier, so only 14 could sleep at a time: 1,000 / 14 × 50 ms ≈ 3.6 s. On Java 24+ all 1,000 slept at once.

### What still pins (Java 24+)

According to JEP 491, a virtual thread still pins its carrier when:

1. It calls **native code** (code compiled for the machine, typically C, reached via JNI — the Java Native Interface — or the Foreign Function & Memory API) and that native code calls back into Java code that blocks.
2. It blocks while **loading a class** (resolving a symbolic reference — turning a class/method *name* in bytecode into the real thing in memory) or **inside a class initializer** (`static { … }`, code that runs once when a class is first used), or waits for another thread to finish initialising a class.

These are rare in application code. The JFR event **`jdk.VirtualThreadPinned`** (on by default, reported when the pin lasts over 20 ms) now also records the *reason* and the carrier thread. The old diagnostic **system property** (a `-Dname=value` setting passed to the JVM) `jdk.tracePinnedThreads` was removed by JEP 491 and has no effect.

### Takeaway

On Java 21–23: avoid long blocking operations inside `synchronized` in virtual-thread code (use `ReentrantLock` there). On Java 24+: choose `synchronized` or `ReentrantLock` on their merits (Chapter 14 §14.9); watch JFR for the remaining native/class-initialisation cases.

---

## 15.6 `ThreadLocal` and scoped values

### The problem

Code deep in a call stack often needs *context* that was established near the top: the current user, a request id for logging, a **transaction** (a group of database operations that succeed or fail together). Passing it as a parameter through every method is noisy, especially through framework code you don't own. Java has two ways to pass it **implicitly**.

### `ThreadLocal` (Java 1.2)

A **`ThreadLocal<T>`** is a variable with a separate value **per thread**: `set(v)`, `get()`, `remove()`, `ThreadLocal.withInitial(supplier)`. Uses: per-request context in older frameworks (Spring Security's `SecurityContextHolder`, logging MDC — *Mapped Diagnostic Context*, key/value pairs automatically added to every log line), and per-thread caches of non-thread-safe objects (the classic being `SimpleDateFormat` — which `java.time.DateTimeFormatter`, being immutable, made unnecessary).

Problems:

1. **Unconstrained mutability** — any code can `set` it at any time; data flow is hard to follow.

2. **Unbounded lifetime** — the value lives until `remove()` or the thread dies. With **thread pools, a forgotten `remove()` leaks** the value into the next, unrelated task (a security bug if it's a user identity) and can **leak memory** (keep objects alive that nothing needs any more, so the garbage collector can't free them).
3. **Expensive inheritance** — `InheritableThreadLocal` copies values into every child thread; with millions of virtual threads that's costly.

### Scoped values (final in Java 25, JEP 506)

A **`ScopedValue<T>`** is an **immutable** value **bound for the duration of a method call**. You bind it with `ScopedValue.where(KEY, value).run(runnable)` (or `.call(callable)` for a result); any code called — directly or indirectly — from inside that call can `KEY.get()` it; when the call returns, the binding is gone automatically. There is no `set`. Code can **rebind** it for a nested call, which shadows the outer value only inside that call. History: **incubator** in Java 20 (JEP 429; an incubator module is an even earlier experimental stage than preview), previews in 21–24 (JEPs 446, 464, 481, 487), **final in Java 25 (JEP 506)** — so it's usable without flags in Java 25 LTS.

`code/ch15/scoped/ScopedValuesDemo.java` (key parts):

```java
static final ThreadLocal<String> REQUEST_ID_TL = new ThreadLocal<>();
static final ScopedValue<String> USER = ScopedValue.newInstance();

// ThreadLocal leak on a pooled thread
single.submit(() -> REQUEST_ID_TL.set("req-OLD")).get();                 // forgot remove()
single.submit(() -> System.out.println(... REQUEST_ID_TL.get())).get();  // sees req-OLD!

ScopedValue.where(USER, "alice").run(() -> handleRequest());            // handler -> service -> repository
static void repository() { System.out.println("3 repository sees USER = " + USER.get()); }

String result = ScopedValue.where(USER, "bob").call(() -> {
    String outer = USER.get();
    String inner = ScopedValue.where(USER, "admin").call(USER::get);    // nested rebinding
    return outer + " -> " + inner + " -> " + USER.get();
});
USER.get();                 // unbound -> NoSuchElementException
USER.orElse("anonymous");
```

Output:

```
1 ThreadLocal in A: requestId = req-A
1 ThreadLocal in B: requestId = req-B
1 main thread sees null (never set here)
2 next task on the same pooled thread sees: req-OLD
2 withInitial -> per-thread scratch buffer
3 bound before? false
3 repository sees USER = alice
3 bound after?  false
4 nested rebinding: bob -> admin -> bob
5 get() when unbound -> NoSuchElementException; orElse -> anonymous
6 plain new thread sees bound? false
```

- Line 2 is the pooled-thread leak in action: the second task inherited the first task's request id.
- Line 3: `repository()` read `alice` three calls deep without any parameter; outside the `run`, nothing is bound.
- Line 6: bindings are **not** inherited by threads you start yourself. They *are* inherited by subtasks forked in a `StructuredTaskScope` (§15.7), which is how context flows into concurrent subtasks safely.

### Trade-offs

| | `ThreadLocal` | `ScopedValue` (25) |
|---|---|---|
| Mutability | `set` anywhere | immutable; rebind only for a nested call |
| Lifetime | until `remove()` / thread death | exactly the `run` / `call` |
| Leak risk with pools | yes | none |
| Inheritance | `InheritableThreadLocal` copies | shared (no copy) with structured-concurrency subtasks |
| Good for | per-thread caches of mutable objects; legacy frameworks | request context: user, tenant (the customer account in a multi-customer system), trace id (an id following one request across services), transaction |

### Common mistakes

- Not calling `remove()` in a `finally` when using `ThreadLocal` with pools.
- Treating a scoped value like a mutable holder — to "change" it, rebind for a nested call.
- Declaring `ScopedValue` keys as instance fields; they're meant to be `static final` constants (often private, exposed through accessors to control who can read or bind them).

### In the interview

> **30-second answer:** "`ThreadLocal` gives each thread its own mutable copy; it's easy to leak across pooled tasks and costly with millions of virtual threads. Scoped values, final in Java 25, bind an immutable value for the duration of a call; callees read it, it disappears when the call returns, and structured-concurrency subtasks share it. I use scoped values for request context and keep `ThreadLocal` for legacy integration and per-thread caches."

#### 15.6.1 `ThreadLocalRandom`

A shared `java.util.Random` is thread-safe, but every `nextInt` CASes its single 48-bit seed, so many threads fight over one variable (its Javadoc warns of "contention and consequent poor performance"). **`ThreadLocalRandom`** (Java 7) keeps the seed **in each `Thread` object**:

- Always call it as **`ThreadLocalRandom.current().nextInt(...)`**, at the point of use. Don't store the result of `current()` in a field or pass it to other threads; the Javadoc recommends this usage form so that an instance can never be shared by accident.
- It offers range helpers (`nextInt(origin, bound)`, `nextLong(bound)`, `nextDouble(a, b)`) and, since Java 17, implements `RandomGenerator`.
- It **cannot be seeded** (`setSeed` throws `UnsupportedOperationException`), so tests that need repeatable numbers inject a seeded `Random` or `SplittableRandom` instead. It is not cryptographically secure: for tokens and keys use `SecureRandom`.
- It works the same on virtual threads (each has its own seed).

`code/ch15/random/RandomContention.java`: one thread per core, 5 million `nextInt(100)` calls each:

```java
long shared = time(threads, () -> SHARED.nextInt(100));                       // static final Random
long local  = time(threads, () -> ThreadLocalRandom.current().nextInt(100));
```

```
4 threads x 5,000,000 nextInt(100)   [machine-dependent]
shared Random      : 1983 ms
ThreadLocalRandom  : 53 ms  (37x faster)
range helper       : nextInt(10, 20) in [10,20): true
setSeed            : UnsupportedOperationException (not reproducible by design)
```

The gap (17× to 60× across runs) is pure contention: both generators do the same arithmetic. Sources: `ThreadLocalRandom`, `Random`, `RandomGenerator` Javadoc (JEP 356).

## 15.7 Structured concurrency (preview)

### Status (verified against the JEPs)

Structured concurrency is **still a preview API** — not final — in Java 25 and 26:

| Release | JEP | Status |
|---|---|---|
| 19, 20 | 428, 437 | incubator |
| 21, 22, 23, 24 | 453, 462, 480, 499 | preview 1–4 |
| **25 (LTS)** | **505** | fifth preview — API redesigned around `StructuredTaskScope.open()` and `Joiner` |
| **26** | **525** | sixth preview — small changes (e.g. `Joiner.anySuccessfulResultOrThrow()` renamed `anySuccessfulOrThrow()`, `allSuccessfulOrThrow()` now returns a list of results, new `Joiner.onTimeout()`) |
| 27 (GA — general availability, the official release — 15 Sept 2026) | 533 | seventh preview — built-in joiners make `join()` throw `ExecutionException`, a third type parameter for the exception type, `awaitAll()` removed, timeouts reworked |

Because it's a preview, you must compile and run with `--enable-preview` (and the exact API can still change between releases), so the runnable code in this book doesn't depend on it. Details below use the part of the API that is the same in Java 25 and 26.

### What it is

**Structured concurrency** treats a group of related concurrent tasks as **one unit of work** with a clear lifetime, the way a code block structures sequential code: subtasks are started inside a scope, and the scope **cannot be exited until they've all finished**. Consequences:

1. **No leaked threads** — when the method returns, no subtask is still running in the background.
2. **Error propagation and cancellation** — if one subtask fails, the others are **cancelled** (interrupted) automatically and the failure surfaces from `join()`.
3. **Observability** — thread dumps show subtasks as children of the task that forked them.
4. **Scoped values** bound in the parent are visible in the forked subtasks.

Compare with `ExecutorService` + `Future`s: if you submit two lookups and the first fails, nothing stops the second; if the caller is interrupted, the subtasks keep running; the relationship between them exists only in your head.

### How it works (Java 25/26 preview API)

1. Open a scope in try-with-resources: `StructuredTaskScope.open()` (default policy: all subtasks must succeed) or `open(Joiner)` for another policy.
2. `fork(callable)` each subtask → a `Subtask<T>`; each runs in its own **virtual thread**.
3. `join()` waits for all subtasks (or until the policy decides — e.g. first failure).
4. Read results with `subtask.get()`; `subtask.state()` is `SUCCESS`, `FAILED` or `UNAVAILABLE`.
5. Closing the scope guarantees every subtask has finished.

```java
// (preview API - run with: java --enable-preview --source 26 StructuredDemo.java ; same output on JDK 25 with --source 25)
static final ScopedValue<String> USER = ScopedValue.newInstance();
record Page(String profile, int orders) {}

static String fetchProfile() throws InterruptedException { Thread.sleep(100); return "profile of " + USER.get(); }
static int fetchOrders() throws InterruptedException { Thread.sleep(150); return 3; }
static int fetchOrdersBroken() throws InterruptedException { Thread.sleep(50); throw new IllegalStateException("orders service down"); }
static String slowProfile() throws InterruptedException {
    try { Thread.sleep(5_000); return "too late"; }
    catch (InterruptedException e) { System.out.println("  slowProfile: cancelled (interrupted)"); throw e; }
}

static Page load() throws InterruptedException {
    try (var scope = StructuredTaskScope.open()) {                  // all must succeed
        Subtask<String> profile = scope.fork(StructuredDemo::fetchProfile);
        Subtask<Integer> orders = scope.fork(StructuredDemo::fetchOrders);
        scope.join();                                               // wait for both, or the first failure
        return new Page(profile.get(), orders.get());
    }
}
static Page loadWithFailure() throws InterruptedException {
    try (var scope = StructuredTaskScope.open()) {
        Subtask<String> profile = scope.fork(StructuredDemo::slowProfile);
        Subtask<Integer> orders = scope.fork(StructuredDemo::fetchOrdersBroken);
        scope.join();                                               // throws FailedException
        return new Page(profile.get(), orders.get());
    }
}
// main:
Page p = ScopedValue.where(USER, "alice").call(StructuredDemo::load);
try { loadWithFailure(); }
catch (StructuredTaskScope.FailedException e) { ... e.getCause().getMessage() ... }
```

Output (JDK 26 and JDK 25, both with `--enable-preview`):

```
page = Page[profile=profile of alice, orders=3] in ~150 ms (max of 100 and 150, not the sum)
  slowProfile: cancelled (interrupted)
join -> FailedException, cause: orders service down; returned after ~50 ms, not 5000
```

- The forked `fetchProfile` read `USER` — scoped values flow into subtasks.
- When `fetchOrdersBroken` failed after 50 ms, the scope **interrupted** `slowProfile` immediately instead of letting it run for 5 s, and `join()` threw `FailedException` carrying the original cause.

Other built-in policies (`Joiner` factory methods): all must succeed (the default policy of `open()`; `allSuccessfulOrThrow` also collects the results), first success wins and cancels the rest (`anySuccessfulOrThrow` in 26 — "hedged" requests: sending the same request to several **replicas**, i.e. identical copies of a service, and taking the fastest answer), wait for all regardless of outcome (`awaitAll`), and a predicate-based `allUntil`. Timeouts are set through the configuration function passed to `open`. Check the Javadoc of the JDK you run, since names changed between previews (Java 27's seventh preview removes `awaitAll` and has the built-in joiners make `join()` throw `ExecutionException`).

### When to use it

Once final, for any "call several things concurrently, then combine" code in a request handler — the fan-out that §15.2 did with `CompletableFuture.allOf`, but with automatic cancellation and ordinary blocking code. Until then: use it for experiments, or use `CompletableFuture`/executors in production and cancel manually.

## In the interview

**30-second answer:** "Structured concurrency groups subtasks into a scope whose lifetime is a code block: fork subtasks in virtual threads, `join`, and on failure the siblings are cancelled and the error propagates; no thread outlives the scope and scoped values flow into subtasks. It's still a preview API — fifth preview in Java 25, sixth in 26, seventh in 27 — while scoped values became final in 25."

---

## 15.8 The Java Memory Model and happens-before

### What it is

The **Java Memory Model (JMM)** — JLS chapter 17 (the JLS is the *Java Language Specification*, the official rulebook), redesigned in Java 5 (JSR-133, a *Java Specification Request*, the older name for a formal spec change) — is the contract that says **which values a read of a shared variable is allowed to return** when several threads run. It exists because compilers, JIT and CPUs all **reorder** (run independent instructions in a different order) and **cache** (keep private, faster copies of) memory operations to go faster. Within a single thread these optimisations are invisible (the thread always sees its own actions in program order), but *another* thread can observe writes late or in a surprising order.

The JMM's central idea is the **happens-before** relation. If action A *happens-before* action B, then A's effects (and everything before A in its thread) are **guaranteed visible** to B. If two conflicting accesses (same variable, at least one write, different threads) are **not** ordered by happens-before, the program has a **data race**, and the read can see **stale** (out-of-date) or reordered values.

### The happens-before rules

| Rule | A happens-before B when… |
|---|---|
| Program order | A comes before B in the same thread |
| Monitor lock | A is an unlock of a monitor and B is a *later* lock of the **same** monitor ( `synchronized` ) |
| Volatile | A is a write to a volatile field and B a *later* read of **that** field |
| Thread start | A is `t.start()` and B is any action in `t` |
| Thread termination | A is any action in `t` and B is another thread returning from `t.join()` (or seeing `isAlive()` false) |
| Interruption | A is `t.interrupt()` and B is `t` detecting the interrupt |
| Transitivity | A hb B and B hb C ⇒ A hb C |
| `java.util.concurrent` | e.g. putting an object into a concurrent collection hb its retrieval; submitting a task hb its execution; task actions hb `Future.get()` returning; `release` / `countDown` / `unlock` hb a successful `acquire` / `await` / `lock` |

Plus **final-field semantics** (JLS 17.5): if a constructor doesn't leak `this`, every thread that obtains a reference to the object sees its `final` fields correctly initialised, even without synchronisation.

### Seeing reordering for real

`code/ch15/jmm/ReorderingDemo.java` runs the classic "store buffering" test millions of times. For each slot `i`, two threads run concurrently:

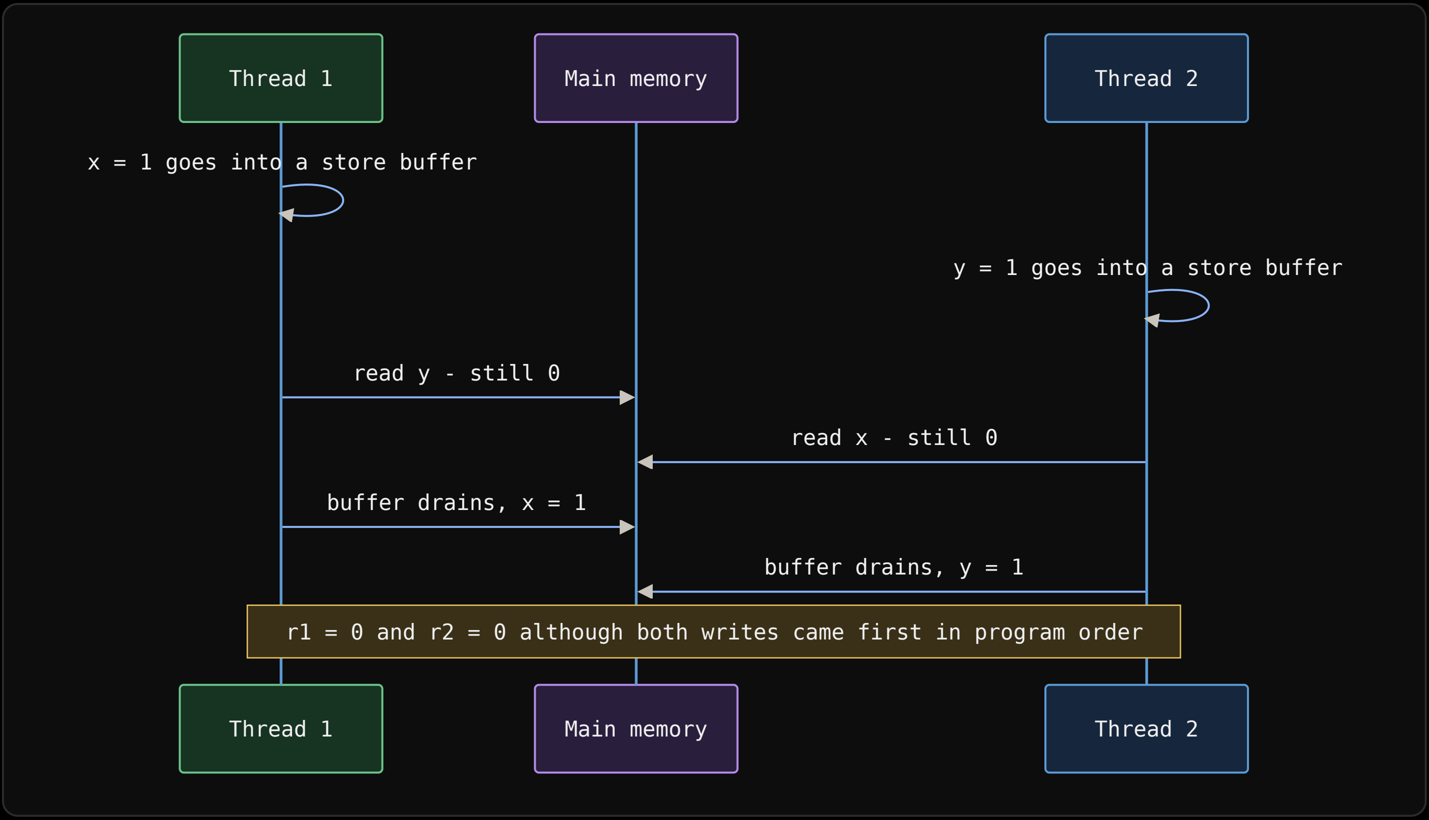
```
// thread 1                          // thread 2
x[i] = 1;   r1[i] = y[i];            y[i] = 1;   r2[i] = x[i];
```

In **any** interleaving of these four statements in program order, at least one thread reads a 1 — so `(r1, r2) = (0, 0)` is impossible without reordering. Then the same with `AtomicIntegerArray` `set`/`get` (volatile semantics):

Output:

```
plain arrays:         (0,0) seen 1,041 times in 50,000,000 trials   [nondeterministic - varies run to run; any
value > 0 proves reordering]
AtomicIntegerArray:   (0,0) seen 0 times in 50,000,000 trials   [always 0 - forbidden by the JMM]
```

Each CPU core put its write into a *store buffer* (a small per-core queue of pending writes) and read the other variable before its own write became visible to the other core — both reads saw 0. With volatile semantics the JMM forbids that outcome, and it never occurred.


Thread 1
Main memory
Thread 2
x = 1 goes into a store buffer
y = 1 goes into a store buffer
read y - still 0
read x - still 0
buffer drains, x = 1
buffer drains, y = 1
r1 = 0 and r2 = 0 although both writes came first in program order
Thread 1
Main memory
Thread 2

### Safe publication

**Publishing** an object means making it reachable by other threads. Publishing *unsafely* (assigning to a plain shared field with no happens-before) can let another thread see the reference **before** the object's fields are initialised. Safe ways (JCIP):

1. Initialise it in a **static initializer** (class initialisation is thread-safe).
2. Store the reference in a `volatile` field or an `AtomicReference`.
3. Store it in a `final` field of a properly constructed object.
4. Store it in a field **guarded by a lock**, or put it in a **concurrent collection**.
5. Make the object **immutable** (all fields `final`, no `this` escape) — then even racy publication shows it correctly.

`code/ch15/jmm/SafePublication.java` (key parts):

```java
// Double-checked locking: correct ONLY because 'instance' is volatile (Java 5+)
private static volatile Registry instance;
static Registry get() {
    Registry r = instance;
    if (r == null) {
        synchronized (Registry.class) {
            r = instance;
            if (r == null) instance = r = new Registry();
        }
    }
    return r;
}
// Initialization-on-demand holder: lazy and thread-safe with no locking code
private static class Holder { static final Heavy INSTANCE = new Heavy(); }
static Heavy get() { return Holder.INSTANCE; }

static volatile Config current;                               // publish through a volatile write
```

Output:

```
64 threads raced: Registry constructed 1 time(s), Heavy constructed 1 time(s)
reader sees fully built config: Config[url=jdbc:postgresql://db/shop, poolSize=10]
```

Without `volatile`, double-checked locking is **broken**: the write of the reference can become visible before the writes inside the constructor, so a second thread skips the lock and uses a half-built object. The holder idiom is usually simpler — the JVM initialises `Holder` exactly once, on first use, under its internal class-initialisation lock. For **singletons** (classes meant to have exactly one instance), an `enum` with one constant is simplest of all (*Effective Java* Item 3; see LLD Chapter 8).

### Common mistakes

- Believing "it worked in testing" proves correctness — reorderings depend on CPU architecture (ARM, as on this laptop, reorders more than x86 — the Intel/AMD chip family), JIT decisions and load.
- Synchronising writes but reading without the lock (no happens-before for the reader).
- Using `volatile` for a reference to a *mutable* object and assuming later changes to its fields are visible — only the reference write is covered.

### In the interview

> **30-second answer:** "The JMM defines when a write by one thread is guaranteed visible to another, through happens-before: program order, unlock→lock on the same monitor, volatile write→read, `start`, `join`, and the `java.util.concurrent` hand-offs, all transitive. Without a happens-before edge you have a data race and can see stale or reordered values. That's why double-checked locking needs `volatile`, and why immutable objects with final fields are always safe to share."

- *Does `volatile` make `i++` atomic?* → No; it gives visibility and ordering only.
- *What does `final` guarantee?* → Correct visibility of final fields after construction, if `this` doesn't escape.

More: LLD §12.6; JCIP chapter 16; JLS §17.4.

## 15.9 Liveness: deadlock, livelock, starvation

**Safety** means "nothing bad happens" (no races). **Liveness** means "something good eventually happens" — the program makes progress. Liveness failures:

| Failure | What happens | Threads are… | Analogy |
|---|---|---|---|
| **Deadlock** | two or more threads each wait for a lock another holds, forever | `BLOCKED`/`WAITING`, 0% CPU | two cars at a one-lane bridge, each waiting for the other to reverse |
| **Livelock** | threads keep reacting to each other and retrying, but none progresses | running, busy CPU | two people in a corridor stepping aside in the same direction again and again |
| **Starvation** | one thread never gets the resource because others keep taking it | runnable but never lucky | the polite customer never served at a crowded bar |

### Deadlock

Four conditions (the **Coffman conditions**) must all hold: **mutual exclusion**, **hold and wait** (holding one lock while waiting for another), **no preemption** (locks can't be taken away), and **circular wait** (A waits for B, B waits for A, or a longer loop). Break any one and deadlock is impossible — in practice, you break *circular wait* or *hold-and-wait*.

`code/ch15/liveness/DeadlockDemo.java` (key parts):

```
static void transferBad(Account from, Account to, int amt, CountDownLatch bothHoldFirstLock) {
    synchronized (from) {
        bothHoldFirstLock.countDown(); bothHoldFirstLock.await();   // make the deadlock certain
        synchronized (to) { ... }
    }
}
// thread 1: transferBad(a, b, ...)     thread 2: transferBad(b, a, ...)
long[] ids = ManagementFactory.getThreadMXBean().findDeadlockedThreads();

// Fix 1 - global lock order: always lock the account with the lower id first
Account first = from.id < to.id ? from : to, second = first == from ? to : from;
synchronized (first) { synchronized (second) { ... } }

// Fix 2 - tryLock with timeout, release everything, random back-off, retry
if (from.lock.tryLock(10, MILLISECONDS)) { try { if (to.lock.tryLock(10, MILLISECONDS)) { ... } } finally {
from.lock.unlock(); } }
Thread.sleep(ThreadLocalRandom.current().nextInt(1, 5));
```

Output:

```
deadlock detected between 2 threads:
  transfer-a-to-b is BLOCKED, waiting for a lock held by transfer-b-to-a
  transfer-b-to-a is BLOCKED, waiting for a lock held by transfer-a-to-b
t1 alive=true, t2 alive=true (stuck forever; only a restart helps)
lock ordering: finished, total money = 2000
tryLock + back-off: finished, total money = 2000
```

- `ThreadMXBean.findDeadlockedThreads()` (`ThreadMXBean` is the JVM's built-in monitoring interface for threads; this is also what `jstack` and `jcmd Thread.print` report as "Found one Java-level deadlock") detects cycles among `synchronized` monitors and `java.util.concurrent` locks. Java can't *recover* from a deadlock — the threads can't be forced to release their locks — so prevention is the only real strategy. (The demo marks the stuck threads as daemons so the program can still exit.)
- Both fixes completed 2,000 opposite-direction transfers with money conserved.

**Prevention checklist:**

1. **Lock ordering** — acquire multiple locks in one global order (by id, by `System.identityHashCode` with a tie-breaker lock).
2. **Timeouts** — `tryLock(timeout)`, then release and retry with random back-off.
3. **Fewer locks, shorter holds** — don't call unknown code (callbacks, listeners, overridable methods) while holding a lock ("open calls" — JCIP 10.1.4).
4. **Higher-level tools** — a single-writer design, `ConcurrentHashMap` atomic methods, message passing via queues.
5. **Thread-starvation deadlock** in pools: tasks waiting for other tasks in the same bounded pool (Chapter 14 §14.11) — use separate pools, `CompletableFuture` composition, or virtual threads.

### Livelock

Threads aren't blocked; they're busy responding to each other. Typical cause: every thread detects a conflict and backs off **in exactly the same way at the same time** — e.g. the `tryLock` version above *without* the random sleep can end up with both threads repeatedly grabbing their first lock, failing on the second, releasing, and trying again in lock-step. The fix is **randomised back-off** (wait a random, growing delay before retrying — as in Ethernet collision handling, where network cards that collide each wait a random time) or an ordering rule that decides who goes first. Also found in message processing: a message that always fails is put back at the head of the queue and retried forever (use a **dead-letter queue** — a side queue where messages that keep failing are parked for a human to inspect).

### Starvation

A thread is ready to run but never gets the lock or CPU time:

- **Unfair locks**: `synchronized` and default `ReentrantLock` allow **barging** — a newly arriving thread can grab a just-released lock ahead of long waiters. That improves throughput but can starve a thread under heavy contention. `new ReentrantLock(true)` / `new Semaphore(n, true)` grant in (roughly) arrival order, at a throughput cost.

- **Readers starving a writer** with read-write locks under constant read traffic (the fair mode of `ReentrantReadWriteLock` helps).
- **Thread priorities** — don't use them to fix starvation; they're only hints (Chapter 14 §14.2).
- **Long-running tasks** hogging a small pool while short tasks queue behind them — separate pools (bulkheads).

Detailed walk-throughs and the dining philosophers: LLD §12.17 Deadlock, §12.18 Livelock, §12.19 Starvation, LLD §13.5.

### In the interview

> **30-second answer:** "Deadlock needs mutual exclusion, hold-and-wait, no preemption and a circular wait; I prevent it with a global lock order or `tryLock` with timeouts and back-off, and by not calling foreign code while holding locks. I diagnose it with a thread dump, which reports Java-level deadlocks. Livelock is threads busily retrying in lock-step — fixed with randomised back-off. Starvation is a thread never getting a resource — fairness settings, separate pools, or less contention fix it."

- *How do you find a deadlock in production?* → `jstack <pid>` / `jcmd <pid> Thread.print` (or `ThreadMXBean` in a **health check** — an endpoint or job that regularly reports whether the service is working).
- *Deadlock vs livelock CPU usage?* → ~0% vs high.

### Self-check

1. Which Coffman condition does lock ordering break? → Circular wait.
2. Why random back-off rather than fixed? → Fixed delays keep contenders in lock-step → livelock.
3. What does a fair lock trade away? → Throughput.

---

## 15.10 Best practices and choosing a concurrency model

**Best practices (JCIP, *Effective Java* Items 78–84, JDK guidance)**

1. **Minimise shared mutable state.** Prefer immutable objects (records), confinement and message passing.
2. **Synchronise all access to shared mutable data** — reads and writes — with the same lock (Item 78).
3. **Keep synchronised regions small; never call alien code** (code you don't control, e.g. callbacks or overridable methods) **inside them** (Item 79).
4. **Prefer executors, tasks and streams to raw threads** (Item 80); prefer **concurrency utilities to `wait`/`notify`** (Item 81).
5. **Document thread safety** of every class you publish (Item 82).
6. **Lazy initialisation**: only when needed; use the holder idiom or volatile double-checked locking (Item 83).
7. **Don't depend on the thread scheduler** — no `Thread.yield`, priorities or busy-waits for correctness (Item 84).
8. **Handle interruption** properly — rethrow or restore the flag.
9. **Always bound things**: queue sizes, pool sizes, timeouts on `get`/`join`/`tryLock`/remote calls.
10. **Never lose exceptions** in async code: join, handle or log every `Future`/`CompletableFuture`.
11. **Use virtual threads for I/O-bound concurrency on Java 21+**; don't pool them; limit scarce resources with semaphores; avoid `ThreadLocal` caches.
12. **Prefer scoped values over `ThreadLocal`** for request context on Java 25+.
13. **Test and observe**: stress tests, thread dumps, JFR; but rely on happens-before reasoning, not on tests passing.

### Choosing a model

| Workload | Recommended tool (Java 25) |
|---|---|
| Many concurrent blocking calls (HTTP, DB, files) | virtual threads — `newVirtualThreadPerTaskExecutor()`, thread-per-request |
| Fan out to N services, combine, with cancellation | `CompletableFuture.allOf` today; `StructuredTaskScope` once final |
| CPU-heavy work on large in-memory data | parallel streams / Fork/Join / fixed pool of ≈ #cores platform threads |
| Periodic or delayed jobs | `ScheduledExecutorService` |
| Producer-consumer pipeline | `BlockingQueue` + executor (virtual threads are fine as consumers) |
| Shared counters, caches, registries | atomics, `LongAdder`, `ConcurrentHashMap` |
| Callback-based library integration | `CompletableFuture` as a promise (`complete`) |
| Very high-throughput event streaming with back-pressure (consumers signalling producers to slow down) | reactive libraries (Project Reactor, RxJava) or messaging systems — beyond this book |

## Real-world examples

- **Web servers**: Tomcat/Jetty with virtual threads let each request run plain blocking JDBC (Java Database Connectivity, the standard low-level database API) code at high concurrency.
- **API gateways / BFFs** (a gateway is the single entry point in front of many services; a BFF, *Backend For Frontend*, is a gateway tailored to one app — see System Design Fundamentals): fan out to several microservices per request (`CompletableFuture.allOf` with per-call `orTimeout` and fallbacks).
- **Batch analytics**: parallel streams over large in-memory collections; Fork/Join merge sorts.
- **Caches**: Caffeine (a popular in-memory caching library) builds on `ConcurrentHashMap` and `CompletableFuture` (async loading).
- **Observability** (being able to see what a running system is doing through logs, metrics and traces): request ids carried in logging context (MDC, `ThreadLocal`-based today, scoped values in newer designs).

### 15.10.1 Concurrency exceptions quick reference

| Exception | Package · checked? | Thrown when | What to do |
|---|---|---|---|
| `InterruptedException` | `java.lang` · checked | a thread is interrupted while in (or on entering) `sleep`, `wait`, `join`, `BlockingQueue.take/put`, `lockInterruptibly`, `Condition.await`, `Future.get`…; the interrupt flag is **cleared** | rethrow, or restore the flag with `Thread.currentThread().interrupt()` (§14.4) |
| `ExecutionException` | `j.u.c` · checked | `Future.get()` (and `CompletableFuture.get()`) when the task threw; the real error is `getCause()` | unwrap the cause and decide: retry, fall back or fail |
| `CompletionException` | `j.u.c` · unchecked | `CompletableFuture.join()`, and the exception passed to dependent stages, when a stage failed; wraps the cause | unwrap in `exceptionally`/`handle` (§15.2) |
| `TimeoutException` | `j.u.c` · checked | `Future.get(t, unit)`, `CyclicBarrier.await(t, unit)`, `Exchanger.exchange(x, t, unit)` ran out of time. A `get` timeout does **not** cancel the task | `cancel(true)` the task if its result is no longer wanted |
| `CancellationException` | `j.u.c` · unchecked (a subclass of `IllegalStateException`) | `get()`/`join()` on a future that was `cancel`led | usually expected after your own `cancel`; check `isCancelled()` |
| `BrokenBarrierException` | `j.u.c` · checked | waiting at a `CyclicBarrier` when another party was interrupted or timed out, the barrier was `reset()`, or the barrier action threw | treat the round as failed; `reset()` or build a new barrier |
| `RejectedExecutionException` | `j.u.c` · unchecked | `execute`/`submit` after `shutdown()`, or when threads and queue are full under `AbortPolicy` | back-pressure: retry later, `CallerRunsPolicy`, or shed load (§14.11) |
| `IllegalMonitorStateException` | `java.lang` · unchecked | `wait`/`notify` without holding that object's monitor; `unlock()` by a non-owner; `Condition.await/signal` without the lock | a bug: fix the locking |
| `IllegalThreadStateException` | `java.lang` · unchecked | `start()` on a thread that was already started | create a new `Thread` (or use an executor) |

Timed waits that don't throw report the timeout through their return value: `tryLock(t, u)` and `Semaphore.tryAcquire(t, u)` → `false`, `CountDownLatch.await(t, u)` and `awaitTermination` → `false`, `BlockingQueue.poll(t, u)` → `null`. `ConcurrentModificationException`, despite its name, is mostly a *single-thread* error (changing a collection while iterating it, §14.12). `code/ch15/exceptions/ExceptionsTour.java` triggers each one:

```
show("Future.get(), task threw", () -> pool.submit(() -> { throw new IllegalArgumentException("bad"); }).get());
show("unlock() by a non-owner", () -> new ReentrantLock().unlock());
show("submit() after shutdown()", () -> pool.submit(() -> 1));
```

```
sleep() on an interrupted thread           -> InterruptedException
Future.get(), task threw                   -> ExecutionException (cause: IllegalArgumentException)
Future.get(50 ms), task too slow           -> TimeoutException
Future.get(), task was cancelled           -> CancellationException
CompletableFuture.join(), stage threw      -> CompletionException (cause: IllegalArgumentException)
CyclicBarrier, another party timed out     -> BrokenBarrierException
Object.wait() without the monitor          -> IllegalMonitorStateException
unlock() by a non-owner                    -> IllegalMonitorStateException
Condition.await() without the lock         -> IllegalMonitorStateException
Thread.start() twice                       -> IllegalThreadStateException
submit() after shutdown()                  -> RejectedExecutionException
submit() to a saturated pool (AbortPolicy) -> RejectedExecutionException
interrupt flag after InterruptedException was thrown: false
```

The last line shows why §14.4's "restore the flag" rule exists: throwing `InterruptedException` cleared the flag. Sources: Javadoc of each exception class and of `Future`, `CyclicBarrier`, `ThreadPoolExecutor`, `Object.wait`, `ReentrantLock.unlock`.

## Chapter TL;DR

| Topic | Remember |
|---|---|
| `Future` | Blocking `get()`, no composition; `state()`/`resultNow()` since 19 |
| `CompletableFuture` | `supplyAsync(…, executor)` → `thenApply`/`thenCompose`/`thenCombine`/`allOf`/`anyOf`; `exceptionally`/`handle`/`whenComplete`; `orTimeout`/`completeOnTimeout` (9); pass your own executor for I/O; never leave failures unobserved |
| Fork/Join | Recursive split + work stealing; `RecursiveTask`; fork one half, compute the other, join; powers parallel streams; CPU-bound only |
| Virtual threads (21) | Cheap JVM-managed threads on ~#cores carriers; unmount on blocking; huge throughput for waiting tasks; don't pool; limit with `Semaphore`; no help for CPU work |
| Pinning | Java 21–23: blocking in `synchronized` pinned carriers; Java 24 (JEP 491) fixed it; native callbacks and class initialisation still pin; `jdk.VirtualThreadPinned` JFR event |
| Context | `ThreadLocal`: per-thread, mutable, leaks with pools; `ScopedValue` (final 25): immutable, bounded to a call, inherited by structured subtasks; `ThreadLocalRandom.current()` instead of a shared `Random` |
| Structured concurrency | Preview in 25 (JEP 505), 26 (JEP 525), 27 (JEP 533): scope = unit of work, auto-cancel on failure, no leaked threads |
| JMM | Happens-before via program order, monitor unlock→lock, volatile write→read, start/join, j.u.c hand-offs; data race = stale/reordered reads; safe publication; DCL needs `volatile`; holder idiom |
| Liveness | Deadlock (Coffman; lock ordering, `tryLock`+timeouts; detect with thread dumps), livelock (random back-off), starvation (fairness, bulkheads) |
| Choosing | I/O-bound → virtual threads; CPU-bound → parallel streams/Fork/Join; fan-out → `CompletableFuture` (structured concurrency later) |
| Exceptions | `ExecutionException`/`CompletionException` wrap the cause; `get` timeout doesn't cancel; `CancellationException` after `cancel`; broken barrier, rejected task, monitor not held (§15.10.1) |

**Where next:** what the JVM does underneath — memory areas, garbage collectors, JIT, profiling with JFR and benchmarking with JMH — in Chapter 16 — JVM internals and performance; the version-by-version feature tour in Chapter 17 — Modern Java 17 to 25. Practise with LLD Chapter 13 — Concurrency problems; exam traps in OCP Chapter 13; compare with Python's `asyncio` and GIL in Python Chapter 16.

# Chapter 16 — JVM Internals and Performance

Chapter 1 gave the one-page picture: `javac` makes bytecode; the JVM loads it lazily, interprets it, JIT-compiles hot code and collects garbage. This chapter opens the hood — with measurements: class loading, object sizes, five garbage collectors on one workload, the JIT deleting allocations, lying benchmarks, the JDK's diagnostic tools, a leak hunt (a **memory leak** is memory never released though no longer needed), a two-module JPMS application (§16.12), and the operating-system mechanics underneath every process (§16.13).

Everything here is about **HotSpot**, the JVM inside OpenJDK and almost every vendor build (Oracle, Temurin, Corretto, Zulu, Microsoft). Other JVMs (OpenJ9, GraalVM native images — programs compiled ahead of time into a standalone executable) differ in details.

**How the code was run.** Programs are in `code/` (package `ch16.*`), compiled with `javac --release 25`, run on **OpenJDK 26** (`26+35`), Apple M4 Pro (14 cores, 48 GiB — **GiB/MiB/KiB** are binary units: 1 GiB = 1024 MiB = 1024 × 1024 KiB — macOS). The JPMS example has its own `jpms/run.sh`. **All timings, byte counts, heap sizes and pauses are machine-dependent** — compare the *shape* of the results. Shenandoah isn't in the Oracle-built OpenJDK 26, so its numbers come from **Amazon Corretto 21.0.10**.

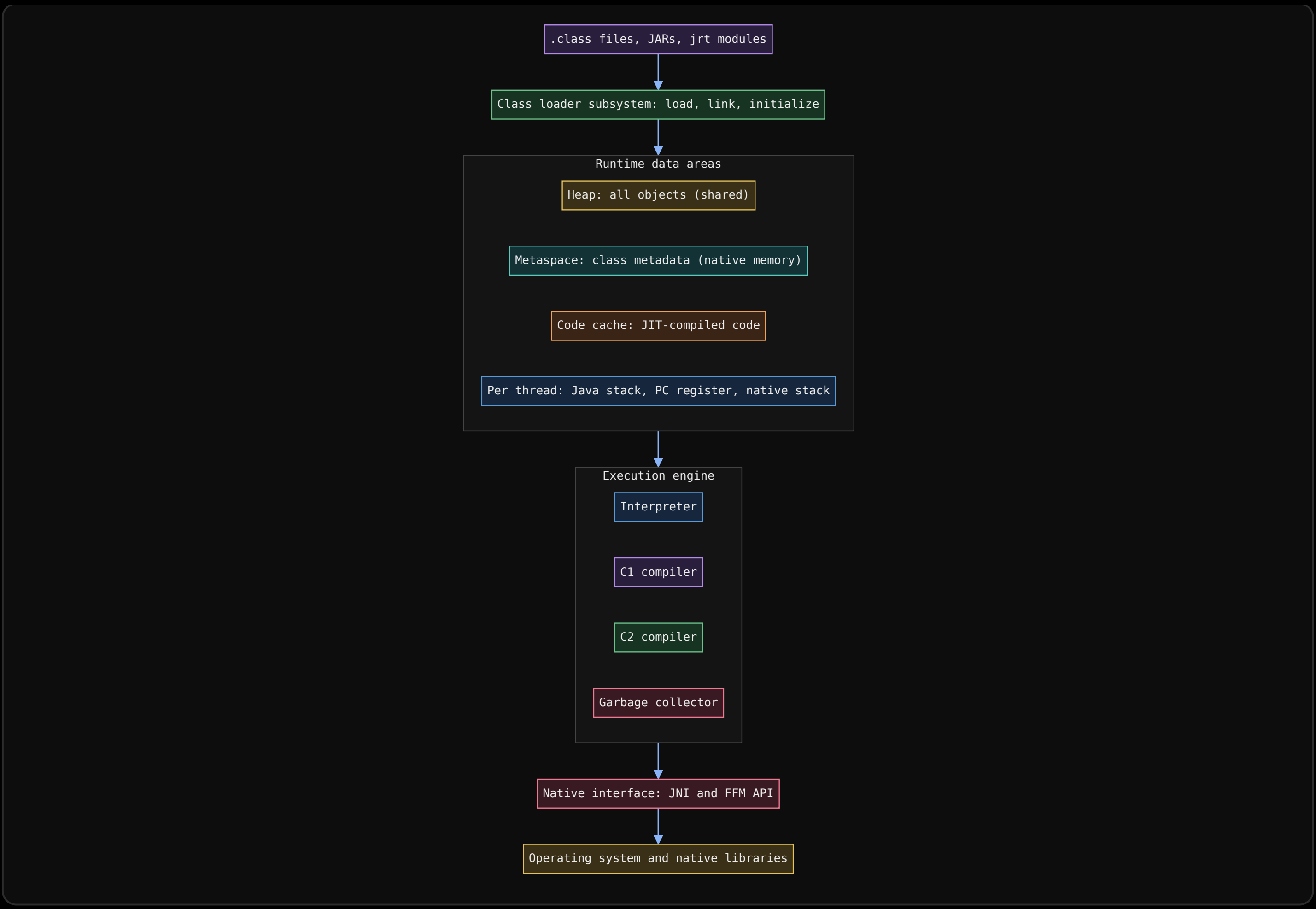
> **Exam depth.** Module directives, command-line options and service-loader traps are drilled in OCP Chapter 12 — Modules. Concurrency-related JVM behaviour (memory model, thread states) is in Chapter 15 and the LLD handbook's concurrency chapters.

---

## 16.1 The JVM's architecture

### What it is

A **Java Virtual Machine** is a small operating system for Java programs: it loads code, gives each thread a **stack** (per-call scratch memory for locals), owns and cleans the shared **heap** of objects, and compiles bytecode to machine code on the fly. The *JVM Specification* says **what** a JVM must do; HotSpot decides **how** (which collectors, which compilers).


.class files, JARs, jrt modules
Class loader subsystem: load, link, initialize
Runtime data areas
Heap: all objects (shared)
Metaspace: class metadata (native memory)
Code cache: JIT-compiled code
Per thread: Java stack, PC register, native stack
Execution engine
Interpreter
C1 compiler
C2 compiler
Garbage collector
Native interface: JNI and FFM API
Operating system and native libraries

| **Subsystem** | **Job** | **Covered in** |
|---|---|---|
| Class loader subsystem | Find class bytes, verify them, prepare statics, run static initialisers | 16.2 |
| Runtime data areas | Memory the JVM manages for you | 16.3, 16.4 |
| Interpreter + JIT compilers | Execute bytecode; compile hot code to machine code | 16.8, Chapter 1 §1.7 |
| Garbage collector | Reclaim unreachable objects, compact memory | 16.5–16.7 |
| Native interface | Call C code: JNI (Java Native Interface, old) or the Foreign Function & Memory API (Java 22, Chapter 17) | — |
| Serviceability | Tools and events that observe a live JVM: JFR, `jcmd`, JMX (Java Management Extensions, the standard monitoring API) | 16.10 |

---

## 16.2 Class loading in depth

### What it is

**Class loading** is how a class goes from bytes on disk to something the JVM can execute. It happens **lazily**: a class is loaded the first time the program needs it, not at start-up.

### How it works

1. **Loading** — a *class loader* finds the bytes (`.class` file, JAR entry, or the JDK's `jrt:` image) and creates a `Class` object plus **metadata** (the JVM's internal description: methods, fields, bytecode) in **metaspace** (§16.3).

2. **Linking** — **verification** (the bytecode is **type-safe**, so hostile class files can't crash the JVM); **preparation** (static fields get default values); **resolution** (symbolic references like "`println` in `java/io/PrintStream`" become direct pointers, lazily on first use).
3. **Initialization** — static initialisers and `static { }` blocks run top to bottom, exactly once, under a lock.

**What triggers initialization?** Creating an instance (`new`), calling a static method, reading or writing a static field that is **not** a compile-time constant (so reading `static final int MAX = 10` doesn't), initialising a subclass (the superclass goes first), **reflection** like `Class.forName("X")`, or being the main class. Merely *loading* a class (e.g. because it is named in a signature) does not initialise it.

### The loader hierarchy and parent delegation

| Loader | Loads | `getClassLoader()` returns |
|---|---|---|
| **Bootstrap** (inside the JVM, written in C++) | `java.base` and other core modules | `null` (e.g. `String.class.getClassLoader()`) |
| **Platform** | Other Java SE / JDK modules such as `java.sql` | the platform loader |
| **Application** (also called *system*) | Your class path (folders/JARs searched for classes) and module path (§16.12) | the app loader |
| Custom (`new URLClassLoader(…)`, frameworks) | Plugins, web apps in **app servers** (Tomcat-style servers hosting several deployed apps), hot-reload tools | your loader |

A loader first **delegates to its parent**; only if the parent can't find the class does it look itself. So nobody can sneak in a fake `java.lang.String`: the bootstrap loader always wins. A class's identity is **its name plus its defining loader** — the same `.class` file loaded by two loaders gives two different, incompatible classes (the source of the infamous "`ClassCastException`: `Foo` cannot be cast to `Foo`" in app servers).

### Seeing it: `code/ch16/classload/LoadingDemo.java`

```java
static class Config {
    static final String NAME = "compile-time constant"; // inlined by javac: does NOT trigger init
    static final long STARTED = System.nanoTime();       // not a constant: reading it DOES trigger init
    static { System.out.println("  [Config static initializer runs]"); }
}
static class Parent { static { System.out.println("  [Parent initialized]"); } static int shared = 1; }
static class Child extends Parent { static { System.out.println("  [Child initialized]"); } }

// in main:
System.out.println("2. reading Config.NAME   -> " + Config.NAME);
long t = Config.STARTED;          // 3.
int s = Child.shared;             // 4. field declared in Parent
new Child();                      // 5.
show("java.lang.String", String.class);
show("java.sql.Date", Class.forName("java.sql.Date"));
```

Output:

```
1. main started
2. reading Config.NAME   -> compile-time constant
3. reading Config.STARTED (triggers init):
  [Config static initializer runs]
4. Child.shared via subclass name (only Parent initializes):
  [Parent initialized]
5. new Child():
  [Child initialized]
6. Who loaded what?
   java.lang.String  loaded by bootstrap (null)
   java.sql.Date     loaded by platform
   LoadingDemo       loaded by app
   app loader's parent  : platform
   platform's parent    : null (bootstrap is native)
   module of String     : java.base
   module of LoadingDemo: unnamed module (class path)
```

What to notice:

- Step 2 printed nothing from `Config`: `NAME` is a `static final` String initialised with a literal — a **constant variable** — so `javac` copied its value into `main`'s bytecode (**inlined** it).
- Step 4 initialised only `Parent`: a static field is "owned" by the class that declares it.

### The class-loading log

`java -Xlog:class+load=info -cp out ch16.classload.LoadingDemo` prints one line per loaded class (real output; paths shortened, timestamps machine-dependent):

```
[0.006s][info][class,load] java.lang.Object source: shared objects file
...
[0.016s][info][class,load] ch16.classload.LoadingDemo source: file:.../out/
[0.017s][info][class,load] ch16.classload.LoadingDemo$Config source: file:.../out/
...
[0.017s][info][class,load] ch16.classload.LoadingDemo$Child source: file:.../out/
...
[0.023s][info][class,load] java.sql.Date source: jrt:/java.sql
```

The run loaded **786 classes**: 770 from the *shared objects file*, 9 from `jrt:/` (the JDK's module image), the rest from `out/`. `Child` is **loaded** at step 4 but **not initialised** until step 5 — separate steps. The **shared objects file** is **Class Data Sharing (CDS)**: a pre-parsed, pre-verified archive of core classes mapped straight into memory, which is why Hello World starts in tens of milliseconds. The JDK ships a default archive (JEP 341, Java 12); you can archive your own classes (AppCDS, JEP 310, Java 10) and, since Java 24–25, a full **AOT cache** (*ahead-of-time*: work done before the program runs and saved) that also records linked classes and method profiles (JEPs 483, 514, 515 — see Chapter 1 §1.7).

### Common mistakes

- Expecting a `static {}` block to run at program start. It runs at first *active use*.
- Heavy work in static initialisers. An exception there becomes `ExceptionInInitializerError`, and every later use of the class throws `NoClassDefFoundError` — the class is permanently broken for that loader.
- Confusing `ClassNotFoundException` (checked; thrown by `Class.forName`/ `loadClass` when a name can't be found) with `NoClassDefFoundError` (an error; the class was present at compile time but is missing or failed to initialise at run time).
- Holding references to a custom loader's classes from shared code: that loader and all its classes can never be unloaded — the *classloader leak* (16.11) behind `OutOfMemoryError: Metaspace` after **redeploys** (replacing a running app without restarting the server).

### In the interview

**Q: Walk me through what happens when the JVM first uses a class.** It is loaded by a class loader using parent delegation (bootstrap → platform → application), linked (verified, statics prepared with defaults, references resolved lazily), then initialised: static initialisers run once, in textual order, under a lock.

- *Why parent delegation?* Core classes can't be spoofed, and each class is loaded once per hierarchy.

- *What is CDS?* A memory-mapped archive of pre-processed classes: faster start-up, metadata shared between JVMs.

### TL;DR

Load (find bytes) → link (verify, prepare, resolve) → initialise (static code, once). Lazy. Loaders delegate to parents first. CDS/AOT caches pre-do this work for faster start-up.

## 16.3 Runtime memory areas

### What it is

The JVM splits its memory into areas with different owners and lifetimes. Knowing them tells you *which* `OutOfMemoryError` you are looking at and which flag controls it.

| Area | Per thread or shared | Holds | Size flag | When it runs out |
|---|---|---|---|---|
| **Heap** | Shared | All objects and arrays | `-Xms` (initial), `-Xmx` (max), `-XX:MaxRAMPercentage` | `OutOfMemoryError: Java heap space` (also `GC overhead limit exceeded` when GC time dominates) |
| **Java stack** | Per thread | One **frame** per active method call: local variables (e.g. a method's local `int`s), operand stack (the scratch stack bytecode pushes and pops values on), return address | `-Xss` / `Thread` constructor `stackSize` | `StackOverflowError` |
| **PC register** (program counter) | Per thread | Address of the current bytecode instruction | — | — |
| **Native method stack** | Per thread | Frames of native (C) code | OS | crash or `StackOverflowError` |
| **Metaspace** | Shared, native memory (straight from the OS, outside the Java heap) | Class metadata: methods, bytecode, constant pools (each class's table of literals and symbolic references), field layouts | `-XX:MaxMetaspaceSize` (unlimited by default) | `OutOfMemoryError: Metaspace` |
| **Compressed class space** | Part of metaspace | Class structures addressed by compressed class pointers | `-XX:CompressedClassSpaceSize` (1 GiB default) | `OutOfMemoryError: Compressed class space` |
| **Code cache** | Shared | JIT-compiled machine code | `-XX:ReservedCodeCacheSize` | Warning "CodeCache is full. Compiler has been disabled" — the app keeps running, slowly |
| **Direct / native memory** | Shared | `ByteBuffer.allocateDirect`, FFM `Arena` memory, thread stacks, GC data structures | `-XX:MaxDirectMemorySize` | `OutOfMemoryError: Direct buffer memory` / `unable to create native thread` |

Since **Java 8** (JEP 122) class metadata lives in **metaspace** (native memory) instead of the old fixed-size *PermGen*, so the classic `PermGen space` error is gone — but an unbounded metaspace can still grow with a classloader leak.

**Seeing it:** `code/ch16/memory/MemoryAreas.java`

The `java.lang.management` API reports the pools the running JVM actually has; recursion in threads of different stack sizes measures stack depth.

```
for (MemoryPoolMXBean p : ManagementFactory.getMemoryPoolMXBeans()) {
    MemoryUsage u = p.getUsage();
    ... p.getType(), p.getName(), u.getUsed() / 1024, u.getMax() / MiB ...
}
for (long stackSize : new long[] {256 * 1024, 1024 * 1024, 8 * 1024 * 1024}) {
    depth = 0;
    Thread t = new Thread(null, () -> {
        try { recurse(); } catch (StackOverflowError e) { /* expected */ }
    }, "deep", stackSize);
    t.start(); t.join();
}
```

Output (machine-dependent: default G1, 48 GiB machine so max heap = 12 288 MiB):

```
Garbage collector(s):
  G1 Young Generation
  G1 Concurrent GC
  G1 Old Generation
Memory pools (type, used KiB / max MiB):
  NON-HEAP CodeHeap 'non-nmethods'               1174 / 5
  NON-HEAP Metaspace                              738 / no limit
  NON-HEAP CodeHeap 'profiled nmethods'           138 / 117
  NON-HEAP Compressed Class Space                  62 / 1024
  HEAP     G1 Eden Space                            0 / no limit
  HEAP     G1 Old Gen                               0 / 12288
  HEAP     G1 Survivor Space                        0 / no limit
  NON-HEAP CodeHeap 'non-profiled nmethods'        38 / 117
Stack   256 KiB -> StackOverflowError after ~1,473 frames
Stack  1024 KiB -> StackOverflowError after ~23,200 frames
Stack  8192 KiB -> StackOverflowError after ~206,700 frames
```

- The **code cache** has three segments: JVM stubs (small helper routines), C1 code still **profiling** usage ("profiled nmethods" — *nmethod* is HotSpot's name for a compiled method), and final C2 code ("non-profiled").
- Depth isn't proportional to stack size: at 256 KiB the method overflowed while still **interpreted** (big frames); with bigger stacks it got **JIT-compiled** (small frames). Never rely on an exact depth.
- Under `-XX:+UseZGC` the pools become `ZGC Young/Old Generation`.

### Native memory tracking

The heap is only part of a Java process's memory. **Native Memory Tracking (NMT)** accounts for the rest. Start the JVM with `-XX:NativeMemoryTracking=summary`, then ask it: `jcmd <pid> VM.native_memory summary scale=MB` (real output for the leak demo of 16.11 with `-Xmx256m`, trimmed):

```
Total: reserved=1722MB, committed=278MB
-                 Java Heap (reserved=256MB, committed=186MB)
-                     Class (reserved=1024MB, committed=0MB)
-                      Code (reserved=244MB, committed=8MB)
-                        GC (reserved=60MB, committed=59MB)
...
```

*Reserved* = address space set aside; *committed* = backed by RAM. The **GC's own bookkeeping** (59 MB: remembered sets and mark bitmaps, §16.5) and other areas push committed memory (278 MB) past the heap — why a 512 MB **container** (an isolated, resource-limited process environment, e.g. Docker) must not get `-Xmx512m`.

### Common mistakes

- Sizing a container by `-Xmx` alone. Budget heap + metaspace + code cache + thread stacks (roughly 1 MB each by default, 2 MB on macOS/AArch64 — 64-bit ARM) + direct buffers + GC overhead. A common start is `-XX:MaxRAMPercentage=75` and measure. If the heap is at 40% but the container OOM-killed the process (§16.13), look at native memory: NMT summary, thread count, direct buffers, metaspace.
- "Fixing" `StackOverflowError` with a giant `-Xss`. Usually the recursion has no base case, or should be a loop (Chapter 4 §4.5).

### TL;DR

Heap = objects (`-Xmx`); stack = frames per thread (`-Xss`); metaspace = class metadata; code cache = JIT output; plus native memory. Each has its own error message — read it.

---

## 16.4 Object layout basics

### What it is

Every heap object carries a hidden **header** before its fields, and sizes are rounded up to a multiple of 8 bytes (**alignment**). Knowing this explains why a million tiny objects cost more than you'd guess, and why `int[]` beats `List<Integer>` for big numeric data.

On 64-bit HotSpot with default settings:

| Part | Size | Contains |
|---|---|---|
| **Mark word** | 8 bytes | Identity hash code (default `hashCode()`), lock state, GC age bits (collections survived) |
| **Class pointer** | 4 bytes (*compressed class pointers*) | Which class this object is |
| Array length (arrays only) | 4 bytes | `length` |
| Fields | `long`/`double` 8, `int`/`float` 4, `short`/`char` 2, `byte`/`boolean` 1, **reference 4** (*compressed oops*) | Your data, reordered by the JVM to reduce gaps |
| Padding | 0–7 bytes | Round total up to a multiple of 8 |

**Compressed oops** ("ordinary object pointers"): with heaps below about 32 GB the JVM stores references as 32-bit offsets and shifts them by 3 (because objects are 8-byte aligned), so each reference costs 4 bytes instead of 8. That is why going from `-Xmx31g` to `-Xmx33g` can *reduce* usable capacity.

**Compact object headers** (JEP 519, a product feature in **Java 25**, opt-in with `-XX:+UseCompactObjectHeaders`; made the default in JDK 27 by JEP 534) squeeze mark word and class pointer into a single **8-byte** header.

### Seeing it: `code/ch16/layout/ObjectSizes.java`

No library needed: HotSpot counts the bytes each thread allocates. Allocate a million objects into an array (so the JIT can't remove them) and divide.

```java
static double bytesPerObject(Maker m) {
    for (int i = 0; i < N; i++) sink[i] = m.make(i);          // warm-up round
    long before = THREADS.getCurrentThreadAllocatedBytes();
    for (int i = 0; i < N; i++) sink[i] = m.make(i);
    long after = THREADS.getCurrentThreadAllocatedBytes();
    return (after - before) / (double) N;
}
... bytesPerObject(i -> new Point(i, i)) ...    // record Point(int x, int y)
```

Output of `java -cp out ch16.layout.ObjectSizes`, then with `-XX:+UseCompactObjectHeaders` (JDK 26):

| Object | Default | Compact headers | Why (default) |
|---|---|---|---|
| `new Object()` | 16.0 | 8.0 | 12-byte header → padded to 16 |
| `record Point(int x, int y)` | 24.0 | 16.0 | 12 + 4 + 4 = 20 → 24 |
| `Node { Object value; Node next; }` | 24.0 | 16.0 | 12 + 4 + 4 (compressed refs) = 20 → 24 |
| `Long` (not cached) | 24.0 | 16.0 | 12 + 8 = 20 → 24 |
| `new int[0]` | 16.0 | 16.0 | 12 + 4 length = 16 |
| `new int[10]` | 56.0 | 56.0 | 16 + 40 = 56 |
| `new byte[10]` | 32.0 | 24.0 | 16 + 10 = 26 → 32 |
| a 6-character `String` | 48.0 | 48.0 | `String` object 24 + its `byte[6]` 24 |

What this means in practice:

- A `List<Integer>` of a million values costs ~4 MB of references **plus** ~16 MB of `Integer`s (outside the -128..127 cache — §7.6); an `int[]` costs 4 MB — 5× less.
- A two-`int` record is 24 bytes for 8 bytes of data: headers dominate small objects. Compact headers cut them by a third (JEP 519 reports 22% less heap and 8% less CPU on SPECjbb2015, an industry-standard Java server benchmark).
- 6 ASCII characters cost 48 bytes as a `String` (Latin-1 strings use 1 byte per char — Java 9 *compact strings*).

### When not to care

Most business code allocates moderately and the GC handles it. Worry about layout when profiling (16.10) shows a type with millions of instances dominating the heap — then consider primitive arrays, fewer wrapper objects, or compact headers.

> **TL;DR**

Header 12 bytes (8 with compact headers), references 4 bytes (compressed oops, heap < ~32 GB), everything padded to 8. Small objects are mostly header; primitive arrays are dense.

---

## 16.5 Garbage-collection algorithms

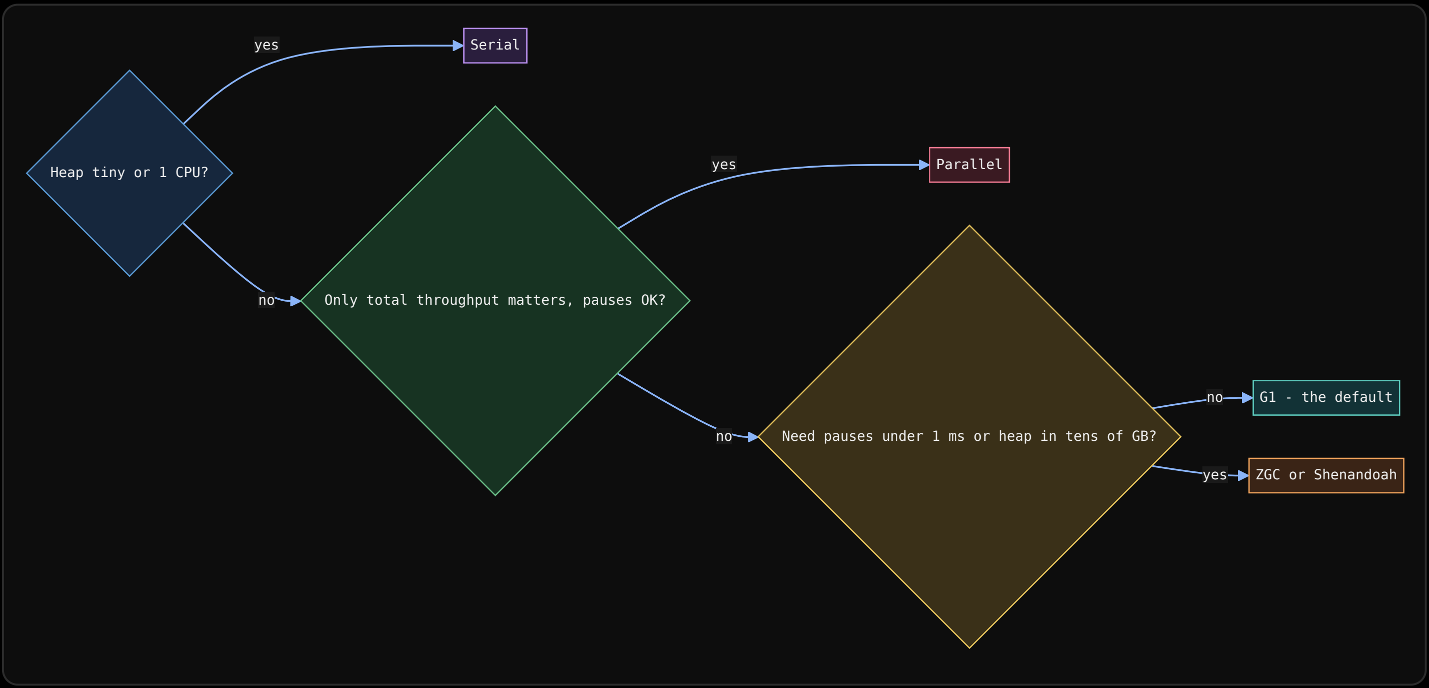
Chapter 1 §1.8 explained **reachability** (alive = reachable by a chain of references), **GC roots** (where chains start: local variables, static fields, active threads) and the **generational** idea (young and old objects kept apart and collected differently). Here are the mechanics behind each HotSpot collector.

## The building blocks

| Technique | How it works | Cost | Used by |
|---|---|---|---|
| **Mark** | Trace from GC roots, marking every reachable object | Proportional to **live** objects | All |
| **Sweep** | Walk memory, add unmarked space to free lists | Proportional to heap size; leaves **fragmentation** (free memory split into many small gaps) | Old gen of Serial (with compaction) |
| **Compact** | Slide live objects together, fix up references | Proportional to live data; no fragmentation | Serial/Parallel old gen, G1 full GC |
| **Copy (evacuate)** | Copy live objects into an empty area; the old area is freed wholesale | Proportional to live data; needs spare space | Young generations everywhere; G1, ZGC, Shenandoah for all regions |
| **Generations** | Collect the young area often (most objects are already dead), the old area rarely | Needs to track old→young references (**card table / remembered sets** — side tables noting which old areas point into young ones) | Serial, Parallel, G1, generational ZGC and Shenandoah |
| **Concurrent phases** | GC threads work *while* application threads run | Needs **barriers** — small code the JIT inserts around reference reads/writes — and extra CPU | G1 marking, ZGC, Shenandoah |

Terms: a **stop-the-world (STW) pause** halts all application threads at a **safepoint** (where every thread's state is known). **Throughput** = share of time running your code, **latency** = pause lengths, **footprint** = memory needed; a collector optimises two, rarely all three.

## The collectors


Heap tiny or 1 CPU?
yes
Serial
no
Only total throughput matters, pauses OK?
yes
Parallel
no
Need pauses under 1 ms or heap in tens of GB?
no
G1 - the default
yes
ZGC or Shenandoah

| Collector | Since | How it works | Pauses | Choose it for |
|---|---|---|---|---|
| **Serial** `-XX:+UseSerialGC` | The original | One GC thread; copying young gen, mark-sweep-compact old gen; all STW | Grow with heap size | Small heaps, 1 CPU, CLI tools, small containers |
| **Parallel** `-XX:+UseParallelGC` | Default before Java 9 | Serial's design with many GC threads | Shorter, still grow with live data | Batch jobs: max throughput, pauses irrelevant |
| **G1** (Garbage-First) `-XX:+UseG1GC` | Default since Java 9 (JEP 248) | Equal **regions** (1–32 MB, sized from the heap). Young regions evacuated in parallel STW pauses; old regions **marked concurrently**, then **mixed** collections evacuate the old regions with the most garbage first (hence the name). Objects ≥ half a region are **humongous** (own regions). *Evacuate* = copy live objects out so the region can be freed. Full GC only as a fallback | Aims at `-XX:MaxGCPauseMillis` (default 200 ms); typically a few to tens of ms | General purpose — the default for most services |
| **ZGC** `-XX:+UseZGC` | Experimental 11, production 15, generational 21 (JEP 439), generational-only since 24 (JEP 490) | Nearly everything concurrent, including **relocation** (moving objects). **Coloured pointers** keep GC state in spare bits of each reference; **load barriers** fix references as the app reads them | Well under 1 ms, independent of heap size (up to 16 TB) | Latency-sensitive services, large heaps |
| **Shenandoah** `-XX:+UseShenandoahGC` | Experimental 12, production 15 (JEP 379); generational mode product in 25 (JEP 521) | Concurrent marking **and** concurrent compaction using load-reference barriers | Sub-millisecond to a few ms | Low latency; shipped in most OpenJDK builds (Temurin, Corretto, Red Hat) but not in Oracle's builds |
| **Epsilon** `-XX:+UnlockExperimentalVMOptions -XX:+UseEpsilonGC` | Java 11 (JEP 318) | Allocates, never frees | None — until the heap is full and the JVM exits | Performance testing, ultra-short jobs |

The removed **CMS** collector (removed in Java 14, JEP 363) is still mentioned in old articles and flags; it no longer exists.

**Default choice**: G1, unless the JVM sees fewer than 2 CPUs or less than 1792 MB of memory, where it picks Serial. JDK 27 (JEP 523) makes G1 the default in all environments.

### How G1 collects, step by step

1. New objects go into **eden** regions (fast: each thread **bump-allocates** — just moves a pointer forward — in its own *TLAB*, thread-local allocation buffer).
2. When eden fills, a **young pause** copies live objects to **survivor** regions; objects that survived enough collections (up to `MaxTenuringThreshold` = 15) are **promoted** (moved) to old regions.
3. When the old generation passes a threshold (*IHOP*, Initiating Heap Occupancy Percent, adaptive), a **concurrent mark cycle** starts: short STW pauses begin and end it (*Concurrent Start*, *Remark*, *Cleanup*); the tracing runs alongside the app.
4. Then several **mixed** pauses evacuate young regions plus a slice of old regions ordered by garbage.
5. If old regions can't be reclaimed fast enough — allocation outruns marking — an **evacuation failure** and eventually a **Full GC** (STW, multi-threaded compaction) occur. Seeing `Pause Full` in a G1 log is a tuning signal (16.6).

### In the interview

**Q: Compare G1 and ZGC.** Both are region-based and generational. G1 copies live objects in stop-the-world pauses aimed at a goal (200 ms default, usually far less) with lower CPU and memory overhead. ZGC also moves objects concurrently (coloured pointers, load barriers), so pauses stay under a millisecond whatever the heap size, at some cost in throughput and **headroom** (spare free memory).

- *Why generational?* Most objects die young, so collecting only the young area is cheap and productive.
- *Why do concurrent collectors need barriers?* The app changes references while the GC traces or moves objects; barriers record or repair those changes.
- *Which collector for a nightly batch job on 32 cores?* Parallel (throughput). ZGC's generational mode arrived in 21, became its default in 23, and is the only mode since 24.

### TL;DR

Serial (tiny), Parallel (throughput), **G1 (default, balanced)**, ZGC and Shenandoah (concurrent compaction, sub-ms pauses), Epsilon (no GC). All modern ones are generational and region-based.

---

## 16.6 Collectors compared on a real workload

**The workload:** `code/ch16/gc/AllocWorkload.java`

A server-like pattern: a cache of 100 000 byte arrays (~100 MB) with random replacement, so objects live a while, get promoted, then die in the old generation — the hard case for generational GC. A **heartbeat** thread sleeps 1 ms in a loop and records its worst extra delay: the latency users would feel.

```
Thread heartbeat = Thread.ofPlatform().daemon().start(() -> {
    long last = System.nanoTime();
    while (running) {
        try { Thread.sleep(1); } catch (InterruptedException e) { return; }
        long now = System.nanoTime();
        long stall = (now - last) / 1_000 - 1_000;          // time beyond the 1 ms we asked for
        if (stall > worstStallMicros) worstStallMicros = stall;
        last = now;
    }
});
byte[][] cache = new byte[100_000][];                         // the live set
SplittableRandom rnd = new SplittableRandom(42);              // fixed seed: same work every run
for (long i = 0; i < allocations; i++) {
    byte[] b = new byte[64 + rnd.nextInt(1_984)];            // 64..2047 bytes, avg ~1 KiB
    b[0] = (byte) i;
    bytes += b.length;
    int slot = rnd.nextInt(cache.length);
    if (cache[slot] != null) checksum += cache[slot][0];     // read the entry we evict
    cache[slot] = b;                                          // old entry becomes garbage
}
```

### G1 vs ZGC with `-Xlog:gc`

`java -Xmx1g -XX:+UseG1GC -Xlog:gc -cp out ch16.gc.AllocWorkload` (real output, start and end):

```
[0.004s][info][gc] Using G1
[0.019s][info][gc] GC(1) Pause Young (Concurrent Start) (G1 Evacuation Pause) 6M->6M(10M) 0.322ms
[0.019s][info][gc] GC(2) Concurrent Mark Cycle
[0.021s][info][gc] GC(2) Pause Remark 12M->12M(33M) 0.112ms
[0.021s][info][gc] GC(2) Pause Cleanup 14M->14M(33M) 0.012ms
[0.021s][info][gc] GC(2) Concurrent Mark Cycle 1.846ms
...
[0.601s][info][gc] GC(49) Pause Young (Mixed) (G1 Evacuation Pause) 746M->282M(1013M) 4.061ms
allocated      : 6,331 MB of payload in 6,000,000 arrays (checksum -2929001)
elapsed        : 641 ms wall, 1,673 ms CPU (whole process)
  G1 Young Generation    count   44, time   132 ms
  G1 Concurrent GC       count   14, time     2 ms
  G1 Old Generation      count    0, time     0 ms
worst stall    : 9.1 ms (seen by the heartbeat thread)
```

Read a G1 line as: GC number, pause kind (Young Normal / Concurrent Start / Prepare Mixed / Mixed / Remark / Cleanup / Full), cause, `before->after(heap size)`, pause length. The concurrent mark cycle itself is not a pause — only Remark and Cleanup are.

`java -Xmx1g -XX:+UseZGC -Xlog:gc -cp out ch16.gc.AllocWorkload`:

```
[0.004s][info][gc] Using The Z Garbage Collector
[0.053s][info][gc] GC(1) Major Collection (Warmup)
[0.059s][info][gc] GC(1) Major Collection (Warmup) 206M(20%)->148M(14%) 0.006s
...
[0.581s][info][gc] GC(13) Minor Collection (Allocation Rate) 640M(62%)->228M(22%) 0.011s
allocated      : 6,331 MB of payload in 6,000,000 arrays (checksum -2929001)
elapsed        : 580 ms wall, 947 ms CPU (whole process)
  ZGC Minor Cycles       count   10, time   110 ms
  ZGC Minor Pauses       count   30, time     0 ms
  ZGC Major Cycles       count    4, time    31 ms
  ZGC Major Pauses       count   20, time     0 ms
worst stall    : 0.8 ms (seen by the heartbeat thread)
```

The time at the end of a ZGC line (`0.011s`) is the **whole concurrent cycle**, not a pause. The actual pauses are logged under `gc+phases` (`-Xlog:gc+phases`):

```
[0.034s][info][gc,phases] GC(0) Y: Pause Mark Start (Major) 0.005ms
[0.039s][info][gc,phases] GC(0) Y: Pause Mark End 0.007ms
[0.039s][info][gc,phases] GC(0) Y: Pause Relocate Start 0.002ms
```

Five **micro**seconds. `Minor` = young-only collection, `Major` = young + old; `Y:`/`O:` = young/old generation.

### Five collectors, same program

Three runs each with `-Xmx1g`, pauses parsed from `-Xlog:gc*` (Shenandoah on Corretto 21.0.10; the rest on OpenJDK 26). **Machine-dependent numbers** — M4 Pro, 14 cores:

| Collector | Wall time | Process CPU | Pauses | Longest pause | Worst stall seen by app |
|---|---|---|---|---|---|
| Serial | 1 570–1 590 ms | ~1 670 ms | 42 | **128–133 ms** | 128–133 ms |
| Parallel | 690–700 ms | ~1 940 ms | 62 | 5–10 ms | 5–10 ms |
| G1 (default) | 630–660 ms | ~1 670 ms | 58 | 9–17 ms | 8–17 ms |
| ZGC | 585–605 ms | ~950–1 010 ms | ~50 | **0.010–0.015 ms** | 0.3–1.8 ms |
| Shenandoah (JDK 21) | 560–570 ms | ~870 ms | 48 | 0.18–0.25 ms | 0.4–0.8 ms |

How to read it:

- **Serial** works on one thread while the app waits: slowest, with 130 ms freezes. **Parallel** spreads the pauses over 11 GC threads: short pauses, most CPU. **G1** matches Parallel here, with tunable pause goals.

- **ZGC/Shenandoah** pauses are tiny; the heartbeat's leftover sub-millisecond "stall" is OS scheduling noise. On this many-core machine they also finished fastest; in a CPU-starved container their concurrent GC threads compete with yours and the picture can flip — measure on production-like hardware.
- JFR agrees (16.10): `jfr view gc-pauses` reported **P99 8.40 ms** (the 99th **percentile**: 99% of pauses were this short or shorter) for G1 and **P99 0.012 ms** for ZGC on a 20-million-allocation run.

### When the heap is too small

The same G1 run with `-Xmx512m` (real log excerpt):

```
[0.207s][info][gc] GC(42) Pause Young (Normal) (G1 Evacuation Pause) (Evacuation Failure: Allocation) 500M->511M(512M) 1.864ms
[0.208s][info][gc] GC(43) Pause Young (Normal) (G1 Evacuation Pause) (Evacuation Failure: Allocation) 512M->512M(512M) 1.192ms
[0.215s][info][gc] GC(44) Pause Full (G1 Compaction Pause) 512M->104M(367M) 6.408ms
```

At ~6.5 GB/s of allocation and ~400 MB of headroom, G1's marking couldn't keep up, evacuation failed (no free region for survivors) and G1 fell back to **six Full GCs** in under a second. The heap after the full GC (104M) is the real **live set** (memory taken by still-reachable objects). The fix is headroom, not a magic flag: at `-Xmx1g` there were no full GCs. ZGC at 512 MB still had no allocation stalls (longest pause 0.019 ms).

> **TL;DR**

Read `before->after(size) pause` in G1 logs; for ZGC, pauses are in `gc+phases`. Serial pauses scale with the heap; G1 gives single-digit milliseconds here; ZGC/Shenandoah give microseconds. `Pause Full` or `Evacuation Failure` in G1 = not enough headroom.

---

## 16.7 GC tuning basics

### The problem it solves

Default settings are good for most programs. Tuning is for a **measured** problem: pauses over an SLA (service-level agreement), throughput too low, or memory too high.

### How to tune (in this order)

1. **Define the goal** — "p99 latency < 50 ms", "batch done in 20 min", "fits a 2 GB container".
2. **Log GC permanently** (cheap): `-Xlog:gc*:file=gc.log:time,uptime,level,tags:filecount=5,filesize=20m`.
3. **Size the heap**: `-Xmx`, or `-XX:MaxRAMPercentage` in containers (**cgroup** limits — the Linux kernel feature containers use to cap CPU and memory — are detected). Rule of thumb: 3–4× the **live set** (heap after a full or mixed GC). `-Xms` = `-Xmx` avoids resizing in long-running services.
4. **Pick the collector** (16.5), then **set a goal, not mechanisms**: G1 `-XX:MaxGCPauseMillis=100`; ZGC mostly needs only `-Xmx`.
5. **Change one thing at a time** and re-measure under realistic load.
6. **Reduce allocation** in code — often the biggest win (JFR allocation views, 16.10, show where).

### The flags you'll actually use

| Flag | Meaning |
|---|---|
| `-Xms<size>` / `-Xmx<size>` | Initial / maximum heap |
| `-XX:MaxRAMPercentage=75` | Max heap as % of machine or container memory (default 25) |
| `-XX:+UseG1GC`, `+UseZGC`, `+UseParallelGC`, `+UseSerialGC`, `+UseShenandoahGC` | Select a collector |
| `-XX:MaxGCPauseMillis=<ms>` | G1/Parallel pause goal (a hint, not a guarantee) |
| `-Xss<size>` | Thread stack size |
| `-XX:MaxMetaspaceSize=<size>` | Cap metaspace (turns a classloader leak into a clear error) |
| `-Xlog:gc*` | Unified GC logging (Java 9+; replaces `-XX:+PrintGCDetails`) |
| `-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=<dir>` | Write a **heap dump** (a file of every object in the heap, for offline analysis) when the heap is exhausted |
| `-XX:+AlwaysPreTouch` | Touch all heap pages at start-up (slower start, no page-fault hiccups later, §16.13) |
| `-XX:+UseCompactObjectHeaders` | Java 25+: 8-byte headers (16.4) |

### What the JVM chose for you: `-XX:+PrintFlagsFinal`

`java -XX:+PrintFlagsFinal -version` prints all ~800 flags with their final values. Excerpt (real, this machine):

```
   size_t G1HeapRegionSize                         = 8388608                                   {product}
{ergonomic}
    uintx MaxGCPauseMillis                         = 200                                       {product} {default}
   size_t MaxHeapSize                              = 12884901888                               {product}
{ergonomic}
     uint ParallelGCThreads                        = 11                                        {product} {default}
     intx ThreadStackSize                          = 2048                                   {pd product} {default}
     bool UseCompactObjectHeaders                  = false                          {product lp64_product}
{default}
     bool UseG1GC                                  = true                                      {product}
{ergonomic}
...
```

`{ergonomic}` means the JVM computed the value from the machine (**ergonomics**): 12 GiB max heap = 25% of 48 GiB, 8 MiB G1 regions for that heap size, G1 selected because the machine is "server class" (at least 2 CPUs and ~2 GB of memory). `{default}` is the built-in default; `{command line}` would mark your own settings. `pd` = platform-dependent (e.g. 2 MiB stacks on macOS/AArch64). `jcmd <pid> VM.flags` shows the same for a running JVM.

### When not to tune

- Without logs or metrics showing a problem.
- By copying flag lists from old blog posts (many flags are obsolete — CMS, PermGen — or harmful, like tiny young generations).
- To hide a leak: a bigger heap only delays the `OutOfMemoryError` (16.11).

### Common mistakes

- Setting `-Xmn`/`NewRatio` with G1: fixing the young size disables G1's pause-time adaptation.
- Calling `System.gc()` in application code (it triggers a full collection; disable with `-XX:+DisableExplicitGC` if a library does it).
- Measuring GC on a laptop and deploying to a 2-CPU container.

**TL;DR**

Measure → size heap from the live set → pick collector → set a goal → one change at a time.

---

## 16.8 JIT compilation, inlining and escape analysis

Chapter 1 §1.7 showed **tiered compilation** (interpreter → C1, a fast simple compiler → C2, a slower optimising one) with `-XX:+PrintCompilation`. This section covers the optimisations that make C2 fast.

### Key optimisations

| Optimisation | What it does | Why it matters |
|---|---|---|
| **Inlining** | Copies a **callee**'s body into the caller (hot methods up to `FreqInlineSize` = 325 bytecode bytes, others up to `MaxInlineSize` = 35) | Removes call overhead and lets other optimisations see across methods; small methods and getters are free |
| **Devirtualisation** | If profiling shows a **virtual call** (target depends on the runtime class) only ever sees one or two **receiver** classes, call them directly (behind a cheap guard checking the assumption) | Interfaces and polymorphism cost almost nothing in hot code |
| **Escape analysis** | Proves an object never leaves the method/thread | **Scalar replacement** (fields become locals — no allocation) and **lock elision** |
| **Loop optimisations** | Unrolling (repeating the body to do fewer jumps), range-check elimination, vectorisation (SIMD — *single instruction, multiple data*) | Array loops approach C speed |
| **Dead-code elimination** | Removes computations whose results are unused | Great for apps, a trap for benchmarks (16.9) |
| **Speculation + deoptimisation** | Optimise for what was observed; fall back to the interpreter if it changes (*deoptimise*) | Aggressive yet correct |

### Seeing escape analysis: `code/ch16/jit/EscapeAnalysis.java`

```java
record Point(int x, int y) {
    Point plus(Point o) { return new Point(x + o.x, y + o.y); }
}
static Point escaped;

static long sumLocal(int n) {                        // Points never leave the method
    long total = 0;
    for (int i = 0; i < n; i++) {
        Point p = new Point(i, i).plus(new Point(1, 2)); // 3 Point allocations per iteration... on paper
        total += p.x() + p.y();
    }
    return total;
}
static long sumEscaping(int n) {                     // one Point per iteration is published
    ... escaped = p; ...
}
```

After warming both methods up, it counts the heap bytes allocated during 10 million iterations.

`java -cp out ch16.jit.EscapeAnalysis`, then `java -XX:-DoEscapeAnalysis -cp out ch16.jit.EscapeAnalysis`:

```
DoEscapeAnalysis = true
local      result=100000020000000  heap allocated for 10M iterations: 0 bytes
escaping   result=100000020000000  heap allocated for 10M iterations: 240,000,000 bytes
DoEscapeAnalysis = false
local      result=100000020000000  heap allocated for 10M iterations: 720,000,000 bytes
escaping   result=100000020000000  heap allocated for 10M iterations: 720,000,000 bytes
```

- With escape analysis, 30 million `Point`s in `sumLocal` cost **zero** bytes: after inlining, C2 kept `x` and `y` in CPU **registers**. In `sumEscaping` only the published `Point` escaped (10M × 24 B = 240 MB); the two temporaries were scalar-replaced.
- Switched off, all 3 × 10M × 24 B = 720 MB are allocated.

Lesson: short-lived small objects in hot code (records, iterators, `Optional`, lambdas) are often free — write clear code first, profile before "optimising" allocations away by hand.

### Interpreter vs C1 vs C2 on real work

The first part of `code/ch16/bench/NaiveBenchmark.java` computes Collatz sequence lengths (halve even numbers, map odd $n$ to $3n+1$, until 1 — a cheap CPU workout) for 20 000 numbers per round. Round times (ms; machine-dependent):

| Mode | Round 1 | Rounds 6–8 |
|---|---|---|
| Default (tiered, C2) | 3.37 | 2.0–2.2 |
| `-XX:TieredStopAtLevel=1` (C1 only) | 4.54 | 4.3–4.6 |
| `-Xint` (interpreter only) | 23.21 | 23.1–23.6 |

The interpreter is ~10× slower than C2 here; C1 alone is about 2× slower. `TieredStopAtLevel=1` is sometimes used for short-lived tools where start-up matters more than peak speed.

### Code cache

Compiled code lives in the code cache (16.3). If it fills, the JIT stops compiling and new hot code stays interpreted — the app runs on, slower. `jcmd <pid> Compiler.codecache` shows its use; increase `-XX:ReservedCodeCacheSize` if logs say *CodeCache is full*.

### In the interview

**Q: What is escape analysis?** C2 proves an object is never seen outside the method or thread that made it, then replaces it with its fields in registers (no allocation, no GC) and drops locks on it. It depends on inlining, so it works best on small, hot code.

- *Does Java allocate on the stack?* Not literally; scalar replacement has the same effect for non-escaping objects.
- *What is deoptimisation?* Discarding compiled code whose speculation failed (e.g. a new subclass appeared) and continuing in the interpreter.

### TL;DR

C2 inlines, devirtualises, removes non-escaping allocations and dead code, and speculates with a safety net. Prove optimisations with measurement (allocated bytes, JFR), not guesses.

---

## 16.9 Benchmarking: why naive timing lies, and JMH

### The problem it solves

"Is `A` faster than `B`?" sounds simple, but on the JVM the same code runs at three different speeds (interpreter, C1, C2), the JIT may delete work whose result you never use, and GC pauses can land in any measurement.

### Two traps, measured

`code/ch16/bench/NaiveBenchmark.java`:

```
for (int i = 0; i < 50_000_000; i++) Math.sqrt(i);          // result discarded
...
double acc = 0;
for (int i = 0; i < 50_000_000; i++) acc += Math.sqrt(i);   // result used
sink = acc;
```

Output (machine-dependent):

```
1) Warm-up: identical rounds of 20,000 Collatz sequences
   round 1:   3.37 ms (result 1834634)
   round 2:   2.42 ms (result 1834634)
   ...
   round 8:   1.99 ms (result 1834634)
2) Dead code: 50M square roots, result ignored vs used
   round 1: ignored    1.55 ms   used   27.41 ms
   round 2: ignored    3.48 ms   used   26.12 ms
   round 3: ignored    0.00 ms   used   26.35 ms
```

1. **Warm-up**: round 1 is ~70% slower than round 8 — measure the first run and you compare the interpreter, not your algorithm.
2. **Dead-code elimination**: 50 million square roots "in 0.00 ms" is impossible — the JIT removed the loop because nobody used `Math.sqrt`'s result.

Other traps: **constant folding** (inputs known at compile time get precomputed), **loop hoisting** (moving invariant work out of the loop), **GC noise** (allocation from an earlier test collected during a later one), **profile pollution** (benchmarking two implementations in one JVM makes a call site polymorphic), **CPU frequency scaling**, and **`System.currentTimeMillis()`** (low resolution — use `System.nanoTime()` for intervals).

### What JMH is

**JMH** (Java Microbenchmark Harness, an OpenJDK project) handles these traps: separate warm-up and measurement iterations, fresh **forked** JVMs per benchmark (isolating profiles and JIT decisions), a `Blackhole` that "consumes" results so the JIT can't eliminate them, state fields that prevent constant folding, and error bars.

```
// (JMH — not run in this book: needs the jmh-core library and its annotation processor)
@State(Scope.Thread)
@BenchmarkMode(Mode.AverageTime) @OutputTimeUnit(TimeUnit.NANOSECONDS)
@Warmup(iterations = 5, time = 1) @Measurement(iterations = 5, time = 1) @Fork(2)
public class SqrtBench {
    double x = 12345.678;                          // field, not a constant: no constant folding

    @Benchmark public double sqrt() { return Math.sqrt(x); }          // returned value is consumed
    @Benchmark public void sqrtBlackhole(Blackhole bh) { bh.consume(Math.sqrt(x)); }
}
```

To run: generate a project from the Maven archetype (Chapter 18) `org.openjdk.jmh:jmh-java-benchmark-archetype`, `mvn package`, `java -jar target/benchmarks.jar`. Results read `Score ± Error`; overlapping error ranges mean no demonstrated difference.

## When not to use microbenchmarks

Microbenchmarks answer tiny questions. For "why is my service slow?", **profile the real application** under load (16.10) — the **bottleneck** is usually I/O, locks or an algorithm.

## TL;DR

Never trust a single `nanoTime` loop. Use JMH for micro questions and a profiler for real ones.

## 16.10 Profiling and diagnostic tools

### The JDK's own toolbox (all in `$JAVA_HOME/bin`)

| Tool | What it does | Typical use |
|---|---|---|
| `jps -l` | Lists running JVMs with PIDs and main classes | Find the process |
| **`jcmd <pid> <command>`** | Swiss-army knife: sends diagnostic commands to a live JVM | Flags, heap info, class histogram, thread dump, JFR start/stop, heap dump, NMT |
| `jstat -gcutil <pid> <ms>` | Samples GC statistics periodically | Watch heap occupancy and GC counts live |
| `jstack <pid>` | Thread dump (same as `jcmd <pid> Thread.print`) | Deadlocks, stuck threads |
| `jmap`, `jinfo` | Older heap/flag tools, mostly superseded by `jcmd` | Legacy scripts |
| `jfr` | Reads Flight Recorder files | Offline analysis |
| `jconsole` / `jhsdb` | Basic JMX GUI / inspect a hung process or **core dump** (snapshot of a crashed process's memory) | Quick look / last resort |

Free external tools: **JDK Mission Control (JMC)**, the JFR GUI (separate download); **VisualVM** (visualvm.github.io) — live monitoring, sampling profiler, heap-dump browser; **async-profiler** — low-overhead CPU/allocation **flame graphs** (each bar is a method; its width shows its share of time or memory) (Linux/macOS).

### `jcmd` on a live process

The leak demo of 16.11 can be kept alive with `java -Xmx256m -cp out ch16.leak.ListenerLeak leak 12`. Real session (PID machine-specific):

```
$ jps -l | grep ListenerLeak
1796 ch16.leak.ListenerLeak
$ jcmd 1796 VM.version
1796:
OpenJDK 64-Bit Server VM version 26+35-2893
JDK 26.0.0
$ jcmd 1796 GC.heap_info
1796:
garbage-first heap   total reserved 262144K, committed 191488K, used 105337K [0x00000007f0000000, 0x0000000800000000)
 region size 1M, 2 eden (2M), 0 survivor (0M), 104 old (104M), 0 humongous (0M), 81 free (81M)
$ jcmd 1796 Thread.print | head
Full thread dump OpenJDK 64-Bit Server VM (26+35-2893 mixed mode, sharing)
...
"main" #3 [6403] prio=5 os_prio=31 cpu=57.63ms elapsed=5.99s tid=0x0000000148019400 nid=6403 waiting on condition [0x000000016d7b6000]
   java.lang.Thread.State: TIMED_WAITING (sleeping)
    ...
    at ch16.leak.ListenerLeak.main(ListenerLeak.java:68)
```

`jcmd <pid> help` lists every command (on JDK 26: `GC.class_histogram`, `GC.heap_dump`, `JFR.start`, `JFR.dump`, `JFR.view`, `Thread.dump_to_file` — which also lists virtual threads —, `VM.flags`, `VM.native_memory`, `VM.metaspace`, `Compiler.codecache` and more).

### `jstat` — GC counters every 500 ms

While `AllocWorkload` ran with 60 million allocations: `jstat -gcutil <pid> 500 6`

```
  S0     S1     E      O      M     CCS    YGC     YGCT     FGC    FGCT     CGC    CGCT       GCT
     - 100.00  68.66  30.61  52.38  13.36     56     0.171     0     0.000    20     0.004     0.174
     - 100.00   7.26  36.10  52.38  13.36     69     0.220     0     0.000    26     0.005     0.225
     ...
     - 100.00  47.49  30.37  52.38  13.36    119     0.406     0     0.000    50     0.010     0.415
```

Columns are percentages of capacity for **S**urvivor 0/1, **E**den, **O**ld, **M**etaspace, **C**ompressed **C**lass **S**pace, then counts and total seconds for **Y**oung GC, **F**ull GC, **C**oncurrent GC. Here: ~25 young GCs per second, no full GCs, old gen oscillating 30–44% as mixed collections clean it — healthy. A steadily rising `O` that never drops after GCs is the signature of a leak.

### Java Flight Recorder (JFR)

**What it is**: an event recorder built into the JVM (open-sourced in JDK 11, JEP 328) — the aircraft "black box". It records GC, JIT, locks, I/O, allocation samples and CPU samples with typically ~1% overhead (default settings), so it is safe to leave on in production.

**How to use it**

1. Start at launch: `java -XX:StartFlightRecording=filename=rec.jfr,settings=profile …` (or attach later: `jcmd <pid> JFR.start duration=60s filename=rec.jfr`).
2. Dump or stop: the recording is written when the program exits or with `jcmd <pid> JFR.dump name=1 filename=rec.jfr`.
3. Analyse: `jfr summary`, `jfr print --events <type>`, `jfr view <view>` (JDK 21+), or open in JMC.

Real session on `AllocWorkload 20000000` (G1, `-Xmx1g`):

```
$ jfr summary rec.jfr
 ...
 Event Type                              Count  Size (bytes)
=============================================================
 jdk.GCPhaseParallel                     37056        978649
 ...
 jdk.ObjectAllocationSample                570          8201
 jdk.ExecutionSample                       117          1166
 jdk.GarbageCollection                      80          1778
 ...
$ jfr print --events jdk.GarbageCollection rec.jfr | head -11
jdk.GarbageCollection {
  ...
  name = "G1New"
  cause = "G1 Evacuation Pause"
  sumOfPauses = 1.72 ms
  ...
}
```

```
$ jfr view gc-pauses rec.jfr          (G1)          $ jfr view gc-pauses recz.jfr    (same run, ZGC)
Total Pause Time: 270 ms                           Total Pause Time: 0.955 ms
Number of Pauses: 94                               Number of Pauses: 173
Median Pause Time: 3.68 ms                         Median Pause Time: 0.00521 ms
P99 Pause Time: 8.40 ms                            P99 Pause Time: 0.0120 ms
Maximum Pause Time: 8.40 ms                        Maximum Pause Time: 0.0121 ms
```

(The two outputs are shown side by side and trimmed to the key lines.)

```
$ jfr view hot-methods rec.jfr
Method                                                          Samples Percent
ch16.gc.AllocWorkload.main(String[])                                116   99.15%
...
$ jfr view allocation-by-class rec.jfr
Object Type                                                 Allocation Pressure
byte[]                                                                  100.00%
...
```

`jfr view` with no name lists all views (`allocation-by-site`, `contention-by-site`, `gc`, `pinned-threads`, …). Recordings can contain environment variables and system properties — review before sharing (`jfr scrub` removes events).

### Which tool for which symptom?

| Symptom | First tool | Then |
|---|---|---|
| High CPU | JFR `hot-methods` / async-profiler flame graph | Check GC CPU in JFR `gc` view |
| Latency spikes | GC log pauses; JFR `gc-pauses`, `safepoints` | Lock contention views, thread dumps |
| App hangs | `jcmd <pid> Thread.print` two or three times | Look for `BLOCKED` threads and "Found one Java-level deadlock" |
| Memory keeps growing | `jstat -gcutil` (old gen after GC); `GC.class_histogram` twice | JFR `OldObjectSample` with `path-to-gc-roots=true`; heap dump in VisualVM/Eclipse MAT (Memory Analyzer Tool) |
| Process RSS (RAM really used, §16.13) > heap | NMT `VM.native_memory` | Thread count, direct buffers, metaspace |
| Slow start-up | `-Xlog:class+load`, `-Xlog:startuptime` | CDS / AOT cache |

### In the interview

**Q: Latency spikes every few minutes. How do you investigate?** Correlate spikes with the GC log (long pauses, full GCs, evacuation failures), then record JFR across a spike and check GC pauses, safepoints, lock **contention** (threads queuing for one lock) and allocation hot spots. If GC is the cause, fix allocation rate or headroom before touching flags.

- *Thread dump vs heap dump?* Thread dump: every thread's stack and locks — small and fast. Heap dump: every object — large, pauses the JVM, may contain secrets.

### TL;DR

`jps` → `jcmd` (everything) → `jstat` (live GC counters) → JFR for low-overhead recordings (`jfr summary/print/view`, JMC). Heap dumps are the heavy tool of last resort.

## 16.11 Memory leaks

### What it is

A **memory leak** in Java is memory that stays **reachable** although the program will never use it again. The GC can't free it — something still points to it.

### Why it matters

Leaks grow slowly: fine for days, then ever more frequent GCs, rising latency, and finally `OutOfMemoryError: Java heap space` — often at peak traffic.

### Common leak shapes

| Shape | Example | Fix |
|---|---|---|
| Unbounded static collection | `static Map<String, Result> cache` that only grows | Bounded cache with **eviction** (removing old or least-used entries: Caffeine, or `LinkedHashMap.removeEldestEntry` — Chapter 10) |
| Listeners never removed | Per-request subscription to an app-wide **event bus** (forwards published events to every subscriber) | Unsubscribe (try-with-resources handle) |
| `ThreadLocal` on pooled threads | Setting per-request data on a thread pool thread and never calling `remove()` | `try { … } finally { tl.remove(); }`; scoped values (Java 25) |
| Wrong `equals` / `hashCode` for map keys | Mutable key changed after insertion; entry can never be found or removed | Immutable keys, e.g. records (Chapter 6 §6.13) |
| Classloader leaks | A shared class keeps a reference into a redeployed web app | Clean up drivers, threads, thread-locals on undeploy |
| Unclosed resources | Streams, connections holding native buffers | try-with-resources (Chapter 8) |
| Inner-class capture | A stored inner-class instance or lambda keeps its outer object alive | Static nested classes; capture less |

### A measured leak: `code/ch16/leak/ListenerLeak.java`

```java
static final class EventBus {
    static final List<Consumer<String>> LISTENERS = new CopyOnWriteArrayList<>();
    static AutoCloseable subscribe(Consumer<String> l) {
        LISTENERS.add(l);
        return () -> LISTENERS.remove(l);                 // the handle that lets callers clean up
    }
}
static final class RequestSession {
    final byte[] buffer = new byte[10 * 1024];            // per-request working memory
    void onEvent(String e) { buffer[0]++; }
}
static void handleRequestLeaky() {
    RequestSession s = new RequestSession();
    EventBus.subscribe(s::onEvent);                       // BUG: never unsubscribed
}
static void handleRequestFixed() throws Exception {
    RequestSession s = new RequestSession();
    try (AutoCloseable sub = EventBus.subscribe(s::onEvent)) {   // unsubscribed when the request ends
        // ... handle the request ...
    }
}
static long usedAfterGcKiB() {
    System.gc();                                          // only for measuring in a demo
    return ManagementFactory.getMemoryMXBean().getHeapMemoryUsage().getUsed() / 1024;
}
```

The bound method reference `s::onEvent` captures its receiver `s`, so the static list → lambda → session → 10 KiB buffer chain keeps every request's memory alive.

Output (KiB figures vary slightly between runs — machine-dependent):

```
mode = leak
  after  2,000 requests: listeners= 2,000  heap growth since start= 20,211 KiB
  after  4,000 requests: listeners= 4,000  heap growth since start= 40,537 KiB
  ...
  after 10,000 requests: listeners=10,000  heap growth since start=101,210 KiB
mode = fixed
  after  2,000 requests: listeners=     0  heap growth since start=      0 KiB
  ...
  after 10,000 requests: listeners=     0  heap growth since start=      0 KiB
```

The key measurement is **heap used *after* a full GC**: linear growth (~10 KiB per request) is a leak; flat is healthy. In production, read the same signal from the GC log's "after" numbers or `jstat`'s `O` column — no `System.gc()` needed.

### Finding the culprit without a heap dump

**Step 1 — class histogram** (`jcmd <pid> GC.class_histogram`, real output, trimmed):

```
 num     #instances         #bytes  class name (module)
-------------------------------------------------------
   1:         20232      103004240  [B (java.base@26)
   ...
   6:         10000         160000  ch16.leak.ListenerLeak$$Lambda/0x000001fc01041800
   7:         10000         160000  ch16.leak.ListenerLeak$RequestSession
```

`[B` is `byte[]`. Exactly 10 000 sessions and 10 000 lambdas after 10 000 requests: suspicious. Take two histograms minutes apart; classes whose counts only rise are the suspects. (By default the histogram runs a full GC first, so it counts only live objects.)

**Step 2 — JFR old-object sampling with paths to GC roots**: `java -XX:StartFlightRecording=filename=leak.jfr,settings=profile,path-to-gc-roots=true … leak`, then `jfr print --events jdk.OldObjectSample --stack-depth 3 leak.jfr` (one of 178 samples):

```
jdk.OldObjectSample {
  ...
  objectSize = 10.0 kB
  object =  [
    byte[10240]
    buffer : ch16.leak.ListenerLeak$RequestSession
    arg$1 : ch16.leak.ListenerLeak$$Lambda.0x0000180001045400
    [9822] : java.lang.Object[10000]
    array : java.util.concurrent.CopyOnWriteArrayList
    LISTENERS : java.lang.Class Class Name: ch16.leak.ListenerLeak$EventBus
    ...
  ]
  ...
  stackTrace = [
    ch16.leak.ListenerLeak$RequestSession.<init>() line: 27
    ch16.leak.ListenerLeak.handleRequestLeaky() line: 32
    ...
  ]
}
```

Read the `object` chain bottom-up and you have the whole story: the `LISTENERS` static field of `EventBus` → its `CopyOnWriteArrayList` → element 9822 → the lambda (its captured `arg$1`) → the session's `buffer`. The stack trace shows **where** the leaked object was allocated. All of this without stopping the JVM to write a multi-gigabyte heap dump.

**Step 3 (if needed) — heap dump**: `jcmd <pid> GC.heap_dump /tmp/app.hprof` or `-XX:+HeapDumpOnOutOfMemoryError`, analysed in Eclipse MAT or VisualVM ("dominator tree", "path to GC roots"). Heap dumps pause the app and contain all data in memory (passwords, personal data) — handle them as secrets.

#### Common mistakes

- Treating a large heap as a leak. A cache that is big but bounded is fine; a leak is **growth without bound** in heap-after-GC.

- "Fixing" with `WeakReference` everywhere (a weak reference doesn't keep its object alive; the GC may clear it). Weak references suit canonicalising maps (one shared instance per value) and listener registries you don't own; caches usually want size- or time-based eviction.

### In the interview

**Q: Can Java leak memory if it has a GC?** Yes: the GC frees only *unreachable* objects, so anything still referenced but no longer needed — static maps, listeners never removed, thread-locals on pooled threads — leaks. Confirm with heap-after-GC growth, find the type with class histograms or JFR old-object samples, and follow the path to the GC root.

- *Leak vs high usage?* A leak keeps growing in heap-after-GC; high usage plateaus.

### TL;DR

Leak = still reachable, never used. Measure heap after GC → class histogram → JFR `OldObjectSample` path to root → fix the reference (unsubscribe, bound the cache, `remove()` the thread-local).

---

## 16.12 Modules (JPMS) basics

### What it is

The **Java Platform Module System** (JPMS, Java 9, JEP 261) adds a level above packages: a **module** is a named group of packages whose `module-info.java` declares **what it needs** (`requires`) and **what it offers** (`exports`). The JDK itself is about 70 modules (`java.base`, `java.sql`, …).

### The problem it solves

The flat **class path** made every `public` class usable by everyone (hence widespread use of internals like `sun.misc.Unsafe`), found missing or duplicate JARs only at run time ("JAR hell"), and couldn't produce a smaller JDK. Modules give **reliable configuration**, **strong encapsulation** (non-exported packages are inaccessible, even to reflection) and a **modular JDK** that `jlink` can trim.

### The directives

| Directive | Meaning |
|---|---|
| `requires M;` | This module reads module `M` (needs it at compile and run time) |
| `requires transitive M;` | …and anyone who requires me also reads `M` (for types that appear in my API) |
| `requires static M;` | Needed at compile time only; optional at run time |
| `exports p;` | Public types in package `p` are accessible to other modules |
| `exports p to M1, M2;` | Qualified export — only to the named modules |
| `opens p;` / `opens p to M;` | Allow deep reflection (private members) on `p` at run time — what Jackson or Hibernate need |
| `open module m { … }` | Opens every package of the module for reflection |
| `uses S;` / `provides S with Impl;` | Service consumer / provider for `ServiceLoader` (the JDK's plug-in lookup) |

`java.base` is required implicitly by every module.

**The example:** `jpms/`

```
jpms/src/
├── com.example.greet/                 (library module)
│   ├── module-info.java
│   └── com/example/greet/Greeter.java, internal/Formatter.java
└── com.example.app/                   (application module)
    ├── module-info.java
    └── com/example/app/Main.java
```

```java
// src/com.example.greet/module-info.java
module com.example.greet {
    exports com.example.greet;             // public types here are usable by other modules
    // com.example.greet.internal is NOT exported: public classes there are still inaccessible outside
}

// src/com.example.app/module-info.java
module com.example.app {
    requires com.example.greet;            // checked at compile AND launch time
    requires java.logging;                 // a JDK module; java.base is always required implicitly
}
```

```java
// src/com.example.app/com/example/app/Main.java
public class Main {
    public static void main(String[] args) throws Exception {
        System.out.println(Greeter.greet("modules"));
        System.out.println("my module      : " + Main.class.getModule().getName());
        System.out.println("Greeter module : " + Greeter.class.getModule().getName());
        System.out.println("Logger module  : " + Logger.class.getModule().getName());
        System.out.println("app reads greet: " + Main.class.getModule().canRead(Greeter.class.getModule()));
        Class<?> hidden = Class.forName("com.example.greet.internal.Formatter");
        try {
            hidden.getConstructor().newInstance();
        } catch (IllegalAccessException e) {
            System.out.println("reflection     : " + e.getMessage());
        }
    }
}
```

`Formatter` is a `public` class with a `public` constructor — yet outside its module it is off-limits. Run with `bash jpms/run.sh` (the book's generic runner skips this folder: it compiles `code/` as one class-path unit). Real output:

```
$ javac -d mods --module-source-path src $(find src -name "*.java")
$ java --module-path mods -m com.example.app/com.example.app.Main
*** Hello, modules ***
my module      : com.example.app
Greeter module : com.example.greet
Logger module  : java.logging
app reads greet: true
reflection     : class com.example.app.Main (in module com.example.app) cannot access class com.example.greet.internal.Formatter (in module com.example.greet) because module com.example.greet does not export com.example.greet.internal to module com.example.app
$ jdeps --module-path mods -s mods/com.example.app
com.example.app -> com.example.greet
com.example.app -> java.base
com.example.app -> java.logging
$ jlink --module-path mods --add-modules com.example.app --output runtime --strip-debug --no-header-files --no-man-pages
$ runtime/bin/java --list-modules
com.example.app
com.example.greet
java.base@26
java.logging@26
$ du -sh runtime
44M  runtime/
```

And if `Main` imports the internal package, compilation fails (real `javac` output):

```
Main.java:4: error: package com.example.greet.internal is not visible
import com.example.greet.internal.Formatter;
                        ^
  (package com.example.greet.internal is declared in module com.example.greet, which does not export it)
```

`--module-source-path` compiles several modules at once (one sub-folder per module); `-m module/mainclass` launches. `jdeps` shows dependencies (also useful on plain JARs to find use of JDK internals). `jlink` built a **44 MB** self-contained runtime with just four modules, compared with the full 377 MB JDK on this machine — ideal for container images.

### Class path vs module path

| Where the JAR is | It becomes | Can read | Exports |
|---|---|---|---|
| Class path | Part of the **unnamed module** | Every module | Everything (but named modules can't `require` it) |
| Module path, has `module-info.class` | **Named (explicit) module** | Only what it `requires` | Only what it `exports` |
| Module path, no `module-info` | **Automatic module** (name from `Automatic-Module-Name` in the JAR's `META-INF/MANIFEST.MF` **manifest**, else the file name) | Every module | Everything |

This is why modules were adoptable gradually: most applications still run on the class path, while the **JDK's** internals are strongly encapsulated for everyone since Java 16–17 (JEPs 396 and 403). Code that reflects into JDK internals needs `--add-opens java.base/java.lang=ALL-UNNAMED` (a temporary workaround, not a fix).

### When not to use it

Most applications stay on the class path — frameworks work fine there, and modularising adds `opens` friction with reflective libraries. Modules pay off for **libraries**, **large codebases** wanting enforced boundaries, and **`jlink` runtimes**.

### Common mistakes

- Module name ≠ package name confusion: `requires` takes a *module* name, `exports` a *package* name.
- Split packages: the same package in two modules is an error on the module path.
- Forgetting `opens` for packages that frameworks access reflectively → `InaccessibleObjectException` at run time.
- Cyclic `requires` between modules — not allowed.

### In the interview

**Q: What does JPMS give you over the class path?** Reliable configuration (dependencies checked at start-up), strong encapsulation (only exported packages are accessible; deep reflection needs `opens`), and a modular JDK that `jlink` can shrink.

- *`exports` vs `opens`?* Compile- and run-time access to public types vs run-time reflective access to all members, private included.

### TL;DR

`module-info.java`: `requires` what you need, `exports` your API, `opens` for reflection. Class path = unnamed module. `jdeps` to analyse, `jlink` to ship a small runtime. Exam-level detail: OCP Chapter 12.

## 16.13 Operating-system fundamentals for engineers

### What it is

The JVM is a program like any other: the **operating system (OS) kernel** gives it CPU time, memory and access to files and the network. Many "JVM" problems — 100% CPU, `Too many open files`, a container killed with no Java stack trace, a slow first request on a cold disk — are really OS behaviour. This section covers the OS ideas every backend engineer is expected to know, each tied to what you see from Java (process vs thread basics are introduced in §14.1). The Python handbook points here too (Python ch.17); the ideas are language-neutral.

### Processes vs threads

| | Process | Thread |
|---|---|---|
| What | a running program with its **own virtual address space** (below), file-descriptor table and resources | a path of execution **inside** a process: its own stack, registers and program counter |
| Shares memory with | nobody (only through explicit IPC) | every other thread in the same process — hence data races (14.5) |
| Create / switch cost | higher: new address space; switching may flush address-translation caches | lower, but a platform thread still has a kernel structure and ~1 MiB of reserved stack by default |
| Crash isolation | one process crashing leaves others alive | one thread corrupting memory can take down the whole process |
| Java | `ProcessBuilder`, `ProcessHandle` (each JVM is one process) | each **platform thread** is one OS thread (1:1). **Virtual threads** (15.4) are scheduled by the JVM onto a few OS carrier threads (M:N) |

On Linux, `fork()` creates a child process as a copy of the parent (using **copy-on-write**: pages are shared until one side writes), and `exec()` replaces its program — that is what `ProcessBuilder.start()` does. Processes talk through **IPC** (inter-process communication): pipes (what `Process.getInputStream()` reads), sockets (including Unix domain sockets, supported by `SocketChannel` since Java 16), shared memory (a memory-mapped file), **signals** (small numbered OS notifications; `kill -TERM` triggers the JVM's **shutdown hooks**, code registered to run at exit) and message queues.

### CPU scheduling

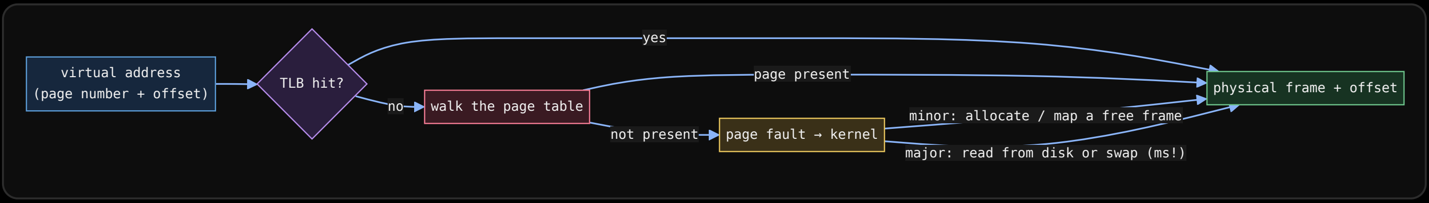
A CPU core runs one thread at a time. The kernel's **scheduler** gives each runnable thread a short **time slice**, then **preempts** it (a timer **interrupt** forces the CPU into kernel code) and saves its registers — a **context switch**, costing a few microseconds plus cache warm-up. Linux's scheduler (CFS, the Completely Fair Scheduler, replaced by EEVDF in kernel 6.6) aims at fairness weighted by priority (`nice`). Threads waiting for I/O or a lock are **blocked** and use no CPU; **load average** counts threads running or waiting to run (on Linux also uninterruptible disk waits).

Consequences for Java: a pool of 500 CPU-bound platform threads on 8 cores gets no faster than ~8 threads, just more switching; blocked threads are cheap in CPU but not in memory; and container CPU **limits** (cgroup quotas) can pause (**throttle**) a whole JVM for the rest of a 100 ms period once its quota is spent — the JVM reads the limit to size GC and fork-join threads (`-XX:ActiveProcessorCount` overrides it).

**Diagnosing 100% CPU:** `top -H -p <pid>` (Linux) lists threads by CPU; convert the hot thread id to hex; `jcmd <pid> Thread.print` shows each Java thread with its OS id as `nid=0x…`; match them to find the stack burning CPU. Take three dumps a few seconds apart — a frame that stays on top is the culprit. JFR's `ExecutionSample` events (16.10) do the same statistically.

### Virtual memory and paging

Every process sees its own **virtual address space**; it never uses physical addresses. Memory is managed in fixed-size **pages** (4 KiB on most x86-64 Linux, 16 KiB on Apple Silicon — the demo prints it). A per-process **page table** maps virtual pages to physical **frames** (page-sized slots of real RAM), and the CPU's **TLB** (*translation lookaside buffer*, a small hardware cache) caches recent translations.



- Memory is **allocated lazily**: reserving address space (`-Xmx`, `mmap`) costs nothing until a page is touched; the first touch triggers a **page fault** and the kernel supplies a frame. That is why a process's **virtual size (VSZ)** is far larger than its **resident set size (RSS)**, the RAM actually used. `-XX:+AlwaysPreTouch` touches the whole heap at start-up to avoid faults later.
- When RAM runs short the kernel drops clean file pages, writes **dirty** ones (modified since they were read) back, and **swaps anonymous pages** (memory not backed by any file, such as the heap) out to a disk area called *swap*; a **major fault** then costs milliseconds instead of nanoseconds. A swapping JVM has terrible GC pauses, so servers usually run with little or no swap.
- Out of memory entirely, Linux's **OOM killer** picks a victim and sends `SIGKILL` (uncatchable, immediate) — the JVM dies with no `OutOfMemoryError` and no stack trace. In containers the cgroup memory limit applies to the *whole* process (heap + metaspace + thread stacks + direct buffers, 16.3), so size `-Xmx` (or `-XX:MaxRAMPercentage`) well below it.
- **Segmentation** (variable-size segments) is the older scheme; modern x86-64 and ARM OSes use flat paging, and "segmentation fault" today just means touching an unmapped or protected page.

**Memory hierarchy — rough latencies** (orders of magnitude; vary by hardware):

| Level | Latency | Size |
|---|---|---|
| CPU register / L1 cache | ~1 ns | tens of KiB per core |
| L2 / L3 cache | ~4–40 ns | MiB |
| Main memory (DRAM) | ~100 ns | GiB |
| NVMe SSD read | ~20–100 µs | TiB |
| Spinning disk seek | ~5–10 ms | TiB |
| Network round trip, same data centre / across a continent | ~0.5 ms / ~50–150 ms | — |

### System calls

Your code runs in **user mode** and cannot touch hardware. To open a file, send bytes, create a thread or allocate pages it makes a **system call** (`open`, `read`, `write`, `mmap` — map a file or memory into the address space — `clone` — create a thread/process — `futex` — the kernel wait/wake primitive behind locks — `epoll_wait` …): the CPU traps into **kernel mode**, the kernel checks permissions and does the work, and control returns. A syscall costs roughly 100 ns to a few µs — cheap once, expensive per byte. That is why buffering matters (13.6): 1,000 one-byte `write` calls are 1,000 syscalls; one buffered 1,000-byte write is one. On Linux, `strace -f -c java …` counts a program's syscalls (not run here: macOS needs `dtruss` with System Integrity Protection relaxed).

### File descriptors

A **file descriptor (fd)** is a small integer index into a process's table of open kernel objects — files, pipes, sockets, the epoll instance itself ("everything is a file"). 0, 1 and 2 are stdin, stdout and stderr (§13.8). Every `FileInputStream`, `FileChannel`, socket and database connection holds one until closed. The table has a per-process limit (`ulimit -n`, often 1,024 by default for shells; servers raise it to tens of thousands); leaked streams or a connection storm end in `IOException: Too many open files` (`EMFILE`). Use try-with-resources (Chapter 8) and watch the count (`ls /proc/<pid>/fd | wc -l` on Linux, or the JMX `OpenFileDescriptorCount`). File **permissions** (`rwx` for owner, group, others) are checked when the fd is opened, not on each read.

### I/O models: blocking, non-blocking, multiplexing, async

| Model | How a thread waits | Java | Scales to |
|---|---|---|---|
| **Blocking** | `read()` sleeps in the kernel until data arrives | `java.io` streams, `Socket` | one thread per connection — fine for hundreds, costly for 100k **platform** threads |
| **Non-blocking** | `read()` returns 0 immediately if nothing is there; the app must retry | `configureBlocking(false)` | useless alone (busy-waiting) |
| **Multiplexing** | one call waits for **readiness** on many fds: `select`/`poll` (O(n) per call), **`epoll`** (Linux) / **`kqueue`** (BSD, macOS) (O(ready)) | NIO `Selector` — the basis of Netty, Tomcat's NIO connector, Redis's **event loop** (one thread looping over "what's ready next?", SQL book §16.11) | one thread, tens of thousands of connections |
| **Asynchronous (completion)** | submit the operation; the kernel tells you when it is *done* | `AsynchronousFileChannel`; Linux `io_uring` (batched async I/O) via libraries | high-throughput storage and networking |
| **Virtual threads** | write blocking code; the JVM parks the virtual thread and uses the OS readiness APIs underneath | `Thread.ofVirtual()`, `Executors.newVirtualThreadPerTaskExecutor()` | blocking style at event-loop scale |

### Page cache and zero-copy

The kernel keeps recently read and written file data in otherwise unused RAM, the **page cache**. A `read()` of cached data is a memory copy; a miss goes to disk. A `write()` normally only dirties cached pages and returns — the kernel flushes them later — so data survives a power cut only after **`fsync`** (`FileChannel.force(true)`), which databases do at commit (SQL book's write-ahead log). This is also why "free" memory on a busy Linux server looks near zero (it's cache, released on demand) and why the second run of a file-heavy job is much faster. **Zero-copy** (`sendfile`, used by `FileChannel.transferTo`) sends file pages from the page cache straight to a socket without copying through the JVM — one reason Kafka is fast.

### Seeing it: `code/ch16/os/OsBasics.java`

```
process  pid=94997 parent=94956 cpus=14
thread   wrote sharedCounter -> main thread reads 42
child    says: child pid=95011 parent=94997 | exit code 3 | it could not see sharedCounter (separate address space)
fds      before=6  with 100 files open=108  after close=8
vm       page size=16384 B, mapped 1024 MiB, touched 1000 pages -> only those 15.6 MiB need physical RAM
nio      selector implementation: KQueueSelectorProvider
nio      one selector thread echoed to 200 of 200 clients
```

Reading it: a thread sees the variable another thread wrote; a child process (created by fork + exec) has its own pid, reports ours as its parent, and could not see our memory. Opening 100 channels added ~100 descriptors (the small leftovers are descriptors the JDK opened for its own use meanwhile, such as the pipes to the `getconf` child). Mapping 1 GiB was instant because pages are only backed when touched. On macOS the NIO selector is kqueue-based (it would print `EPollSelectorProvider` on Linux), and a single thread served 200 connections. Pids and counts change on every run.

**How the demo's program itself ran** — C and Go are compiled ahead of time to machine code; CPython interprets bytecode; the JVM interprets bytecode and **JIT-compiles** hot methods (16.8). **Endianness** (byte order of multi-byte numbers) also comes from the CPU: x86 and ARM are little-endian, network protocols are big-endian, and Java's `ByteBuffer` defaults to big-endian (`ByteOrder.nativeOrder()` tells you the CPU's).

### Common mistakes

- **Thread-per-request with thousands of platform threads** → memory for stacks and scheduler overhead; use a bounded pool, virtual threads, or an event loop.
- **Setting `-Xmx` equal to the container memory limit** → the OOM killer ends the JVM because non-heap memory has nowhere to go; leave 25% or more headroom.
- **Unbuffered small writes** → one syscall each; wrap streams in buffers.
- **Not closing streams, sockets or connections** → fd exhaustion under load.
- **Assuming a successful `write()` is on disk** → call `force()`/`fsync` when durability matters.
- **Treating `top`'s low "free" memory as a leak** → most of it is page cache; look at RSS and "available".

### In the interview

> **30-second answer — process vs thread:** "A process is a running program with its own virtual address space and file-descriptor table; a thread is an execution path inside a process that shares its memory. Processes are isolated and heavier to create and switch; threads are cheaper but need synchronisation. In Java each platform thread is an OS thread, while virtual threads are multiplexed by the JVM onto a few carrier threads."

- *What is a page fault?* → The CPU touched a virtual page with no valid mapping; the kernel maps a frame (minor) or loads it from disk (major), or kills the access (segfault).
- *epoll vs select?* → `select` re-scans every fd on each call (O(n), limited set size); `epoll` registers fds once and returns only the ready ones (O(ready)).
- *Why did my container JVM vanish without an `OutOfMemoryError`?* → The kernel's OOM killer sent `SIGKILL` because total process memory exceeded the cgroup limit.
- *What happens on a system call?* → Switch from user to kernel mode via a trap, the kernel validates and performs the operation, then returns — cheap but not free, so batch I/O.

### TL;DR

Processes own address spaces and fd tables; threads share them. The scheduler time-slices runnable threads (context switches aren't free; cgroup CPU limits throttle). Virtual memory = pages + page tables + TLB, allocated lazily on page faults; out of RAM → swap or OOM killer. Syscalls cross into the kernel (batch them); fds are per-process handles with a limit. I/O: blocking, non-blocking, multiplexed (epoll/kqueue — NIO, Netty, Redis), async (io_uring), or virtual threads. The page cache makes repeat reads fast, writes lazy (`fsync` for durability) and enables zero-copy.

Sources: Arpaci-Dusseau & Arpaci-Dusseau, *Operating Systems: Three Easy Pieces* (processes, scheduling, paging, TLBs, I/O); Linux man pages `fork(2)`, `mmap(2)`, `epoll(7)`, `select(2)`, `fsync(2)`, `proc(5)`, `getrlimit(2)`; Linux kernel docs (EEVDF scheduler, cgroup v2 memory/CPU); JDK 25 API docs (`ProcessHandle`, `Selector`, `FileChannel`, `ByteOrder`); JEP 444 (virtual threads).

---

## Chapter TL;DR

- **Class loading**: lazy load → link → initialise; parent-first delegation; CDS/AOT caches speed start-up.
- **Memory areas**: heap, stacks, metaspace, code cache, native memory — each with its own `OutOfMemoryError`.
- **Objects**: 12-byte header (8 with compact headers), 4-byte references, 8-byte alignment: `new Object()` = 16 B, `Point(int,int)` = 24 B.
- **Collectors**: Serial, Parallel, **G1 (default)**, ZGC/Shenandoah (sub-ms pauses). Here: Serial 130 ms, G1 ~9 ms, ZGC ~0.01 ms pauses.
- **Tuning**: measure, size from the live set, set a goal, one change at a time.
- **JIT**: inlining unlocks escape analysis (30 M `Point`s, 0 bytes). **Benchmarks**: use JMH.

- **Tools**: `jcmd`, `jstat`, **JFR**; heap dumps last. **Leaks**: heap-after-GC growth → histogram → `OldObjectSample` path.
- **JPMS**: `requires`/`exports`/`opens`; `jdeps`; `jlink` (44 MB runtime vs 377 MB JDK).
- **OS fundamentals**: processes own address spaces, threads share them; time-sliced scheduling; virtual memory is lazy (page faults, RSS ≪ VSZ, OOM killer); syscalls and fds are per-process; NIO `Selector` = epoll/kqueue; page cache + `fsync` + zero-copy.
- Next: Chapter 17 — Modern Java 17 to 25.

# Chapter 17 — Modern Java: 17 to 25

Java ships every six months, and a lot changed between the two **long-term-support** (LTS) releases — versions vendors patch for years, so most companies standardise on them — most teams run today: **Java 17** (September 2021) and **Java 25** (September 2025). This chapter is a **concise tour**: one table of everything finalised in each release, one tiny runnable example per final feature the rest of the book doesn't already teach, and an exact list of what is still *preview* or *incubating* — what you can and can't use in production today. Features taught elsewhere get a two-line reminder and a link: this chapter is the map, the linked sections the territory.

**How the code was run.** Programs are in `code/` (package `ch17.v17` … `ch17.v25`, one sub-package per release), compiled with `javac --release 25` **without** `--enable-preview`, and run on OpenJDK 26. Only **final** features appear in runnable code. Every JEP number and status was checked against the release pages on openjdk.org (`openjdk.org/projects/jdk/<N>/`) and the individual JEP pages.

---

## 17.1 Reading a feature's status

Each change to the JDK is described in a **JEP** (JDK Enhancement Proposal) — see Chapter 1 §1.12 for the release model. A JEP title tells you the maturity:

| **Title contains** | **Status** | **Can you use it in production?** |
|---|---|---|
| nothing (e.g. *JEP 444: Virtual Threads*) | **Final / permanent** | Yes |
| *(Preview)*, *(Second Preview)* … | **Preview**: complete, but may still change | Only with `--enable-preview` on both `javac` and `java`, for experiments |
| *(Incubator)* | **Incubating API** in a `jdk.incubator.*` module | Only with `--add-modules jdk.incubator.…`; API will change |
| *(Experimental)* | **Experimental VM feature** | Needs `-XX:+UnlockExperimentalVMOptions`; not for production |
| status *Closed / Withdrawn* | **Withdrawn** | No — removed before finalisation |

A feature usually goes preview → preview → … → final, and sometimes gets renamed along the way (e.g. *Statements before super(...)* became *Flexible Constructor Bodies*). This book marks every feature with the release that **finalised** it.

---

## 17.2 The version table

GA = general availability date. "Final" rows list what you can rely on; previews are in 17.11.

| Release | GA | Language and library — final | JVM, GC and tools — final |
|---|---|---|---|
| **17 LTS** | 2021-09-14 | Sealed classes (409), enhanced pseudo-random generators (356), always-strict floating point (306), context-specific deserialization filters (415) | Strong encapsulation of JDK internals (403), macOS/AArch64 port (391), new macOS rendering pipeline (382); experimental AOT/Graal JIT removed (410); RMI activation removed (407) |
| **18** | 2022-03-22 | UTF-8 by default (400), code snippets in Javadoc `@snippet` (413), internet-address resolution SPI (418) | Simple web server `jwebserver` (408), core reflection reimplemented with method handles (416); finalization deprecated for removal (421) |
| **19** | 2022-09-20 | *(no final language/API features)* | Linux/RISC-V port (422) |
| **20** | 2023-03-21 | *(no final features — previews and incubators only)* | — |
| **21 LTS** | 2023-09-19 | Record patterns (440), pattern matching for `switch` (441), virtual threads (444), sequenced collections (431), key encapsulation mechanism API (452) | Generational ZGC (439); warnings for dynamically loaded agents (451); Windows 32-bit x86 port deprecated (449) |
| **22** | 2024-03-19 | Unnamed variables and patterns (456), Foreign Function & Memory API (454), launch multi-file source programs (458) | Region pinning for G1 (423) |
| **23** | 2024-09-17 | Markdown documentation comments (467) | ZGC generational mode by default (474); `sun.misc.Unsafe` memory-access methods deprecated for removal (471) |
| **24** | 2025-03-18 | Stream gatherers (485), Class-File API (484), quantum-resistant ML-KEM (496) and ML-DSA (497) | Virtual threads synchronize without pinning (491), ahead-of-time class loading and linking (483), ZGC non-generational mode removed (490), Security Manager permanently disabled (486), linking run-time images without JMODs (493), late barrier expansion for G1 (475), JNI restriction warnings (472), `Unsafe` memory-access warnings (498), Windows 32-bit port removed (479), 32-bit x86 port deprecated (501) |
| **25 LTS** | 2025-09-16 | Compact source files and instance `main` methods (512), module import declarations (511), flexible constructor bodies (513), scoped values (506), key derivation function API (510) | Compact object headers (519, opt-in), generational Shenandoah (521), AOT command-line ergonomics (514) and method profiling (515), JFR cooperative sampling (518) and method timing and tracing (520); 32-bit x86 port removed (503) |

Numbers in parentheses are JEP numbers. Glosses for the less obvious entries: **sealed** classes restrict which classes may extend them; **deserialization filters** reject unwanted classes when turning bytes back into objects; **SPI** = *service provider interface*, a plug-in point; **method handles** are fast, typed references to methods; **finalization** is the old `finalize()` clean-up hook; a *port* is a build for another OS/CPU (**RISC-V** is an open CPU architecture, **x86** the Intel/AMD family); **agents** are JARs that instrument (modify) classes as they load; **ML-KEM/ML-DSA** are *quantum-resistant* algorithms meant to stay secure against future quantum computers; the **Security Manager** was an old in-process sandbox; **JMODs** are the JDK's module packaging files. In three words each: **21** = virtual threads + patterns, **22** = FFM + `_`, **24** = gatherers, **25** = simpler Java (compact source files, module imports, flexible constructors) + scoped values.

## 17.3 The Java 17 baseline

Teams moving from Java 11 got a whole set of features finalised in 14–17. They are taught in depth elsewhere:

| Feature | Final in | JEP | Where in this book |
|---|---|---|---|
| Switch expressions (`->`, `yield`) | 14 | 361 | Chapter 3 §3.3 |
| Helpful `NullPointerException` messages (on by default since 15) | 14 | 358 | below |
| Text blocks `"""` | 15 | 378 | Chapter 2 §2.12 |
| Records | 16 | 395 | Chapter 6 §6.8 |
| Pattern matching for `instanceof` | 16 | 394 | Chapter 3 §3.5 |
| `Stream.toList()` | 16 | — | Chapter 12 |
| `jpackage` (builds native installers such as `.msi`, `.dmg`, `.deb`) | 16 | 392 | — |
| Sealed classes and interfaces | 17 | 409 | Chapter 6 §6.7 |

All of them in one small program: `code/ch17/v17/Java17Baseline.java`

```java
sealed interface Shape permits Circle, Square {}                 // sealed classes: Java 17
record Circle(double r) implements Shape {}                      // records: Java 16
record Square(double side) implements Shape {}

static double area(Shape s) {
    if (s instanceof Circle c) return Math.PI * c.r() * c.r();   // pattern matching for instanceof: Java 16
    if (s instanceof Square q) return q.side() * q.side();
    throw new AssertionError();
}
static String kind(int day) {
    return switch (day) {                                         // switch expression: Java 14
        case 6, 7 -> "weekend";
        case 1, 2, 3, 4, 5 -> "weekday";
        default -> throw new IllegalArgumentException("day " + day);
    };
}
// main: a text block, then c.address().city().length() with a null address
```

Output:

```
text block : {"shape": "circle", "r": 1.5}
areas      : 7.07 4.00
switch     : weekend, weekday
record     : Circle[r=1.5]
NPE says   : Cannot invoke "ch17.v17.Java17Baseline$Address.city()" because the return value of
"ch17.v17.Java17Baseline$Customer.address()" is null
```

The **helpful NPE** message names exactly which part of a chain was `null` — no more guessing which of three calls on one line failed.

### Enhanced pseudo-random generators (JEP 356, Java 17)

**What it is:** a common interface `java.util.random.RandomGenerator` for every random source (old `Random`, `SplittableRandom`, `ThreadLocalRandom` and new *LXM* algorithms — mixing a linear congruential generator with an XOR-based one), selectable by name. (*Pseudo-random* = from a deterministic formula; the same **seed** gives the same sequence.) **Why it matters:** code can accept "any generator", and you can pick algorithms with better statistical quality or splitting support for parallel work. `code/ch17/v17/RandomGenerators.java`:

```java
RandomGenerator g = RandomGeneratorFactory.of("L64X128MixRandom").create(42); // seeded: repeatable
System.out.println("L64X128MixRandom dice: " + g.ints(8, 1, 7).boxed().toList());
RandomGenerator legacy = new java.util.Random(42);           // old classes implement the interface too
```

Output:

```
L64X128MixRandom dice: [1, 2, 1, 1, 4, 4, 6, 2]
java.util.Random dice: [3, 4, 1, 3, 1, 2, 6, 3]
default algorithm    : L32X64MixRandom
a few algorithms     : [L128X1024MixRandom, L128X128MixRandom, L128X256MixRandom, L32X64MixRandom,
L64X1024MixRandom, L64X128MixRandom]
```

For everyday use `ThreadLocalRandom.current()` or `RandomGenerator.getDefault()` is fine; for security use `SecureRandom` (unpredictable, suitable for passwords and keys; Chapter 7 §7.12).

### Strong encapsulation (JEP 403, Java 17)

The JDK's internal packages (`sun.*`, `jdk.internal.*`) are closed to reflection by default — the `--illegal-access` escape hatch was removed. Old libraries that poke at internals fail with `InaccessibleObjectException` unless you add `--add-opens`. This is the #1 upgrade issue from Java 8/11; see Chapter 16 §16.12.

---

## 17.4 Java 18

### UTF-8 by default (JEP 400)

**The problem it solved:** before 18, APIs such as `FileReader`, `FileWriter`, `InputStreamReader`, `PrintStream` and `String.getBytes()` used the **platform** charset — UTF-8 on Linux/macOS but often Windows-1252 on Windows — so a file written on one machine could be garbled on another. Since 18 the default is **UTF-8 everywhere**; the OS encoding is still available as the `native.encoding` property. (Console output uses `stdout.encoding`, which still follows the terminal.) Full treatment: §13.7.

`code/ch17/v18/Utf8Default.java`:

```java
System.out.println("Charset.defaultCharset(): " + Charset.defaultCharset());
System.out.println("native.encoding (OS)    : " + System.getProperty("native.encoding"));
Files.writeString(tmp, "café ☕");                           // no charset argument needed
try (var reader = new java.io.FileReader(tmp.toFile())) { ... }   // FileReader also uses UTF-8 now
```

Output (macOS — on Windows `native.encoding` would typically be `Cp1252`, but the first two lines would still say UTF-8):

```
Charset.defaultCharset(): UTF-8
file.encoding           : UTF-8
native.encoding (OS)    : UTF-8
read back               : café ☕ (9 bytes)
```

9 bytes for 6 characters: `é` takes 2 bytes and `☕` 3 in UTF-8. Still best practice: pass a charset explicitly when reading data you did not write.

### Simple web server (JEP 408)

`jwebserver` serves the current directory as **static files** over HTTP — handy for testing HTML or sharing files on a LAN, **not** for production (HTTP/1.1 only, no HTTPS). Run `jwebserver -p 8000` (`-p` = port) then open `http://localhost:8000/` (not run here — the book's programs don't use the network). The same server is available as an API: `com.sun.net.httpserver.SimpleFileServer`.

### Code snippets in Javadoc (JEP 413)

`{@snippet : …}` replaces `<pre>{@code …}</pre>` for examples in Javadoc; it supports highlighting, regions and snippets pulled from real, compiled source files — so documentation examples can be tested.

### Finalization deprecated for removal (JEP 421)

`Object.finalize()` still works but is *deprecated for removal* (discouraged and scheduled for deletion); use try-with-resources and `java.lang.ref.Cleaner` (registers clean-up code to run after an object becomes unreachable) instead (Chapter 8).

## 17.5 Java 19 and 20: the preview years

Neither release finalised a language or library feature. They matter because the big Java 21 features were previewed here:

| JEP | Feature in 19 | JEP | Feature in 20 |
|---|---|---|---|
| 425 | Virtual Threads (Preview) | 436 | Virtual Threads (Second Preview) |
| 405 | Record Patterns (Preview) | 432 | Record Patterns (Second Preview) |
| 427 | Pattern Matching for switch (Third Preview) | 433 | Pattern Matching for switch (Fourth Preview) |
| 424 | Foreign Function & Memory API (Preview) | 434 | Foreign Function & Memory API (Second Preview) |
| 428 | Structured Concurrency (Incubator) | 437 | Structured Concurrency (Second Incubator) |
| 426 | Vector API (Fourth Incubator) | 438 | Vector API (Fifth Incubator) |
| — | — | 429 | Scoped Values (Incubator) |

Java 19 also added the Linux/RISC-V port (JEP 422). If you are on 17 and skipping to 21 or 25, you can ignore 19 and 20 entirely.

---

## 17.6 Java 21 LTS

### Pattern matching for `switch` and record patterns (JEPs 441, 440)

`switch` can match on **types** and **deconstruct records** (pull components into variables), with `when` **guards**, explicit `case null`, and **exhaustiveness** checking over sealed hierarchies — no `default` needed. Full treatment: Chapter 3 §3.6 and §3.7. `code/ch17/v21/PatternsDemo.java` evaluates a small expression tree:

```java
sealed interface Expr permits Num, Add, Mul, Neg {}
record Num(int value) implements Expr {}
record Add(Expr left, Expr right) implements Expr {}
record Mul(Expr left, Expr right) implements Expr {}
record Neg(Expr e) implements Expr {}

static int eval(Expr e) {
    return switch (e) {                                          // exhaustive: no default needed (sealed)
        case Num(int v) -> v;
        case Add(Expr l, Expr r) -> eval(l) + eval(r);
        case Mul(Num(int a), Num(int b)) when a == 0 || b == 0 -> 0;   // nested pattern + guard
        case Mul(Expr l, Expr r) -> eval(l) * eval(r);
        case Neg(Neg(Expr inner)) -> eval(inner);                // double negation
        case Neg(Expr inner) -> -eval(inner);
    };
}
static String describe(Object o) {
    return switch (o) {
        case null -> "null!";
        case Integer i when i > 100 -> "big int " + i;
        case Integer i -> "int " + i;
        case String s -> "string of length " + s.length();
        default -> "something else: " + o.getClass().getSimpleName();
    };
}
```

Output:

```
Add[left=Num[value=2], right=Mul[left=Num[value=3], right=Neg[e=Neg[e=Num[value=4]]]]] = 14
int 7
big int 700
string of length 3
something else: Double
null!
```

If you add a new `Expr` subtype, every `switch` like `eval` stops compiling until you handle it — the compiler finds the places to update. This "data-oriented" style (sealed types + records + pattern switch) is the modern alternative to the Visitor pattern (LLD handbook, behavioural patterns).

### Virtual threads (JEP 444)

**Virtual threads** are lightweight JVM-managed threads: you can have millions, and a **blocking** call (sleep, socket read, JDBC query) unmounts the virtual thread from its **carrier** OS thread instead of blocking it. They make the simple **thread-per-request** style scale for **I/O-bound** servers. Full coverage, including pinning and when they *don't* help: Chapter 15. `code/ch17/v21/VirtualThreadsDemo.java`:

```java
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {   // close() waits for all tasks
    for (int i = 0; i < 10_000; i++) {
        executor.submit(() -> {
            Thread.sleep(Duration.ofMillis(200));            // simulated blocking I/O
            done.incrementAndGet();
            return null;
        });
    }
}
```

Output:

```
tasks finished: 10000
took < 2 s    : true   (10,000 x 200 ms sequentially would be 2,000 s)
is virtual    : true
```

(The program prints a yes/no instead of the elapsed time, which varies by machine; on the author's machine it was a few hundred milliseconds.)

### Sequenced collections (JEP 431)

New interfaces `SequencedCollection`, `SequencedSet` and `SequencedMap` give every ordered collection `getFirst`, `getLast`, `addFirst`, `addLast`, `removeFirst/Last` and `reversed()` (maps: `firstEntry`, `lastEntry`, `pollFirstEntry`, `putFirst`, …). Full coverage: Chapter 10 §10.11. `code/ch17/v21/SequencedDemo.java`:

```java
List<String> list = new ArrayList<>(List.of("b", "c"));
list.addFirst("a"); list.addLast("d");
LinkedHashSet<Integer> set = new LinkedHashSet<>(List.of(3, 1, 2));
set.addFirst(2);                                             // moves an existing element to the front
SequencedMap<String, Integer> map = new LinkedHashMap<>();
map.put("x", 1); map.put("y", 2); map.putFirst("w", 0);
```

Output:

```
[a, b, c, d] first=a last=d reversed=[d, c, b, a]
[2, 3, 1] reversed=[1, 3, 2]
{w=0, x=1, y=2} firstEntry=w=0 pollLast=y=2 -> {w=0, x=1}
```

### Also final in 21

- **Generational ZGC** (JEP 439) — opt-in with `-XX:+UseZGC -XX:+ZGenerational` in 21; default mode from 23, the only mode from 24 (Chapter 16 §16.5).
- **Key Encapsulation Mechanism API** (JEP 452) — `javax.crypto.KEM`, the building block for hybrid and **post-quantum** key exchange.
- **Warnings for dynamically loaded agents** (JEP 451) — attaching a Java agent to a running JVM now prints a warning; tools like Mockito's inline mock maker (Chapter 18) should be loaded with `-javaagent` at

start-up.

---

## 17.7 Java 22

### Unnamed variables and patterns (JEP 456)

Use `_` for anything you must declare but never read: loop variables, `catch` parameters, lambda parameters, try-with-resources variables, and parts of record patterns. It documents intent and silences "unused" warnings. `code/ch17/v22/UnnamedDemo.java`:

```
for (var _ : List.of("a", "b", "c")) count++;                 // loop variable not needed
catch (NumberFormatException _) { ... }                       // exception object not needed
m.forEach((k, _) -> System.out.print(k + " "));               // lambda parameter not needed
if (o instanceof Line(Point(var x1, _), Point(var x2, _)))    // ignore the y components
    System.out.println("horizontal distance = " + Math.abs(x2 - x1));
String kind = switch (o) {
    case Line(Point _, Point(int x, int y)) when x == y -> "ends on the diagonal";
    case Line _ -> "a line";                                  // unnamed pattern variable
    default -> "other";
};
```

Output:

```
count = 3
not a number
x y
horizontal distance = 3
a line
```

`_` has been a reserved keyword since Java 9, so no valid old code breaks. You can use several `_` in the same scope.

### Foreign Function & Memory API (JEP 454)

**What it is:** a pure-Java way to call **native** (C) libraries and to work with *off-heap* memory — the safe, fast replacement for JNI (the Java Native Interface, which needed hand-written C glue) and for `sun.misc.Unsafe` memory access. **Key types:** `Linker` (makes a Java `MethodHandle` for a C function), `FunctionDescriptor` (the C signature), `Arena` (owns off-heap memory and frees it when closed), `MemorySegment` (a bounded, checked region of memory), `ValueLayout` (C types like `int`, `long`, pointer).

`code/ch17/v22/ForeignStrlen.java` calls the C library's `strlen`:

```
Linker linker = Linker.nativeLinker();
MethodHandle strlen = linker.downcallHandle(
        linker.defaultLookup().find("strlen").orElseThrow(),
        FunctionDescriptor.of(ValueLayout.JAVA_LONG, ValueLayout.ADDRESS));   // size_t strlen(const char*)

try (Arena arena = Arena.ofConfined()) {                     // off-heap memory freed at the end of the block
    MemorySegment cString = arena.allocateFrom("Hello, native world!");     // UTF-8, NUL-terminated
    long len = (long) strlen.invokeExact(cString);
    MemorySegment ints = arena.allocate(ValueLayout.JAVA_INT, 5);            // like int[5] in C
    for (int i = 0; i < 5; i++) ints.setAtIndex(ValueLayout.JAVA_INT, i, i * i);
}
```

Output of `java -cp out ch17.v22.ForeignStrlen` (warnings go to standard error; path shortened):

```
WARNING: A restricted method in java.lang.foreign.Linker has been called
WARNING: java.lang.foreign.Linker::downcallHandle has been called by ch17.v22.ForeignStrlen in an unnamed module
(file:.../out17/)
WARNING: Use --enable-native-access=ALL-UNNAMED to avoid a warning for callers in this module
WARNING: Restricted methods will be blocked in a future release unless native access is enabled

strlen = 20
off-heap ints: [0, 1, 4, 9, 16], 20 bytes
```

Calling native code can crash the JVM, so such methods are **restricted**: grant access explicitly with `--enable-native-access=ALL-UNNAMED` (or a module name). Real-world users: Apache Lucene (the search library behind Elasticsearch; memory-mapped index files), database drivers, and **bindings** (ready-made Java wrappers for a C library) generated by the **jextract** tool from C header files.

### Launch multi-file source-code programs (JEP 458)

`java Main.java` now compiles and runs a program spread over several source files in a directory tree — no build step for small tools and learning. See Chapter 1 §1.10.

### Region pinning for G1 (JEP 423)

Native code holding a pointer into a Java array (a JNI "critical region") no longer blocks G1 garbage collection; G1 just avoids moving that one region.

---

## 17.8 Java 23

### Markdown documentation comments (JEP 467)

Javadoc comments can be written in **Markdown** using `///` lines instead of HTML inside `/** … */`. `code/ch17/v23/MarkdownDoc.java`:

```java
/// JEP 467 (Java 23): documentation comments written in **Markdown**.
///
/// Lines start with `///` instead of the `/** ... */` block, and use Markdown instead of HTML:
///
/// - lists like this one
/// - `inline code` and **bold**
/// - links to program elements: [String#strip()]
///
/// @param <T> the element type (block tags such as `@param` still work)
public class MarkdownDoc<T> {
```

`javadoc -d docs MarkdownDoc.java` turns it into this HTML (real output, the class description block):

```html
<p>JEP 467 (Java 23): documentation comments written in <strong>Markdown</strong>.</p>
<p>Lines start with <code>///</code> instead of the <code>/** ... */</code> block, and use Markdown instead of HTML:</p>
<ul>
<li>lists like this one</li>
<li><code>inline code</code> and <strong>bold</strong></li>
<li>links to program elements: <a href="https://docs.oracle.com/en/java/javase/26/docs/api/java.base/java/lang/String.html#strip()" class="external-link"><code>String.strip()</code></a></li>
</ul>
```

`[String#strip()]` became a link to the JDK's API page — reference links to program elements are the Markdown replacement for `{@link …}`.

### Also in 23

- **ZGC generational mode by default** (JEP 474).
- **`sun.misc.Unsafe` memory-access methods deprecated for removal** (JEP 471) — the FFM API and `VarHandle` (a typed handle for reading/writing a variable with chosen memory-ordering guarantees) are the replacements.
- **String templates withdrawn.** They were previewed in 21 (JEP 430) and 22 (JEP 459); a third preview (JEP 465) was proposed for 23 but **withdrawn**, and the feature was removed from JDK 23. The designers decided the syntax (`STR."Hello \{name}"`) and processor model needed rethinking. There is no replacement yet — use `String.formatted`, `+`, `StringBuilder` or `MessageFormat` (Chapter 7 §7.4). Don't trust tutorials from 2023–24 that show `STR.` templates: that code doesn't compile on any current JDK.

---

## 17.9 Java 24

### Stream gatherers (JEP 485)

`Stream.gather(Gatherer)` adds **custom intermediate operations** — the missing counterpart to custom collectors. Built-ins in `java.util.stream.Gatherers`: `windowFixed`, `windowSliding`, `fold`, `scan`, `mapConcurrent`. Full coverage, including writing your own: Chapter 12 §12.9. `code/ch17/v24/GatherersDemo.java`:

```
Stream.of(1, 2, 3, 4, 5, 6, 7).gather(Gatherers.windowFixed(3)).toList()
Stream.of(1, 2, 3, 4, 5).gather(Gatherers.windowSliding(2)).toList()
Stream.of(1, 2, 3, 4, 5).gather(Gatherers.scan(() -> 0, Integer::sum)).toList()
Stream.of("a", "b", "c").gather(Gatherers.fold(() -> "", String::concat)).toList()
```

Output:

```
[[1, 2, 3], [4, 5, 6], [7]]
[[1, 2], [2, 3], [3, 4], [4, 5]]
[1, 3, 6, 10, 15]
[abc]
```

### Class-File API (JEP 484)

**What it is:** a standard API (`java.lang.classfile`) for reading, writing and transforming `.class` files. **Why:** frameworks and tools (Spring, Hibernate, Mockito, the JDK itself) generate or rewrite bytecode, traditionally with the third-party ASM library, which always lags behind new class-file versions. The JDK's own API is updated in lock-step with each release. `code/ch17/v24/ClassFileDemo.java` parses its own class file:

```
ClassModel cm = ClassFile.of().parse(bytes);
for (MethodModel m : cm.methods()) {
    for (CodeElement e : m.code().get()) {                // walk the bytecode as typed objects
        if (e instanceof Instruction) instructions++;
        if (e instanceof InvokeInstruction inv) calls.add(inv.name().stringValue());
    }
}
```

Output:

```
class   : ch17/v24/ClassFileDemo  (class-file major version 69)
method  : <init>   ()V                        3 instructions, 1 calls [<init>]
method  : square   (I)I                       4 instructions, 0 calls []
method  : main     ([Ljava/lang/String;)V   133 instructions, 36 calls
square(7) = 49
```

Major version 69 = Java 25 (Chapter 1 §1.6). Note how pattern matching (`instanceof InvokeInstruction inv`) makes walking a model natural. Application developers rarely need this API directly.

### Virtual threads without pinning (JEP 491)

Before 24, a virtual thread that blocked **inside a `synchronized` block** stayed *pinned* to its carrier OS thread, which could starve the scheduler of free carriers — the standard advice was "replace `synchronized` with `ReentrantLock`". Since 24, virtual threads can release their carrier while holding or waiting for a **monitor** (the lock behind `synchronized`), so that advice is largely obsolete. (Pinning still happens in native frames.) Details: Chapter 15.

### Also final in 24

- **Ahead-of-time class loading and linking** (JEP 483): record a **training run** into an AOT cache, start faster in production (Chapter 1 §1.7).
- **Security Manager permanently disabled** (JEP 486): `System.setSecurityManager` throws; `-Djava.security.manager` is an error.

- **Post-quantum cryptography**: ML-KEM key encapsulation (JEP 496) and ML-DSA signatures (JEP 497), standardised by NIST in FIPS 203/204.
- **ZGC non-generational mode removed** (JEP 490), **late barrier expansion for G1** (JEP 475, a JIT-internal simplification of GC barrier code), **linking run-time images without JMODs** (JEP 493, lets `jlink` work on JDKs shipped without the `jmods` folder).
- Integrity warnings: **JNI** use prints a warning unless enabled (JEP 472); **`Unsafe` memory access** prints a warning on first use (JEP 498).

---

## 17.10 Java 25 LTS

### Compact source files and instance `main` methods (JEP 512)

A source file can now contain just methods and fields — the class around them is implicit — and `main` can be an instance method without `String[] args`. A new class `java.lang.IO` offers `IO.println` and `IO.readln`, and compact source files automatically import all of `java.base`. This is covered with runnable examples in Chapter 1 §1.10 and Chapter 4 §4.9:

```
void main() {
    IO.println("Hello, Java 25");
}
```

It was previewed four times under different names: *Unnamed Classes and Instance Main Methods* (21, JEP 445), *Implicitly Declared Classes…* (22 JEP 463, 23 JEP 477), *Simple Source Files…* (24, JEP 495).

### Module import declarations (JEP 511)

`import module M;` imports every public top-level type in every package that module `M` exports (and that it transitively reads through `requires transitive`). It saves long import lists in scripts, prototypes and teaching code; ordinary code can use it too. `code/ch17/v25/ModuleImportDemo.java`:

```
package ch17.v25;

import module java.base;           // JEP 511 (Java 25): imports every package exported by java.base

public class ModuleImportDemo {
    public static void main(String[] args) {
        List<String> words = List.of("modules", "import", "java", "base");
        Map<Integer, List<String>> byLength = words.stream().collect(Collectors.groupingBy(String::length));
        IO.println("by length : " + new TreeMap<>(byLength));                 // java.lang.IO: Java 25 (JEP 512)
        IO.println("a path    : " + Path.of("docs", "ch17.md"));
        IO.println("a date    : " + LocalDate.of(2025, 9, 16).getDayOfWeek()); // Java 25 GA day
    }
}
```

Output:

```
by length : {4=[java, base], 6=[import], 7=[modules]}
a path    : docs/ch17.md
a date    : TUESDAY
```

**Trade-off:** ambiguity. Add `import module java.desktop;` next to `java.base` and a plain `List` becomes ambiguous (`java.util.List` or `java.awt.List`?) — fix it with a normal single-type import (`import java.util.List;`), which always wins over module imports. The importing class does not have to be in a named module: this demo lives on the class path.

### Flexible constructor bodies (JEP 513)

Statements may now appear **before** `super(…)` or `this(…)`, as long as they don't use the object being constructed (except to assign its own fields) — validation, computing arguments, early field assignment. **The problem it solves:** validating or computing arguments used to need a static helper or wasted superclass work; and a superclass constructor calling an overridden method saw the subclass's fields still at default values. `code/ch17/v25/FlexibleConstructors.java`:

```java
static class Account {
    final String id;
    Account(String id) { this.id = id; describe(); }
    void describe() { System.out.println("  Account(" + id + ")"); }
}
static class SavingsAccount extends Account {
    final double rate;
    SavingsAccount(String id, double rate) {
        if (rate < 0 || rate > 0.2)                                // validate BEFORE the superclass runs
            throw new IllegalArgumentException("bad rate " + rate);
        this.rate = rate;                                          // allowed: initialise own field early
        super(id.strip().toUpperCase());                           // prepare the argument, then call super
    }
    @Override void describe() {                                   // called from Account's constructor:
        System.out.println("  SavingsAccount(" + id + ", rate=" + rate + ")");   // rate is already set
    }
}
```

Output:

```
  SavingsAccount(SA-1, rate=0.03)
  rejected before any superclass work: bad rate 5.0
```

Before Java 25 the overridden `describe()` would have printed `rate=0.0` (the field wasn't assigned yet). Calling overridable methods from constructors is still a design smell (Chapter 6); this feature makes it safe where it can't be avoided. Not allowed in the **prologue** (the code before `super(…)`): reading fields, calling instance methods, using `this` as a value.

### Scoped values (JEP 506)

A `ScopedValue` passes read-only context (the current user, a request ID, a transaction) to code deep in a call chain **without** method parameters — like a `ThreadLocal`, but immutable, automatically unbound when the scope ends (no leaks on thread pools), and cheap to inherit into child threads in **structured concurrency** (still preview, §17.11). Details and comparison with `ThreadLocal`: Chapter 15. `code/ch17/v25/ScopedValueDemo.java`:

```java
static final ScopedValue<String> USER = ScopedValue.newInstance();

static void handle() { audit("read order 17"); }                  // no user parameter threaded through
static void audit(String action) {
    System.out.println("  audit: " + (USER.isBound() ? USER.get() : "<anonymous>") + " -> " + action);
}
// main:
ScopedValue.where(USER, "ana").run(ScopedValueDemo::handle); // bound only while run() executes
ScopedValue.where(USER, "bo").run(() -> {
    handle();
    ScopedValue.where(USER, "admin").run(ScopedValueDemo::handle);   // rebinding for a nested scope
    handle();                                                       // back to "bo"
});
handle();                                                            // unbound again
```

Output:

```
  audit: ana -> read order 17
  audit: bo -> read order 17
  audit: admin -> read order 17
  audit: bo -> read order 17
  audit: <anonymous> -> read order 17
bound outside? false
```

### Key derivation function API (JEP 510)

`javax.crypto.KDF` derives **keys** from a secret, e.g. with **HKDF** (HMAC-based Key Derivation Function, used by TLS 1.3 and many protocols). `code/ch17/v25/HkdfDemo.java` checks the JDK against test case 1 of RFC 5869, the HKDF standard:

```
KDF hkdf = KDF.getInstance("HKDF-SHA256");
byte[] okm = hkdf.deriveData(HKDFParameterSpec.ofExtract()
        .addIKM(ikm).addSalt(salt).thenExpand(info, 42));    // extract-then-expand, 42 bytes out
```

Output:

```
OKM      = 3cb25f25faacd57a90434f64d0362f2a2d2d0a90cf1a5a4c5db02d56ecc4c5bf34007208d5b887185865
RFC 5869 = 3cb25f25faacd57a90434f64d0362f2a2d2d0a90cf1a5a4c5db02d56ecc4c5bf34007208d5b887185865
match    : true
```

Use `deriveKey("AES", spec)` to get a `SecretKey` instead of raw bytes.

### JVM and tooling in 25

| JEP | Feature | One line |
|---|---|---|
| 519 | Compact object headers | 8-byte object headers with `-XX:+UseCompactObjectHeaders`: smaller heaps (Chapter 16 §16.4) |
| 521 | Generational Shenandoah | `-XX:+UseShenandoahGC -XX:ShenandoahGCMode=generational`, now a product feature |
| 514 | AOT command-line ergonomics | One-step AOT cache creation: `-XX:AOTCacheOutput=app.aot` |
| 515 | AOT method profiling | The AOT cache also stores method profiles, so the JIT warms up faster |
| 518 | JFR cooperative sampling | Safer, more robust stack sampling in Flight Recorder |
| 520 | JFR method timing and tracing | Time or trace chosen methods via JFR without changing code |
| 503 | Remove the 32-bit x86 port | Only 64-bit x86 remains |

## 17.11 Still preview or incubating in Java 25 — and what came next

These are **not** final in Java 25. Don't use them in production code; the book has no runnable examples of them. Each row shows every round up to JDK 27 (GA 15 September 2026).

| Feature | History (JEP numbers) | Status in 25 | After 25 |
|---|---|---|---|
| **Structured concurrency** — treat a group of subtasks as one unit (fail together, cancel together) | Incubator 19 (428), 20 (437); preview 21 (453), 22 (462), 23 (480), 24 (499) | **Fifth preview** (505) | Sixth preview in 26 (525), seventh in 27 (533) |
| **Primitive types in patterns, `instanceof` and `switch`** — e.g. `case int i when i > 0`, `x instanceof byte b` | Preview 23 (455), 24 (488) | **Third preview** (507) | Fourth preview in 26 (530), fifth in 27 (532) |
| **Stable values** — lazily initialised constants the JIT can treat as `final` | — | **Preview** (502) | Renamed **Lazy Constants**: second preview in 26 (526), third in 27 (531) |
| **PEM encodings of cryptographic objects** — read/write keys and certificates as PEM text (the `-----BEGIN …-----` Base64 format) | — | **Preview** (470) | Second preview in 26 (524), third in 27 (538) |
| **Vector API** — portable SIMD computations (`jdk.incubator.vector`) | Incubating since 16; 17 (414) … 24 (489) | **Tenth incubator** (508) | 11th incubator in 26 (529), 12th in 27 (537) — waiting for Project Valhalla's value classes (objects without identity, stored as compactly as primitives) |
| **JFR CPU-time profiling** (Linux only) | — | **Experimental** (509) | — |

To try a preview on JDK 25: `javac --release 25 --enable-preview Foo.java` and `java --enable-preview Foo`; single-file launch: `java --enable-preview --source 25 Foo.java`.

**What JDK 26 (GA 17 March 2026) and JDK 27 (GA 15 September 2026) finalised** — both non-LTS, noted briefly:

- **26**: HTTP/3 (over the UDP-based QUIC protocol) for the HTTP Client API (517), AOT object caching with any GC (516), G1 throughput improvement by reducing synchronization (522), *Prepare to Make Final Mean Final* (500 — warnings when deep reflection mutates `final` fields), Applet API removed (504).
- **27**: G1 the default collector in all environments (523), compact object headers on by default (534), post-quantum hybrid key exchange for TLS 1.3 (527), JFR in-process data redaction (536).

The next LTS is expected to be **Java 29** (September 2027).

---

## 17.12 Upgrading 17 → 21 → 25: removals and deprecations

| Change | JEP, release | What breaks / what to do |
|---|---|---|
| JDK internals strongly encapsulated | 403, 17 | Reflection into `sun.*`/`java.*` private members fails → upgrade the library, or `--add-opens` temporarily |
| Security Manager deprecated → permanently disabled | 411 (17) → 486 (24) | Code calling `System.setSecurityManager` fails; remove it |
| Finalization deprecated for removal | 421, 18 | Replace `finalize()` with try-with-resources / `Cleaner` |
| Applet API deprecated → removed | 398 (17) → 504 (26) | Nothing for server code |
| Dynamic agent loading warns | 451, 21 | Load agents with `-javaagent` at start-up (e.g. Mockito in tests: add the agent to the test JVM) |
| `sun.misc.Unsafe` memory access deprecated → warns | 471 (23) → 498 (24) | Upgrade libraries (Netty, older serializers) to versions using FFM/`VarHandle` |
| JNI use warns | 472, 24 | Add `--enable-native-access=…` for libraries that use native code |
| ZGC non-generational mode removed | 490, 24 | Drop `-XX:-ZGenerational` |
| 32-bit ports removed | 479 (24, Windows x86), 503 (25, x86) | Use 64-bit JDKs |
| String templates withdrawn | 465, 23 | Code written against the 21/22 preview must be rewritten |

**A practical upgrade checklist**

1. Run the test suite on the new JDK with `--release` still at the old version, then raise `--release`.
2. Upgrade build plugins (Chapter 18) and bytecode-heavy libraries first (Lombok, Mockito, Byte Buddy, Spring, Hibernate): they must understand the new class-file version.
3. Run `jdeps --jdk-internals your.jar` to find uses of JDK internals and `jdeprscan your.jar` to find deprecated APIs.
4. Watch the startup warnings (agents, JNI, `Unsafe`) — they become errors in future releases.
5. Re-check GC and memory settings; defaults changed (generational ZGC, compact headers in 27).

---

## 17.13 In the interview

**Q: What changed in Java between 17 and 21, and between 21 and 25?** 17 → 21: pattern matching for `switch` and record patterns (with sealed types, a new data-oriented style), **virtual threads**, sequenced collections, generational ZGC. 21 → 25: unnamed `_` variables, the Foreign Function & Memory API, stream gatherers, the Class-File API, virtual threads no longer pinned by `synchronized`, and in 25 itself compact source files with instance `main`, module import declarations, flexible constructor bodies, scoped values and AOT start-up improvements.

- *Is structured concurrency final?* No — it is still in preview (fifth preview in 25, seventh in 27).
- *What happened to string templates?* Previewed in 21 and 22, withdrawn in 23; no replacement yet.
- *Why upgrade from 21 to 25?* LTS support horizon, no `synchronized` pinning with virtual threads, faster start-up with AOT caches, smaller heaps with compact object headers, and the new language conveniences.
- *Can you use a preview feature in production?* Technically with `--enable-preview`, but it may change or disappear in the next release and class files compiled with previews only run on that exact release — so no.

## Chapter TL;DR

- Status words matter: **final** = use it; **preview/incubator/experimental** = experiments only; **withdrawn** = gone. Check the JEP title on openjdk.org.
- **17**: sealed classes, new random generators, strong encapsulation (plus records, text blocks, `switch` expressions, `instanceof` patterns from 14–16).
- **18**: UTF-8 by default, `jwebserver`, `@snippet`. **19–20**: previews only.
- **21 LTS**: pattern matching for `switch`, record patterns, **virtual threads**, sequenced collections, generational ZGC.
- **22**: `_` unnamed variables and patterns, **FFM API**, multi-file source launch. **23**: Markdown doc comments; string templates withdrawn.
- **24**: **stream gatherers**, Class-File API, virtual threads without `synchronized` pinning, AOT class loading, Security Manager disabled, ML-KEM/ML-DSA.
- **25 LTS**: compact source files + instance `main` + `IO`, `import module`, flexible constructor bodies, **scoped values**, KDF API, compact object headers (opt-in), generational Shenandoah.
- Still preview in 25: structured concurrency, primitive patterns, stable values (→ lazy constants), PEM; Vector API incubating. JDK 26/27 finalised HTTP/3, G1-everywhere and compact headers by default.
- Next: Chapter 18 — Testing, build and tooling.

# Chapter 18 — Testing, Build and Tooling

Everything so far was compiled with a bare `javac` and run with `java`. Real projects add four things: a **build tool** that downloads **libraries** (other people's code, shipped as JARs) and runs every step (compile, test, package) in order, **automated tests** that prove the code still works after each change, **logging** that tells you what a running program did, and a **package** (a JAR, §18.12) you can ship. This chapter teaches all four on one small shop application, plus the JDBC (Java Database Connectivity) basics for talking to a database, and ends with a map of where to go next (Spring Boot, the most popular Java framework for services). **How the code was run.** All sources are in `code/`:

| Folder | What | How it runs |
|---|---|---|
| `code/ch18/shop/` | the application: `Cart`, `Discounts`, `OrderService`, … and `ShopDemo` (has `main`) | the book's runner (`javac --release 25`, JDK 26) |
| `code/test/ch18/shop/` | JUnit tests (marked `// tools_run: skip` — they have no `main`) | `code/run_tests.sh` |
| `code/ch18/logging/` | `JulDemo` | the book's runner |
| `code/ch18/jdbc/` | `JdbcOrderRepository`, `JdbcDemo` (skip-marked: needs a database) | by hand against the SQL Handbook's private PostgreSQL (a popular open-source relational database) (18.11) |
| `code/build-files/` | `pom.xml`, `build.gradle.kts`, `settings.gradle.kts` | Maven 3.9.14 and Gradle 9.4.1 **offline**, from the local artifact cache (18.2, 18.3) |
| `code/package_jar.sh` | builds and runs a JAR with the `jar` tool | by hand (18.12) |

Test libraries were downloaded from Maven Central (`repo1.maven.org`, SHA-1 checksum verified) into `ROOT/.java-books/lib/`: **JUnit 6.1.3** (`junit-platform-console-standalone`, the latest release at the time of writing), **Mockito 5.24.0** (`mockito-core`, `mockito-junit-jupiter`) with **Byte Buddy 1.18.14** (`byte-buddy`, `byte-buddy-agent`) and **Objenesis 3.3**, and **AssertJ 3.27.7**. The Maven and Gradle builds used the versions already in the local cache (JUnit 6.0.3, Mockito 5.23.0) because they ran without network. Every output below is real.

---

## 18.1 The shape of a real Java project

### What it is

A Java project is more than `.java` files. It has **dependencies** (other people's JARs that your code needs), **tests**, **resources** (config files, templates), and **build outputs** (compiled classes, the final JAR). Both mainstream build tools agree on one **standard directory layout**, so any Java developer can open any project:

```
shop/
├── pom.xml                  (Maven)  or  build.gradle.kts + settings.gradle.kts (Gradle)
├── mvnw / gradlew           the build-tool "wrapper" script (commit it)
└── src/
    ├── main/
    │   ├── java/ch18/shop/Cart.java        production code (package = folder path)
    │   └── resources/                       copied as-is onto the class path (e.g. logback.xml)
    └── test/
        ├── java/ch18/shop/CartTest.java     tests: same package, separate folder
        └── resources/                       test-only resources
target/  (Maven)  or  build/  (Gradle)        generated — never commit
```

Tests live in the **same package** as the code they test but in a **separate source folder**. That gives them access to **package-private** members (§5.6), while the build keeps test classes out of the final JAR. This chapter's `code/` folder mirrors it: `code/ch18/shop` is "main", `code/test/ch18/shop` is "test".

### Coordinates and repositories

Every library is identified by three **coordinates**, written `groupId:artifactId:version` — for example `org.assertj:assertj-core:3.27.7`.

| Term | Meaning |
|---|---|
| **groupId** | the organisation, reversed domain style (`org.junit.jupiter`) |
| **artifactId** | the library's name (`junit-jupiter-api`) |
| **version** | a release (`6.1.3`) or a **SNAPSHOT** (`1.0.0-SNAPSHOT`: work in progress, may change) |
| **artifact** | a file published under those coordinates — usually a JAR plus a `.pom` describing its own dependencies |
| **repository** | a server storing artifacts; the public one is **Maven Central** (`repo1.maven.org/maven2`, searchable at central.sonatype.com) |
| **local repository** | the cache on your disk (`~/.m2/repository` for Maven, `~/.gradle/caches` for Gradle) — downloaded once, reused by every project |
| **transitive dependency** | a dependency of a dependency, pulled in automatically (Mockito brings Byte Buddy and Objenesis) |

A repository is laid out by coordinates: `org.assertj:assertj-core:3.27.7` lives at `repo1.maven.org/maven2/org/assertj/assertj-core/3.27.7/assertj-core-3.27.7.jar` — how this chapter fetched its jars by hand, and what a build tool does for every (transitive) dependency.

### Why a build tool

Without one you'd download jars by hand, keep versions consistent, type long `-cp` lines and remember to test before packaging. A build tool turns that into one file and one command (`mvn verify`, `./gradlew build`) that behaves the same on every machine and on **CI** (*continuous integration*: a server that builds and tests every pushed change).

#### TL;DR

Standard layout (`src/main/java`, `src/test/java`), libraries identified by `group:artifact:version`, downloaded from Maven Central into a local cache. The build tool does the rest.

---

## 18.2 Maven

### What it is

**Apache Maven** (2004) is a build tool driven by a **declarative** XML file, `pom.xml` (**P**roject **O**bject **M**odel). You describe *what* the project is — coordinates, dependencies, **plugin** settings (a plugin is an add-on JAR that does one kind of build work) — and Maven's fixed **lifecycle** decides *how* and *in which order* things happen. The current stable line is Maven 3.9 (3.9.16 at the time of writing); Maven 4.0 was still a release candidate.

### The `pom.xml` — `code/build-files/maven/pom.xml`

Key parts (the full file adds pinned plugin versions):

```
<project ...>
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>shop</artifactId>
  <version>1.0.0-SNAPSHOT</version>
  <packaging>jar</packaging>

  <properties>
    <maven.compiler.release>25</maven.compiler.release>   <!-- same as javac option: release 25 -->
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencyManagement>                        <!-- versions only; adds nothing by itself -->
    <dependencies>
      <dependency>                              <!-- a BOM pins every JUnit module's version -->
        <groupId>org.junit</groupId> <artifactId>junit-bom</artifactId> <version>6.0.3</version>
        <type>pom</type> <scope>import</scope>
      </dependency>
      <dependency>                              <!-- one Byte Buddy for Mockito AND AssertJ -->
        <groupId>net.bytebuddy</groupId> <artifactId>byte-buddy</artifactId> <version>1.18.11</version>
      </dependency>
      <!-- ... byte-buddy-agent 1.18.11 ... -->
    </dependencies>
  </dependencyManagement>

  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId> <artifactId>junit-jupiter</artifactId>
      <scope>test</scope>                       <!-- no version: taken from the BOM -->
    </dependency>
    <dependency>
      <groupId>org.mockito</groupId> <artifactId>mockito-junit-jupiter</artifactId>
      <version>5.23.0</version> <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>org.assertj</groupId> <artifactId>assertj-core</artifactId>
      <version>3.27.7</version> <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <!-- maven-dependency-plugin 'properties' goal: exposes the mockito-core jar path as a property -->
      <plugin>
        <artifactId>maven-surefire-plugin</artifactId> <version>3.5.4</version>
        <configuration>
          <argLine>-javaagent:${org.mockito:mockito-core:jar} -Xshare:off</argLine>
          <excludedGroups>demo-failure</excludedGroups>        <!-- JUnit tags to skip -->
        </configuration>
      </plugin>
      <plugin>
        <artifactId>maven-jar-plugin</artifactId> <version>3.5.1</version>
        <configuration>
          <archive><manifest><mainClass>ch18.shop.ShopDemo</mainClass></manifest></archive>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

What each part does:

- **Coordinates** of *this* project — what other projects would write to depend on it.
- **`maven.compiler.release`** — passes `--release 25` to `javac` (Chapter 1 §1.5).
- **`dependencyManagement`** — a table of versions; it *declares* nothing. A **BOM** (bill of materials) is a pom that is only such a table, imported with `<scope>import</scope>`, so all JUnit modules share one version.
- **`dependencies`** — what the project actually uses, each with a **scope**.
- **`build/plugins`** — configure the plugins that do the work (compile, test, jar).

### Dependency scopes

| Scope | On compile class path | On test class path | Packaged / at run time | Typical example |
|---|---|---|---|---|
| `compile` (default) | yes | yes | yes | Jackson, Guava |
| `provided` | yes | yes | no — the runtime supplies it | Servlet API inside an **app server** (e.g. Tomcat) |
| `runtime` | no | yes | yes | JDBC **driver** (the vendor library that talks to a specific database; code compiles against `java.sql` only) |
| `test` | no (main code) | yes | no | JUnit, Mockito, AssertJ |
| `import` | — | — | — | only for a BOM inside `dependencyManagement` |

`system` also exists but is deprecated — don't use it.

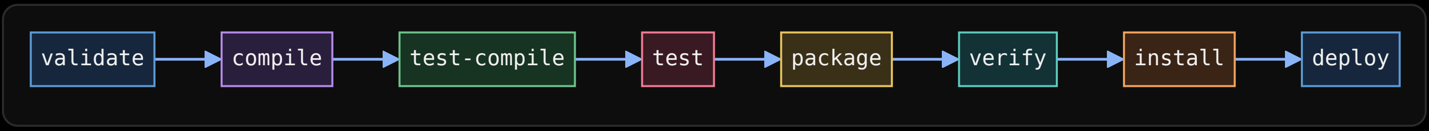
### Transitive dependencies and version conflicts

Maven builds a dependency **tree**. When two paths bring *different versions* of one library, Maven picks the **nearest** one in the tree; on a tie, the one declared **first**. This build hit a real conflict: `mockito-core` wants Byte Buddy 1.17.7 at depth 3, `assertj-core` wants 1.18.3 at depth 2 — so "nearest wins" picked AssertJ's version, and Mockito's `byte-buddy-agent` would have stayed at a different version. The fix is to **pin** the version in `dependencyManagement`. The real tree after pinning (`mvn -o dependency:tree`):

```
[INFO] com.example:shop:jar:1.0.0-SNAPSHOT
[INFO] +- org.junit.jupiter:junit-jupiter:jar:6.0.3:test
[INFO] |  +- org.junit.jupiter:junit-jupiter-api:jar:6.0.3:test
[INFO] |  |  +- org.opentest4j:opentest4j:jar:1.3.0:test
[INFO] |  |  +- org.junit.platform:junit-platform-commons:jar:6.0.3:test
[INFO] |  |  +- org.apiguardian:apiguardian-api:jar:1.1.2:test
[INFO] |  |  \- org.jspecify:jspecify:jar:1.0.0:test
[INFO] |  +- org.junit.jupiter:junit-jupiter-params:jar:6.0.3:test
[INFO] |  \- org.junit.jupiter:junit-jupiter-engine:jar:6.0.3:test
[INFO] |     \- org.junit.platform:junit-platform-engine:jar:6.0.3:test
[INFO] +- org.mockito:mockito-junit-jupiter:jar:5.23.0:test
[INFO] |  \- org.mockito:mockito-core:jar:5.23.0:test
[INFO] |     +- net.bytebuddy:byte-buddy-agent:jar:1.18.11:test
[INFO] |     \- org.objenesis:objenesis:jar:3.3:test
[INFO] \- org.assertj:assertj-core:jar:3.27.7:test
[INFO]    \- net.bytebuddy:byte-buddy:jar:1.18.11:test
```

Other conflict tools: `<exclusions>` on a dependency removes one transitive artifact; the **Maven Enforcer** plugin can fail the build on version-convergence problems (different versions of one library requested).

### The lifecycle: phases, plugins and goals



- A **phase** is a step in the lifecycle. Running a phase runs **every phase before it**: `mvn package` = validate, compile, test-compile, test, then package.
- A **plugin** is a JAR that does work; each unit of work is a **goal**, written `plugin:goal` (`compiler:compile`, `surefire:test`). The `jar` packaging binds default goals to phases:

| Phase | Default goal bound (packaging `jar` ) | Result |
|---|---|---|
| process-resources | `resources:resources` | copy `src/main/resources` into `target/classes` |
| compile | `compiler:compile` | `target/classes` |
| test-compile | `compiler:testCompile` | `target/test-classes` |
| test | `surefire:test` | runs unit tests; fails the build on a failure |
| package | `jar:jar` | `target/shop-1.0.0-SNAPSHOT.jar` |
| integration-test / verify | `failsafe:*` if configured | integration tests ( `*IT.java` ) and checks |
| install | `install:install` | copy the JAR into `~/.m2/repository` for other local projects |
| deploy | `deploy:deploy` | upload to a remote repository (company Nexus/Artifactory, Maven Central) |

Two other lifecycles exist: **clean** ( `mvn clean` deletes `target/` ) and **site** (project reports). Surefire runs test classes whose names match `Test*` , `*Test` , `*Tests` or `*TestCase` by default.

### Everyday commands

| Command | Does |
|---|---|
| `mvn clean verify` | full build from scratch with all checks — what CI usually runs |
| `mvn test` | compile and run unit tests |
| `mvn -Dtest=CartTest test` | run one test class ( `CartTest#newCartIsEmpty` for one method) |
| `mvn package -DskipTests` | build the JAR without running tests (compiles them; `-Dname=value` sets a property) |
| `mvn dependency:tree` | print the resolved tree |
| `mvn -o …` | offline: use only the local repository |
| `./mvnw …` | the **Maven Wrapper**: downloads the exact Maven version the project pins ( `mvn wrapper:wrapper` creates it); commit it so everyone and CI use the same Maven without installing it |

### A real build

The sources were copied into the standard layout next to this `pom.xml` and built **offline** ( `mvn -o -B package` ; `-B` = batch mode, no prompts or colours) from the local cache. Trimmed real output:

```
[INFO] --- dependency:3.10.0:properties (default) @ shop ---
[INFO] --- resources:3.3.1:resources (default-resources) @ shop ---
[INFO] --- compiler:3.15.0:compile (default-compile) @ shop ---
[INFO] Compiling 11 source files with javac [debug release 25] to target/classes
[INFO] --- resources:3.3.1:testResources (default-testResources) @ shop ---
[INFO] --- compiler:3.15.0:testCompile (default-testCompile) @ shop ---
[INFO] Compiling 6 source files with javac [debug release 25] to target/test-classes
[INFO] --- surefire:3.5.4:test (default-test) @ shop ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO] Running ch18.shop.DiscountsTest
[INFO] Tests run: 27, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.105 s -- in ch18.shop.DiscountsTest
[INFO] Running ch18.shop.AssertjStyleTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.395 s -- in ch18.shop.AssertjStyleTest
[INFO] Running ch18.shop.OrderServiceTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.166 s -- in ch18.shop.OrderServiceTest
...
[INFO] Results:
[WARNING] Tests run: 48, Failures: 0, Errors: 0, Skipped: 1
[INFO] --- jar:3.5.1:jar (default-jar) @ shop ---
[INFO] Building jar: .../java18_mvn/target/shop-1.0.0-SNAPSHOT.jar
[INFO] BUILD SUCCESS
[INFO] Total time:  2.645 s
```

That is the lifecycle table in action. The real run taught two lessons: the first attempt failed because Maven 3.9.14's *default* resources plugin (3.4.0) wasn't cached — **pin every plugin version** so a Maven upgrade can't silently change your build — and the Byte Buddy conflict above. `java -jar target/shop-1.0.0-SNAPSHOT.jar` runs `ShopDemo` (output in 18.12).

### Multi-module builds (in brief)

Big projects split into **modules** (`shop-domain`, `shop-web`, …) under a parent pom with `<packaging>pom</packaging>` and a `<modules>` list. The parent holds shared `dependencyManagement` and plugin settings; `mvn verify` at the root builds modules in dependency order.

### Common mistakes

- Omitting plugin versions (build changes when Maven is upgraded) or using version ranges.
- Test libraries without `<scope>test</scope>` — they end up in the product.
- Fighting a conflict with random exclusions instead of reading `dependency:tree` and pinning in `dependencyManagement`.
- Running `mvn install` just to build — `verify` is enough unless another local project needs the JAR.

### In the interview

> **30-second answer:** "Maven is a declarative build tool. The `pom.xml` gives coordinates, dependencies with scopes, and plugin configuration. It resolves transitive dependencies from Maven Central into `~/.m2`, choosing the nearest version on conflicts, and runs a fixed lifecycle — compile, test, package, verify, install, deploy — where running a phase runs all earlier ones. Plugins' goals are bound to phases."

- *`dependencies` vs `dependencyManagement`?* → The first adds dependencies; the second only fixes versions (and scopes) for when they are used, including transitive ones — and overrides "nearest wins".
- *What is a BOM?* → A pom that only contains `dependencyManagement`; importing it aligns versions of a family of libraries (JUnit, Spring, Jackson).
- *`install` vs `deploy`?* → Local repository vs remote repository.

### TL;DR

`pom.xml` = coordinates + dependencies (with scopes) + plugins. Fixed lifecycle; a phase runs all earlier phases. Pin plugin and conflicting library versions; read `dependency:tree`.

## 18.3 Gradle

### What it is

**Gradle** (2008) is a build tool whose build file is **code** — a Kotlin (`build.gradle.kts`, the default for new builds) or Groovy (`build.gradle`) script. Instead of a fixed lifecycle it builds a **graph of tasks** and runs only what's needed, skipping **up-to-date** tasks (**incremental builds**: only redo work whose inputs changed — why a second `./gradlew build` with no changes is fast) and reusing outputs from a **build cache** (stored results of earlier runs, possibly shared between machines). It uses the same Maven repositories and coordinates. Gradle 9 (9.4.1 here) needs Java 17+ to run the build itself.

### The build file — `code/build-files/gradle/build.gradle.kts`

```
plugins {
    application                                   // adds 'java' + the run task
}

group = "com.example"
version = "1.0.0-SNAPSHOT"

java {
    toolchain { languageVersion = JavaLanguageVersion.of(25) }   // build with a JDK 25
}

repositories {
    mavenCentral()
}

val mockitoAgent = configurations.create("mockitoAgent")   // a separate list holding just the agent jar

dependencies {
    testImplementation(platform("org.junit:junit-bom:6.0.3"))
    testImplementation("org.junit.jupiter:junit-jupiter")
    testRuntimeOnly("org.junit.platform:junit-platform-launcher")
    testImplementation("org.mockito:mockito-junit-jupiter:5.23.0")
    testImplementation("org.assertj:assertj-core:3.27.7")
    testImplementation("net.bytebuddy:byte-buddy:1.18.11")        // one Byte Buddy version for everybody
    testImplementation("net.bytebuddy:byte-buddy-agent:1.18.11")
    mockitoAgent("org.mockito:mockito-core:5.23.0") { isTransitive = false }
}

application { mainClass = "ch18.shop.ShopDemo" }

tasks.test {
    useJUnitPlatform { excludeTags("demo-failure") }
    jvmArgs("-javaagent:${mockitoAgent.asPath}", "-Xshare:off")
    testLogging { events("passed", "skipped", "failed") }
}

tasks.jar { manifest { attributes["Main-Class"] = "ch18.shop.ShopDemo" } }
```

`settings.gradle.kts` holds one line, `rootProject.name = "shop"` (multi-project builds list their subprojects there with `include(...)`).

- **Toolchain** — Gradle finds (or downloads) a JDK 25 to compile and test with, independent of the JDK running Gradle.
- **`platform(...)`** — imports a BOM, like Maven's `import` scope.
- **`testRuntimeOnly("org.junit.platform:junit-platform-launcher")`** — required since Gradle 9 when running on the JUnit Platform.
- **`useJUnitPlatform()`** — without it Gradle looks for JUnit 4 tests and finds none.

### Configurations (Gradle's scopes)

| Gradle configuration | Closest Maven scope | Meaning |
|---|---|---|
| `implementation` | compile | needed to compile and run, **not** exposed to consumers' compile class path |
| `api` (plugin `java-library`) | compile | part of your public API: consumers compile against it too |
| `compileOnly` | provided | compile time only (annotations, provided APIs) |
| `runtimeOnly` | runtime | run time only (JDBC driver, logging backend) |
| `testImplementation` / `testRuntimeOnly` | test | the same, for tests |

`implementation` vs `api` is a real improvement over Maven: hiding implementation dependencies means fewer recompilations and less accidental **coupling**.

### A real build

The same sources, built offline with Gradle 9.4.1 (`gradle --offline clean build --console=plain`; for the offline run `mavenCentral()` was replaced by `mavenLocal()` so Gradle could read the `~/.m2` cache):

```
> Task :clean
> Task :compileJava
...                              (classes, jar, startScripts, distTar, distZip, assemble, compileTestJava, testClasses)
> Task :test
Cart > itemsViewIsReadOnly() PASSED
Cart > shippingIsFreeAbove50() SKIPPED
Cart > adding the same SKU twice merges the quantities PASSED
...
> Task :check
> Task :build
BUILD SUCCESSFUL in 5s
8 actionable tasks: 8 executed
```

`build` depends on `assemble` (jar + distributions from the `application` plugin) and `check` (tests). `gradle run` then ran `ShopDemo`. Useful commands: `./gradlew test --tests 'ch18.shop.CartTest'`, `./gradlew dependencies --configuration testRuntimeClasspath`, `./gradlew tasks`, `./gradlew build --scan` (online build report).

**The Gradle Wrapper.** `gradle wrapper` generates `gradlew`, `gradlew.bat` and `gradle/wrapper/gradle-wrapper.properties` (holding `distributionUrl`). Commit them and always run `./gradlew`. The user's OCP study repo is a real example: its `build.gradle` (Groovy DSL) pins a **Java 21 toolchain**, gives every chapter its own *source set* with its own compile task, and its wrapper pins Gradle 9.4.1.

### Maven vs Gradle

| | Maven | Gradle |
|---|---|---|
| Build file | XML, declarative | Kotlin/Groovy script |
| Model | fixed lifecycle of phases | task graph, incremental, build cache, daemon (a background process kept between builds) |
| Speed on big builds | fine | usually faster (up-to-date checks, caching) |
| Flexibility | limited — write a plugin for custom logic | very high (and easy to over-engineer) |
| Learning curve | gentle, very uniform across projects | steeper; builds differ more |
| Where you'll meet it | most enterprise/back-end Java, Spring examples | Android (mandatory), many newer projects, Kotlin |

Both are good choices; follow the team. Spring Initializr (start.spring.io) generates either.

### Common mistakes

- Using `gradle` from your PATH instead of `./gradlew`.
- Forgetting `useJUnitPlatform()` — "0 tests" and a green build.
- Putting everything in `api` "to be safe".
- Custom logic scattered in the build script; move it to a plugin in `buildSrc` or a convention plugin (a shared plugin applying a team's standard settings).

### TL;DR

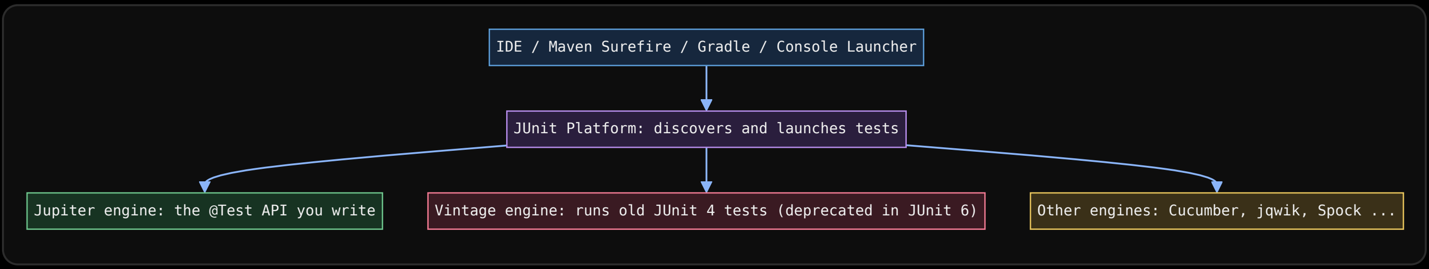
Gradle = scripted task graph with incremental builds; same repositories and coordinates as Maven. Use the wrapper, `implementation` by default, and remember `useJUnitPlatform()`.

---

## 18.4 JUnit: your first automated tests

### What it is

An **automated test** runs code with known inputs and checks the result, reporting pass or fail with no human reading output. A **unit test** checks one small piece (a class or method) in isolation, in milliseconds. **JUnit** is Java's standard test **framework** (a library that calls *your* test methods). Its modern generation (JUnit 5, 2017; **JUnit 6**, released 30 September 2025) has three parts:


IDE / Maven Surefire / Gradle / Console Launcher
JUnit Platform: discovers and launches tests
Jupiter engine: the @Test API you write
Vintage engine: runs old JUnit 4 tests (deprecated in JUnit 6)
Other engines: Cucumber, jqwik, Spock ...

- **JUnit Platform** — the launcher that tools talk to.
- **JUnit Jupiter** — the programming model (`org.junit.jupiter.api.*`) and its engine.
- **JUnit Vintage** — an engine for legacy JUnit 4 tests.

JUnit 6 kept the same Jupiter API (these examples are identical in JUnit 5), moved the minimum to **Java 17**, gave Platform, Jupiter and Vintage **one shared version number** (6.x instead of 1.x/5.x), added JSpecify nullness annotations (marking whether a value may be `null`), a `--fail-fast` launcher option and test cancellation, and deprecated Vintage. Most JUnit 5 test code runs unchanged.

### Why it matters

Tests let you **refactor** (restructure without changing behaviour), upgrade or fix without fear — hundreds of checks in a second. They document behaviour precisely ("SAVE25 needs a subtotal of at least 100"), catch **regressions** (things that used to work and broke) before users do, and push toward better design (hard-to-test code is usually too coupled).

### The code under test

`Cart` holds `Item` records; adding the same SKU (*stock-keeping unit*, a product's id) twice merges quantities; money is `BigDecimal` (Chapter 7 §7.7).

```java
public final class Cart {
    private final Map<String, Item> items = new LinkedHashMap<>();
    public void add(Item item) {
        items.merge(item.sku(), item,
                (old, extra) -> new Item(old.sku(), old.name(), old.unitPrice(), old.quantity() +
extra.quantity()));
    }
    public void remove(String sku) {
        if (items.remove(sku) == null) throw new NoSuchElementException("not in cart: " + sku);
    }
    public List<Item> items()   { return List.copyOf(items.values()); }
    public boolean isEmpty()    { return items.isEmpty(); }
    public int count()          { return items.values().stream().mapToInt(Item::quantity).sum(); }
    public BigDecimal subtotal() { /* sum of line totals, setScale(2, HALF_UP) */ }
}
```

### The tests — `code/test/ch18/shop/CartTest.java`

```java
@Tag("fast")
@DisplayName("Cart")
class CartTest {

    private Cart cart;                                   // a NEW CartTest instance is created for every test

    @BeforeEach
    void setUp() { cart = new Cart(); }

    @Test
    void newCartIsEmpty() {
        assertTrue(cart.isEmpty());
        assertEquals(0, cart.count());
        assertEquals(new BigDecimal("0.00"), cart.subtotal());
    }

    @Test
    @DisplayName("adding the same SKU twice merges the quantities")
    void addSameSkuMerges() {
        cart.add(new Item("MUG", "Mug", new BigDecimal("12.50"), 1));
        cart.add(new Item("MUG", "Mug", new BigDecimal("12.50"), 2));

        assertAll("merged line",                         // reports EVERY failing check, not just the first
                () -> assertEquals(1, cart.items().size()),
                () -> assertEquals(3, cart.count()),
                () -> assertEquals(new BigDecimal("37.50"), cart.subtotal()));
    }

    @Test
    void removingUnknownSkuThrows() {
        NoSuchElementException e = assertThrows(NoSuchElementException.class, () -> cart.remove("NOPE"));
        assertEquals("not in cart: NOPE", e.getMessage());
    }

    @Test
    @Disabled("shipping costs not implemented yet")
    void shippingIsFreeAbove50() { fail("not written yet"); }

    @Nested
    @DisplayName("with a book and a mug")
    class WithTwoItems {
        @BeforeEach
        void fill() {                                    // runs AFTER the outer setUp()
            cart.add(new Item("BK-1", "Java book", new BigDecimal("45.00"), 2));
            cart.add(new Item("MUG", "Mug", new BigDecimal("12.50"), 1));
        }
        @Test void subtotalAddsLineTotals() { assertEquals(new BigDecimal("102.50"), cart.subtotal()); }
        @Test void removeDropsTheWholeLine() { cart.remove("BK-1"); assertEquals(1, cart.count()); }
    }
    // ... itemsViewIsReadOnly()
}
```

How it works:

1. `@Test` marks a test method. Test classes and methods don't need to be `public` (package-private is the convention); they must not be `private`.
2. The static import `org.junit.jupiter.api.Assertions.*` gives the `assertXxx` methods (an **assertion** fails the test if its check isn't true). Argument order is **(expected, actual)**, with an optional message **last**.
3. A failed assertion throws `AssertionFailedError`; any other uncaught exception also fails the test. A test that finishes normally passes.
4. `assertThrows` runs the lambda, checks the exception type and **returns** the exception so you can inspect it.
5. `@Nested` inner classes group tests sharing a situation; the outer `@BeforeEach` runs first.

### The assertions you'll use

| Assertion | Checks |
|---|---|
| `assertEquals(exp, act)` / `assertNotEquals` | `equals` (with a `delta` overload for doubles) |
| `assertTrue` / `assertFalse` | a condition |
| `assertNull` / `assertNotNull` | null-ness |
| `assertSame` / `assertNotSame` | same reference (`==`) |
| `assertArrayEquals`, `assertIterableEquals` | element by element |
| `assertLinesMatch` | lists of strings, regex lines allowed |
| `assertInstanceOf(Type.class, obj)` | type, returns the cast value |
| `assertThrows` / `assertThrowsExactly` | an exception of that type (or subtype / exactly that type) |
| `assertDoesNotThrow` | no exception; returns the result |
| `assertAll(...)` | runs every check, reports all failures together |
| `assertTimeout(Duration, ...)` | finishes in time (`assertTimeoutPreemptively` aborts it) |
| `fail("msg")` | fails immediately |

Messages can be a `Supplier<String>` (`() -> "got " + r`), built only on failure.

### Lifecycle — `LifecycleTest.java`

```java
class LifecycleTest {
    static int instances;
    LifecycleTest()            { System.out.println("  constructor  (instance #" + ++instances + ")"); }

    @BeforeAll static void beforeAll() { System.out.println("@BeforeAll   (static, once)"); }
    @BeforeEach void beforeEach()      { System.out.println("  @BeforeEach"); }
    @Test void first()                 { System.out.println("    test: first"); }
    @Test void second()                { System.out.println("    test: second"); }
    @AfterEach void afterEach()        { System.out.println("  @AfterEach"); }
    @AfterAll static void afterAll()   { System.out.println("@AfterAll    (static, once)"); }
}
```

Output (from the run below):

```
@BeforeAll   (static, once)
  constructor  (instance #1)
  @BeforeEach
    test: second
  @AfterEach
  constructor  (instance #2)
  @BeforeEach
    test: first
  @AfterEach
@AfterAll    (static, once)
```

Three facts to remember:

- **A fresh instance per test method** (default `@TestInstance(Lifecycle.PER_METHOD)`), so fields can't leak state between tests. That is also why `@BeforeAll`/`@AfterAll` must be `static` — unless you annotate the class `@TestInstance(Lifecycle.PER_CLASS)`.
- **Method order is deterministic but deliberately not obvious** — here `second` ran before `first`. Tests must not depend on each other. If you truly need an order: `@TestMethodOrder(MethodOrderer.OrderAnnotation.class)` + `@Order(n)`.
- `@AfterEach` runs even when the test fails — use it for cleanup.

### More annotations

| Annotation | Purpose |
|---|---|
| `@DisplayName("…")` | readable name in reports |
| `@Disabled("reason")` | skip, with a reason shown in the report (`↷` below) |
| `@Tag("fast")` | label for filtering (`--include-tag`, Maven `<groups>`, Gradle `includeTags`) |
| `@Timeout(2)` | fail if slower than 2 s |
| `@TempDir Path dir` | a fresh temporary directory, deleted afterwards (18.7) |
| `@ExtendWith(X.class)` | plug in an **extension** (hooks into the test lifecycle, e.g. Mockito, Spring) — replaces JUnit 4 runners and rules |
| `@EnabledOnOs`, `@EnabledIfEnvironmentVariable`, `@EnabledForJreRange` … | conditional execution |

**Assumptions** (`assumeTrue(cond)`, `assumingThat`) stop a test as *aborted* rather than failed when a precondition doesn't hold (e.g. "only on CI"). Aborted = an assumption didn't hold; failed = an assertion or unexpected exception.

### Running the tests from the command line

IDEs and build tools run JUnit for you. Without them, the **Console Launcher** (`junit-platform-console-standalone`, one JAR containing the Platform, Jupiter, Vintage and Suite engines) does it. The plain form — compile, then point the launcher at the classes:

```
$ javac --release 25 -cp '.java-books/lib/*' -d /tmp/java18t $(find ch18/shop test -name '*.java')
$ java -jar junit-platform-console-standalone-6.1.3.jar execute \
      --class-path /tmp/java18t:assertj-core-3.27.7.jar --scan-class-path \
      --include-classname '.*(Cart|Discounts)Test' --details=summary --disable-banner

Test run finished after 101 ms
...                              (13 containers found, started and successful)
[        34 tests found           ]
[         1 tests skipped         ]
[        33 tests successful      ]
[         0 tests failed          ]
```

`--scan-class-path` discovers every class whose name matches the default pattern (`Test*`, `*Test`, `*Tests`). The **exit code** is 0 only if nothing failed — that's what CI checks. Filter by tag with `--include-tag fast` (console), `<groups>fast</groups>` (Surefire) or `useJUnitPlatform { includeTags("fast") }` (Gradle).

For the Mockito tests the launcher must be started differently, because Mockito wants to be attached as a **Java agent** (a JAR the JVM loads before your code so it can modify classes as they load; 18.6) and the agent must be the *same* JAR the tests load. So `code/run_tests.sh` puts everything on one class path and starts the launcher by its main class:

```
LIB="../../../../.java-books/lib"; CP="$LIB/*"; OUT=$(mktemp -d)
javac --release 25 -cp "$CP" -d "$OUT" $(find ch18/shop test -name '*.java') || exit 1
if [ "${1:-}" = fail ]; then TAGS=(--include-tag demo-failure); else TAGS=(--exclude-tag demo-failure); fi
java -Xshare:off -javaagent:"$(ls "$LIB"/mockito-core-*.jar)" \
     -Djava.util.logging.SimpleFormatter.format='%4$s: %5$s%n' \
     -cp "$OUT:$CP" org.junit.platform.console.ConsoleLauncher execute \
     --scan-class-path "$OUT" "${TAGS[@]}" \
     --disable-banner --disable-ansi-colors --details=tree
```

`bash code/run_tests.sh` output (tree trimmed; notes in parentheses added):

```
WARNING: declined: customer=bob total=102.50
INFO: order placed: id=ORD-1 total=102.50 ref=PAY-9
INFO: order placed: id=ORD-1 total=76.88 ref=PAY-9
INFO: order placed: id=ORD-7 total=92.25 ref=ok
@BeforeAll   (static, once)
  ... (lifecycle lines shown above)
│
├─ JUnit Platform Suite ✔
├─ JUnit Jupiter ✔
│  ├─ AssertjStyleTest ✔          (6 tests, 18.7)
│  ├─ DiscountsTest ✔             (27 tests, 18.5)
│  ├─ Cart ✔
│  │  ├─ itemsViewIsReadOnly() ✔
│  │  ├─ shippingIsFreeAbove50() ↷ shipping costs not implemented yet
│  │  ├─ adding the same SKU twice merges the quantities ✔
│  │  ├─ newCartIsEmpty() ✔
│  │  ├─ removingUnknownSkuThrows() ✔
│  │  └─ with a book and a mug ✔
│  │     ├─ removeDropsTheWholeLine() ✔
│  │     └─ subtotalAddsLineTotals() ✔
│  ├─ OrderServiceTest ✔          (6 tests, 18.6)
│  └─ LifecycleTest ✔
│     ├─ second() ✔
│     └─ first() ✔
└─ JUnit Vintage ✔

Test run finished after 566 ms
...
[        48 tests found           ]
[         1 tests skipped         ]
[        47 tests started         ]
[        47 tests successful      ]
[         0 tests failed          ]
```

The `INFO:`/`WARNING:` lines at the top are the application's own logging (18.10) printed while `OrderServiceTest` ran.

### Coming from JUnit 4

| JUnit 4 | Jupiter (JUnit 5/6) |
|---|---|
| `org.junit.Test`, public classes/methods | `org.junit.jupiter.api.Test`, package-private OK |
| `@Before` / `@After` | `@BeforeEach` / `@AfterEach` |
| `@BeforeClass` / `@AfterClass` | `@BeforeAll` / `@AfterAll` |
| `@Ignore` | `@Disabled` |
| `@Test(expected = X.class)` | `assertThrows(X.class, ...)` |
| `@RunWith`, `@Rule` | `@ExtendWith` extensions |
| `@Category` | `@Tag` |
| `assertEquals("msg", exp, act)` — message first | `assertEquals(exp, act, "msg")` — message last |

### Common mistakes

- Swapping expected and actual — failure messages then read backwards.
- Forgetting `@Test` (the method never runs, silently).
- Mixing JUnit 4's `org.junit.Test` with Jupiter assertions — the Jupiter engine ignores JUnit 4 tests.
- Tests that depend on each other's order or on shared static state.
- Printing results instead of asserting them — a test that can't fail is not a test.

### In the interview

> **30-second answer:** "JUnit 5 and 6 split into the Platform, which launches tests for IDEs and build tools, and Jupiter, the API: `@Test`, `@BeforeEach` / `@AfterEach`, static `@BeforeAll` / `@AfterAll`, assertions like `assertEquals`, `assertThrows` and `assertAll`, plus `@Nested`, `@Tag`, `@DisplayName`, `@Disabled` and extensions via `@ExtendWith`. A new test instance is created per method so tests stay independent."

- *Why is `@BeforeAll` static?* → One instance per test method exists (2 tests → 2 `LifecycleTest` instances), so a once-per-class hook can't belong to an instance — unless `PER_CLASS` lifecycle is used.
- *`assertAll` vs several asserts?* → Several asserts stop at the first failure; `assertAll` reports all of them.
- *What changed in JUnit 6?* → Java 17 baseline, a single version for all modules, JSpecify annotations, Vintage deprecated; the Jupiter API is essentially the same.

### TL;DR

`@Test` + `Assertions` + lifecycle hooks, one fresh instance per test, no reliance on order. Run from IDE, build tool, or the Console Launcher.

## 18.5 Parameterized, repeated and dynamic tests

### The problem it solves

Rules like "a code is 4+ capital letters followed by 2 digits" need many inputs. Copy-pasting a test per input is noisy; a loop inside one test stops at the first failure and reports one anonymous result. **Parameterized tests** run the same method once per input, each reported separately. (They live in `junit-jupiter-params`, included in the `junit-jupiter` aggregate artifact.)

### The rules under test — `Discounts.java`

```java
public static boolean isValidCode(String code) {
    return code != null && code.matches("[A-Z]{4,}\\d{2}");
}
public static int percentFor(String code, BigDecimal subtotal) {
    if (!isValidCode(code)) return 0;
    return switch (code) {
        case "SAVE10" -> 10;
        case "SAVE25" -> subtotal.compareTo(BIG_ORDER) >= 0 ? 25 : 0;   // BIG_ORDER = 100.00
        default -> 0;
    };
}
public static BigDecimal apply(BigDecimal amount, int percent) {
    if (percent < 0 || percent > 100) throw new IllegalArgumentException("percent out of range: " + percent);
    return amount.multiply(BigDecimal.valueOf(100 - percent)).divide(BigDecimal.valueOf(100), 2,
RoundingMode.HALF_UP);
}
```

### The tests — `DiscountsTest.java`

```java
@ParameterizedTest(name = "{0} is a valid code")
@ValueSource(strings = {"SAVE10", "SAVE25", "WELCOME99"})
void validCodes(String code) { assertTrue(Discounts.isValidCode(code)); }

@ParameterizedTest(name = "[{index}] {0} is rejected")
@NullAndEmptySource                                   // adds null and ""
@ValueSource(strings = {"save10", "SAV10", "SAVE1", "SAVE100"})
void invalidCodes(String code) { assertFalse(Discounts.isValidCode(code)); }

@ParameterizedTest(name = "{0} on {1} -> {2}%")
@CsvSource({
        "SAVE10,  20.00, 10",
        "SAVE25,  99.99,  0",     // boundary: just below 100
        "SAVE25, 100.00, 25",     // boundary: exactly 100
        "SAVE99, 500.00,  0",     // well-formed but unknown
})
void percentForCode(String code, BigDecimal subtotal, int expected) {   // strings converted to BigDecimal/int
    assertEquals(expected, Discounts.percentFor(code, subtotal));
}

static Stream<Arguments> roundingCases() {
    return Stream.of(
            Arguments.of("10.00", 10, "9.00"),
            Arguments.of("0.05", 50, "0.03"),              // 0.025 rounds half-up to 0.03
            Arguments.of("102.50", 25, "76.88"),           // 76.875 -> 76.88
            Arguments.of("19.99", 100, "0.00"));
}

@ParameterizedTest
@MethodSource("roundingCases")
void applyRoundsHalfUpToCents(String amount, int percent, String expected) {
    assertEquals(new BigDecimal(expected), Discounts.apply(new BigDecimal(amount), percent));
}

@RepeatedTest(3)
void applyIsDeterministic() { assertEquals(new BigDecimal("9.00"), Discounts.apply(BigDecimal.TEN, 10)); }

@TestFactory
Stream<DynamicTest> everyPercentKeepsTheResultWithinRange() {
    return Stream.of(0, 33, 50, 99, 100).map(p -> dynamicTest("percent " + p, () -> {
        BigDecimal r = Discounts.apply(new BigDecimal("80.00"), p);
        assertTrue(r.signum() >= 0 && r.compareTo(new BigDecimal("80.00")) <= 0, () -> "got " + r);
    }));
}
```

Output (part of the same run; some rows collapsed, notes in parentheses added):

```
│  ├─ DiscountsTest ✔
│  │  ├─ validCodes(String) ✔
│  │  │  ├─ "SAVE10" is a valid code ✔
│  │  │  ├─ "SAVE25" is a valid code ✔
│  │  │  └─ "WELCOME99" is a valid code ✔
│  │  ├─ applyRoundsHalfUpToCents(String, int, String) ✔   (4 rows)
│  │  ├─ invalidCodes(String) ✔
│  │  │  ├─ [1] null is rejected ✔
│  │  │  ├─ [2] "" is rejected ✔
│  │  │  ├─ [3] "save10" is rejected ✔
│  │  │  ├─ [4] "SAV10" is rejected ✔
│  │  │  ├─ [5] "SAVE1" is rejected ✔
│  │  │  └─ [6] "SAVE100" is rejected ✔
│  │  ├─ applyRejectsBadPercent(int) ✔                     (2 rows)
│  │  ├─ percentForCode(String, BigDecimal, int) ✔
│  │  │  ├─ "SAVE10" on "20.00" -> "10"% ✔
│  │  │  ├─ "SAVE25" on "99.99" -> "0"% ✔
│  │  │  ├─ "SAVE25" on "100.00" -> "25"% ✔
│  │  │  └─ "SAVE99" on "500.00" -> "0"% ✔
│  │  ├─ everyPercentKeepsTheResultWithinRange() ✔
│  │  │  ├─ percent 0 ✔
│  │  │  ├─ ... percent 33, 50, 99 ✔
│  │  │  └─ percent 100 ✔
│  │  └─ applyIsDeterministic() ✔
│  │     ├─ repetition 1 of 3 ✔
│  │     ├─ repetition 2 of 3 ✔
│  │     └─ repetition 3 of 3 ✔
```

Notice: the name pattern placeholders are `{0}`, `{1}` … and `{index}`; this JUnit version shows text arguments **in quotes**, so `null` (no quotes) and `""` (the empty string) are easy to tell apart. `@CsvSource` values start as text and are **converted** to the parameter types (`BigDecimal`, `int`) automatically.

### Argument sources

| Source | Supplies | Example use |
|---|---|---|
| `@ValueSource(strings/ints/…)` | one literal argument per run | valid/invalid inputs |
| `@NullSource`, `@EmptySource`, `@NullAndEmptySource` | `null`, empty string/collection/array | edge cases |
| `@EnumSource(Level.class)` | each enum constant (filterable with `names`) | behaviour per enum value |
| `@CsvSource({...})` / `@CsvFileSource(resources = "/x.csv")` | rows of several arguments | input → expected tables |
| `@MethodSource("factory")` | a `static` method in the test class (or `Class#method` elsewhere; non-static allowed with `PER_CLASS`) returning `Stream<Arguments>` or a collection | objects that can't be written as text |
| `@FieldSource` (5.11+) | a static field holding the arguments | shared fixed data |
| `@ArgumentsSource(MyProvider.class)` | your own provider | reusable generators |

Related features: `@RepeatedTest(n)` runs a test n times (useful for suspected flakiness); `@TestFactory` returns **dynamic tests** built at run time, so the number of tests can depend on data (parameterized = fixed method fed by a source; dynamic = arbitrary tests from a factory). Since JUnit 5.13 a whole class can be parameterized with `@ParameterizedClass`.

### Common mistakes

- Many inputs in one `@Test` with a loop — the first failure hides the rest.
- Hiding the *expected* value computation inside the test (re-implementing the logic). Expected values should be literal, worked out by hand — like `76.88` above.

- Forgetting boundaries: the `99.99` / `100.00` rows are the most valuable ones in that table.

### TL;DR

One method, many inputs, each reported separately: `@ParameterizedTest` + a source. Put boundary values in the table.

---

## 18.6 Test doubles and Mockito

### The problem it solves

`OrderService.checkout` charges a card and saves an order. A unit test must not charge real cards or need a real database — slow, flaky and expensive. So the service **receives its collaborators** through the constructor (**dependency injection**: an object is handed its dependencies instead of creating them; the *Dependency Inversion Principle*, LLD §4.10), and tests pass in stand-ins called **test doubles**.

```java
public final class OrderService {
    private static final Logger LOG = Logger.getLogger(OrderService.class.getName());
    private final PaymentGateway gateway;          // interface: the card processor
    private final OrderRepository repository;      // interface: storage
    private final Clock clock;                     // time is a dependency too
    private final Supplier<String> ids;            // so is id generation

    public OrderService(PaymentGateway gateway, OrderRepository repository, Clock clock, Supplier<String> ids) { ... }

    public Order checkout(String customer, Cart cart, String discountCode) {
        if (cart.isEmpty()) throw new IllegalStateException("cart is empty");
        BigDecimal subtotal = cart.subtotal();
        int percent = Discounts.percentFor(discountCode, subtotal);
        BigDecimal total = Discounts.apply(subtotal, percent);

        PaymentGateway.PaymentResult result = gateway.charge(customer, total);
        if (!result.approved()) {
            LOG.warning(() -> "declined: customer=" + customer + " total=" + total);
            throw new PaymentDeclinedException(customer, total);
        }
        Order order = new Order(ids.get(), customer, total, clock.instant());
        repository.save(order);
        LOG.info(() -> "order placed: id=" + order.id() + " total=" + total + " ref=" + result.reference());
        return order;
    }
}
```

Injecting a `java.time.Clock` (Chapter 7 §7.10) and an id `Supplier` makes the result fully predictable: `Clock.fixed(...)` always returns the same instant.

### The five kinds of test double

| Double | What it is | In this chapter |
|---|---|---|
| **Dummy** | passed only to fill a parameter, never used | — |
| **Stub** | returns canned answers | the lambda `(c, amt) -> new PaymentResult(true, "ok")` |
| **Fake** | a real, working but simplified implementation | `InMemoryOrderRepository` (a `LinkedHashMap`) |
| **Spy** | a real object that also records calls | `spy(new InMemoryOrderRepository())` |
| **Mock** | programmed with expectations; you **verify** how it was called | `@Mock PaymentGateway gateway` |

Rule of thumb: check **state** (the result, the fake's contents) when you can; **verify interactions** only when the interaction *is* the behaviour — "the card was charged the discounted amount", "nothing was saved after a decline". A longer discussion of doubles and test strategy is in LLD — Testing strategies.

## Mockito in practice — `OrderServiceTest.java`

```java
@ExtendWith(MockitoExtension.class)                      // creates @Mock fields, checks for unused stubs
class OrderServiceTest {
    static final Instant NOW = Instant.parse("2025-09-16T10:00:00Z");

    @Mock PaymentGateway gateway;
    @Mock OrderRepository repository;
    @Captor ArgumentCaptor<Order> saved;
    OrderService service;
    Cart cart;

    @BeforeEach
    void setUp() {
        service = new OrderService(gateway, repository, Clock.fixed(NOW, ZoneOffset.UTC), () -> "ORD-1");
        cart = new Cart();
        cart.add(new Item("BK-1", "Java book", new BigDecimal("45.00"), 2));
        cart.add(new Item("MUG", "Mug", new BigDecimal("12.50"), 1));    // subtotal 102.50
    }

    @Test
    void approvedPaymentSavesTheOrder() {
        when(gateway.charge("ann", new BigDecimal("76.88"))).thenReturn(new PaymentResult(true, "PAY-9"));  // arrange

        Order order = service.checkout("ann", cart, "SAVE25");                                              // act

        assertEquals(new Order("ORD-1", "ann", new BigDecimal("76.88"), NOW), order);                       // assert
        verify(repository).save(saved.capture());         // called exactly once, capture the argument
        assertEquals("ORD-1", saved.getValue().id());
    }

    @Test
    void chargesBeforeSaving() {
        when(gateway.charge(anyString(), any())).thenReturn(new PaymentResult(true, "PAY-9"));
        service.checkout("ann", cart, null);
        InOrder inOrder = inOrder(gateway, repository);
        inOrder.verify(gateway).charge(eq("ann"), eq(new BigDecimal("102.50")));  // mixing matchers: all must be matchers
        inOrder.verify(repository).save(any(Order.class));
    }

    @Test
    void declinedPaymentThrowsAndSavesNothing() {
        when(gateway.charge(anyString(), any())).thenReturn(new PaymentResult(false, null));
        var e = assertThrows(PaymentDeclinedException.class, () -> service.checkout("bob", cart, null));
        assertEquals("payment of 102.50 declined for bob", e.getMessage());
        verifyNoInteractions(repository);
    }

    @Test
    void gatewayOutagePropagates() {
        when(gateway.charge(anyString(), any())).thenThrow(new IllegalStateException("gateway timeout"));
        assertThrows(IllegalStateException.class, () -> service.checkout("ann", cart, null));
        verify(repository, never()).save(any());
    }

    @Test
    void fakeRepositoryPlusSpy() {
        InMemoryOrderRepository fake = spy(new InMemoryOrderRepository());
        var svc = new OrderService((c, amt) -> new PaymentResult(true, "ok"),   // a stub written as a lambda
                fake, Clock.fixed(NOW, ZoneOffset.UTC), () -> "ORD-7");
        svc.checkout("cy", cart, "SAVE10");
        assertEquals(new BigDecimal("92.25"), fake.findById("ORD-7").orElseThrow().total());
        verify(fake, times(1)).save(any());
    }
    // ... emptyCartNeverReachesTheGateway()
}
```

All six passed (tree in 18.4). How Mockito works, step by step:

1. `@Mock` (or `mock(PaymentGateway.class)`) creates a generated subclass/**instrumented** class (bytecode rewritten at run time) whose methods record each call and return **defaults**: `null`, `0`, `false`, empty collections (an unstubbed `List<String> find()` returns an empty list), `Optional.empty()`.
2. `when(mock.method(args)).thenReturn(x)` **stubs** a call: when those arguments arrive, return `x`. Also `thenThrow(...)`, `thenAnswer(inv -> ...)`, and several values in a row (`thenReturn(a, b)`).
3. The code under test runs.
4. `verify(mock).method(args)` checks a call happened (exactly once by default). Variants: `times(n)`, `never()`, `atLeastOnce()`, `atMost(n)`, `verifyNoInteractions(...)`, `verifyNoMoreInteractions(...)`; `inOrder(...)` checks ordering.
5. **Argument matchers** — `any()`, `anyString()`, `eq(x)`, `argThat(pred)` — loosen matching. Rule: if one argument uses a matcher, *all* must (hence `eq("ann")` above); `verify(gateway).charge("ann", any())` throws `InvalidUseOfMatchersException`.
6. An **`ArgumentCaptor`** grabs what was passed so you can assert on it afterwards.

### More Mockito you'll meet

| Need | API |
|---|---|
| Stub a `void` method to throw | `doThrow(new X()).when(mock).voidMethod()` |
| Stub a spy without calling the real method | `doReturn(x).when(spy).method()` |
| Given/when/then wording (BDD — *behaviour-driven development* — style) | `BDDMockito.given(...).willReturn(...)`, `then(mock).should().method()` |
| Mock a `static` method | `try (MockedStatic<Util> m = mockStatic(Util.class)) { m.when(Util::now).thenReturn(...); }` |
| Allow an unused stub | `lenient().when(...)` or `@MockitoSettings(strictness = LENIENT)` |
| Mock without the extension | `mock(X.class)` or `MockitoAnnotations.openMocks(this)` |

**Mockito 5 notes.** Mockito 5 needs Java 11+, and its default *inline* mock maker instruments classes at run time, so it can mock `final` classes, records and static methods. It does that through a Java agent. On JDK 21+ loading an agent *dynamically* (into an already-running JVM) prints a warning (JEP 451), and a future JDK will block it by default — without `-javaagent` the first run of this chapter printed:

```
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of
the JDK. Please add Mockito as an agent to your build as described in Mockito's documentation: ...
WARNING: A Java agent has been loaded dynamically (.../byte-buddy-agent-1.18.14.jar)
```

Hence `-javaagent:mockito-core.jar` in `run_tests.sh`, Surefire's `argLine`, and Gradle's `jvmArgs`. (`-Xshare:off` only silences a harmless class-data-sharing notice caused by the agent — Chapter 16 §16.2.)

### When not to mock

- **Value objects** (defined only by their data) **and records** (`Item`, `Order`, `BigDecimal`) — just create real ones.
- **Types you don't own** (e.g. a JDBC `ResultSet`, an HTTP client): wrap them behind your own interface and mock that, or use an **integration test** (against real infrastructure such as a database, §18.9).
- **Everything** — tests that mock every collaborator end up restating the implementation ("calls A then B then C") and break on every refactor while catching few bugs. Prefer fakes for storage-like collaborators.

### Common mistakes

- Stubbing with the wrong arguments — the stub never matches and the mock returns `null`. **Strict stubs** (the `MockitoExtension` default) turn that into an `UnnecessaryStubbingException`, as 18.8 shows.
- Verifying calls that aren't part of the behaviour (logging, getters).
- `when(spy.method())` on a spy calls the real method during stubbing — use `doReturn`.
- Mixing raw values and matchers in one call.

### In the interview

> **30-second answer:** "A test double replaces a real collaborator: dummy, stub, fake, spy or mock. Mockito creates mocks, stubs them with `when(...).thenReturn(...)`, and verifies interactions with `verify`, argument matchers and captors. I inject collaborators through the constructor so tests can swap them, use fakes for things like repositories, and mock only true boundaries like payment gateways — checking state first and interactions only when they are the behaviour."

- *Mock vs stub?* → A stub provides answers; a mock is also verified for how it was called.
- *Mock vs spy?* → A mock is a hollow object with defaults; a spy wraps a real object and calls real methods unless stubbed.
- *How do you test code that uses `LocalDateTime.now()`?* → Inject a `Clock`, pass `Clock.fixed(...)` in tests.

### TL;DR

Design for injection, then choose the lightest double: real object → fake → stub → mock. Mockito: `@Mock`, `when/thenReturn`, `verify`, matchers, captors; run it with `-javaagent`.

---

## 18.7 AssertJ fluent assertions

### What it is

**AssertJ** is an assertion library: one entry point, `assertThat(actual)`, returns a type-specific object whose methods you chain — auto-complete shows only assertions that make sense for that type. Failure messages are detailed (see 18.8). It works with any test framework; Spring Boot's test **starter** (one dependency bundling related libraries) includes it. (Hamcrest — `assertThat(x, is(y))` — is the older matcher-style alternative.)

**Code — `AssertjStyleTest.java`**

```java
import static org.assertj.core.api.Assertions.*;

@Test
void collections() {
    assertThat(sampleCart().items())
            .hasSize(2)
            .extracting(Item::sku)
            .containsExactly("BK-1", "MUG");
}

@Test
void bigDecimalsCompareByValue() {
    assertThat(sampleCart().subtotal())
            .isEqualByComparingTo("102.5")               // ignores scale: 102.5 == 102.50
            .isGreaterThan(BigDecimal.ZERO);
}

@Test
void exceptions() {
    assertThatThrownBy(() -> new Item("X", "x", BigDecimal.ONE, 0))
            .isInstanceOf(IllegalArgumentException.class)
            .hasMessage("quantity must be positive: 0");
    assertThatIllegalArgumentException().isThrownBy(() -> Discounts.apply(BigDecimal.ONE, 200));
}

@Test
void recursiveComparisonIgnoringAField() {
    Order actual = new Order("ORD-42", "ann", new BigDecimal("9.99"), Instant.now());
    Order expected = new Order("ignored", "ann", new BigDecimal("9.99"), Instant.EPOCH);
    assertThat(actual).usingRecursiveComparison().ignoringFields("id", "placedAt").isEqualTo(expected);
}

@Test
void softAssertionsCollectAllFailures() {
    Cart cart = sampleCart();
    SoftAssertions.assertSoftly(soft -> {
        soft.assertThat(cart.count()).isEqualTo(3);
        soft.assertThat(cart.isEmpty()).isFalse();
        soft.assertThat(cart.items()).allSatisfy(i -> assertThat(i.quantity()).isPositive());
    });
}

@Test
void writesCsvIntoATemporaryDirectory(@TempDir Path dir) throws IOException {   // deleted after the test
    Order o = new Order("ORD-1", "ann", new BigDecimal("76.88"), Instant.parse("2025-09-16T10:00:00Z"));
    Path csv = OrderExporter.writeCsv(List.of(o), dir);
    assertThat(csv).exists().hasFileName("orders.csv");
    assertThat(Files.readAllLines(csv)).containsExactly(
            "id,customer,total,placed_at",
            "ORD-1,ann,76.88,2025-09-16T10:00:00Z");
}
```

All six passed (tree in 18.4). The last test uses JUnit's `@TempDir`: a real, empty directory per test, deleted afterwards — the right way to test file code (Chapter 13) without touching your disk.

### JUnit assertions vs AssertJ

| Check | JUnit `Assertions` | AssertJ |
|---|---|---|
| equality | `assertEquals(3, n)` | `assertThat(n).isEqualTo(3)` |
| list order and content | `assertEquals(List.of(...), list)` | `assertThat(list).containsExactly(...)` / `containsExactlyInAnyOrder` / `contains` |
| property of each element | loop | `extracting(Item::sku)`, `filteredOn(...)`, `allSatisfy(...)` |
| `BigDecimal` by value | `assertEquals(0, a.compareTo(b))` | `isEqualByComparingTo("102.5")` |
| exception and message | `assertThrows` then `assertEquals(msg, e.getMessage())` | `assertThatThrownBy(...).isInstanceOf(...).hasMessage(...)` |
| many checks, one report | `assertAll(...)` | `SoftAssertions.assertSoftly(...)` |
| object graph field by field | manual | `usingRecursiveComparison()` |
| strings | `assertTrue(s.startsWith("x"))` | `assertThat(s).startsWith("x").contains("y")` |
| files | manual | `assertThat(path).exists().hasContent(...)` |
| description | message argument | `.as("subtotal of %s", cart)` before the assertion |

### Common mistakes

- `assertThat(x);` with nothing chained — asserts nothing. (**Static analysis** tools — which inspect source for bugs without running it — and AssertJ's own `@CheckReturnValue` catch this.)
- Calling `.as("…")` **after** the assertion — too late; it must come first.
- `isEqualTo` on `BigDecimal` when scale doesn't matter — use `isEqualByComparingTo`.

### TL;DR

`assertThat(actual).<fluent checks>` — readable, type-aware, great messages. Especially strong for collections, exceptions, `BigDecimal`, and object graphs.

---

## 18.8 Reading a failing test report

A test suite is only useful if its failures are quick to understand. `FailingDemoTest` fails **on purpose** — it carries `@Tag("demo-failure")`, which every normal run excludes. `bash code/run_tests.sh fail` runs only it:

```
@Tag("demo-failure")
@ExtendWith(MockitoExtension.class)
class FailingDemoTest {
    @Mock PaymentGateway gateway;

    @Test
    void bigDecimalEqualsChecksScale() {
        // BigDecimal.equals compares value AND scale: 102.5 is not equal to 102.50
        assertEquals(new BigDecimal("102.5"), AssertjStyleTest.sampleCart().subtotal());
    }

    @Test
    void assertjListDiff() {
        List<String> skus = AssertjStyleTest.sampleCart().items().stream().map(Item::sku).toList();
        Assertions.assertThat(skus).containsExactly("MUG", "BK-1", "PEN");
    }

    @Test
    void groupedAssertionsReportEveryFailure() {
        Cart cart = AssertjStyleTest.sampleCart();
        assertAll("cart",
                () -> assertEquals(2, cart.count(), "count"),
                () -> assertEquals(2, cart.items().size(), "lines"),
                () -> assertEquals("100.00", cart.subtotal().toPlainString(), "subtotal"));
    }

    @Test
    void unusedStubIsReported() {
        // Strict stubs: a stubbing the code under test never uses is a smell (wrong argument? dead setup?)
        Mockito.when(gateway.charge("ann", BigDecimal.TEN)).thenReturn(new PaymentGateway.PaymentResult(true,
"x"));
        Assertions.assertThat(AssertjStyleTest.sampleCart().count()).isEqualTo(3);
    }
}
```

Real output (exit code 1; stack frames trimmed where marked):

```
|
├─ JUnit Platform Suite ✔
├─ JUnit Jupiter ✔
│  └─ FailingDemoTest ✔
│     ├─ bigDecimalEqualsChecksScale() ✘ expected: <102.5> but was: <102.50>
│     ├─ groupedAssertionsReportEveryFailure() ✘ cart (2 failures)
│     │         org.opentest4j.AssertionFailedError: count ==> expected: <2> but was: <3>
│     │         org.opentest4j.AssertionFailedError: subtotal ==> expected: <100.00> but was: <102.50>
│     ├─ unusedStubIsReported() ✘
│     │        Unnecessary stubbings detected.
│     │        Clean & maintainable test code requires zero unnecessary code.
│     │        Following stubbings are unnecessary (click to navigate to relevant line of code):
│     │          1. -> at ch18.shop.FailingDemoTest.unusedStubIsReported(FailingDemoTest.java:47)
│     │        Please remove unnecessary stubbings or use 'lenient' strictness. More info: javadoc for
UnnecessaryStubbingException class.
│     └─ assertjListDiff() ✘
│              Expecting actual:
│                ["BK-1", "MUG"]
│              to contain exactly (and in same order):
│                ["MUG", "BK-1", "PEN"]
│              but could not find the following elements:
│                ["PEN"]
└─ JUnit Vintage ✔

Failures (4):
  JUnit Jupiter:FailingDemoTest:bigDecimalEqualsChecksScale()
    MethodSource [className = 'ch18.shop.FailingDemoTest', methodName = 'bigDecimalEqualsChecksScale',
methodParameterTypes = '']
    => org.opentest4j.AssertionFailedError: expected: <102.5> but was: <102.50>
       org.junit.jupiter.api.Assertions.assertEquals(Assertions.java:1199)
       ch18.shop.FailingDemoTest.bigDecimalEqualsChecksScale(FailingDemoTest.java:26)
  ... (the other three repeat the messages above, each with its stack frames)

Test run finished after 67 ms
...
[         4 tests found           ]
[         0 tests successful      ]
[         4 tests failed          ]
```

How to read each one:

1. **`expected: <102.5> but was: <102.50>`** — looks equal, isn't: `BigDecimal.equals` compares value *and scale* (Chapter 7 §7.7). The last frame from *your* code (`FailingDemoTest.java:26`) is the line to open.
2. **`cart (2 failures)`** — `assertAll` ran all three checks and reports the two that failed (`count`, `subtotal`), each with its message label; the `lines` check passed.
3. **Unnecessary stubbings** — the test stubbed `charge("ann", 10)` but never called it. In real tests this usually means the stub's arguments don't match what the code passes, so the code got `null` instead. Mockito tells you the exact line.
4. **AssertJ's diff** prints actual, expected and exactly what's missing (and, when relevant, what's unexpected or out of order).

Build tools report the same information: Surefire writes XML/text reports under `target/surefire-reports/`, Gradle an HTML report at `build/reports/tests/test/index.html`; CI servers read the XML.

### TL;DR

Read the first line (expected vs actual), then the first stack frame in your own code. Use `assertAll`/soft assertions and AssertJ to get complete, precise messages.

---

## 18.9 Test design

### What makes a good unit test — FIRST

| Letter | Property | How the tests here achieve it |
|---|---|---|
| **F**ast | milliseconds, so you run them constantly | no network, no DB, no sleeps: 47 tests in ~0.6 s |
| **I**ndependent | any order, any subset | fresh instance per test, `@BeforeEach` setup |
| **R**epeatable | same result everywhere, every time | fixed `Clock`, fixed ids, `@TempDir` |
| **S**elf-validating | pass/fail without reading output | assertions, not `println` |
| **T**imely | written with (or before) the code | — |

### Structure: Arrange–Act–Assert

Every test has three blocks — **arrange** (inputs, doubles), **act** (one call to the code under test), **assert** (check outcome) — the same idea as BDD's *given–when–then*. `approvedPaymentSavesTheOrder` above is commented that way. One *behaviour* per test; several assertions about that one behaviour are fine.

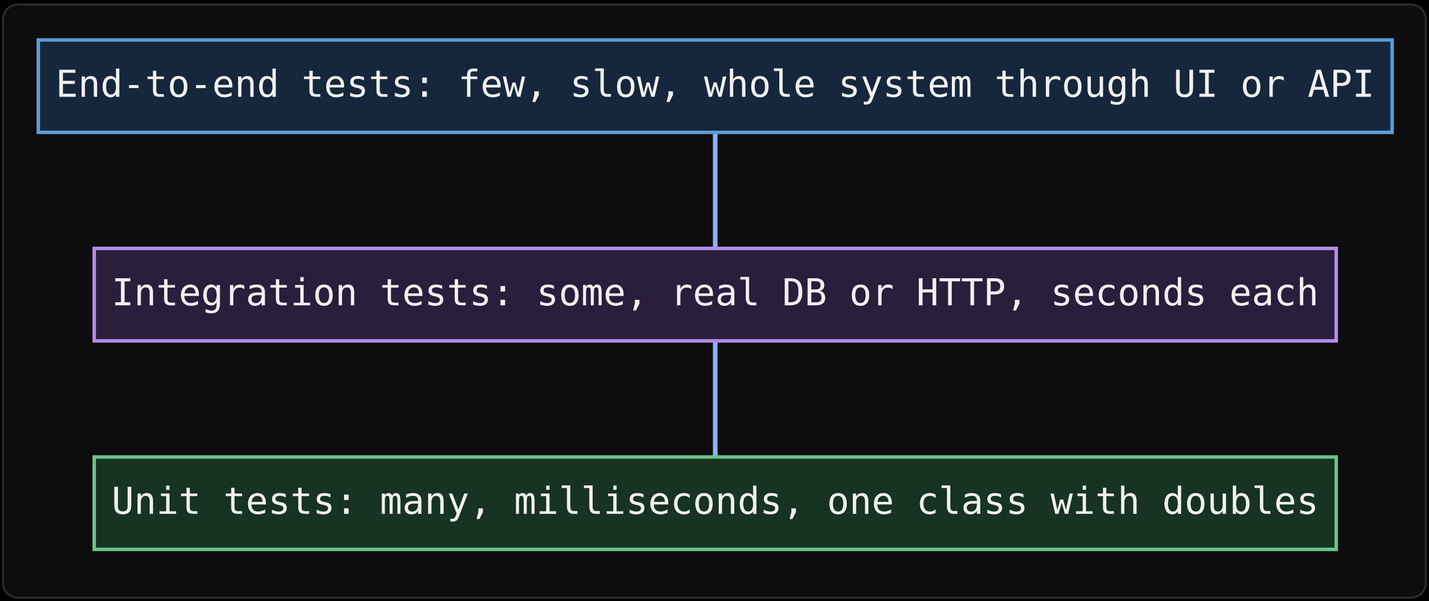
### Naming

Name tests after behaviour, not methods: `declinedPaymentThrowsAndSavesNothing` beats `testCheckout2`. A failing test's name should already tell you what broke. `@DisplayName` allows full sentences.

### Choosing inputs

- **Equivalence classes**: one representative per group that behaves the same (a valid code, a lowercase code, a too-short code).
- **Boundaries**: just below, at, just above each limit (`99.99`, `100.00` for "SAVE25 needs at least 100"; `-1`, `0`, `100`, `101` for percentages).
- **Special values**: `null`, empty, zero, negative, very large, duplicates, non-ASCII Unicode (accents, emoji).
- **Errors**: each exception path, including exceptions thrown by collaborators (`gatewayOutagePropagates`).

### The test pyramid


End-to-end tests: few, slow, whole system through UI or API
Integration tests: some, real DB or HTTP, seconds each
Unit tests: many, milliseconds, one class with doubles

Most tests should be fast unit tests; fewer **integration tests** exercise real infrastructure (a real database, real HTTP); a handful of **end-to-end** tests cover whole user journeys. **Testcontainers** (2.x at the time of writing) starts real databases/**brokers** (Kafka, RabbitMQ) in **Docker** containers for integration tests — the modern replacement for in-memory lookalikes such as H2, whose SQL **dialect** differs from production (Testcontainers gives the same engine and dialect). Maven runs integration tests with the **Failsafe** plugin (`*IT.java`, `verify` phase); Gradle typically uses a separate test **source set** (a named group of source folders with its own compile task) or the JVM Test Suite plugin.

### Coverage — useful, but not a goal

**Code coverage** measures which lines/**branches** (each path through an `if`/`switch`) the tests executed; **JaCoCo** is the standard Java tool (Maven plugin `jacoco-maven-plugin`, Gradle plugin `jacoco`), producing an HTML report. Low coverage reliably shows *untested* code; high coverage does **not** prove the tests check anything (90% line coverage only means 90% of lines ran; a test without assertions still "covers" lines). **Mutation testing** (PIT/pitest) is stronger: it makes small changes to your code (flip `>=` to `>`) and checks that some test fails — our `99.99`/`100.00` rows would kill that **mutant** (the altered copy).

### Flaky tests

A **flaky** test sometimes passes and sometimes fails without code changes. Common causes and cures:

| Cause | Cure |
|---|---|
| Current time, time zones | inject `Clock`; fixed zone |
| Randomness | inject `Random` with a seed, or a `Supplier` |
| Test order / shared static state | fresh state per test; no static mutables |
| `Thread.sleep` to "wait" for async work | wait for a condition (Awaitility library, latches — Chapter 14) |
| Real network or external services | doubles in unit tests; Testcontainers/WireMock (a fake HTTP server) in integration tests |
| Hash-order assumptions (`HashMap` iteration) | assert with `containsExactlyInAnyOrder` or sort first |

### TDD in one paragraph

**Test-driven development** repeats a short cycle: write a failing test (**red**), write the simplest code that passes (**green**), clean up (**refactor**) with the tests as a safety net. It keeps designs small and testable; many teams use it selectively (bug fixes: first write the test that reproduces the bug). See also LLD — refactoring with tests as a safety net.

### In the interview

> **30-second answer:** "Good unit tests are fast, independent, repeatable and self-checking. I structure them arrange–act–assert, name them after behaviour, and pick inputs by equivalence classes and boundaries. I keep most tests at unit level with injected doubles, add integration tests against real infrastructure with Testcontainers, and treat coverage as a way to find untested code, not a target."

- *How do you test time-dependent code?* → Inject a `Clock`.
- *Is 100% coverage worth it?* → No; diminishing returns and it can't show assertion quality — mutation testing is a better signal.
- *What's a flaky test and how do you fix one?* → Nondeterministic outcome; remove the hidden input (time, randomness, order, network, sleeps).

### TL;DR

FIRST, AAA, behaviour names, boundary inputs, mostly unit tests, real infrastructure for integration tests, no hidden time/random/order dependencies.

---

# 18.10 Logging

### What it is

**Logging** records what a running program does, each record with a **level**, a **logger name** (usually the class) and a timestamp, sent to destinations (console, file, a central **log collector**) chosen by **configuration**, not code. `System.out.println` has no levels, no way to turn it down in production, no timestamps and no routing.

### How a logging system works

1. Code asks for a **logger** named after its class: `Logger.getLogger(OrderService.class.getName())`.
2. Loggers form a **hierarchy** by name: `ch18.shop.OrderService` → `ch18.shop` → `ch18` → root. A logger without its own level **inherits** the nearest ancestor's.
3. Each call has a **level**; if it is below the logger's effective level, the call is discarded cheaply.
4. Accepted records go to **handlers/appenders** (console, file…), which have their own level and a **formatter/layout**.

### Levels across the libraries

| `java.util.logging` | SLF4J / Logback / Log4j 2 | `System.Logger` | Use for |
|---|---|---|---|
| `SEVERE` | `ERROR` | `ERROR` | failures needing attention |
| `WARNING` | `WARN` | `WARNING` | unexpected but handled |
| `INFO` | `INFO` | `INFO` | important business/lifecycle events |
| `CONFIG` | — | — | configuration details (JUL only) |
| `FINE` | `DEBUG` | `DEBUG` | diagnostic detail |
| `FINER`, `FINEST` | `TRACE` | `TRACE` | very fine detail |

### `java.util.logging` (JUL), run — `code/ch18/logging/JulDemo.java`

JUL is built into the JDK (module `java.logging`), so it needs no dependency. By default the root logger is at `INFO` and a `ConsoleHandler` writes to **System.err** (settings in `$JAVA_HOME/conf/logging.properties`).

```
// What you would normally put in a logging.properties file (-Djava.util.logging.config.file=...)
static final String CONFIG = """
        handlers = java.util.logging.ConsoleHandler
        .level = INFO
        java.util.logging.ConsoleHandler.level = ALL
        java.util.logging.SimpleFormatter.format = %4$-7s [%3$s] %5$s%6$s%n
        ch18.logging.level = FINE
        """;

public static void main(String[] args) throws IOException {
    System.setErr(System.out);                        // ConsoleHandler writes to System.err; merge for the demo
    LogManager.getLogManager().readConfiguration(
            new ByteArrayInputStream(CONFIG.getBytes(StandardCharsets.UTF_8)));
    // Normally a static final field; created AFTER the configuration here because the demo configures in code.
    Logger log = Logger.getLogger(JulDemo.class.getName());                     // "ch18.logging.JulDemo"

    System.out.println("-- 1. levels (ch18.logging is set to FINE, the root to INFO)");
    log.severe("disk full");
    log.warning("slow response: 1200 ms");
    log.info("server started on port 8080");
    log.config("pool size = 10");
    log.fine("cache miss for key 42");
    log.finer("not shown: FINER is below FINE");

    System.out.println("-- 2. another package inherits the root level INFO");
    Logger other = Logger.getLogger("com.acme.billing");
    other.info("billing ready");
    other.fine("not shown: com.acme.billing is at INFO");

    System.out.println("-- 3. parameters and lazy messages");
    log.log(Level.INFO, "user {0} bought {1} items", new Object[]{"ann", 3});   // MessageFormat style
    int[] calls = {0};
    Logger quiet = Logger.getLogger("com.acme.quiet");
    quiet.fine(() -> "expensive " + (++calls[0]));   // Supplier: not evaluated, FINE is disabled here
    quiet.fine("eager " + (++calls[0]));               // string built anyway, then thrown away
    System.out.println("message expressions evaluated: " + calls[0]);
    System.out.println("isLoggable(FINE) = " + quiet.isLoggable(Level.FINE));

    System.out.println("-- 4. logging an exception (message + stack trace)");
    try { Integer.parseInt("12x"); }
    catch (NumberFormatException e) { log.log(Level.WARNING, "bad quantity in request", e); }

    System.out.println("-- 5. a custom handler with its own format");
    Logger audit = Logger.getLogger("ch18.audit");
    audit.setUseParentHandlers(false);                // don't ALSO send to the root's ConsoleHandler
    Handler h = new StreamHandler(System.out, new java.util.logging.Formatter() {
        @Override public String format(LogRecord r) {
            return "AUDIT|" + r.getLevel() + "|" + r.getLoggerName() + "|" + formatMessage(r) + "\n";
        }
    });
    audit.addHandler(h);
    audit.info("ann changed her e-mail");
    h.flush();

    System.out.println("-- 6. System.Logger (Java 9): a facade that routes to JUL by default");
    System.Logger sys = System.getLogger("ch18.logging.system");
    sys.log(System.Logger.Level.INFO, "hello from System.Logger, {0}", "ann");
    sys.log(System.Logger.Level.DEBUG, "DEBUG maps to FINE -> shown, ch18.logging is FINE");
}
```

Output:

```
-- 1. levels (ch18.logging is set to FINE, the root to INFO)
SEVERE  [ch18.logging.JulDemo] disk full
WARNING [ch18.logging.JulDemo] slow response: 1200 ms
INFO    [ch18.logging.JulDemo] server started on port 8080
CONFIG  [ch18.logging.JulDemo] pool size = 10
FINE    [ch18.logging.JulDemo] cache miss for key 42
-- 2. another package inherits the root level INFO
INFO    [com.acme.billing] billing ready
-- 3. parameters and lazy messages
INFO    [ch18.logging.JulDemo] user ann bought 3 items
message expressions evaluated: 1
isLoggable(FINE) = false
-- 4. logging an exception (message + stack trace)
WARNING [ch18.logging.JulDemo] bad quantity in request
java.lang.NumberFormatException: For input string: "12x"
    at java.base/java.lang.NumberFormatException.forInputString(NumberFormatException.java:67)
    at java.base/java.lang.Integer.parseInt(Integer.java:527)
    at java.base/java.lang.Integer.parseInt(Integer.java:624)
    at ch18.logging.JulDemo.main(JulDemo.java:56)

-- 5. a custom handler with its own format
AUDIT|INFO|ch18.audit|ann changed her e-mail
-- 6. System.Logger (Java 9): a facade that routes to JUL by default
INFO    [ch18.logging.system] hello from System.Logger, ann
FINE    [ch18.logging.system] DEBUG maps to FINE -> shown, ch18.logging is FINE
```

What it shows:

- **Levels and inheritance**: `ch18.logging.level = FINE` applies to every logger below `ch18.logging`; `com.acme.billing` has no setting, so it inherits the root's `INFO`.
- **Two filters**: a record must pass the *logger's* level and then the *handler's* level (`ConsoleHandler.level = ALL` here; its default is `INFO`). JUL's default `ConsoleHandler` writes to `System.err`.
- **Formatter placeholders** in `SimpleFormatter.format` (a `String.format` pattern): `%1$` date/time, `%2$` source, `%3$` logger name, `%4$` level, `%5$` message, `%6$` exception with stack trace.
- **Lazy messages**: the `Supplier` overload skipped the string building; the plain overload built a string that was then discarded ("evaluated: 1"). Use suppliers (or `isLoggable` guards) for expensive messages.
- **Exceptions**: pass the exception as the last argument — you get the message *and* the stack trace (Chapter 8 §8.9).
- **Configuration timing**: levels from a configuration file are applied to loggers as they are created; configure logging at start-up (`-Djava.util.logging.config.file=logging.properties`) before your classes create loggers — that's why this demo, which configures in code, creates its logger afterwards.
- **`System.Logger`** (Java 9) is a tiny **facade** (a thin front-end API forwarding to whichever implementation is installed) in `java.base`; without other setup it forwards to JUL.

### SLF4J + Logback: what most applications use (not run)

Real applications usually code against **SLF4J** (Simple Logging Facade for Java, 2.0.x) — an API only — and pick a **backend** (the library that actually writes the logs) at deployment: **Logback** (Spring Boot's default) or **Log4j 2**. Libraries log through SLF4J so the *application* decides where output goes. **Bridges** (adapter JARs) route JUL, Log4j 1, and Commons Logging calls into SLF4J so one configuration controls everything.

```java
// (SLF4J — not run: needs slf4j-api plus a backend such as logback-classic on the class path)
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.slf4j.MDC;

public final class OrderService {
    private static final Logger log = LoggerFactory.getLogger(OrderService.class);

    public Order checkout(String customer, Cart cart, String code) {
        MDC.put("customer", customer);                                   // added to every line on this thread
        try {
            log.debug("checkout started, {} items", cart.count());       // {} placeholders: no string built if DEBUG is off
            // ...
            log.info("order placed id={} total={}", order.id(), total);
            log.atInfo().addKeyValue("orderId", order.id()).log("order placed");  // SLF4J 2 fluent API: structured field
            return order;
        } catch (RuntimeException e) {
            log.error("checkout failed for {}", customer, e);            // exception LAST, no placeholder for it
            throw e;
        } finally {
            MDC.remove("customer");
        }
    }
}
```

```xml
<!-- src/main/resources/logback.xml (not run) -->
<configuration>
  <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
    <encoder><pattern>%d{HH:mm:ss.SSS} %-5level [%thread] %logger{36} %X{customer} - %msg%n</pattern></encoder>
  </appender>
  <logger name="com.example.shop" level="DEBUG"/>
  <root level="INFO"><appender-ref ref="CONSOLE"/></root>
</configuration>
```

- `{}` **placeholders** replace string concatenation and are cheap when the level is off.
- **MDC** (Mapped Diagnostic Context) attaches key/values (request id, user) to every log line of the current thread — `%X{customer}` prints it. (With virtual threads and thread pools, always clear it in `finally`.)
- **Structured logging** (JSON lines with fields) makes logs searchable in tools such as Elasticsearch/OpenSearch, Loki or Splunk.

## Common mistakes

- Logging and rethrowing the same exception at every layer — one failure, five stack traces. Log once, where it's handled.
- Logging secrets or personal data (passwords, tokens, card numbers).
- String concatenation in hot debug logs (`log.debug("x=" + expensive())`).
- Several backends on the class path — SLF4J warns and picks one; keep exactly one.
- Treating the logging library as harmless: **Log4Shell** (CVE-2021-44228, December 2021) let attackers run code through message lookups in Log4j 2 versions before 2.15.0 — keep logging libraries patched like any other dependency.

## In the interview

> **30-second answer:** "I code against SLF4J with a backend like Logback, one logger per class, levels ERROR to TRACE, parameterized `{}` messages, exceptions passed as the last argument, MDC for request ids, configuration outside the code. The JDK has java.util.logging and the `System.Logger` facade, fine for small tools."

- *Why a facade?* → Libraries shouldn't force a backend; the application chooses one, and bridges unify everything.

- *Why `{}` instead of concatenation?* → No string is built when the level is disabled.

**TL;DR**

Loggers per class, levels, hierarchy, handlers/appenders, config outside code. JUL is built in; SLF4J + Logback/Log4j 2 is the industry default.

---

## 18.11 JDBC basics

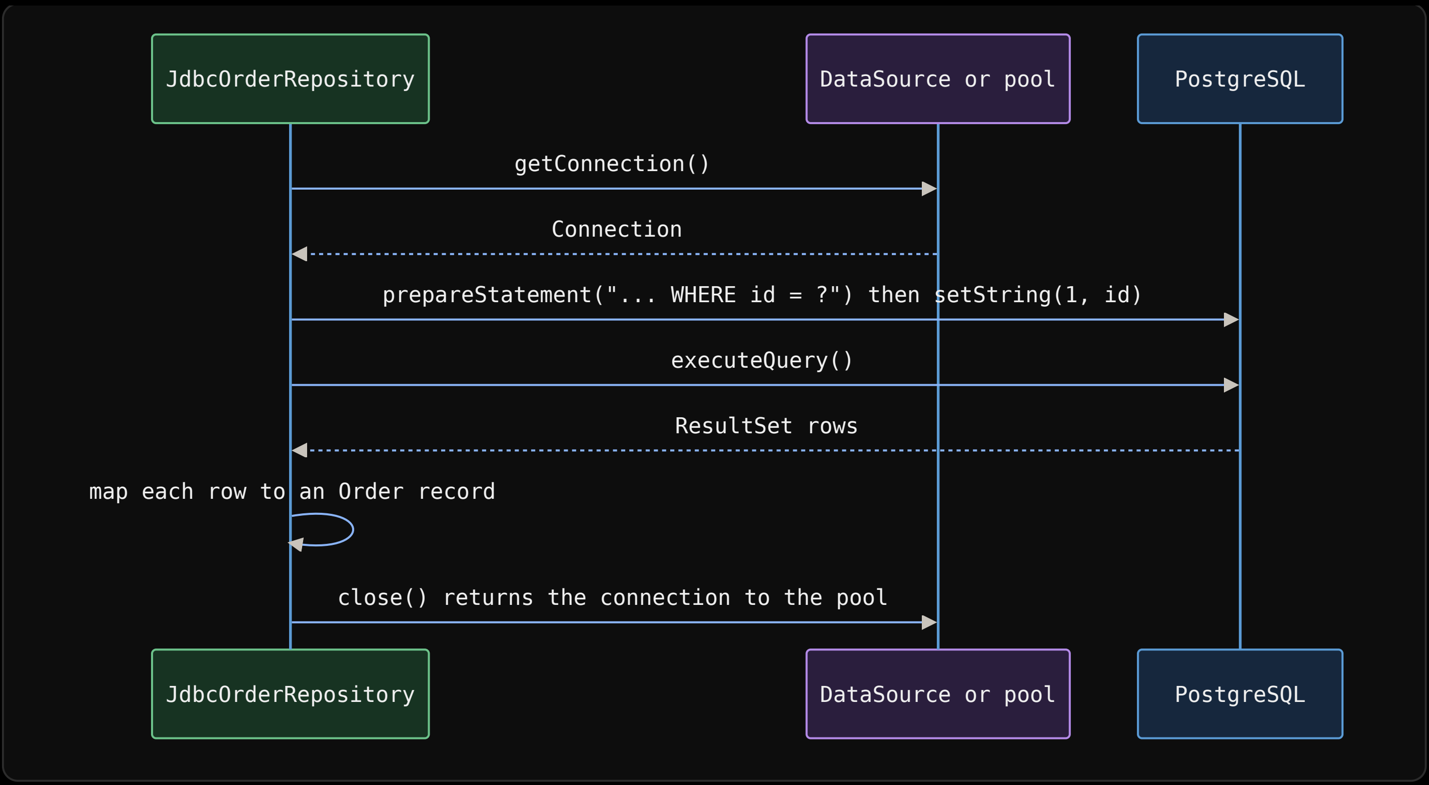
### What it is

**JDBC** (Java Database Connectivity, packages `java.sql` and `javax.sql`) is the standard API for **relational databases** (tables of rows and columns, queried with SQL). Your code uses JDBC **interfaces**; a vendor's **driver** JAR (here PostgreSQL's `postgresql-42.7.13.jar`) implements them. Since JDBC 4.0 (Java 6) drivers register themselves through the **service loader** — the old `Class.forName("org.postgresql.Driver")` line is unnecessary. Every higher-level tool (Spring's `JdbcTemplate`, jOOQ, Hibernate/JPA — the Java Persistence API) is built on JDBC.

The SQL Handbook's Chapter 14 runs a longer JDBC program against the same private PostgreSQL (SQL injection demonstrated, batches, isolation levels); this section focuses on the Java side: resource handling, mapping rows to records, transactions, and hiding JDBC behind the `OrderRepository` interface so the rest of the app — and its tests — don't care.

### The pieces

| Type | Role |
|---|---|
| `DriverManager` | gets a `Connection` from a URL — simple, for scripts and tools |
| `DataSource` | a connection factory configured once — what applications inject; a **connection pool** (reusable open connections — HikariCP, Spring Boot's default) implements it |
| `Connection` | one session with the database; owns transactions |
| `PreparedStatement` | SQL with `?` placeholders, values bound by index (from **1**) with typed setters |
| `ResultSet` | a **cursor** over result rows (one row at a time): `next()`, then `getString("col")`, `getBigDecimal`, `getObject(col, LocalDate.class)` |
| `SQLException` | **checked** exception; `getSQLState()` gives a standard 5-character code (`23505` = unique violation) |


JdbcOrderRepository
DataSource or pool
PostgreSQL
getConnection()
Connection
prepareStatement("... WHERE id = ?") then setString(1, id)
executeQuery()
ResultSet rows
map each row to an Order record
close() returns the connection to the pool
JdbcOrderRepository
DataSource or pool
PostgreSQL

### The repository — `code/ch18/jdbc/JdbcOrderRepository.java`

```java
public final class JdbcOrderRepository implements OrderRepository {
    private final DataSource dataSource;
    public JdbcOrderRepository(DataSource dataSource) { this.dataSource = dataSource; }

    @Override
    public void save(Order o) {
        String sql = "INSERT INTO orders (id, customer, total, placed_at) VALUES (?, ?, ?, ?)";
        try (Connection c = dataSource.getConnection();
             PreparedStatement ps = c.prepareStatement(sql)) {
            bind(ps, o);
            ps.executeUpdate();
        } catch (SQLException e) {
            throw new RepositoryException("save failed for " + o.id(), e);
        }
    }

    /** All or nothing: one transaction for the whole list. */
    public void saveAll(List<Order> orders) {
        String sql = "INSERT INTO orders (id, customer, total, placed_at) VALUES (?, ?, ?, ?)";
        try (Connection c = dataSource.getConnection()) {
            c.setAutoCommit(false);                           // start a transaction
            try (PreparedStatement ps = c.prepareStatement(sql)) {
                for (Order o : orders) { bind(ps, o); ps.addBatch(); }
                ps.executeBatch();
                c.commit();
            } catch (SQLException e) {
                c.rollback();                                 // undo every insert of this call
                throw e;
            }
        } catch (SQLException e) {
            throw new RepositoryException("saveAll failed", e);
        }
    }

    private List<Order> query(String sql, String param) {
        try (Connection c = dataSource.getConnection();
             PreparedStatement ps = c.prepareStatement(sql)) {
            ps.setString(1, param);                           // bound parameter: never concatenate user input
            try (ResultSet rs = ps.executeQuery()) {
                List<Order> out = new ArrayList<>();
                while (rs.next()) out.add(map(rs));
                return out;
            }
        } catch (SQLException e) {
            throw new RepositoryException("query failed: " + sql, e);
        }
    }

    private static Order map(ResultSet rs) throws SQLException {       // row -> record
        return new Order(rs.getString("id"), rs.getString("customer"),
                rs.getBigDecimal("total"), rs.getTimestamp("placed_at").toInstant());
    }
    // ... findById / findByCustomer call query(...); bind(...) sets the four parameters;
    //     RepositoryException is an unchecked wrapper for SQLException
}
```

What to notice:

1. **try-with-resources** closes `ResultSet`, `PreparedStatement`, then `Connection`, even on exceptions (Chapter 8 §8.6). With a pool, `close()` returns the connection instead of disconnecting.
2. **Always `PreparedStatement` with `?`** for values — the driver sends them separately from the SQL text, which prevents **SQL injection** (input crafted to change the SQL command itself) and lets the database reuse the query **plan**.
3. **Transactions**: a connection starts in **auto-commit** mode (each statement is its own committed transaction). For several statements that must succeed together, `setAutoCommit(false)`, then `commit()` or `rollback()` (undo everything since the transaction began).

4. **Exception translation** (Chapter 8 §8.8): the checked `SQLException` is wrapped, with the cause kept, so the `OrderRepository` interface stays storage-neutral.
5. The class names only `java.sql`/`javax.sql` types — the driver is a **runtime** dependency (`runtime` scope / `runtimeOnly`).

### Running it — `code/ch18/jdbc/JdbcDemo.java`

The demo creates a database `ch_18_jdbc` on the SQL Handbook's private server (port 5544, reachable only through a **Unix socket** — a local inter-process connection that looks like a file — hence the `junixsocket` socket-factory parameters in the URL; against a normal server the URL is just `jdbc:postgresql://localhost:5432/shop`), creates the table, and uses the repository through a `PGSimpleDataSource`:

```java
PGSimpleDataSource ds = new PGSimpleDataSource();          // the driver's simple, non-pooled DataSource
ds.setURL(BASE + "ch_18_jdbc" + SOCKET);
ds.setUser("postgres");
JdbcOrderRepository repo = new JdbcOrderRepository(ds);
repo.save(new Order("ORD-1", "ann", new BigDecimal("76.88"), t));
repo.saveAll(List.of(/* ORD-2 for bob, ORD-3 for ann */));
System.out.println("findById(ORD-1)   = " + repo.findById("ORD-1").orElseThrow());
// ... findById("ORD-404"), findByCustomer("ann"), then saveAll(ORD-4, duplicate ORD-1) inside try/catch,
//     printing the SQLState of the cause, and finally whether ORD-4 exists
```

Commands (from the repository root) and real output:

```
$ bash tools_pg.sh start
postgres ready on port 5544 (socket /tmp)
$ javac --release 25 -cp '.java-books/lib/*' -d /tmp/java18o $(find java/chapters/18-testing-build-and-tooling/code/ch18 -name '*.java')
$ java --enable-native-access=ALL-UNNAMED -cp "/tmp/java18o:.java-books/lib/*" ch18.jdbc.JdbcDemo
driver: PostgreSQL JDBC Driver 42.7.13
findById(ORD-1)   = Order[id=ORD-1, customer=ann, total=76.88, placedAt=2025-09-16T10:00:00Z]
findById(ORD-404) = Optional.empty
ann's orders      = [ORD-1, ORD-3]
saveAll failed: SQLState 23505 (BatchUpdateException)
ORD-4 present after rollback? false
```

(`--enable-native-access=ALL-UNNAMED` only silences the JDK's warning about the socket library loading native code — without it the program prints four `WARNING: A restricted method…` lines first.) The rollback worked: the valid `ORD-4` was undone together with the failing duplicate. `BatchUpdateException` is the `SQLException` subclass thrown by `executeBatch`.

### Java ↔ SQL types

| SQL (PostgreSQL) | Java | Getter / setter |
|---|---|---|
| `text`, `varchar` | `String` | `getString` / `setString` |
| `integer`, `bigint` | `int`, `long` (use `wasNull()` for SQL NULL) or `Integer`/`Long` via `getObject(col, Integer.class)` | `getInt`, `getLong` |
| `numeric` | `BigDecimal` — never `double` for money | `getBigDecimal` / `setBigDecimal` |
| `boolean` | `boolean` | `getBoolean` |
| `date`, `time` | `LocalDate`, `LocalTime` (JDBC 4.2) | `getObject(col, LocalDate.class)` / `setObject` |
| `timestamp` | `LocalDateTime` | `getObject(col, LocalDateTime.class)` |
| `timestamptz` | `OffsetDateTime` (pgjdbc does not accept `Instant` in `setObject`); `Timestamp.from(instant)` also works, as above | `getObject(col, OffsetDateTime.class)` |

### Testing database code

- **Unit tests** of services use the fake `InMemoryOrderRepository` — the reason the interface exists.
- **Integration tests** of `JdbcOrderRepository` itself need a real database: Testcontainers starts PostgreSQL in Docker, the test creates the **schema**, runs the repository, and checks rows. Mocking `Connection`/`ResultSet` tests nothing useful.

### When not to write raw JDBC

For a handful of queries, raw JDBC is fine. For a real application, use a helper that removes the **boilerplate**: **Spring `JdbcClient`/`JdbcTemplate`** (SQL stays visible, mapping is a lambda), **jOOQ** (type-safe SQL builder), or **JPA/Hibernate** (an **ORM**, *object-relational mapper*: maps objects to tables and generates SQL — powerful, but hides queries; watch for the **N+1 problem**: one query for a list, then one extra query per item). The SQL book compares ORMs vs SQL in its Chapter 14.

### Common mistakes

- Concatenating user input into SQL.
- Not closing connections — the pool runs dry and the app hangs.
- `getInt` on a NULL column returns `0`; check `wasNull()` or use `getObject(col, Integer.class)`.
- Using `double` for money columns.
- Forgetting `rollback()` (or `setAutoCommit(true)` before returning a connection to a pool that doesn't reset it).

### In the interview

> **30-second answer:** "JDBC is the standard API; a driver implements it. Apps get connections from a pooled `DataSource`, use `PreparedStatement` with bound parameters to avoid SQL injection, map `ResultSet` rows to objects, close everything with try-with-resources, and control transactions with `setAutoCommit(false)`, `commit` and `rollback`. I hide it behind a repository interface and test it against a real database with Testcontainers."

- *`Statement` vs `PreparedStatement`?* → Prepared: parameters bound separately (safe, typed, plan reuse).
- *Why a connection pool?* → Opening a connection costs a network handshake, authentication and a server process; a pool reuses a bounded number of them.

### TL;DR

`DataSource` → `Connection` → `PreparedStatement` → `ResultSet`, all in try-with-resources; bind parameters; explicit transactions for multi-statement work; hide JDBC behind an interface; test it against a real DB.

---

## 18.12 Packaging: JAR files

### What it is

A **JAR** (Java ARchive) is a ZIP file of `.class` files and resources plus a **manifest**, `META-INF/MANIFEST.MF`, holding metadata (e.g. which class to start). A manifest with a `Main-Class` makes it an **executable JAR**, started with `java -jar`.

### Building one by hand — `code/package_jar.sh`

This is what `jar:jar` (Maven) and the `jar` task (Gradle) do for you. Real run:

```
$ javac --release 25 -d classes $(find src -name '*.java')
$ jar --create --file shop.jar --main-class ch18.shop.ShopDemo -C classes .
$ jar --list --file shop.jar | head -5
META-INF/
META-INF/MANIFEST.MF
ch18/
ch18/shop/
ch18/shop/Cart.class
$ unzip -p shop.jar META-INF/MANIFEST.MF
Manifest-Version: 1.0
Created-By: 26 (Oracle Corporation)
Main-Class: ch18.shop.ShopDemo

$ java -jar shop.jar
items=3 subtotal=102.50
Order[id=ORD-1, customer=ann, total=76.88, placedAt=2025-09-16T10:00:00Z]
Sep 28, 2026 3:37:08 PM ch18.shop.OrderService checkout
WARNING: declined: customer=bob total=899.00
error: payment of 899.00 declined for bob
$ ls -l shop.jar
13629 bytes
```

- `-C classes .` means "change into `classes/` and add everything" — package folders become ZIP paths.
- `--main-class` (short form `-e`) writes the `Main-Class` manifest line; `Created-By` records the JDK that built it (26 here, compiled for 25).
- The `WARNING` lines come from JUL's *default* format (timestamp, source, level) because no custom configuration was loaded — compare 18.10.
- Short, older option spellings still work: `jar cfe shop.jar ch18.shop.ShopDemo -C classes .`, `jar tf shop.jar`.

### Dependencies at run time

`java -jar` **ignores** `-cp` (so `java -cp lib/* -jar app.jar` won't find the libraries); a JAR that needs libraries must say where they are, or contain them:

| Approach | How | Notes |
|---|---|---|
| Plain JAR + class path | `java -cp "shop.jar:lib/*" ch18.shop.ShopDemo` | simplest; ship a `lib/` folder (Gradle's `application` plugin builds exactly this into `distZip` with start scripts) |
| `Class-Path` manifest entry | `Class-Path: lib/postgresql-42.7.13.jar` | relative paths, space-separated |
| **Fat / uber JAR** | Maven Shade plugin, Gradle Shadow plugin | one file with all classes merged; watch for clashing `META-INF/services` files and signatures; still needs an installed JRE (unlike a `jlink` image) |
| **Spring Boot JAR** | `spring-boot-maven-plugin` / Gradle plugin | dependencies kept as nested JARs, loaded by Boot's own launcher |
| **Custom runtime** | `jlink` (Java 9) | a trimmed JDK containing only needed modules — see Chapter 16 §16.12 |
| **Native installer** | `jpackage` (final in Java 16) | `.dmg`/`.pkg`, `.msi`/`.exe`, `.deb`/`.rpm` bundling a runtime |
| **Container image** | Dockerfile, Jib, Cloud Native Buildpacks | the common deployment unit for services |
| **Native executable** | GraalVM Native Image | **ahead-of-time** compiled to machine code, fast start-up; restrictions on reflection |

Other JAR facts: a **multi-release JAR** (Java 9) holds version-specific classes under `META-INF/versions/N/`; a **modular JAR** contains `module-info.class` (Chapter 16, and the OCP modules chapter for exam depth); JARs can be **signed** with `jarsigner`. Libraries are published as a JAR + `.pom` (+ `-sources.jar` and `-javadoc.jar`) to a repository with `mvn deploy` or Gradle's `maven-publish` plugin.

**TL;DR**

JAR = ZIP + manifest. `jar --create --main-class`, run with `java -jar`. For dependencies: class path folder, fat JAR, Boot JAR, or container; `jlink`/`jpackage` to ship a runtime.

---

## 18.13 Other everyday tools

| Need | Tool(s) | One-line note |
|---|---|---|
| IDE | IntelliJ IDEA, Eclipse, VS Code + Java extensions | import the Maven/Gradle build; don't hand-configure class paths (Chapter 1 §1.11) |
| Debugger | IDE debugger (**breakpoints** — where execution pauses to inspect variables — conditional breakpoints, watches, step into/over, evaluate expression); `jdb` on the command line | start the JVM with `-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=*:5005` to attach remotely |
| Quick experiments | `jshell` (Java's REPL), single-file launch `java Foo.java` | Chapter 1 §1.9–1.10 |
| Formatting | google-java-format, Palantir Java Format, Spotless (build plugin) | format automatically; stop debating style in reviews |
| Static analysis | compiler warnings (`-Xlint:all`), Error Prone, SpotBugs, PMD, Checkstyle, SonarQube | catch bugs and **smells** (patterns suggesting a design problem) before tests run |
| Null-safety | JSpecify annotations + NullAway | JUnit 6 and Spring Boot 4 both adopted JSpecify |
| Coverage / mutation | JaCoCo / PIT | 18.9 |
| Dependency hygiene | `dependency:tree`, `gradle dependencies`, Dependabot/Renovate (bots opening dependency-update pull requests), OWASP Dependency-Check | keep libraries current and free of known vulnerabilities |
| Documentation | `javadoc` (Markdown doc comments since Java 23 — Chapter 17) | generated HTML API docs |
| Profiling / monitoring | JFR, JMC, VisualVM, async-profiler | Chapter 16 §16.10 |
| Benchmarks | JMH | Chapter 16 §16.9 |
| Continuous integration | GitHub Actions, GitLab CI, Jenkins | run `./mvnw verify` / `./gradlew build` on every push |

A typical CI job is only a few lines — check out the code, set up a JDK, run the wrapper:

```
# .github/workflows/build.yml (GitHub Actions — not run)
name: build
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-java@v5
        with: { distribution: temurin, java-version: '25', cache: maven }
      - run: ./mvnw -B verify
```

---

## 18.14 Where next: Spring Boot and friends

### What it is

Most server-side Java uses a **framework** for dependency injection, configuration, web **endpoints** (URLs your service answers), data access, security, **observability** (logs, metrics, traces) and testing. The most widely used is **Spring Boot** (on the Spring Framework). **Spring Boot 4.0** was released on 20 November 2025 on Spring Framework 7 (4.1 followed in 2026); it runs on Java 17+ and supports Java 25.

### What Spring Boot gives you

- **Dependency injection container**: you write classes like our `OrderService` with constructor parameters; Spring creates them (*beans*) and passes the right collaborators — exactly the design that made 18.6 testable, without the hand wiring in `ShopDemo`.
- **Starters**: one dependency per capability (`spring-boot-starter-webmvc`, `-jdbc`, `-data-jpa`, `-security`, `-actuator`) with compatible versions managed by Boot's BOM.
- **Auto-configuration**: sees a JDBC driver and a URL in `application.properties` → creates a HikariCP `DataSource`; sees the web starter → starts an **embedded** Tomcat (running inside your application, not as a separate server).
- **Executable JAR**: `./mvnw spring-boot:run` in development, `java -jar app.jar` in production.
- **Testing**: the test starter bundles JUnit Jupiter, Mockito and AssertJ (this chapter's tools); `@SpringBootTest` for full context tests, *slice* tests like `@WebMvcTest` and `@DataJdbcTest`, `@MockitoBean` to replace a bean with a mock, and Testcontainers integration.
- **Actuator**: **health checks**, **metrics** (via Micrometer), info endpoints for operations.

```
// (Spring Boot — not run: needs the Spring Boot starters; shows the shape of an app)
@SpringBootApplication                               // component scan + auto-configuration
public class ShopApplication {
    public static void main(String[] args) { SpringApplication.run(ShopApplication.class, args); }
}

@RestController
@RequestMapping("/orders")
class OrderController {
    private final OrderService service;              // injected by Spring through the constructor
    OrderController(OrderService service) { this.service = service; }

    @GetMapping("/{id}")
    Order byId(@PathVariable String id) { return service.find(id).orElseThrow(); }
}
```

```
# src/main/resources/application.properties (not run)
spring.datasource.url=jdbc:postgresql://localhost:5432/shop
spring.datasource.username=shop
logging.level.com.example.shop=DEBUG
```

Generate a project at **start.spring.io** (Spring Initializr): choose Maven or Gradle, Java 25, and the starters you need.

### Alternatives worth knowing

| Framework | In one line |
|---|---|
| **Quarkus** | Kubernetes-oriented, fast start-up, strong GraalVM native-image support; Jakarta EE / MicroProfile APIs |
| **Micronaut** | compile-time dependency injection (no run-time reflection), small **footprint** (memory use) |
| **Jakarta EE** (formerly Java EE) | the standard specifications (Servlet, JPA, CDI, REST) implemented by application servers and runtimes such as WildFly, Open Liberty, Payara |
| **Helidon**, **Javalin**, **Vert.x** | lighter or **reactive** (non-blocking, event-driven) options for services |

### A learning path from here

1. Build a small **REST API** with Spring Boot (web + JDBC or JPA + validation), tested with this chapter's tools and Testcontainers.
2. Learn the design side: LLD Handbook — SOLID, patterns, and the case studies (parking lot, booking systems) — all in Java.
3. Learn the data side: SQL Handbook and *Designing Data-Intensive Applications* notes in `ddia/`.
4. Prepare for interviews: Chapter 19 — Java for coding interviews and the OCP Java 21 study repo for certification-level detail.

> **TL;DR**
>
> Spring Boot = DI container + starters + auto-configuration + executable JAR + first-class testing. Everything in this chapter — constructor injection, JUnit/Mockito/AssertJ, SLF4J logging, JDBC/DataSource, Maven/Gradle, JARs — is what Boot builds on.

---

## Chapter TL;DR

- **Layout & coordinates**: `src/main/java`, `src/test/java`; libraries are `group:artifact:version` from Maven Central, cached locally.
- **Maven**: `pom.xml`; scopes; nearest-wins conflicts fixed by `dependencyManagement`/BOMs; lifecycle validate → … → deploy. Pin plugin versions; use `mvnw`.
- **Gradle**: scripted incremental task graph; `implementation`/`api`/`runtimeOnly`/`testImplementation`; `useJUnitPlatform()`; `gradlew`, toolchains.
- **JUnit (5/6 Jupiter)**: `@Test`, `assertEquals(expected, actual)`, `assertThrows`, `assertAll`; new instance per test; `@Nested`, `@Tag`, `@TempDir`; parameterized tests. JUnit 6 = Java 17 baseline, unified versions, same API.
- **Mockito**: inject collaborators, mock boundaries: `when/thenReturn`, `verify`, matchers (all or none), captors, strict stubs; prefer fakes; attach with `-javaagent`.
- **AssertJ**: `assertThat(x).…` fluent, type-aware, precise failure messages; soft assertions; recursive comparison.
- **Test design**: FIRST, arrange–act–assert, behaviour names, boundaries, pyramid, Testcontainers for integration, coverage ≠ quality, remove time/random/order/network nondeterminism.
- **Logging**: logger per class, levels, hierarchy, handlers; JUL built in; SLF4J + Logback/Log4j 2 in apps; `{}` placeholders, exception last, MDC, no secrets.
- **JDBC**: `DataSource` → `Connection` → `PreparedStatement` → `ResultSet` in try-with-resources; bind parameters; explicit transactions; wrap behind a repository.
- **Packaging**: JAR = ZIP + manifest; `jar --create --main-class`, `java -jar`; fat JARs, Boot JARs, `jlink`, `jpackage`, containers.
- **Next**: Spring Boot builds on all of the above.

Next: Chapter 19 — Java for coding interviews.

# Chapter 19 — Java for Coding Interviews

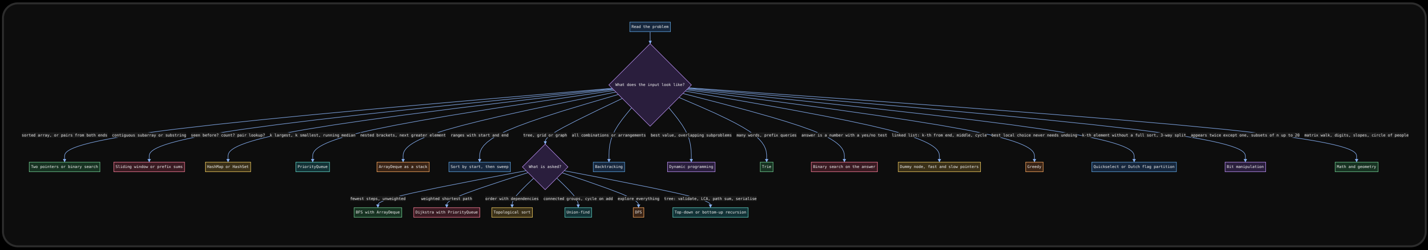
A coding interview (the "LeetCode-style" round) checks two things: do you recognise the classic **data-structure and algorithm (DSA) patterns**, and can you write them correctly in your language under time pressure? Java is a strong interview language — fast, predictable collections and a compiler that catches type slips — but it is verbose, and a few Java-specific traps (integer overflow, `Integer` compared with `==`, `Stack` instead of `ArrayDeque`, an `a - b` comparator) quietly fail the hidden test cases the judge runs but never shows you.

Three parts: **the Java DSA toolkit** (19.1–19.7: classes, idioms, pitfalls, a complexity table); **twenty patterns** (19.8–19.27, each with how to recognise it, the steps, runnable Java with checks, complexity and common mistakes) plus how to talk through a solution (19.28); and **Java interview Q&A** (19.29), which points to the question bank's answers. It assumes Chapters 2–6, collections (Chapter 10) and lambdas (Chapter 11). The same patterns in Python: Python Handbook, Chapter 19 — reading both separates the *algorithm* from the *language*.

> **Key term — complexity (Big-O):** how running time (and extra memory) grows with input size *n*. O(1) constant; O(log n) grows very slowly (halving each step); O(n) in step with n; O(n log n) a good sort; O(n²) a loop inside a loop. Constants are ignored (O(2n) is O(n)). §19.7 collects the numbers for every Java collection.

> **Key terms — pattern names** (each has its own section): a **graph** is *nodes* joined by *edges*; a **tree** is a graph with one *root* and no loops. **BFS** (breadth-first search) explores ring by ring outward; **DFS** (depth-first search) goes as deep as possible before backing up. **Dijkstra** finds cheapest routes when edges have costs. **Topological sort** orders tasks so prerequisites come first. **Union-find** tracks which items share a group. **DP** (dynamic programming) reuses answers to smaller sub-problems; **backtracking** tries choices and undoes them; **greedy** takes the best-looking step now. A **trie** is a tree of letters for prefix lookups; a **heap** keeps the smallest (or largest) item instantly available; a **BST** (binary search tree) keeps smaller keys left, larger right; the **LCA** (lowest common ancestor) is the deepest node above two given nodes. **Quickselect** finds the k-th smallest without a full sort.

**How the code was run.** Every program is in `code/` (toolkit demos in `toolkit/`, patterns in `patterns/`, facts in `qa/InterviewFacts.java`), compiled with `javac --release 25` and run on OpenJDK 26 (`bash tools_run_java.sh java` from the repository root). Every pattern program ends with a line such as `10 checks passed`; the output shown is real.


Read the problem
What does the input look like?
sorted array, or pairs from both ends
Two pointers or binary search
contiguous subarray or substring
Sliding window or prefix sums
seen before? count? pair lookup?
HashMap or HashSet
k largest, k smallest, running median
PriorityQueue
nested brackets, next greater element
ArrayDeque as a stack
ranges with start and end
Sort by start, then sweep
tree, grid or graph
What is asked?
fewest steps, unweighted
BFS with ArrayDeque
weighted shortest path
Dijkstra with PriorityQueue
order with dependencies
Topological sort
connected groups, cycle on add
Union-find
explore everything
DFS
tree: validate, LCA, path sum, serialise
Top-down or bottom-up recursion
all combinations or arrangements
Backtracking
best value, overlapping subproblems
Dynamic programming
many words, prefix queries
Trie
answer is a number with a yes/no test
Binary search on the answer
linked list: k-th from end, middle, cycle
Dummy node, fast and slow pointers
best local choice never needs undoing
Greedy
k-th element without a full sort, 3-way split
Quickselect or Dutch flag partition
appears twice except one, subsets of n up to 20
Bit manipulation
matrix walk, digits, slopes, circle of people
Math and geometry

---

# Part 1 — The Java DSA toolkit

## 19.1 Testing your solutions: `assert` is off, use a helper

### What it is

Interview practice means writing a function and checking it against a few inputs. Java has an `assert` statement, but the JVM **ignores it unless started with `-ea`** (enable assertions), so a "passing" run proves nothing. This chapter therefore uses a ten-line helper, `ch19.Check`, whose `eq(actual, expected, label)` always runs, compares arrays by content, prints `ok …`, and throws `AssertionError` on a mismatch.

```java
// code/ch19/toolkit/AssertKeywordDemo.java (key part)
boolean enabled = false;
assert enabled = true;                  // the side effect runs ONLY when assertions are on (java -ea)
System.out.println("assert keyword enabled? " + enabled);
assert 1 + 1 == 3 : "never reported without -ea";
System.out.println("assert 1 + 1 == 3 did not stop the program");
eq(1 + 1, 2, "Check.eq always runs");
try { eq(1 + 1, 3, "deliberately wrong"); }
catch (AssertionError e) { System.out.println("Check.eq failed loudly: " + e.getMessage()); }
```

Output (plain `java`, no `-ea`):

```
assert keyword enabled? false
assert 1 + 1 == 3 did not stop the program
ok  Check.eq always runs -> 2
Check.eq failed loudly: deliberately wrong: expected 3 but got 2
```

With `java -ea` the first line becomes `true` and the second `assert` throws `AssertionError: never reported without -ea`.

In a real project you'd write JUnit tests (Chapter 18); in an interview you'd just run `main` with a few calls. Use `assert` in production code only for internal invariants you're happy to have switched off — never for argument validation.

---

## 19.2 Arrays, strings and char arithmetic

Arrays are fixed-size and zero-based; `java.util.Arrays` is the toolbox of static helpers (§4.6, §4.8). `String` is immutable (§7.1); build strings with `StringBuilder` (§7.3). A `char` is an unsigned 16-bit number (0–65,535), so `c - 'a'` turns a lowercase letter into 0–25 — the basis of the `int[26]` frequency table used in dozens of string problems.

```java
// code/ch19/toolkit/ArraysAndStrings.java (key parts)
int[] a = {5, 2, 9, 1};
Arrays.sort(a);                                        // in place, ascending
int[] part = Arrays.copyOfRange(a, 1, 3);              // [from, to) -> indexes 1..2
int[] dist = new int[4];
Arrays.fill(dist, Integer.MAX_VALUE);                  // "infinity" for shortest paths
int[][] grid = new int[2][3];                          // 2 rows x 3 cols, all 0
System.out.println(Arrays.equals(new int[]{1, 2}, new int[]{1, 2}) + " " + (new int[]{1, 2} == new int[]{1, 2}));

int[] freq = new int[26];
for (char c : "banana".toCharArray()) freq[c - 'a']++;  // 'a'->0, 'b'->1, ...
char next = (char) ('a' + 2);                          // int arithmetic, cast back to char
int digit = '7' - '0';                                 // 7

char[] cs = "dcba".toCharArray();
Arrays.sort(cs);                                       // sort a string's letters
String sorted = new String(cs);

StringBuilder sb = new StringBuilder();
for (int i = 0; i < 5; i++) sb.append(i).append(',');
sb.setLength(sb.length() - 1);                         // drop the trailing comma
```

Output:

```
sorted      [1, 2, 5, 9]
copyOfRange [2, 5]
fill        [2147483647, 2147483647, 2147483647, 2147483647]
grid        [[0, 0, 0], [0, 0, 7]]
equals      true  ==  false
sum/max     17 9
freq a,b,n  3,1,2
'a'+2       99 as char c
digit value 7  Character.getNumericValue 7
helpers     false true q
trim/strip  [Hello, World] lower=hello, world
charAt/sub  e el 2
split       [a, b, , c]  words=[two, spaces]
sorted chars abcd  valueOf=abcd
builder     0,1,2,3,4  reversed=cba
edited      [X,1,2,3,4]  deleteCharAt(1)=ac
join/repeat a-b-c ababab
parse       -41 1010 10
```

| Need | Java | Note |
|---|---|---|
| Length | `arr.length`, `s.length()`, `list.size()` | Field, method, method — a classic compile error |
| Print an array | `Arrays.toString(a)`, `Arrays.deepToString(grid)` | `println(a)` prints `[I@1b6d3586` |
| Compare arrays | `Arrays.equals(a, b)` | `a == b` and `a.equals(b)` compare references, not contents |
| Character at i | `s.charAt(i)` | O(1); `s.toCharArray()` copies (O(n)) — fine once, not in a loop |
| Substring | `s.substring(from, to)` | End exclusive; copies, O(to − from) |
| Split on whitespace | `s.trim().split("\\s+")` | `split` takes a regex; trailing empty strings are dropped |
| Letter test | `Character.isLetterOrDigit`, `isDigit`, `isLetter`, `toLowerCase` | |
| Build a string | `StringBuilder.append`, `insert`, `reverse`, `setCharAt`, `deleteCharAt`, `setLength` | `s += x` in a loop is O(n²) |
| Number ↔ text | `Integer.parseInt(s)`, `Integer.parseInt(s, 2)`, `String.valueOf(n)`, `Integer.toBinaryString(n)` | |
| 2D grid | `new int[rows][cols]`, `grid[r][c]` | Rows first. `char[][] g` from `String[]` via `toCharArray()` |

**Common mistakes.** Comparing strings with `==` (§19.6); forgetting that `'a' + 1` is the `int` 98, not `"b"`; using `split(".")` (the dot is a regex meaning "any character" — use `split("\\.")`); mutating an array you were given when the caller still needs it (clone first).

---

## 19.3 `HashMap` and `HashSet` idioms

A `HashMap` gives O(1) average `get`/`put`/`containsKey`; a `HashSet` is a `HashMap` with only keys (§10.4, internals in §10.5). Since Java 8 `Map`'s default methods turn the old "if key present … else …" into one line:

| Idiom | One-liner | Replaces |
|---|---|---|
| Count | `count.merge(key, 1, Integer::sum)` | `if (!m.containsKey(k)) m.put(k, 1); else m.put(k, m.get(k) + 1);` |
| Count (alternative) | `count.put(k, count.getOrDefault(k, 0) + 1)` | same |
| Group | `groups.computeIfAbsent(key, k -> new ArrayList<>()).add(v)` | get, null check, put, add |
| Put once | `m.putIfAbsent(k, v)` | `if (!m.containsKey(k)) m.put(k, v)` |
| Seen before? | `if (!seen.add(x)) …` | `if (seen.contains(x)) … else seen.add(x)` |
| Iterate | `for (var e : m.entrySet()) e.getKey() / e.getValue()` | `for (k : keySet()) m.get(k)` (a second lookup per key) |

```java
// code/ch19/toolkit/MapSetIdioms.java (key parts)
for (String w : words) count.merge(w, 1, Integer::sum);
for (String w : words) byLength.computeIfAbsent(w.length(), k -> new ArrayList<>()).add(w);
int n = count.get("missing");                  // null unboxed -> NullPointerException
if (!seen.add(x)) { firstDup = x; break; }     // add returns false for a duplicate
Set<int[]> arrays = ...;                       // arrays use identity equals/hashCode
record Cell(int r, int c) {}                   // records get value-based equals/hashCode
```

Output:

```
merge        {be=2, not=1, or=1, to=2}  same=true
group        {2=[to, be, or, to, be], 3=[not]}
be=2 to=2
unboxing null -> NullPointerException
first dup    1
int[] key    contains? false
List key     contains? true
record key   contains? true
linked order [to, be, or, not]
```

**Keys.** A key needs value-based `equals` and `hashCode` (§6.13). `int[]` doesn't have them, so `Set<int[]>` can't find an equal array. Use `List.of(r, c)`, a `record Cell(int r, int c)` (records get value-based `equals`/`hashCode`), a `String` like `r + "," + c`, or encode a grid cell as the `int` `r * cols + c` (fastest). Never mutate an object while it is a key.

**Order.** `HashMap` iteration order is unspecified and can change between runs or JDK versions — don't let an answer depend on it. Use `LinkedHashMap` (insertion order) or `TreeMap` (sorted) when order matters, as in `Hashing.firstUnique` (§19.10).

**Arrays beat maps for small dense keys.** When keys are lowercase letters or values 0–1000, an `int[26]` or `int[1001]` is faster than a `HashMap<Character, Integer>` (no boxing, no hashing).

---

## 19.4 `ArrayDeque`, `PriorityQueue` and `TreeMap`

### `ArrayDeque` for stacks and queues — and why not `Stack`

`ArrayDeque` is a resizable circular array with amortised O(1) operations at both ends (§10.6). Use it for **both** stacks and queues:

| Role | Add | Remove | Look | Empty remove |
|---|---|---|---|---|
| Stack (LIFO: last in, first out) | `push(x)` | `pop()` | `peek()` | `pop()` throws `NoSuchElementException` |
| Queue (FIFO: first in, first out) | `offer(x)` | `poll()` | `peek()` | `poll()` returns `null` |

Declare the variable as `Deque<Integer>` (a stack) or `Queue<Integer>`/`Deque<Integer>` (a queue). `ArrayDeque` rejects `null` elements.

Why not `java.util.Stack`? It dates from Java 1.0, extends `Vector` (every method `synchronized` — a cost for nothing in single-threaded code), exposes index methods like `get(0)` that break the stack abstraction, and iterates **bottom-to-top** — the opposite of `ArrayDeque`. Its own Javadoc recommends `Deque` instead. `LinkedList` also implements `Deque` but allocates a node per element; `ArrayDeque` is faster.

### `PriorityQueue` — a binary heap

`PriorityQueue` is a **min-heap** by default (an array-backed tree that keeps the smallest element at the root): `peek()` is O(1), `offer`/`poll` are O(log n). For a max-heap pass `Comparator.reverseOrder()`. For arrays like `{distance, node}` use `Comparator.comparingInt(x -> x[0])`. Printing or iterating a `PriorityQueue` shows **heap order, not sorted order**; only repeated `poll()` gives sorted output. `contains` and `remove(Object)` are O(n).

### `TreeMap` / `TreeSet` — sorted with navigation

A red-black tree (self-balancing BST): O(log n) `get`/`put`/`remove`, and navigation methods a hash map can't offer. Remember the four: **`floor`** (≤), **`ceiling`** (≥), **`lower`** (<), **`higher`** (>). On a `TreeMap` they are `floorKey`/`floorEntry` and so on; all return `null` when nothing qualifies. Typical uses: the nearest booking before a given time, calendar clashes, leaderboards, medians over a moving window.

```java
// code/ch19/toolkit/DequesHeapsTrees.java (key parts)
Deque<Integer> stack = new ArrayDeque<>();
stack.push(1); stack.push(2); stack.push(3);
Deque<String> queue = new ArrayDeque<>();
queue.offer("a"); queue.offer("b"); queue.offer("c");
PriorityQueue<Integer> max = new PriorityQueue<>(Comparator.reverseOrder());
PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(x -> x[0]));
TreeMap<Integer, String> prices = new TreeMap<>(Map.of(10, "ten", 20, "twenty", 30, "thirty"));
prices.floorKey(25); prices.ceilingKey(25); prices.lowerKey(20); prices.higherKey(30);
```

Output:

```
stack peek=3 pop=3 iterates [2, 1]
legacy Stack iterates [1, 2, 3] and allows get(0)=1
queue poll=a peek=b left [b, c]
empty poll -> null (pop() would throw)
min.peek=1 max.peek=8  toString is heap order, not sorted: [1, 3, 8, 5]
poll order 1 3 5 8
int[] heap first [2, 9]
floorKey(25)=20 ceilingKey(25)=30 lowerKey(20)=10 higherKey(30)=null
first=10 last=30=thirty headMap(20)={10=ten} tailMap(20)={20=twenty, 30=thirty}
TreeSet floor(10)=9 ceiling(10)=16 pollFirst=1 descending=[16, 9, 4]
```

---

## 19.5 Sorting with comparators

| What you sort | Call | Algorithm and guarantees |
|---|---|---|
| `int[]`, `long[]`, `char[]` … | `Arrays.sort(a)` | Dual-Pivot Quicksort (§19.25); the Javadoc promises O(n log n) on all inputs. Ascending only — no comparator overload for primitives |
| `Integer[]`, `String[]`, `int[][]`, objects | `Arrays.sort(a, comparator)` | Stable (equal elements keep their order), TimSort (a merge sort tuned for partly sorted data), O(n log n) |
| `List` | `list.sort(cmp)` or `Collections.sort(list, cmp)` | Same stable sort; `Collections.sort` just calls `List.sort` |

Build comparators from parts instead of writing `compare` by hand (§10.8): `Comparator.comparingInt(Person::age).reversed().thenComparing(Person::name)`. For `int[]` rows the lambda needs a type witness: `Comparator.<int[]>comparingInt(x -> x[0]).thenComparingInt(x -> x[1])` — otherwise the compiler infers `Object` once you chain.

```
// code/ch19/toolkit/SortingDemo.java (key parts)
Integer[] boxed = {3, -1, 2};
Arrays.sort(boxed, Collections.reverseOrder());                       // descending needs boxes
int[][] intervals = {{5, 6}, {1, 9}, {1, 3}};
Arrays.sort(intervals, Comparator.<int[]>comparingInt(x -> x[0]).thenComparingInt(x -> x[1]));
people.sort(Comparator.comparingInt(Person::age).reversed().thenComparing(Person::name));
Integer[] big = {Integer.MIN_VALUE, 1, Integer.MAX_VALUE};
Arrays.sort(big, (x, y) -> x - y);                                   // WRONG: overflows
Arrays.sort(big, Integer::compare);                                  // right
```

Output:

```
int[] asc        [-1, 2, 3]
Integer[] desc   [3, 2, -1]
stream desc      [3, 2, -1]
intervals        [[1, 3], [1, 9], [5, 6]]
age desc, name   [Person[name=Bo, age=30], Person[name=Ravi, age=30], Person[name=Ana, age=25]]
by length        [fig, pear, kiwi, apple]
(a-b) comparator [1, 2147483647, -2147483648]
Integer::compare [-2147483648, 1, 2147483647]
```

The `(a, b) -> a - b` comparator is the most common silent bug: `1 - Integer.MIN_VALUE` overflows to a negative number, so the sort puts `MIN_VALUE` last. Always use `Integer.compare(a, b)` or `Comparator.comparingInt`. Note also "by length" — `pear` stays before `kiwi` because the sort is stable.

---

## 19.6 Numeric and equality pitfalls

These are the bugs that pass the sample tests and fail the hidden ones. Background: overflow in §2.10, wrappers and caching in §7.6.

```
// code/ch19/toolkit/NumericPitfalls.java (key parts)
int lo = 2_000_000_000, hi = 2_100_000_000;
int bad  = (lo + hi) / 2;  // overflows
int mid  = lo + (hi - lo) / 2;  // safe midpoint
int mid2 = (lo + hi) >>> 1;     // also safe for non-negative lo, hi (unsigned shift)
long wrong = n * n;        // int * int overflows BEFORE widening
long right = (long) n * n;
Math.multiplyExact(n, n);  // throws ArithmeticException instead of wrapping
Integer c = 128, d = 128;
boolean same = c == d;              // false: compares references
list.remove(1);                      // removes INDEX 1
list.remove(Integer.valueOf(10));    // removes VALUE 10
```

Output:

```
MAX + 1            -2147483648
(lo+hi)/2          -97483648  lo+(hi-lo)/2=2050000000  (lo+hi)>>>1=2050000000
n*n wrong/right    1410065408 / 10000000000
multiplyExact      ArithmeticException: integer overflow
toIntExact ok      42
abs(MIN_VALUE)     -2147483648
7/2, -7/2          3, -3   7/2.0=3.5
-7 % 3, floorMod   -1, 2
127 == 127         true
128 == 128         false   equals=true
map.get == get     false   intValue== true
remove idx/value   [30]
String ==, equals  false, true
'a' + 1, "" + 'a' + 1  98, a1
for-each remove    ConcurrentModificationException
removeIf           [1, 3]
```

| **Trap** | **Why** | **Fix** |
|---|---|---|
| `int` overflow | `int` wraps silently past ±2.1 billion | Use `long` for sums, products, and "infinity + edge"; `Math.addExact` / `multiplyExact` / `toIntExact` throw instead |
| Midpoint | `lo + hi` can overflow | `lo + (hi - lo) / 2` |
| `Math.abs(Integer.MIN_VALUE)` | $+2^{31}$ doesn't fit in `int` | Work in `long`, or `Math.absExact` (Java 15) to get an exception |
| `Integer.MAX_VALUE` as "infinity" | `INF + 1` wraps to a negative number | Guard before adding (see coin change, §19.19) or use `Integer.MAX_VALUE / 2` |
| `-7 % 3 == -1` | `%` keeps the dividend's sign | `Math.floorMod(x, m)` for a non-negative index (circular arrays, hashing) |
| `7 / 2 == 3` | Integer division truncates toward zero | Cast one side: `7 / 2.0`, or `(double) a / b` |
| `Integer == Integer` | Compares references; only −128…127 are guaranteed cached by `Integer.valueOf` | `.equals`, `Integer.compare`, or unbox to `int`. Especially `map.get(a) == map.get(b)` |
| Unboxing `null` | `int x = map.get(missing)` | `getOrDefault`, or keep it as `Integer` and check |
| `list.remove(int)` | `List<Integer>` has `remove(int index)` and `remove(Object)` | `remove(Integer.valueOf(v))` to remove a value |
| Removing in for-each | Fail-fast iterator throws `ConcurrentModificationException` | `removeIf`, an explicit `Iterator.remove()`, or build a new list |
| Deep recursion | Default thread stack overflows around $10^4$–$10^5$ frames (depends on frame size) | Iterate with an explicit `ArrayDeque` stack for big grids/lists (see islands, §19.16) |

## 19.7 Complexity cheat table

Big-O describes how cost grows with input size n; "amortised" means averaged over many calls (an occasional resize is paid for by many cheap operations). Details and a measured demo: §10.12.

| Structure | Access / search | Insert / add | Remove | Notes |
|---|---|---|---|---|
| array `int[]` | O(1) index, O(n) search | — (fixed size) | — | `Arrays.binarySearch` O(log n) if sorted |
| `ArrayList` | O(1) `get(i)`, O(n) `contains` | O(1) amortised at end, O(n) in middle | O(n) (shifts), O(1) at end | Default list |
| `LinkedList` | O(n) `get(i)` | O(1) at ends | O(1) at ends | Rarely the right choice |
| `ArrayDeque` | O(1) `peek` at ends | O(1) amortised `push`/`offer` | O(1) `pop`/`poll` | Stack and queue |
| `HashMap` / `HashSet` | O(1) average | O(1) average | O(1) average | Worst case O(log n) with treeified buckets of `Comparable` keys |
| `LinkedHashMap` | O(1) average | O(1) | O(1) | Keeps insertion (or access) order — LRU cache |
| `TreeMap` / `TreeSet` | O(log n) | O(log n) | O(log n) | `floor`/`ceiling`/`first`/`last` also O(log n) |
| `PriorityQueue` | O(1) `peek` | O(log n) `offer` | O(log n) `poll`, O(n) `remove(obj)` | Build from a collection in O(n) |
| `String` | O(1) `charAt` | `+` copies: O(n) | `substring` copies | `StringBuilder.append` amortised O(1) |

| Algorithm | Time | Extra space |
|---|---|---|
| `Arrays.sort` / `List.sort` | O(n log n) | Small for primitives; up to O(n) buffer for objects (merge sort) |
| Binary search | O(log n) | O(1) |
| BFS / DFS on V vertices, E edges | O(V + E) | O(V) |
| Dijkstra with a binary heap | O((V + E) log V) | O(V + E) |
| Topological sort (Kahn) | O(V + E) | O(V) |
| Union-find with path compression + union by size | ≈ O(1) per op (inverse Ackermann, §19.18) | O(n) |
| Subsets / permutations | $O(2^n \cdot n)$ / $O(n! \cdot n)$ | O(n) recursion depth |

Rough budget: about $10^8$ simple operations per second in Java once the JIT has warmed up, so $n = 10^5$ needs O(n log n) or better and $n \le 20$ allows $O(2^n)$.

# Part 2 — Twenty patterns

Each pattern follows the same shape: **what it is**, **recognise it when** (the wording in a problem that should trigger it), **how it works**, the code, its **complexity**, and **common mistakes**. Full programs are in `code/ch19/patterns/`; the README shows the key methods. Python versions of the same ideas: Python §19.2–19.11.

## 19.8 Two pointers

### What it is

Two indexes move through the data in a coordinated way so that one pass replaces a nested loop. Three flavours: **opposite ends** moving inward (sorted arrays, palindromes), **read/write** pointers moving the same way (in-place filtering), and **fast/slow** pointers moving at different speeds (linked lists, §19.22).

**Recognise it when**: the input is sorted; you need a pair/triple with a target sum; "in place, O(1) extra space"; palindrome; linked-list middle, cycle, or k-th from the end.

### How it works (pair sum in a sorted array)

1. Put `left` at index 0 and `right` at the last index.
2. If the sum is too small, only moving `left` right can increase it; if too big, move `right` left.
3. Stop when they meet — each step discards one element for good, so the loop runs at most n times.

```java
// code/ch19/patterns/TwoPointers.java (key parts)
static int[] pairWithSum(int[] sorted, int target) {
    int left = 0, right = sorted.length - 1;
    while (left < right) {
        int sum = sorted[left] + sorted[right];
        if (sum == target) return new int[]{left, right};
        if (sum < target) left++;          // need a bigger sum
        else right--;                      // need a smaller sum
    }
    return new int[]{-1, -1};
}

static int dedupe(int[] sorted) {          // read/write pointers
    if (sorted.length == 0) return 0;
    int write = 1;                         // sorted[0..write) holds the unique values
    for (int read = 1; read < sorted.length; read++)
        if (sorted[read] != sorted[write - 1]) sorted[write++] = sorted[read];
    return write;
}

static boolean hasCycle(ListNode head) {   // fast/slow (Floyd)
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next; fast = fast.next.next;
        if (slow == fast) return true;     // reference comparison on purpose
    }
    return false;
}

static ListNode reverse(ListNode head) {   // prev / cur / next
    ListNode prev = null, cur = head;
    while (cur != null) { ListNode next = cur.next; cur.next = prev; prev = cur; cur = next; }
    return prev;
}
```

`ListNode` is a tiny class with `int val` and `ListNode next` — the shape interview platforms use. The file also has `isPalindrome` (skip non-alphanumerics, compare lower-cased) and `middle`.

Output:

```
ok  pairWithSum 15 -> [2, 4]
ok  pairWithSum none -> [-1, -1]
ok  dedupe -> [1, 2, 3, 4]
ok  middle odd -> 3
...
10 checks passed
```

**Complexity**: O(n) time, O(1) space (a sort first makes it O(n log n)).

### Common mistakes

- Using opposite-end pointers on **unsorted** input — use hashing (§19.10) instead.
- `while (left <= right)` for pairs — an element would pair with itself.
- In fast/slow, checking `fast.next.next` without first checking `fast != null && fast.next != null`.
- For 3Sum (find all triples that add up to zero), forgetting to skip duplicate values after sorting.

## 19.9 Sliding window

### What it is

A special two-pointer pattern for **contiguous** ranges (elements next to each other, with no gaps): keep a window `[left, right]` and a running summary of it (sum, counts, distinct characters). Moving `right` adds one element; moving `left` removes one — each element enters and leaves once, so the work is O(n) instead of O(n²) for re-examining every subarray (a contiguous slice of an array).

**Recognise it when**: "longest/shortest substring or subarray such that…", "at most k distinct", "window of size k", and the condition changes monotonically (only ever in one direction) as the window grows (for sums, this needs non-negative numbers — otherwise use prefix sums, §19.11).

### How it works (variable-size window)

1. For each `right`, add `a[right]` to the window state.
2. **While** the window is invalid (or, for "shortest", while it is valid), update the answer as appropriate and remove `a[left++]`.
3. Record the answer for the current window.

```java
// code/ch19/patterns/SlidingWindow.java (key parts)
static int shortestWithSumAtLeast(int[] a, int target) {
    int left = 0, sum = 0, best = Integer.MAX_VALUE;
    for (int right = 0; right < a.length; right++) {
        sum += a[right];                           // grow
        while (sum >= target) {                    // valid: record, then shrink
            best = Math.min(best, right - left + 1);
            sum -= a[left++];
        }
    }
    return best == Integer.MAX_VALUE ? 0 : best;
}

static int longestUniqueSubstring(String s) {
    Map<Character, Integer> lastSeen = new HashMap<>();
    int left = 0, best = 0;
    for (int right = 0; right < s.length(); right++) {
        Integer prev = lastSeen.put(s.charAt(right), right);    // put returns the old value
        if (prev != null && prev >= left) left = prev + 1;      // jump past the duplicate
        best = Math.max(best, right - left + 1);
    }
    return best;
}

static int longestWithKDistinct(String s, int k) {
    int[] count = new int[128];                    // ASCII counts: faster than a HashMap
    int distinct = 0, left = 0, best = 0;
    for (int right = 0; right < s.length(); right++) {
        if (count[s.charAt(right)]++ == 0) distinct++;
        while (distinct > k) if (--count[s.charAt(left++)] == 0) distinct--;
        best = Math.max(best, right - left + 1);
    }
    return best;
}
```

The file also has the fixed-size version, `maxSumOfK`: add `a[right]`, subtract `a[right - k]`.

Output:

```
ok  max sum of 3 -> 9
ok  shortest sum>=7 -> 2
ok  shortest none -> 0
ok  unique abcabcbb -> 3
...
6 checks passed
```

**Complexity**: O(n) time; O(1) or O(alphabet) space.

### Common mistakes

- The `prev >= left` check in `longestUniqueSubstring`: without it, `"abba"` jumps `left` **backwards** at the last `a` and returns 3 instead of 2.
- Using `if` instead of `while` to shrink.
- Applying "shrink while sum ≥ target" to arrays with **negative** numbers — shrinking no longer reliably lowers the sum.

---

## 19.10 Hashing

### What it is

Store what you've seen in a `HashMap`/`HashSet` so the question "have I seen X?" or "how many X so far?" costs O(1) instead of a scan. You trade O(n) memory for a factor of n in time.

**Recognise it when**: pairs with a target sum in **unsorted** data, duplicates, counting frequencies, grouping by a signature (anagrams — words with the same letters, like *tea* and *eat*), "first unique", "longest consecutive run".

### How it works (Two Sum)

1. Walk the array once.
2. For each value x, look up `target - x` among the values already seen.
3. If found, you have the pair; otherwise record x → its index and continue. Looking only backwards also stops an element pairing with itself.

```java
// code/ch19/patterns/Hashing.java (key parts)
static int[] twoSum(int[] a, int target) {
    Map<Integer, Integer> indexOf = new HashMap<>();
    for (int i = 0; i < a.length; i++) {
        Integer j = indexOf.get(target - a[i]);
        if (j != null) return new int[]{j, i};
        indexOf.put(a[i], i);
    }
    return new int[0];
}

static List<List<String>> groupAnagrams(String[] words) {
    Map<String, List<String>> groups = new LinkedHashMap<>();   // keeps first-seen order
    for (String w : words) {
        char[] cs = w.toCharArray();
        Arrays.sort(cs);                                        // "tea" -> "aet"
        groups.computeIfAbsent(new String(cs), k -> new ArrayList<>()).add(w);
    }
    return new ArrayList<>(groups.values());
}

static int longestConsecutive(int[] a) {
    Set<Integer> set = new HashSet<>();
    for (int x : a) set.add(x);
    int best = 0;
    for (int x : set) {
        if (set.contains(x - 1)) continue;         // only count from the START of a run
        int len = 1;
        while (set.contains(x + len)) len++;
        best = Math.max(best, len);
    }
    return best;
}
```

Output:

```
ok  twoSum -> [2, 3]
ok  groupAnagrams -> [[eat, tea, ate], [tan, nat], [bat]]
ok  longestConsecutive -> 4
ok  firstUnique -> w
4 checks passed
```

**Complexity**: O(n) average time (`groupAnagrams` is O(n · k log k) for words of length k), O(n) space. `longestConsecutive` is O(n) because each run is walked only from its first element.

### Common mistakes

- `Integer j = …; if (j != 0)` style checks on a missing key (it's `null`, not 0).
- Comparing two `Integer` values from a map with `==` (§19.6).
- Using `int[]` as a map key (identity `hashCode`) — use a `String`, `List`, or record.

---

## 19.11 Prefix sums

### What it is

Precompute `prefix[i]` = sum of the first i elements. Then the sum of any range `a[l..r]` is `prefix[r + 1] - prefix[l]` — one subtraction instead of a loop. Combined with a hash map, it counts subarrays with a given sum even when numbers are **negative** (where a sliding window fails).

**Recognise it when**: many range-sum queries; "number of subarrays whose sum equals k"; 2D sub-rectangle sums; many "add v to every element in [l, r]" updates (the inverse trick, a **difference array**).

### How it works (subarrays summing to k)

1. Keep a running sum and a map from "prefix sum value" → "how many times seen", starting with `{0: 1}` (the empty prefix).
2. A subarray ending here sums to k exactly when an earlier prefix equals `sum - k`. Add that count.
3. Record the current sum.

```java
// code/ch19/patterns/PrefixSums.java (key parts)
static long[] prefix(int[] a) {                 // length n+1, prefix[0] = 0; long avoids overflow
    long[] p = new long[a.length + 1];
    for (int i = 0; i < a.length; i++) p[i + 1] = p[i] + a[i];
    return p;
}
static long rangeSum(long[] p, int from, int to) { return p[to + 1] - p[from]; }

static int subarraysSumToK(int[] a, int k) {
    Map<Integer, Integer> seen = new HashMap<>();
    seen.put(0, 1);                              // empty prefix
    int sum = 0, count = 0;
    for (int x : a) {
        sum += x;
        count += seen.getOrDefault(sum - k, 0);
        seen.merge(sum, 1, Integer::sum);
    }
    return count;
}

// 2D: p[r+1][c+1] = g[r][c] + above + left - overlap (inclusion-exclusion)
static long rect(long[][] p, int r1, int c1, int r2, int c2) {
    return p[r2 + 1][c2 + 1] - p[r1][c2 + 1] - p[r2 + 1][c1] + p[r1][c1];
}
```

The difference array records `+v` at `l` and `-v` at `r + 1` for each update, then one running-sum pass produces the final array.

Output:

```
ok  rangeSum 1..3 -> 6
ok  rangeSum all -> 23
ok  subarrays sum 3 -> 4
ok  2D rect -> 28
...
5 checks passed
```

**Complexity**: O(n) to build, O(1) per query; 2D is O(rows · cols) to build.

### Common mistakes

- Off-by-one (an index one too high or too low): the `n + 1` length and the `prefix[0] = 0` sentinel (a dummy value placed to remove a special case) exist precisely so that `l = 0` needs no special case.
- Forgetting `seen.put(0, 1)` — misses subarrays that start at index 0.
- `int` prefix sums of large inputs overflowing — use `long`.

---

## 19.12 Stacks and monotonic stacks

### What it is

A **stack** (last in, first out) matches nested structure: brackets, expression evaluation, undo, DFS. A **monotonic stack** keeps its contents in increasing or decreasing order by popping everything that breaks the order before pushing; each popped element has just met its "next greater" (or smaller) element. A **monotonic deque** does the same at both ends to give the max of a sliding window in O(n).

**Recognise it when**: balanced brackets; "next greater/smaller element", "days until warmer", "stock span", "largest rectangle in histogram"; "maximum of every window of size k".

### How it works (next warmer day)

1. Keep a stack of **indexes** whose temperatures are decreasing from bottom to top.
2. For each new day i, pop every index j whose temperature is lower — day i is j's answer, `i - j`.
3. Push i. Indexes left at the end never found a warmer day (answer 0).

```
// code/ch19/patterns/Stacks.java (key parts)
static boolean validBrackets(String s) {
    Deque<Character> stack = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        switch (c) {
            case '(' -> stack.push(')');          // push the EXPECTED closer
            case '[' -> stack.push(']');
            case '{' -> stack.push('}');
            default -> { if (stack.isEmpty() || stack.pop() != c) return false; }
        }
    }
    return stack.isEmpty();
}

static int[] daysUntilWarmer(int[] t) {
    int[] ans = new int[t.length];
    Deque<Integer> stack = new ArrayDeque<>();
    for (int i = 0; i < t.length; i++) {
        while (!stack.isEmpty() && t[stack.peek()] < t[i]) {
            int j = stack.pop();
            ans[j] = i - j;                        // i is j's "next greater"
        }
        stack.push(i);
    }
    return ans;
}

static int[] windowMax(int[] a, int k) {
    int[] out = new int[a.length - k + 1];
    Deque<Integer> dq = new ArrayDeque<>();        // indexes, values decreasing front to back
    for (int i = 0; i < a.length; i++) {
        if (!dq.isEmpty() && dq.peekFirst() <= i - k) dq.pollFirst();       // fell out of window
        while (!dq.isEmpty() && a[dq.peekLast()] <= a[i]) dq.pollLast();    // can never be max again
        dq.offerLast(i);
        if (i >= k - 1) out[i - k + 1] = a[dq.peekFirst()];
    }
    return out;
}
```

The file also evaluates Reverse Polish Notation (postfix notation, where each operator follows its operands: `"4 13 5 / +"` = 6) with an arrow-label `switch` (§3.3).

Output:

```
ok  daysUntilWarmer -> [1, 1, 4, 2, 1, 1, 0, 0]
ok  windowMax -> [3, 3, 5, 5, 6, 7]
ok  RPN -> 6
...
6 checks passed
```

**Complexity**: O(n) — every index is pushed once and popped at most once, even though there's a loop inside a loop.

### Common mistakes

- Storing values instead of indexes when you need distances.
- Forgetting the final `stack.isEmpty()` check (`"(("` would pass).
- Popping in RPN in the wrong order: the **second** pop is the left operand (the value on the left of the operator).
- Using `java.util.Stack` (§19.4).

## 19.13 Heaps and top-k

### What it is

A heap (`PriorityQueue`) always hands you the smallest (or largest) element in O(log n) per change. The key trick for "top k" is counter-intuitive: to keep the k **largest**, use a **min**-heap of size k — its root is the weakest of the current top k, the one to evict when something bigger arrives.

**Recognise it when**: "k-th largest/smallest", "k most frequent", "k closest points", "merge k sorted lists", "running median" (the median of everything seen so far, updated as each value arrives) and "maximise capital" (two heaps, below), scheduling by earliest end time, Dijkstra.

### How it works (k-th largest)

1. Offer each element into a min-heap.
2. If the heap grows past k, poll (remove the smallest).
3. After all elements, the root is the k-th largest. Cost O(n log k) — better than sorting when k ≪ n.

```
// code/ch19/patterns/HeapTopK.java (key parts)
static int kthLargest(int[] a, int k) {
    PriorityQueue<Integer> heap = new PriorityQueue<>();
    for (int x : a) {
        heap.offer(x);
        if (heap.size() > k) heap.poll();          // evict the smallest
    }
    return heap.peek();
}

static List<Integer> mergeSorted(int[][] arrays) {   // heap holds {value, arrayIndex, elementIndex}
    PriorityQueue<int[]> heap = new PriorityQueue<>(Comparator.comparingInt(x -> x[0]));
    for (int i = 0; i < arrays.length; i++) if (arrays[i].length > 0) heap.offer(new int[]{arrays[i][0], i, 0});
    List<Integer> out = new ArrayList<>();
    while (!heap.isEmpty()) {
        int[] top = heap.poll();
        out.add(top[0]);
        int arr = top[1], next = top[2] + 1;
        if (next < arrays[arr].length) heap.offer(new int[]{arrays[arr][next], arr, next});
    }
    return out;
}

static final class MedianFinder {                    // two heaps
    private final PriorityQueue<Integer> low = new PriorityQueue<>(Comparator.reverseOrder()); // max-heap
    private final PriorityQueue<Integer> high = new PriorityQueue<>();                         // min-heap
    void add(int x) {
        low.offer(x);
        high.offer(low.poll());                                    // largest of low moves up
        if (high.size() > low.size()) low.offer(high.poll());      // keep low >= high in size
    }
    double median() {
        return low.size() > high.size() ? low.peek() : (low.peek() + (long) high.peek()) / 2.0;
    }
}
```

`topKFrequent` counts with `merge`, then keeps a size-k min-heap of map entries ordered by `Map.Entry.comparingByValue()`, breaking ties by key.

Output:

```
ok  kthLargest -> 5
ok  topKFrequent -> [1, 2]
ok  mergeSorted -> [0, 1, 2, 4, 5, 7, 9]
ok  median of 5,15 -> 10.0
...
8 checks passed
```

**Complexity**: top-k O(n log k); merging k lists with N total elements O(N log k); median O(log n) per add, O(1) per query; maximise capital O(n log n) (each project enters and leaves each heap at most once).

### Two heaps

Some problems need the best element from **two different groups** at once. Keep one heap per group and move elements across the boundary as the situation changes. Two shapes come up:

- **Split one stream into halves** (running median). A max-heap `low` holds the smaller half, a min-heap `high` the larger half, and the invariant (the rule kept true after every step) is `low.size() == high.size()` or one more. The median is then `low.peek()` or the average of both roots — O(1) — and each `add` costs O(log n). Every new value passes through `low` into `high` and, if `high` got too big, one value comes back; that keeps every element of `low` ≤ every element of `high` without a special case (`MedianFinder` above).
- **Locked vs available** (maximise capital, scheduling). A min-heap orders items by the threshold that **unlocks** them; a max-heap holds the unlocked ones by **value**. Each round: move everything whose threshold is now met from the first heap to the second, then take the best from the second. Example: you may run at most k projects; project i needs `capital[i]` to start and pays `profit[i]`; start with `start` money.

```java
// code/ch19/patterns/HeapTopK.java (key part)
static long maximizeCapital(int k, long start, int[] profit, int[] capital) {
    PriorityQueue<Integer> locked = new PriorityQueue<>(Comparator.comparingInt(i -> capital[i]));
    PriorityQueue<Integer> affordable = new PriorityQueue<>(Comparator.comparingInt((Integer i) ->
profit[i]).reversed());
    for (int i = 0; i < profit.length; i++) locked.offer(i);
    long money = start;
    for (int round = 0; round < k; round++) {
        while (!locked.isEmpty() && capital[locked.peek()] <= money) affordable.offer(locked.poll()); // unlock
        if (affordable.isEmpty()) break;                   // nothing we can start
        money += profit[affordable.poll()];                // greedy: the most profitable affordable project
    }
    return money;
}
```

Heaps store **indexes** here, and the comparators look the values up — cheaper than creating `int[]` pairs. The `(Integer i) ->` type on the lambda is needed because `.reversed()` stops Java inferring it. Taking the most profitable affordable project is a greedy choice (§19.24): money only grows, so nothing unlocked now can become locked later, and a bigger profit never unlocks fewer projects.

**Recognise it when**: "median of a stream / sliding window median", "maximise capital with at most k projects", "schedule tasks as they become available", anything that says "the best among those that currently qualify".

### Common mistakes

- A max-heap of everything for top-k: O(n log n), and uses O(n) memory.
- Writing the max-heap comparator as `(a, b) -> b - a` (overflow, §19.5) — use `Comparator.reverseOrder()`.
- Expecting `heap.toString()` or a for-each loop to be sorted.
- Changing an object's priority field while it sits in the heap — the heap won't re-order; remove and re-insert (or insert a new entry and skip stale ones, as Dijkstra does in §19.16).

---

## 19.14 Binary search (and binary search on the answer)

### What it is

Halve a sorted search space each step: O(log n). Beyond "find x in a sorted array", the powerful version is **binary search on the answer**: when the answer is a number and a yes/no test `feasible(x)` flips from false to true exactly once as x grows, binary-search x itself.

**Recognise it when**: sorted input; "first/last position of", "insert position"; rotated sorted array; "minimum capacity/speed/time such that…" or "maximise the minimum…" (search on the answer).

### How it works

1. Decide the range and write down the **invariant** (what's true about `lo` and `hi` at every step).

2. Compute `mid = lo + (hi - lo) / 2` (never `(lo + hi) / 2`, §19.6).
3. Discard the half that cannot contain the answer, keeping the invariant.
4. Two templates cover almost everything: **closed** `[lo, hi]` with `while (lo <= hi)` for "find exactly", and **half-open / first-true** (the range includes `lo` but not `hi`) with `while (lo < hi)`, `hi = mid` for "first index where the condition holds".

```java
// code/ch19/patterns/BinarySearch.java (key parts)
static int lowerBound(int[] a, int target) {       // first index with a[i] >= target
    int lo = 0, hi = a.length;                      // answer is in [lo, hi]
    while (lo < hi) {
        int mid = (lo + hi) >>> 1;
        if (a[mid] < target) lo = mid + 1; else hi = mid;
    }
    return lo;
}

static int searchRotated(int[] a, int target) {    // e.g. [4,5,6,7,0,1,2]
    int lo = 0, hi = a.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == target) return mid;
        if (a[lo] <= a[mid]) {                     // left half is sorted
            if (a[lo] <= target && target < a[mid]) hi = mid - 1; else lo = mid + 1;
        } else {                                   // right half is sorted
            if (a[mid] < target && target <= a[hi]) lo = mid + 1; else hi = mid - 1;
        }
    }
    return -1;
}

// Binary search on the answer: smallest ship capacity that ships everything in `days` days.
static int minCapacity(int[] weights, int days) {
    int lo = Arrays.stream(weights).max().getAsInt();   // must fit the heaviest package
    int hi = Arrays.stream(weights).sum();              // everything in one day
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (daysNeeded(weights, mid) <= days) hi = mid; else lo = mid + 1;
    }
    return lo;
}
```

`daysNeeded` greedily fills each day up to the capacity. Library versions: `Arrays.binarySearch` and `Collections.binarySearch` return the index if found, otherwise **`-(insertionPoint) - 1`**; with duplicates, they don't promise *which* equal element they return — write `lowerBound` yourself when you need the first. `TreeSet.ceiling` is the other ready-made lower bound.

Output:

```
ok  search hit -> 3
ok  search miss -> -1
ok  lowerBound first 2 -> 1
ok  lowerBound past end -> 5
...
9 checks passed
```

**Complexity**: O(log n); on the answer, O(n · log(range)).

### Common mistakes

- Mixing templates: `while (lo < hi)` with `hi = mid - 1`, or `while (lo <= hi)` with `hi = mid` (infinite loop).
- `lo = mid` with a downward-rounded mid: when `hi = lo + 1` the loop never moves.
- Search-on-answer with the wrong bounds (capacity below the heaviest item is never feasible) or a non-monotonic test.

# 19.15 Intervals

## What it is

Problems about ranges `[start, end]`: merge overlaps, insert a new range, count simultaneous meetings. Almost all start the same way: **sort by start**, then sweep left to right, comparing each interval with the last one you kept.

**Recognise it when**: meetings, bookings, time ranges, "overlapping", "minimum rooms/platforms", "free time".

## How it works (merge)

1. Sort by start.
2. For each interval: if it starts at or before the end of the last merged interval, they overlap — extend that end to the max of the two ends.
3. Otherwise start a new merged interval.

```java
// code/ch19/patterns/Intervals.java (key parts)
static int[][] merge(int[][] intervals) {
    int[][] sorted = intervals.clone();                         // don't reorder the caller's array
    Arrays.sort(sorted, Comparator.comparingInt(x -> x[0]));
    List<int[]> out = new ArrayList<>();
    for (int[] cur : sorted) {
        if (!out.isEmpty() && cur[0] <= out.getLast()[1]) {     // overlaps the last merged one
            out.getLast()[1] = Math.max(out.getLast()[1], cur[1]);
        } else {
            out.add(new int[]{cur[0], cur[1]});
        }
    }
    return out.toArray(new int[0][]);
}

static int minRooms(int[][] meetings) {                          // [start, end) semantics
    int[][] sorted = meetings.clone();
    Arrays.sort(sorted, Comparator.comparingInt(x -> x[0]));
    PriorityQueue<Integer> ends = new PriorityQueue<>();         // end times of rooms in use
    for (int[] m : sorted) {
        if (!ends.isEmpty() && ends.peek() <= m[0]) ends.poll(); // reuse the room that frees first
        ends.offer(m[1]);
    }
    return ends.size();
}
```

`List.getLast()` is from sequenced collections (Java 21, §10.11); on older Java write `out.get(out.size() - 1)`. `insert` copies intervals that end before the new one, merges all that overlap it, then copies the rest — O(n) with no sort.

Output:

```
ok  merge -> [[1, 6], [8, 10], [15, 18]]
ok  merge touching -> [[1, 5]]
ok  insert -> [[1, 2], [3, 10]]
ok  minRooms -> 2
...
5 checks passed
```

**Complexity**: O(n log n) for the sort; the sweep is O(n) (O(n log n) with the heap).

## Common mistakes

- Not agreeing whether `[1, 4]` and `[4, 5]` overlap: for merging closed ranges, `<=` says yes; for meetings `[start, end)`, back-to-back meetings share a room. Ask the interviewer.
- Keeping `max(end)` out: `[1, 10]` then `[2, 3]` must stay `[1, 10]`.
- `clone()` on `int[][]` is **shallow** — rows are shared, so writing into `out` rows copied by reference would modify the input. That's why `merge` adds `new int[]{…}`.

## 19.16 BFS and DFS: trees, grids and graphs

### What it is

Two ways to visit every node reachable from a start. **Breadth-first search (BFS)** uses a **queue** and visits nodes in rings of increasing distance — so in an unweighted graph (all edges cost the same) the first time it reaches a node is via a shortest path. **Depth-first search (DFS)** uses a **stack** (or recursion, i.e. the call stack) and goes as deep as possible before backing up — good for "explore everything", connected components (groups of mutually reachable nodes), cycle detection and path enumeration. **Dijkstra's algorithm** is BFS generalised to non-negative edge weights, with a priority queue instead of a plain queue.

A graph is usually given as an edge list; build an **adjacency list** `List<List<Integer>>` (for each node, the list of nodes it links to). A grid is an implicit graph: each cell's neighbours are the four cells reached by the `DIRS` offsets.

**Recognise it when**: tree depth or level order; "number of islands/regions"; "fewest moves/steps" (BFS); "can you reach"; weighted "cheapest route" (Dijkstra).

### How it works (BFS)

1. Put the start in the queue and mark it seen.
2. Poll a node; for each unseen neighbour, mark it seen **when you enqueue it** and record its distance.
3. For level-by-level output, read `q.size()` at the start of each level and poll exactly that many.

```
// code/ch19/patterns/Graphs.java (key parts)
record TreeNode(int val, TreeNode left, TreeNode right) {}

static int depth(TreeNode n) {                       // DFS, recursive
    return n == null ? 0 : 1 + Math.max(depth(n.left()), depth(n.right()));
}

static List<List<Integer>> levels(TreeNode root) {  // BFS, level by level
    List<List<Integer>> out = new ArrayList<>();
    Deque<TreeNode> q = new ArrayDeque<>();
    if (root != null) q.offer(root);
    while (!q.isEmpty()) {
        List<Integer> level = new ArrayList<>();
        for (int size = q.size(); size > 0; size--) {
            TreeNode n = q.poll();
            level.add(n.val());
            if (n.left() != null) q.offer(n.left());
            if (n.right() != null) q.offer(n.right());
        }
        out.add(level);
    }
    return out;
}

static final int[][] DIRS = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};

static int shortestPath(String[] maze) {            // BFS on a grid, '#' = wall
    int rows = maze.length, cols = maze[0].length();
    int[][] dist = new int[rows][cols];
    for (int[] row : dist) Arrays.fill(row, -1);    // -1 = not seen
    Deque<int[]> q = new ArrayDeque<>();
    q.offer(new int[]{0, 0});
    dist[0][0] = 0;
    while (!q.isEmpty()) {
        int[] cur = q.poll();
        if (cur[0] == rows - 1 && cur[1] == cols - 1) return dist[cur[0]][cur[1]];
        for (int[] d : DIRS) {
            int nr = cur[0] + d[0], nc = cur[1] + d[1];
            if (nr < 0 || nc < 0 || nr >= rows || nc >= cols) continue;
            if (maze[nr].charAt(nc) == '#' || dist[nr][nc] != -1) continue;
            dist[nr][nc] = dist[cur[0]][cur[1]] + 1;
            q.offer(new int[]{nr, nc});
        }
    }
    return -1;
}

static long[] dijkstra(int n, int[][] edges, int src) {   // edges = {from, to, weight}
    // ... build List<List<int[]>> g
    long[] dist = new long[n];
    Arrays.fill(dist, Long.MAX_VALUE);
    dist[src] = 0;
    PriorityQueue<long[]> pq = new PriorityQueue<>(Comparator.comparingLong(x -> x[0]));
    pq.offer(new long[]{0, src});
    while (!pq.isEmpty()) {
        long[] top = pq.poll();
        int u = (int) top[1];
        if (top[0] > dist[u]) continue;             // stale entry: a shorter path was found already
        for (int[] e : g.get(u)) {
            long nd = dist[u] + e[1];
            if (nd < dist[e[0]]) { dist[e[0]] = nd; pq.offer(new long[]{nd, e[0]}); }
        }
    }
    return dist;
}
```

`islands` in the same file counts connected groups of `'1'` cells with an **explicit** `ArrayDeque` **stack** instead of recursion, so a 1000 × 1000 grid (~$10^6$ deep) can't cause `StackOverflowError`; `dfs` shows the recursive adjacency-list version. Java's `PriorityQueue` has no "decrease key", so Dijkstra re-inserts a node with its new distance and skips stale entries on the way out ("lazy deletion").

Output:

```
ok  tree depth -> 3
ok  level order -> [[3], [9, 20], [15, 7]]
ok  islands -> 3
ok  maze BFS -> 4
...
7 checks passed
```

**Complexity**: BFS/DFS O(V + E) (a grid: O(rows · cols)); Dijkstra O((V + E) log V).

### Common mistakes

- Marking a cell seen when it's **polled** rather than when it's **offered** — the same cell gets queued many times.
- `new boolean[rows][cols]` vs `[cols][rows]` swapped; `r < rows` vs `c < cols` bound mix-ups.
- Recursive DFS on huge inputs → `StackOverflowError`.
- Dijkstra with negative edge weights (wrong answers — use Bellman-Ford); forgetting the stale-entry check (still correct, but slower).
- Building an undirected graph (edges work in both directions) with only one direction of each edge.

---

## 19.17 Topological sort

### What it is

An ordering of a **directed acyclic graph** (DAG) in which every edge `u → v` puts u before v — a valid order to take courses, run build steps or apply database migrations. It exists only if there's no cycle, so the same algorithm doubles as cycle detection.

**Recognise it when**: prerequisites, dependencies, "order of tasks", "can all courses be finished", build systems, alien-dictionary letter order.

### How it works (Kahn's algorithm)

1. Count each node's **in-degree** (number of incoming edges).
2. Queue every node with in-degree 0 — nothing has to come before them.
3. Poll a node, append it to the order, and decrement its neighbours' in-degrees; enqueue any that reach 0.
4. If the order has fewer than n nodes, the rest are stuck in a cycle.

```
// code/ch19/patterns/TopoSort.java (key parts)
static List<Integer> kahn(int n, int[][] prereqs) {        // {a, b}: b before a
    List<List<Integer>> g = new ArrayList<>();
    for (int i = 0; i < n; i++) g.add(new ArrayList<>());
    int[] indegree = new int[n];
    for (int[] p : prereqs) { g.get(p[1]).add(p[0]); indegree[p[0]]++; }

    Deque<Integer> ready = new ArrayDeque<>();
    for (int i = 0; i < n; i++) if (indegree[i] == 0) ready.offer(i);
    List<Integer> order = new ArrayList<>();
    while (!ready.isEmpty()) {
        int u = ready.poll();
        order.add(u);
        for (int v : g.get(u)) if (--indegree[v] == 0) ready.offer(v);
    }
    return order.size() == n ? order : List.of();           // leftovers are in a cycle
}

// DFS version: colour 0 = unvisited, 1 = on the current path, 2 = finished.
private static boolean visit(List<List<Integer>> g, int u, int[] colour, Deque<Integer> finished) {
    colour[u] = 1;
    for (int v : g.get(u)) {
        if (colour[v] == 1) return false;                  // back edge -> cycle
        if (colour[v] == 0 && !visit(g, v, colour, finished)) return false;
    }
    colour[u] = 2;
    finished.push(u);                                      // reverse post-order
    return true;
}
```

Output:

```
ok  kahn order -> [0, 1, 2, 3]
ok  kahn cycle -> []
ok  dfs order -> [0, 2, 1, 3]
ok  dfs cycle -> []
4 checks passed
```

Both `[0, 1, 2, 3]` and `[0, 2, 1, 3]` are valid — a DAG usually has many topological orders. For the lexicographically *smallest* order, swap the queue for a `PriorityQueue`.

**Complexity**: O(V + E) time and space.

### Common mistakes

- Reversing the edge direction (read `{a, b}` carefully).
- In DFS, using a single `visited` flag: it can't tell "on the current path" (a cycle) from "finished earlier" (fine) — that's why there are three colours.

---

## 19.18 Union-find (disjoint set union)

### What it is

A structure that tracks which elements belong to the same group and merges groups quickly. Each group is a tree; its root is the group's representative. `find(x)` walks to the root; `union(a, b)` links one root under the other. With **path compression** (make nodes point closer to the root while finding) and **union by size** (hang the smaller tree under the larger), each operation costs effectively O(1) — formally O(α(n)), the inverse Ackermann function, which is below 5 for any realistic n.

**Recognise it when**: "number of connected components" with edges arriving over time; "does adding this edge create a cycle?"; grouping accounts/emails that share an attribute; Kruskal's minimum spanning tree (the cheapest set of edges that connects every node without forming a cycle).

### How it works

1. Start with every element as its own root (`parent[i] = i`).

2. `find`: follow `parent` pointers to the root, halving the path as you go.
3. `union`: find both roots; if equal they were already connected (return false); otherwise attach the smaller tree to the larger and decrement the component count.

```
// code/ch19/patterns/UnionFind.java (key parts)
int find(int x) {
    while (parent[x] != x) {
        parent[x] = parent[parent[x]];         // path halving: point to grandparent
        x = parent[x];
    }
    return x;
}

boolean union(int a, int b) {
    int ra = find(a), rb = find(b);
    if (ra == rb) return false;                // already connected: this edge closes a cycle
    if (size[ra] < size[rb]) { int t = ra; ra = rb; rb = t; }
    parent[rb] = ra;                           // smaller tree under the larger
    size[ra] += size[rb];
    components--;
    return true;
}

static int kruskal(int n, int[][] edges) {     // minimum spanning tree weight
    int[][] sorted = edges.clone();
    Arrays.sort(sorted, Comparator.comparingInt(e -> e[2]));
    UnionFind uf = new UnionFind(n);
    int total = 0;
    for (int[] e : sorted) if (uf.union(e[0], e[1])) total += e[2];
    return total;
}
```

Output:

```
ok  components -> 3
ok  redundant edge -> [2, 3]
ok  kruskal MST -> 19
...
5 checks passed
```

**Complexity**: O(α(n)) amortised per operation; O(n) space. Kruskal is O(E log E) for the sort.

### Common mistakes

- Comparing `parent[a] == parent[b]` instead of `find(a) == find(b)`.
- Linking `a` to `b` instead of **root** to root.
- Using union-find for **directed** reachability — it only models undirected connectivity.

---

## 19.19 Dynamic programming

### What it is

**Dynamic programming (DP)** solves a problem by combining answers to smaller versions of the same problem, computing each smaller answer **once**. It applies when a problem has **optimal substructure** (the best answer is built from best answers to subproblems) and **overlapping subproblems** (plain recursion would recompute the same subproblem many times — naive Fibonacci is exponential).

Two ways to write it:

| | **Memoisation (top-down)** | **Tabulation (bottom-up)** |
|---|---|---|
| Shape | Recursive function + cache | Loops filling an array |
| Computes | Only the states actually reached | Every state in the table |
| Order | Natural — recursion finds it | You must choose a loop order where dependencies come first |
| Risk in Java | `StackOverflowError` on deep recursion; `HashMap` memo boxes keys | Off-by-one indexes |
| Space optimisation | Hard | Easy — keep only the rows you need |

Java has no `@cache` decorator (unlike Python's `functools.cache`); use an array (fast; pick a sentinel such as 0 or -1 that can't be a real answer) or a `HashMap` keyed by a record for multi-part states. Don't use `computeIfAbsent` with a recursive mapping function on a `HashMap` — modifying the map inside its own mapping function throws `ConcurrentModificationException` since Java 9 (memoisation note in §11.8); use get-then-put as in `editDistance`.

**Recognise it when**: "number of ways", "minimum/maximum cost", "longest/shortest … subsequence" (order kept, not necessarily adjacent), "can you reach/make", choices at each step whose effects add up, and a brute force that branches.

### How to design a DP (the five questions)

1. **State**: what does `dp[i]` (or `dp[i][j]`) mean, in words?
2. **Transition**: how is a state computed from smaller ones?
3. **Base cases**.
4. **Order**: which loop direction guarantees dependencies are ready?
5. **Answer**: which state holds it? Then shrink space if only the last row/two values are used.

```java
// code/ch19/patterns/DynamicProgramming.java (key parts)
// 1D, top-down: ways to climb n stairs taking 1 or 2 steps
static long climbMemo(int n, long[] memo) {
    if (n <= 1) return 1;
    if (memo[n] != 0) return memo[n];
    return memo[n] = climbMemo(n - 1, memo) + climbMemo(n - 2, memo);
}
// 1D, bottom-up in O(1) space
static long climbTab(int n) {
    long prev2 = 1, prev1 = 1;
    for (int i = 2; i <= n; i++) { long cur = prev1 + prev2; prev2 = prev1; prev1 = cur; }
    return prev1;
}

// Coin change: dp[x] = fewest coins that make x
static int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, Integer.MAX_VALUE);
    dp[0] = 0;
    for (int x = 1; x <= amount; x++)
        for (int c : coins)
            if (c <= x && dp[x - c] != Integer.MAX_VALUE)   // guard: MAX_VALUE + 1 would overflow
                dp[x] = Math.min(dp[x], dp[x - c] + 1);
    return dp[amount] == Integer.MAX_VALUE ? -1 : dp[amount];
}

// 2D: longest common subsequence, dp[i][j] = LCS of a[0..i) and b[0..j)
static int lcs(String a, String b) {
    int[][] dp = new int[a.length() + 1][b.length() + 1];
    for (int i = 1; i <= a.length(); i++)
        for (int j = 1; j <= b.length(); j++)
            dp[i][j] = a.charAt(i - 1) == b.charAt(j - 1)
                    ? dp[i - 1][j - 1] + 1
                    : Math.max(dp[i - 1][j], dp[i][j - 1]);
    return dp[a.length()][b.length()];
}

// 0/1 knapsack in one row: iterate capacity DOWNWARDS so each item is used at most once
static int knapsack(int[] weight, int[] value, int capacity) {
    int[] dp = new int[capacity + 1];
    for (int item = 0; item < weight.length; item++)
        for (int c = capacity; c >= weight[item]; c--)
            dp[c] = Math.max(dp[c], dp[c - weight[item]] + value[item]);
    return dp[capacity];
}
```

The file also has house robber (`dp[i] = max(dp[i-1], dp[i-2] + a[i])` in two variables), longest increasing subsequence in O(n log n) with `Arrays.binarySearch` on a `tails` array, edit distance top-down with a `HashMap<Key, Integer>` memo (`record Key(int i, int j)`), and grid paths with a single rolling row.

Output:

```
ok  climb memo n=50 -> 20365011074
ok  climb tab n=50 -> 20365011074
ok  house robber -> 12
ok  coin change 15 -> 3
...
10 checks passed
```

Note `coinChange({1, 5, 11}, 15) = 3` (5 + 5 + 5): a greedy "largest coin first" would take 11 + 1 + 1 + 1 + 1 = 5 coins — the classic reason this problem needs DP. And `climb(50)` is 20,365,011,074, beyond `int` range — hence `long`.

**Complexity**: states × work per state. Climbing O(n); coin change O(amount · coins); LCS and edit distance O(m · n); knapsack O(items · capacity).

### Common mistakes

- Undefined state ("dp[i] is… something about i") — write the sentence first.

- 0/1 knapsack with the capacity loop going **up** (items get reused — that's the unbounded version).
- `Integer.MAX_VALUE + 1` as "impossible + one coin" wrapping to negative.
- `int` answers for counting problems that exceed $2^{31}$ — use `long` or the requested modulus (`% 1_000_000_007`).

---

## 19.20 Backtracking

### What it is

A systematic search that builds a candidate one choice at a time, recurses, then **undoes** the choice to try the next one. **Pruning** (abandoning a branch as soon as it can't succeed) is what makes it fast enough.

**Recognise it when**: "all subsets / permutations / combinations", "all valid arrangements", N-Queens, Sudoku, word search in a grid, partitioning a string; small n (≤ ~20).

### How it works (template)

1. If the current path is a complete answer, record a **copy** of it.
2. For each available choice: **choose** (add to path, mark used) → **explore** (recurse) → **un-choose** (remove, unmark).
3. Skip choices that already violate a constraint (prune).

```java
// code/ch19/patterns/Backtracking.java (key parts)
private static void subsets(int[] nums, int start, List<Integer> path, List<List<Integer>> out) {
    out.add(new ArrayList<>(path));                   // COPY: path keeps changing
    for (int i = start; i < nums.length; i++) {
        path.add(nums[i]);                            // choose
        subsets(nums, i + 1, path, out);              // explore
        path.removeLast();                            // un-choose (Java 21; or remove(size()-1))
    }
}

private static void combo(int[] c, int remaining, int start, List<Integer> path, List<List<Integer>> out) {
    if (remaining == 0) { out.add(List.copyOf(path)); return; }
    for (int i = start; i < c.length && c[i] <= remaining; i++) {   // sorted input: prune
        path.add(c[i]);
        combo(c, remaining - c[i], i, path, out);                    // i, not i+1: reuse allowed
        path.removeLast();
    }
}

private static int place(int n, int row, boolean[] cols, boolean[] diag, boolean[] anti) {  // N-Queens
    if (row == n) return 1;
    int count = 0;
    for (int c = 0; c < n; c++) {
        int d = row - c + n, a = row + c;             // each diagonal has a unique index
        if (cols[c] || diag[d] || anti[a]) continue;
        cols[c] = diag[d] = anti[a] = true;
        count += place(n, row + 1, cols, diag, anti);
        cols[c] = diag[d] = anti[a] = false;
    }
    return count;
}
```

`permutations` uses a `boolean[] used` instead of a start index, since every element can appear at any position.

Output:

```
ok  subsets -> [[], [1], [1, 2], [1, 2, 3], [1, 3], [2], [2, 3], [3]]
ok  permutations count -> 6
ok  permutations of 2 -> [[1, 2], [2, 1]]
ok  combination sum -> [[2, 2, 3], [7]]
...
5 checks passed
```

**Complexity**: subsets $O(2^n \cdot n)$, permutations $O(n! \cdot n)$; pruning improves the practical cost, not the worst case.

### Common mistakes

- `out.add(path)` without copying: every stored answer is the same list object, empty at the end.
- `path.remove(x)` on a `List<Integer>` with an `int` x removes by **index** (§19.6). `removeLast()` (Java 21) or `remove(path.size() - 1)` is unambiguous.
- Forgetting to un-mark `used[i]` or the diagonal flags.
- Duplicates in the input: sort, then skip `nums[i] == nums[i - 1]` at the same depth.

---

## 19.21 Tries

### What it is

A **trie** (prefix tree, pronounced "try") stores strings character by character: each node has one child per possible next character and a flag saying "a word ends here". Words sharing a prefix share the path, so "does any word start with `car`?" costs O(length of `car`) however many words are stored. Real uses: autocomplete, spell-check, IP routing tables (longest-prefix match), word games.

**Recognise it when**: many prefix queries; autocomplete; "word search II" (many words in one grid); replacing words by their shortest root.

### How it works

1. Start at the root. For each character, follow (or create, when inserting) the child for that character.
2. At the end of an insert, set `isWord = true`.
3. `contains` needs the path **and** `isWord`; `startsWith` only needs the path.

```java
// code/ch19/patterns/Trie.java (key parts)
private static final class Node {
    final Node[] next = new Node[26];     // lowercase a-z; HashMap<Character, Node> for big alphabets
    boolean isWord;
    int prefixCount;                      // how many inserted words pass through this node
}
private final Node root = new Node();

void insert(String word) {
    Node n = root;
    for (char ch : word.toCharArray()) {
        int i = ch - 'a';
        if (n.next[i] == null) n.next[i] = new Node();
        n = n.next[i];
        n.prefixCount++;
    }
    n.isWord = true;
}

private Node walk(String s) {
    Node n = root;
    for (char ch : s.toCharArray()) {
        n = n.next[ch - 'a'];
        if (n == null) return null;
    }
    return n;
}

boolean contains(String word)     { Node n = walk(word); return n != null && n.isWord; }
boolean startsWith(String prefix) { return walk(prefix) != null; }
```

`complete(prefix)` walks to the prefix node and then does a DFS over children `a`…`z` with a shared `StringBuilder` (append, recurse, `deleteCharAt` — backtracking again), so results come out in alphabetical order.

Output:

```
ok  count prefix car -> 3
ok  autocomplete car -> [car, card, care]
ok  autocomplete none -> []
...
6 checks passed
```

**Complexity**: insert and lookup O(L) for a word of length L; space O(total characters × alphabet) with arrays — a `HashMap` per node uses less memory for sparse alphabets but is slower.

### Common mistakes

- Treating "path exists" as "word exists" (`ca` is only a prefix).
- `ch - 'a'` on uppercase or non-letters → negative index `ArrayIndexOutOfBoundsException`. Normalise input or use a map.

## 19.22 Linked lists and fast/slow pointers

### What it is

A **singly linked list** is a chain of nodes, each holding a value and a reference `next` to the following node; the last node's `next` is `null`. There is no index: reaching the k-th node means walking k steps. Interview list problems are therefore about **pointer bookkeeping** — which references to keep so that you never lose the rest of the chain. Two tools do most of the work:

- A **dummy (sentinel) node** placed in front of the head, so "delete or insert at the head" is not a special case.
- **Fast and slow pointers**: two references walking at different speeds (usually 1 and 2 steps) or with a fixed gap. When fast reaches the end, slow is at the middle; if the list loops, fast eventually laps

slow — **Floyd's cycle-finding algorithm** ("tortoise and hare").

§19.8 already shows `reverse`, `middle` and `hasCycle`; this section adds the other classic list problems.

**Recognise it when**: "linked list" in the signature; k-th node from the end; middle; cycle or where it starts; two lists that merge; palindrome list in O(1) space; any sequence `x → f(x)` that might repeat (happy number, "find the duplicate number").

## How it works

1. **k-th from the end in one pass**: move `lead` k steps ahead of `trail`, then move both until `lead` is at the last node; `trail` is now just **before** the node to remove.
2. **Intersection**: pointer `p` walks list A then list B, `q` walks B then A. Both travel `lenA + lenB` steps, so they arrive at the first shared node together (or both reach `null`).
3. **Palindrome in O(1) space**: find the middle with fast/slow, reverse the second half, compare the halves, then reverse it back so the caller's list is unchanged.
4. **Where does the cycle start?** After slow and fast meet, start a new pointer at the head and step it and slow one node at a time; they meet at the cycle's first node. Why: if the tail before the loop has length a and they met b steps into the loop of length L, slow walked a + b and fast 2(a + b), and the difference a + b is a multiple of L — so a steps from the meeting point also lands on the loop's start.

```java
// code/ch19/patterns/LinkedLists.java (key parts)
static Node removeKthFromEnd(Node head, int k) {
    Node dummy = new Node(0);                  // a dummy in front makes "remove the head" a normal case
    dummy.next = head;
    Node lead = dummy, trail = dummy;
    for (int i = 0; i < k; i++) lead = lead.next;          // open a gap of k
    while (lead.next != null) { lead = lead.next; trail = trail.next; }
    trail.next = trail.next.next;              // trail stops just BEFORE the node to delete
    return dummy.next;
}

static Node intersection(Node a, Node b) {
    Node p = a, q = b;
    while (p != q) {                           // reference comparison: same node object, not same value
        p = (p == null) ? b : p.next;
        q = (q == null) ? a : q.next;
    }
    return p;
}

static boolean isPalindrome(Node head) {
    Node slow = head, fast = head;
    while (fast != null && fast.next != null) { slow = slow.next; fast = fast.next.next; }
    Node secondHalf = reverse(slow);
    boolean ok = true;
    for (Node p = head, q = secondHalf; q != null; p = p.next, q = q.next)
        if (p.val != q.val) { ok = false; break; }
    reverse(secondHalf);                       // put the caller's list back the way it was
    return ok;
}

static Node cycleStart(Node head) {
    Node slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next; fast = fast.next.next;
        if (slow == fast) {
            Node p = head;
            while (p != slow) { p = p.next; slow = slow.next; }
            return p;
        }
    }
    return null;
}
```

The file also has `isHappy(n)`: a number is "happy" if repeatedly replacing it by the sum of the squares of its digits reaches 1. The sequence either hits 1 or repeats forever, so fast/slow on **numbers** (`slow =

`next(slow)`, `fast = next(next(fast))`) detects the loop without a `HashSet`.

Output:

```
ok  remove 2nd from end -> 1->2->3->5
ok  remove the head -> 2
ok  intersection node -> 8
ok  no intersection -> null
...
12 checks passed
```

**Complexity**: all O(n) time, O(1) extra space (intersection O(m + n)).

### Common mistakes

- Losing the rest of the list: save `next` **before** you overwrite `cur.next`.
- No dummy node, so removing the head needs its own `if` (and is usually the bug).
- Comparing nodes with `equals` or by `val` when the question is "the same node" — use `==` on references.
- Checking `fast.next.next` without `fast != null && fast.next != null` first → `NullPointerException`.
- Leaving the input modified (a reversed half) when the caller still uses the list.

---

## 19.23 Trees: recursion templates, LCA and serialisation

### What it is

A **binary tree** node has a value and up to two children. Almost every tree problem is solved by recursion in one of two shapes:

- **Top-down**: pass information **down** as parameters (the allowed range, the depth so far, the path so far). Used for "validate a binary search tree", "root-to-leaf path sum".
- **Bottom-up**: each call **returns** a summary of its subtree (height, "did I find p?", best downward path) and the parent combines its children's answers. Used for height, diameter, balance check, lowest common ancestor, maximum path sum. When the answer is not the same thing a parent needs, return one value and record the overall best in a side variable.

A **binary search tree (BST)** keeps every key in the left subtree smaller and every key in the right subtree larger than the node — for the **whole** subtree, not just the direct children. The **lowest common ancestor (LCA)** of p and q is the deepest node that has both below it (a node counts as its own descendant). **Serialising** a tree means turning it into a string that can be turned back into the identical tree. §19.16 has depth and level-order BFS; this section adds the recursion templates.

**Recognise it when**: "tree" plus height, depth, balanced, diameter, path sum, "is it a valid BST", "lowest common ancestor", "serialize / deserialize", "build the tree from traversals".

### How it works

1. **Validate a BST (top-down)**: each node must lie strictly inside a range `(low, high)`. Going left, the node's value becomes the new `high`; going right, the new `low`. Use `long` bounds so a key equal to `Integer.MIN_VALUE` or `Integer.MAX_VALUE` is still allowed.
2. **LCA (bottom-up)**: return the node itself if it is p or q; otherwise ask both children. If both sides found something, this node is where p and q split — it is the LCA. Otherwise pass up whichever side found something.
3. **Maximum path sum (bottom-up with a side result)**: a call returns the best path going **down** from its node (a parent can extend only one branch); at each node also record the best path that **bends** through it (left + node + right). Negative branches are replaced by 0 — better not to take them.
4. **Serialise**: preorder (node, then left subtree, then right) and write `#` for every `null`. The null markers make the shape unambiguous, so one traversal is enough. **Deserialise** reads tokens in the same order with a queue.

```
// code/ch19/patterns/Trees.java (key parts)
record TreeNode(int val, TreeNode left, TreeNode right) {}

private static boolean valid(TreeNode n, long low, long high) {        // top-down
    if (n == null) return true;
    if (n.val() <= low || n.val() >= high) return false;
    return valid(n.left(), low, n.val()) && valid(n.right(), n.val(), high);
}

static TreeNode lca(TreeNode root, int p, int q) {                      // bottom-up
    if (root == null || root.val() == p || root.val() == q) return root;
    TreeNode left = lca(root.left(), p, q), right = lca(root.right(), p, q);
    if (left != null && right != null) return root;       // p and q split here
    return left != null ? left : right;
}

private static int gain(TreeNode n, int[] best) {                      // bottom-up + side result
    if (n == null) return 0;
    int left = Math.max(0, gain(n.left(), best));          // a negative branch is better skipped
    int right = Math.max(0, gain(n.right(), best));
    best[0] = Math.max(best[0], n.val() + left + right);   // path that bends at n
    return n.val() + Math.max(left, right);                // a parent can extend only one side
}

private static void write(TreeNode n, StringJoiner out) {              // serialise: preorder with "#"
    if (n == null) { out.add("#"); return; }
    out.add(String.valueOf(n.val()));
    write(n.left(), out);
    write(n.right(), out);
}

private static TreeNode read(Deque<String> tokens) {                   // deserialise: same order
    String t = tokens.poll();
    if (t.equals("#")) return null;
    TreeNode left = read(tokens);
    TreeNode right = read(tokens);
    return new TreeNode(Integer.parseInt(t), left, right);
}
```

The node is a `record`, so it is immutable (children are built first, then the parent) and two trees with the same shape and values are `equals` — which makes the round-trip check a one-liner. `int[] best` is the usual Java trick for a mutable "side result" inside a recursive helper (a field works too).

Output:

```
ok  grandchild breaks the BST rule -> false
ok  lca(5, 1) -> 3
ok  lca(7, 4) -> 2
ok  lca where one is the ancestor -> 5
...
11 checks passed
```

**Complexity**: each is O(n) time — every node visited once — and O(h) stack space, where the height h is O(log n) for a balanced tree and O(n) for a degenerate one (a "linked list" tree of $10^5$ nodes can throw `StackOverflowError`; then use an explicit `ArrayDeque`, §19.16).

## Common mistakes

- Validating a BST by comparing each node only with its direct children — a grandchild can break the rule (the second check above).
- `int` bounds with `Integer.MIN_VALUE`/`MAX_VALUE` as "no limit" — a legal key equal to them is rejected; use `long` or `Integer` with `null`.
- Starting `best` at 0 in max path sum — an all-negative tree must return its largest (negative) value.
- Serialising without null markers, or with preorder for write and a different order for read.
- Forgetting that a node can be its own ancestor in LCA.

## 19.24 Greedy

### What it is

A **greedy algorithm** builds the answer one step at a time, taking the choice that looks best **right now** and never undoing it. It is fast (often one pass after a sort) but only correct when the problem has the **greedy-choice property**: some optimal answer starts with the locally best choice. Unlike DP (§19.19), which tries all choices for each subproblem, greedy commits to one. Example: "largest coin that fits" is right for euro coins but wrong for coins {1, 3, 4} and amount 6 (greedy gives 4 + 1 + 1, the best is 3 + 3) — exactly why coin change is a DP problem.

Proving a greedy choice is safe usually takes one of two arguments: **exchange** ("take any optimal answer that didn't make my choice; swapping my choice in doesn't make it worse") or **stays ahead** ("after every step my partial answer is at least as good as any other").

**Recognise it when**: "minimum number of …", "maximum number of non-overlapping …", "can you reach …", jump game, gas station, candies, assigning cookies/boats/meeting rooms, intervals sorted by end time (§19.15), and when constraints ($n = 10^5$) rule out DP over pairs.

### How it works

1. **Jump game** (each value is the maximum jump from that index): sweep left to right keeping `farthest`, the furthest index reachable so far. If you ever stand on an index beyond `farthest`, you're stuck. For the **fewest jumps**, treat it as BFS without a queue: the indexes reachable with j jumps form a range; when you reach its end, take another jump and the new range ends at `farthest`.
2. **Gas station** (circular route; `gas[i]` fuel at station i, `cost[i]` to reach the next): if total gas < total cost, no start works. Otherwise sweep with a running tank; whenever it drops below 0 at station i, no station from the current start to i can be the start (each would arrive at i with even less), so restart at i + 1.
3. **Candies** (every child gets ≥ 1; a child with a higher rating than a neighbour gets more than that neighbour): one pass left→right satisfies the left-neighbour rule, one pass right→left satisfies the right-neighbour rule, taking the `max` so the first pass's constraint is kept.

```
// code/ch19/patterns/Greedy.java (key parts)
static boolean canJump(int[] jumps) {
    int farthest = 0;
    for (int i = 0; i < jumps.length; i++) {
        if (i > farthest) return false;                  // stuck: i is beyond every reachable index
        farthest = Math.max(farthest, i + jumps[i]);
    }
    return true;
}

static int minJumps(int[] jumps) {
    int count = 0, end = 0, farthest = 0;
    for (int i = 0; i < jumps.length - 1; i++) {
        farthest = Math.max(farthest, i + jumps[i]);
        if (i == end) { count++; end = farthest; }         // the current range is used up: jump
    }
    return count;
}

static int gasStation(int[] gas, int[] cost) {
    int total = 0, tank = 0, start = 0;
    for (int i = 0; i < gas.length; i++) {
        int diff = gas[i] - cost[i];
        total += diff;
        tank += diff;
        if (tank < 0) { start = i + 1; tank = 0; }        // no station in [start..i] can work
    }
    return total >= 0 ? start : -1;
}

static int candies(int[] ratings) {
    int n = ratings.length;
    int[] c = new int[n];
    Arrays.fill(c, 1);
    for (int i = 1; i < n; i++)
        if (ratings[i] > ratings[i - 1]) c[i] = c[i - 1] + 1;
    for (int i = n - 2; i >= 0; i--)
        if (ratings[i] > ratings[i + 1]) c[i] = Math.max(c[i], c[i + 1] + 1);
    int sum = 0;
    for (int x : c) sum += x;
    return sum;
}
```

Output:

```
ok  canJump blocked by 0 -> false
ok  minJumps -> 2
ok  minJumps already there -> 0
ok  gas station start -> 3
...
9 checks passed
```

**Complexity**: jump game and gas station O(n) time, O(1) space; candies O(n) time, O(n) space. Greedy problems that sort first are O(n log n).

### Common mistakes

- Using greedy without a reason it is safe — try a small counter-example (like coins {1, 3, 4}) before coding; if one exists, it's DP.
- In `minJumps`, looping to the last index too — you'd count one extra jump when a range ends exactly there.
- Candies in one pass, or overwriting the first pass's value instead of taking `max`.
- Gas station with a nested loop trying every start: O(n²) and times out.

---

## 19.25 Sort and search: quicksort, quickselect, Dutch flag, merge sort on a list

### What it is

You normally call `Arrays.sort` (§19.5), but interviews ask you to **write** the pieces of sorting algorithms because they solve other problems:

- **Partition**: rearrange an array around a **pivot** value so that everything smaller is on its left. It is the core of **quicksort** (partition, then sort both sides) and of **quickselect** (partition, then continue on the **one** side that holds the k-th element — average O(n), no full sort). The **Lomuto** partition used here keeps a `store` index: everything left of it is `< pivot`.
- **Three-way partition** (the **Dutch national flag** problem, named by Edsger Dijkstra after the flag's three stripes): sort an array of only 0s, 1s and 2s in one pass.
- **Merge sort** splits in half, sorts each half and **merges** two sorted runs. It needs no random access, so it is the natural way to sort a **linked list** in O(n log n).

**Recognise it when**: "k-th largest/smallest" without needing the rest sorted; "sort colors", "move all x to the end", "partition around a value"; "sort a linked list"; "median without sorting"; "count inversions" (a merge-sort variant).

### How it works

1. **Partition**: swap a **random** element to the end as the pivot (random pivots make the $O(n^2)$ worst case, e.g. already-sorted input, practically impossible). Walk `i` over the rest; each value smaller than the pivot is swapped to `store++`. Finally swap the pivot into `store` — its final sorted position.
2. **Quickselect for the k-th largest**: the target index in ascending order is `n - k`. Partition; if the pivot landed on the target you're done, otherwise narrow `lo`/`hi` to the side containing it.
3. **Dutch flag**: three regions `[0, low)` = 0s, `[low, mid)` = 1s, `(high, end]` = 2s, and `[mid, high]` still unknown. Look at `a[mid]`: 0 → swap to `low`, advance both; 1 → advance `mid`; 2 → swap with `high`, shrink `high`, and **don't** advance `mid` (the value just swapped in hasn't been looked at).
4. **Merge sort on a list**: find the middle with fast/slow (§19.22), cut the list there, sort each half recursively, and merge by relinking nodes behind a dummy head.

```
// code/ch19/patterns/SortSearch.java (key parts)
static int partition(int[] a, int lo, int hi) {
    swap(a, lo + RNG.nextInt(hi - lo + 1), hi);           // random pivot defeats sorted-input worst case
    int pivot = a[hi], store = lo;
    for (int i = lo; i < hi; i++) if (a[i] < pivot) swap(a, i, store++);
    swap(a, store, hi);
    return store;
}

static int kthLargest(int[] input, int k) {                // quickselect
    int[] a = input.clone();                               // don't reorder the caller's array
    int target = a.length - k;
    int lo = 0, hi = a.length - 1;
    while (true) {
        int p = partition(a, lo, hi);
        if (p == target) return a[p];
        if (p < target) lo = p + 1; else hi = p - 1;
    }
}

static void sortColors(int[] a) {                          // Dutch national flag
    int low = 0, mid = 0, high = a.length - 1;
    while (mid <= high) {
        if (a[mid] == 0) swap(a, low++, mid++);
        else if (a[mid] == 1) mid++;
        else swap(a, mid, high--);                         // don't advance mid
    }
}

static Node mergeSort(Node head) {                         // merge sort on a linked list
    if (head == null || head.next == null) return head;
    Node slow = head, fast = head.next;                   // slow stops at the END of the left half
    while (fast != null && fast.next != null) { slow = slow.next; fast = fast.next.next; }
    Node right = slow.next;
    slow.next = null;                                      // cut the list in two
    return merge(mergeSort(head), mergeSort(right));
}
```

`quickSort` is the three-line recursion around `partition`, and `merge` is the classic two-pointer merge with `<=` so equal values keep their order (stable). `RNG` is a `Random` with a fixed seed so runs are repeatable.

Output:

```
ok  quickSort -> [-3, 0, 1, 2, 5, 5, 6, 9]
ok  quickselect 2nd largest -> 5
ok  quickselect with duplicates -> 4
ok  input left untouched -> [3, 2, 1, 5, 6, 4]
...
7 checks passed
```

**Complexity**: quicksort O(n log n) average, $O(n^2)$ worst, O(log n) stack on average; quickselect O(n) average, $O(n^2)$ worst; Dutch flag O(n), O(1) space; merge sort on a list O(n log n) time, O(log n) recursion stack. Compare quickselect with the size-k heap of §19.13 (O(n log k), streams well, never quadratic) — say both in the interview.

### Common mistakes

- A fixed pivot (first or last element) on sorted or all-equal input → $O(n^2)$ and deep recursion. Randomise; for many duplicates a three-way partition helps.
- Mixing up k-th **largest** (index `n - k`) with k-th smallest (index `k - 1`).
- Advancing `mid` after swapping with `high` in the Dutch flag.
- Finding the middle with `fast = head` in list merge sort — for two nodes the split never shrinks and recursion never ends.
- Quickselect reordering the caller's array without saying so — clone, or state it.

---

## 19.26 Bit manipulation

### What it is

An `int` is 32 bits in **two's complement** (the top bit is the sign; `-x == ~x + 1`). Bitwise operators work on all 32 bits at once in one CPU instruction: `&` (and), `|` (or), `^` (xor: 1 when the bits differ), `~` (not), `<<` (shift left, × 2 per step), `>>` (arithmetic shift right, copies the sign bit) and `>>>` (logical shift right, fills with 0) — see §2.8. Interview problems use a handful of identities:

| Identity | Meaning |
|---|---|
| `x ^ x == 0`, `x ^ 0 == x` | equal values cancel under xor |
| `x & (x - 1)` | x with its lowest 1-bit cleared |
| `x & -x` | only the lowest 1-bit of x |
| `(x >> i) & 1` | bit i of x |
| `x \| (1 << i)`, `x & ~(1 << i)`, `x ^ (1 << i)` | set, clear, toggle bit i |
| `1 << n` | $2^n$, and the number of subsets of n items |

Java also has the library calls `Integer.bitCount`, `Integer.highestOneBit`, `Integer.numberOfTrailingZeros` and `Integer.toBinaryString` — say you'd use them in real code.

**Recognise it when**: "every element appears twice except one"; "without extra space" with integers; count 1-bits; powers of two; "swap without a temp"; subsets of up to ~20 items (a bitmask — an integer whose bits are on/off flags — per subset); flags packed into one number.

### How it works

1. **Single number**: xor everything together — pairs cancel and the lone value remains. O(n), O(1) space, no `HashSet`.
2. **Counting bits for 0..n**: a tiny DP — `i >> 1` is i without its last bit, so `bits[i] = bits[i >> 1] + (i & 1)`.
3. **Swap odd and even bits**: mask the even positions with `0x55555555` (binary 0101…) and the odd with `0xAAAAAAAA` (1010…), shift each one place toward the other, and `|` them. Use `>>>` so a set sign bit doesn't copy itself in.
4. **Power of two**: exactly one 1-bit, so `n > 0 && (n & (n - 1)) == 0`.
5. **Subsets by mask**: loop `mask` from 0 to $2^n - 1$; bit i of `mask` says whether item i is in this subset — an iterative alternative to backtracking (§19.20).

```java
// code/ch19/patterns/BitManipulation.java (key parts)
static int singleNumber(int[] a) {
    int x = 0;
    for (int v : a) x ^= v;
    return x;
}

static int[] countBits(int n) {
    int[] bits = new int[n + 1];
    for (int i = 1; i <= n; i++) bits[i] = bits[i >> 1] + (i & 1);
    return bits;
}

static int swapOddEvenBits(int x) {
    int even = x & 0x55555555;                 // bits 0, 2, 4, ...
    int odd  = x & 0xAAAAAAAA;                 // bits 1, 3, 5, ...
    return (even << 1) | (odd >>> 1);
}

static boolean isPowerOfTwo(int n) {
    return n > 0 && (n & (n - 1)) == 0;
}

static List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> out = new ArrayList<>();
    for (int mask = 0; mask < (1 << nums.length); mask++) {
        List<Integer> s = new ArrayList<>();
        for (int i = 0; i < nums.length; i++) if ((mask >> i & 1) == 1) s.add(nums[i]);
        out.add(s);
    }
    return out;
}
```

Output:

```
ok  singleNumber -> 4
ok  singleNumber negative -> -7
ok  countBits 0..5 -> [0, 1, 1, 2, 1, 2]
ok  matches Integer.bitCount -> 2
...
11 checks passed
```

184 is `1011 1000` in binary; swapping each pair of bits gives `0111 0100` = 116.

**Complexity**: single number and counting bits O(n); the bit tricks O(1); subsets O($2^n$ · n).

### Common mistakes

- Operator precedence: `x & 1 == 0` parses as `x & (1 == 0)` and doesn't compile for `int`; write `(x & 1) == 0`.
- `>>` on negative numbers when you meant `>>>` — the sign bit smears in.
- `1 << 32` is **1**, not 0 (the shift count is taken mod 32); use `1L << k` for 64-bit masks.
- Forgetting `n > 0` in the power-of-two test: `0 & -1 == 0` would say 0 is a power of two, and `Integer.MIN_VALUE` also has a single 1-bit.

---

## 19.27 Math and geometry

### What it is

A small family of problems where the trick is arithmetic or coordinates rather than a data structure: walking a matrix in a spiral, reversing an integer's digits **without overflow**, counting points on one straight line, and closed-form or recurrence answers such as the **Josephus problem** (n people in a circle, every k-th is removed; who survives?). The Java-specific part is almost always **overflow** (§19.6) and **exact arithmetic** — avoid `double` for anything you compare for equality.

**Recognise it when**: "spiral order", "rotate the matrix"; "reverse the digits", "string to integer", "return 0 if it overflows"; "points on the same line", slopes; "circle of people, every k-th"; GCD/LCM, primes (sieve), modular arithmetic `mod 1_000_000_007`.

## How it works

1. **Spiral order**: keep four boundaries `top`, `bottom`, `left`, `right`. Walk the top row left→right, then the right column down, then (if a row remains) the bottom row right→left, then (if a column remains) the left column up — shrinking the boundary after each side. The two `if`s stop a single remaining row or column being printed twice.
2. **Reverse an int**: pop the last digit with `% 10`, push it with `result * 10 + digit`. Before multiplying, check that `result` is within `Integer.MAX_VALUE / 10` and `Integer.MIN_VALUE / 10`; outside that range the next step overflows, so return 0. Java's `%` keeps the sign of the dividend, so negatives work without special-casing.
3. **Max points on a line**: fix each point as an anchor and group every later point by the **slope** to it. Represent the slope exactly as the reduced fraction `(dx/g, dy/g)` with g = gcd(|dx|, |dy|), normalised so dx ≥ 0 (and dy > 0 when dx = 0 — vertical lines). Pack the pair into a `long` key. The best group + 1 (the anchor) is the answer for that anchor.
4. **Josephus**: number the people 0…n−1. After the first removal the circle is a Josephus problem of size n − 1 whose numbering is shifted by k, so `J(1) = 0` and `J(n) = (J(n−1) + k) mod n` — an O(n) loop instead of simulating a list (O(n·k)).

```java
// code/ch19/patterns/MathGeometry.java (key parts)
static List<Integer> spiral(int[][] m) {
    List<Integer> out = new ArrayList<>();
    if (m.length == 0) return out;
    int top = 0, bottom = m.length - 1, left = 0, right = m[0].length - 1;
    while (top <= bottom && left <= right) {
        for (int c = left; c <= right; c++) out.add(m[top][c]);
        top++;
        for (int r = top; r <= bottom; r++) out.add(m[r][right]);
        right--;
        if (top <= bottom) { for (int c = right; c >= left; c--) out.add(m[bottom][c]); bottom--; }
        if (left <= right) { for (int r = bottom; r >= top; r--) out.add(m[r][left]); left++; }
    }
    return out;
}

static int reverse(int x) {
    int result = 0;
    while (x != 0) {
        int digit = x % 10;                    // negative for negative x
        x /= 10;
        if (result > Integer.MAX_VALUE / 10 || result < Integer.MIN_VALUE / 10) return 0;
        result = result * 10 + digit;
    }
    return result;
}

static int maxPointsOnLine(int[][] pts) {
    int best = Math.min(pts.length, 1);
    for (int i = 0; i < pts.length; i++) {
        Map<Long, Integer> slopes = new HashMap<>();
        for (int j = i + 1; j < pts.length; j++) {
            int dx = pts[j][0] - pts[i][0], dy = pts[j][1] - pts[i][1];
            int g = gcd(Math.abs(dx), Math.abs(dy));
            dx /= g; dy /= g;
            if (dx < 0 || (dx == 0 && dy < 0)) { dx = -dx; dy = -dy; }   // (1,2) and (-1,-2) are the same line
            long key = ((long) dx << 32) ^ (dy & 0xFFFFFFFFL);          // exact key, no double rounding
            best = Math.max(best, 1 + slopes.merge(key, 1, Integer::sum));
        }
    }
    return best;
}

static int josephus(int n, int k) {
    int survivor = 0;
    for (int size = 2; size <= n; size++) survivor = (survivor + k) % size;
    return survivor;
}
```

Why is the overflow check enough in `reverse` when `result` equals `MAX_VALUE / 10` exactly (214748364)? That can only happen on the last digit of a 10-digit input, and a 10-digit `int` starts with 1 or 2, so the digit added is at most 2 and `2147483642` still fits.

Output:

```
ok  spiral 3x3 -> [1, 2, 3, 6, 9, 8, 7, 4, 5]
ok  spiral 3x4 -> [1, 2, 3, 4, 8, 12, 11, 10, 9, 5, 6, 7]
ok  spiral one column -> [1, 2, 3]
ok  reverse 123 -> 321
...
12 checks passed
```

With 7 people and k = 3 the survivor is index 3 — person 4 when counting from 1.

**Complexity**: spiral O(rows · cols); reverse O(number of digits); max points O(n²) time, O(n) space (the gcd adds a log factor); Josephus O(n) time, O(1) space.

**Common mistakes**

- Slopes as `double dy / dx`: division by zero for vertical lines, `0.0` vs `-0.0`, and rounding that makes different slopes equal. Use reduced integer pairs.
- Checking overflow **after** `result * 10` — the value has already wrapped.
- `Math.abs(Integer.MIN_VALUE)` is still negative (§19.6) — an input like that breaks "take abs, reverse, restore sign" solutions.
- Spiral without the two `if`s: a single middle row or column comes out twice.
- Simulating Josephus by removing from an `ArrayList` — O(n²) — when the recurrence is O(n).

---

## 19.28 Talking through a solution in Java

Interviewers grade communication as much as code. A dependable rhythm:

1. **Clarify** (2 min): input sizes, value ranges (overflow?), duplicates, empty input, sorted?, what to return when there's no answer.
2. **Examples**: work one small example by hand, plus one edge case.
3. **Brute force first**, state its complexity, then name the pattern that improves it ("sorted → two pointers", "subarray sum with negatives → prefix sums + map").
4. **Code** with clear names and a signature like the platform's (`public int[] twoSum(int[] nums, int target)`). Say out loud why you pick `ArrayDeque`, `long`, or `Integer.compare`.
5. **Test** by tracing your example through the code, then the edge cases: empty, one element, all equal, negatives, `Integer.MAX_VALUE`.
6. **State time and space** complexity, including the sort and the recursion stack.

Java-specific polish: import `java.util.*`; declare interface types (`Map<String, Integer> m = new HashMap<>()`); prefer `var` only where the type is obvious (§2.6); return `new int[0]` or `List.of()` rather than `null`; and don't reach for streams in tight loops — a plain loop is clearer to trace and avoids boxing (see Q28).

---

# Part 3 — Java interview questions

## 19.29 Java interview Q&A

Short answers — say the first sentence, then add detail if asked. Most of these questions are answered in full in the Chapter 21 question bank (cited as **21·Qn**); here each keeps only the core answer plus any detail the bank doesn't repeat, and a "→" link to the teaching section. Certification-level traps: OCP Java 21 study notes (e.g. collections and generics); design questions: the LLD handbook. Claims marked ✔ are demonstrated in `code/ch19/qa/InterviewFacts.java` (output at the end of this section).

### Strings, objects and the language

**Q1. Why is `String` immutable?** 21·Q39. "Modifying" methods return new strings, so a `String` is thread-safe without locks, a stable `HashMap` key (hash cached), poolable, and secure for checked-then-used values (paths, class names). → §7.1

**Q2. String pool; how many objects does `new String("hi")` create?** 21·Q40 — "one or two". Extra: `"hi" == new String("hi")` is false; run-time concatenation makes a new object, while `"h" + "i"` with literals is folded at compile time and pooled ✔. → §7.1

**Q3. `String` vs `StringBuilder` vs `StringBuffer`?** 21·Q41: `+=` in a loop is O(n²); `StringBuffer`'s locking rarely helps because a single string is rarely built by several threads. → §7.3

**Q4. The `equals`/`hashCode` contract; overriding only `equals`?** 21·Q51. `equals` must also be reflexive, symmetric, transitive, consistent and `false` for `null`; with only `equals` overridden, equal objects keep different identity hash codes, land in different buckets, and `HashSet.contains` misses them ✔. → §6.13

**Q5. `==` vs `equals`?** `==` compares primitive values or object **references**; `equals` compares content when overridden (`String`, wrappers, collections, records do; arrays don't). `Integer a = 127, b = 127; a == b` is true only because `Integer.valueOf` caches −128…127 (see 21·Q42, Q211). → §7.6, §19.6

**Q6. Pass-by-value or by reference?** Always by value; for objects the value is a copy of the reference ✔ (21·Q7). → §4.2

**Q7. `final` vs `finally` vs `finalize`?** 21·Q11. A `return` inside `finally` overrides the try's return — never do that ✔. → §5.8, §8.4

**Q8. How do you make a class immutable?** 21·Q54 (`final` class or private constructors, `private final` fields set in the constructor, no setters, defensive copies, no `this` escape; records are only shallowly immutable). → §5.9, §6.8

**Q9. Interface vs abstract class?** 21·Q23; interfaces hold only constants, abstract classes allow any access level. → §6.5, §6.6, §6.12

**Q10. What is a record (Java 16), and its limits?** 21·Q128: generated private final fields, canonical constructor, accessors `x()`, component-based `equals`/`hashCode`/`toString`; implicitly `final`, extends `java.lang.Record`, no extra instance fields; may implement interfaces and add static members, methods and a compact constructor (no parameter list) for validation. → §6.8

**Q11. Sealed classes (Java 17)?** 21·Q130; adding a new subtype makes non-exhaustive switches fail to compile. → §6.7, §3.6

**Q12. Overloading vs overriding?** 21·Q20: overloads are chosen from the *static* argument types, overrides by dynamic dispatch on the object's class; a same-signature static method *hides* rather than overrides. → §4.3, §6.3

## Collections

**Q13. How does `HashMap` work internally?** 21·Q80 (16 buckets, load factor 0.75, spread hash `& (capacity − 1)`, lists treeified past 8 entries when ≥ 64 buckets — otherwise it resizes — doubling on resize, hash then `equals`). Average O(1); worst O(log n) for a tree bucket. → §10.5

**Q14. Mutating a key after `put`?** 21·Q84: the entry is effectively lost (found only by iterating); use immutable keys. → §10.5

**Q15. `HashMap` vs `Hashtable` vs `synchronizedMap` vs `ConcurrentHashMap`?** 21·Q85. Details: `HashMap` allows one `null` key and `null` values; `Hashtable` and `ConcurrentHashMap` allow no nulls; with `synchronizedMap` you must still lock manually while iterating or doing check-then-act; `ConcurrentHashMap` offers atomic `merge`/`compute`/`putIfAbsent`. → §14.12

**Q16. How is `ConcurrentHashMap` thread-safe?** 21·Q86. Also: reads see volatile fields; writers to different buckets don't block each other; `size()` is an estimate under concurrent change; nulls are forbidden so that `get` returning `null` unambiguously means "absent". → §14.12, §14.10

**Q17. `ArrayList` vs `LinkedList`?** 21·Q88: `ArrayList` has amortised O(1) append and O(n) middle insert; `LinkedList` costs more memory per element; for a queue or stack use `ArrayDeque`. → §10.2

**Q18. `HashMap` vs `LinkedHashMap` vs `TreeMap`; an LRU cache?** 21·Q92, Q102: O(1) / O(1) ordered / O(log n) sorted with navigation; LRU = `new LinkedHashMap<>(16, 0.75f, true)` + `removeEldestEntry` returning `size() > capacity`. → §10.4

**Q19. `Comparable` vs `Comparator`?** 21·Q58; comparators also cover classes you don't own; build them with `comparing`/`thenComparing`/`reversed`, never by subtraction. → §10.8, §19.5

**Q20. Fail-fast vs fail-safe iterators?** 21·Q93; fail-fast detection is best-effort ✔. → §10.7

**Q21. `ArrayDeque` vs `Stack`?** 21·Q97 and §19.4 (`Stack` also iterates bottom-to-top; its Javadoc recommends `Deque`). → §10.6

**Q22. `List.of` vs `Arrays.asList` vs `Collections.unmodifiableList`?** 21·Q95. Details: `List.of` (Java 9) rejects nulls; `Arrays.asList` allows `set` (writing through to the array) but `add`/`remove` throw; `unmodifiableList` shows later changes to the backing list; `List.copyOf` (Java 10) takes an unmodifiable snapshot ✔. → §10.10

## Exceptions

**Q23. Checked vs unchecked?** 21·Q60: checked = subclasses of `Exception` but not `RuntimeException`, enforced by the compiler; unchecked = `RuntimeException` and subclasses; `Error`s (`OutOfMemoryError`) are serious JVM problems you normally don't catch. → §8.2, §8.3

**Q24. How does try-with-resources work?** 21·Q63: resources close in reverse order ✔ on normal or exceptional exit; a `close()` failure is added via `getSuppressed()` instead of replacing the body's exception as a hand-written `finally` would. → §8.6

**Q25. Can a `finally` block not run?** 21·Q64: `System.exit`, JVM crash/kill, an infinite loop or deadlock in the `try`, or a daemon thread killed at shutdown; an exception or `return` doesn't skip it. → §8.4

## Generics, lambdas and streams

**Q26. Type erasure?** 21·Q73 ✔; also, two overloads differing only in type arguments clash. → §9.7, §9.8

**Q27. `? extends` vs `? super` (PECS)?** 21·Q74. Example: `Collections.copy(List<? super T> dest, List<? extends T> src)`. → §9.6

**Q28. Streams vs loops — which is faster, which is better?** For simple work on small collections a loop is usually as fast or faster (no pipeline objects, no boxing with primitives). Streams win on readability for declarative transforms (filter–map–group) and make parallelism a one-word change — though parallel streams only pay off for large, CPU-bound, easily split work (21·Q123). Loops are better for early exits with complex state, checked exceptions, and index arithmetic (most DSA code). → §12.11, §12.8

**Q29. How do you misuse `Optional`?** 21·Q111; `get()` on empty throws `NoSuchElementException` ✔ — prefer `orElse`, `orElseGet`, `orElseThrow`, `map`, `ifPresent`. → §11.7

**Q30. Functional interface; why effectively final?** 21·Q105, Q109 (e.g. `Runnable`, `Function<T,R>`). → §11.2, §11.5

## Concurrency

**Q31. `volatile` vs `synchronized`?** `volatile` gives **visibility and ordering** (writes before a volatile write are visible to a thread that later reads it) but no atomicity for `count++` (21·Q152); `synchronized` gives **mutual exclusion** *and* visibility on release/acquire (21·Q151). Use `volatile` for a flag or a safely published reference, locks or atomics (`AtomicInteger`) for read-modify-write. → §14.6, §14.7

**Q32. What is "happens-before"?** 21·Q153: edges are program order, monitor unlock → later lock, volatile write → later read, `Thread.start()` → the started thread, a thread's actions → a successful `join()`. → §15.8

**Q33. Deadlock?** 21·Q158; also avoid shared locks altogether (immutable data, message passing). → §15.9

**Q34. Virtual threads (Java 21)?** 21·Q134, Q135: use `Executors.newVirtualThreadPerTaskExecutor()`; limit access to scarce resources with a `Semaphore`, not a pool; JEP 491 (Java 24) ended `synchronized` pinning, but native frames and class initialisation still pin. → §15.4, §15.5

**Q35. `Runnable` vs `Callable`; `Future` vs `CompletableFuture`?** 21·Q164: `Runnable.run()` returns nothing and can't throw checked exceptions; `CompletableFuture` (Java 8) can be completed explicitly and composed — `thenApply`, `thenCompose`, `thenCombine`, `allOf`, `exceptionally`/`handle`. → §15.1, §15.2

**Q36. `ExecutorService` vs `new Thread()`?** 21·Q160: unbounded platform-thread creation is expensive and exhausts memory; size CPU-bound pools near the core count, prefer virtual threads for blocking I/O, and always shut executors down (`AutoCloseable` since Java 19). → §14.11

## The JVM

**Q37. JVM memory areas?** 21·Q172; metaspace replaced PermGen in Java 8; a too-deep stack throws `StackOverflowError`. → §16.3, §1.8

**Q38. Garbage collection; the default collector?** 21·Q174. GC roots include thread stacks, static fields and JNI references; the young generation is collected by copying survivors, long-lived objects are promoted; JDK 27 makes G1 the default everywhere. → §16.5, §16.7

**Q39. Can Java leak memory?** 21·Q180; mutated `HashMap` keys leak too; take a heap dump with `jcmd <pid> GC.heap_dump`. → §16.11

**Q40. How does the JVM run bytecode?** Class loaders load classes lazily with parent delegation (21·Q181); the JVM verifies, links and initialises them; bytecode starts interpreted and hot methods are JIT-compiled by C1, then C2 with inlining and escape analysis (21·Q183) — why Java "warms up". → §1.7, §16.2, §16.8

**Q41. `static`; when does a static initialiser run?** 21·Q10: once, when the class is first initialised (first instance, static method call, or use of a non-constant static field). → §5.5, §5.10

**Q42. Modern Java for interviews?** 21·Q127: `var` (10), switch expressions (14), text blocks (15), records and `instanceof` patterns (16), sealed types (17), pattern-matching `switch`, record patterns, virtual threads and sequenced collections (`getFirst`/`getLast`/`removeLast`, 21), `_` (22), gatherers (24), compact source files with instance `main` (25). → §17.2

**Proof of the ✔ claims**

```
ok  literals share one pooled object -> true
ok  new String makes a separate object -> true
ok  intern() returns the pooled copy -> true
ok  runtime concatenation creates a new object -> true
ok  compile-time constant concatenation is pooled -> true
ok  BadPoint equals says equal -> true
ok  but HashSet cannot find it (different hashCode) -> true
ok  record works as a key -> true
ok  return in finally overrides try -> 2
ok  try-with-resources order -> [open A, open B, body, close B, close A]
ok  same runtime class -> true
ok  adding during for-each throws ConcurrentModificationException -> true
ok  callee can mutate the object but not rebind the caller's variable -> x!
ok  view sees later changes, copy does not -> 2,1
ok  orElse instead of get() -> default
ok  get() on empty throws NoSuchElementException -> true
16 checks passed
```

`BadPoint` overrides `equals` but not `hashCode`; `javac -Xlint` even warns about it (`[overrides] Class BadPoint overrides equals, but neither it nor any superclass overrides hashCode method`).

---

## Chapter TL;DR

- **Test with a helper, not `assert`** — the keyword is disabled unless you run `java -ea`.
- **Toolkit**: `int[]` + `Arrays` (sort, fill, copyOfRange, toString); `c - 'a'` for letter indexes; `StringBuilder` for building; `HashMap.merge`/`getOrDefault`/`computeIfAbsent`; `ArrayDeque` for stacks **and** queues (never `Stack`); `PriorityQueue` (min-heap; `Comparator.reverseOrder()` for max); `TreeMap`/`TreeSet` for `floor`/`ceiling`; comparators from `comparingInt`/`thenComparing`, never `a - b`.
- **Traps**: `int` overflow (use `long`, `Math.*Exact`, `lo + (hi - lo) / 2`); `Integer == Integer` outside −128…127; unboxing `null`; `list.remove(int)`; `%` with negatives (`Math.floorMod`); removing inside for-each; deep recursion.
- **Patterns**: two pointers (sorted/in-place/linked lists) · sliding window (contiguous, non-negative) · hashing (seen/count/group) · prefix sums (range sums, subarray sum = k) · stack/monotonic stack (nesting, next greater) · heap (top-k, merge k, median) · binary search (sorted, first-true, on the answer) · intervals (sort by start, sweep) · BFS (fewest steps) / DFS (explore) / Dijkstra (weights) · topological sort (dependencies, cycle check) · union-find (dynamic connectivity) · DP (state, transition, base, order, answer) · backtracking (choose, explore, un-choose, copy the path) · tries (prefix queries) · linked lists (dummy node; fast/slow for middle, cycle start, k-th from end) · trees (top-down ranges, bottom-up summaries; LCA, max path, preorder + `#` serialisation) · greedy (only with an exchange argument; jump game, gas station, candies) · sort & search (random-pivot partition, quickselect, Dutch flag, merge sort on lists) · bits (`^` cancels pairs, `x & (x - 1)`, masks, `>>>`) · math & geometry (spiral boundaries, overflow check before `* 10`, gcd-reduced slopes, Josephus recurrence). Two heaps handle running medians and "best among the currently eligible".
- **Interview talk**: clarify → example → brute force → pattern → code → trace tests → complexity.

- **Language Q&A** centres on a few ideas: immutability and the string pool, the `equals`/`hashCode` contract, `HashMap`/`ConcurrentHashMap` internals, checked vs unchecked, erasure, `volatile` vs `synchronized` and happens-before, virtual threads, JVM memory and GC — each linked above to its full chapter.
- Same patterns in Python: Python Chapter 19.
- Next: Chapter 20 — Cheat sheet and glossary for one-page reminders of the whole book.

# Chapter 20 — Cheat Sheets and Glossary

This chapter compresses Chapters 1–19 into pages you can scan in a minute: one **cheat sheet** per area, a **feature-by-version table** for Java 17 → 25, everyday build and test **commands**, an **interview question index**, a **"where next"** guide, and an alphabetical **glossary** of 685 terms (§20.19) — every technical term on the sheets is defined there once, linked to the section that teaches it. References like **§10.5** mean Chapter 10, section 5. To revise, cover the right-hand column of a sheet and say the answer before you look.

**Every concrete value on the sheets is tested.** Primitive ranges, the `Integer` cache, the string pool, collection behaviours, stream results, `BigDecimal` and `java.time` edge cases, concurrency results and more are asserted by `code/ch20/CheatsheetChecks.java` (package `ch20`), compiled with `javac --release 25` and run on JDK 26 by the book's runner (`bash tools_run_java.sh java`). A failed check throws `AssertionError`. Real output:

```
OK   20.1 syntax, types, primitives          29 checks
OK   20.2 operators & precedence             16 checks
OK   20.3 control flow & patterns            12 checks
OK   20.4 methods & arrays                   16 checks
OK   20.5 OOP                                13 checks
OK   20.6 core APIs                          45 checks
OK   20.7 exceptions                         12 checks
OK   20.8 generics                            7 checks
OK   20.9 collections                        28 checks
OK   20.10 functional interfaces & Optional  12 checks
OK   20.11 streams & collectors              18 checks
OK   20.12 I/O & NIO.2                       11 checks
OK   20.13 concurrency                       13 checks
OK   20.14 JVM                                9 checks
OK   20.15 modern Java 17-25                  9 checks
all 250 checks passed on Java 26
```

The helper is three small methods — `eq(actual, expected, label)`, `yes(condition, label)` and `throwsA(ExceptionType.class, code, label)` — because Java's own `assert` keyword is switched off unless you run `java -ea` (§19.1). A few lines as a taste:

```java
Integer i1 = 127, i2 = 127, i3 = 128, i4 = 128;
eq(i1 == i2, true, "Integer cache -128..127");
eq(i3 == i4, false, "outside the cache: different objects");
throwsA(IllegalStateException.class, () -> emps.stream().collect(Collectors.toMap(Emp::dept, Emp::name)),
        "toMap duplicate key throws");
eq(LocalDate.of(2024, 1, 31).plusMonths(1), LocalDate.of(2024, 2, 29), "plusMonths clamps to month end");
```

---

## 20.1 Syntax, types and primitive ranges

**Program shapes** (§1.5, §1.10, §4.9):

```java
// Classic: file Hello.java, class name = file name
public class Hello {
    public static void main(String[] args) { System.out.println("Hi " + args.length); }
}

// Java 25 compact source file (JEP 512): no class declaration, instance main, java.lang.IO
void main() { IO.println("Hi"); }
```

| Command | Does | Since |
|---|---|---|
| `javac Hello.java` → `java Hello` | compile to `Hello.class`, then run it on the JVM | 1.0 |
| `javac -d out $(find src -name '*.java')` → `java -cp out pkg.Main` | compile a tree into `out/`, run with a class path | — |
| `java Hello.java` | compile in memory and run a single source file | 11 |
| `java Main.java` (other files next to it) | launch a multi-file source program | 22 |
| `jshell` | interactive REPL: type expressions, see results | 9 |
| `javap -c Hello` | disassemble bytecode | — |

**Basic rules** (§2.1): statements end with `;`; blocks use `{}`; Java is case-sensitive; identifiers start with a letter, `_` or `$` (a lone `_` is not a name — since 22 it means "unnamed"); comments are `//`, `/* */` and `/** Javadoc */` (Markdown `///` doc comments since 23). Naming: `ClassName`, `methodAndVariable`, `CONSTANT_NAME`, `com.example.pkg`. Contextual keywords — `var`, `record`, `sealed`, `non-sealed`, `permits`, `yield`, `when`, `module` — are keywords only in their positions.

**The eight primitives** (§2.3) — everything else is a reference type (§2.5):

| Type | Bits | Range | Default (fields) | Literal |
|---|---|---|---|---|
| `byte` | 8 | −128 … 127 | `0` | `(byte) 1` |
| `short` | 16 | −32 768 … 32 767 | `0` | `(short) 1` |
| `int` | 32 | −2 147 483 648 … 2 147 483 647 (±2.1 × $10^{9}$) | `0` | `42`, `0x2A`, `0b101010`, `052`, `1_000_000` |
| `long` | 64 | −9 223 372 036 854 775 808 … 9 223 372 036 854 775 807 (±9.2 × $10^{18}$) | `0L` | `42L` |
| `float` | 32 | ±3.4028235 × $10^{38}$, ~7 significant digits | `0.0f` | `3.14f` |
| `double` | 64 | ±1.7976931348623157 × $10^{308}$, ~15–16 digits | `0.0` | `3.14`, `1e-9` |
| `char` | 16 | 0 … 65 535 (unsigned UTF-16 code unit) | `'\u0000'` | `'A'`, `'\n'`, `'\u0041'` |
| `boolean` | — | `true`, `false` | `false` | `true` |

- `Double.MIN_VALUE` (4.9E-324) is the smallest **positive** double; the most negative is `-Double.MAX_VALUE`.
- Fields get defaults (`0`, `false`, `null`); **local variables don't** — reading one before assigning it is a compile error (§2.2).
- Money: `long` cents or `BigDecimal`, never `double` (§2.11).

**Conversions** (§2.7):

| Rule | Example | Result |
|---|---|---|
| Widening is automatic: `byte → short → int → long → float → double`, `char → int` | `long l = 42;` | ok |
| Narrowing needs a cast and keeps the low bits | `(byte) 200` | `-56` |
| Casting a floating value to an integer truncates toward zero | `(int) 3.99` | `3` |
| Arithmetic promotes `byte`/`short`/`char` to `int` | `byte b = 10; b = b + 1;` | compile error |
| Compound assignment hides a cast | `byte b = 10; b += 300;` | `54` |
| `char` is a number | `(int) 'A'`, `(char) ('a' + 1)` | `65`, `'b'` |

**Integer and floating-point facts** (§2.10, §2.11):

| Expression | Value | Why |
|---|---|---|
| `Integer.MAX_VALUE + 1` | `-2147483648` | silent wrap-around; use `Math.addExact` to get an `ArithmeticException` |
| `7 / 2`, `-7 / 2` | `3`, `-3` | integer division truncates toward zero |
| `-7 % 3` / `Math.floorMod(-7, 3)` | `-1` / `2` | `%` takes the dividend's sign |
| `7 / 2.0` | `3.5` | one `double` operand → `double` arithmetic |
| `0.1 + 0.2` | `0.30000000000000004` | binary fractions can't represent 0.1 exactly |
| `1.0 / 0`, `0.0 / 0`, `1 / 0` | `Infinity`, `NaN`, `ArithmeticException` | floats have special values; ints throw |
| `Double.NaN == Double.NaN` | `false` | test with `Double.isNaN` |

**`var`** (Java 10, §2.6): local variables with an initializer only (also in `for` loops and lambda parameters); not for fields, parameters or return types; `var x = null;` and `var a = {1, 2};` don't compile. The type is fixed at compile time — `var` is not dynamic typing.

**Text blocks** (Java 15, §2.12): `"""` + newline opens; common indentation is stripped (the closing `"""` position counts); a closing delimiter on its own line adds a trailing `\n`; `\` at line end joins lines; `\s` keeps a trailing space.

**Console I/O** (§2.13, §13.8): `System.out.println/print/printf`; input with `Scanner(System.in)` (`nextInt`, `nextLine` — beware the leftover newline), `BufferedReader`, or `IO.readln("prompt")` in Java 25. `System.console()` returns `null` without an interactive terminal (IDEs, pipes) — always check it (Java 22–24 briefly returned a JLine-based console even then; 25 reverted).

## 20.2 Operators and precedence

From highest to lowest (§2.8, §2.9). Operands are always evaluated **left to right**; associativity only decides grouping.

| Level | Operators | Associativity |
|---|---|---|
| 1 Postfix | `x++` `x--` | left |
| 2 Unary | `++x` `--x` `+` `-` `~` `!` `(type)` | right |
| 3 Multiplicative | `*` `/` `%` | left |
| 4 Additive | `+` `-` (and `String` `+`) | left |
| 5 Shift | `<<` `>>` `>>>` | left |
| 6 Relational | `<` `>` `<=` `>=` `instanceof` | left |
| 7 Equality | `==` `!=` | left |
| 8–10 Bitwise | `&` then `^` then `\|` | left |
| 11–12 Logical | `&&` then `\|\|` (short-circuit) | left |
| 13 Ternary | `? :` | right |
| 14 Assignment, lambda | `=` `+=` `-=` `*=` `/=` `%=` `&=` `\|=` `^=` `<<=` `>>=` `>>>=` `->` | right |

Memory hook: **unary** → **multiply** → **add** → **shift** → **compare** → **equal** → **bitwise** → **logical** → **ternary** → **assign.**

| Expression | Value | Lesson |
|---|---|---|
| `2 + 3 * 4` | `14` | `*` before `+` |
| `10 - 4 - 3` | `3` | left-associative |
| `1 + 2 + "3" + 4 + 5` | `"3345"` | numbers add until the first `String`, then everything concatenates |
| `true \|\| false && false` | `true` | `&&` binds tighter than `\|\|` |
| `x & 1 == 0` | compile error for `int x` | `==` binds tighter than `&` — write `(x & 1) == 0` |
| `i = 5; i++ + ++i` | `12` | `5 + 7`; never write this in real code |
| `-8 >> 1`, `-8 >>> 28` | `-4`, `15` | `>>` keeps the sign, `>>>` fills with zeros |
| `5 ^ 3`, `~5`, `1 << 10` | `6`, `-6`, `1024` | XOR, complement (−(n+1)), shift = ×$2^n$ |
| `a = b = c = 7` | all `7` | assignment is right-associative |
| `s != null && s.length() > 0` with `s = null` | `false`, no exception | `&&` stops at the first `false` |
| `o instanceof String str && str.length() == 4` | `true` for `"text"` | pattern variable in scope after `&&` (Java 16) |

`==` on objects compares **references**; use `equals` for content (§6.13, §7.6).

## 20.3 Control flow, `switch` and patterns

**Branches and loops** (§3.1, §3.8, §3.9):

| Construct | Shape | Notes |
|---|---|---|
| `if` / `else if` / `else` | `if (cond) { … } else { … }` | condition must be `boolean` — `if (1)` doesn't compile |
| Ternary | `cond ? a : b` | an expression; keep it short |
| `for` | `for (int i = 0; i < n; i++)` | index loops |
| Enhanced `for` | `for (String s : list)` | arrays and any `Iterable`; can't remove safely inside |
| `while` / `do … while` | test first / body runs at least once | |
| `break` / `continue` | leave / skip to next iteration of the innermost loop | `break outer;` / `continue outer;` with a label |

**The four `switch` shapes** (§3.2, §3.3, §3.4):

| Shape | Falls through? | Produces a value? | Must be exhaustive? |
|---|---|---|---|
| Statement, `case X:` | **yes** — until `break` | no | no (unless it uses patterns or `null`) |
| Statement, `case X ->` (14) | no | no | no (same exception) |
| Expression, `case X ->` (14) | no | yes; blocks use `yield` | **yes** |
| Expression, `case X:` + `yield` | yes | yes | yes |

```java
String type = switch (day) {
    case 1, 2, 3, 4, 5 -> "weekday";
    case 6, 7 -> "weekend";
    default -> { yield "invalid"; }
};
```

Selector types: `int` and smaller (`char`, `byte`, `short`) and their wrappers, `String`, enums — and, with patterns (21), **any reference type**. `long`, `float`, `double` and `boolean` selectors need *primitive patterns*, which are still **preview** in Java 25.

**Pattern matching** (§3.5, §3.6, §3.7):

| Feature | Syntax | Final in |
|---|---|---|
| Type pattern in `instanceof` | `if (o instanceof String s && !s.isEmpty())` | 16 |
| Type pattern in `switch` | `case Integer i ->` | 21 |
| Guard | `case Integer i when i < 0 ->` | 21 |
| `null` label | `case null ->` or `case null, default ->` | 21 |
| Record pattern (nestable) | `case Point(int x, int y) when x == y ->` | 21 |
| `var` in a record pattern | `case Point(var x, var y) ->` | 21 |
| Unnamed pattern / variable | `case Point(var x, _) ->`, `catch (Exception _)`, `for (var _ : list)` | 22 |

Rules to remember: without `case null`, a `null` selector throws `NullPointerException`; the first matching label wins, and a label that can never match (dominated, e.g. `CharSequence` before `String`) is a compile error; a pattern `switch` must be exhaustive — add `default` or cover every permitted subtype of a **sealed** type:

```
sealed interface Shape permits Circle, Square {}
record Circle(double r) implements Shape {}
record Square(double side) implements Shape {}

static double area(Shape s) {
    return switch (s) {                 // no default needed: the compiler knows all subtypes
        case Circle c -> Math.PI * c.r() * c.r();
        case Square q -> q.side() * q.side();
    };
}
```

**Loop pitfalls** (§3.10): off-by-one (`<=` vs `<`), modifying a collection inside for-each (→ `ConcurrentModificationException`), `float` loop counters, `==` on strings in conditions, and infinite `while` loops whose variable never changes.

---

## 20.4 Methods and arrays

**Methods** (§4.1–§4.5):

| Topic | Rule |
|---|---|
| Signature | name + parameter types (not the return type, not parameter names) |
| Passing | **always by value**; for objects the value is a copy of the reference — the callee can mutate the object but not rebind the caller's variable |
| Overload resolution | phase 1: exact match or widening; phase 2: boxing/unboxing; phase 3: varargs. `over(5)` with `over(long)`, `over(Integer)`, `over(int...)` picks `long` |
| Varargs | `int... nums` is an `int[]`; must be the last parameter; one per method; zero arguments allowed |
| `return` | a non-`void` method must return on every path |
| Recursion | needs a base case; each call adds a stack frame → `StackOverflowError` if too deep (Java has no tail-call elimination) |
| `static` | called on the class, has no `this` (§5.5) |

**Arrays** (§4.6, §4.7):

```
int[] a = new int[3];             // [0, 0, 0] — elements get default values
int[] b = {3, 1, 2};              // initializer
String[] names = new String[2];   // [null, null]
int[][] grid = new int[3][4];     // rectangular
int[][] jagged = {{1}, {2, 3}, {}};  // rows of different lengths
a.length                           // a field, not a method; fixed forever
```

Index `0 … length − 1`; anything else → `ArrayIndexOutOfBoundsException`. Arrays are objects: `a.equals(b)` and `==` compare identity; `a.toString()` prints something like `[I@1b6d3586`.

**`java.util.Arrays`** (§4.8):

| Method | Does | Example result |
|---|---|---|
| `toString(a)` / `deepToString(m)` | print 1D / nested | `[1, 2, 3]`, `[[1], [2, 3], []]` |
| `equals(a, b)` / `deepEquals` | compare contents | `true` for `{3,1,2}` vs `{3,1,2}` |
| `sort(a)`, `sort(a, from, to)`, `sort(T[], cmp)` | sort in place (dual-pivot quicksort for primitives, stable merge sort — TimSort — for objects) | |
| `binarySearch(a, key)` | on a **sorted** array; not found → `-(insertionPoint) - 1` | `binarySearch([1,2,3], 5)` = `-4` |
| `copyOf(a, n)`, `copyOfRange(a, from, to)` | new array, padded with defaults | `copyOf([1,2,3], 5)` = `[1, 2, 3, 0, 0]` |
| `fill(a, v)`, `setAll(a, i -> …)` | set every element | |
| `asList(…)` | fixed-size `List` **view** of the array — `add` throws, `set` writes through | |
| `stream(a)` | `IntStream` / `Stream<T>` | |
| `mismatch(a, b)`, `compare(a, b)` | first differing index / lexicographic order (9) | |

**Command-line arguments**: `args` is never `null` (empty array when none); all elements are `String`s; the program name is not included (§4.9).

---

## 20.5 OOP: classes, interfaces, records, enums and sealed types

**Access modifiers** (§5.6):

| Modifier | Same class | Same package | Subclass (other package) | Everywhere |
|---|---|---|---|---|
| `private` | ✓ | | | |
| *(none)* package-private | ✓ | ✓ | | |
| `protected` | ✓ | ✓ | ✓ (through inheritance) | |
| `public` | ✓ | ✓ | ✓ | ✓ |

Top-level classes can only be `public` or package-private; one `public` top-level class per file, named like the file.

**Constructors and initialisation** (§5.3, §5.10):

- If you write no constructor, the compiler adds a no-argument one; write any constructor and it doesn't.
- `this(...)` calls another constructor of the same class, `super(...)` the parent's; if you write neither, `super()` is inserted. Since **Java 25** (JEP 513) statements that don't use `this` — validation, computing arguments, assigning the class's own fields — may appear **before** `super(...)`/`this(...)`.
- Order: static fields and `static {}` blocks run **once**, in source order, when the class is initialised → for each `new`: parent constructor → instance field initialisers and `{}` blocks in source order → the rest of the constructor body.

**`static` and `final`** (§5.5, §5.8): `static` = one per class, shared; a static method has no `this`. `final` variable = assigned once; `final` method = can't be overridden; `final` class = can't be extended. `final` on a reference doesn't freeze the object.

**Inheritance and overriding** (§6.1–§6.4):

| **Overriding rule** | **Detail** |
|---|---|
| Same signature | otherwise it's an overload; use `@Override` to let the compiler check |
| Return type | same or a subtype (covariant) |
| Access | same or wider (`protected` → `public` ok, not the reverse) |
| Checked exceptions | same, narrower or none — never new or broader |
| Not overridable | `private`, `static` (hidden instead), `final` methods; constructors |
| Dispatch | instance methods use the object's run-time class; fields and static methods use the compile-time type |

```
class Base    { static String who() { return "Base"; }    String me() { return "Base"; }
                String call() { return me() + "/" + who(); } }
class Derived extends Base { static String who() { return "Derived"; }
                @Override String me() { return "Derived"; } }
new Derived().call()   // "Derived/Base"
```

**Choosing a type** (§6.12):

| **Need** | **Use** | **Key facts** |
|---|---|---|
| Mutable object with identity and lifecycle | **class** (`final` unless designed for extension) | full control of encapsulation |
| Shared state/code for a family of related classes | **abstract class** | fields, constructors, any access; single inheritance; can't be instantiated (§6.5) |
| A capability many unrelated types can have | **interface** | many per class; `abstract`, `default` (8), `static` (8), `private` (9) methods; fields are `public static final` constants; no instance state (§6.6) |
| Immutable data where equal values are interchangeable | **record** (16) | private final fields, canonical constructor, accessors `x()`, `equals`/`hashCode`/`toString`; implicitly `final`; extends `Record`; no extra instance fields; compact constructor for validation (§6.8) |
| A fixed set of instances known at compile time | **enum** | fields, constructors (private), methods, per-constant bodies; `values()`, `valueOf`, `ordinal()`; `EnumSet`/`EnumMap` (§6.9) |
| A closed set of variants | **sealed** interface/class (17) + records | `permits` lists subtypes; each is `final`, `sealed` or `non-sealed`; exhaustive `switch` without `default` (§6.7) |

```
record Range(int lo, int hi) {
    Range {                                       // compact constructor
        if (lo > hi) throw new IllegalArgumentException(lo + " > " + hi);
    }
}
new Point(1, 2).equals(new Point(1, 2))          // record Point(int x, int y): true;  toString() = "Point[x=1, y=2]"
```

**Nested types** (§6.10, §6.11):

| Kind | Declared | Has an outer instance? | Typical use |
|---|---|---|---|
| Static nested class | `static class Node` inside a class | no | helpers, builders, `Map.Entry`-like types — the default choice |
| Inner class | `class Itr` inside a class | yes (`Outer.this`) | iterators that read the outer object's fields |
| Local class | inside a method body | yes if in an instance method | one-off helper with a name |
| Anonymous class | `new Interface() { … }` | yes if in an instance method | one-off implementation with state or several methods |
| Lambda | `x -> …` | captures effectively final locals | single-method (functional) interfaces |

Nested records, enums and interfaces are implicitly `static`.

**`Object` methods contract** (§6.13): `equals` must be reflexive, symmetric, transitive, consistent, and `false` for `null`; equal objects **must** have equal `hashCode`s; override both or neither (`Objects.equals`, `Objects.hash` help). `toString` defaults to `ClassName@hexHash`. Immutability recipe (§5.9): `final` class, `private final` fields, no setters, defensive copies in and out (`List.copyOf`), don't leak `this` from the constructor. Design-level treatment: LLD handbook.

---

## 20.6 Core APIs: String, numbers, dates

**Strings** (§7.1, §7.2) are immutable; every "modifying" method returns a new `String`.

| Fact | Value |
|---|---|
| `"java" == "java"` (two literals) | `true` — both are the pooled object |
| `"java" == new String("java")` | `false` — `new` always makes a new object |
| `"java" == new String("java").intern()` | `true` |
| `"java" == part + "va"` (run time, `part = "ja"`) | `false`; `"ja" + "va"` (constants) is folded → `true` |
| `"Java".substring(1, 3)` | `"av"` — end index exclusive |
| `"a,b,,".split(",").length` / `split(",", -1)` | `2` / `4` — trailing empties dropped unless limit is negative |
| `"a.b".replace(".", "-")` / `replaceAll(".", "-")` | `"a-b"` / `"---"` — `replaceAll` takes a **regex** |
| `" ".isBlank()` / `" ".isEmpty()` | `true` / `false` |
| `compareTo` | negative, zero or positive — not necessarily −1/1 |

Everyday methods: `length`, `charAt`, `indexOf` (−1 if absent), `contains`, `startsWith`/`endsWith`, `substring`, `strip` (Unicode-aware; prefer to `trim`), `toLowerCase(Locale.ROOT)`, `equalsIgnoreCase`, `split`, `String.join`, `repeat` (11), `lines` (11), `formatted` (15), `chars`, `codePoints`.

**Builders** (§7.3): `StringBuilder` (mutable, not synchronised — use it for loops: `append`, `insert`, `reverse`, `deleteCharAt`, `setLength(0)`); `StringBuffer` (synchronised, legacy); `StringJoiner` / `String.join` / `Collectors.joining` for delimited lists. `s += x` in a loop is O($n^2$).

**Formatting** (§7.4): `String.format(Locale.US, "%5.2f|%-4s|%04d|%,d", 3.14159, "ab", 42, 1234567)` → `" 3.14|ab |0042|1,234,567"`. Conversions `%s %d %f %e %x %c %b %n %%`; flags `-` left, `0` pad, `,` grouping, `+` sign; `%2$s` argument index. Always pass a `Locale` when the output is parsed by machines.

**`Math`** (§7.5):

| Call | Result | Note |
|---|---|---|
| `Math.round(2.5)`, `Math.round(-2.5)` | `3`, `-2` | ties toward +∞; returns `long` for `double` |
| `Math.rint(2.5)` | `2.0` | ties to even |
| `Math.floorDiv(-7, 2)`, `floorMod(-7, 3)` | `-4`, `2` | round toward −∞ |
| `Math.abs(Integer.MIN_VALUE)` | `-2147483648` | overflow! `absExact` throws |
| `Math.clamp(15, 0, 10)` | `10` | Java 21 |
| `addExact`, `multiplyExact`, `toIntExact`, `powExact` (25) | throw `ArithmeticException` on overflow | |
| `Math.random()` | `double` in [0.0, 1.0) | prefer `RandomGenerator` |

**Wrappers and autoboxing** (§7.6):

| Fact | Detail |
|---|---|
| `Integer a = 127, b = 127; a == b` | `true` — `Integer.valueOf` caches **−128 … 127** (also `Short`, `Byte`, `Long`, `Character` ≤ 127, both `Boolean`s) |
| `Integer c = 128, d = 128; c == d` | `false` — always compare wrappers with `equals` |
| `int x = (Integer) null` | `NullPointerException` on unboxing |
| `list.remove(1)` vs `list.remove(Integer.valueOf(1))` | removes **index** 1 vs the **value** 1 |
| `Integer.parseInt("ff", 16)` | `255`; bad input → `NumberFormatException` |
| `Boolean.parseBoolean("YES")` | `false` — only `"true"` (any case) is true |

**`BigInteger`** / **`BigDecimal`** (§7.7): immutable, arbitrary precision.

| Rule | Example |
|---|---|
| Build from a `String` (or `BigDecimal.valueOf(double)`) | `new BigDecimal("0.1").add(new BigDecimal("0.2"))` = `0.3`; `new BigDecimal(0.1)` = `0.1000000000000000055511…` |
| `equals` checks value **and scale**; `compareTo` checks value only | `2.0` vs `2.00`: `equals` false, `compareTo` 0 — so `HashSet<BigDecimal>` can hold both |
| Non-terminating division needs a scale or `MathContext` | `ONE.divide(valueOf(3))` → `ArithmeticException`; `divide(valueOf(3), 4, HALF_UP)` = `0.3333` |
| Rounding modes | `2.345` → `HALF_UP` `2.35`, `HALF_EVEN` (banker's) `2.34` |
| `BigInteger` | `BigInteger.TWO.pow(100)` has 31 digits |

**`java.time`** (§7.8–§7.11) — immutable, thread-safe; months are 1–12:

| Class | Represents | Example |
|---|---|---|
| `LocalDate` / `LocalTime` / `LocalDateTime` | wall-calendar date/time, **no zone** | birthdays, opening hours |
| `ZonedDateTime` | date-time in a region ( `ZoneId.of("Europe/Paris")` ), DST-aware | meetings |
| `OffsetDateTime` | date-time with a fixed offset ( `+02:00` ) | API timestamps |
| `Instant` | a point on the UTC time-line | logs, DB timestamps |
| `Duration` | seconds + nanos (machine time) | `Duration.ofMinutes(90)` → `PT1H30M` |
| `Period` | years, months, days (calendar) | 2024-01-15 → 2025-03-20 = `P1Y2M5D` |
| `DateTimeFormatter` | format/parse, thread-safe | `ofPattern("yyyy-MM-dd HH:mm")` |

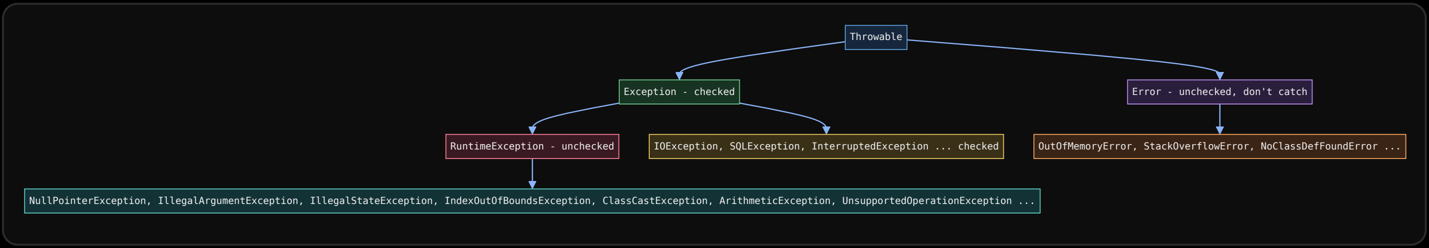
Edge cases (all checked): `LocalDate.of(2024, 1, 31).plusMonths(1)` → `2024-02-29` (clamped); in `America/New_York`, 2025-03-09 01:30 plus 1 hour → 03:30, and that day lasts 23 hours; `yyyy` vs `YYYY` for 2025-12-29 → `2025` vs `2026` (week-based year — a classic bug); `MM` month vs `mm` minute; `HH` 0–23 vs `hh` 1–12; `LocalDate.of(2025, 2, 29)` → `DateTimeException`. Legacy `Date` / `Calendar` / `SimpleDateFormat` are mutable and not thread-safe — convert at the edges.

**Random numbers** (§7.12): `new Random(42)` is reproducible (same seed → same sequence); `ThreadLocalRandom.current()` for concurrent code; `SecureRandom` for tokens and passwords; `RandomGenerator.of("L64X128MixRandom")` (17) for choosing an algorithm; `random.nextInt(lo, hi)` (17) excludes `hi`.

---

## 20.7 Exceptions

**The hierarchy** (§8.2):


Throwable
Exception - checked
Error - unchecked, don't catch
RuntimeException - unchecked
IOException, SQLException, InterruptedException ... checked
OutOfMemoryError, StackOverflowError, NoClassDefFoundError ...
NullPointerException, IllegalArgumentException, IllegalStateException, IndexOutOfBoundsException, ClassCastException, ArithmeticException, UnsupportedOperationException ...

| Kind | Which classes | Compiler rule | Meaning |
|---|---|---|---|
| **Checked** | `Exception` and subclasses **except** the `RuntimeException` branch | must be caught or declared with `throws` (§8.3) | a recoverable problem outside the program's control (file, network, DB) |
| **Unchecked** | `RuntimeException` and subclasses | no rule | a programming error (bad argument, bad state, `null` ) |
| **Error** | `Error` and subclasses (also unchecked) | no rule | the JVM or environment is in trouble — let it propagate |

| Common exception | Thrown when | Kind |
|---|---|---|
| `NullPointerException` | dereferencing `null` (helpful messages name the `null` part: Java 14, on by default since 15) | unchecked |
| `IllegalArgumentException` / `NumberFormatException` | bad argument / unparsable number | unchecked |
| `IllegalStateException` | object not in the right state (stream reused, `toMap` duplicate key) | unchecked |
| `ArrayIndexOutOfBoundsException`, `StringIndexOutOfBoundsException` | bad index | unchecked |
| `ClassCastException`, `ArithmeticException` | bad cast / integer `/ 0`, overflow in `*Exact` | unchecked |
| `UnsupportedOperationException` | mutating an unmodifiable collection | unchecked |
| `ConcurrentModificationException` | structural change during iteration | unchecked |
| `IOException`, `FileNotFoundException`, `NoSuchFileException` | I/O failure | **checked** |
| `InterruptedException` | a blocked thread was interrupted | **checked** |
| `SQLException` | JDBC failure | **checked** |
| `UncheckedIOException` | an `IOException` wrapped for lambdas/streams | unchecked |

**Syntax rules** (§8.4–§8.6):

| Construct | Rule |
|---|---|
| `try { } catch (A e) { } finally { }` | `catch` or `finally` (or both) required; `finally` runs on normal exit, exception, `return`, `break` |
| Catch order | subclasses before superclasses, else "exception has already been caught" compile error |
| Multi-catch `catch (A \| B e)` | `A` and `B` must not be subtype-related; `e` is effectively final |
| `return` in `finally` | overrides the `try`'s return **and swallows exceptions** — never do it (`try { return 1; } finally { return 2; }` returns `2`) |
| try-with-resources `try (var r = open())` | any `AutoCloseable`; closed automatically in **reverse** order; a close failure is added as **suppressed** to the body's exception (`getSuppressed()`) |
| `finally` doesn't run | only if the JVM exits (`System.exit`, crash) or the thread never leaves `try` |
| Rethrow | `catch (Exception e) { throw e; }` — precise rethrow keeps the narrower checked types |
| Chaining | `throw new ServiceException("load failed", e)` — keep the cause (§8.8) |

```java
try (Res a = new Res("A", log); Res b = new Res("B", log)) {
    throw new RuntimeException("body failed");
}   // log: open A, open B, body, close B, close A — both close() failures become suppressed
```

**Custom exceptions** (§8.7): extend `Exception` (checked) or `RuntimeException` (unchecked); provide `(String message)` and `(String message, Throwable cause)` constructors; add fields for context.

**Best practices** (§8.10): throw early (validate arguments with `Objects.requireNonNull`, `IllegalArgumentException`), catch late (where you can really handle it); never swallow (`catch (Exception e) {}`); don't use exceptions for control flow; catch the most specific type; translate low-level exceptions at layer boundaries and keep the cause; log **or** rethrow, not both; clean up with try-with-resources. Reading traces: the first line is the type and message, then frames newest-first; read `Caused by:` sections bottom-up to find the root (§8.9).

---

## 20.8 Generics and PECS

**Syntax** (§9.1–§9.5):

| Form | Example |
|---|---|
| Generic class / record | `class Box<T> { T value; }`, `record Pair<A, B>(A first, B second) {}` |
| Generic interface | `interface Repository<T, ID> { Optional<T> findById(ID id); }` |
| Generic method | `static <T> List<T> repeat(T x, int n)` — type parameter before the return type |
| Explicit type argument | `Collections.<String>emptyList()` (rarely needed — inference) |
| Diamond | `Map<String, List<Integer>> m = new HashMap<>();` |
| Bounded type | `<T extends Number>`, several bounds: `<T extends Number & Comparable<T>>` (class first) |
| Recursive bound | `<T extends Comparable<? super T>> T max(List<? extends T> xs)` |
| Wildcards | `List<?>`, `List<? extends Number>`, `List<? super Integer>` |

Naming convention: `T` type, `E` element, `K`/`V` key/value, `R` result, `ID` identifier.

**Invariance and PECS** (§9.6): `List<Integer>` is **not** a `List<Number>` (arrays are covariant — a `String[]` is an `Object[]` — and fail at run time with `ArrayStoreException`; generics refuse at compile time).

| Wildcard | You can read as | You can add | Use when the parameter… | Example |
|---|---|---|---|---|
| `List<? extends Number>` | `Number` | nothing (except `null`) | **produces** values for you | `double sumAll(List<? extends Number> xs)` |
| `List<? super Integer>` | `Object` | `Integer` | **consumes** values you give it | `void fill(List<? super Integer> sink)` accepts `List<Number>`, `List<Object>` |
| `List<?>` | `Object` | nothing | you only use `Object` methods / size | `printAll(List<?> xs)` |
| `List<T>` | `T` | `T` | you both read and write | `swap(List<T> xs, int i, int j)` |

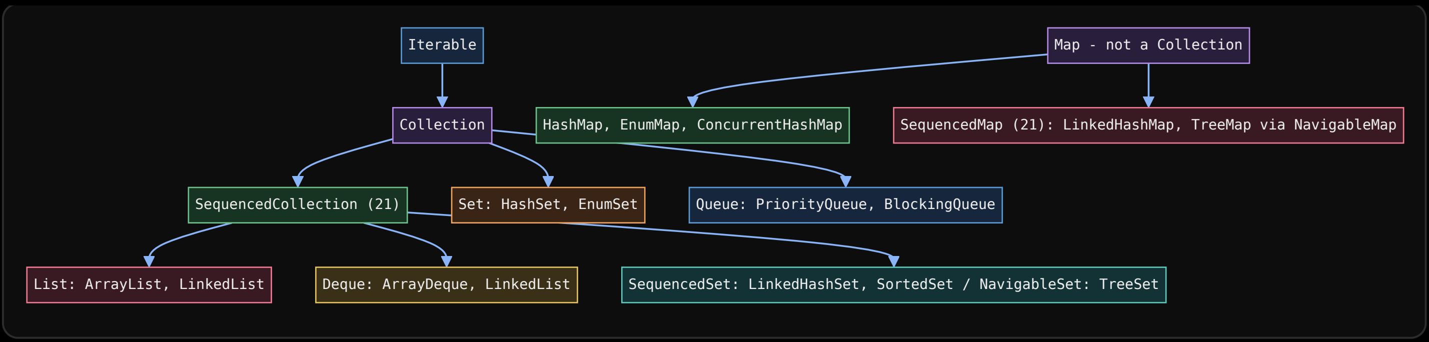
**P**roducer **E**xtends, **C**onsumer **S**uper — e.g. `Collections.copy(List<? super T> dest, List<? extends T> src)`. Don't use wildcards in return types.

**Type erasure** (§9.7, §9.8): generics are checked by the compiler, then erased to their bounds (`Object` if unbounded) with casts inserted. `new ArrayList<String>().getClass() == new ArrayList<Integer>().getClass()` is `true`. So you **can't**: use primitives (`List<int>`), `new T()`, `new T[n]`, `instanceof List<String>`, use `T` in static members, overload `f(List<String>)` with `f(List<Integer>)` (same erasure), or make a generic `Throwable`. Workarounds: `Supplier<T>` factories, `Class<T>` tokens, `List<T>` instead of `T[]`. Raw types (`List raw`) switch checking off — they exist only for pre-Java-5 code. Eliminate unchecked warnings; suppress only on the smallest scope with a comment; mark safe generic varargs methods `@SafeVarargs`.

---

## 20.9 Collections: big-O and which to choose

**The hierarchy** (§10.1):


Iterable
Map - not a Collection
Collection
HashMap, EnumMap, ConcurrentHashMap
SequencedMap (21): LinkedHashMap, TreeMap via NavigableMap
SequencedCollection (21)
Set: HashSet, EnumSet
Queue: PriorityQueue, BlockingQueue
List: ArrayList, LinkedList
Deque: ArrayDeque, LinkedList
SequencedSet: LinkedHashSet, SortedSet / NavigableSet: TreeSet

**Big-O** (§10.12) — `*` amortised, `†` average (worst case O(log n) in a tree-ified bucket):

| **Collection** | **add at end** | **add/remove at front** | `get(i)` | `contains` / `get(key)` | **remove element** | **Order** | `null` **?** |
|---|---|---|---|---|---|---|---|
| `ArrayList` | O(1)* | O(n) | O(1) | O(n) | O(n) | index | yes |
| `LinkedList` | O(1) | O(1) | O(n) | O(n) | O(n) | index | yes |
| `ArrayDeque` | O(1)* | O(1)* | — | O(n) | O(n) | head → tail | **no** |
| `PriorityQueue` | O(log n) | `poll` O(log n), `peek` O(1) | — | O(n) | O(n) | heap (iteration unsorted) | no |
| `HashSet` / `HashMap` | O(1)† | — | — | O(1)† | O(1)† | none | one `null` key |
| `LinkedHashSet` / `LinkedHashMap` | O(1)† | — | — | O(1)† | O(1)† | insertion (or access) | one `null` key |
| `TreeSet` / `TreeMap` | O(log n) | — | — | O(log n) | O(log n) | sorted | no `null` keys (natural order) |
| `EnumSet` / `EnumMap` | O(1) | — | — | O(1) | O(1) | enum order | no |
| `CopyOnWriteArrayList` | O(n) | O(n) | O(1) | O(n) | O(n) | index | yes |
| `ConcurrentHashMap` | O(1)† | — | — | O(1)† | O(1)† | none | **no** |

**Which one?** (§10.13) Key → value: `HashMap`; insertion order or LRU: `LinkedHashMap`; sorted or range queries: `TreeMap`. Unique elements: `HashSet` / `LinkedHashSet` / `TreeSet` by the same rule. Indexed or append: `ArrayList` (the default list). Stack or queue: `ArrayDeque` (not `Stack`, not `LinkedList`). "Smallest next": `PriorityQueue`. Enum keys: `EnumMap`/`EnumSet`. Fixed data: `List.of`/`Set.of`/`Map.of`. Shared between threads: `ConcurrentHashMap`, `CopyOnWriteArrayList`, `BlockingQueue`.

**`HashMap` internals** (§10.5): table of buckets, default capacity 16, load factor 0.75 (resize — double — when size > capacity × 0.75); index = `(h ^ (h >>> 16)) & (capacity − 1)`; collisions chain in a list that turns into a red-black tree past 8 entries once the table has ≥ 64 buckets; lookup = hash, then `equals`. Keys must have consistent `equals`/`hashCode` and must not change while in the map.

**Unmodifiable vs views** (§10.10):

| Factory | Changes through it? | Sees later changes to the source? | `null` s |
|---|---|---|---|
| `List.of` , `Set.of` , `Map.of` (9) | no — `UnsupportedOperationException` | — (no source) | **rejected** (NPE); `Set.of("a","a")` → `IllegalArgumentException` |
| `List.copyOf` (10), `stream.toList()` (16) | no | no — a snapshot | `copyOf` rejects; `toList()` **allows** `null` (unlike `Collectors.toUnmodifiableList()` ) |
| `Collections.unmodifiableList(src)` | no | **yes** — a read-only view | allowed |
| `Arrays.asList(array)` | `set` yes (writes through to the array); `add` / `remove` no | yes | allowed |

**API you use daily** (§10.2–§10.6, §10.9, §10.11):

| Want | Call | Result in the checks |
|---|---|---|
| Count words | `map.merge(w, 1, Integer::sum)` | `{a=1, b=3, c=1}` in a `TreeMap` |
| Group into lists | `map.computeIfAbsent(k, x -> new ArrayList<>()).add(v)` | |
| Default | `map.getOrDefault(k, -1)` | `-1` |
| Previous value | `map.put("a", 2)` | returns the old value ( `null` if none) |
| Nearest keys | `treeMap.floorKey(25)` , `ceilingKey(25)` on {10, 20, 30} | `20` , `30` ; `floorKey(5)` = `null` |
| Sub-ranges | `headMap(20)` / `tailMap(20)` | `[10]` / `[20, 30]` (head exclusive, tail inclusive) |
| Stack | `deque.push(x)` , `pop()` , `peek()` | `push` = `addFirst` |
| Queue | `deque.offer(x)` , `poll()` , `peek()` | `offer` = `addLast` ; `poll` returns `null` when empty, `remove` throws |
| Min-heap / max-heap | `new PriorityQueue<>()` / `new PriorityQueue<>(Comparator.reverseOrder())` | `poll()` of {5, 1, 4}: `1` / `5` |
| LRU cache | `new LinkedHashMap<>(16, 0.75f, true)` + override `removeEldestEntry` → `size() > cap` | |
| Ends of an ordered collection (21) | `getFirst()` , `getLast()` , `addFirst` , `removeLast` , `reversed()` | `[x, y, z]` → `x` , `z` , `[z, y, x]` |
| Sort with several keys | `list.sort(comparing(String::length).thenComparing(reverseOrder()))` | `[fig, pear, kiwi, apple]` |
| Remove while iterating | `removeIf(pred)` or `iterator.remove()` | for-each + `list.remove` → `ConcurrentModificationException` |
| Utilities | `Collections.sort` , `reverse` , `shuffle` , `frequency` , `max` , `nCopies` , `emptyList` | |

**Ordering** (§10.8): `Comparable<T>.compareTo` = the class's one natural order; `Comparator<T>` = any number of external orders (`comparing`, `thenComparing`, `reversed`, `nullsFirst`). Never compare by subtraction (`a - b` overflows) — use `Integer.compare`. `TreeSet`/`TreeMap` treat `compareTo == 0` as "equal", so keep it consistent with `equals`.

## 20.10 Functional interfaces, lambdas and `Optional`

**`java.util.function`** (§11.2, §11.3) — a *functional interface* has exactly one abstract method:

| Interface | Shape | Method | Typical use |
|---|---|---|---|
| `Supplier<T>` | `() → T` | `get` | factories, lazy defaults, `orElseGet` |
| `Consumer<T>` / `BiConsumer<T,U>` | `T → void` / `(T,U) → void` | `accept` | `forEach`, `Map.forEach` |
| `Predicate<T>` / `BiPredicate<T,U>` | `T → boolean` | `test` | `filter`, `removeIf` |
| `Function<T,R>` / `BiFunction<T,U,R>` | `T → R` / `(T,U) → R` | `apply` | `map`, `computeIfAbsent`, `merge` |
| `UnaryOperator<T>` / `BinaryOperator<T>` | `T → T` / `(T,T) → T` | `apply` | `replaceAll`, `reduce` |
| `IntPredicate`, `IntFunction<R>`, `ToIntFunction<T>`, `IntUnaryOperator`, `IntBinaryOperator`, `IntSupplier`, `ObjIntConsumer<T>` … | primitive versions (also `Long…`, `Double…`) | `test`, `apply`, `applyAsInt`, `getAsInt` … | avoid boxing in hot code |
| `Runnable` / `Callable<V>` | `() → void` / `() → V`, may throw | `run` / `call` | tasks |
| `Comparator<T>` | `(T,T) → int` | `compare` | sorting |

**Lambda syntax**: `() -> 42`, `x -> x * 2`, `(a, b) -> a + b`, `(String s) -> s.length()`, `(var x, var y) -> …`, block form `x -> { …; return r; }`. Captured local variables must be **effectively final** (§11.5); fields can change. `this` in a lambda is the enclosing object (in an anonymous class it's the anonymous object).

**Method references** (§11.4):

| Kind | Syntax | Equivalent lambda |
|---|---|---|
| Static method | `Integer::sum` | `(a, b) -> Integer.sum(a, b)` |
| Bound instance (specific object) | `System.out::println`, `"*"::repeat` | `x -> System.out.println(x)` |
| Unbound instance (first argument is the receiver) | `String::toUpperCase` | `s -> s.toUpperCase()` |
| Constructor | `ArrayList::new`, `int[]::new` | `() -> new ArrayList<>()` |

**Composition** (§11.6): `f.andThen(g)` runs `f` then `g`; `f.compose(g)` runs `g` then `f` (with `plus1`, `times2` on 3: `8` vs `7`); `Predicate.and/or/negate`, `Predicate.not(String::isBlank)` (11); `Function.identity()`; `Comparator.comparing(...).thenComparing(...)`.

**`Optional<T>`** (§11.7) — a return type for "maybe no result":

| **Do** | **Don't** |
|---|---|
| `opt.map(f).filter(p).orElse(default)` | `opt.get()` without checking (→ `NoSuchElementException`) |
| `orElseGet(() -> expensive())` for costly defaults | `orElse(expensive())` — the argument is **always** evaluated |
| `orElseThrow()` (10) when absence is a bug | `Optional` fields, parameters, collection elements |
| `ifPresentOrElse`, `or`, `stream()` (9) | returning `null` from a method that returns `Optional` |
| return an empty collection for "no results" | `Optional<List<…>>` |
| `Optional.ofNullable(x)` for maybe-null values | `Optional.of(null)` (→ `NullPointerException`) |
| `OptionalInt` / `OptionalDouble` for primitives | `Optional<Integer>` in hot numeric code |

---

## 20.11 Streams and collectors

A **stream** is a one-shot, lazy pipeline over a source: **source → zero or more intermediate operations → one terminal operation** (§12.1, §12.2). Nothing runs until the terminal operation; elements flow one at a time, and short-circuiting operations stop early:

```
Stream.of(1, 2, 3, 4).peek(x -> trace.add("saw " + x)).filter(x -> x > 1).findFirst();
// result 2, trace [saw 1, saw 2] — elements 3 and 4 are never touched
```

| **Stage** | **Operations** |
|---|---|
| Sources | `coll.stream()`, `Stream.of(…)`, `Arrays.stream(a)`, `IntStream.range(0, n)` / `rangeClosed`, `Stream.iterate(seed, hasNext, next)` (9), `Stream.generate(s)`, `Files.lines(p)`, `str.chars()`, `map.entrySet().stream()` |
| Intermediate, stateless | `filter`, `map`, `mapToInt/Long/Double`, `mapToObj`, `boxed`, `flatMap`, `mapMulti` (16), `peek` (debugging only) |
| Intermediate, stateful | `distinct`, `sorted`, `limit`, `skip`, `takeWhile` / `dropWhile` (9), `gather(…)` (24) |
| Terminal | `forEach`, `collect`, `toList()` (16), `toArray`, `reduce`, `count`, `min` / `max`, `sum` / `average` (primitive streams), `findFirst` / `findAny`, `anyMatch` / `allMatch` / `noneMatch` |

**Terminal facts** (§12.4): on an empty stream `anyMatch` → `false`, `allMatch` and `noneMatch` → `true`; `reduce(identity, op)` returns `T`, `reduce(op)` returns `Optional<T>`; `IntStream.average()` returns `OptionalDouble`; `Stream.toList()` is unmodifiable but accepts `null` elements; reusing a consumed stream → `IllegalStateException`.

**Collectors** (§12.5) — the examples use five employees (eng: Ann 120, Bob 100; ops: Cid 90, Dee 95; hr: Eve 80):

| **Collector** | **Result in the checks** | **Watch out** |
|---|---|---|
| `groupingBy(Emp::dept, TreeMap::new, counting())` | `{eng=2, hr=1, ops=2}` | default map is a `HashMap` (no order) |
| `partitioningBy(e -> e.salary() >= 100, mapping(Emp::name, toList()))` | `{false=[Cid, Dee, Eve], true=[Ann, Bob]}` | always has both keys, even for an empty stream |
| `toMap(Emp::dept, Emp::name)` | `IllegalStateException` | duplicate keys throw; `null` values → NPE |
| `toMap(Emp::dept, Emp::salary, Math::max, TreeMap::new)` | `{eng=120, hr=80, ops=95}` | merge function resolves duplicates |
| `joining(", ", "[", "]")` | `[Ann, Bob, Cid, Dee, Eve]` | elements must be `CharSequence` |
| `counting()`, `summingInt`, `averagingDouble`, `minBy`, `maxBy`, `summarizingInt` | numbers, `Optional`, `IntSummaryStatistics` | usually as downstream collectors |
| `mapping`, `filtering` (9), `flatMapping` (9), `collectingAndThen`, `teeing` (12) | adapted results | |
| `toList()`, `toSet()`, `toUnmodifiableList()` (10), `toCollection(TreeSet::new)` | collections | `Collectors.toList()` mutability is unspecified |

**Primitive streams** (§12.6): `IntStream`, `LongStream`, `DoubleStream` have `sum`, `average`, `summaryStatistics`, `range`; convert with `mapToInt` / `boxed` / `mapToObj`. `IntStream.rangeClosed(1, 5).reduce(1, (a, b) -> a * b)` = `120`.

**Gatherers** (Java 24, §12.9): `stream.gather(g)` is a custom intermediate operation. Built-ins in `Gatherers`: `windowFixed(2)` on 1..5 → `[[1, 2], [3, 4], [5]]`; `windowSliding(n)`; `scan(() -> 0, Integer::sum)` on 1, 2, 3 → `[1, 3, 6]`; `fold`; `mapConcurrent(n, f)` (runs `f` on virtual threads, keeps order).

**Rules and mistakes** (§12.7, §12.8, §12.10, §12.11): lambdas in a pipeline should be stateless and must not modify the source; don't use `peek`/`forEach` to fill an outside list — collect instead; `sorted`/`distinct` on infinite streams never finish; close streams from `Files.lines`/`list`/`walk`. Parallel streams pay off only for large, CPU-bound, easily split, stateless work (they use the common `ForkJoinPool`); never for I/O. Loops remain better for early exits with complex state, checked exceptions and index arithmetic.

---

## 20.12 I/O and NIO.2

**Which API?** (§13.1):

| **Need** | **Use** | **Since** |
|---|---|---|
| Name a file or directory | `Path.of("dir", "a.txt")` (was `Paths.get`) | 7 (`Path.of` 11) |
| Small text file in one call | `Files.readString(p)`, `Files.writeString(p, s)` | 11 |
| Lines of a small file | `Files.readAllLines(p)`, `Files.write(p, lines)` | 7 |
| Huge text file | `try (Stream<String> s = Files.lines(p))` or `Files.newBufferedReader(p)` | 8 / 7 |
| Raw bytes from anywhere | `InputStream` / `OutputStream` (+ `transferTo`, `readAllBytes`) | 1.0 (9) |
| Text from anywhere | `Reader` / `Writer` with an explicit charset | 1.1 |
| Big files, random access, memory mapping | `FileChannel` + `ByteBuffer` | 1.4 |
| Legacy code that wants `File` | `path.toFile()`, `file.toPath()` | 7 |

**`Path`** (§13.2) never touches the disk: `Path.of("/a/b/../c/./d.txt").normalize()` → `/a/c/d.txt`; `Path.of("/a/b").relativize(Path.of("/a/c/d"))` → `../c/d`; `Path.of("/a").resolve("b.txt")` → `/a/b.txt`; `getFileName`, `getParent`, `getRoot`, `toAbsolutePath`, `startsWith`.

**`Files`** (§13.3, §13.4): `exists`, `isDirectory`, `createDirectories` (no error if present), `createFile` (fails if the file exists — `FileAlreadyExistsException`), `createTempFile`/`createTempDirectory`, `copy`/`move` (add `REPLACE_EXISTING`, `ATOMIC_MOVE`), `delete` (throws if missing or a non-empty directory) / `deleteIfExists`, `size`, `mismatch` (12), `list` / `walk` / `find` (lazy `Stream<Path>` — **close them**). `Files.size` counts **bytes**: `"ünï\n"` is 6 bytes in UTF-8.

**Streams and decorators** (§13.5, §13.6): byte streams (`InputStream`/`OutputStream`) vs character streams (`Reader`/`Writer`); bridge with `InputStreamReader`/`OutputStreamWriter` and a charset; wrap in `BufferedReader`/`BufferedWriter`/`BufferedInputStream` (8 KB buffer by default) — unbuffered byte-at-a-time I/O is very slow, and a writer you forget to `flush`/`close` loses data. Always use try-with-resources.

**Charsets** (§13.7): the default charset is **UTF-8** since Java 18 (`Charset.defaultCharset()`); still pass `StandardCharsets.UTF_8` explicitly for files other systems produce. Mojibake = bytes decoded with the wrong charset.

**Binary data** (§13.9): `DataInputStream`/`DataOutputStream` for primitives; `FileChannel`, `ByteBuffer` (`flip` before reading what you wrote), memory-mapped files; `WatchService` for directory change events.

**Serialization** (§13.10): `implements Serializable`, `transient` fields are skipped, declare `serialVersionUID`. Deserialising untrusted data is a **security risk** (gadget chains) — prefer JSON or protobuf; if you must, use deserialization filters (`ObjectInputFilter`, 9; context-specific filters, 17).

**Also** (§13.8, §13.11–§13.13): `Properties` for `key=value` config (`load`, `getProperty(k, default)`); JSON needs a library (Jackson, Gson) — the JDK has none; `System.in`/`out`/`err`, `Console`, `Scanner`, `IO` (25).

---

## 20.13 Concurrency

**Threads** (§14.1–§14.4):

```
Thread t = Thread.ofPlatform().name("worker").start(task);      // or new Thread(task).start()
Thread v = Thread.ofVirtual().start(task);                       // Java 21
t.join();                                                        // wait for it to finish
```

Call `start()`, not `run()` (which runs on the *current* thread). States: `NEW` → `RUNNABLE` → (`BLOCKED` on a monitor · `WAITING` on `wait`/`join`/`park` · `TIMED_WAITING` on `sleep`/timeouts) → `TERMINATED`. Interruption is cooperative: `t.interrupt()` sets a flag; blocking methods throw `InterruptedException` — rethrow it or restore the flag with `Thread.currentThread().interrupt()`, never swallow it. There is no safe way to kill a thread: `Thread.stop()` threw `UnsupportedOperationException` from Java 20 and was removed in JDK 26. The JVM exits when only daemon threads remain.

**The primitives** (§14.5–§14.13):

| **Tool** | **Gives you** | **Use for** | **Remember** |
|---|---|---|---|
| `synchronized` (method or block) | mutual exclusion **and** visibility; reentrant | protecting shared mutable state, multi-variable invariants | instance method locks `this`, static method locks the `Class` object; released automatically |
| `volatile` field | visibility and ordering only | a stop flag, safely publishing a reference | `count++` on a volatile is **still a race** |
| `AtomicInteger`, `AtomicLong`, `AtomicReference` | lock-free single-variable updates via CAS | counters, sequence numbers | `compareAndSet(expected, new)` → `true` only if the value matched; `LongAdder` for very hot counters |
| `ReentrantLock` | `tryLock`, timeouts, interruptible, fair mode, several `Condition`s | when `synchronized` isn't flexible enough | **`unlock()` in `finally`** |
| `ReadWriteLock` / `StampedLock` | many readers or one writer / optimistic reads | read-mostly data | `StampedLock` is not reentrant |
| `wait` / `notifyAll` | guarded blocks | legacy code — prefer `java.util.concurrent` | call in a `while` loop, holding the lock |
| `ConcurrentHashMap` | thread-safe map with atomic `merge`, `compute`, `computeIfAbsent`, `putIfAbsent` | shared caches, counters | no `null` keys/values; iterators weakly consistent |
| `CopyOnWriteArrayList` | snapshot iteration, never throws CME | listeners, read-mostly lists | every write copies the array |
| `BlockingQueue` (`ArrayBlockingQueue`, `LinkedBlockingQueue`) | `put` waits when full, `take` waits when empty | producer–consumer | prefer bounded queues |
| `CountDownLatch(n)` | wait until n `countDown()` calls | start/finish gates | one-shot |
| `CyclicBarrier(n)` | n threads wait for each other, repeatedly | phased algorithms | reusable |
| `Semaphore(k)` | at most k holders at a time | limit concurrent DB/API calls (also with virtual threads) | `release` in `finally` |
| `ThreadLocal` / `ScopedValue` (25) | per-thread value / immutable per-call context | request context | remove `ThreadLocal`s on pooled threads |

**Executors** (§14.11): `try (ExecutorService pool = Executors.newFixedThreadPool(n)) { Future<T> f = pool.submit(callable); }` — `ExecutorService` is `AutoCloseable` since Java 19, and `close()` waits for submitted tasks. Factories: `newFixedThreadPool(n)` (unbounded queue!), `newCachedThreadPool()` (unbounded threads!), `newSingleThreadExecutor()`, `newScheduledThreadPool(n)`, `newWorkStealingPool()`, `newVirtualThreadPerTaskExecutor()` (21). For full control use `new ThreadPoolExecutor(core, max, keepAlive, unit, boundedQueue, factory, rejectionPolicy)`. Size CPU-bound pools ≈ number of cores.

**`CompletableFuture`** (§15.1, §15.2):

| Method | Does | Checked result |
|---|---|---|
| `supplyAsync(supplier[, executor])` / `runAsync` | start async work (default: common `ForkJoinPool`) | |
| `thenApply(fn)` | transform (like `map`) | |
| `thenCompose(fn)` | chain another async step (like `flatMap`) | `5` → `6` |
| `thenCombine(other, fn)` | combine two independent results | price 20 × qty 3 = `60` |
| `allOf(…)` / `anyOf(…)` | wait for all / the first | |
| `exceptionally(fn)` | recover on failure | `-1` |
| `handle((v, ex) -> …)` / `whenComplete` | always runs; can change result / can't | |
| `orTimeout(d, unit)` / `completeOnTimeout(v, d, unit)` (9) | time limits | |
| `join()` / `get()` | block for the result | `join` wraps failures in `CompletionException` (unchecked); `get` in `ExecutionException` (checked) |

**Virtual threads** (Java 21, §15.4, §15.5): cheap threads scheduled by the JVM onto a few carrier (platform) threads; blocking I/O unmounts them. Use one per task (`newVirtualThreadPerTaskExecutor()`) for **I/O-bound**, high-concurrency work; they don't speed up CPU-bound work. Don't pool them; cap scarce resources with a `Semaphore`; avoid big `ThreadLocal` caches. They are always daemon threads. Since **Java 24** (JEP 491) blocking inside `synchronized` no longer pins the carrier; native calls that call back into blocking Java code, and class initialisation, still do.

**Also modern** (§15.3, §15.6, §15.7): Fork/Join (`RecursiveTask`, work stealing, used by parallel streams); **scoped values** are final in Java 25 (`ScopedValue.where(USER, "alice").call(() -> USER.get())` → `"alice"`, and unbound again afterwards); **structured concurrency** (`StructuredTaskScope.open()` plus a `Joiner` policy) is still **preview** in 25 (JEP 505) and kept changing in 26 (JEP 525) and 27 (JEP 533: built-in joiners make `join()` throw `ExecutionException`, `awaitAll()` removed) — don't write production code against it yet.

**Java Memory Model** (§15.8): without a *happens-before* edge another thread may never see your write. Edges: program order in one thread; unlock → later lock of the same monitor; volatile write → later read of that field; `Thread.start()` → actions in the thread; actions in a thread → a successful `join()`; `java.util.concurrent` hand-offs (queue put → take, `countDown` → `await`, task submit → run, task → `Future.get`); transitivity. Safe publication: `final` fields, volatile, locks, concurrent collections, static initialisers.

**Liveness** (§15.9): **deadlock** (threads wait for each other's locks forever — fix with one global lock order, `tryLock` with timeout, fewer locks; diagnose with `jstack`/`jcmd Thread.print`), **livelock** (threads keep reacting to each other without progress — add randomised back-off), **starvation** (a thread never gets CPU or a lock — fair locks, bounded work).

**Choosing a model** (§15.10): blocking I/O at scale → virtual threads; fan-out/fan-in with composition → `CompletableFuture` (structured concurrency once final); CPU-heavy data work → parallel streams / Fork/Join / fixed pool of ≈ cores; periodic jobs → `ScheduledExecutorService`; shared counters/caches → atomics, `LongAdder`, `ConcurrentHashMap`. For design-level concurrency case studies see the LLD handbook.

---

## 20.14 The JVM: memory, GC, flags and tools

**How code runs** (§1.6, §1.7, §16.1, §16.2, §16.8): `javac` → bytecode in `.class` files → class loaders load classes lazily (bootstrap → platform → application, **parent-first delegation**) → verify, prepare, resolve, initialise (static initialisers run once) → the interpreter runs bytecode; hot methods are compiled by the tiered JIT (**C1** fast, then **C2** heavily optimising: inlining, devirtualisation, escape analysis, loop optimisations, deoptimisation when assumptions break). Hence "warm-up". Class loaders in the checks:

`String.class.getClassLoader()` is `null` (bootstrap), `java.sql` classes use the `platform` loader, your classes the `app` loader.

**Memory areas** (§16.3, §1.8):

| Area | Per thread? | Holds | Size flag | When it runs out |
|---|---|---|---|---|
| Heap (young: eden + survivors; old) | shared | all objects and arrays | `-Xms`, `-Xmx`, `-XX:MaxRAMPercentage` | `OutOfMemoryError: Java heap space` |
| Java stack | per thread | frames: locals, operand stack | `-Xss` | `StackOverflowError` |
| Metaspace (native memory; replaced PermGen in 8) | shared | class metadata | `-XX:MaxMetaspaceSize` (unlimited by default) | `OutOfMemoryError: Metaspace` |
| Code cache | shared | JIT-compiled code | `-XX:ReservedCodeCacheSize` | JIT stops compiling (slow, not a crash) |
| Direct / native memory | shared | direct `ByteBuffer`s, thread stacks, GC structures | `-XX:MaxDirectMemorySize` | `OutOfMemoryError: Direct buffer memory` / `unable to create native thread` |

Object size basics (§16.4): 12-byte header (8 with `-XX:+UseCompactObjectHeaders`, Java 25; the default from JDK 27), 4-byte compressed references, padded to 8 bytes — `new Object()` is 16 bytes; `record Point(int x, int y)` 24.

**Garbage collectors** (§16.5–§16.7) — an object is garbage when it isn't reachable from GC roots (thread stacks, static fields, JNI handles):

| Collector | Flag | Pauses | Choose it for |
|---|---|---|---|
| **G1** (default since 9 on machines with ≥ 2 CPUs and ≥ 1792 MB; in all environments from JDK 27, JEP 523) | `-XX:+UseG1GC` | goal `-XX:MaxGCPauseMillis` (200 ms default); typically a few-tens of ms | most services — the default for a reason |
| Parallel | `-XX:+UseParallelGC` | longer, grows with live data | batch jobs: maximum throughput |
| Serial | `-XX:+UseSerialGC` | stop-the-world, single thread | tiny heaps, 1 CPU, CLI tools (before JDK 27 the JVM picked it on machines with < 2 CPUs or < 1792 MB) |
| **ZGC** (generational; generational-only since 24) | `-XX:+UseZGC` | typically < 1 ms, independent of heap size | latency-sensitive services, huge heaps |
| Shenandoah (generational mode product in 25) | `-XX:+UseShenandoahGC` | sub-ms to a few ms | low latency; in most OpenJDK builds, not Oracle's |
| Epsilon | `-XX:+UnlockExperimentalVMOptions -XX:+UseEpsilonGC` | none — never collects | benchmarks, ultra-short jobs |

**Key flags** (§16.7):

| Flag | Meaning |
|---|---|
| `-Xms512m -Xmx2g` | initial / maximum heap (set them equal for servers to avoid resizing) |
| `-XX:MaxRAMPercentage=75` | heap as % of container memory (default 25) |
| `-Xss1m` | thread stack size (default platform-dependent: about 1 MB on 64-bit Linux, 2 MB on macOS/AArch64) |
| `-Xlog:gc*` (or `-Xlog:gc:file=gc.log`) | unified GC logging (9+) |
| `-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/dumps` | heap dump on `OutOfMemoryError` |
| `-XX:+PrintFlagsFinal` | print every flag's final value |
| `-XX:StartFlightRecording=duration=60s,filename=app.jfr` | record with JDK Flight Recorder at start-up |
| `-XX:TieredStopAtLevel=1` | C1 only: faster start-up, lower peak speed |
| `-ea` | enable `assert` statements |
| `--enable-preview` | allow preview features (needs `--release N` at compile time) |
| `-XX:+UseCompactObjectHeaders` | 8-byte object headers (25, opt-in; on by default from JDK 27, JEP 534) |

**Diagnostic tools** (§16.10, §16.11):

| Tool | Use |
|---|---|
| `jps -l` | list running JVMs and PIDs |
| `jcmd <pid> help` | the Swiss-army knife: `VM.flags`, `GC.heap_info`, `GC.class_histogram`, `Thread.print`, `GC.heap_dump file.hprof`, `JFR.start` / `JFR.dump`, `VM.native_memory` |
| `jstat -gcutil <pid> 1000` | heap occupancy and GC counts every second |
| `jstack <pid>` | thread dump — look for `BLOCKED` and "Found one Java-level deadlock" |
| JFR + `jfr view hot-methods app.jfr` / JDK Mission Control | low-overhead production profiling |
| VisualVM, Eclipse MAT, async-profiler | GUI monitoring, heap-dump analysis (dominator tree), flame graphs |
| JMH | micro-benchmarks that avoid warm-up and dead-code traps (§16.9) |
| `jdeps`, `jlink` | module dependencies / build a trimmed runtime (§16.12) |

Symptom → first tool: high CPU → JFR hot methods; latency spikes → GC log pauses; hangs → two or three thread dumps; memory growth → `jstat` old-gen trend + class histogram twice, then a heap dump; RSS far above heap → native memory tracking. Classic leaks: unbounded static caches, unregistered listeners, `ThreadLocal`s on pooled threads, mutated `HashMap` keys, class-loader leaks.

**Modules (JPMS)** (§16.12): `module-info.java` with `requires`, `exports`, `opens` (for reflection), `uses`/`provides` (services); the class path is the *unnamed module*; `import module java.base;` (25) imports every exported package of a module.

## 20.15 Modern Java by version: 17 to 25

Final features only — the version that **finalised** each one (§17.2; JEP numbers in parentheses). The book's baseline before 17: `var` (10), `HttpClient` and `java Hello.java` (11), switch expressions and helpful NPE messages (14; the messages on by default since 15), text blocks (15), records, `instanceof` patterns and `Stream.toList()` (16).

| Release | Final features (JEP numbers and the JVM/tool changes: §17.2) |
|---|---|
| **17 LTS** (Sep 2021) | sealed classes, enhanced pseudo-random generators, always-strict floating point, context-specific deserialization filters; strong encapsulation of JDK internals |
| **18** (Mar 2022) | UTF-8 by default, `@snippet` in Javadoc, `jwebserver`; finalization deprecated for removal |
| **19, 20** | previews and incubators only (Linux/RISC-V port in 19) |
| **21 LTS** (Sep 2023) | record patterns, pattern matching for `switch`, **virtual threads**, sequenced collections, KEM API; generational ZGC |
| **22** (Mar 2024) | unnamed variables and patterns `_`, Foreign Function & Memory API, multi-file source launch |
| **23** (Sep 2024) | Markdown doc comments `///`; ZGC generational by default; `Unsafe` memory access deprecated for removal |
| **24** (Mar 2025) | **stream gatherers**, Class-File API, ML-KEM/ML-DSA; no `synchronized` pinning, AOT class loading, non-generational ZGC removed, Security Manager disabled |
| **25 LTS** (Sep 2025) | compact source files and instance `main`, module imports, flexible constructor bodies, **scoped values**, KDF API; compact object headers (opt-in), generational Shenandoah, AOT ergonomics and method profiling |

**Still not final in Java 25** (§17.11): structured concurrency (fifth preview, 505; sixth in 26, seventh in 27), primitive types in patterns (third preview, 507), stable values (preview, 502; renamed *lazy constants* in 26), PEM encodings (preview, 470), Vector API (tenth incubator, 508), JFR CPU-time profiling (experimental, 509). **Withdrawn**: string templates (previewed in 21 and 22, removed in 23). Try a preview with `javac --release 25 --enable-preview` and `java --enable-preview`. After 25 (§17.11): JDK 26 (Mar 2026) finalised HTTP/3 in `HttpClient` (517) and removed the Applet API (504) and `Thread.stop`; JDK 27 (Sep 2026) made G1 the default everywhere (523) and compact object headers the default (534). LTS releases: 8, 11, 17, 21, 25 (every two years); the next is expected to be 29 in September 2027 (§1.12).

Upgrading 17 → 25 (§17.12): raise the JDK first and `--release` second; upgrade build plugins and bytecode libraries (Lombok, Mockito, Byte Buddy, Spring); run `jdeps --jdk-internals` and `jdeprscan`; read the start-up warnings (agents, JNI, `Unsafe`); drop removed flags such as `-XX:-ZGenerational`.

---

## 20.16 Testing, build and tooling commands

**JDK tools** (§1.4–§1.11, §18.12, §18.13):

| Command | Does |
|---|---|
| `java -version`, `javac -version` | check the installed JDK |
| `javac --release 25 -d out $(find src -name '*.java')` | compile for a target Java version |
| `java -cp out:lib/* com.example.Main` | run with a class path (`;` on Windows) |
| `jar --create --file app.jar --main-class com.example.Main -C out .` | build an executable JAR; run with `java -jar app.jar` |
| `javadoc -d docs …`, `javap -c`, `jshell` | docs, bytecode, REPL |
| `jlink`, `jpackage` | custom runtime image / native installer |
| `bash tools_run_java.sh java` | this book's runner: compiles each chapter's `code/` with `--release 25` and runs every `main` |

**Maven** (§18.2; this book: Maven 3.9.14, Gradle 9.4.1) — `pom.xml`; coordinates `groupId:artifactId:version`; scopes `compile`, `test`, `provided`, `runtime`; "nearest wins" for version conflicts (pin with `dependencyManagement`); pin every plugin version too. Lifecycle: `validate → compile → test-compile → test → package → verify → install → deploy` — running a phase runs every phase before it.

| Command | Does |
|---|---|
| `mvn clean verify` | full build with all checks (what CI runs) |
| `mvn test`, `mvn -Dtest=CartTest test` | run all / one test class (`CartTest#method` for one method) |
| `mvn package -DskipTests` | build the JAR without running tests |
| `mvn dependency:tree` | print the resolved dependency tree |
| `./mvnw …` | the Maven Wrapper: pinned Maven version, committed to the repo |

**Gradle** (§18.3) — `build.gradle(.kts)`; configurations `implementation`, `testImplementation`, `runtimeOnly`; incremental and cached tasks.

| Command | Does |
|---|---|
| `./gradlew build` | compile, test, assemble |
| `./gradlew test --tests 'ch18.shop.CartTest'` | run matching tests |
| `./gradlew run` | run the `application` plugin's main class |
| `./gradlew dependencies --configuration testRuntimeClasspath`, `./gradlew tasks` | inspect |

**JUnit Jupiter** (§18.4, §18.5, §18.8) — the same API in JUnit 5 and JUnit 6 (6.0, 2025: Java 17 baseline, one version number for Platform/Jupiter/Vintage, JSpecify annotations, Vintage deprecated); this book ran JUnit 6.1.3:

| Annotation | Meaning |
|---|---|
| `@Test` | a test method (package-private is fine) |
| `@BeforeEach` / `@AfterEach` | run around every test (a new test-class instance per test by default) |
| `@BeforeAll` / `@AfterAll` | once per class (`static`) |
| `@DisplayName`, `@Disabled`, `@Tag`, `@Timeout`, `@Nested` | naming, skipping, filtering, time limit, grouping |
| `@ParameterizedTest` + `@ValueSource`, `@CsvSource`, `@MethodSource`, `@EnumSource`, `@NullAndEmptySource` | one test, many inputs |
| `@TempDir Path dir` | fresh temporary directory |
| `@ExtendWith(MockitoExtension.class)` | plug in an extension |

Assertions: `assertEquals(expected, actual)` (expected first), `assertTrue`, `assertNull`, `assertThrows(X.class, () -> …)` (returns the exception), `assertAll(…)` (report every failure), `assertIterableEquals`, `assertTimeout`. Console launcher: `java -jar junit-platform-console-standalone-<version>.jar execute --class-path out:libs --scan-class-path`.

**Mockito** (§18.6): `mock(Repo.class)` or `@Mock`; stub with `when(repo.find("a")).thenReturn(Optional.of(x))` / `thenThrow`; `verify(repo).save(any())`, `verify(repo, never()).delete(…)`, `times(2)`; `ArgumentCaptor`; `@InjectMocks`; stub only what the test needs (strict stubs flag the rest). Fakes beat mocks for simple collaborators. Mockito 5 (this book: 5.24.0) mocks `final` classes through an agent — on JDK 21+ add `-javaagent:mockito-core.jar` to the test JVM (Surefire `argLine`, Gradle `jvmArgs`) instead of letting it self-attach.

**AssertJ** (§18.7): `assertThat(list).hasSize(3).containsExactly("a", "b", "c")`; `assertThat(s).startsWith("J").contains("av")`; `assertThatThrownBy(() -> …).isInstanceOf(IllegalArgumentException.class).hasMessageContaining("negative")`; `extracting(Order::id)`; `usingRecursiveComparison()`.

**Test design** (§18.9): Arrange–Act–Assert; one behaviour per test; name tests after behaviour; FIRST (fast, independent, repeatable, self-validating, timely); inject clocks and random seeds; test pyramid (many unit, some integration — Testcontainers — few end-to-end). Testcontainers starts throwaway real services (such as a database) in Docker containers for tests.

**Logging** (§18.10): code against SLF4J (`private static final Logger log = LoggerFactory.getLogger(Foo.class); log.info("order {} placed", id);` — placeholders, not concatenation), with Logback or Log4j 2 as the backend; levels `TRACE < DEBUG < INFO < WARN < ERROR`; `java.util.logging` is built in (`FINE`, `INFO`, `WARNING`, `SEVERE`). Pass the exception as the last argument to keep the stack trace.

**JDBC** (§18.11): `try (Connection c = dataSource.getConnection(); PreparedStatement ps = c.prepareStatement("SELECT … WHERE id = ?")) { ps.setLong(1, id); try (ResultSet rs = ps.executeQuery()) { while (rs.next()) … } }` — always `PreparedStatement` (prevents SQL injection), a connection pool (HikariCP) in real apps, `setAutoCommit(false)` + `commit`/`rollback` for transactions. SQL itself: the SQL handbook.

---

## 20.17 Interview question index

The coding side — the Java DSA toolkit (§19.1–§19.7) and twenty patterns from two pointers to math and geometry (§19.8–§19.27) — is in the same chapter. Pattern finder:

| Problem says… | Pattern | Section |
|---|---|---|
| sorted input, pair with target sum, in place | two pointers | §19.8 |
| longest/shortest contiguous substring or subarray | sliding window | §19.9 |
| seen before, count, group, pair in unsorted data | hashing | §19.10 |
| range sums, subarrays summing to k (negatives allowed) | prefix sums | §19.11 |
| brackets, next greater element | stack / monotonic stack | §19.12 |
| k largest, merge k lists, running median, best among currently eligible | heap, two heaps | §19.13 |
| sorted, first true, minimum capacity such that | binary search (on the answer) | §19.14 |
| meetings, overlaps, rooms | intervals | §19.15 |
| fewest steps, islands, weighted route | BFS / DFS / Dijkstra | §19.16 |
| prerequisites, order with dependencies | topological sort | §19.17 |
| connected groups as edges arrive | union-find | §19.18 |
| number of ways, min cost, longest subsequence | dynamic programming | §19.19 |
| all subsets, permutations, N-Queens | backtracking | §19.20 |
| many prefix queries, autocomplete | trie | §19.21 |
| k-th from end, middle, cycle start, intersection, palindrome list | linked lists, fast/slow pointers | §19.22 |
| validate BST, lowest common ancestor, path sum, serialise | tree recursion (top-down / bottom-up) | §19.23 |
| jump game, gas station, candies, local best is safe | greedy | §19.24 |
| k-th element without full sort, sort colours, sort a list | quickselect, Dutch flag, merge sort | §19.25 |
| appears twice except one, count 1-bits, subsets of ≤ 20 | bit manipulation | §19.26 |
| spiral matrix, reverse digits, points on a line, Josephus | math and geometry | §19.27 |

**Language questions.** The full bank is Chapter 21 (Q1–Q280 by topic: basics, OOP, strings, `equals`/`hashCode`, exceptions, generics, collections, lambdas, streams, modern Java, concurrency, JVM, I/O, testing, scenarios, output puzzles, coding tasks, DSA and multithreading problems). §19.29 condenses the 42 most-asked into short answers keyed to that bank: Q1–12 strings, objects and the language · Q13–22 collections · Q23–25 exceptions · Q26–30 generics, lambdas and streams · Q31–36 concurrency · Q37–42 the JVM.

More interview material: each chapter's **In the interview** sections, the modern-Java round in §17.13, and exam-level traps in the OCP Java 21 study notes.

## 20.18 Where next

| Direction | First step |
|---|---|
| **Spring Boot** (4.0, Nov 2025, on Spring Framework 7; Java 17+: DI, auto-configuration, starters, Actuator — §18.14) | build a small REST API at start.spring.io with JDBC or JPA, tested with JUnit, Mockito and Testcontainers |
| **Jakarta EE** (formerly Java EE: Servlet, CDI, JPA, JAX-RS, Bean Validation on WildFly, Open Liberty, Payara) | learn JPA and CDI — Spring uses many of the same specifications |
| **Quarkus, Micronaut, Helidon** (fast start-up, small memory, GraalVM native images) | port your Spring REST API to compare |
| **Kotlin** (JetBrains JVM language, Java-interoperable; null-safe types, data classes, coroutines; Android's preferred language since 2019); also Scala, Groovy, Clojure if a job needs them | rewrite one small program; call it from Java |
| **Android** (Kotlin-first, Java supported) | Android Developers' basic course |
| **Data and messaging** (JDBC/JPA, Kafka clients, Spark, Flink) | the SQL handbook and the DDIA notes |
| **Design** (SOLID, patterns, LLD case studies in Java) | the LLD handbook |
| **Certification** (Oracle Certified Professional: Java SE 21 Developer, 1Z0-830) | the OCP Java 21 study notes |
| **Performance** (JFR, JMH, GC tuning, async-profiler) | revisit Chapter 16 with a real service |

Keep up to date by reading the JEP list for each release on openjdk.org and the dev.java articles; a new feature release ships every March and September.

---

## 20.19 Glossary (A–Z)

Each term has a short definition and a link to the section that teaches it. Backticks and symbols are ignored when sorting; *italic* names point to another entry. 685 terms.

**A**

- **Abstract class** — A class declared `abstract` that can't be instantiated; it may hold state, constructors and abstract methods for subclasses to implement. §6.5
- **Abstract method** — A method with no body that a concrete subclass (or implementing class) must implement. §6.5
- **Access modifier** — `private`, package-private (none), `protected` or `public` — controls where a member can be used. §5.6
- **Accessor (record)** — The generated method named like a record component, e.g. `x()` for `record Point(int x, int y)`. §6.8
- **Actuator** — The Spring Boot module that exposes health, metrics and info endpoints for operations. §18.14
- **Adjacency list** — A graph stored as, for each node, the list of nodes it has an edge to. §19.16
- **Algebraic data type** — A type defined as "exactly one of these variants, each with these fields" — in Java, a sealed interface plus records. §3.7, §6.7
- **Algorithm** — A precise, step-by-step procedure that solves a problem; its cost is described with *Big-O notation*. Ch 19
- **Allocation rate** — How fast a program creates new objects (MB per second); a high rate means more frequent young collections. §16.10
- **Amortised O(1)** — Constant cost on average over many calls, even if an occasional call (such as an `ArrayList` resize) is O(n). §10.12
- **Anagram** — A word made by rearranging another word's letters (*tea*, *eat*); grouped by sorting each word's letters. §19.10

- **Annotation** — Metadata written `@Name` on code (`@Override`, `@FunctionalInterface`, `@Test`) that the compiler, tools or frameworks read. §6.3, §18.4
- **Anonymous class** — A class declared and instantiated in one expression, `new Interface() { … }`, with no name. §6.11
- **AOT cache** — Ahead-of-time class loading and linking data (Java 24–25) that the JVM reuses to start faster. §17.9, §17.10
- **API** — Application Programming Interface: the classes and methods a library, the JDK or a service offers for other code to call. §1.1
- **`ArithmeticException`** — Unchecked exception for integer division by zero or overflow detected by `Math.*Exact` methods. §2.10, §7.5
- **Arrange-Act-Assert** — The three-part shape of a clear test: set up the inputs, run the code under test, then check the result. §18.9
- **Array** — A fixed-length, indexed, zero-based container of one element type; an object with a `length` field. §4.6
- **`ArrayDeque`** — A resizable circular-array `Deque`; the recommended stack and queue; rejects `null`. §10.6
- **`ArrayIndexOutOfBoundsException`** — Thrown when an array index is negative or ≥ `length`. §4.6
- **`ArrayList`** — A growable-array `List` with O(1) indexed access and amortised O(1) append; the default list. §10.2
- **`Arrays.asList`** — Returns a fixed-size `List` view backed by an array: `set` writes through, `add`/`remove` throw. §4.8, §10.10
- **Arrow label** — `case X ->` in a `switch` (Java 14): runs only that case, with no fall-through. §3.3
- **ASCII** — The original 7-bit character set of 128 English letters, digits and symbols; the first 128 Unicode characters are the same. §7.1, §13.7
- **`assert`** — A statement that checks a condition only when the JVM runs with `-ea`; disabled by default. §19.1
- **AssertJ** — A fluent assertion library: `assertThat(x).isEqualTo(y)`. §18.7
- **Associativity** — The grouping order for operators of equal precedence — left for most, right for assignment and the ternary. §2.9
- **Asynchronous** — Started now and finished later without the caller waiting; the result arrives through a `Future`, a callback or a `CompletableFuture`. §15.1, §15.2
- **Atomic variable** — A class such as `AtomicInteger` whose single-variable updates are atomic without locks, using CAS. §14.10
- **Atomicity** — An operation is atomic when other threads can see it only as not started or fully done, never half-done. §14.5
- **Autoboxing / boxing / unboxing** — Automatic conversion from a primitive to its *wrapper class* (`int` → `Integer`, boxing) and back (unboxing); unboxing `null` throws `NullPointerException`. §7.6
- **`AutoCloseable`** — The interface with `close()` that try-with-resources calls automatically. §8.6

### B

- **Backing array** — The array behind an `ArrayList` or an `Arrays.asList` view. §10.2, §10.10
- **Backtracking** — Searching every candidate solution by making a choice, exploring further, then undoing the choice to try the next one. §19.20
- **Banker's rounding** — `RoundingMode.HALF_EVEN`: a tie rounds to the even neighbour (2.345 → 2.34), which avoids drift in large sums. §7.7
- **Behaviour parameterisation** — Passing behaviour (today, a lambda) as an argument so one method can serve many cases. §11.1
- **Bellman-Ford** — A shortest-path algorithm that, unlike Dijkstra's, allows negative edge weights; slower at O(V · E). §19.16
- **Benchmark** — A measured timing of code; a fair Java microbenchmark needs warm-up and a harness such as *JMH*. §16.9
- **BFS (breadth-first search)** — Visiting a graph level by level from a start node using a queue; it finds shortest paths when every edge costs the same. §19.16
- **`BigDecimal`** — Immutable arbitrary-precision decimal number with an explicit scale; used for money. §7.7

- **`BigInteger`** — Immutable arbitrary-size integer. §7.7
- **Big-O notation** — How an operation's cost grows with input size n: O(1), O(log n), O(n), O(n log n), $O(n^2)$. §10.12, §19.7
- **Binary search** — Finding a key in sorted data by halving the range each step, O(log n). §4.8, §19.14
- **Binary search tree (BST)** — A binary tree where every key in a node's left subtree is smaller, and every key in its right subtree larger, than the node's key. §19.23
- **Binary tree** — A tree in which every node has at most two children, called left and right. §19.23
- **Bit manipulation** — Solving problems with bitwise operators on the binary form of integers — masks, shifts and xor tricks. §19.26
- **Bitmask** — An integer whose individual bits act as on/off flags, for example one bit per item of a subset. §19.26
- **Bitwise operators** — `&`, `\|`, `^`, `~`, `<<`, `>>`, `>>>` — operate on the individual bits of integers. §2.8
- **`BLOCKED`** — The thread state of a thread waiting to acquire a monitor held by another thread. §14.3
- **`BlockingQueue`** — A queue whose `put` waits when full and `take` waits when empty; the core of producer–consumer. §14.12
- **BOM** — (1) Byte order mark: optional bytes at the start of a text file that announce its encoding. (2) In Maven, a "bill of materials" POM that pins versions for a family of libraries. §13.7, §18.2
- **Bootstrap class loader** — The JVM's built-in loader for core modules such as `java.base`; shown as `null` by `getClassLoader()`. §16.2
- **Bounded type parameter** — A type parameter with a limit, e.g. `<T extends Number>`. §9.5
- **`break`** — Leaves the innermost loop or `switch` (or a labelled statement with `break label`). §3.9
- **Bridge method** — A hidden method `javac` generates so that overriding still works after generic types are erased. §9.7
- **Brute force** — Solving a problem by trying every possibility; usually too slow, but a good first answer to improve on. §19.28
- **Bucket** — One slot of a hash table's array, holding the entries whose hashes map to that index. §10.5
- **Buffer / buffering** — A block of memory that collects data so it is read or written in large chunks, avoiding many slow system calls. §13.6, §13.9
- **Build tool** — Software (Maven, Gradle) that resolves dependencies, compiles, tests and packages a project. §18.2, §18.3
- **Byte Buddy** — A library for generating and changing classes at run time; Mockito uses it to build mocks. §17.12
- **Byte stream** — `InputStream`/`OutputStream`: I/O in raw 8-bit bytes. §13.5
- **`ByteBuffer`** — A NIO container of bytes with position, limit and capacity, used with channels. §13.9
- **Bytecode** — The platform-independent instruction set in `.class` files that the JVM executes. §1.6

## C

- **C1 / C2** — HotSpot's two JIT compilers: C1 compiles quickly with light optimisation, C2 slowly with heavy optimisation. §1.7, §16.8
- **Cache** — A fast store of recently used or precomputed results kept so the next request is cheaper; it needs a size limit or eviction rule, or it becomes a memory leak. §10.4, §16.11, System Design Fundamentals
- **Call stack** — The stack of frames for the method calls a thread currently has in progress: a call pushes a frame, a return pops it. §4.5, §16.3
- **`Callable`** — A task interface `V call() throws Exception` that returns a value; see `Runnable`. §14.11
- **Callback** — A function handed to other code so that code can call it later, for example the lambda given to `thenApply`. §15.1, §15.2
- **Canonical constructor** — A record's constructor that takes every component in declaration order; generated unless you write it. §6.8
- **Card table / remembered set** — GC bookkeeping that records which old-generation areas point into the young generation, so a young collection needn't scan the whole heap. §16.5
- **Carrier thread** — A platform thread on which the JVM mounts virtual threads to run them. §15.4

- **CAS (compare-and-set)** — An atomic CPU instruction that updates a value only if it still equals an expected value. §14.10
- **Casting** — Explicit conversion `(Type) value` — narrowing a primitive, or checking an object's run-time type. §2.7, §6.4
- **CDI** — Jakarta Contexts and Dependency Injection: the Jakarta EE standard for dependency injection. §1.2
- **CDS (Class Data Sharing)** — Saving already-parsed classes in an archive that later JVM runs map in, so they start faster; the *AOT cache* builds on it. §16.2
- **Channel** — An NIO connection to a file or socket (such as `FileChannel`) that reads and writes through `ByteBuffer`s. §13.9
- **`char`** — The 16-bit unsigned primitive holding one UTF-16 code unit (0 … 65 535). §2.3
- **Character stream** — `Reader`/`Writer`: I/O in 16-bit characters, decoded with a charset. §13.5
- **Charset (encoding)** — The rules mapping characters to bytes and back, such as UTF-8; the JDK default since Java 18 is UTF-8. §13.7
- **Checked exception** — An `Exception` outside the `RuntimeException` branch; must be caught or declared with `throws`. §8.2, §8.3
- **Check-then-act** — Testing a condition and then acting on it as two separate steps; another thread can change things in between, so the pair must be made atomic. §14.5
- **CI (continuous integration)** — A server that automatically builds and tests every change pushed to a shared repository. §18.1, §18.2
- **Circular array** — An array used as a ring whose indexes wrap from the end back to the start, so a queue can grow at both ends cheaply (`ArrayDeque`). §10.6
- **Class** — A blueprint that defines fields, constructors and methods; objects are its instances. §5.1
- **Class file** — The `.class` file `javac` produces, containing bytecode and a constant pool. §1.6
- **Class initialisation** — Running a class's static field initialisers and `static {}` blocks, once, on first active use. §5.10, §16.2
- **Class loader** — The object that finds and loads class bytes; loaders delegate to their parent first. §16.2
- **Class path** — The list of directories and JARs where the JVM looks for classes (`-cp`). §1.5
- **Class-File API** — The `java.lang.classfile` API (final in Java 24) for reading and writing class files. §17.9
- **`Clock`** — `java.time.Clock`: a replaceable source of "now"; inject it so tests can fix the time. §7.8, §18.9
- **Closure** — A function together with the variables it captured from its surrounding scope; lambdas capture effectively final locals. §11.5
- **Code cache** — Memory where the JIT stores compiled machine code. §16.3
- **Code coverage** — The share of lines or branches that the tests execute, measured with tools such as JaCoCo; high coverage doesn't prove good tests. §18.9
- **Code point** — The number Unicode gives one character; a code point above U+FFFF needs two `char`s (a *surrogate pair*) in Java. §7.2
- **Code unit** — One 16-bit `char` of UTF-16 text; most characters take one code unit, some take two. §2.3, §7.2
- **Coffman conditions** — The four conditions that must all hold for a deadlock: mutual exclusion, hold and wait, no preemption, circular wait. Break any one to prevent it. §15.9
- **Collection** — The root interface of lists, sets and queues (maps are separate). §10.1
- **`Collections`** — Utility class with `sort`, `reverse`, `shuffle`, `frequency`, `unmodifiableList` and more. §10.9
- **Collector** — A recipe for accumulating stream elements into a result, e.g. `groupingBy`, `toMap`, `joining`. §12.5
- **Coloured pointers** — ZGC's trick of storing a few GC status bits inside object references themselves. §16.5
- **Compact constructor** — A record constructor without a parameter list, used to validate or normalise components. §6.8

- **Compact object headers** — Java 25 option (`-XX:+UseCompactObjectHeaders`) that shrinks object headers from 12 to 8 bytes; on by default from JDK 27. §16.4, §17.10
- **Compact source file** — A Java 25 source file with top-level methods and no class declaration; run with `java File.java`. §4.9, §17.10
- **Compaction** — Moving live objects next to each other so free memory forms one large block. §16.5
- **`Comparable`** — Interface whose `compareTo` defines a class's natural order. §10.8
- **`Comparator`** — A separate object whose `compare` defines an ordering; built with `comparing`, `thenComparing`, `reversed`. §10.8
- **Compiler** — A program that translates source code into another form: `javac` turns Java into bytecode, the JIT turns bytecode into machine code. §1.5, §1.7
- **Compile-time constant** — A `final` primitive or `String` initialised with a constant expression; folded by `javac`. §7.1
- **`CompletableFuture`** — A `Future` you can complete manually and compose asynchronously (`thenApply`, `thenCompose`, `allOf`). §15.2
- **`CompletionException`** — The unchecked wrapper `CompletableFuture.join()` throws when the computation failed. §15.2
- **Composition** — Building a class from other objects held in fields and delegating work to them — often preferred to inheritance. §6.1
- **Compound assignment** — Operators such as `+=`, which include an implicit cast back to the variable's type. §2.8
- **Compressed oops** — Storing object references in 32 bits on a 64-bit JVM (heaps up to about 32 GB) to save memory. §16.4
- **Concurrency** — Several tasks in progress over the same period, possibly interleaved on one core. §14.1
- **Concurrent collection** — A collection from `java.util.concurrent`, such as `ConcurrentHashMap` or `BlockingQueue`, designed for safe use by many threads at once. §14.12
- **`ConcurrentHashMap`** — A thread-safe, highly scalable hash map with atomic `merge`/`compute`; no `null` keys or values. §14.12
- **`ConcurrentModificationException` (CME)** — Thrown by fail-fast iterators when a collection is structurally changed during iteration. §10.7
- **`Condition`** — A waiting queue tied to an explicit `Lock` (`await`, `signal`); one lock can have several conditions. §14.9
- **Connected component** — A group of graph nodes that can all reach each other; counted with DFS, BFS or union-find. §19.16, §19.18
- **Connection pool** — A set of open database connections reused across requests, because opening one is slow (e.g. HikariCP). §18.11
- **Constant folding** — The compiler working out a constant expression such as `"ja" + "va"` at compile time. §7.1
- **Constant pool** — The table inside a class file that holds the literals and symbolic names its bytecode refers to. §1.6
- **Constructor** — A special method with the class's name that initialises a new object. §5.3
- **Constructor chaining** — Calling another constructor with `this(...)` or the parent's with `super(...)`. §5.3, §6.1
- **Container** — A lightweight, isolated package of an application with its own CPU and memory limits (e.g. Docker); the JVM reads those limits to size itself. §16.3, §16.7
- **Context switch** — The OS pausing one thread and running another on the same CPU core. §14.1
- **`continue`** — Skips to the next iteration of the innermost (or labelled) loop. §3.9
- **`CopyOnWriteArrayList`** — A thread-safe list that copies its array on every write; iterators see a snapshot. §14.12
- **`CountDownLatch`** — A one-shot synchronizer: threads `await()` until `countDown()` has been called n times. §14.13
- **Covariant return type** — An overriding method may return a subtype of the overridden method's return type. §6.3
- **CPU** — Central processing unit: the processor that executes instructions; its cores each run one thread at a time. §14.1

- **CPU-bound** — Work limited by computation rather than by waiting for I/O; more threads than cores won't speed it up. §12.8, §15.4
- **Critical section** — Code that touches shared mutable state and must not run in two threads at once. §14.1
- **Currying** — Turning a function of several arguments into a chain of one-argument functions. §11.8
- **`CyclicBarrier`** — A reusable synchronizer where n threads wait for each other before all continue. §14.13

### D

- **Daemon thread** — A background thread that doesn't keep the JVM alive; virtual threads are always daemon. §14.2
- **DAG (directed acyclic graph)** — A graph whose edges have a direction and that contains no cycle; it always has a topological order. §19.17
- **Dangling else** — The question of which `if` an `else` belongs to when braces are missing; Java pairs it with the nearest `if`. §3.1
- **Data race** — Two threads access the same variable without a happens-before order and at least one writes. §15.8
- **Data structure** — A way of organising data in memory — array, list, hash table, tree, heap — chosen for the operations you need to be fast. §10.1, Ch 19
- **`DateTimeFormatter`** — Immutable, thread-safe formatter and parser for `java.time` values. §7.11
- **Daylight saving time (DST)** — The seasonal shift of clocks in many regions, which makes some local times skip or repeat. §7.10
- **Dead-code elimination** — The JIT removing computation whose result is never used — the classic trap in hand-written benchmarks. §16.9
- **Deadlock** — Threads wait forever for locks each other holds. §15.9
- **Declarative style** — Describing what result you want (filter, map, group) instead of writing the loop that produces it. §11.8, §12.11
- **Decorator** — A wrapper that adds behaviour to an object while offering the same interface, e.g. `BufferedReader` around a `Reader`. §13.5
- **Default constructor** — The no-argument constructor the compiler adds when a class declares no constructor. §5.3
- **Default method** — An interface method with a body (Java 8), inherited by implementing classes unless overridden. §6.6
- **Default value** — The initial value of a field or array element: `0`, `false` or `null`; local variables have none. §2.2, §2.3
- **Defensive copy** — Copying a mutable input or output so outside code can't change an object's internal state. §5.9
- **Definite assignment** — The compiler rule that a local variable must certainly have been assigned before it is read. §2.2
- **Deoptimisation** — The JIT discarding compiled code whose speculative assumptions stopped holding. §16.8
- **Dependency** — A library your project uses; build tools download it together with the libraries it needs in turn. §18.1, §18.2
- **Dependency injection** — Supplying an object's collaborators from outside (usually through the constructor) instead of creating them inside. §18.6, §18.14
- **`Deque`** — A double-ended queue: insert and remove at both ends; used as stack or queue. §10.6
- **Deserialisation** — Rebuilding objects from bytes; dangerous with untrusted input. §13.10
- **Devirtualisation** — The JIT replacing an overridable (virtual) call with a direct call when only one implementation has been seen. §16.8
- **DFS (depth-first search)** — Visiting a graph by following each path as deep as possible before backing up, using recursion or an explicit stack. §19.16
- **Diamond operator** — `<>` in `new HashMap<>()` — lets the compiler infer the type arguments. §9.2
- **Difference array** — An array of "change starts here" marks: add v at l and −v at r + 1, then one running sum applies every range update. §19.11

- **Dijkstra's algorithm** — Finds the cheapest paths from one node in a graph with non-negative edge weights, using a priority queue. §19.16
- **Direct memory** — Native memory outside the heap used by direct `ByteBuffer`s. §16.3
- **Divide and conquer** — Splitting a problem into smaller parts, solving each (often recursively or in parallel) and combining the results. §4.5, §15.3
- **Docker** — The most widely used tool for building and running containers. §16.3, §18.9
- **`double`** — The 64-bit IEEE 754 floating-point primitive; the default for decimals. §2.3, §2.11
- **`do-while`** — A loop that runs its body once before testing the condition. §3.8
- **Downcasting / upcasting** — Upcasting treats an object as one of its supertypes (automatic); downcasting casts back to a subtype and can throw `ClassCastException`. §6.4
- **Downstream collector** — A collector applied to each group inside `groupingBy` or `partitioningBy`, such as `counting()`. §12.5
- **DSA** — Data structures and algorithms — the subject of coding interviews. Ch 19
- **DSL** — Domain-specific language: a small language for one job, such as Gradle's Kotlin build scripts. §18.3
- **DTO** — Data transfer object: a plain data carrier passed between layers or over the network; a record is a natural fit. §6.8
- **Dummy node / sentinel** — A special marker value (such as −1 for "not seen") or a placeholder node in front of a linked list's head that removes a special case from the code. §19.11, §19.22
- **`Duration`** — A `java.time` amount of time in seconds and nanoseconds (e.g. `PT1H30M`). §7.10
- **Dutch national flag** — Splitting an array into three regions (less than, equal to, greater than a value) in one pass. §19.25
- **Dynamic dispatch** — Choosing which overridden method to run from the object's run-time class. §6.4
- **Dynamic programming (DP)** — Solving a problem from the answers to overlapping smaller subproblems, computing each answer once (memoisation or tabulation). §19.19

**E**

- **Eclipse Temurin** — The Adoptium project's free, widely used OpenJDK build. §1.2
- **Eden** — The part of the young generation where new objects are first allocated. §16.5
- **Effectively final** — A local variable that is never reassigned after initialisation; lambdas may capture only these. §11.5
- **Encapsulation** — Hiding an object's state behind methods so its invariants can't be broken from outside. §5.7
- **Encounter order** — The order in which a stream meets its elements: a `List` has one, a `HashSet` does not. §12.4
- **Endianness** — The byte order of multi-byte numbers: big-endian (Java's default) stores the most significant byte first. §13.9
- **End-to-end test** — A test of the whole running system through its real entry points, as a user would use it. §18.9
- **Enhanced `for`** — The for-each loop, `for (T x : iterable)`. §3.8
- **Enum** — A type with a fixed set of named instances; may have fields, constructors and methods. §6.9
- **`EnumMap` / `EnumSet`** — Very compact, fast map/set implementations for enum keys/elements. §10.4, §10.3
- **Environment variable** — A named setting the operating system passes to every program it starts, such as `JAVA_HOME` or `PATH`. §1.4
- **Epoch** — The fixed instant 1970-01-01T00:00:00Z from which `Instant` and `System.currentTimeMillis()` count. §7.10
- **Epsilon GC** — A no-op collector that never frees memory; for testing. §16.5
- **`equals`** — The method that defines logical equality; must be consistent with `hashCode`. §6.13
- **Equivalence class** — A group of inputs that the code treats the same way, so one test (plus the boundaries) covers it. §18.9
- **`Error`** — The `Throwable` branch for serious JVM problems such as `OutOfMemoryError`; unchecked, normally not caught. §8.2
- **Escape analysis** — A JIT analysis proving an object doesn't leave a method, enabling scalar replacement and lock elision. §16.8

- **Escape sequence** — A backslash code for a special character in a literal: `\n`, `\t`, `\"`, `\\`, `A`. §2.4
- **Exception** — An object signalling an abnormal condition, thrown with `throw` and caught with `catch`. §8.1
- **Exception chaining** — Wrapping a low-level exception as the cause of a higher-level one. §8.8
- **Exception translation** — Catching an exception at a layer boundary and throwing one that fits the higher layer's abstraction. §8.8
- **Executable JAR** — A JAR whose manifest names a `Main-Class`, so it runs with `java -jar app.jar`. §18.12
- **`ExecutorService`** — A thread-pool abstraction that accepts tasks and returns `Future`s; `AutoCloseable` since Java 19. §14.11
- **Exhaustive switch** — A `switch` whose cases cover every possible value (all enum constants, all permitted subtypes, or `default`). §3.3, §3.6
- **Explicit lock** — A `java.util.concurrent.locks.Lock` such as `ReentrantLock`, acquired and released in code. §14.9
- **Expression** — Code that produces a value, such as `a + b`, `x > 0` or a method call; a statement is a complete instruction. §2.1

## F

- **Factory method** — A static method that creates instances (`List.of`, `Integer.valueOf`); unlike a constructor it has a name and may cache or return a subtype. §5.3
- **Fail-fast iterator** — An iterator that throws `ConcurrentModificationException` on a best-effort basis when its collection changes structurally. §10.7
- **Failure atomicity** — A method that fails leaves its object exactly as it was before the call. §8.7, §8.10
- **Fair lock** — A lock that grants access in waiting order, avoiding starvation at some cost in speed. §14.9
- **Fake** — A working but simplified test double, such as an in-memory repository. §18.6
- **Fall-through** — In a classic `case X:` switch, execution running on into the next case when there is no `break`. §3.2
- **Fast and slow pointers (Floyd's cycle-finding algorithm)** — Two references moving through a list at different speeds (1 and 2 steps): fast reaching the end puts slow at the middle; they meet only if there is a cycle. §19.22
- **Fat JAR** — A single JAR containing the application and all of its dependencies (also "uber JAR"). §18.12
- **Feature release** — One of the two Java versions released every year, in March and September. §1.12
- **Field** — A variable declared in a class that holds an object's state (or, if `static`, the class's). §5.1
- **FIFO / LIFO** — First in, first out (a queue) / last in, first out (a stack). §10.1, §10.6
- **File descriptor** — The operating system's small-integer handle for an open file or socket; each process may hold only a limited number. §16.13
- **`final`** — Modifier: a final variable is assigned once, a final method can't be overridden, a final class can't be extended. §5.8
- **Finalization** — The deprecated-for-removal `finalize()` mechanism (Java 18, JEP 421); use try-with-resources or `Cleaner`. §17.12
- **`finally`** — A block after `try` that always runs when control leaves the `try`. §8.4
- **Flaky test** — A test that sometimes passes and sometimes fails with no code change, usually because of timing, randomness or shared state. §18.9
- **Flexible constructor bodies** — Java 25 feature allowing statements that don't use `this` before `super(...)` or `this(...)`. §17.10
- **`float`** — The 32-bit IEEE 754 floating-point primitive (~7 significant digits). §2.3
- **Floating-point error** — Rounding error because binary fractions can't represent most decimals exactly (`0.1 + 0.2`). §2.11
- **Foreign Function & Memory API** — The `java.lang.foreign` API (final in Java 22) to call native code and manage off-heap memory, replacing JNI. §17.7

- **Fork/Join** — A framework for recursive divide-and-conquer tasks run by a work-stealing `ForkJoinPool`. §15.3
- **Formatter specifier** — A pattern such as `%5.2f` or `%-10s` in `String.format`/`printf`. §7.4
- **Fragile base class** — A subclass breaking because its superclass's internals changed — a risk of inheritance across teams or libraries. §6.1
- **Frame (stack frame)** — The record one method call keeps on its thread's stack: parameters, local variables and where to return. §4.5, §16.3
- **`fsync`** — An operating-system call that forces written data out of memory caches onto the disk. §16.13
- **Full GC** — A collection of the whole heap, usually with a long stop-the-world pause. §16.5
- **Functional interface** — An interface with exactly one abstract method; a lambda or method reference can implement it. §11.2
- **Functional programming** — A style built from pure functions, immutable data and functions passed around as values. §11.8
- **`Future`** — A handle to an asynchronous result, obtained with a blocking `get()`. §15.1

**G**

- **GA (general availability)** — The official release of a Java version for production use. §20.15
- **Garbage collection (GC)** — Automatic reclamation of memory used by objects that are no longer reachable. §1.8, §16.5
- **Garbage-First (G1)** — The default collector since Java 9 (in all environments from JDK 27); region-based, with a pause-time goal. §16.5
- **Gatherer (stream gatherers)** — A custom intermediate stream operation used with `stream.gather(...)` (final in Java 24). §12.9
- **GC roots** — Starting points for reachability: thread stacks, static fields, JNI references. §16.5
- **GCD / LCM** — Greatest common divisor (the largest number dividing both) / least common multiple (the smallest number both divide). §19.27
- **Generational hypothesis** — Most objects die young, so collectors split the heap into young and old generations. §16.5
- **Generic method** — A method that declares its own type parameters: `static <T> T first(List<T> xs)`. §9.4
- **Generics** — Type parameters on classes, interfaces and methods for compile-time type safety without casts. §9.1
- **Getter / setter** — Methods that read (`getName()`) or change (`setName(...)`) a private field. §5.7
- **GIL** — Python's global interpreter lock, which lets one thread run Python code at a time; Java has none, so its threads truly run in parallel. §14.14
- **GraalVM** — Oracle's JDK distribution with an advanced JIT and *Native Image*. §1.2, §18.14
- **Gradle** — A build tool using a Groovy or Kotlin DSL with incremental, cached tasks. §18.3
- **Graph** — A set of nodes joined by edges, which may have a direction and a weight (cost); trees, grids and road maps are graphs. §19.16
- **Greedy algorithm** — Building an answer by always taking the choice that looks best right now and never undoing it; correct only when the greedy-choice property holds. §19.24
- **Greedy-choice property** — Some optimal answer starts with the locally best choice — the condition that makes a greedy algorithm correct. §19.24
- **Guard clause** — An early `return` (or `throw`) at the top of a method for special cases, keeping the main logic unindented. §3.1
- **Guard (pattern)** — A `when` condition attached to a `case` pattern. §3.6
- **GUI** — Graphical user interface: windows, buttons and menus. §16.10

**H**

- **Handle-or-declare rule** — A checked exception must be caught or listed in the method's `throws` clause. §8.3
- **Happens-before** — The Java Memory Model relation guaranteeing one action's effects are visible to another. §15.8
- **Hash table** — An array of buckets indexed by each key's hash code, giving O(1) average lookups; `HashMap` is one. §10.5

- **`hashCode`** — Returns an `int` used by hash-based collections; equal objects must return equal hash codes. §6.13
- **`HashMap`** — An unordered hash-table map with average O(1) operations; allows one `null` key. §10.4, §10.5
- **`HashSet`** — A set backed by a `HashMap`; unordered, average O(1). §10.3
- **Heap** — The JVM memory area where all objects and arrays live, shared by all threads and garbage-collected. §16.3
- **Heap (data structure)** — A tree kept in an array whose root is always the smallest (min-heap) or largest (max-heap) item; O(log n) insert and remove; `PriorityQueue` is a min-heap by default. Unrelated to the JVM *heap*. §10.6, §19.13
- **Heap dump** — A snapshot of every object in the heap, analysed with Eclipse MAT or VisualVM. §16.10, §16.11
- **Heap pollution** — A variable of a parameterized type referring to an object of the wrong type, caused by unchecked operations. §9.8
- **Helpful NullPointerException** — NPE messages that name exactly which expression was `null` (Java 14, on by default since 15). §8.1
- **Hexadecimal** — Base-16 notation with digits 0–9 and a–f, written `0x2A` in Java. §2.4
- **Hibernate** — The most popular JPA implementation, which maps Java objects to database tables. §18.11, §18.14
- **Hiding** — A static method or field in a subclass with the same name as the parent's; chosen by compile-time type, not overridden. §6.2
- **Higher-order function** — A function that takes or returns another function, such as `map` or `Comparator.comparing`. §11.8
- **HikariCP** — A fast JDBC connection pool; Spring Boot's default. §18.11
- **HotSpot** — The JVM inside OpenJDK and most Java distributions, named after its compilation of "hot" code. §1.3
- **HTTP** — Hypertext Transfer Protocol: the request/response protocol of the web, spoken by browsers, REST APIs and Java's `HttpClient`. §18.14
- **Humongous object** — In G1, an object at least half a region in size, which gets regions of its own. §16.5

### I

- **IDE** — Integrated development environment, such as IntelliJ IDEA, Eclipse or VS Code: editor, compiler, debugger and tools in one. §1.11
- **Identity** — Whether two references point to the very same object (`==`), as opposed to being equal in value (`equals`). §6.13
- **`IdentityHashMap`** / **`WeakHashMap`** — Special maps: `IdentityHashMap` compares keys with `==`; `WeakHashMap` drops an entry once nothing else references its key. §10.4
- **IEEE 754** — The international standard for binary floating-point numbers that `float` and `double` follow. §2.11
- **`IllegalArgumentException`** — Unchecked exception for a method argument that violates a precondition. §8.7
- **`IllegalStateException`** — Unchecked exception for a call made when the object is in the wrong state. §8.7
- **Immutable object** — An object whose state can't change after construction (the opposite of **mutable**), such as `String` or a record of immutable values. §5.9
- **Incidental indentation** — The common leading whitespace that a text block strips from every line. §2.12
- **Incubator module** — An experimental API in a `jdk.incubator.*` module, e.g. the Vector API. §17.1
- **In-degree** — The number of edges pointing into a node; Kahn's algorithm starts from the nodes with in-degree 0. §19.17
- **Inheritance** — A subclass acquiring a superclass's members with `extends`; Java has single class inheritance. §6.1
- **Initialisation order** — Static initialisers once per class, then for each object: parent constructor, field initialisers and instance blocks, constructor body. §5.10

- **Inlining** — The JIT copying a callee's body into its caller to remove call overhead and enable further optimisation. §16.8
- **Inner class** — A non-static nested class; each instance holds a reference to an enclosing instance. §6.10
- **In-place** — Changing the input directly with O(1) extra memory instead of building a new copy. §19.8
- **Instance** — An object created from a class; instance members belong to each object, `static` ones to the class. §5.1
- **Instance initialiser block** — A `{ … }` block in a class body that runs for every new object, before the rest of the constructor body. §5.10
- **Instance main method** — A Java 25 `main` that is not `static` and may take no parameters: `void main()`. §4.9, §17.10
- **`instanceof`** — Operator testing an object's run-time type; with a pattern it also binds a variable (Java 16). §3.5
- **`Instant`** — A point on the UTC time-line, stored as seconds and nanoseconds since the epoch; use it for timestamps. §7.10
- **`int`** — The 32-bit signed integer primitive, range $-2^{31}$ … $2^{31}-1$. §2.3
- **`Integer` cache** — `Integer.valueOf` returns cached objects for −128 … 127, so `==` happens to work only in that range. §7.6
- **Integer division** — Dividing two integers drops the fraction, rounding toward zero: `7 / 2` is 3 and `-7 / 2` is −3. §2.8, §19.6
- **Integer overflow** — A result outside a type's range wraps around silently (`Integer.MAX_VALUE + 1` is `MIN_VALUE`). §2.10
- **Integration test** — A test that runs several real parts together (for example code plus a real database) instead of one class in isolation. §18.9
- **IntelliJ IDEA** — JetBrains' widely used Java IDE. §1.11
- **Interface** — A contract type: abstract, default, static and private methods plus constants; a class can implement many. §6.6
- **Intermediate operation** — A lazy stream operation such as `filter` or `map` that returns a new stream. §12.3
- **Interning** — Obtaining the pooled copy of a string with `intern()`. §7.1
- **Interpreter** — The JVM component that executes bytecode instruction by instruction before the JIT compiles hot code. §1.7
- **Interruption** — Cooperative cancellation: `interrupt()` sets a flag that blocking methods answer with `InterruptedException`. §14.4
- **Interval** — A range with a start and an end, such as a meeting `[9, 11)`; interval problems usually sort by start and sweep. §19.15
- **Invariance (generics)** — `List<Integer>` is not a `List<Number>` even though `Integer` is a `Number`; wildcards add the flexibility back. §9.6
- **Invariant** — A condition that must always hold — for an object after every public method, or for a loop on every iteration. §5.7
- **Inverse Ackermann function** — α(n), a function that grows so slowly it is below 5 for any realistic n; union-find costs O(α(n)) per operation. §19.18
- **`invokedynamic`** — A bytecode instruction whose call target is linked at run time; Java uses it for lambdas and string concatenation. §1.6
- **I/O** — Input/output: reading and writing files, network sockets, the console and databases. §13.1
- **`IO` class** — `java.lang.IO` (Java 25): `println`, `print` and `readln` for simple console programs. §4.9, §17.10
- **`Iterable`** — Anything with an `iterator()`; usable in an enhanced `for`. §10.1, §10.7
- **`Iterator`** — An object with `hasNext`, `next` and optional `remove` for walking a collection. §10.7

## J

- **Jackson / Gson** — Popular third-party libraries for reading and writing JSON in Java. §13.12
- **Jakarta EE** — The Eclipse Foundation's enterprise Java specifications (formerly Java EE): Servlet, JPA, CDI, REST. §1.2, §18.14

- **JAR** — A ZIP archive of class files and resources with a manifest; executable with `java -jar` when it names a `Main-Class`. §18.12
- **Java agent** — A JAR loaded with `-javaagent` that can change classes as they load (used by profilers and Mockito); loading one into a running JVM now prints a warning. §17.6, §18.6
- **Java Memory Model (JMM)** — The rules defining when writes by one thread become visible to others. §15.8
- **Java SE** — Java Standard Edition: the core language, JVM and standard library this book covers. §1.2
- **`javac`** — The Java compiler; turns `.java` source into `.class` bytecode. §1.5
- **Javadoc** — Documentation comments `/** … */` and the tool that turns them into HTML API pages. §1.5, §2.1
- **`JAVA_HOME`** — The environment variable naming the JDK directory that build tools should use. §1.4
- **`java.time`** — The modern date-time API (Java 8): immutable, thread-safe classes such as `LocalDate` and `Instant`. §7.8
- **JCIP** — *Java Concurrency in Practice*, the classic book on Java threads and locking. §14.4
- **`jcmd`** — The command-line tool that sends diagnostic commands to a running JVM. §16.10
- **JDBC** — The standard API for talking to relational databases: `Connection`, `PreparedStatement`, `ResultSet`. §18.11
- **JDK** — Java Development Kit: the JVM, the class library and tools such as `javac`, `jar`, `jshell`, `jcmd`. §1.3
- **JDK Mission Control (JMC)** — A desktop application for analysing JFR recordings. §16.10
- **JEP** — JDK Enhancement Proposal: the numbered document describing a new feature. §1.12, §17.1
- **JFR (JDK Flight Recorder)** — Low-overhead event recording built into the JVM, for production profiling. §16.10
- **JIT compiler** — Just-in-time compiler that turns hot bytecode into optimised machine code while the program runs. §1.7, §16.8
- **`jlink`** — Builds a custom runtime image containing only the modules an application needs. §16.12
- **JLS** — The Java Language Specification: the official definition of the language's rules. §4.3
- **JMH** — The Java Microbenchmark Harness, which handles warm-up, forks and dead-code elimination for benchmarks. §16.9
- **JMX** — Java Management Extensions: the JVM's API for publishing and reading monitoring data and controls. §16.1
- **JNI** — Java Native Interface: the older way to call C or C++ code from Java, being replaced by the *Foreign Function & Memory API*. §16.1
- **Josephus problem** — n people stand in a circle and every k-th is removed; the survivor's position follows $J(n) = (J(n - 1) + k) \bmod n$. §19.27
- **JPA** — Jakarta Persistence: the standard API for mapping objects to database tables, usually through Hibernate. §1.2, §18.14
- **JPMS** — The Java Platform Module System (Java 9): `module-info.java` with `requires`, `exports`, `opens`. §16.12
- **JRE** — Java Runtime Environment: the JVM plus class library, enough to run but not compile programs; since Java 11 Oracle ships only the JDK (trimmed runtimes are built with `jlink`). §1.3
- **`jshell`** — The Java REPL (Java 9) for trying expressions interactively. §1.9
- **JSON** — JavaScript Object Notation: a common text format of objects, arrays, strings and numbers for exchanging data. §13.12
- **`jstack`** — Prints a thread dump of a running JVM. §16.10
- **JUL** — Short for `java.util.logging`, the JDK's built-in logging API. §18.10
- **JUnit** — The standard Java testing framework (tests are `@Test` methods): the Platform discovers and launches tests, Jupiter is the modern test API, Vintage runs old JUnit 4 tests. §18.4
- **JVM** — Java Virtual Machine: loads, verifies and runs bytecode, manages memory and threads. §1.3, §16.1

**K**

- **Kafka** — A distributed, durable message log used to stream events between services. §20.18

- **Kahn's algorithm** — Topological sort by repeatedly taking a node with in-degree 0 and removing its outgoing edges. §19.17
- **Kernel** — The core of the operating system that manages memory, devices and scheduling; programs reach it through system calls. §16.13
- **Key derivation function API** — `javax.crypto.KDF` (Java 25) for deriving keys, e.g. with HKDF. §17.10
- **Keyword** — A word reserved by the language (`class`, `if`, `int`); contextual keywords such as `var` and `record` are special only in certain positions. §2.1
- **Kotlin** — A JetBrains language for the JVM that is fully interoperable with Java. §20.18
- **Kruskal's algorithm** — Building a minimum spanning tree by taking edges cheapest first and skipping any that would close a cycle (checked with union-find). §19.18

**L**

- **Label** — A name before a loop (`outer:`) that `break` or `continue` can target. §3.9
- **Lambda expression** — An anonymous function, `(params) -> body`, implementing a functional interface. §11.2
- **Latency** — The time one operation takes from start to finish — a request, a disk read, a GC pause. §16.5, §16.6, System Design Fundamentals
- **Lazy evaluation** — Deferring work until a result is needed; streams do nothing until a terminal operation runs. §12.2
- **LeetCode** — A popular website of coding-interview practice problems; "LeetCode-style" means a timed algorithm question. Ch 19
- **Lexicographic order** — Dictionary order: compare the first elements and look at the next ones only on a tie. §4.8
- **Lifecycle (Maven)** — Maven's fixed sequence of build phases (`validate` … `compile` … `test` … `package` … `deploy`); running a phase runs every earlier one. §18.2
- **Linked list** — A chain of nodes, each holding a value and a reference to the next node (and, if doubly linked, the previous one). §10.2, §19.22
- **`LinkedHashMap`** — A hash map that keeps insertion (or access) order; the basis of a simple LRU cache. §10.4
- **`LinkedHashSet`** — A hash set that remembers insertion order. §10.3
- **`LinkedList`** — A doubly linked `List` and `Deque`; O(1) at the ends, O(n) indexing. §10.2
- **Liskov substitution principle** — A subclass must work anywhere its parent is expected, without surprising the caller. §6.1
- **`ListIterator`** — An iterator over a `List` that can move in both directions and `set` or `add` elements as it goes. §10.2
- **Literal** — A value written directly in source code: `42`, `3.14`, `'a'`, `"hi"`, `true`. §2.4
- **Little's law** — Requests in flight = arrival rate × time per request; e.g. 1,000 requests/s each taking 0.1 s keep 100 busy at once. §15.4
- **Livelock** — Threads keep changing state in response to each other but make no progress. §15.9
- **Liveness** — A program's ability to keep making progress; deadlock, livelock and starvation are liveness failures. §15.9
- **Load average** — On Linux, the average number of threads running or waiting to run over the last 1, 5 and 15 minutes. §16.13
- **Load barrier** — A tiny piece of code ZGC adds to every reference read so it can move objects while the application keeps running. §16.5
- **Load factor** — The fullness ratio (default 0.75) at which a `HashMap` doubles its table. §10.5
- **Local class** — A class declared inside a method body. §6.11
- **`LocalDate`** — A date without time or zone, e.g. `2025-03-09`. §7.9
- **`LocalDateTime`** — A date and time without a zone. §7.9
- **Locale** — A language-and-region setting (`Locale.US`, `Locale.ROOT`) that controls number, date and case formatting. §7.2, §7.4
- **`LocalTime` / `OffsetDateTime`** — `LocalTime` is a time of day with no date or zone; `OffsetDateTime` is a date-time with a fixed offset from UTC. §7.8
- **Lock elision** — The JIT removing locking on objects proved not to escape their thread. §16.8

- **Lock ordering** — Always acquiring several locks in one agreed global order, which makes deadlock impossible. §15.9
- **Logback / Log4j 2** — Logging back-ends that write SLF4J log calls to the console, files or other destinations. §18.10
- **Logging** — Recording events with a level and logger name to configurable destinations (SLF4J, Logback, `java.util.logging`). §18.10
- **Lombok** — A library that generates boilerplate such as getters at compile time by hooking into `javac`. §17.12
- **Lomuto partition** — A simple partition scheme that keeps one `store` index; everything left of it is smaller than the pivot. §19.25
- **`long`** — The 64-bit signed integer primitive, about $\pm 9.2 \times 10^{18}$. §2.3
- **`LongAdder`** — A counter spread over several cells to avoid contention; faster than `AtomicLong` when many threads increment it. §14.10
- **Loop unrolling** — The JIT repeating a loop body several times per iteration to cut loop overhead. §16.8
- **Lowest common ancestor (LCA)** — The deepest tree node that has both given nodes beneath it (a node counts as its own descendant). §19.23
- **LRU cache** — A cache that evicts the least-recently-used entry; built with an access-ordered `LinkedHashMap`. §10.4
- **LTS release** — A long-term-support Java release (8, 11, 17, 21, 25) that vendors patch for years. §1.12

M

- **`main` method** — The program entry point: classically `public static void main(String[] args)`. §4.9
- **Manifest** — `META-INF/MANIFEST.MF` inside a JAR: metadata such as the `Main-Class` to run. §18.12
- **`Map`** — A key → value collection with unique keys; not a subtype of `Collection`. §10.4
- **Mark word** — The part of an object header that holds the identity hash code, lock state and GC age. §16.4
- **Mark-and-sweep** — GC that first marks every reachable object, then frees the rest. §16.5
- **Marker interface** — An interface with no methods that just tags a class, such as `Serializable`. §6.2
- **`Math`** — Utility class for arithmetic: `abs`, `pow`, `sqrt`, `round`, `floorMod`, `clamp`, `*Exact`. §7.5
- **Maven** — A build tool driven by `pom.xml`, with a fixed lifecycle of phases and a central repository. §18.2
- **Maven Central** — The main public repository from which Maven and Gradle download libraries. §18.1, §18.2
- **MDC (mapped diagnostic context)** — Per-thread key-value data, such as a request id, that the logging framework adds to every log line. §18.10
- **Megamorphic call** — A call site that has seen many different receiver classes, which the JIT can't inline well. §11.8, §16.8
- **Memoisation** — Caching a function's results by its arguments so each is computed only once — top-down dynamic programming. §19.19
- **Memory leak (Java)** — Objects that are no longer needed but stay reachable, so the GC can't free them. §16.11
- **Memory-mapped file** — A file mapped into the process's address space, so reading or writing memory reads or writes the file. §13.9
- **Merge sort** — Sorting by splitting the input in half, sorting each half and merging them; O(n log n) and stable. §19.25
- **Metaspace** — Native memory holding class metadata (replaced PermGen in Java 8). §16.3
- **Method** — A named block of code in a class that can take parameters and return a value. §4.1
- **Method overloading** — Several methods with the same name but different parameter lists, chosen at compile time. §4.3
- **Method overriding** — A subclass providing its own implementation of an inherited instance method, chosen at run time. §6.3

- **Method reference** — Shorthand for a lambda that calls one method: `String::length`, `System.out::println`, `ArrayList::new`. §11.4
- **Method signature** — A method's name plus its parameter types. §4.1
- **Microbenchmark** — A timing of a tiny piece of code; easy to get wrong without a harness such as JMH. §16.9
- **Minimum spanning tree** — The cheapest set of edges that connects every node of a weighted graph without a cycle (Kruskal's algorithm). §19.18
- **Mock** — A test double that records calls and returns programmed answers (e.g. with Mockito). §18.6
- **Mockito** — The most widely used Java mocking library: `mock`, `when(...).thenReturn`, `verify`. §18.6
- **Modular arithmetic** — Working with remainders after division by a fixed number m; answers are often reported "mod 1,000,000,007" to keep them small. §19.6, §19.27
- **Modular JAR** — A JAR that contains a `module-info.class`. §18.12
- **Module** — A named, self-describing group of packages declared in `module-info.java`. §16.12
- **Module import declaration** — `import module java.base;` (Java 25): imports every package a module exports. §17.10
- **Module path** — Where the JVM looks for modules (`--module-path`), as opposed to the class path. §16.12
- **Mojibake** — Garbled text produced by decoding bytes with the wrong charset. §13.7
- **Monitor (intrinsic lock)** — The lock every Java object has, used by `synchronized`, `wait` and `notify`. §14.6
- **Monotonic stack** — A stack kept in increasing or decreasing order by popping out-of-order items first; finds each "next greater element" in O(n). §19.12
- **Mounting / unmounting** — A virtual thread being attached to, or detached from, a carrier thread; blocking I/O unmounts it. §15.4
- **Multi-catch** — `catch (A \| B e)` — one handler for several unrelated exception types. §8.5
- **Multi-release JAR** — A JAR that holds different versions of some classes for different Java releases. §18.12
- **Mutable reduction** — Accumulating stream elements into a mutable container such as a list or map — what `collect` does. §12.5
- **Mutation testing** — Judging tests by planting small bugs ("mutants") and checking that the tests fail. §18.9
- **Mutual exclusion** — Letting only one thread at a time into a critical section. §14.6

## N

- **NaN** — "Not a Number": the `double` result of undefined operations such as `0.0 / 0`; it is not even equal to itself. §2.11
- **Narrowing conversion** — Converting to a type with a smaller range (`long` → `int`); requires a cast and may lose information. §2.7
- **Native Image** — GraalVM's ahead-of-time compiler that turns a Java application into a fast-starting stand-alone executable. §18.12, §18.14
- **Native memory** — Memory the JVM uses outside the Java heap: metaspace, thread stacks, direct buffers, GC data. §16.3
- **Native Memory Tracking (NMT)** — A JVM option (`-XX:NativeMemoryTracking=summary`) that reports native memory use by area. §16.3
- **Natural order** — The order a class defines for itself by implementing `Comparable` (numbers ascending, strings alphabetically). §10.8
- **Nested class** — A class declared inside another; static nested or inner. §6.10
- **Netty** — A high-performance asynchronous networking framework for the JVM. §13.9
- **NIO.2** — The `java.nio.file` API (Java 7): `Path`, `Files`, `FileSystem`, `WatchService`. §13.1
- **Node** — One element of a linked structure (list, tree or graph) holding a value and references to its neighbours. §19.16, §19.22
- **Non-blocking** — An operation that returns immediately instead of making the thread wait, such as a CAS update. §14.10
- **`null`** — The reference value meaning "no object". §2.5

- `NullPointerException` **(NPE)** — Thrown when code uses a `null` reference as if it pointed to an object. §8.1

**O**

- **Object** — An instance of a class, with state (fields) and behaviour (methods), living on the heap. §5.1
- `Object` **class** — The root of every class hierarchy; defines `equals`, `hashCode`, `toString`, `getClass`, `wait`, `notify`. §6.2, §6.13
- **Object header** — The bookkeeping at the start of every object — mark word and class pointer: 12 bytes, or 8 with compact headers. §16.4
- **Observer / listener** — An object registered to be told about events; forgetting to unregister listeners is a classic memory leak. §16.11
- **OCP** — Oracle Certified Professional: the Java SE 21 Developer certification exam (1Z0-830). §20.18
- **Off-by-one error** — A loop or index that goes one step too far or stops one too short (`<=` instead of `<`). §3.10
- **Old generation** — The heap area for objects that survived several young collections. §16.5
- **On-stack replacement (OSR)** — Switching a long-running loop from interpreted to compiled code while it is still running. §16.8
- **OOP** — Object-oriented programming: organising code as objects that combine state and behaviour, using encapsulation, inheritance and polymorphism. §5.1
- **OpenJDK** — The open-source project and code base from which every mainstream Java distribution is built. §1.2
- **Operating system (OS)** — The software (Linux, Windows, macOS) that manages hardware, processes, memory and files for programs. §16.13
- **Operator** — A symbol that computes a value from operands, such as `+`, `==`, `&&` or `?:`. §2.8
- **Optimal substructure** — The best answer is built from the best answers to subproblems — one of the two conditions for dynamic programming. §19.19
- `Optional` — A container holding one value or none; a return type for "maybe no result". §11.7
- **Ordinal** — An enum constant's zero-based position in its declaration (`ordinal()`). §6.9
- `OutOfMemoryError` **(OOM)** — Thrown when the JVM can't allocate memory in the heap, metaspace or native memory. §16.3
- **Overlapping subproblems** — Plain recursion would solve the same subproblem many times — the other condition for dynamic programming. §19.19

**P**

- **P99** — The 99th-percentile latency: 99 % of operations finished at least this fast. §16.6
- **Package** — A namespace grouping related types, declared with `package`; mirrors the directory structure. §5.4
- **Package-private** — The default access (no modifier): visible within the same package only. §5.6
- **Page cache** — Operating-system memory holding recently read or written file data, so repeated reads skip the disk. §16.13
- **Page fault** — A CPU trap when a program touches memory that isn't currently mapped to RAM; the OS must load or allocate the page. §16.13
- **Palindrome** — Text (or a list) that reads the same forwards and backwards. §19.8
- **Parallel GC** — A throughput collector that uses many threads in stop-the-world pauses. §16.5
- **Parallel stream** — A stream whose work is split across the common `ForkJoinPool`; only worth it for large CPU-bound tasks. §12.8
- **Parallelism** — Tasks literally running at the same instant on different CPU cores; concurrency only requires them to overlap in time. §14.1
- **Parameterized test** — A test run once per set of arguments (`@ParameterizedTest` with a source). §18.5
- **Partition** — Rearranging an array around a pivot so smaller values end up on its left; the core step of quicksort and quickselect. §19.25
- **Pass-by-value** — Java always copies the argument value into the parameter; for objects, the value copied is the reference. §4.2
- `Path` — An NIO.2 object naming a file-system location; doesn't touch the disk by itself. §13.2

- **Path compression** — In union-find, pointing nodes at (or nearer to) the root during `find`, so later finds are almost O(1). §19.18
- **Path traversal** — An attack that uses `../` in a user-supplied file name to escape the intended directory. §13.2
- **Pattern matching** — Testing a value's shape and extracting parts in one step: type, record and unnamed patterns. §3.5, §3.6, §3.7
- **Pattern variable** — The variable a pattern binds when it matches, such as `s` in `o instanceof String s`. §3.5
- **PECS** — "Producer `extends`, consumer `super`": the wildcard rule for flexible generic APIs. §9.6
- **`Period`** — A `java.time` amount in years, months and days (e.g. `P1Y2M5D`). §7.9
- **`Phaser`** — A reusable barrier-like synchronizer whose number of parties can change between phases. §14.13
- **Pinning** — A virtual thread blocking without unmounting, holding its carrier; `synchronized` no longer pins since Java 24. §15.5
- **Pivot** — The value a partition splits around; picking it at random avoids quicksort's $O(n^2)$ worst case. §19.25
- **Platform thread** — A Java thread backed one-to-one by an OS thread. §15.4
- **Plugin (build)** — An add-on that gives Maven or Gradle a job to do, such as compiling, running tests or packaging. §18.2
- **Poison pill** — A special item put on a queue to tell its consumer to stop. §14.8, §14.12
- **Polymorphism** — One reference type, many run-time behaviours: a call runs the implementation of the object's actual class. §6.4
- **Precedence** — The order in which operators bind (`*` before `+`, `&&` before `\|\|`). §2.9
- **Precise rethrow** — `catch (Exception e) { throw e; }` keeps the narrower checked types the `try` can actually throw. §8.5
- **Prefix sum** — Running totals where `prefix[i]` is the sum of the first i elements, so any range sum costs one subtraction. §19.11
- **`PreparedStatement`** — A pre-compiled JDBC statement with `?` parameters; prevents SQL injection. §18.11
- **Preview feature** — A complete feature that may still change; needs `--enable-preview` on `javac` and `java`. §1.12, §17.1
- **Primitive stream** — `IntStream`, `LongStream` or `DoubleStream`, which avoid boxing and add `sum`/`average`. §12.6
- **Primitive type** — One of the eight built-in value types: `byte`, `short`, `int`, `long`, `float`, `double`, `char`, `boolean`. §2.3
- **`PriorityQueue`** — A binary-heap queue that always removes the smallest element (by natural order or a comparator). §10.6
- **`private`** — Access modifier: visible only inside the declaring top-level class. §5.6
- **Process** — A running program with its own memory; the threads inside it share that memory. §14.1, §16.13
- **Producer-consumer** — One set of threads creating work items and another processing them, joined by a shared (usually blocking) queue. §14.12
- **Profiling** — Measuring where a running program spends CPU time or allocates memory (JFR, async-profiler). §16.10
- **Promotion (numeric)** — Automatic widening of `byte`, `short` and `char` to `int` (or to the wider operand type) in arithmetic. §2.7
- **`Properties`** — A key=value configuration map loaded from `.properties` files. §13.11
- **`protected`** — Access modifier: visible in the package and to subclasses. §5.6
- **Pseudo-random number generator** — An algorithm whose output looks random but is fully determined by its seed. §7.12
- **Pure function** — A function whose result depends only on its arguments and that has no side effects. §11.8

## Q

- **Quarkus / Micronaut** — Cloud-native Java frameworks designed for fast start-up and low memory. §18.14, §20.18
- **Queue** — A collection that hands elements out in arrival order (FIFO): `offer` adds at the tail, `poll` removes from the head. §10.6
- **Quickselect** — Finding the k-th smallest element by partitioning and continuing on one side only; O(n) on average. §19.25
- **Quicksort** — Sorting by partitioning around a pivot and recursing on both sides; O(n log n) on average. §19.25

**R**

- **Race condition** — A result that depends on the timing of threads, e.g. lost updates from `count++`. §14.5
- **Raw type** — A generic type used without type arguments (`List`); disables generic type checking. §9.7
- **Reachability** — An object is live if a chain of references leads to it from a GC root. §16.5
- **Read-modify-write** — Reading a value, computing a new one and writing it back (`count++`); not atomic unless protected. §14.5
- **`ReadWriteLock`** — A lock pair allowing many concurrent readers or one writer. §14.9
- **Record** — A concise immutable data carrier (Java 16) with generated constructor, accessors, `equals`, `hashCode`, `toString`. §6.8
- **Record pattern** — A pattern that deconstructs a record: `case Point(int x, int y)` (Java 21). §3.7
- **Recurrence** — A formula that defines each answer from earlier ones, such as J(n) = (J(n − 1) + k) mod n. §19.19, §19.27
- **Recursion** — A method calling itself; needs a base case to stop. §4.5
- **Red-black tree** — A self-balancing binary search tree that keeps every operation O(log n); `TreeMap` and `TreeSet` use one. §10.4
- **Reduction** — Combining stream elements into one result with `reduce` or a collector. §12.4
- **Reentrant** — A lock its holder can acquire again without blocking itself, as `synchronized` and `ReentrantLock` allow. §14.6
- **`ReentrantLock`** — An explicit reentrant lock with `tryLock`, timeouts, fairness and conditions. §14.9
- **Refactoring** — Restructuring code to make it clearer without changing what it does; tests make it safe. §18.4, §18.9
- **Reference type** — Any non-primitive type; variables hold references to objects (or `null`). §2.5
- **Reflection** — Inspecting and using classes, methods and fields at run time (`getClass()`, `java.lang.reflect`). §16.2
- **Region (G1)** — One of the equal-sized blocks G1 divides the heap into; each region is eden, survivor, old or humongous at any time. §16.5
- **Regular expression** — A pattern language for matching text, used by `split`, `replaceAll`, `matches`, `Pattern`. §7.2
- **Reifiable type** — A type fully known at run time (`String`, `List<?>`, `int[]`); `List<String>` is not, because of erasure. §9.8
- **Rejection policy** — What a `ThreadPoolExecutor` does with a task when its queue is full and no thread is free — throw, run it in the caller, or drop it. §14.11
- **Relative / absolute path** — An absolute path starts at the file-system root; a relative path is resolved against the current directory. §13.2
- **REPL** — Read-eval-print loop: an interactive prompt such as `jshell`. §1.9
- **Resident set size (RSS)** — The physical memory a process actually occupies, as reported by the operating system. §16.10, §16.13
- **Resource leak** — Failing to close a file, socket or connection; prevented by try-with-resources. §8.6, §13.13
- **REST API** — A web API in which URLs name resources and HTTP methods (GET, POST …) act on them, usually exchanging JSON. §18.14
- **Reverse Polish Notation** — Postfix notation in which each operator follows its operands (`3 4 +`), evaluated with a stack. §19.12

- **RMI** — Remote Method Invocation: an old Java API for calling methods in another JVM, built on serialization. §13.10
- `Runnable` — A task interface with `void run()` and no result. §14.2
- `RuntimeException` — The superclass of unchecked exceptions. §8.2

### S

- **Safepoint** — A point at which all Java threads can be paused safely, for example for parts of a garbage collection. §16.5
- **Scalar replacement** — Replacing a non-escaping object's fields with local variables so it's never allocated. §16.8
- **Scale (`BigDecimal`)** — The number of digits after the decimal point; `2.0` and `2.00` have different scales. §7.7
- `Scanner` — A `java.util` class that reads and parses console or file input token by token. §2.13
- `ScheduledExecutorService` — An executor that runs tasks after a delay or repeatedly at a fixed rate. §14.11
- **Scheduler** — The part of the OS (or, for virtual threads, of the JVM) that decides which thread runs on which core next. §15.4, §16.13
- **Scoped value** — An immutable value bound for the duration of a call and readable by code it invokes (final in Java 25). §15.6
- **SDKMAN!** — A command-line tool for installing and switching between JDK versions. §1.4
- **Sealed class** — A class or interface (Java 17) that lists its *permitted subclasses* with `permits`. §6.7
- `SecureRandom` — A random-number generator strong enough for passwords, keys and tokens. §7.12
- **Seed** — The starting value of a pseudo-random generator; the same seed reproduces the same sequence. §7.12
- **Selector** — The value a `switch` examines. §3.2
- `Semaphore` — A synchronizer handing out a fixed number of permits to limit concurrent access. §14.13
- **Sequenced collection** — Java 21 interfaces (`SequencedCollection`, `SequencedSet`, `SequencedMap`) with `getFirst`, `getLast`, `reversed`. §10.11
- **Serial GC** — A single-threaded, stop-the-world collector for small heaps. §16.5
- **Serialization** — Converting objects to bytes with `ObjectOutputStream`; risky with untrusted data. §13.10
- `serialVersionUID` — The version number checked when deserialising a `Serializable` class. §13.10
- `Set` — A collection with no duplicate elements. §10.3
- **Shadowing** — A local variable or parameter hiding a field with the same name; use `this.name` to reach the field. §5.2
- **Shallow immutability** — Fields can't be reassigned, but the objects they refer to may still be mutable (true of records). §5.9, §6.8
- **Shebang** — A first line such as `#!/usr/bin/java --source 25` that lets a Java source file run as a Unix script. §1.10
- **Shenandoah** — A low-pause OpenJDK collector that compacts concurrently. §16.5
- **Short-circuit evaluation** — `&&` and `\|\|` skip the right operand when the left decides the result; also short-circuiting stream operations. §2.8, §12.4
- **Side effect** — Anything a function does besides returning a value: changing a field, printing, writing a file. §11.8
- **Sieve of Eratosthenes** — Finding every prime up to n by crossing out the multiples of each prime in turn. §19.27
- **SIGKILL** — The Unix signal that ends a process at once, with no chance to clean up (sent, for example, by the OOM killer). §16.13
- **SIMD** — Single instruction, multiple data: one CPU instruction that works on several numbers at once. §16.8
- **Single-file source launch** — Running `java Hello.java` without a separate compile step (Java 11). §1.10
- **Singleton** — A class with exactly one instance, often written as an `enum` with one constant. §5.3
- **SLF4J** — Simple Logging Facade for Java: a logging API with pluggable backends such as Logback. §18.10
- **Sliding window** — Keeping a running summary of a contiguous range while moving its ends forward, so subarray problems take O(n). §19.9

- **SNAPSHOT version** — A Maven version ending in `-SNAPSHOT`, meaning "work in progress — may change". §18.1
- **SOLID** — Five object-oriented design principles: single responsibility, open/closed, Liskov substitution, interface segregation, dependency inversion. §20.18
- **Space complexity** — How much extra memory an algorithm needs as its input grows, in Big-O terms. §19.7
- **`Spliterator`** — The splittable iterator behind every stream, which divides the work for parallel streams. §12.2
- **Spring Boot** — The most widely used Java framework for back-end services, with auto-configuration and starters. §18.14
- **Spy** — A test double that wraps a real object and records calls, letting you stub only some methods. §18.6
- **SQL** — Structured Query Language: the language for querying and changing relational databases. §18.11
- **SQL injection** — An attack in which user input is pasted into SQL text and runs as commands; `PreparedStatement` parameters prevent it. §18.11
- **Stable sort** — A sort that keeps equal elements in their original relative order. §10.8, §19.5
- **Stack (data structure)** — A LIFO collection: `push` adds on top and `pop` removes from the top; use `ArrayDeque`, not `Stack`. §10.6, §19.4
- **Stack (JVM)** — Per-thread memory of frames for active method calls, holding locals and partial results. §16.3
- **Stack trace** — The list of method frames active when an exception was created, newest first. §8.9
- **Stack unwinding** — An exception travelling up the call stack, abandoning each frame until a matching `catch` is found. §8.1
- **`StackOverflowError`** — Thrown when a thread's stack is exhausted, usually by runaway recursion. §4.5, §16.3
- **`StampedLock`** — A lock with an optimistic-read mode for read-mostly data; not reentrant. §14.9
- **Standard streams** — `System.in`, `System.out` and `System.err`: the input, output and error channels every program starts with. §13.8
- **Starvation** — A thread never gets the CPU or lock it needs because others always win. §15.9
- **Stateless / stateful operation** — A stateless stream step (`map`, `filter`) looks at one element at a time; a stateful one (`sorted`, `distinct`) must remember others. §12.3
- **`static`** — Belonging to the class rather than to an instance; one copy shared by all. §5.5
- **Static nested class** — A nested class declared `static`; it has no enclosing instance. §6.10
- **Stop-the-world (STW) pause** — A GC phase during which all application threads are paused. §16.5
- **Stream** — A lazy, one-shot pipeline of operations over a source of elements (`java.util.stream`). §12.1
- **`String`** — An immutable sequence of UTF-16 characters. §2.12, §7.1
- **String pool** — The JVM table of shared `String` objects for literals and interned strings. §7.1
- **String templates** — A preview feature in Java 21 and 22 that was withdrawn in Java 23. §17.8
- **`StringBuilder`** — A mutable, unsynchronised character buffer for building strings efficiently. §7.3
- **Strong encapsulation** — Since Java 17 JDK internals are inaccessible by default; `--add-opens` is required to reach them. §17.3
- **Structured concurrency** — Treating a group of concurrent subtasks as one unit (`StructuredTaskScope`); still preview in Java 25. §15.7
- **Structured logging** — Logging events as named fields (often JSON) that machines can search, rather than only free text. §18.10
- **Stub** — A test double that returns canned answers. §18.6
- **Subarray / subsequence** — A subarray (or substring) is a **contiguous** slice (no gaps); a subsequence keeps the original order but may skip elements. §19.9, §19.19
- **`super`** — Refers to the parent class: `super.method()` calls the parent's version, `super(...)` its constructor. §6.1
- **Suppressed exception** — An exception from `close()` attached to the primary exception in try-with-resources. §8.6
- **Surefire** — The Maven plugin that runs unit tests (its sibling Failsafe runs integration tests). §18.2

- **Surrogate pair** — Two `char`s encoding one Unicode character outside the Basic Multilingual Plane (many emoji). §7.2
- **Survivor space** — Young-generation areas to which objects that survived a collection are copied before promotion to the old generation. §16.5
- **`switch` expression** — A `switch` that yields a value (Java 14), using arrow labels or `yield`. §3.3
- **`synchronized`** — Keyword that acquires an object's monitor for a method or block, giving mutual exclusion and visibility. §14.6
- **Synchronizer** — A coordination object such as a latch, barrier, semaphore or phaser. §14.13
- **System call** — A request from a program to the operating-system kernel, such as reading a file. §16.13

### T

- **Tabulation** — Bottom-up dynamic programming: filling a table in an order where every entry's dependencies are already computed. §19.19
- **Template Method pattern** — A base class fixes the steps of an algorithm and lets subclasses fill in some of them. §6.5
- **`TemporalAdjusters`** — Ready-made date adjustments such as "last day of the month" or "next Monday". §7.9
- **Terminal operation** — The stream operation that produces a result and triggers execution (`collect`, `count`, `forEach`). §12.4
- **Ternary operator** — `cond ? a : b` — the conditional expression. §2.8
- **Test double** — Any stand-in for a real collaborator in a test: dummy, stub, fake, spy or mock. §18.6
- **Testcontainers** — A library that starts real services (databases, message brokers) in throwaway Docker containers for tests. §18.9
- **Test-driven development (TDD)** — Writing a failing test first, then the code to make it pass, then tidying up. §18.9
- **Text block** — A multi-line string literal delimited by `"""` (Java 15). §2.12
- **`this`** — A reference to the current object; `this(...)` calls another constructor. §5.2, §5.3
- **Thread** — An independent path of execution within a process, sharing the heap with other threads. §14.1
- **Thread confinement** — Keeping data reachable by only one thread, so it needs no locking. §14.14
- **Thread dump** — A snapshot of every thread's state and stack; shows deadlocks. §16.10
- **Thread pool** — A set of reusable worker threads that execute submitted tasks. §14.11
- **Thread priority** — A hint from 1 to 10 for the scheduler; the operating system may ignore it. §15.9
- **`ThreadLocal`** — A variable with an independent value per thread. §15.6
- **Thread-per-request** — A server model that gives every request its own thread; affordable at scale with virtual threads. §15.4
- **`ThreadPoolExecutor`** — The configurable thread-pool class behind most `Executors` factory methods. §14.11
- **Thread-safe** — Behaving correctly when used by several threads at once without extra coordination by callers. §14.1, §14.14
- **Throughput** — Work completed per unit of time — requests per second, or for a GC the share of time not spent collecting. §16.5, System Design Fundamentals
- **`throw` / `throws`** — `throw` raises an exception; `throws` in a method signature declares checked exceptions it may raise. §8.3, §8.7
- **`Throwable`** — The root class of everything that can be thrown: `Error` and `Exception`. §8.2
- **Tiered compilation** — Running code in the interpreter, then C1, then C2 as it gets hotter. §1.7
- **Time slice** — The short period a thread runs before the scheduler may switch the core to another thread. §16.13
- **Time zone** — A region's rules for its offset from UTC, including daylight saving, such as `Europe/Paris`. §7.10
- **TimSort** — The stable merge sort used by `Arrays.sort` and `List.sort` for objects; very fast on partly sorted data. §19.5
- **TLAB** — Thread-local allocation buffer: a private chunk of eden from which one thread allocates without locking. §16.5

- **TLB** — Translation lookaside buffer: the CPU's cache of virtual-to-physical address translations. §16.13
- **TLS** — Transport Layer Security: the encryption behind HTTPS. §17.10
- **Toolchain** — A build-tool setting that picks the exact JDK used to compile and test, independent of the JDK running the build. §18.3
- **Topological sort** — Ordering a DAG's nodes so every edge points forward — prerequisites before the tasks that need them. §19.17
- **`toString`** — Returns a text representation; `Object`'s default is `ClassName@hash`. §6.13
- **Transaction** — A group of database changes that succeed or fail together (`commit` or `rollback`). §18.11
- **Transitive dependency** — A library you get indirectly because one of your dependencies needs it. §18.1, §18.2
- **Treeification** — `HashMap` turning a bucket of more than 8 entries into a small red-black tree (Java 8+, once the table has ≥ 64 buckets). §10.5
- **`TreeMap` / `TreeSet`** — Red-black-tree map/set kept in sorted order, O(log n), with navigation methods such as `floorKey`. §10.4, §10.3
- **Trie** — A prefix tree that stores strings character by character, so prefix lookups cost O(length of the prefix). §19.21
- **try-with-resources** — `try (Resource r = …)`, which closes every declared `AutoCloseable` automatically, in reverse order. §8.6
- **Two pointers** — Two indexes or references ("pointers", such as `left` and `right`) moving through data in a coordinated way, so one pass replaces a nested loop. §19.8
- **Two's complement** — How Java stores signed integers: the top bit is the sign and −x equals `~x + 1`. §2.10, §19.26
- **Type erasure (erasure)** — Compiling generics by checking types and then removing type arguments, inserting casts. §9.7
- **Type inference** — The compiler deducing types (`var`, diamond, generic method arguments, lambda parameters). §2.6, §9.4
- **Type parameter** — A placeholder type such as `T` in `class Box<T>`. §9.2
- **Type pattern** — A pattern that tests a type and binds a variable: `instanceof String s`, `case Integer i`. §3.5
- **Type witness** — An explicit type argument written before a generic method's name, e.g. `Comparator.<int[]>comparingInt(...)`. §9.4, §19.5

**U**

- **Unchecked exception** — `RuntimeException`, `Error` and their subclasses — not checked by the compiler. §8.2
- **Unchecked warning** — A compiler warning that a generic operation can't be type-checked (raw types, unchecked casts). §9.8
- **`UncheckedIOException`** — An unchecked wrapper for `IOException`, used inside lambdas and streams. §13.4
- **Unicode** — The universal character set; Java strings use the UTF-16 encoding of it. §7.2
- **Union-find (disjoint set union)** — A structure that tracks which elements belong to the same group and merges groups in nearly O(1) per operation. §19.18
- **Unit test** — A fast test of one small piece of code — a method or a class — in isolation. §18.4, §18.9
- **Unmodifiable collection** — A collection whose mutators throw `UnsupportedOperationException`, such as `List.of`. §10.10
- **Unnamed module** — The module that holds everything loaded from the class path. §16.12
- **Unnamed variable `_`** — An underscore for a variable or pattern you don't use (Java 22). §3.7, §17.7
- **`UnsupportedOperationException`** — Thrown by an operation a collection or object doesn't support. §10.10
- **UTC** — Coordinated Universal Time: the world reference time, with no time-zone or daylight-saving shifts. §7.10
- **UTF-16** — The encoding Java strings use: each character is one or two 16-bit `char`s. §2.3, §7.2
- **UTF-8** — A variable-length encoding using 1–4 bytes per character; the Java default charset since 18. §13.7

**V**

- **Value object** — A small immutable object defined only by its values (an amount of money, a date range); records fit well. §6.8, §18.6
- `var` **(local variable type inference)** — Java 10: the compiler infers a local variable's type from its initializer. §2.6
- **Varargs** — A parameter `T... values` accepting zero or more arguments as an array. §4.4
- **Vector API** — An incubating API for SIMD calculations — one instruction applied to many numbers at once. §17.11
- **Virtual memory** — Each process sees its own large address space, which the OS maps onto physical RAM page by page. §16.13
- **Virtual thread** — A lightweight thread (Java 21) scheduled by the JVM onto carrier threads; cheap enough to create one per task. §15.4
- **Visibility (memory)** — Whether one thread's writes are guaranteed to be seen by another. §14.7, §15.8
- **VisualVM / Eclipse MAT / async-profiler** — Free diagnostic tools: VisualVM monitors a running JVM, Eclipse MAT analyses heap dumps, async-profiler samples CPU and allocations into flame graphs. §16.10
- `volatile` — A field modifier guaranteeing visibility and ordering of reads and writes, but not atomic compound actions. §14.7

### W

- `wait` / `notify` — `Object` methods for guarded blocks; must be called holding the object's monitor. §14.8
- **Wait set** — The threads waiting in `wait()` on an object's monitor until they are notified. §14.8
- **Warm-up** — The start-up period while the JVM loads classes and the JIT compiles hot code; timings taken then are misleadingly slow. §16.9
- `WatchService` — The NIO.2 API that reports files being created, changed or deleted in a directory. §13.9
- **Weakly consistent iterator** — A concurrent-collection iterator that never throws CME and may or may not show concurrent updates. §14.12
- **Week-based year** — `YYYY` in a date pattern: the year a week belongs to, which differs from the calendar year `yyyy` around New Year. §7.11
- **Widening conversion** — Automatic conversion to a type with a larger range (`int` → `long` → `double`). §2.7
- **Wildcard** — `?` in a generic type: `List<?>`, `List<? extends T>`, `List<? super T>`. §9.6
- **Work stealing** — Idle threads taking tasks from the back of busy threads' queues, as `ForkJoinPool` does. §15.3
- **Wrapper class** — The object form of a primitive: `Integer`, `Long`, `Double`, `Character`, `Boolean`, … §7.6

### X

- **XML** — Extensible Markup Language: a tag-based text format, used for example by Maven's `pom.xml`. §18.2

### Y

- `yield` — Returns a value from a block inside a `switch` expression. §3.3
- **Young generation** — The heap area (eden plus survivor spaces) where new objects are allocated; collected often and cheaply. §16.5

### Z

- **Zero-copy** — Moving data between files and sockets without copying it through the Java heap (e.g. `transferTo`). §13.9
- **ZGC** — A scalable low-latency collector with sub-millisecond pauses; generational-only since Java 24. §16.5
- `ZonedDateTime` — A date-time with a time-zone region, aware of daylight-saving rules. §7.10

---

## Chapter TL;DR

- **Types**: eight primitives with fixed sizes (`int` $\pm 2.1 \times 10^{9}$, `long` $\pm 9.2 \times 10^{18}$, `char` 0–65 535); integer overflow wraps silently; `0.1 + 0.2 != 0.3`; money in `BigDecimal` built from strings.
- **Equality**: `==` compares references for objects; use `equals` — the `Integer` cache (−128…127) and the string pool only make `==` *look* right.
- **Modern syntax**: switch expressions, pattern matching with guards, record patterns and `_`, sealed types for exhaustive switches, records for data, `var` for locals, text blocks.
- **Collections**: `ArrayList` and `HashMap` by default, `ArrayDeque` for stacks and queues, `TreeMap` for order and ranges, `LinkedHashMap` for LRU; `List.of` is unmodifiable and rejects `null`.
- **Functional and streams**: `Supplier`/`Consumer`/`Predicate`/`Function`; streams are lazy and one-shot; `groupingBy`, `partitioningBy`, `toMap` (duplicates throw); gatherers in 24; `Optional` only as a return type.
- **Exceptions**: checked = recoverable and declared, unchecked = bugs; try-with-resources closes in reverse and keeps close failures as suppressed; never `return` from `finally`.
- **Concurrency**: `synchronized`/locks for mutual exclusion, `volatile` for visibility only, atomics and `ConcurrentHashMap` for counters and caches, `CompletableFuture` for composition, virtual threads for blocking I/O at scale; happens-before decides visibility.
- **JVM**: heap, stacks, metaspace, code cache; G1 by default, ZGC for low latency; `-Xmx`, `-Xlog:gc*`, heap dumps on OOM; `jcmd`, JFR, `jstack` to diagnose.
- **Versions**: 17 sealed · 21 virtual threads, pattern `switch`, record patterns, sequenced collections · 22 `_`, FFM · 24 gatherers · 25 compact source files, module imports, flexible constructors, scoped values. Structured concurrency is still preview.
- **Tools**: `mvn clean verify` / `./gradlew build`, JUnit + Mockito + AssertJ, SLF4J for logging, `PreparedStatement` for SQL.
- **Next**: Spring Boot (or Jakarta EE, Quarkus), Kotlin if you like, and the LLD, SQL and DDIA notes — then build something real.

# Chapter 21 — Interview Question Bank

## How to use this bank

This chapter collects the Java questions that come up again and again in real interviews — gathered from big public question lists (GitHub collections, LeetCode Discuss interview-experience posts, Java 8 stream-coding lists) — and worded the way interviewers usually say them.

- ★ marks a question that appears in many sources: expect it.
- Each answer is short enough to **say out loud** in under a minute. Where this book teaches the topic in full, a → **More** link takes you there. Chapter 19 §19.29 already answers 42 core questions in more depth; those entries here are deliberately brief and link to it.
- Questions the rest of the book does not cover still get a complete short answer here, so nothing is missing.
- Every "what does this print" output and every coding answer was run on a real JDK; the printed results are real.
- **How to drill:** cover the answer, say yours out loud, then uncover and compare. Mark the ones you missed and repeat them the next day.

Versions: the book targets **Java 25 LTS** (LTS = long-term support release). Features are tagged with the Java version that made them final.

## 1. Platform and language basics

**Q1. ★ What is the difference between JDK, JRE and JVM?** The **JVM** (Java Virtual Machine) runs bytecode. The **JRE** (Java Runtime Environment) was JVM + standard libraries, enough to run programs. The **JDK** (Java Development Kit) adds the compiler `javac` and tools (`jshell`, `jcmd`, `jlink`…). Since Java 11 Oracle no longer ships a separate JRE (some vendors, such as Eclipse Temurin, still offer JRE builds); you can build a trimmed runtime for your app with `jlink`. → *More:* §1.3 JDK, JRE and JVM

**Q2. Why is Java platform independent? Is the JVM platform independent?** `javac` compiles source to **bytecode** (`.class` files), a portable instruction set; any JVM on any OS runs the same bytecode ("write once, run anywhere"). The JVM itself is *not* platform independent — each OS/CPU pair has its own JVM build. → *More:* §1.6 Bytecode and class files

**Q3. Is Java a pure object-oriented language?** No. It has eight **primitive** types (`int`, `double`…) that are not objects, and `static` members that belong to no object. Everything else is an object, and wrappers (`Integer`) let primitives act as objects.

**Q4. ★ What are the primitive types and their sizes?** `byte` 8 bits, `short` 16, `char` 16 (unsigned UTF-16 code unit), `int` 32, `long` 64, `float` 32, `double` 64, `boolean` (true/false, size not specified). Integer types are signed two's complement. Default field values are `0`, `0.0`, `'\u0000'`, `false`; local variables have no default and must be assigned before use. → *More:* §2.3 Primitive types

**Q5. Primitive vs reference types?** A primitive variable holds the value itself; a reference variable holds a pointer to an object on the heap (or `null`). Copying a reference copies the pointer, so both variables see the same object. Primitives can't be `null` and can't be used as generic type arguments. → *More:* §2.5 Reference types and null

**Q6. ★ What are wrapper classes and autoboxing? What can go wrong?** Each primitive has a wrapper object (`Integer`, `Long`, `Character`…). **Autoboxing** converts `int → Integer` automatically; **unboxing** does the reverse. Pitfalls: unboxing `null` throws `NullPointerException`; `==` on wrappers compares references (works by luck inside the −128…127 cache); boxing in hot loops allocates objects. → *More:* §7.6 Wrapper classes and autoboxing

**Q7. ★ Is Java pass-by-value or pass-by-reference?** Always pass-by-value; for objects the value copied is the reference, so a method can mutate the caller's object but not make the caller's variable point elsewhere. → *More:* §4.2 · §19.29 Q6

**Q8. Widening vs narrowing conversion; what is casting?** **Widening** (`int → long → double`) is automatic and safe (except precision loss for big `long → float/double`). **Narrowing** (`double → int`, `int → byte`) needs an explicit **cast** `(int) 3.99` and may truncate or overflow: `(byte) 200` is `-56`. In expressions, `byte`/`short`/`char` are promoted to `int`, which is why `short s = 1; s = s + 1;` doesn't compile but `s += 1` does (compound assignment casts implicitly). → *More:* §2.7 Type conversion

**Q9. What does `var` do, and where can't you use it?** `var` (Java 10) lets the compiler infer a **local** variable's type from its initializer; the variable is still statically typed. Not allowed for fields, method parameters, return types, without an initializer, or with a bare `null` initializer. Use it when the type is obvious from the right-hand side. → *More:* §2.6 var

**Q10. ★ What does `static` mean? Can a static method use instance members?** A static member belongs to the class, one copy shared by all instances. A static method has no `this`, so it can't use instance fields or methods directly (it needs an object reference). Static blocks run once when the class is initialised. → *More:* §5.5 Static members · §19.29 Q41

**Q11. ★ Explain `final` vs `finally` vs `finalize`.** `final` = can't reassign / override / extend; `finally` = block that always runs after `try`; `finalize()` = deprecated-for-removal GC callback (JEP 421, Java 18) — use try-with-resources or `Cleaner`. → *More:* §5.8 final · §19.29 Q7

**Q12. What are the access modifiers?** `private` (same class only), *package-private* (no keyword: same package), `protected` (same package plus subclasses in other packages), `public` (everyone). Top-level classes can only be `public` or package-private. Since Java 9, modules can additionally hide whole packages. → *More:* §5.6 Access modifiers

**Q13. Why is `main` `public static void main(String[] args)`? What changed in Java 25?** The launcher must call it without creating an object (`static`), from outside the package (`public`), and expects no result (`void`). Java 25 (JEP 512) allows **instance main methods** and **compact source files**: a file containing just `void main() { IO.println("Hi"); }` runs with `java Hello.java`. → *More:* §4.9 main and simpler main

**Q14. What is the difference between `this` and `super`?** `this` refers to the current object (disambiguate fields, call another constructor with `this(...)`, pass yourself). `super` reaches the parent's version of a member (`super.toString()`) or calls a parent constructor with `super(...)`. Before Java 25, `this(...)`/`super(...)` had to be the first statement; Java 25's flexible constructor bodies (JEP 513) allow statements before it (validation, computing arguments) as long as they don't read fields or call instance methods; the only use of `this` allowed there is assigning the class's own fields that have no initializer. → *More:* §6.1 Inheritance and super

**Q15. What happens on integer overflow? How do you detect it?** It silently wraps around: `Integer.MAX_VALUE + 1 == Integer.MIN_VALUE`. Use `Math.addExact`/`multiplyExact` (throw `ArithmeticException`), a wider type (`long`), or `BigInteger`. → *More:* §2.10 Integer overflow

**Q16. Why shouldn't you use `double` for money? What do you use instead?** Binary floating point can't represent most decimal fractions exactly (`0.1 + 0.2` prints `0.30000000000000004`). Use `BigDecimal` created from a **string** or `BigDecimal.valueOf(double)` (not `new BigDecimal(0.1)`, which captures the binary error), with an explicit `RoundingMode`, or store amounts as `long` cents. → *More:* §7.7 BigInteger and BigDecimal

## 2. OOP: classes, inheritance and polymorphism

**Q17. ★ What are the four principles of OOP? Encapsulation** — hide state behind methods so invariants hold. **Abstraction** — expose *what* an object does, not *how* (interfaces, abstract classes). **Inheritance** — a subclass reuses and specialises a parent. **Polymorphism** — one call, different behaviour depending on the object's run-time class. → *More:* §5.7 Encapsulation · LLD Handbook

**Q18. ★ Abstraction vs encapsulation?** Abstraction is a *design* idea: model only the essential behaviour and hide complexity behind a simple contract. Encapsulation is the *mechanism*: private fields, controlled access through methods. Abstraction decides what to show; encapsulation enforces what to hide.

**Q19. ★ What is polymorphism? Compile-time vs run-time?** Compile-time (static) polymorphism is **overloading**, resolved by the compiler from argument types. Run-time (dynamic) polymorphism is **overriding**: `Animal a = new Dog(); a.speak()` runs `Dog.speak()` because the JVM dispatches on the actual object's class. → *More:* §6.4 Polymorphism and casting

**Q20. ★ Overloading vs overriding?** Overloading: same name, different parameters, chosen at compile time. Overriding: same signature in a subclass, chosen at run time; can't reduce visibility, can't throw broader checked exceptions, may return a subtype (**covariant return**). → *More:* §6.3 Overriding · §19.29 Q12

**Q21. ★ Can you override a static method? A private method? Can you overload `main`?** Static: no — a same-signature static method in a subclass **hides** the parent's; which runs depends on the reference's declared type. Private: no — it isn't inherited, so a same-named subclass method is unrelated. `main`: yes, you can overload it, but the launcher only calls the standard signature. → *More:* §6.2 Members, hiding and Object

**Q22. ★ Why doesn't Java support multiple inheritance of classes? How do you get it?** Inheriting state and implementations from two parents creates the "diamond problem" (which parent's field or method wins?). Java allows one superclass but many interfaces. If two interfaces provide the same `default` method, the class must override it and may pick one with `A.super.method()`. → *More:* §6.6 Interfaces

**Q23. ★ Interface vs abstract class (and what changed in Java 8/9)?** Interfaces: multiple implementation, no instance state, `default`/`static` methods (8) and `private` methods (9). Abstract class: single inheritance, fields, constructors. Interface for a capability, abstract class for shared state among related classes. → *More:* §6.5 · §19.29 Q9

**Q24. Can an abstract class have a constructor? Can it have no abstract methods?** Yes to both. The constructor runs when a subclass is created (via `super(...)`), initialising shared state. A class can be `abstract` with zero abstract methods simply to forbid direct instantiation.

**Q25. ★ Composition vs inheritance — which do you prefer?** Inheritance ("is-a") couples the subclass to the parent's implementation details; a change in the parent can break it (the "fragile base class" problem). Composition ("has-a") holds another object in a field and delegates to it, so you can swap implementations and expose only what you choose. Prefer composition; use inheritance only for a true is-a relation designed for extension. → *More:* LLD Handbook

**Q26. Association vs aggregation vs composition?** **Association**: objects know each other. **Aggregation**: a "has-a" where the part can live without the whole (a team has players). **Composition**: the part's lifetime is owned by the whole (an order owns its order lines).

**Q27. What is a constructor? Default constructor? Constructor chaining?** A constructor initialises a new object; it has the class name and no return type. If you write none, the compiler adds a no-arg **default constructor**; if you write any, it doesn't. Chaining: `this(...)` calls another constructor of the same class, `super(...)` a parent one; if neither is written, the compiler inserts `super()`, so a parent without a no-arg constructor causes a compile error. → *More:* §5.3 Constructors

**Q28. What is a private constructor used for?** To stop outside instantiation: utility classes with only static methods, singletons, and classes that hand out instances through static factory methods (`List.of`, `Integer.valueOf`).

**Q29. What is the initialisation order of a class?** Once per class: static field initialisers and static blocks, top to bottom (parent class first). Per object: parent constructor chain first, then the child's instance field initialisers and instance blocks in order, then the rest of the child constructor body. So a parent constructor that calls an overridden method sees the child's fields still at default values (see Q218). → *More:* §5.10 Initialisation order

**Q30. What methods does `Object` have?** `equals`, `hashCode`, `toString`, `getClass`, `clone` (protected), `wait`/`notify`/`notifyAll` (thread coordination on the object's monitor), and the deprecated `finalize`. Every class inherits them. → *More:* §6.13 equals, hashCode and toString

**Q31. What is an enum, and why is it better than int constants?** An `enum` is a class with a fixed set of instances: type-safe (you can't pass 7 where a `Day` is expected), can have fields, constructors, methods and per-constant behaviour, works in `switch`, and comes with `values()`, `valueOf()`, `EnumMap`, `EnumSet`. Enum constants are singletons, safe even across serialization. → *More:* §6.9 Enums

**Q32. Static nested vs inner class vs anonymous class?** A **static nested** class has no link to an outer instance (prefer it). An **inner** (non-static) class holds a hidden reference to its outer object — it can use the outer fields but can leak memory if it outlives the outer. An **anonymous** class is an unnamed one-off implementation, mostly replaced by lambdas for single-method interfaces. → *More:* §6.10 Nested classes

**Q33. What is a marker interface? Give examples.** An interface with no methods that tags a class for special treatment: `Serializable`, `Cloneable`, `RandomAccess`. Today annotations usually do this job, but a marker interface is a real type, so the compiler can check it.

**Q34. What is a covariant return type?** An overriding method may declare a *subtype* of the parent's return type, e.g. `Animal reproduce()` overridden as `Dog reproduce()`, which saves callers a cast.

**Q35. What is upcasting and downcasting? When do you get `ClassCastException`?** Upcasting (`Animal a = dog`) is implicit and always safe. Downcasting (`(Dog) a`) is explicit and checked at run time; if the object isn't really a `Dog` it throws `ClassCastException`. Guard with pattern matching: `if (a instanceof Dog d) d.bark();`. → *More:* §3.5 Pattern matching for instanceof

**Q36. What are coupling and cohesion? Coupling** is how much one class depends on another's details — aim low. **Cohesion** is how focused a class is on one responsibility — aim high. Low coupling plus high cohesion makes code easier to change and test.

**Q37. ★ What are the SOLID principles? S**ingle responsibility (one reason to change), **O**pen/closed (extend without editing), **L**iskov substitution (subtypes must work wherever the parent is expected), **I**nterface segregation (small focused interfaces), **D**ependency inversion (depend on abstractions, inject concrete classes). → *More:* LLD Handbook

**Q38. ★ How do you write a thread-safe singleton?** Best: an `enum` with one constant (serialization- and reflection-safe), or the **holder idiom** (a nested static class holds the instance; the JVM's class initialisation makes it lazy and thread-safe). Double-checked locking also works but the field **must** be `volatile`. Code in Q248.

## 3. Strings

**Q39. ★ Why is `String` immutable?** Safe sharing between threads, safe as a hash key (hash cached), poolable, and secure for values that are checked then used (paths, class names). → *More:* §7.1 · §19.29 Q1

**Q40. ★ What is the String pool? How many objects does `new String("hi")` create?** A JVM table (on the heap since Java 7) with one shared instance per literal; `new String("hi")` makes one new object plus the pooled literal if not already there — "one or two". `intern()` returns the pooled copy. → *More:* §7.1 · §19.29 Q2

**Q41. ★ `String` vs `StringBuilder` vs `StringBuffer`?** Immutable vs mutable-unsynchronised vs mutable-synchronised (legacy). Build strings in loops with `StringBuilder`. → *More:* §7.3 · §19.29 Q3

**Q42. ★ `==` vs `equals()` for strings?** `==` asks "same object?", `equals` asks "same characters?". Literals happen to share a pooled object, so `==` sometimes "works" — a classic trap; always use `equals` (or `"x".equals(var)` to be null-safe). See Q210. → *More:* §7.1 Strings and the pool

**Q43. Why is `+=` on a `String` in a loop slow?** Each `+=` copies the whole string into a new object, so n appends cost $O(n^2)$ character copies. Outside loops, `+` is fine: since Java 9 the compiler emits an `invokedynamic` call to `StringConcatFactory`, which builds the result efficiently. → *More:* §7.3 StringBuilder

**Q44. How is a `String` stored internally?** Since Java 9 (**compact strings**, JEP 254), as a `byte[]` plus a coder flag: Latin-1 strings use one byte per character, others UTF-16 (two bytes). The hash code is computed lazily and cached in a field. → *More:* §7.1 Strings and the pool

**Q45. Why prefer `char[]` over `String` for passwords?** A `String` can't be wiped and may linger in memory (and in the pool) until GC, where heap dumps can expose it; a `char[]` can be overwritten with `Arrays.fill(pw, '\0')` after use. That's why `Console.readPassword()` returns `char[]`.

**Q46. `trim()` vs `strip()`; what else did Java 11 add?** `trim()` removes characters ≤ U+0020 (ASCII spaces and control chars); `strip()` (11) removes all Unicode whitespace. Java 11 also added `isBlank()`, `lines()`, `repeat(n)`, `stripLeading/Trailing()`. → *More:* §7.2 The String API

**Q47. What does `intern()` do, and when would you use it?** Returns the canonical pooled instance with the same content, adding it to the pool if absent. Rarely needed: it can save memory when you hold millions of duplicate strings, but `-XX:+UseStringDeduplication` (G1 since Java 8u20, the other collectors since Java 18) usually does that for you.

**Q48. What are text blocks?** Multi-line string literals (Java 15) delimited by `"""`; incidental indentation is stripped, so JSON/SQL/HTML can be pasted readably. They are ordinary `String`s. → *More:* §2.12 Strings and text blocks

**Q49. How do you split, join and format strings?** `"a,b".split(",")` (regex; trailing empty strings dropped), `String.join(",", list)`, `Collectors.joining(", ", "[", "]")`, `StringJoiner`, and `"%s=%d".formatted(k, v)` (Java 15) or `String.format`. → *More:* §7.4 Formatting

**Q50. Is `String` thread-safe? Is `StringBuilder`?** `String` yes (immutable). `StringBuilder` no — don't share one between threads; give each thread its own, or use `StringBuffer` in the rare case one buffer really is shared.

## 4. equals, hashCode, immutability and cloning

**Q51. ★ What is the `equals`/`hashCode` contract? What breaks if you override only `equals`?** Equal objects must have equal hash codes; override only `equals` and `HashSet`/`HashMap` stop finding "equal" objects because they hash to different buckets (Q224 shows it). → *More:* §6.13 · §19.29 Q4

**Q52. How do you write a correct `equals`?** Check identity (`this == o`), then type (`o instanceof Point p`, or `getClass() == o.getClass()` if subclasses must never equal the parent), then compare significant fields (`Objects.equals` for references, `==` for primitives except `Double.compare`/`Float.compare`). Make the class final or use `getClass()` to stay symmetric with subclasses. Build `hashCode` from the same fields: `Objects.hash(x, y)`. Records generate both. → *More:* §6.13 equals and hashCode

**Q53. Can two unequal objects have the same hash code?** Yes — that's a **collision** and is legal (`"Aa"` and `"BB"` both hash to 2112). Collisions only cost performance; the map falls back to `equals`. → *More:* §6.13 equals and hashCode

**Q54. ★ How do you make a class immutable?** `final` class, `private final` fields, no setters, defensive copies in and out (`List.copyOf`), no `this` escaping. Records cover most of it, shallowly. → *More:* §5.9 Immutability basics · §19.29 Q8

**Q55. What are the benefits of immutable objects?** Thread-safe without locks, safe as map keys and set elements, can be cached and shared freely, easier to reason about (no state changes after construction), and failure-atomic (an exception can't leave them half-updated). The cost is allocating a new object for every "change".

**Q56. Shallow copy vs deep copy?** A shallow copy duplicates the top object but shares the objects its fields point to; a deep copy recursively copies them too. `Object.clone()` and `new ArrayList<>(list)` are shallow.

**Q57. Why is `Cloneable`/`clone()` considered broken? What do you use instead?** `Cloneable` has no `clone` method (it just switches on `Object.clone`'s field-by-field copy), `clone` is protected, returns `Object`, bypasses constructors, is shallow, and clashes with `final` fields. Prefer a **copy constructor** (`new Point(other)`) or a static factory (`Point.copyOf(p)`).

**Q58. `Comparable` vs `Comparator`?** `compareTo` defines one natural order inside the class; `Comparator` objects define any number of external orders. Never compare by subtraction (overflow); use `Integer.compare` or `Comparator.comparing`. → *More:* §10.8 · §19.29 Q19

## 5. Exceptions

**Q59. ★ Explain the exception hierarchy.** `Throwable` has two branches: `Error` (serious JVM problems: `OutOfMemoryError`, `StackOverflowError` — don't catch) and `Exception`. Under `Exception`, `RuntimeException` and its subclasses are **unchecked**; all other exceptions are **checked**. → *More:* §8.2 The Throwable hierarchy

**Q60. ★ Checked vs unchecked exceptions?** Checked must be caught or declared (recoverable conditions like `IOException`); unchecked signal bugs (`NullPointerException`, `IllegalArgumentException`) and needn't be declared. → *More:* §8.3 · §19.29 Q23

**Q61. Error vs Exception?** An `Error` means the JVM or environment is in trouble (out of memory, stack overflow, class-linkage failure); the application usually can't recover, so you let it propagate. An `Exception` is a condition the program can reasonably handle. → *More:* §8.2 The Throwable hierarchy

**Q62. `throw` vs `throws`?** `throw` is a statement that throws one exception object now: `throw new IllegalStateException("closed")`. `throws` is part of a method signature listing checked exceptions the method may pass to its caller. → *More:* §8.7 Throwing exceptions

**Q63. ★ How does try-with-resources work?** Closes every `AutoCloseable` declared in the header in reverse order; a close failure after a body failure is attached as a suppressed exception (Q228 shows the order). → *More:* §8.6 · §19.29 Q24

**Q64. ⋆ When does `finally` not run? What if both `try` and `finally` return?** Only on `System.exit`, JVM crash/kill, or a thread that never leaves the `try`. A `return` in `finally` overrides the `try`'s return and swallows any exception — never do it (Q212). → *More:* §8.4 · §19.29 Q25

**Q65. Is `try` without `catch` allowed?** Yes: `try`–`finally`, or try-with-resources with no `catch` at all. A `try` alone, with neither `catch` nor `finally` nor resources, doesn't compile. → *More:* §8.4 try/catch/finally

**Q66. Why must `catch` blocks go from specific to general? What is multi-catch?** A `catch (Exception e)` before `catch (IOException e)` makes the second unreachable — a compile error. Multi-catch `catch (IOException | SQLException e)` handles unrelated types in one block; the variable is implicitly final, and the alternatives can't be subclasses of each other. → *More:* §8.5 Multi-catch

**Q67. How do you create a custom exception? Checked or unchecked?** Extend `RuntimeException` (unchecked, the common modern choice) or `Exception` (checked, when callers must handle it), and provide constructors that take a message and a **cause** so the original exception is kept (`super(msg, cause)`). → *More:* §8.7 Custom exceptions

**Q68. Can an overriding method throw a broader checked exception?** No. It may throw the same checked exceptions, narrower ones, or none, and any unchecked exceptions — otherwise callers using the parent type would face exceptions they didn't expect. → *More:* §6.3 Overriding

**Q69. `ClassNotFoundException` vs `NoClassDefFoundError`?** `ClassNotFoundException` is a checked exception from code that loads a class *by name* (`Class.forName`, a class loader) and can't find it. `NoClassDefFoundError` is an `Error`: a class that was present at compile time is missing at run time, or its static initialiser failed earlier.

**Q70. What is exception chaining, and why does it matter?** Wrapping a low-level exception in a higher-level one while keeping it as the **cause**: `throw new OrderException("save failed", e)`. The stack trace then shows "Caused by: …", so you don't lose the root cause. → *More:* §8.8 Chaining

**Q71. What are exception-handling best practices?** Don't swallow exceptions (empty `catch`); catch only what you can handle; keep the cause; log *or* rethrow, not both; prefer specific exceptions; validate arguments early (`Objects.requireNonNull`); don't use exceptions for normal control flow; clean up with try-with-resources. → *More:* §8.10 Best practices

## 6. Generics

**Q72. Why use generics?** Compile-time type safety (a `List<String>` can't receive an `Integer`), no casts on read, and reusable algorithms that work for many types. → *More:* §9.1 Why generics

**Q73. ⋆ What is type erasure?** Generic types are checked by the compiler, then erased to their bounds with casts inserted; at run time `List<String>` and `List<Integer>` are the same class. So no `new T()`, no `new T[]`, no `instanceof List<String>`, no `List<int>`. → *More:* §9.7 · §19.29 Q26

**Q74. ⋆ Explain `? extends T` vs `? super T` (PECS).** Producer `extends` (read as `T`, can't add), consumer `super` (can add `T`, reads give `Object`). → *More:* §9.6 · §19.29 Q27

**Q75. Is `List<Integer>` a subtype of `List<Number>`?** No — generics are **invariant**. Otherwise you could add a `Double` to a list of integers through the `List<Number>` reference. Use `List<? extends Number>` to accept both read-only. Arrays, by contrast, are covariant and fail at run time with `ArrayStoreException`. → *More:* §9.6 Invariance and wildcards

**Q76. What is a bounded type parameter?** `<T extends Comparable<T>> T max(List<T> xs)` restricts `T` to types that have `compareTo`, so the method can call it. Multiple bounds: `<T extends Number & Comparable<T>>` (class first). → *More:* §9.5 Bounded type parameters

**Q77. What is a raw type, and why avoid it?** Using a generic class without type arguments (`List list`) — kept for pre-Java-5 compatibility. It turns off type checks, so a wrong element blows up later with a `ClassCastException` far from the bug. → *More:* §9.8 Erasure restrictions

**Q78. Can you overload `void f(List<String>)` and `void f(List<Integer>)`?** No. After erasure both become `f(List)` — same signature, compile error ("name clash"). → *More:* §9.8 Erasure restrictions

## 7. Collections

**Q79. Explain the Collections Framework hierarchy.** `Iterable` → `Collection` → `List` (ordered, indexed: `ArrayList`, `LinkedList`), `Set` (no duplicates: `HashSet`, `LinkedHashSet`, `TreeSet`), `Queue`/`Deque` (`ArrayDeque`, `PriorityQueue`). `Map` (`HashMap`, `LinkedHashMap`, `TreeMap`) is separate because it stores key–value pairs, not single elements. Java 21 added `SequencedCollection`/`SequencedMap` for types with a defined order. → *More:* §10.1 The hierarchy

**Q80. ★ How does `HashMap` work internally?** Bucket array (16, load factor 0.75), index = spread hash `& (n−1)`, collisions chained, bucket becomes a red-black tree beyond 8 entries (when the table has ≥ 64 buckets), table doubles when size > capacity × load factor; lookup compares hash then `equals`. → *More:* §10.5 HashMap internals · §19.29 Q13

**Q81. Why is `HashMap` capacity a power of two? What does "spreading" the hash do?** With n a power of two, `hash & (n − 1)` is a cheap replacement for `hash % n`. Because that only uses the low bits, `HashMap` XORs the high 16 bits into the low 16 (`h ^ (h >>> 16)`) so keys whose hash codes differ only in high bits still land in different buckets. On resize each entry either stays at index *i* or moves to *i + oldCapacity*. → *More:* §10.5 HashMap internals

**Q82. What happens when two keys have the same hash code?** They go to the same bucket; `put` walks the bucket and uses `equals` to decide "same key → replace value" or "different key → add a node". `get` does the same. Many collisions degrade a bucket from O(1) to O(n) (O(log n) once treeified). → *More:* §10.5 HashMap internals

**Q83. Can `HashMap` have a null key? `Hashtable`? `ConcurrentHashMap`? `TreeMap`?** `HashMap`: one null key (stored in bucket 0) and null values. `Hashtable` and `ConcurrentHashMap`: no nulls at all. `TreeMap`: no null key with natural ordering (it calls `compareTo` → NPE), null values are fine. → *More:* §10.5 HashMap internals

**Q84. Why must map keys be immutable? What if you mutate a key?** The entry stays in the bucket of the old hash, so lookups miss it. → *More:* §10.5 · §19.29 Q14

**Q85. ★ `HashMap` vs `Hashtable` vs `synchronizedMap` vs `ConcurrentHashMap`?** Unsynchronised with nulls / legacy fully synchronised / one lock wrapper / fine-grained concurrent with atomic compound operations. → *More:* §14.12 Concurrent collections · §19.29 Q15

**Q86. ★ How does `ConcurrentHashMap` achieve thread safety? How did Java 7 differ?** Java 8+: lock-free reads, CAS into empty buckets, `synchronized` on a bucket's head node for updates, cooperative resizing; Java 7 used 16 lock "segments". Iterators are weakly consistent. → *More:* §14.12 · §19.29 Q16

**Q87. ★ Is `map.get(k) == null ? map.put(k, v) : …` safe on a `ConcurrentHashMap`?** No — it's a check-then-act race: two threads can both see `null` and both put. Use the atomic methods: `putIfAbsent`, `computeIfAbsent(k, key -> new ArrayList<>())`, `merge(word, 1, Integer::sum)`. Keep the functions short — they run while the bucket is locked. → *More:* §14.12 Concurrent collections

**Q88. ★ `ArrayList` vs `LinkedList`?** Array-backed, O(1) index access, cache-friendly vs doubly linked, O(1) only at ends or an iterator. Default to `ArrayList`. → *More:* §10.2 · §19.29 Q17

**Q89. How does `ArrayList` grow?** `new ArrayList<>()` starts with a shared empty array and allocates 10 slots on the first `add`. When full it grows to about 1.5× (`old + old/2`) and copies the elements. Each copy is O(n) but rare, so `add` is **amortised O(1)**. Pass an initial capacity if you know the size. → *More:* §10.2 Lists

**Q90. `ArrayList` vs `Vector`?** `Vector` is the legacy synchronised version (every method locked, grows 2×). Don't use it; use `ArrayList`, or `CopyOnWriteArrayList`/`Collections.synchronizedList` when shared. → *More:* §10.2 Lists

**Q91. `HashSet` vs `LinkedHashSet` vs `TreeSet`? How does `HashSet` work internally?** `HashSet` is unordered O(1); `LinkedHashSet` keeps insertion order; `TreeSet` keeps elements sorted, O(log n), and supports `floor`/`ceiling`/ranges. Internally a `HashSet` *is* a `HashMap` whose keys are the elements and whose values are one shared dummy object; `add` returns `false` if the key was already there. → *More:* §10.3 Sets

**Q92. ★ `HashMap` vs `LinkedHashMap` vs `TreeMap`? How is `TreeMap` implemented?** Unordered / insertion- or access-ordered / sorted. `TreeMap` is a **red-black tree** (a self-balancing binary search tree), so `get`/`put`/`remove` are O(log n) and it offers `floorKey`, `ceilingKey`, `headMap`, `tailMap`, `firstKey`. It uses `compareTo`/`Comparator`, not `equals`, to detect duplicates. → *More:* §10.4 Maps · §19.29 Q18

**Q93. ★ Fail-fast vs fail-safe iterators? How do you remove while iterating?** Fail-fast iterators throw `ConcurrentModificationException` on structural change through another path (tracked by `modCount`); concurrent collections' iterators are weakly consistent or snapshot-based. Remove with `Iterator.remove()` or `removeIf`. → *More:* §10.7 · §19.29 Q20

**Q94. `Comparable` vs `Comparator` — how do you sort a list of employees by salary, then name?** `list.sort(Comparator.comparingDouble(Employee::salary).thenComparing(Employee::name))`; add `.reversed()` for descending. `Collections.sort` and `List.sort` use a stable merge sort (TimSort) for objects. → *More:* §10.8

**Q95. `List.of` vs `Arrays.asList` vs `Collections.unmodifiableList`?** Truly unmodifiable and null-hostile / fixed-size view that writes through / read-only view of a list that may still change. → *More:* §10.10 · §19.29 Q22

**Q96. What is `CopyOnWriteArrayList` and when do you use it?** A thread-safe list that copies the whole underlying array on every write; reads and iteration take no lock and see a snapshot (no `ConcurrentModificationException`). Great for read-mostly data such as listener lists; terrible for frequent writes. → *More:* §14.12 Concurrent collections

**Q97. `Queue`, `Deque`, `PriorityQueue` — and why prefer `ArrayDeque` to `Stack`?** `Queue` is FIFO (`offer`/`poll`/`peek`), `Deque` adds both ends (stack or queue), `PriorityQueue` is a binary heap that always yields the smallest element (O(log n) `offer`/`poll`). `Stack` is synchronised legacy extending `Vector` (so it also exposes index access that breaks the stack abstraction); `ArrayDeque` is unsynchronised, faster, and gives `push`/`pop`/`peek` through the `Deque` interface. → *More:* §10.6 · §19.29 Q21

**Q98. What are `EnumMap`, `WeakHashMap` and `IdentityHashMap`?** `EnumMap`: a compact array indexed by enum ordinal — fastest map for enum keys. `WeakHashMap`: keys held by weak references, so an entry disappears once nothing else references the key (used for metadata caches). `IdentityHashMap`: compares keys with `==` instead of `equals` (graph traversal, serialization).

**Q99. What Java 8 methods did `Map` gain?** `getOrDefault`, `putIfAbsent`, `computeIfAbsent`, `computeIfPresent`, `compute`, `merge`, `forEach`, `replaceAll`, `remove(k, v)`. Word count in one line: `counts.merge(word, 1, Integer::sum)`. → *More:* §10.4 Maps

**Q100. What are sequenced collections (Java 21)?** New interfaces `SequencedCollection`, `SequencedSet`, `SequencedMap` give every ordered collection uniform `getFirst()`, `getLast()`, `addFirst/Last`, `removeFirst/Last` and `reversed()` — so `list.getLast()` replaces `list.get(list.size() - 1)`. → *More:* §10.11 Sequenced collections

**Q101. What is the time complexity of common collection operations?** `ArrayList`: get O(1), add-at-end amortised O(1), insert/remove middle O(n), `contains` O(n). `HashMap`/`HashSet`: O(1) average. `TreeMap`/`TreeSet`: O(log n). `PriorityQueue`: offer/poll O(log n), peek O(1). `ArrayDeque`: O(1) at both ends. → *More:* §10.12 Big-O cheat sheet · §19.7

**Q102. ★ How would you implement an LRU cache?** `LinkedHashMap` in access order with `removeEldestEntry` returning `size() > capacity` (Q249). Without it: a `HashMap` from key to node plus a doubly linked list — move a node to the front on access, evict from the tail; both O(1). Make it thread-safe with a lock, or use a library like Caffeine in production. → *More:* §10.4

## 8. Lambdas, functional interfaces and Optional

**Q103. ★ What are the main features of Java 8?** Lambdas and method references, functional interfaces (`java.util.function`), the Stream API, `Optional`, `default` and `static` interface methods, the `java.time` date/time API, `CompletableFuture`, new `Map` methods, and Metaspace replacing PermGen.

**Q104. What is a lambda expression?** A compact way to write an implementation of a functional interface: `(a, b) -> a + b`. It's not an anonymous class: it has no own `this` (it uses the enclosing one), and the JVM creates it through `invokedynamic`, usually without a new class file. → *More:* §11.2 Lambda syntax

**Q105. ★ What is a functional interface? Is `@FunctionalInterface` required?** An interface with exactly one abstract method (`default`/`static` methods and `Object`'s public methods don't count). The annotation is optional; it makes the compiler reject a second abstract method. → *More:* §11.2 · §19.29 Q30

**Q106. Name the core functional interfaces.** `Function<T,R>` (T → R), `BiFunction<T,U,R>`, `Supplier<T>` (() → T), `Consumer<T>` (T → nothing), `Predicate<T>` (T → boolean), `UnaryOperator<T>` (T → T), `BinaryOperator<T>` ((T,T) → T), plus primitive versions like `IntPredicate`, `ToIntFunction` that avoid boxing. → *More:* §11.3

**Q107. Supplier vs Consumer?** A `Supplier<T>` takes nothing and produces a value (`get()`), e.g. a lazy default in `orElseGet`. A `Consumer<T>` takes a value and returns nothing (`accept()`), e.g. the action in `forEach`. → *More:* §11.3 Built-in functional interfaces

**Q108. What are method references? Which kinds exist?** Shorthand for a lambda that only calls a method: static (`Integer::parseInt`), bound instance (`System.out::println`, `str::startsWith`), unbound instance (`String::length` — the first argument becomes the receiver), and constructor (`ArrayList::new`). → *More:* §11.4 Method references

**Q109. Why must variables used in a lambda be effectively final?** The lambda captures a copy of the value; allowing later reassignment would make the copy and the variable silently disagree, and lambdas may run on other threads. → *More:* §11.5

**Q110. ★ What is `Optional`, and how should you use it?** A container that holds one value or none, used as a **return type** for "maybe no result" instead of returning `null`. Consume it with `map`, `filter`, `orElse`, `orElseGet`, `orElseThrow`, `ifPresentOrElse`. `Optional.of(null)` throws NPE; use `ofNullable`. → *More:* §11.7 Optional

**Q111. ★ `orElse` vs `orElseGet`? Common misuses of `Optional`?** `orElse(x)` always evaluates `x`; `orElseGet(() -> x)` only when empty — matters for expensive defaults. Misuses: `get()` without a check, `Optional` fields/parameters, `Optional<List>`, returning `null` from an `Optional` method. → *More:* §11.7 · §19.29 Q29

**Q112. What are `default` methods for? Can a default method be overridden?** They let an interface gain new methods without breaking existing implementations — that's how `Collection.stream()` and `Iterable.forEach` were added in Java 8. Implementations can override them; static interface methods can't be overridden and are called as `Interface.method()`. → *More:* §6.6 Interfaces

## 9. Streams

**Q113. What is a stream? Collection vs Stream?** A stream is a one-shot pipeline of operations over a data source: it stores nothing, is lazy, doesn't modify its source, and can be consumed only once (a second terminal operation throws `IllegalStateException`). A collection stores elements and can be iterated many times. → *More:* §12.1 What a stream is

**Q114. ★ Intermediate vs terminal operations? What does "lazy" mean?** Intermediate operations (`filter`, `map`, `sorted`, `distinct`, `limit`) return a new stream and do nothing yet. A terminal operation (`collect`, `forEach`, `reduce`, `count`, `findFirst`) starts the work. Elements flow one at a time through the whole pipeline, and short-circuiting operations stop early (Q223). → *More:* §12.2 Laziness

**Q115. ★ `map` vs `flatMap`?** `map` turns each element into exactly one element (`Stream<List<Integer>>` stays a stream of lists). `flatMap` turns each element into a stream and concatenates them (`lists.stream().flatMap(List::stream)` → `Stream<Integer>`). → *More:* §12.3 Intermediate operations

**Q116. How do you create a stream?** `collection.stream()`, `Stream.of(a, b)`, `Arrays.stream(arr)`, `IntStream.range(0, n)`, `Stream.iterate(seed, next)`, `Stream.generate(supplier)`, `Files.lines(path)`, `string.chars()`, `Pattern.splitAsStream`. → *More:* §12.1 What a stream is

**Q117. What does `collect` do? Name the useful collectors.** `collect` is a mutable reduction into a container. Collectors: `toList()`, `toSet()`, `toMap(k, v, merge)`, `joining(sep)`, `groupingBy(classifier, downstream)`, `partitioningBy(predicate)`, `counting()`, `summingInt`, `averagingDouble`, `mapping`, `collectingAndThen`, `teeing` (Java 12). → *More:* §12.5 Collectors

**Q118. ★ `groupingBy` vs `partitioningBy`?** `groupingBy` splits into any number of groups by a key (`Map<K, List<T>>`); `partitioningBy` splits into exactly two by a predicate (`Map<Boolean, List<T>>`, both keys always present). → *More:* §12.5 Collectors

**Q119. `Collectors.toList()` vs `Stream.toList()`?** `stream.toList()` (Java 16) returns an **unmodifiable** list and allows nulls. `Collectors.toList()` currently returns an `ArrayList` but promises nothing about mutability. Use `Collectors.toCollection(ArrayList::new)` when you need to modify the result. → *More:* §12.5 Collectors

**Q120. ★ What happens with duplicate keys in `Collectors.toMap`?** It throws `IllegalStateException` ("Duplicate key"). Pass a merge function: `toMap(Emp::dept, Emp::salary, Double::sum)`; add a map supplier such as `LinkedHashMap::new` to control the map type. → *More:* §12.5 Collectors

**Q121. What does `reduce` do?** Combines all elements into one value with an associative function: `nums.stream().reduce(0, Integer::sum)`. Without an identity it returns `Optional<T>`. For sums prefer `mapToInt(...).sum()` (no boxing). → *More:* §12.4 Terminal operations and reduce

**Q122. `findFirst` vs `findAny`; `anyMatch`/`allMatch`/`noneMatch`?** `findFirst` respects encounter order; `findAny` may return any element (cheaper in parallel). The match operations short-circuit and return `boolean`; `allMatch` on an empty stream is `true`, `anyMatch` is `false`. → *More:* §12.4 Terminal operations

**Q123. ★ Are parallel streams always faster? When do you use them?** No. They split work over the common `ForkJoinPool`; worth it only for large, CPU-bound, easily splittable sources (arrays, `ArrayList`, ranges) with independent, stateless operations. Bad for I/O, small data, `LinkedList`/`iterate` sources, ordered operations like `limit`, and shared mutable state (adding to a shared `ArrayList` from `forEach` loses elements or throws). → *More:* §12.8 Parallel streams

**Q124. Why have `IntStream`, `LongStream`, `DoubleStream`?** To avoid boxing every number into an object. They add `sum`, `average`, `max`, `summaryStatistics`, `range`. Convert with `mapToInt`, back with `boxed()`. → *More:* §12.6 Primitive streams

**Q125. `forEach` vs `forEachOrdered`; `peek`?** In parallel streams `forEach` may process in any order; `forEachOrdered` keeps encounter order (slower). `peek` is for debugging only — it runs only if a terminal operation pulls elements through it. → *More:* §12.8 Parallel streams

**Q126. What are stream gatherers (Java 24)?** `Stream.gather(Gatherer)` lets you write custom intermediate operations, the way collectors are custom terminal ones. Built-ins in `Gatherers`: `windowFixed(3)`, `windowSliding(3)`, `fold`, `scan`, `mapConcurrent`. → *More:* §12.9 Stream gatherers

## 10. Modern Java (9 → 25)

**Q127. ★ What are the main features from Java 9 to 25?** 9: modules, `List.of`, JShell. 10: `var`. 11: `String` helpers, `HttpClient`, single-file launch. 14: switch expressions. 15: text blocks. 16: records, `instanceof` patterns, `Stream.toList`. 17: sealed classes. 21: virtual threads, pattern matching for `switch`, record patterns, sequenced collections, generational ZGC. 22: unnamed `_`, Foreign Function & Memory API. 24: gatherers, no `synchronized` pinning. 25: compact source files, module imports, flexible constructor bodies, scoped values. → *More:* §17.2 The version table · §19.29 Q42

**Q128. ★ What is a record? Can it have extra fields or extend a class?** A class whose whole job is to hold a fixed set of values: the compiler writes the constructor, the accessors (`name()`, not `getName()`), and `equals`/`hashCode`/`toString`. Implicitly final, extends `Record` (so it can't extend another class), fields are `private final` (shallowly immutable), no extra instance fields (static ones are fine); can implement interfaces and validate in a compact constructor. → *More:* §6.8 Records · §19.29 Q10

**Q129. Record vs Lombok `@Data` vs a normal class?** A record is a language feature: final, immutable, fixed shape, no setters, works with record patterns. Lombok `@Data` generates getters, setters, `equals`/`hashCode` for a mutable class through an annotation processor. Use a normal class (or Lombok) when you need mutability, inheritance or hidden state (e.g. JPA entities). → *More:* §6.8 Records

**Q130. ★ What are sealed classes and why use them?** `sealed ... permits A, B` limits subclasses; each must be `final`, `sealed` or `non-sealed`. The compiler then knows all cases, so `switch` over the type can be exhaustive without `default`. → *More:* §6.7 · §19.29 Q11

**Q131. ★ What is pattern matching for `instanceof` and `switch`?** `if (obj instanceof String s)` tests and binds in one step (Java 16). Switch patterns (Java 21) match on type, with guards and null handling:

```java
String describe(Object o) {
    return switch (o) {
        case null -> "null";
        case Integer i when i > 0 -> "positive int";
        case Integer i -> "int";
        case String s -> "string of " + s.length();
        default -> "other";
    };
}
```

→ *More:* §3.6 Pattern matching for switch

**Q132. What are record patterns?** Patterns that deconstruct a record in place (Java 21): `if (p instanceof Point(int x, int y))` or `case Circle(Point(var x, var y), double r) ->`. With sealed types they give algebraic-data-type style code. Java 22's unnamed pattern `_` skips components you don't need. → *More:* §3.7 Record patterns

**Q133. Switch statement vs switch expression?** The arrow form (`case A -> ...`) has no fall-through. A switch **expression** yields a value (`int n = switch (d) { ... };`), must be exhaustive, and uses `yield` inside a block. The old colon form falls through without `break`. → *More:* §3.3 switch expressions

**Q134. ★ What are virtual threads?** JVM-managed lightweight threads mounted on a few carrier threads; blocking I/O unmounts them, so millions of blocking tasks are cheap. Help I/O-bound concurrency, not CPU-bound speed. Don't pool them. → *More:* §15.4 Virtual threads · §19.29 Q34

**Q135. What is pinning, and is it still a problem?** A virtual thread is **pinned** when it can't unmount from its carrier while blocked, so it ties up an OS thread. Before Java 24, blocking inside `synchronized` pinned; JEP 491 (Java 24) fixed that. It still pins when blocking under a native frame (a `native` method or FFM call that calls back into Java) and when blocking during class loading or class initialisation. Diagnose with the JFR event `jdk.VirtualThreadPinned`. → *More:* §15.5 Pinning and JEP 491

**Q136. Virtual threads vs reactive programming (WebFlux, RxJava)?** Both serve many concurrent I/O operations with few OS threads. Reactive code does it with non-blocking callbacks and operator chains — efficient, but hard to read and debug. Virtual threads keep simple blocking code and normal stack traces. Reactive still fits streaming and back-pressure use cases.

**Q137. What are scoped values (Java 25)?** `ScopedValue` passes immutable context (user, request ID) down a call chain for a bounded scope — `ScopedValue.where(USER, u).run(task)` — without parameters. Unlike `ThreadLocal` it can't be changed, is unbound automatically when `run`/`call` returns, and child threads forked in a `StructuredTaskScope` inherit it without copying. Final in Java 25 (JEP 506). → *More:* §15.6 ThreadLocal and scoped values

**Q138. What is structured concurrency? Is it final?** It treats a group of subtasks as one unit: fork them in a scope, join, and if one fails the others are cancelled — no leaked threads. Still a **preview** in Java 25 (fifth preview, JEP 505), so it needs `--enable-preview`. → *More:* §15.7 Structured concurrency

**Q139. What are Java modules (JPMS)?** Java 9's module system: a `module-info.java` declares what a module `requires` and which packages it `exports`. It gives strong encapsulation (non-exported packages are inaccessible, even by reflection unless `opens`), reliable configuration, and `jlink` custom runtimes. Java 25 (JEP 511) adds `import module java.base;` to import all packages a module exports. → *More:* §16.12 Modules

**Q140. Why upgrade from Java 8/11/17 to 21 or 25?** Language productivity (records, patterns, text blocks), virtual threads, better GCs (generational ZGC, G1 improvements), lower memory (compact object headers, a product option in 25 enabled with `-XX:+UseCompactObjectHeaders`), faster start-up (the AOT cache, Java 24–25), security and support lifetime (LTS), and frameworks such as Spring Boot 3+ requiring Java 17+. → *More:* §17.12 Upgrading

## 11. Concurrency

**Q141. Process vs thread?** A **process** is a running program with its own memory space; a **thread** is a path of execution inside a process, sharing the heap with other threads but having its own stack. Threads are cheaper to create and communicate through shared memory — which is why they need synchronisation. → *More:* §14.1 Concurrency basics

**Q142. ★ How do you create a thread? `Runnable` vs extending `Thread`?** Implement `Runnable`/`Callable` and hand it to a `Thread` or, better, an `ExecutorService`; Java 21 adds `Thread.ofPlatform()`/`Thread.ofVirtual()` builders. Prefer `Runnable` over extending `Thread`: it separates the task from the mechanism and leaves your class free to extend something else. → *More:* §14.2 Creating threads

**Q143. ★ `start()` vs `run()`?** `start()` creates a new thread that then calls `run()`. Calling `run()` directly just executes the method on the current thread — no concurrency (Q232). Calling `start()` twice throws `IllegalThreadStateException`. → *More:* §14.2 Creating threads

**Q144. What are the thread states?** `NEW`, `RUNNABLE` (running or ready), `BLOCKED` (waiting for a monitor lock), `WAITING` (`wait()`, `join()`, `park` with no timeout), `TIMED_WAITING` (`sleep`, timed `wait`/`join`), `TERMINATED`. → *More:* §14.3 Thread lifecycle

**Q145. ★ `sleep()` vs `wait()`?** `Thread.sleep` pauses the current thread and keeps any locks it holds. `obj.wait()` must be called while holding `obj`'s monitor (else `IllegalMonitorStateException`), **releases** it, and waits until `notify`/`notifyAll`, a timeout, or interruption. Always call `wait` in a `while` loop that re-checks the condition (spurious wake-ups). → *More:* §14.8 wait/notify

**Q146. `notify()` vs `notifyAll()`?** `notify` wakes one arbitrary waiter; `notifyAll` wakes all, and each re-checks its condition. Prefer `notifyAll` unless all waiters wait for the same condition and any one can proceed — a wrong `notify` can wake the "wrong" thread and stall everyone. → *More:* §14.8 wait/notify

**Q147. Why are `wait`/`notify` in `Object` and not in `Thread`?** Because every object has a monitor and the waiting is on a *condition protected by that object's lock*, not on a particular thread. → *More:* §14.8 wait/notify

**Q148. What is `join()`? What does `yield()` do?** `t.join()` blocks the caller until thread `t` finishes (and guarantees its writes are visible). `Thread.yield()` hints that the scheduler may run another thread — rarely useful. → *More:* §14.3 Thread lifecycle

**Q149. How do you stop a thread?** Cooperatively: call `t.interrupt()`, and have the task check `Thread.currentThread().isInterrupted()` or catch `InterruptedException` from blocking calls and exit. If you catch `InterruptedException` and can't exit, restore the flag with `Thread.currentThread().interrupt()`. `Thread.stop()` is gone (throws `UnsupportedOperationException` since Java 20). → *More:* §14.4 Interruption

**Q150. ★ What is a race condition? Why isn't `count++` thread-safe?** A race condition is when correctness depends on the timing of threads. `count++` is three steps — read, add, write — so two threads can read the same value and one increment is lost. Fix with `synchronized`, `AtomicInteger.incrementAndGet()`, or `LongAdder`. → *More:* §14.5 Race conditions

**Q151. ★ How does `synchronized` work? Method vs block; static synchronized?** Every object has an intrinsic lock (**monitor**). A synchronized instance method locks `this`; a static synchronized method locks the `Class` object; a block locks the object you name (prefer a private lock object and the smallest block). The lock is **reentrant** (the owning thread can re-acquire it) and gives visibility on release/acquire. → *More:* §14.6 synchronized

**Q152. ★ What does `volatile` do? Is it enough for a counter?** Visibility and ordering of reads/writes of that field; no atomicity, so not enough for `count++`. Fine for a stop flag or a published reference. → *More:* §14.7 volatile · §19.29 Q31

**Q153. What is the Java Memory Model / happens-before?** The rules that say when one thread's writes must be visible to another: unlock → lock, volatile write → read, `start()`, `join()`, program order. Without such an edge, reordering and caching can hide writes. → *More:* §15.8 · §19.29 Q32

**Q154. ★ `synchronized` vs `ReentrantLock`?** `ReentrantLock` does the same mutual exclusion but adds `tryLock()` (with timeout), interruptible locking, optional fairness, and several `Condition`s per lock. You must `unlock()` in `finally`. Use `synchronized` by default (simpler, can't forget to release); use `ReentrantLock` for those extras. → *More:* §14.9 Explicit locks

**Q155. What are `ReadWriteLock` and `StampedLock`?** `ReentrantReadWriteLock` lets many readers in at once but only one writer — useful when reads dominate. `StampedLock` (Java 8) adds **optimistic reads**: read without locking, then `validate(stamp)` and retry under a real lock if a write happened. `StampedLock` isn't reentrant. → *More:* §14.9 Explicit locks

**Q156. What are atomic classes and CAS?** `AtomicInteger`, `AtomicLong`, `AtomicReference` update a single variable without locks using **CAS** (compare-and-set: "set to new value only if it still equals the expected value", one CPU instruction, retried on failure). Under heavy contention on a counter, `LongAdder` scales better by spreading updates over cells. → *More:* §14.10 Atomic variables

**Q157. What is the ABA problem?** A CAS sees value A, another thread changes it to B and back to A, and the CAS succeeds even though the state changed in between. Fix with a version stamp: `AtomicStampedReference`.

**Q158. ★ What is a deadlock? How do you prevent and detect it?** Threads holding locks while waiting for each other's locks forever. Prevent with a global lock order, short critical sections, `tryLock` timeouts; detect with a thread dump (`jstack`, `jcmd <pid> Thread.print`), which reports "Found one Java-level deadlock". Code in Q252. → *More:* §15.9 Liveness · §19.29 Q33

**Q159. Livelock vs starvation? Livelock**: threads keep reacting to each other and changing state but make no progress (two people stepping aside in a corridor forever). **Starvation**: a thread never gets the CPU or lock because others always win; fair locks or bounded queues help. → *More:* §15.9 Liveness

**Q160. ★ Why use `ExecutorService` instead of `new Thread()`? What types are there?** Reuse, bounded threads, task queue, futures, one place to shut down. Factories: `newFixedThreadPool(n)`, `newCachedThreadPool()`, `newSingleThreadExecutor()`, `newScheduledThreadPool(n)`, `newWorkStealingPool()`, `newVirtualThreadPerTaskExecutor()` (21). → *More:* §14.11 Executors · §19.29 Q36

**Q161. ★ Explain `ThreadPoolExecutor`'s parameters and what happens when a task is submitted.** `corePoolSize`, `maximumPoolSize`, `keepAliveTime`, `workQueue`, `threadFactory`, `RejectedExecutionHandler`. On submit: start a new thread if fewer than *core* are running; else queue the task; if the queue is full, start threads up to *max*; if that's full too, **reject** (default `AbortPolicy` throws; alternatives `CallerRunsPolicy`, `DiscardPolicy`, `DiscardOldestPolicy`). Note `newFixedThreadPool` uses an **unbounded** queue — tasks can pile up until out of memory; in production build a `ThreadPoolExecutor` with a bounded queue. → *More:* §14.11 Executors

**Q162. How big should a thread pool be?** CPU-bound: about the number of cores (`Runtime.getRuntime().availableProcessors()`). I/O-bound: cores × (1 + wait time / compute time), or simply virtual threads with a `Semaphore` around the scarce resource. Measure, don't guess. → *More:* §14.11 Executors

**Q163. `shutdown()` vs `shutdownNow()`?** `shutdown()` stops accepting new tasks and lets queued ones finish; `shutdownNow()` also interrupts running tasks and returns the queued ones. Follow with `awaitTermination(timeout)`. Since Java 19, `ExecutorService` is `AutoCloseable`, so try-with-resources waits for all tasks. → *More:* §14.11 Executors

**Q164. `Runnable` vs `Callable`; `Future` vs `CompletableFuture`?** `Callable` returns a value and may throw checked exceptions. `Future.get()` blocks; `CompletableFuture` composes asynchronously. → *More:* §15.2 CompletableFuture · §19.29 Q35

**Q165. ★ `thenApply` vs `thenCompose` vs `thenCombine`; how are errors handled in `CompletableFuture`?** `thenApply` maps the result (like `map`); `thenCompose` chains another async call that returns a `CompletableFuture` (like `flatMap`); `thenCombine` joins two independent futures; `allOf`/`anyOf` wait for many. Errors: `exceptionally` (recover), `handle` (result or error), `whenComplete` (side effect). Without an executor argument, `*Async` methods run on the common `ForkJoinPool` — pass your own executor for blocking I/O. `orTimeout`/`completeOnTimeout` (Java 9) add timeouts. Code in Q255. → *More:* §15.2 CompletableFuture

**Q166. What is the Fork/Join framework?** A pool for divide-and-conquer tasks (`RecursiveTask`/`RecursiveAction`): split a task, `fork()` one half, compute the other, `join()`. Idle workers **steal** tasks from busy workers' queues. Parallel streams and default `CompletableFuture` async methods use the common pool. → *More:* §15.3 Fork/Join

**Q167. ★ `CountDownLatch` vs `CyclicBarrier` vs `Semaphore`?** `CountDownLatch`: threads wait until a count reaches zero; one-shot (e.g. wait for 3 services to start). `CyclicBarrier`: N threads wait for each other at a point, then all continue; reusable. `Semaphore`: N permits limit how many threads use a resource at once (e.g. at most 10 DB calls). → *More:* §14.13 Synchronizers

**Q168. What is a `BlockingQueue`? Which implementations exist?** A thread-safe queue where `put` blocks when full and `take` blocks when empty — the standard producer–consumer tool (Q253). `ArrayBlockingQueue` (bounded array), `LinkedBlockingQueue` (optionally bounded), `PriorityBlockingQueue`, `DelayQueue`, `SynchronousQueue` (no capacity: each put waits for a take). → *More:* §14.12 Concurrent collections

**Q169. ★ What is `ThreadLocal`, and what is its danger?** A variable with a separate value per thread (e.g. a `SimpleDateFormat` or the current request's user). In thread pools threads live forever, so values leak or bleed into the next task unless you `remove()` in `finally`. With virtual threads prefer scoped values (Q137). → *More:* §15.6

**Q170. What is a daemon thread?** A background thread that doesn't keep the JVM alive: when only daemon threads remain, the JVM exits without running their `finally` blocks. Set with `setDaemon(true)` before `start()`. Virtual threads are always daemon threads. → *More:* §14.2 Creating threads

**Q171. How do you design a thread-safe class?** Prefer immutability; otherwise confine state (one thread owns it), guard every access to shared mutable state with the same lock, use thread-safe building blocks (`ConcurrentHashMap`, atomics), don't let `this` escape the constructor, and document the policy (`@GuardedBy`). Compound actions (check-then-act) need one atomic step. → *More:* §14.14 Designing thread-safe classes

## 12. JVM, memory and garbage collection

**Q172. ★ What are the JVM memory areas? Heap vs stack?** Heap (objects, shared, GC'd), one stack per thread (frames, locals), metaspace (class metadata, native memory), code cache, direct/native memory. The stack is per thread, LIFO, freed automatically on return, and holds primitives and references; objects themselves live on the heap (unless escape analysis removes the allocation). → *More:* §16.3 · §19.29 Q37

**Q173. ★ What replaced PermGen, and why?** Java 8 replaced PermGen (a fixed-size heap region for class metadata, prone to `OutOfMemoryError: PermGen space`) with **Metaspace** in native memory, which grows as needed (cap it with `-XX:MaxMetaspaceSize`). Interned strings and static fields had already moved to the heap in Java 7. → *More:* §16.3 Memory areas

**Q174. ★ How does garbage collection work? Which collector is the default?** Objects not reachable from GC roots are reclaimed; generational collectors collect the young generation often and the old one rarely. G1 is the default on typical servers since Java 9; on a machine or container with one CPU or under about 1.8 GB of memory the JVM picks Serial instead (still true in 25; JEP 523 makes G1 the default everywhere in a later release). Parallel suits throughput; ZGC (generational only since Java 24) and Shenandoah target sub-millisecond pauses. → *More:* §16.5 GC algorithms · §19.29 Q38

**Q175. What are GC roots, and how does an object become eligible for GC?** Roots are references the JVM always considers live: local variables on thread stacks, static fields, active threads, JNI references. An object is eligible once no chain of references from a root reaches it — even if objects reference each other in a cycle. → *More:* §16.5 GC algorithms

**Q176. Explain young generation, old generation, minor and major GC.** New objects go to **Eden**; a **minor GC** copies survivors to a **survivor** space, and objects that survive several collections are **promoted** to the **old generation**. A **major/full GC** collects the old generation (and usually everything), typically with longer pauses. The design relies on the observation that most objects die young. → *More:* §16.5 GC algorithms

**Q177. Can you force garbage collection?** No. `System.gc()` is only a request (and can be disabled with `-XX:+DisableExplicitGC`). Calling it in application code is almost always wrong. → *More:* §16.5 GC algorithms

**Q178. Strong, soft, weak and phantom references?** **Strong**: normal; never collected while reachable. **Soft**: cleared only when memory is short (memory-sensitive caches). **Weak**: cleared at the next GC once only weakly reachable (`WeakHashMap`). **Phantom**: only tells you (through a `ReferenceQueue`) that the object was collected; used for clean-up, as `Cleaner` does.

**Q179. Name the types of `OutOfMemoryError` and `StackOverflowError` causes.** `Java heap space` (too many live objects or a leak), `Metaspace` (too many classes, class-loader leak), `GC overhead limit exceeded` (GC running nearly all the time), `unable to create native thread`, `Direct buffer memory`. `StackOverflowError` comes from too-deep recursion (usually infinite recursion); `-Xss` sets the stack size. → *More:* §16.3 Memory areas

**Q180. ★ Can Java have memory leaks? How do you find one?** Yes — anything still referenced: unbounded caches, static collections, listeners, `ThreadLocal`s, class-loader leaks. Watch heap-after-GC grow, then use `jcmd GC.class_histogram`, JFR, or a heap dump in Eclipse MAT. → *More:* §16.11 Memory leaks · §19.29 Q39

**Q181. ★ What is a class loader? Explain parent delegation.** A class loader turns `.class` bytes into `Class` objects. Built in: **bootstrap** (core `java.*`), **platform** (other JDK modules; replaced the "extension" loader in Java 9), **application/system** (your class path). A loader first asks its parent; only if the parent can't find the class does it load it itself — so nobody can replace `java.lang.String`. A class is identified by its name **and** its loader. → *More:* §16.2 Class loading · §19.29 Q40

**Q182. What are the phases of class loading? Loading** (find bytes, create `Class`), **linking** — *verification* (bytecode is safe), *preparation* (static fields get default values), *resolution* (symbolic references → real ones) — and **initialisation** (static initialisers run), which happens lazily on first active use. → *More:* §16.2 Class loading

**Q183. What is the JIT compiler?** The just-in-time compiler turns frequently executed ("hot") bytecode into native machine code at run time. HotSpot uses tiered compilation: C1 compiles quickly with light optimisation, C2 later with heavy optimisation (inlining, escape analysis, loop unrolling), using profiling data gathered while interpreting. → *More:* §16.8 JIT compilation

**Q184. Which JVM flags do you know?** `-Xms`/`-Xmx` (initial/max heap), `-Xss` (thread stack), `-XX:+UseG1GC`/`UseZGC`/`UseParallelGC`, `-XX:MaxGCPauseMillis`, `-XX:+HeapDumpOnOutOfMemoryError`, `-Xlog:gc*` (GC logging), `-XX:MaxRAMPercentage` (heap as % of container memory), `-XX:+UseCompactObjectHeaders` (Java 25). → *More:* §16.7 GC tuning basics

**Q185. How do you choose a garbage collector?** G1 (default) balances throughput and pauses for most services. Parallel GC for batch jobs where total throughput matters and pauses don't. ZGC (generational) or Shenandoah for large heaps and latency-sensitive services needing pauses around a millisecond. Serial for tiny heaps or single-CPU containers. → *More:* §16.6 Collectors compared

**Q186. What tools do you use to diagnose a running JVM?** `jcmd` (thread dumps, heap dumps, GC info, JFR control), `jstack` (thread dump), `jmap`/`jcmd GC.heap_dump`, `jstat` (GC statistics), JDK Flight Recorder + JDK Mission Control (low-overhead profiling), VisualVM, async-profiler, and Eclipse MAT for heap dumps. → *More:* §16.10 Profiling and diagnostic tools

**Q187. What is escape analysis?** A JIT analysis that proves an object never leaves a method or thread; the JIT can then skip the heap allocation (scalar replacement — keep the fields in registers) and remove locks on it. → *More:* §16.8 JIT and escape analysis

## 13. I/O, serialization, reflection and annotations

**Q188. Byte streams vs character streams? Why buffer?** `InputStream`/`OutputStream` move raw bytes (images, binary); `Reader`/`Writer` move characters and need a **charset** to convert (always specify UTF-8). `BufferedReader`/`BufferedOutputStream` batch many small reads/writes into few system calls — often 10–100× faster. → *More:* §13.5 Byte and character streams · §13.6 Buffering

**Q189. `java.io` vs NIO / NIO.2?** `java.io` is stream-based and blocking (`File`, `FileInputStream`). NIO (Java 4) added buffers, channels and selectors for non-blocking I/O. NIO.2 (Java 7) added `Path` and `Files` — the modern API: `Files.readString`, `Files.lines`, `Files.walk`, better errors, symbolic links, file attributes. → *More:* §13.1 java.io vs NIO.2

**Q190. How do you read a large file line by line efficiently?** `try (Stream<String> lines = Files.lines(path, UTF_8)) { ... }` or `BufferedReader.readLine()` in a loop — both stream the file rather than loading it all. Use `Files.readString`/`readAllLines` only for small files. → *More:* §13.4 Reading and writing whole files

**Q191. ★ What is serialization? What is `serialVersionUID`?** Converting an object graph to bytes (`ObjectOutputStream`) and back (`ObjectInputStream`); the class must implement `Serializable`. `serialVersionUID` is the class's version number: if the stream's value differs from the class's, deserialization fails with `InvalidClassException`. Declare it explicitly, otherwise the compiler-computed default changes with almost any class edit. → *More:* §13.10 Serialization

**Q192. What does `transient` do? Are static fields serialized?** A `transient` field is skipped and comes back as its default value (`null`, 0). Static fields belong to the class, not the object, so they're never serialized. → *More:* §13.10 Serialization

**Q193. `Serializable` vs `Externalizable`? What if a parent isn't serializable?** `Serializable` is automatic (reflection-based); `Externalizable` makes you write `writeExternal`/`readExternal` yourself and requires a public no-arg constructor. If a superclass isn't serializable, its fields aren't saved and its no-arg constructor runs during deserialization (it must exist).

**Q194. Why is Java serialization considered dangerous? How do singletons survive it?** Deserializing untrusted bytes can execute "gadget chains" of existing classes — a classic remote-code-execution hole; it also bypasses constructors and ties you to class internals. Prefer JSON/Protobuf, and if you must, use deserialization filters (`ObjectInputFilter`). A singleton needs `readResolve()` returning the instance, or simply use an enum. → *More:* §13.10 Serialization

**Q195. What is reflection? Pros and cons?** The ability to inspect and use classes at run time: `Class.forName`, `getDeclaredMethods`, `Field.setAccessible(true)`, `Method.invoke`. Frameworks (Spring, Hibernate, JUnit, Jackson) depend on it. Cons: slower, no compile-time checks, breaks encapsulation, and module boundaries restrict it.

**Q196. How do annotations work? What is retention?** Annotations are metadata on code elements. `@Retention` says how long they live: `SOURCE` (dropped by compiler, e.g. `@Override`), `CLASS` (in the class file, not at run time — the default), `RUNTIME` (readable by reflection, e.g. `@Test`, Spring's annotations). `@Target` limits where they may appear. A custom one: `@Retention(RUNTIME) @Target(METHOD) @interface Timed {}`.

**Q197. How many ways can you create an object?** `new`, reflection (`clazz.getDeclaredConstructor().newInstance()`), `clone()`, deserialization, and factory methods / builders (which use `new` underneath). Records and enums add their own restrictions (enums can't be instantiated reflectively).

## 14. Testing and tooling

**Q198. ★ How do you write a unit test in JUnit (5/6)?** Annotate a method with `@Test`, arrange inputs, act, assert with `assertEquals`/`assertTrue` or AssertJ's `assertThat(x).isEqualTo(y)`. Lifecycle: `@BeforeEach`/`@AfterEach` around each test, `@BeforeAll`/`@AfterAll` (static) once per class; JUnit 4 called these `@Before`/`@BeforeClass`. The book uses JUnit 6, which keeps the JUnit 5 ("Jupiter") API. → *More:* §18.4 JUnit

**Q199. How do you test that a method throws an exception?** `var ex = assertThrows(IllegalArgumentException.class, () -> account.withdraw(-1)); assertEquals("negative amount", ex.getMessage());` or AssertJ `assertThatThrownBy(...).isInstanceOf(...).hasMessage(...)`. → *More:* §18.4 JUnit

**Q200. ★ What is mocking? Mock vs stub vs spy?** A **test double** replaces a real dependency. A **stub** returns canned answers; a **mock** also lets you verify interactions (`verify(repo).save(order)`); a **spy** wraps a real object and calls real methods unless stubbed. With Mockito: `@Mock`, `@InjectMocks`, `when(x.f()).thenReturn(v)`, `ArgumentCaptor`. Static methods can be mocked with `mockStatic` (Mockito 5's default inline mock maker), though needing it is often a design smell. → *More:* §18.6 Test doubles and Mockito

**Q201. What are parameterized tests?** One test method run with many inputs: `@ParameterizedTest` with `@ValueSource`, `@CsvSource`, `@MethodSource` or `@EnumSource`. → *More:* §18.5 Parameterized tests

**Q202. Unit vs integration vs end-to-end tests; what is TDD?** Unit: one class in isolation, fast. Integration: several real components together (database via Testcontainers, HTTP layer). End-to-end: the whole system as a user sees it. Keep many unit tests, fewer integration, few E2E (the test pyramid). **TDD** (test-driven development): write a failing test, make it pass, refactor. → *More:* §18.9 Test design

**Q203. Maven vs Gradle? What are the Maven lifecycle phases?** Both resolve dependencies and build. Maven: XML `pom.xml`, fixed conventions, lifecycle `validate → compile → test → package → verify → install → deploy`. Gradle: Kotlin/Groovy build scripts, incremental builds and build cache, more flexible and usually faster. Dependency scopes like `test` or `provided` keep libraries out of the runtime. → *More:* §18.2 Maven · §18.3 Gradle

**Q204. How should you log in Java?** Through the SLF4J API with Logback or Log4j 2 behind it; use parameterised messages (`log.info("user {} logged in", id)`) so strings are built only when the level is enabled; never log secrets; log an exception once, with its stack trace. → *More:* §18.10 Logging

## 15. Scenario and senior-level questions

**Q205. ★ Your service's CPU is at 100%. How do you investigate?** Find the hot threads (`top -H -p <pid>`, convert the thread ID to hex) and match them in a thread dump (`jcmd <pid> Thread.print`), taking several dumps a few seconds apart. Check GC logs — constant GC from a nearly full heap looks like high CPU. Profile with JFR or async-profiler for a flame graph. Typical culprits: infinite loops, regex backtracking, busy-wait spin loops, and GC thrashing. → *More:* §16.10 Diagnostic tools

**Q206. ★ The application throws `OutOfMemoryError` in production. What do you do?** Read the message (heap, Metaspace, native thread…). Make sure `-XX:+HeapDumpOnOutOfMemoryError` is on, open the dump in Eclipse MAT and look at the dominator tree / biggest retained sizes. Check whether heap-after-GC grows steadily (leak) or the heap is simply too small for the load. Fix the leak or bound the cache before raising `-Xmx`. → *More:* §16.11 Memory leaks

**Q207. Requests hang and threads pile up. What would you look at?** Thread dumps: are many threads `BLOCKED` on one lock, `WAITING` on a connection pool, or stuck in socket reads with no timeout? Typical fixes: set timeouts on every remote call, size or bound pools, remove the lock contention, and look for a deadlock reported in the dump. → *More:* §16.10 Diagnostic tools

**Q208. How would you make a `HashMap`-based cache thread-safe and bounded?** Use `ConcurrentHashMap` with `computeIfAbsent` for atomic load-on-miss, and add eviction (size or time based) — a hand-rolled version gets hard quickly, so in production use Caffeine. For an LRU cache with a lock, wrap a `LinkedHashMap` (Q249) with `synchronized` methods.

**Q209. Why can a `parallelStream()` make a web service slower?** All parallel streams share the JVM-wide common `ForkJoinPool` (sized cores − 1). Under many concurrent requests they compete for the same few threads, and blocking calls inside them stall the pool for everyone. In servers, prefer sequential streams, or an explicit executor for parallel work. → *More:* §12.8 Parallel streams

## 16. "What does this print?" output puzzles

All outputs below were produced by running the code.

**Q210. ★ String pool and `==`.**

```
String s1 = "Java", s2 = "Ja" + "va", s3 = new String("Java");
String part = "Ja", s4 = part + "va";
final String fp = "Ja";
System.out.println((s1 == s2) + " " + (s1 == s3) + " " + (s1 == s4) + " "
        + (s1 == s4.intern()) + " " + (s1 == fp + "va"));
```

Prints `true false false true true`. Literal-only concatenation (and a `final` constant) is folded at compile time into the pooled `"Java"`; `new` and run-time concatenation create new objects; `intern()` returns the pooled one. → *More:* §7.1 Strings and the pool

**Q211. ★ The `Integer` cache.**

```
Integer a = 127, b = 127, c = 128, d = 128;
System.out.println((a == b) + " " + (c == d) + " " + c.equals(d));
```

Prints `true false true`. Autoboxing calls `Integer.valueOf`, which caches −128…127; 128 gives two different objects. Compare wrappers with `equals`. → *More:* §7.6 Wrappers and autoboxing

**Q212. ★ `return` in `try` and `finally`.**

```
static int f1() { try { return 1; } finally { return 2; } }
static int f2() { int x = 1; try { return x; } finally { x = 2; } }
```

`f1()` returns `2` — the `finally` return replaces the first one. `f2()` returns `1` — the return value was already evaluated before `finally` changed `x`. → *More:* §8.4 try/catch/finally

**Q213. String concatenation order.** `System.out.println(1 + 2 + "3" + 4 + 5);` prints `3345`. `+` is left-associative: `1 + 2` is int addition (3), then everything after the first string is concatenation. → *More:* §7.1 Strings and the pool

**Q214. `char` arithmetic.** `System.out.println('a' + 'b');` prints `195` (both promoted to `int`: 97 + 98), but `"" + 'a' + 'b'` prints `ab`. And `char ch = 'a'; ch += 1;` gives `b` (compound assignment casts back), while `System.out.println(ch + 1)` with the original `ch = 'a'` prints `98` (an `int`).

**Q215. Floating point and overflow.** `0.1 + 0.2 == 0.3` is `false` (sum is `0.30000000000000004`). `Math.abs(Integer.MIN_VALUE)` is `-2147483648` (its positive value doesn't fit in an `int`), and `Integer.MAX_VALUE + 1` is `-2147483648`. → *More:* §2.11 Floating point · §2.10 Integer overflow

**Q216. ★ Overloading with `null`.**

```
static void m(Object o) { System.out.println("Object"); }
static void m(String s) { System.out.println("String"); }
m(null);
```

Prints `String`: the compiler picks the **most specific** applicable overload. Add `m(Integer)` too and the call becomes ambiguous — a compile error. → *More:* §4.3 Overloading

**Q217. ★ Overriding vs hiding (fields and static methods).**

```
class P { String name = "P"; static String s() { return "P.s"; } String who() { return "P.who"; } }
class C extends P { String name = "C"; static String s() { return "C.s"; } String who() { return "C.who"; } }
P p = new C();
System.out.println(p.name + " " + p.s() + " " + p.who());
```

Prints `P P.s C.who`. Fields and static methods are resolved from the **declared** type (`P`); only instance methods are dispatched on the run-time object. → *More:* §6.2 Hiding

**Q218. ★ Calling an overridable method from a constructor.**

```
class P { P() { init(); } void init() {} }
class C extends P { String v = "set"; void init() { System.out.println("init sees v=" + v); } }
new C();
```

Prints `init sees v=null`. `P`'s constructor runs before `C`'s field initialisers, but `init()` dispatches to `C`. Never call overridable methods from constructors. → *More:* §5.10 Initialisation order

**Q219. `i = i++`.** `int i = 0; i = i++;` leaves `i` at `0`: `i++` yields the old value (0), increments `i` to 1, then the assignment writes the old value 0 back. Similarly `int x = 5; x += x++ * 2;` gives `15` (5 + 5·2, with the left `x` read before the increment).

**Q220. `List.remove(int)` vs `remove(Object)`.**

```
List<Integer> list = new ArrayList<>(List.of(1, 2, 3));
list.remove(1);                    // index 1 → [1, 3]
list.remove(Integer.valueOf(1));   // the value 1 → [3]
```

An `int` argument picks `remove(int index)` because exact match beats boxing.

**Q221. `Arrays.asList` traps.** `Arrays.asList(new int[]{1, 2, 3}).size()` is `1` — a primitive array is one object, giving a `List<int[]>`; `Arrays.asList(1, 2, 3).size()` is `3`. And `Arrays.asList(3, 1, 2).add(4)` throws `UnsupportedOperationException` (fixed-size view).

**Q222. ★ Removing inside a for-each loop.**

```
List<String> l = new ArrayList<>(List.of("a", "b", "c"));
for (String x : l) if (x.equals("b")) l.remove(x);
System.out.println(l);
```

Prints `[a, c]` — **no** exception, because removing the second-to-last element shrinks the list so `hasNext()` returns false before the check runs. Removing `"a"` instead throws `ConcurrentModificationException`. Either way, use `l.removeIf(...)`. → *More:* §10.7 Iterators

**Q223. ★ Stream laziness and short-circuiting.**

```
Stream.of(1, 2, 3).peek(x -> System.out.println("never")).map(x -> x * 2);
String r = Stream.of("a", "bb", "ccc")
        .filter(x -> { System.out.print(x + " "); return x.length() > 1; })
        .findFirst().get();
```

The first line prints nothing (no terminal operation). The second prints `a bb `and returns `bb`: `findFirst` stops the pipeline, so `"ccc"` is never examined. → *More:* §12.2 Laziness

**Q224. ★ `equals` without `hashCode`.**

```
class Key { int k; Key(int k) { this.k = k; }
    public boolean equals(Object o) { return o instanceof Key x && x.k == k; } }
Set<Key> set = new HashSet<>(); set.add(new Key(1)); set.add(new Key(1));
System.out.println(set.size());
```

Prints `2` even though the keys are equal: each has a different identity hash code, so they land in different buckets. → *More:* §6.13 equals and hashCode

**Q225. Ternary unboxing NPE.**

```
Integer n = null;
int y = flag ? n : 0;   // NullPointerException when flag is true
```

With one `Integer` and one `int` operand, the result type is `int`, so `n` is unboxed.

**Q226. Narrowing casts.** `(byte) 200` is `-56` (keeps the low 8 bits), `(int) 3.99` is `3` (truncates toward zero), `(char) 65` is `A`.

**Q227. Strings are immutable.** `String s = "hello"; s.toUpperCase(); System.out.println(s);` prints `hello`. The result was discarded — write `s = s.toUpperCase();`.

**Q228. try-with-resources close order.**

```
try (R r1 = new R("first"); R r2 = new R("second")) { System.out.println("body"); }
```

Prints `body`, `close second`, `close first` — resources close in reverse order of declaration. → *More:* §8.6 try-with-resources

**Q229. Division edge cases.** `10 / 0` throws `ArithmeticException: / by zero`; `10.0 / 0` is `Infinity`; `0.0 / 0` is `NaN`; `Double.NaN == Double.NaN` is `false` (use `Double.isNaN`). `Math.round(-2.5)` is `-2` and `Math.round(2.5)` is `3` (it rounds half up: `floor(x + 0.5)`). → *More:* §2.11 Floating point

**Q230. Overload resolution order: widening, boxing, varargs.** With `m2(long)` and `m2(Integer)`, `m2(5)` prints `long`. With `m3(long)` and `m3(int...)`, `m3(5)` prints `long3`. The compiler tries, in phases: exact/widening first, then boxing, then varargs. → *More:* §4.3 Overloading

**Q231. Immutable factory traps.** `List.of("a", null)` throws `NullPointerException`; `Map.of("k", 1, "k", 2)` throws `IllegalArgumentException` (duplicate key).

**Q232. `run()` instead of `start()`.** `new Thread(() -> System.out.println(Thread.currentThread().getName())).run();` prints `main` — no new thread was started.

**Q233. `HashMap` and the null key.** `map.put(null, 1); map.put(null, 2);` leaves `{null=2}`. `put` returns the previous value, so `map.put("z", 3)` on an absent key returns `null`.

**Q234. `StringBuilder` equality.** `new StringBuilder("x").equals(new StringBuilder("x"))` is `false` — `StringBuilder` doesn't override `equals`. Compare `sb1.toString().equals(sb2.toString())` or use `sb1.compareTo(sb2) == 0` (Java 11).

**Q235. `BigDecimal` in `HashSet` vs `TreeSet`.** Adding `new BigDecimal("1.0")` and `new BigDecimal("1.00")`: a `HashSet` has size `2` (`equals` compares scale too), a `TreeSet` has size `1` (`compareTo` says they're equal). Also `new BigDecimal(0.1)` prints `0.1000000000000000055511151231257827021181583404541015625`, while `BigDecimal.valueOf(0.1)` prints `0.1`.

**Q236. `switch` on a null string.** `String s = null; switch (s) { case "a" -> ...; default -> ...; }` throws `NullPointerException` — `default` doesn't match `null`. Since Java 21 you can add `case null ->`.

**Q237. Overflow before widening.** `long big = Integer.MAX_VALUE * 2;` is `-2`: the multiplication happens in `int` and overflows *before* the assignment widens it. `Integer.MAX_VALUE * 2L` gives `4294967294`. → *More:* §2.10 Integer overflow

**Q238. Integer literal prefixes.** `System.out.println(010 + 0x10 + 0b10);` prints `26`: octal 8 + hex 16 + binary 2.

**Q239. `split` drops trailing empty strings.** `"a,b,,".split(",").length` is `2`. Use `split(",", -1)` to keep them (length 4).

## 17. Coding tasks asked in Java rounds

These are the "write it in Java" tasks that keep appearing in Java-developer interviews (especially Java 8 stream tasks). Each solution was run; the result is shown in the comment.

**Q240. ★ Reverse a string.**

```
String s = "programming";
String r1 = new StringBuilder(s).reverse().toString();   // gnimmargorp
String r2 = "";
for (int i = s.length() - 1; i >= 0; i--) r2 += s.charAt(i);         // O(n²): mention it
```

Interviewers often forbid `reverse()`: swap `char`s in a `char[]` with two pointers instead (O(n)). → *More:* §19.2

**Q241. Check whether a string is a palindrome.**

```
boolean pal = IntStream.range(0, w.length() / 2)
        .allMatch(i -> w.charAt(i) == w.charAt(w.length() - 1 - i));  // "racecar" → true
```

For "ignore case and punctuation" variants, skip non-letters with two pointers and compare `Character.toLowerCase`.

**Q242. Check whether two strings are anagrams.**

```
char[] a = "listen".toCharArray(), b = "silent".toCharArray();
Arrays.sort(a); Arrays.sort(b);
boolean anagram = Arrays.equals(a, b);   // true
```

O(n log n). O(n): count letters in an `int[26]`, increment for one string, decrement for the other, check all zeros.

**Q243. ★ Count character frequency and find the first non-repeated character (streams).**

```
Map<Character, Long> freq = "programming".chars().mapToObj(c -> (char) c)
        .collect(Collectors.groupingBy(Function.identity(), LinkedHashMap::new, Collectors.counting()));
// {p=1, r=2, o=1, g=2, a=1, m=2, i=1, n=1}
char first = freq.entrySet().stream().filter(e -> e.getValue() == 1)
        .map(Map.Entry::getKey).findFirst().orElseThrow();   // p
```

`LinkedHashMap::new` keeps first-appearance order — that's what makes "first" correct. → *More:* §19.3 HashMap idioms

**Q244. Find duplicate elements.**

```
Set<Character> seen = new HashSet<>();
Set<Character> dups = "programming".chars().mapToObj(c -> (char) c)
        .filter(c -> !seen.add(c))
        .collect(Collectors.toCollection(LinkedHashSet::new));   // [r, m, g]
```

`Set.add` returns `false` for an element already present. Alternative: `groupingBy` + `counting()`, keep counts > 1. (The stateful `filter` is fine sequentially, not in a parallel stream.) → *More:* §19.3 HashMap idioms

**Q245. ★ Find the second-highest number.**

```
int second = List.of(5, 9, 2, 9, 7, 2, 4).stream().distinct()
        .sorted(Comparator.reverseOrder()).skip(1).findFirst().orElseThrow();   // 7
```

`distinct()` matters: without it the answer would be the duplicate 9. In a loop, track `max` and `second` in one O(n) pass.

**Q246. ★ Employee stream tasks: count per department, average salary, top earner per department.**

```
record Employee(String name, String dept, double salary, int age) {}
var emps = List.of(new Employee("Ana", "Eng", 120, 28), new Employee("Bo", "Eng", 100, 35),
        new Employee("Cy", "Ops", 90, 40), new Employee("Di", "Ops", 95, 25), new Employee("Ed", "HR", 70, 45));
emps.stream().collect(groupingBy(Employee::dept, TreeMap::new, counting()));
// {Eng=2, HR=1, Ops=2}
emps.stream().collect(groupingBy(Employee::dept, TreeMap::new, averagingDouble(Employee::salary)));
// {Eng=110.0, HR=70.0, Ops=92.5}
emps.stream().collect(groupingBy(Employee::dept, TreeMap::new,
        collectingAndThen(maxBy(comparingDouble(Employee::salary)), o -> o.orElseThrow().name())));
// {Eng=Ana, HR=Ed, Ops=Di}
emps.stream().filter(e -> e.age() > 30).sorted(comparingDouble(Employee::salary).reversed())
        .map(Employee::name).collect(joining(", "));   // Bo, Cy, Ed
```

(Static imports from `Collectors` and `Comparator`.) Second-highest salary: `map(Employee::salary).distinct().sorted(reverseOrder()).skip(1).findFirst()`. → *More:* §12.5 Collectors

**Q247. ★ Sort a map by its values.**

```
Map<String, Integer> scores = Map.of("a", 3, "b", 1, "c", 2);
Map<String, Integer> sorted = scores.entrySet().stream()
        .sorted(Map.Entry.comparingByValue())
        .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue,
                (a, b) -> a, LinkedHashMap::new));   // {b=1, c=2, a=3}
```

The `LinkedHashMap` supplier is essential — a plain `toMap` returns a `HashMap` and loses the order.

**Q248. ★ Write a thread-safe singleton.**

```
final class Config {                          // holder idiom: lazy, thread-safe, no locking
    private Config() {}
    private static class Holder { static final Config INSTANCE = new Config(); }
    static Config get() { return Holder.INSTANCE; }
}
final class Dcl {                             // double-checked locking
    private static volatile Dcl instance;     // volatile is mandatory
    static Dcl get() {
        Dcl r = instance;
        if (r == null) synchronized (Dcl.class) { r = instance; if (r == null) instance = r = new Dcl(); }
        return r;
    }
}
```

Without `volatile`, another thread could see a non-null reference to a not-yet-fully-constructed object. Simplest of all: `enum Config { INSTANCE; }`.

**Q249. ★ Implement an LRU cache.**

```
class LRU<K, V> extends LinkedHashMap<K, V> {
    private final int cap;
    LRU(int cap) { super(16, 0.75f, true); this.cap = cap; }   // true = access order
    @Override protected boolean removeEldestEntry(Map.Entry<K, V> e) { return size() > cap; }
}
// cap 2: put 1, put 2, get 1, put 3 → keys [1, 3]
```

If asked to do it "from scratch", describe `HashMap<K, Node>` + doubly linked list (Q102). → *More:* §10.4 Maps

**Q250. ★ Write an immutable class.**

```
final class Money {
    private final String currency;
    private final List<String> tags;
    Money(String currency, List<String> tags) {
        this.currency = Objects.requireNonNull(currency);
        this.tags = List.copyOf(tags);          // defensive, unmodifiable copy
    }
    String currency() { return currency; }
    List<String> tags() { return tags; }       // safe to return: can't be modified
}
```

Then point out a `record Money(String currency, List<String> tags)` with `tags = List.copyOf(tags)` in a compact constructor does the same. → *More:* §5.9 Immutability

**Q251. Implement a simple `HashMap`.**

```
class SimpleHashMap<K, V> {
    private static class Node<K, V> { final K key; V value; Node<K, V> next;
        Node(K k, V v, Node<K, V> n) { key = k; value = v; next = n; } }
    private Node<K, V>[] table = newTable(16);
    private int size;
    private static int index(Object key, int n) { int h = Objects.hashCode(key); return (h ^ (h >>> 16)) & (n - 1);
}
    public V get(K key) {
        for (Node<K, V> e = table[index(key, table.length)]; e != null; e = e.next)
            if (Objects.equals(e.key, key)) return e.value;
        return null;
    }
    public void put(K key, V value) {
        int i = index(key, table.length);
        for (Node<K, V> e = table[i]; e != null; e = e.next)
            if (Objects.equals(e.key, key)) { e.value = value; return; }
        table[i] = new Node<>(key, value, table[i]);
        if (++size > table.length * 3 / 4) resize();   // rehash every node into a 2× table
    }
    // newTable: @SuppressWarnings("unchecked") cast of new Node[n]; resize: loop over old buckets
}
```

Talk through: power-of-two capacity, hash spreading, chaining, load factor 0.75, `equals` for key match, null key via `Objects.hashCode`. (The full version with `resize` was run: 100 keys in, updates and lookups correct.) → *More:* §10.5 HashMap internals

**Q252. ★ Write a program that deadlocks — then fix it.**

```
Object a = new Object(), b = new Object();
new Thread(() -> { synchronized (a) { sleep(100); synchronized (b) { } } }).start();
new Thread(() -> { synchronized (b) { sleep(100); synchronized (a) { } } }).start();
// both threads end up BLOCKED forever
```

Fix: make both threads take `a` then `b` (one global lock order), or use `ReentrantLock.tryLock(timeout)` and back off. → *More:* §15.9 Liveness

**Q253. ★ Producer–consumer with a `BlockingQueue`.**

```
BlockingQueue<Integer> q = new ArrayBlockingQueue<>(2);
Thread producer = Thread.ofPlatform().start(() -> {
    try { for (int v = 1; v <= 5; v++) q.put(v); q.put(-1); }        // -1 = "poison pill" end marker
    catch (InterruptedException e) { Thread.currentThread().interrupt(); } });
Thread consumer = Thread.ofPlatform().start(() -> {
    try { for (int v; (v = q.take()) != -1; ) got.add(v); }
    catch (InterruptedException e) { Thread.currentThread().interrupt(); } });
// got = [1, 2, 3, 4, 5]
```

`put` blocks when full, `take` when empty, so no explicit locking is needed. → *More:* §14.12 Concurrent collections

**Q254. Producer–consumer with `wait`/`notify` (a bounded buffer).**

```
final class BoundedQueue<T> {
    private final Deque<T> items = new ArrayDeque<>();
    private final int cap;
    BoundedQueue(int cap) { this.cap = cap; }
    synchronized void put(T t) throws InterruptedException {
        while (items.size() == cap) wait();      // while, not if: re-check after waking
        items.addLast(t); notifyAll();
    }
    synchronized T take() throws InterruptedException {
        while (items.isEmpty()) wait();
        T t = items.removeFirst(); notifyAll(); return t;
    }
}
```

→ *More:* §14.8 wait/notify

**Q255. ★ Run two async calls and combine them with `CompletableFuture`.**

```
try (var ex = Executors.newVirtualThreadPerTaskExecutor()) {
    var price = CompletableFuture.supplyAsync(() -> 100, ex);
    var rate  = CompletableFuture.supplyAsync(() -> 2, ex);
    int total = price.thenCombine(rate, (p, r) -> p * r)
            .orTimeout(1, TimeUnit.SECONDS)
            .exceptionally(t -> -1)       // fallback on failure or timeout
            .join();                      // 200
}
```

For N calls: `CompletableFuture.allOf(fs).thenApply(v -> fs.stream().map(CompletableFuture::join).toList())`.

**Q256. ★ Print odd and even numbers alternately with two threads.**

```
static void printTurn(Object lock, int[] counter, int parity, int max) {
    synchronized (lock) {
        while (counter[0] <= max) {
            if (counter[0] % 2 == parity) { System.out.print(counter[0]++ + " "); lock.notifyAll(); }
            else try { lock.wait(); } catch (InterruptedException e) { Thread.currentThread().interrupt(); return;
}
        }
        lock.notifyAll();   // release the other thread so it can exit
    }
}
// thread A: printTurn(lock, counter, 1, 10); thread B: printTurn(lock, counter, 0, 10)
// 1 2 3 4 5 6 7 8 9 10
```

Alternatives: two `Semaphore`s handing a permit back and forth, or a `ReentrantLock` with two `Condition`s. → *More:* §14.8 wait/notify

**Q257. Count words; flatten a list of lists.**

```
Map<String, Long> wc = Arrays.stream("to be or not to be".split(" "))
        .collect(Collectors.groupingBy(Function.identity(), TreeMap::new, Collectors.counting()));
// {be=2, not=1, or=1, to=2}
List<Integer> flat = List.of(List.of(1, 2), List.of(3), List.<Integer>of())
        .stream().flatMap(List::stream).toList();   // [1, 2, 3]
```

Loop version of word count: `counts.merge(word, 1, Integer::sum)`. → *More:* §12.5 Collectors

**Q258. Fibonacci and primes.**

```
Stream.iterate(new long[]{0, 1}, f -> new long[]{f[1], f[0] + f[1]})
        .limit(10).map(f -> f[0]).toList();         // [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
IntStream.rangeClosed(2, 30)
        .filter(n -> IntStream.rangeClosed(2, (int) Math.sqrt(n)).noneMatch(d -> n % d == 0))
        .boxed().toList();                          // [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
```

Mention: recursive Fibonacci without memoisation is $O(2^n)$; the iterative one is O(n). For factorials beyond 20!, `long` overflows — use `BigInteger` (25! = 15511210043330985984000000).

**Q259. Merge two sorted arrays; find the missing number in 1..n.**

```
int i = 0, j = 0, k = 0;
while (i < a.length && j < b.length) m[k++] = a[i] <= b[j] ? a[i++] : b[j++];
while (i < a.length) m[k++] = a[i++];
while (j < b.length) m[k++] = b[j++];      // {1,3,5} + {2,4,6,8} → [1, 2, 3, 4, 5, 6, 8]
int missing = n * (n + 1) / 2 - Arrays.stream(arr).sum();   // {1,2,4,5}, n=5 → 3
```

For large n the sum can overflow — use `long` or XOR all values and indices.

**Q260. String compression (`aaabccdddd` → `a3b1c2d4`).**

```
StringBuilder sb = new StringBuilder();
for (int i = 0; i < s.length(); ) {
    int j = i;
    while (j < s.length() && s.charAt(j) == s.charAt(i)) j++;
    sb.append(s.charAt(i)).append(j - i);
    i = j;
}
return sb.length() < s.length() ? sb.toString() : s;    // a3b1c2d4
```

**Q261. Reverse the words of a sentence; count vowels.**

```
String.join(" ", Arrays.asList("the quick brown fox".split("\\s+")).reversed());  // fox brown quick the
"Interview".toLowerCase().chars().filter(c -> "aeiou".indexOf(c) >= 0).count();    // 4
```

`List.reversed()` is a Java 21 sequenced-collection view; before 21 use `Collections.reverse`.

**Q262. Swap two numbers without a temporary variable.** `a = a ^ b; b = a ^ b; a = a ^ b;` (3, 7 → 7, 3). The arithmetic version `a = a + b; b = a - b; a = a - b;` can overflow. In real code just use a temp variable.

**Q263. Count events from many threads correctly.**

```
var counter = new AtomicInteger();
try (var ex = Executors.newFixedThreadPool(4)) {
    for (int t = 0; t < 1000; t++) ex.submit(counter::incrementAndGet);
}                                      // close() waits for all tasks
System.out.println(counter.get());     // 1000 (a plain int++ would usually lose updates)
```

→ *More:* §14.10 Atomic variables

## 18. DSA problems in Java rounds

Java rounds usually include one or two classic LeetCode problems. Chapter 19 solves each pattern in idiomatic Java; here is the problem, the one-line approach, and where the full solution lives.

**Q264. ★ Two Sum.** One pass with a `HashMap<value, index>`: for each `x`, check whether `target − x` is already in the map. O(n). → §19.10 Hashing

**Q265. ★ Longest substring without repeating characters.** Sliding window: move `right`, and when a character repeats jump `left` past its previous position (stored in a map or `int[128]`). O(n). → §19.9 Sliding window

**Q266. Valid parentheses.** Push opening brackets on an `ArrayDeque`; on a closing bracket pop and check it matches; valid if the stack ends empty. → §19.12 Stacks

**Q267. Reverse a linked list; detect a cycle.** Reverse with three pointers `prev`/`cur`/`next`. Cycle: Floyd's fast/slow pointers meet if there is a cycle. → §19.22 Linked lists

**Q268. ★ K-th largest element / top-k frequent.** Keep a min-heap (`PriorityQueue`) of size k; the root is the answer. O(n log k). Quickselect gives O(n) average. → §19.13 Heaps

**Q269. Binary search (and its overflow bug).** Compute the middle as `lo + (hi - lo) / 2` (or `(lo + hi) >>> 1`); `(lo + hi) / 2` overflows for large indices. → §19.14 Binary search

**Q270. ★ Merge overlapping intervals.** Sort by start (`Comparator.comparingInt(a -> a[0])`), then extend the last merged interval or start a new one. O(n log n). → §19.15 Intervals

**Q271. Group anagrams.** Key each word by its sorted characters; `computeIfAbsent(key, k -> new ArrayList<>()).add(word)`. → §19.10 Hashing

**Q272. Level-order traversal; number of islands.** BFS with an `ArrayDeque` queue (process one level per `size()` snapshot); islands = count BFS/DFS starts over unvisited land cells. → §19.16 BFS and DFS

**Q273. Maximum subarray, climbing stairs, coin change.** Dynamic programming: Kadane's algorithm keeps the best sum ending here; stairs is Fibonacci; coin change fills `dp[amount]` bottom-up. → §19.19 Dynamic programming

**Q274. Which Java-specific pitfalls cost points in coding rounds?** `==` on `Integer`/`String`, `int` overflow in sums and midpoints, `Arrays.sort` on `int[]` with a comparator (needs `Integer[]`), comparators written as `a - b`, using `Stack`/`LinkedList` instead of `ArrayDeque`, string `+=` in loops, and forgetting `assert` is disabled by default. → §19.6 Pitfalls

## 19. Multithreading problems (Chapter 22)

**Q275. ★ Implement a counting semaphore without `java.util.concurrent`.** One `int permits` under the object's monitor: `acquire` = `while (permits == 0) wait(); permits--`, `release` = `permits++; notifyAll()`. An interrupted waiter takes nothing; a timed version loops on a fixed `nanoTime` deadline; FIFO fairness needs a queue where only the head may take a permit. → §22.2

**Q276. Build a read-write lock with writer preference.** State `readers`, `writing`, `waitingWriters`. Readers wait `while (writing || waitingWriters > 0)`; writers increment `waitingWriters`, wait `while (writing || readers > 0)`, and must decrement it again if interrupted. The last reader and every leaving writer call `notifyAll()`. Writer preference can starve readers; the fair `ReentrantReadWriteLock` bounds both. → §22.3

**Q277. Why does a naive barrier break when reused, and how do you fix it?** Without a reset, round 2 sees `count >= N` and nobody waits; resetting while waiters loop on `count < N` strands them. Use a **generation** number: waiters loop `while (myGeneration == generation)`, and the last arriver resets the count, bumps the generation and calls `notifyAll()`. → §22.4

**Q278. Design a deferred callback executor (run each callback after its delay).** A min-heap of `(nanoTime deadline, seq, callback)` under a lock and one worker that `awaitNanos(head − now)`. `schedule()` signals only when the new task becomes the head; the worker re-reads the head after every wake-up, so no signal can be missed. Run callbacks outside the lock and catch their exceptions. → §22.5

**Q279. ★ Turn a callback-based async API into a blocking call.** Create a per-call mailbox, let the callback record the outcome *in state* (first completion wins) and wake the caller, who waits `while (!done)` with a timeout. Simplest: complete a `CompletableFuture` in the callback and call `get(timeout)`. Never block on the library's own callback thread. → §22.6

**Q280. Three threads call `first()`, `second()`, `third()` in any order; print them in order.** Two one-shot gates that remember being opened: a `CountDownLatch(1)` or `Semaphore(0)` per gate, or a monitor with an `int stage` and `while (stage != mine) wait()` + `notifyAll()`. A bare `notify()` sent before the waiter arrives would be lost. → §22.7

## Sources of the question lists

Question wording and frequency were taken from these public lists and reports (answers are this book's own):

- teamlead/java-interview-questions (GitHub) — questions ranked by how often they were asked in real interviews (high/moderate/low occurrence)
- learning-zone/java-basics (GitHub) — "Java Basics" interview question collection
- in28minutes/JavaInterviewQuestionsAndAnswers (GitHub) — 200+ Java interview questions list
- Devinterview-io/java-interview-questions (GitHub) — top Java interview questions
- DopplerHQ/awesome-interview-questions (GitHub) — Java section and the lists it links (Toptal "Essential Java Interview Questions", Java Code Geeks "115 Java Interview Questions")
- sats17/Java-8-Interview-Sample-Coding-Questions (GitHub) and the widely shared "commonly asked Java 8 stream coding questions" articles (Medium)
- LeetCode Discuss interview-experience posts for Java back-end roles (e.g. "Senior Software Engineer | Java tech stack": Java 8 features, `HashMap` and `ConcurrentHashMap` internals)
- Glassdoor interview reports for Java developer roles (`HashMap` vs `ConcurrentHashMap`, stream tasks on employee lists)
- LeetCode "Top Interview 150" problem list (for the DSA problems in group 18)

# Chapter 22 — Multithreading Interview Problems

The classic "build it yourself" concurrency problems of senior Java interviews. Each section gives the problem, what the interviewer is testing, the approach, key code (full programs in `code/`, package `ch22.*`), real output, and pitfalls and follow-ups. The primitives (`synchronized`, `wait`/`notify`, `Lock`/`Condition`, `Semaphore`, latches, barriers, interrupts, the memory model) are taught in Chapter 14 and Chapter 15; this chapter links there instead of repeating them.

Run any problem with: `cd code && javac -d out $(find . -name '*.java') && java -cp out ch22.<topic>.Main` (for example `ch22.semaphore.Main`). Lines marked `[nondeterministic]` change from run to run; lines marked `[machine-dependent]` depend on your hardware.

## 22.1 Blocking queue / bounded buffer — four ways

### The problem

Build a **bounded blocking queue** (also called a **bounded buffer**): a first-in, first-out (FIFO) container with a fixed **capacity** shared by **producer** threads (which call `put(item)`) and **consumer** threads (which call `take()`).

- `put` must **block** — wait without returning — while the queue is full.
- `take` must block while the queue is empty.
- No item may be lost, duplicated or reordered, whatever the number of producers and consumers.

This is the **producer-consumer** problem. The JDK answer is `ArrayBlockingQueue` (§14.12). In the interview you build it yourself, usually in two or three of the following ways, and you explain why the fourth is wrong.

| Way | Waiting mechanism | Verdict |
|---|---|---|
| 1 | object monitor: `synchronized` + `wait`/`notifyAll` | correct; the classic whiteboard answer |
| 2 | one `ReentrantLock` + two `Condition`s | correct; this is how `ArrayBlockingQueue` is built |
| 3 | two counting semaphores + a mutex | correct; the textbook (Dijkstra-style) answer |
| 4 | busy-wait: loop on "lock, check, unlock" | correct but wasteful: it burns a CPU core while waiting |

### What the interviewer is testing

- **Guarded waits** (§14.8): `while`, not `if`, because of spurious wakeups and **barging** (another thread takes the lock and changes the state before the woken thread runs, §14.8.2); check and act under one lock.
- **Who to wake:** when `notify()`/`signal()` is enough and when it wakes the wrong kind of thread.
- **Acquisition order** in the semaphore version, where one wrong order deadlocks.
- **Interrupts and timeouts** (§14.4): a cancellable `take`, a timed `poll`.
- **Why busy-waiting is bad**, with numbers rather than "it's slow".

### Approach

All four versions store items in a **circular array** (also called a ring buffer): an `Object[]` of size `capacity`, plus `head` (the index of the oldest item) and `count` (or `tail`). An insert writes at `(head + count) % capacity`. A removal reads `items[head]` and advances `head`. The array never moves, so both operations are O(1) and allocate nothing.

What changes between the versions is **how a thread waits** for "not full" or "not empty".

**Way 1 — the object's monitor.** Make `put` and `take` `synchronized`, so one intrinsic lock guards `items`, `head` and `count`.

1. `put`: `while (count == capacity) wait();` then insert, `count++`, `notifyAll()`.
2. `take`: `while (count == 0) wait();` then remove, `count--`, `notifyAll()`.

Why `notifyAll()`: producers and consumers sleep in the *same* wait set, so a single `notify()` can wake a thread of the kind that cannot proceed (a producer when the buffer is still full); it goes back to sleep and the wake-up the other kind needed is gone (§14.8.3). `notifyAll()` costs extra wake-ups but is always correct.

**Way 2 — `ReentrantLock` with two `Condition`s** (§14.9): producers wait on `notFull`, consumers on `notEmpty`. After an insert, `notEmpty.signal()`: one item can satisfy exactly one consumer, and only consumers wait there, so waking one is enough. After a removal, `notFull.signal()`. No thread is woken for nothing, and `lockInterruptibly()` plus `awaitNanos` make `poll(timeout)` easy.

**Way 3 — semaphores** (§14.13). The permit counts carry the "full" and "empty" conditions:

1. `freeSlots = new Semaphore(capacity)`: a producer takes a free slot before inserting.
2. `filledSlots = new Semaphore(0)`: a consumer takes a filled slot before removing.
3. `mutex = new Semaphore(1)` (a binary semaphore used as a lock) protects `head` and `tail`: two producers holding free-slot permits still must not write the same index.
4. `put` = `freeSlots.acquire()` → `mutex.acquire()` → write → `mutex.release()` → `filledSlots.release()`; `take` is the mirror image.

The order in step 4 is the point. A producer that took `mutex` first and then blocked on `freeSlots` would sleep *holding the mutex*, so no consumer could free a slot: **deadlock**. Block on the counting semaphore while holding nothing, then take the short-lived mutex.

**Way 4 — busy-wait.** Loop: lock, check, act if possible, unlock, try again. It is *correct* (check and act share one lock), but a waiting thread never sleeps: every pass is wasted CPU, and the spinner competes for the lock with the threads that could make progress. `Thread.onSpinWait()` (Java 9, JEP 285) is only a CPU hint (save power, let a sibling hyper-thread run); the thread still occupies its core. Spinning suits waits of a few hundred nanoseconds inside lock implementations, not a queue whose consumers may wait for seconds.

### Code

Full files: `code/ch22/blockingqueue/` — `BoundedBuffer` (interface), `MonitorBuffer`, `LockConditionBuffer`, `SemaphoreBuffer`, `SpinBuffer`, `Main`.

Way 2, `LockConditionBuffer` (key parts):

```java
private final ReentrantLock lock = new ReentrantLock();
private final Condition notFull = lock.newCondition();
private final Condition notEmpty = lock.newCondition();

public void put(T item) throws InterruptedException {
    lock.lockInterruptibly();
    try {
        while (count == items.length) notFull.await();
        items[(head + count) % items.length] = item;
        count++;
        notEmpty.signal();               // exactly one consumer can use one new item
    } finally {
        lock.unlock();
    }
}

public T poll(long timeout, TimeUnit unit) throws InterruptedException {
    long nanos = unit.toNanos(timeout);
    lock.lockInterruptibly();
    try {
        while (count == 0) {
            if (nanos <= 0L) return null;
            nanos = notEmpty.awaitNanos(nanos);   // returns the time still left
        }
        return removeHead();                      // removeHead() ends with notFull.signal()
    } finally {
        lock.unlock();
    }
}
```

Way 3, `SemaphoreBuffer.put` (`take` is symmetric):

```
freeSlots.acquire();                 // 1) reserve a free slot (may block) - holding nothing
try {
    mutex.acquire();                 // 2) exclusive access to the array
} catch (InterruptedException e) {
    freeSlots.release();             // give the reservation back, or capacity leaks
    throw e;
}
try {
    items[tail] = item;
    tail = (tail + 1) % items.length;
} finally {
    mutex.release();
}
filledSlots.release();               // 3) announce one more item
```

Way 4, `SpinBuffer.take` (the anti-pattern):

```
while (true) {
    if (Thread.interrupted()) throw new InterruptedException();  // nothing else would notice
    lock.lock();
    try {
        if (count > 0) { /* remove and return the head item */ }
    } finally {
        lock.unlock();
    }
    Thread.onSpinWait();                     // CPU hint only; the thread keeps running
}
```

`Main` runs the same workload on all four: 3 producers × 2,000 distinct numbers and 3 consumers, capacity 4, then one **poison pill** (`-1`, "stop") per consumer. It checks that nothing was lost or duplicated and that each consumer saw each producer's items in increasing order (FIFO), measures the CPU time a consumer burns while the buffer stays empty for 300 ms (`ThreadMXBean.getThreadCpuTime`), and interrupts a blocked consumer.

### Output

```
== 1. correctness: 3 producers x 2000 items, 3 consumers, capacity 4 == [ms: machine-dependent]
monitor (wait/notifyAll)           received=6000 unique=6000 FIFO=true left=0 (87 ms)
Lock + 2 Conditions                received=6000 unique=6000 FIFO=true left=0 (129 ms)
semaphores (free/filled + mutex)   received=6000 unique=6000 FIFO=true left=0 (67 ms)
busy-wait (spin + lock)            received=6000 unique=6000 FIFO=true left=0 (164 ms)

== 2. what a waiting consumer costs (buffer empty for 300 ms) == [ms: machine-dependent]
monitor (wait/notifyAll)           got 42, CPU used while waiting =   0 ms, burned a core? false
Lock + 2 Conditions                got 42, CPU used while waiting =   0 ms, burned a core? false
semaphores (free/filled + mutex)   got 42, CPU used while waiting =   0 ms, burned a core? false
busy-wait (spin + lock)            got 42, CPU used while waiting = 299 ms, burned a core? true

== 3. interrupting a consumer blocked on an empty buffer ==
monitor (wait/notifyAll)           InterruptedException, interrupt flag now false
Lock + 2 Conditions                InterruptedException, interrupt flag now false
semaphores (free/filled + mutex)   InterruptedException, interrupt flag now false
busy-wait (spin + lock)            InterruptedException, interrupt flag now false

poll(100 ms) on empty buffer -> null, waited >= 100 ms: true
```

What to notice:

- All four are *correct*; busy-waiting is wrong in cost, not in result.
- Section 2 is the interview argument: a blocked thread used 0 ms of CPU in 300 ms, the spinning one a whole core. The spin version was also the slowest overall in every run on this 4-core machine (140 ms to over 1 s depending on load, against 50–130 ms), because up to six spinners compete for four cores with the threads that could make progress.
- The flag reads `false` after the exception: throwing `InterruptedException` clears it, so code that can't rethrow must call `Thread.currentThread().interrupt()` (§14.4). The spin version must poll `Thread.interrupted()` itself, since no blocking call is there to notice.

### Pitfalls and follow-ups

- **`if` instead of `while`**: with `notifyAll`, two consumers wake for one item and the second removes from an empty buffer, corrupting `count` (demonstrated in §14.8.2).
- **Changing state without signalling** (or the reverse): every change that can make a waiter's condition true must signal under the same lock — inserts signal `notEmpty`, removals `notFull`.
- **Semaphore order.** Counting semaphore before the mutex; release the mutex before releasing the other counting semaphore. If the mutex `acquire()` is interrupted, give the slot permit back, or every interrupt shrinks the capacity for good.
- **Spinning on unprotected fields.** `while (count == 0) {}` without a lock or `volatile` may never see the update: the JIT may hoist the read out of the loop (§15.8). That is the *wrong* busy-wait; Way 4 is the correct-but-wasteful one.
- **Fairness.** None of these is fair: a newcomer can barge ahead of a long waiter. `new ReentrantLock(true)`, `new Semaphore(n, true)` and `new ArrayBlockingQueue<>(n, true)` buy FIFO hand-off with throughput.
- **Follow-up: "faster under contention."** Two locks (`putLock`, `takeLock`) and an `AtomicInteger` count let a producer and a consumer work at the same time — the `LinkedBlockingQueue` design.
- **Follow-ups: timed `offer`/`poll`** (above, with `awaitNanos`); **`drainTo`**: remove everything under one lock, then wake producers for every freed slot; **stopping consumers**: one poison pill each, or interrupt them (`shutdownNow()`).
- Related, in less depth: the two-way version in LLD §13.8.

### TL;DR

> - Ring buffer + two guarded waits ("not full", "not empty"). Monitor: `while` + `wait()` and `notifyAll()` (one shared wait set). Lock: two `Condition`s, so `signal()` suffices. Semaphores: `freeSlots(capacity)`, `filledSlots(0)`, then the mutex, in that order.
> - Busy-waiting is correct but burns a core per waiter and slows the threads doing real work — measure it and say so.

**Sources:** JDK 21 API — `Object.wait` (spurious wakeups, call in a loop), `Condition`, `Semaphore`, `ArrayBlockingQueue`, `Thread.onSpinWait`; JLS §17.2 (wait sets and notification); JEP 285 (spin-wait hints).

---

## 22.2 Implement a counting semaphore

### The problem

Implement your own `CountingSemaphore(int permits)` with `acquire()`, `release()` and `tryAcquire(timeout)`, without using `java.util.concurrent.Semaphore`. Typical extensions: `acquire(n)` for several permits at once, a **bounded** semaphore that never exceeds its initial count, and a **fair** one that serves waiters in arrival order.

Semantics to state before you start (they match the JDK class):

- The state is a single number of available permits, never negative.
- `acquire()` waits while the count is 0, then decrements it.
- `release()` increments the count and wakes waiters.
- There is **no owner**. Any thread may release, even one that never acquired. That is what separates a semaphore from a lock, and what makes it usable for signalling between threads.

### What the interviewer is testing

- Turning a spec into **state + invariant + guarded wait**: state `permits`, invariant `permits >= 0`, guard `permits >= n`.
- `notify` vs `notifyAll`, clean interrupts, and a **timed wait** with a deadline computed once and the remaining time recomputed after every wake-up.
- **Fairness**: `synchronized` + `notifyAll` give *no* ordering; how would you add one?
- Why a semaphore cannot lose a signal: its count *remembers* releases (§14.8.1).

## Approach

**Monitor version (`CountingSemaphore`).**

1. One field, `int permits`, guarded by the object's own monitor. Every method is `synchronized`.
2. `acquire(n)`: `while (permits < n) wait(); permits -= n;`.
3. `release(n)`: `permits += n; notifyAll();`.
4. `tryAcquire()` without a timeout: if `permits == 0`, return `false` immediately, otherwise decrement.
5. `tryAcquire(timeout)`: compute `deadline = now + timeout` once, then loop. If permits are available, take one. Otherwise compute `left = deadline − now`; if `left <= 0`, return `false`; else `wait(left)`. A spurious or "wrong" wake-up then simply waits again for the *remaining* time. Use `System.nanoTime()`, a **monotonic clock**: it never jumps backwards when the wall clock is corrected.

**Why `notifyAll()` and not `notify()`.** With multi-permit `acquire(n)`, waiters want different amounts. Suppose `release(1)` uses `notify()` and wakes a thread that needs 3 permits, while a thread that needed only 1 keeps sleeping. Nobody makes progress, although a waiter could have. If every waiter asks for exactly one permit, `release(1)` with `notify()` is enough, and it avoids waking N threads to let one through. Choose based on whether every waiter can use whatever was released.

**Interrupts.** `wait()` throws `InterruptedException` and the thread leaves *without* taking a permit, because we decrement only after the loop, so the count stays consistent with no extra code. JLS §17.2.4 also says a notification cannot be lost to an interrupt: after a `notify`, at least one waiter returns normally unless all of them leave by `InterruptedException`.

**Bounded variant.** Add `maxPermits`; a `release` beyond it is a caller bug (more releases than acquires), so throw `IllegalStateException`. Some textbooks instead *block* the releaser until the count drops, which turns the semaphore into a two-sided permit buffer; mention it, choose the exception.

**Fair version (`FairSemaphore`).** "Fair" means FIFO: a thread that started waiting earlier gets a permit earlier. Two things are needed:

1. A **queue of waiting threads** (`ArrayDeque<Thread>`): a thread may take a permit only at the **head** of the queue *and* with a permit free; newcomers join the tail, so nobody barges.
2. `ReentrantLock` + one `Condition`, `signalAll()` after every change so the new head re-checks (per-thread `Condition`s would avoid waking everyone — an optimisation).

`new ReentrantLock(true)` alone is **not** a fair semaphore: it orders who gets the *lock*, not the *permit*, and a fresh caller that finds a permit simply takes it.

## Code

Full files: `code/ch22/semaphore/CountingSemaphore.java`, `FairSemaphore.java`, `Main.java`.

```
public synchronized void acquire(int n) throws InterruptedException {
    checkCount(n);
    while (permits < n) {          // while, not if: spurious wakeups and barging threads
        wait();
    }
    permits -= n;
}

public synchronized boolean tryAcquire(long timeout, TimeUnit unit) throws InterruptedException {
    long deadline = System.nanoTime() + unit.toNanos(timeout);
    while (permits == 0) {
        long leftNanos = deadline - System.nanoTime();
        if (leftNanos <= 0) return false;
        long ms = TimeUnit.NANOSECONDS.toMillis(leftNanos);
        int ns = (int) (leftNanos - TimeUnit.MILLISECONDS.toNanos(ms));
        wait(ms, ns == 0 && ms == 0 ? 1 : ns);   // wait(0, 0) would mean "forever"
    }
    permits--;
    return true;
}

public synchronized void release(int n) {
    checkCount(n);
    if (permits > maxPermits - n)                 // overflow-safe form of permits + n > max
        throw new IllegalStateException("release would exceed maxPermits=" + maxPermits);
    permits += n;
    notifyAll();                  // waiters may want different n, so wake them all to re-check
}
```

`FairSemaphore.acquire` — only the head of the queue may take a permit:

```
lock.lockInterruptibly();
try {
    Thread me = Thread.currentThread();
    queue.addLast(me);
    try {
        while (queue.peekFirst() != me || permits == 0) changed.await();
        permits--;
    } finally {
        queue.remove(me);       // normal exit or interrupt: leave the queue either way
        changed.signalAll();    // the next thread in line may now be at the head
    }
} finally {
    lock.unlock();
}
```

`Main` runs seven checks: 10 workers on 3 permits; a release before anyone waits; `tryAcquire(100 ms)` while the only permit is held; `acquire(3)` fed by three single releases; an over-release on a bounded semaphore; an interrupt while waiting; FIFO order on the fair semaphore.

### Output

```
1 limit: 10 workers, 3 permits -> max inside at once = 3, permits afterwards = 3
2 signal before wait: waiter finished = true, quickly (< 500 ms) = true
3 tryAcquire(100 ms) while the permit is held -> false, waited >= 100 ms: true, tryAcquire() now -> false
4 acquire(3): woke after 3 single releases, permits left = 0
5 bounded: extra release -> IllegalStateException: release would exceed maxPermits=2
6 interrupt while waiting -> InterruptedException, permits after one release = 1
7 fair semaphore: arrival order 0 1 2 3 4 -> served order [0, 1, 2, 3, 4]
```

What to notice:

- Line 2 is the **missed-signal** test: the count remembers the early release, so the late `acquire()` returns at once (a bare `notify()` would be lost).
- Line 6: the interrupted thread took no permit.
- Line 7 is deterministic only because `Main` releases one permit at a time and waits until it is taken; with five releases at once, threads still *acquire* in FIFO order but may *print* in any order.

### Pitfalls and follow-ups

- **Release in `finally`.** Otherwise one exception loses a permit for good, and with no owner nothing detects it; enough failures and every permit is "in use" by nobody.
- **Over-release.** The JDK `Semaphore` lets the count grow; the bounded version fails loudly.
- **Deadline arithmetic.** Restarting the full timeout after each wake-up can make a 100 ms timeout last forever. Compare `nanoTime` values by subtraction (`deadline - now <= 0`), never with `<`, because `nanoTime` may overflow.
- **`wait(0)` means "forever"**, a classic bug when the remaining time rounds down to 0 ms; the code passes at least 1 ns.
- **Starvation.** An `acquire(3)` can wait forever while single-permit callers take each released permit. The fair queue fixes it; the JDK offers `new Semaphore(n, true)`, whose untimed `tryAcquire()` still barges by design.
- **Follow-up: "a lock from your semaphore"**: `new CountingSemaphore(1)` — non-reentrant (a second `acquire` by the holder deadlocks) and with no owner check.
- **Virtual threads**: on Java 21–23 a virtual thread in `wait()` inside `synchronized` **pins** its carrier (§15.5); the `ReentrantLock` version never did, and JDK 24 removed the pinning (JEP 491).

### TL;DR

- One non-negative integer under a lock: `acquire` = `while (permits < n) wait(); permits -= n`; `release` = `permits += n; notifyAll()` (waiters want different amounts).
- Timed waits from a fixed `nanoTime` deadline; an interrupted waiter takes nothing; fairness = a queue where only the head proceeds (a fair lock alone is not enough).

**Sources:** JDK 21 API — `Semaphore` (no ownership, fairness, `tryAcquire` barging), `Object.wait(long, int)`, `System.nanoTime` (compare with subtraction); JLS §17.2.4 (interaction of waits, notification and interruption); JEP 491.

---

## 22.3 Implement a read-write lock

### The problem

Implement a **read-write lock**: any number of **readers** at once, *or* exactly one **writer**, never both — `lockRead()`, `unlockRead()`, `lockWrite()`, `unlockWrite()`. Then answer the real question: **who goes first when both are waiting?** Reader preference can starve writers and writer preference can starve readers (starvation, §15.9). Show it.

### What the interviewer is testing

- **Shared/exclusive state** in two variables (`readers`, `writing`) with the right guard for each side.
- **Policy, not just safety**: why the natural first version starves writers, and the one-counter fix.
- **Who wakes whom**: the last reader out wakes writers; a leaving writer wakes everyone.
- **The interrupt edge case**: a waiting writer that gives up must undo its "waiting" mark.
- The JDK's `ReentrantReadWriteLock`: fairness, reentrancy, no upgrade.

### Approach

**State** (all guarded by the object monitor):

- `int readers`: readers currently inside.
- `boolean writing`: whether a writer is inside.
- `int waitingWriters`: writers blocked in `lockWrite()`. Only the writer-preference version needs it.

**Invariant:** `writing` implies `readers == 0`, and at most one writer is inside.

**Reader preference.**

1. `lockRead`: `while (writing) wait(); readers++;`. A reader enters whenever no writer is *inside*.

2. `unlockRead`: `readers--; if (readers == 0) notifyAll();`. Only the last reader out can let a writer in, so earlier ones need not wake anybody.
3. `lockWrite`: `while (writing || readers > 0) wait(); writing = true;`.
4. `unlockWrite`: `writing = false; notifyAll();`. Wake *all* waiting readers, so they enter together, and the writers.

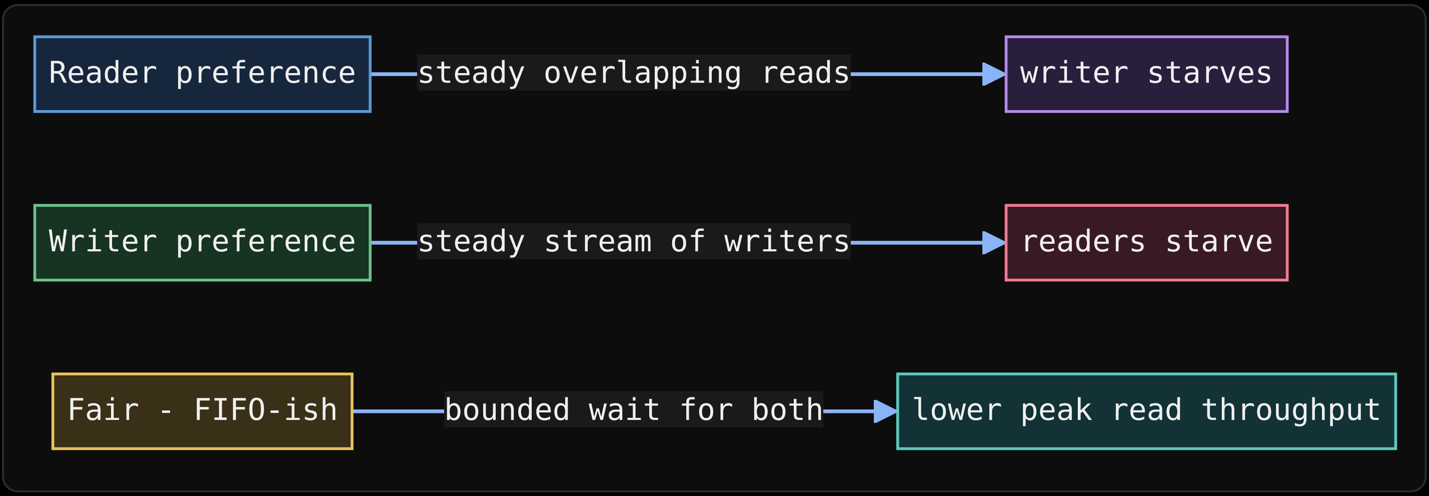
Why writers starve: if reads overlap (six readers, 30 ms each, staggered starts), `readers` never drops to 0, so a writer waits until the stream of reads stops.

**Writer preference** — the one-line change:

1. `lockWrite`: `waitingWriters++`, wait as before, then `waitingWriters--` and set `writing = true`.
2. `lockRead`: `while (writing || waitingWriters > 0) wait();`. A new reader stays out as soon as a writer is *waiting*.

Readers inside finish, the count reaches 0, the writer enters. The opposite failure follows: writers arriving back to back keep `waitingWriters > 0` and readers starve.

**Fair** is the third answer: roughly arrival order. A waiting writer blocks *later* readers only; when it leaves, the readers that queued meanwhile go in together. That is `new ReentrantReadWriteLock(true)`: on release, either the longest-waiting writer gets the lock, or the group of readers that waited longer than every writer. By hand it needs a ticket queue as in `FairSemaphore` (§22.2); in the interview, describing it and naming the JDK class is usually enough.


Reader preference
steady overlapping reads
writer starves
Writer preference
steady stream of writers
readers starve
Fair - FIFO-ish
bounded wait for both
lower peak read throughput

### Code

Full files: `code/ch22/rwlock/` — `SimpleRWLock` (interface), `ReaderPreferenceRWLock`, `WriterPreferenceRWLock`, `Main` (which also wraps the JDK fair lock as a third contestant).

`WriterPreferenceRWLock` (the reader-preference class is the same without `waitingWriters`):

```
public synchronized void lockRead() throws InterruptedException {
    while (writing || waitingWriters > 0) wait();   // the one-line difference
    readers++;
}

public synchronized void unlockRead() {
    if (readers <= 0) throw new IllegalMonitorStateException("no read lock held");
    readers--;
    if (readers == 0) notifyAll();
}

public synchronized void lockWrite() throws InterruptedException {
    waitingWriters++;
    try {
        while (writing || readers > 0) wait();
    } catch (InterruptedException e) {
        waitingWriters--;          // give up our place, or readers stay blocked forever
        notifyAll();               // readers held back by us must re-check
        throw e;
    }
    waitingWriters--;
    writing = true;
}

public synchronized void unlockWrite() {
    if (!writing) throw new IllegalMonitorStateException("no write lock held");
    writing = false;
    notifyAll();
}
```

`Main` runs three experiments on each lock: (1) **safety** — 4 readers and 2 writers, 2,000 operations each; writers keep `pair[0] == pair[1]`, readers count torn pairs or overlap with a writer; (2) a writer arriving at 100 ms while six readers overlap for 600 ms; (3) a reader arriving at 100 ms while two writers alternate for 600 ms with a 1 ms gap.

### Output

```
== 1. safety: 4 readers + 2 writers, 2000 operations each ==
reader-preference                  violations=0  pair=(4000,4000)  readers overlapped (max>1)=true
writer-preference                  violations=0  pair=(4000,4000)  readers overlapped (max>1)=true
JDK ReentrantReadWriteLock(fair)   violations=0  pair=(4000,4000)  readers overlapped (max>1)=true

== 2. writer arrives while 6 readers keep overlapping for 600 ms == [ms: nondeterministic]
reader-preference                  writer waited 505 ms, starved (>= 400 ms)? true
writer-preference                  writer waited  27 ms, starved (>= 400 ms)? false
JDK ReentrantReadWriteLock(fair)   writer waited  16 ms, starved (>= 400 ms)? false

== 3. reader arrives while 2 writers keep alternating (1 ms gap) for 600 ms == [ms: nondeterministic]
reader-preference                  reader waited  22 ms, starved (>= 400 ms)? false
writer-preference                  reader waited 521 ms, starved (>= 400 ms)? true
JDK ReentrantReadWriteLock(fair)   reader waited  25 ms, starved (>= 400 ms)? false
```

What to notice:

- All three are **safe**: starvation is a *liveness* failure (someone never gets in), not a *safety* one (nothing is ever corrupted).
- Experiment 2: under reader preference the writer got in only when the readers stopped (~500 ms); otherwise it waited about one read (≤ 30 ms). Experiment 3 mirrors it: under writer preference one writer is always waiting, so the reader is locked out all run.
- The 1 ms gap: without it the releasing writer re-acquires the lock before any woken thread runs, and even reader preference starved the reader. That is **barging**, a third source of starvation: `notifyAll()` gives woken threads no priority over a running one.

### Pitfalls and follow-ups

- **Waking too few.** `unlockWrite` needs `notifyAll`: with `notify`, one reader enters and the others sleep although they could share.

- **Forgetting `waitingWriters--` on interrupt** blocks every future reader — the subtle bug of the writer-preference version.
- **Unlock checks.** Unlocking a lock you don't hold should throw `IllegalMonitorStateException`, not drive `readers` negative; the JDK tracks per-thread read-hold counts.
- **Reentrancy.** These locks aren't reentrant: in the writer-preference lock, a reader re-entering `lockRead()` while a writer waits deadlocks with it. `ReentrantReadWriteLock` is reentrant.
- **Upgrade deadlock.** Two readers that both try to take the write lock wait for each other forever. The JDK forbids upgrading and allows **downgrading** (write lock → read lock → release write); `StampedLock.tryConvertToWriteLock` covers "read, then maybe write" (§14.9).
- **When it doesn't pay.** With short reads, the bookkeeping (every reader writes `readers` under a lock) costs more than plain mutual exclusion; RW locks win for long, frequent reads and rare writes (LLD §12.23).
- **Follow-up: two `Condition`s** (`canRead`, `canWrite`): `unlockWrite` signals one writer if any waits, else `canRead.signalAll()`, so writers aren't woken for nothing.

### TL;DR

- State `readers`, `writing`, plus `waitingWriters` for writer preference. Readers wait while a writer is inside (or waiting); writers wait while anyone is inside.
- Reader preference starves writers, writer preference starves readers; fair hand-off (`new ReentrantReadWriteLock(true)`) bounds both. Wake everyone when a writer leaves, undo `waitingWriters` on interrupt, never upgrade.

**Sources:** JDK 21 API — `ReentrantReadWriteLock` (fair and non-fair acquisition, reentrancy, downgrading allowed and upgrading not), `ReadWriteLock` (when a read-write lock helps), `StampedLock`; JLS §17.2.

---

## 22.4 Implement a reusable barrier

### The problem

Implement a **barrier** for `N` threads: each calls `await()`, nobody returns until all `N` have, then all are released together. It must be **reusable** (cyclic) for round 2, 3, … of an iterative algorithm, such as a simulation where no thread may start step $k + 1$ until every slice of step $k$ is done. The JDK class is `CyclicBarrier` (§14.13); the interview wants it built, and an explanation of why the obvious version breaks on second use.

### What the interviewer is testing

- Seeing that **"N have arrived" and "my round is over" are different facts**; a count that gets reset can't express the second.
- The **generation** number waiters compare against, so a fast thread can return for round 2 while slow ones are still leaving round 1.
- Interrupt semantics, a **barrier action** (run once per round by the last arriver) and the return value of `await`.

### Approach

**Naive attempt A: never reset.**

```
count++; if (count == N) notifyAll(); else while (count < N) wait();
```

Round 1 works. In round 2 `count` is already ≥ N, so `while (count < N)` is false immediately and every thread runs straight through without waiting. The barrier has silently stopped being a barrier.

**Naive attempt B: reset in the last thread.**

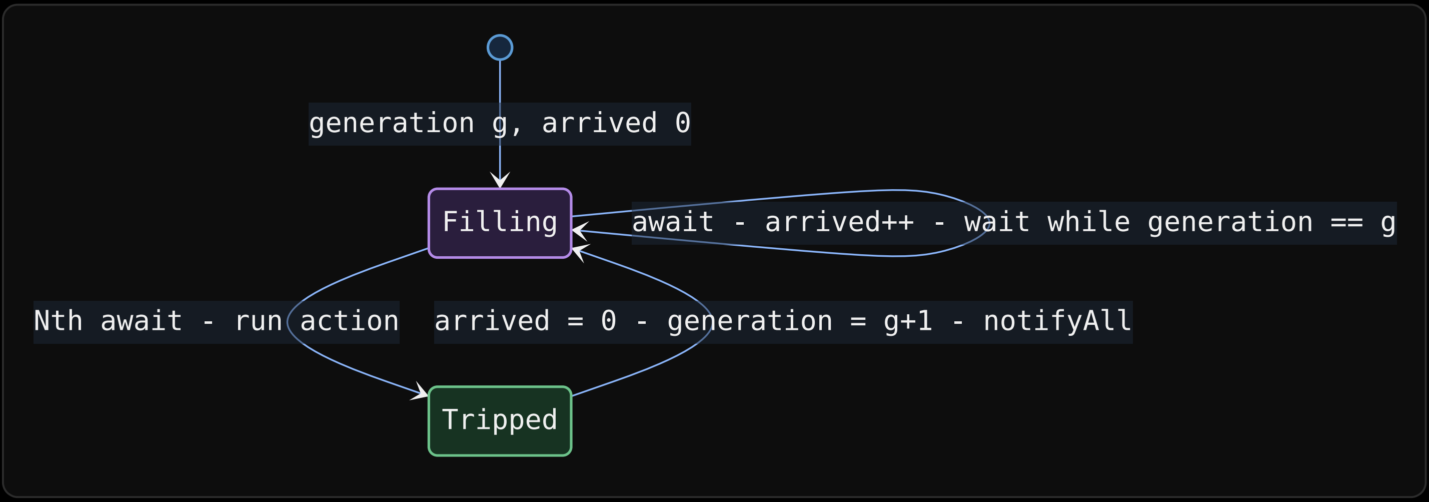
```
count++; if (count == N) { count = 0; notifyAll(); } else while (count < N) wait();
```

The last thread resets `count` to 0 and calls `notifyAll()`. The woken threads re-check `count < N`, see `0 < N`, and go back to sleep: everyone except the last thread is stuck in round 1 forever. The `while` loop, correct elsewhere, now re-checks a condition that has *already been undone*.

The problem underneath both attempts is that a waiter asks "have N arrived?", while the question it needs answered is "**is the round I arrived in over?**". Once the count is reset for the next round, it can no longer answer that.

**The generation barrier.**

1. State: `arrived` (arrivals so far in this round) and `generation` (a `long` round number), both guarded by the monitor.
2. On `await()`, record `myGeneration = generation`, then `arrived++`.
3. **Last arriver** (`arrived == N`): run the optional barrier action, set `arrived = 0` for the next round, `generation++` to end this one, `notifyAll()`, and return.
4. **Everyone else**: `while (myGeneration == generation) wait();`. Spurious wake-ups loop back. The real wake-up finds that the generation has changed and returns.
5. Reuse is now safe. A fast thread that comes back for round 2 increments the fresh `arrived` and records the *new* generation, while slow threads from round 1 are still leaving. They compare against *their* generation, which has already ended.


generation g, arrived 0
Filling
await - arrived++ - wait while generation == g
Nth await - run action
arrived = 0 - generation = g+1 - notifyAll
Tripped

**Interrupts.** Interrupted *before* its round completes, a waiter must withdraw its arrival (`arrived--`), or the round would trip with N−1 real parties. If the round completed at the same moment, it has passed: return normally and restore the interrupt flag. `CyclicBarrier` chooses differently: an interrupt **breaks** the barrier and every other waiter gets `BrokenBarrierException`. Name both; our code withdraws.

**Return value.** As in `CyclicBarrier`, `await()` returns the arrival index (`N-1` for the first, `0` for the last), so one thread can do per-round work without a barrier action.

### Code

Full files: `code/ch22/barrier/ReusableBarrier.java`, `NaiveBarriers.java` (the two broken versions), `Main.java`.

```java
public synchronized int await() throws InterruptedException {
    final long myGeneration = generation;
    int index = parties - 1 - arrived;
    arrived++;
    if (arrived == parties) {           // last one in trips the barrier
        if (barrierAction != null) barrierAction.run();
        arrived = 0;                    // reset for the next round...
        generation++;                   // ...and end this one
        notifyAll();
        return 0;
    }
    try {
        while (myGeneration == generation) {   // not "arrived < parties"!
            wait();
        }
    } catch (InterruptedException e) {
        if (myGeneration == generation) {       // still waiting: withdraw our arrival
            arrived--;
            throw e;
        }
        Thread.currentThread().interrupt();     // the round already completed: keep the interrupt, return normally
    }
    return index;
}
```

`Main` checks: (1) correct reuse — 3 workers × 4 rounds with random work, each verifying after `await()` that all 3 had arrived; (2) naive A with one thread 200 ms late for round 2; (3) naive B on daemon threads, so stuck ones don't keep the JVM alive; (4) a lone waiter interrupted, then a fresh full round.

### Output

```
1 barrier action: round 0 complete, all 3 arrived
1 barrier action: round 1 complete, all 3 arrived
1 barrier action: round 2 complete, all 3 arrived
1 barrier action: round 3 complete, all 3 arrived
1 generation barrier: 4 rounds, threads that left a round early = 0
2 NoReset: round 2, fast threads that passed before the slow one arrived = 2 of 2  (BUG)
3 ResetTooEarly: round 1, threads still stuck after 300 ms = 2 of 3  (BUG)
4 interrupted waiter -> InterruptedException, waiting count after withdraw = 0, next full round completed = true
```

What to notice: the barrier action prints exactly once per round, in order, because it runs inside the monitor in the last arriver before anyone is released. Naive A fails *silently* (occasional wrong numbers in a real algorithm); naive B hangs. Both survive a single-round test — which is why interviewers ask for reuse.

### Pitfalls and follow-ups

- **Testing one round only.** Test at least two rounds with uneven arrivals.
- **`int` generation wrap-around** is harmless (only "changed or not" matters); a `long` makes it moot.
- **A throwing barrier action** escapes with `arrived` still at N and no `notifyAll()`: everyone is stuck. Wrap it in `try/finally` and still advance the generation, or break the barrier as `CyclicBarrier` does.
- **Timeouts.** `await(timeout)` is the same loop with a deadline (as in §22.2); on timeout withdraw, or break the barrier (`CyclicBarrier` throws `TimeoutException` to that thread and `BrokenBarrierException` to the others).
- **Alternative: two turnstiles.** The semaphore solution opens gate 1 when N have arrived and gate 2 when N have *left*, which makes it safe to close gate 1 again; correct but harder to explain.
- **Follow-ups:** a varying number of parties is `Phaser` (§14.13); a latch is one-shot and its counters need not be its waiters, while a barrier is cyclic and arrivers are the waiters.

### TL;DR

- Waiters must ask "is *my* round over?": loop `while (myGeneration == generation) wait()`; the last arriver resets the count, increments the generation and calls `notifyAll()`.

- No reset passes everyone in round 2; a reset with a count-based wait strands them in round 1. On interrupt, withdraw (ours) or break the barrier ( `CyclicBarrier` ).

**Sources:** JDK 21 API — `CyclicBarrier` (barrier action, arrival index, broken-barrier rules), `Phaser`, `CountDownLatch`; JLS §17.2.

---

## 22.5 Thread-safe deferred callback executor

### The problem

Design `schedule(Runnable callback, long delay, TimeUnit unit)`, callable from any thread. Each callback runs **no earlier than** `now + delay`, as soon as possible after, on one dedicated **worker thread**. A callback scheduled later may be due *earlier* than everything waiting. The worker must not **poll** (wake every few ms to look); it sleeps exactly until the next deadline. In production: `ScheduledThreadPoolExecutor` or `DelayQueue` (§14.11).

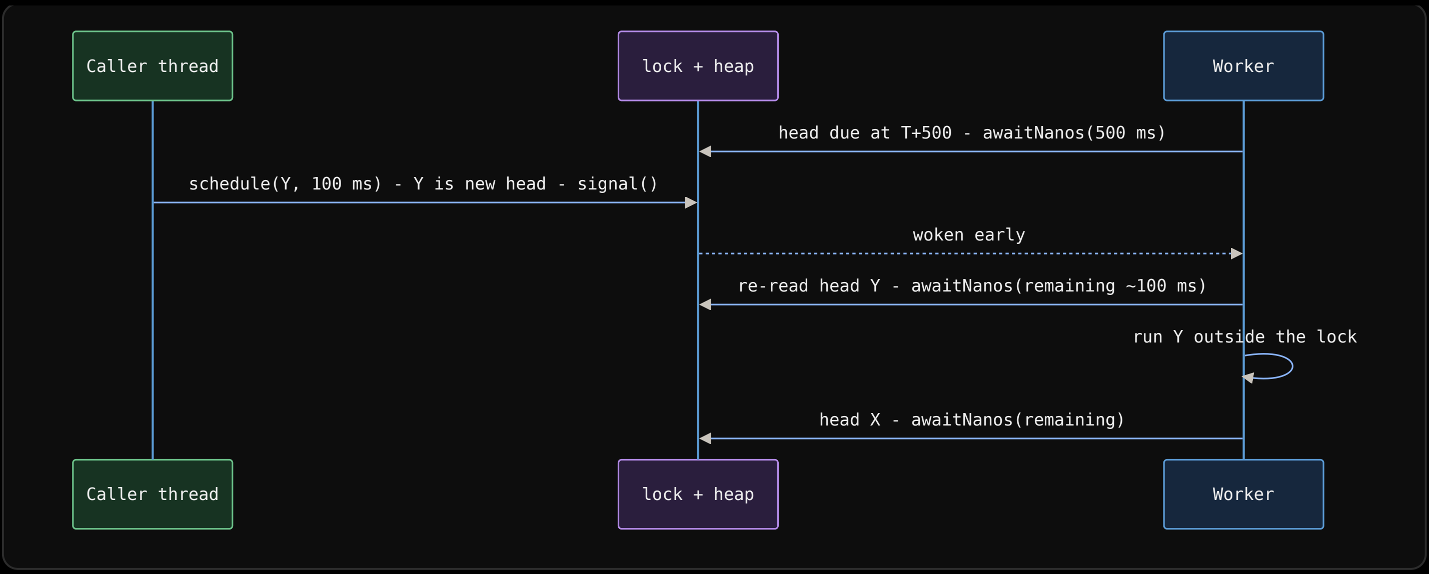
### What the interviewer is testing

- **Data structure**: a **min-heap** keyed by deadline — O(log n) insert, O(1) "next due".
- **A timed wait on a moving target**: the earliest deadline can change while the worker sleeps. Do you wake it and re-read the head?
- **Missed signals**: a signal sent while the worker is running a callback wakes nobody. Still correct?
- Callbacks **outside the lock**, surviving one that throws, **clean shutdown**, and `nanoTime` vs `currentTimeMillis`.

### Approach

1. **Task record** `(deadlineNanos, seq, callback)` with `deadlineNanos = System.nanoTime() + delay`: `nanoTime` is monotonic, so a wall-clock correction (NTP) can't fire a callback early or hours late. `seq` (from an `AtomicLong`) breaks ties in scheduling order.
2. **`PriorityQueue<Task>`** (not thread-safe) guarded by a `ReentrantLock`; one `Condition`, `changed`, means "the head may have changed, or shutdown was requested".
3. **`schedule()`**: under the lock, add the task; if it is now the head (`queue.peek() == task`), `changed.signal()` so the worker recomputes its sleep. Otherwise the worker's sleep target is still right.
4. **Worker loop** (`takeNextDue()`, under the lock):
    - empty queue → `changed.await()` (no timeout: nothing to do until `schedule` or `close`);
    - head due (`deadline − now <= 0`) → `poll()` it and return its callback;
    - head not yet due → `changed.awaitNanos(deadline − now)`, a **timed wait on the earliest deadline**, then loop and **re-read the head**, whatever woke us: timeout, signal or spurious wake-up.
5. **Run the callback after releasing the lock**, in `try/catch (RuntimeException)`: a slow callback must not block `schedule()`, and a throwing one must not kill the only worker.
6. **`close()`**: set `shutdown = true` under the lock and signal; later `schedule()` calls throw `IllegalStateException`; the worker runs the queued tasks as they fall due, exits when the queue is empty, and `close()` joins it. (Dropping pending tasks, `shutdownNow`-style, is the other valid choice; say which.)

**Why the signal cannot be missed.** The worker waits *for a condition on the queue*, re-read under the lock before every wait, never "for a signal". If `schedule()` adds an earlier task while the worker runs a callback, the `signal()` wakes nobody, but the worker sees the new head the next time it takes the lock. The heap carries the information; the signal only shortens a sleep in progress.


Caller thread
lock + heap
Worker
head due at T+500 - awaitNanos(500 ms)
schedule(Y, 100 ms) - Y is new head - signal()
woken early
re-read head Y - awaitNanos(remaining ~100 ms)
run Y outside the lock
head X - awaitNanos(remaining)
Caller thread
lock + heap
Worker

### Code

Full files: `code/ch22/deferredcallback/DeferredCallbackExecutor.java`, `Main.java`.

```java
private record Task(long deadlineNanos, long seq, Runnable callback) implements Comparable<Task> {
    public int compareTo(Task o) {
        int c = Long.compare(deadlineNanos - o.deadlineNanos, 0);   // overflow-safe nanoTime compare
        return c != 0 ? c : Long.compare(seq, o.seq);              // tie -> first scheduled runs first
    }
}

public void schedule(Runnable callback, long delay, TimeUnit unit) {
    Task task = new Task(System.nanoTime() + unit.toNanos(delay), seq.getAndIncrement(), callback);
    lock.lock();
    try {
        if (shutdown) throw new IllegalStateException("executor is shut down");
        queue.add(task);
        if (queue.peek() == task) changed.signal();   // new earliest deadline: worker must recompute its sleep
    } finally {
        lock.unlock();
    }
}

private Runnable takeNextDue() throws InterruptedException {     // called by the worker loop
    lock.lock();
    try {
        while (true) {
            Task head = queue.peek();
            if (head == null) {
                if (shutdown) return null;
                changed.await();                            // nothing to do: sleep until schedule()/close()
                continue;
            }
            long waitNanos = head.deadlineNanos() - System.nanoTime();
            if (waitNanos <= 0) return queue.poll().callback();   // due now
            changed.awaitNanos(waitNanos);              // loop re-reads the head whatever woke us
        }
    } finally {
        lock.unlock();
    }
}
```

The worker loop calls `takeNextDue()`, then runs the callback outside the lock in `try { due.run(); } catch (RuntimeException e) { log }`.

`Main` checks: (1) four threads schedule A (400 ms), B (100), C (300), D (200) at once; (2) Y (due 150 ms) arrives while the worker sleeps until X (500 ms); (3) a callback throws, then `close()` with one task pending.

**Output**

```
1 scheduled A(400) B(100) C(300) D(200) from 4 threads -> ran B D C A, none ran before its deadline: true
   B due ~119 ms, ran at 121 ms [nondeterministic]
   D due ~220 ms, ran at 220 ms [nondeterministic]
   C due ~320 ms, ran at 320 ms [nondeterministic]
   A due ~415 ms, ran at 418 ms [nondeterministic]
2 X due at 500 ms, Y (due 150 ms) scheduled while worker sleeps -> first ran: Y, Y ran between 150 and 300 ms:
true, then X
   [worker] callback threw java.lang.IllegalStateException: boom - worker keeps going
3 callbacks that ran after the throwing one: [after-boom, pending-at-close]
3 schedule() after close() -> IllegalStateException: executor is shut down
```

What to notice: lateness is 0–5 ms (timer resolution and wake-up time) — the guarantee is *not early*, never *exactly on time*. Line 2 is the core: without the `signal()` in `schedule()`, Y would run at ~500 ms, after X. `close()` drained `pending-at-close` before stopping.

### Pitfalls and follow-ups

- **Sleeping for the head's delay, then running whatever is at the head** without re-checking: an earlier task inserted meanwhile runs late, a spurious wake-up runs a task early. Always re-read, recompute, wait again.
- **`Thread.sleep(delay)`** can't be shortened by a newly scheduled earlier task; **polling** (`awaitNanos(10 ms)` in a loop) wakes 100 times a second and is late by up to the interval.
- **Callbacks under the lock**: a 2 s callback blocks every `schedule()` caller for 2 s. **Uncaught callback exceptions** would end the only worker silently (`ScheduledThreadPoolExecutor` captures them in the task's `Future`).
- **One slow callback delays the rest.** Follow-up: the worker only *dispatches* to an `ExecutorService`, or several workers use the **leader–follower** scheme of `DelayQueue` (its source comment names it): one leader does the timed wait on the head, the others wait untimed, and the leader signals a follower when it takes a task — otherwise every worker wakes at every deadline.
- **Starting the worker in the constructor** lets `this` escape; it is safe here only because the class is `final` and `start()` is the last statement (`Thread.start()` happens-before the new thread's actions). Otherwise use a factory (§22.8).
- **Cancellation (follow-up)**: return a handle; remove the task under the lock (O(n)) or mark it and skip it at the head (O(1)) — `ScheduledThreadPoolExecutor.setRemoveOnCancelPolicy` chooses between the two.
- **Huge delays** overflow `nanoTime() + delay`; compare by subtraction and cap the delay.

### TL;DR

- Min-heap of `(deadline, seq, callback)` behind one lock; one worker loops: peek, run if due, else `awaitNanos(deadline − now)`. `schedule()` signals only for a new head; the worker re-reads the head after every wake-up, so state carries the information.
- `nanoTime` deadlines, callbacks outside the lock with exceptions caught, defined shutdown. Production: `ScheduledThreadPoolExecutor` or `DelayQueue`.

**Sources:** JDK 21 API — `Condition.awaitNanos` (returns the remaining time), `PriorityQueue` (not synchronized, O(log n) offer/poll), `DelayQueue`, `ScheduledThreadPoolExecutor` (`setRemoveOnCancelPolicy`), `System.nanoTime`; JLS §17.4.5 (`Thread.start` happens-before rule).

---

## 22.6 Asynchronous to synchronous

### The problem

A library offers only an **asynchronous**, callback-based API:

```
void fetch(String key, Callback<String> cb);   // returns at once; later calls cb.onSuccess(v) or cb.onFailure(e)
```

The callback runs on the library's own I/O thread. Your callers want a **synchronous** (blocking) method:

```
String fetch(String key, Duration timeout) throws InterruptedException, ExecutionException, TimeoutException;
```

It blocks until the result or error arrives, then returns it or throws (sync vs async, §14.1.2). Implement it and compare `wait`/`notify`, `Lock` + `Condition`, `CountDownLatch` and `CompletableFuture`.

### What the interviewer is testing

- The **missed-signal trap**: the callback may run *before* the caller waits — even synchronously inside `fetch()` for a cached value — so a bare `notify()` is lost (§14.8.3).
- **Per-call state**, and **safe publication** of the result from the I/O thread (a happens-before edge, §15.8).
- **Errors, timeouts, interrupts**, and a library that calls back twice.
- The design trap: calling the blocking wrapper *from* the library's callback thread **deadlocks** it.

### Approach

Every correct version has the same shape:

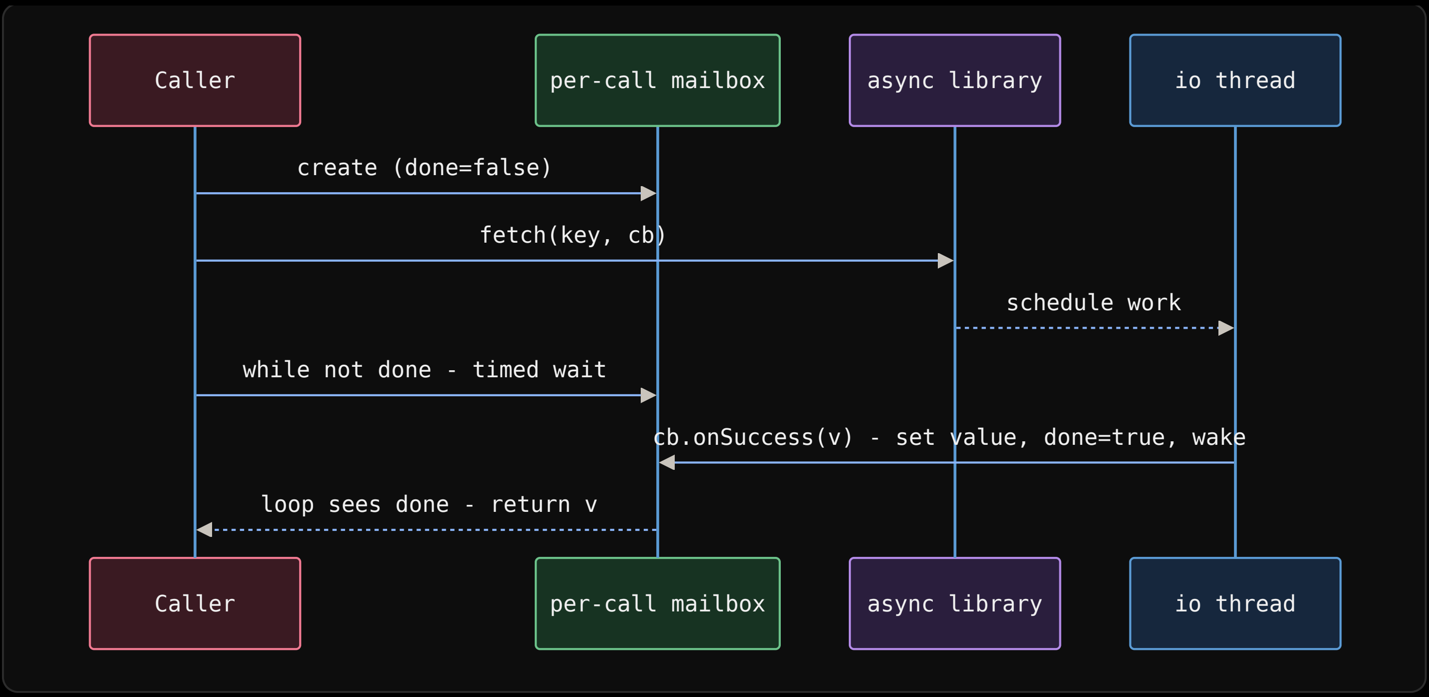
1. **Create a fresh "mailbox" per call**: a small object holding `done`, `value` and `error`.
2. **Register a callback that completes the mailbox**: record the outcome *in state*, then wake the caller. If the mailbox is already complete, ignore the call, so that the first completion wins and a second callback is harmless.
3. **Call the async API.**
4. **Wait on the state**, not on a signal: `while (!done)` with a deadline. If the callback already ran, the loop never waits.
5. **Return the value**, or throw `ExecutionException(cause)` for a failure, or `TimeoutException`.

The four tools differ only in how steps 2 and 4 are written:

| Tool | Mailbox | Wait | Notes |
|---|---|---|---|
| `wait` / `notifyAll` | fields guarded by the mailbox's monitor | `while (!done) TimeUnit.NANOSECONDS.timedWait(box, left)` | most code; you write the deadline loop yourself |
| `Lock` + `Condition` | fields guarded by a per-call lock | `while (!done) nanos = finished.awaitNanos(nanos)` | interruptible lock acquisition; no virtual-thread pinning on Java 21 |
| `CountDownLatch(1)` | `AtomicReference<Outcome>` + latch | `latch.await(timeout)` | the latch *is* the done flag; `compareAndSet` makes the first completion win |
| `CompletableFuture` | the future itself | `future.get(timeout)` | shortest; `complete` ignores later calls; failures arrive as `ExecutionException` |

The latch and the future need no lock: `countDown()` happens-before a successful `await()` return, and completing a `CompletableFuture` happens-before `get()` returns, so the I/O thread's result is visible.

**Preferred answer:** wrap the callback API once as `fetchAsync(key)` returning a `CompletableFuture`; the blocking version is `fetchAsync(key).get(timeout)`, and callers that can stay asynchronous compose (`thenCombine`, `thenApply`) and block nobody. The monitor, `Condition` and latch versions show you can derive it from first principles.


Caller
per-call mailbox
async library
io thread
create (done=false)
fetch(key, cb)
schedule work
while not done - timed wait
cb.onSuccess(v) - set value, done=true, wake
loop sees done - return v
Caller
per-call mailbox
async library
io thread

### Code

Full files: `code/ch22/asynctosync/AsyncService.java` (the simulated library, with one key per edge case), `SyncWrappers.java` (all four wrappers), `Main.java`.

The monitor version (`MonitorSync`):

```java
private static final class Mailbox {          // one per call; its monitor guards its fields
    boolean done;
    String value;
    Throwable error;

    synchronized void complete(String v, Throwable e) {
        if (done) return;                      // first completion wins
        value = v;
        error = e;
        done = true;                           // the STATE that makes a missed signal impossible
        notifyAll();
    }
}

public String fetch(String key, Duration timeout) throws InterruptedException, ExecutionException, TimeoutException
{
    Mailbox box = new Mailbox();
    service.fetch(key, new AsyncService.Callback<>() {
        public void onSuccess(String v) { box.complete(v, null); }
        public void onFailure(Throwable e) { box.complete(null, e); }
    });
    long deadline = System.nanoTime() + timeout.toNanos();
    synchronized (box) {
        while (!box.done) {                    // checks state first: fine if the callback already ran
            long left = deadline - System.nanoTime();
            if (left <= 0) throw new TimeoutException("no reply within " + timeout.toMillis() + " ms");
            TimeUnit.NANOSECONDS.timedWait(box, left);
        }
        if (box.error != null) throw new ExecutionException(box.error);
        return box.value;
    }
}
```

The latch and future versions (`LatchSync`, `FutureSync`), core lines:

```
// CountDownLatch
CountDownLatch latch = new CountDownLatch(1);
AtomicReference<Outcome> result = new AtomicReference<>();
service.fetch(key, new AsyncService.Callback<>() {
    public void onSuccess(String v) { if (result.compareAndSet(null, new Outcome(v, null))) latch.countDown(); }
    public void onFailure(Throwable e) { if (result.compareAndSet(null, new Outcome(null, e))) latch.countDown(); }
});
if (!latch.await(timeout.toNanos(), TimeUnit.NANOSECONDS)) throw new TimeoutException(...);

// CompletableFuture: the reusable bridge, then block only if the caller must
public CompletableFuture<String> fetchAsync(String key) {
    CompletableFuture<String> f = new CompletableFuture<>();
    service.fetch(key, new AsyncService.Callback<>() {
        public void onSuccess(String v) { f.complete(v); }               // later calls return false
        public void onFailure(Throwable e) { f.completeExceptionally(e); }
    });
    return f;
}
// fetch(key, timeout) = fetchAsync(key).get(timeout); on TimeoutException also f.cancel(false)
```

`Main` runs every wrapper against every edge case (`slow` = success after 150 ms, `fail` = `onFailure`, `cached` = callback runs *synchronously* inside `fetch()`, `never` = no callback, `twice` = duplicate callbacks), then 50 concurrent callers, an interrupt while blocked, a blocking call made *from* the I/O thread, and composition without blocking.

### Output

```
== 1. every wrapper against every edge case (timeout 300 ms) ==
1 slow    -> value-of-slow   (same for all 4 wrappers)
1 fail    -> ExecutionException(connection reset)   (same for all 4 wrappers)
1 cached  -> value-of-cached   (same for all 4 wrappers)
1 never   -> TimeoutException   (same for all 4 wrappers)
1 twice   -> first   (same for all 4 wrappers)

== 2. 50 threads call the latch wrapper at once, each with its own key ==
2 callers that got exactly their own result: 50 of 50

== 3. interrupting a caller that is blocked waiting ==
3 blocked caller interrupted -> InterruptedException, left within 100 ms (not after the 10 s timeout): true

== 4. calling the blocking wrapper FROM a callback on the io thread ==
4 sync call from the io thread -> TimeoutException (would hang forever without a timeout)

5 no blocking needed when the caller can compose: value-of-k1+value-of-k2
```

What to notice:

- `cached` passes everywhere because each wrapper records completion in state (`done`, the latch count, the future) before waking anyone; a `notify()`-only version hangs here.
- `twice` → `first`: the second completion is ignored, so a buggy library can't change an answer already returned (or, with a reused mailbox, deliver it to the *next* call).
- Check 4 is the **self-deadlock**: a callback on the only I/O thread called the blocking wrapper, whose reply needs that same thread. Only the timeout saved it; without one, every request in the application would stall.

### Pitfalls and follow-ups

- **Signal without state** — the missed-signal bug the problem is built around. **Shared mailbox** — concurrent callers overwrite each other (check 2). **Bare fields** written by the I/O thread — the caller may never see `value`, or see `done` before it.
- **No timeout**: a lost request (`never`) blocks forever. On timeout make the late callback harmless (mailbox already done, or `cancel` the future).
- **Failures**: throw `ExecutionException(cause)` to keep the original stack trace; never return `null`. **Interrupts**: propagate (§14.4) and cancel the library call if it can be cancelled.

- **Blocking on the callback or event-loop thread** (check 4): chain with `thenApply`/`thenCompose` there instead of `get()`/`join()`.
- **Follow-up: thousands of callers.** Run them on **virtual threads** (§15.4): `CountDownLatch.await`, `Condition.await` and `CompletableFuture.get` unmount cheaply; the `synchronized`/`wait` version pins the carrier before JDK 24 (JEP 491).
- **Follow-up: sync to async** is `CompletableFuture.supplyAsync(() -> blockingCall(), executor)` — some thread still blocks; you choose which.

### TL;DR

- A **per-call mailbox**: the callback records the outcome *in state* (first wins) and wakes the caller, who waits `while (!done)` with a deadline and returns or throws `ExecutionException` / `TimeoutException`. `CountDownLatch` and `CompletableFuture` give state, wake-up and happens-before for free; best is `fetchAsync()` + `get(timeout)` only where needed.
- Traps: callback before the wait, shared result fields, no timeout, blocking on the library's callback thread.

**Sources:** JDK 21 API — `CountDownLatch` (memory-consistency effects), `CompletableFuture` (`complete`, `get(long, TimeUnit)`, `cancel`), `TimeUnit.timedWait`, `Condition`, `java.util.concurrent` package summary (memory visibility guarantees); JLS §17.4.5; JEP 444 (virtual threads), JEP 491.

---

## 22.7 Ordered printing (three threads print in order)

### The problem

One object has three methods, `first()`, `second()` and `third()`. Three different threads call them, one method each, and the threads may arrive in **any** order. Whatever the arrival order, the output must be `first second third`. (This is LeetCode 1114, "Print in Order".) Each method receives a `Runnable` that does the actual printing, so your job is purely the **ordering**: deciding when each thread may go.

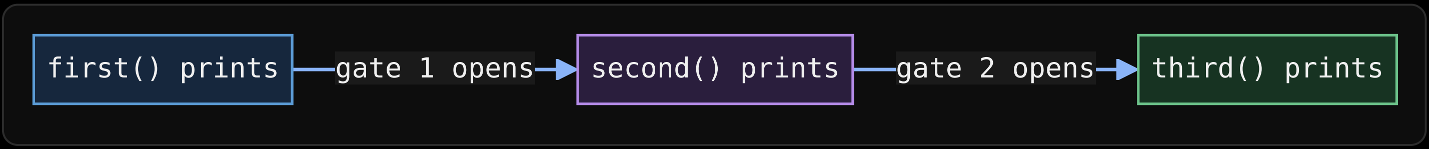
### What the interviewer is testing

- Seeing a **signalling** problem, not mutual exclusion: nothing is protected; one thread tells another "you may go" (LLD §12.20).
- A signal that survives being sent **before** anyone waits (no missed signal, §14.8.3).
- Comparing a latch, a semaphore, a monitor and a spin loop.

### Approach

Draw the dependencies: `second` waits for `first`, and `third` waits for `second`. That makes two **gates**, and each gate is opened exactly once. Any primitive that **remembers** that it was opened will work:

1. **`CountDownLatch(1)` per gate** (§14.13): `first()` prints then `countDown()`; `second()` `await()`s first. A latch at zero stays open, so a late waiter passes straight through. Reads most like the problem statement.
2. **`Semaphore(0)` per gate**: a gate that starts closed; `release()` stores a permit even if nobody waits yet. Unlike a latch it is reusable, which matters in looping variants such as FooBar (§22.13).
3. **A monitor** (§14.8) guarding `int stage`: each method waits `while (stage != myStage)`, prints, increments `stage`, `notifyAll()`. The state makes the signal durable: a late thread checks `stage` before waiting.
4. **A `volatile` stage with busy-waiting** (§14.7): correct, since only the thread whose turn it is writes `stage` (no CAS needed), but each waiter burns a core; only for microsecond waits.



### Code

`code/ch22/orderedprinting/OrderedPrinters.java` (two of the four solutions):

```java
public static final class LatchPrinter implements Printer {
    private final CountDownLatch firstDone = new CountDownLatch(1);
    private final CountDownLatch secondDone = new CountDownLatch(1);

    public void first(Runnable p) { p.run(); firstDone.countDown(); }
    public void second(Runnable p) throws InterruptedException {
        firstDone.await();                 // returns immediately if first() already finished
        p.run();
        secondDone.countDown();
    }
    public void third(Runnable p) throws InterruptedException { secondDone.await(); p.run(); }
}

public static final class MonitorPrinter implements Printer {
    private int stage = 1;                 // whose turn it is; guarded by 'this'

    private synchronized void runAt(int myStage, Runnable p) throws InterruptedException {
        while (stage != myStage) wait();   // while, not if: spurious and wrong-turn wakeups
        p.run();
        stage++;
        notifyAll();                       // wake everyone; the right thread sees its stage
    }
    public void first(Runnable p) throws InterruptedException  { runAt(1, p); }
    public void second(Runnable p) throws InterruptedException { runAt(2, p); }
    public void third(Runnable p) throws InterruptedException  { runAt(3, p); }
}
```

`Main.java` starts the three callers 20 ms apart in all 6 arrival orders, so the "wrong" thread really arrives first, collects output in a (synchronized) `StringBuffer`, and joins every thread before reading it.

### Output

```
-- broken version: prints in arrival order --
arrival 3,2,1 -> third second first
arrival 2,3,1 -> second third first
-- correct versions, all 6 arrival orders each --
CountDownLatch    6/6 arrival orders printed 'first second third'
Semaphore(0)      6/6 arrival orders printed 'first second third'
wait/notifyAll    6/6 arrival orders printed 'first second third'
volatile spin     6/6 arrival orders printed 'first second third'
```

### Pitfalls and follow-ups

- **`if` instead of `while`**: with `notifyAll()`, `third` also wakes when `first` finishes although it is not its turn (and spurious wakeups exist).
- **`notify()` instead of `notifyAll()`**: if it wakes `third` while `second` should run, `third` sleeps again and nobody wakes `second` — a hang.
- **A plain flag without `volatile` or a lock**: the JIT may hoist the read out of the spin loop (§15.8). The other versions need no `volatile`: `countDown()` happens-before a successful `await()`, `release()` before a successful `acquire()`.
- **Follow-up: N stages**: `N-1` gates where stage *i* waits on gate *i−1* and opens gate *i*, or the monitor with an integer. **Repeat the sequence**: latches can't be reset, so semaphores or `stage` modulo N — the FooBar and zero-even-odd problems (§22.13).

### TL;DR

Two one-time gates: a `CountDownLatch(1)` or `Semaphore(0)` each is shortest, because both remember the signal; a monitor needs a state variable plus `while`/`notifyAll`; spinning on a `volatile` works but wastes a core.

**Sources:** JDK 21 javadoc for `java.util.concurrent.CountDownLatch`, `Semaphore` and `Object.wait` (spurious wakeups); JLS §17.4.5 (happens-before); `java.util.concurrent` package summary, "Memory Consistency Properties".

---

## 22.8 Token-bucket rate limiter — blocking and background-refill versions

### The problem

A **token bucket** holds up to `capacity` tokens (the burst size) and gains `rate` tokens per second; a caller needs a token before working. Two twists on the yes/no limiter of LLD §13.11:

1. `getToken()` **blocks** until a token is available.
2. Write it twice: **lazy refill** (tokens computed from elapsed time, no extra thread) and a **background daemon thread** that adds tokens — and explain why that thread must not be started in the constructor.

### What the interviewer is testing

- "Refill, check, take" as one **atomic** step.
- Waiting *while releasing the lock* (`wait`/`await`, never `sleep` under the lock), re-checking after waking, interrupts.
- Safe publication and **`this` escape** from a constructor; the trade-off between the two designs.

### Approach

**Lazy refill (`LazyRefillBucket`).**

1. State `tokens` and `lastRefillNanos` (monotonic `nanoTime`), guarded by `this`.
2. `refill()` adds `elapsed / nanosPerToken` whole tokens, capped at `capacity`, and advances `lastRefillNanos` by exactly the time those tokens represent, so the fraction carries over. A full bucket resets the clock on every call, so idle time is never banked (resetting it only when a token is earned would let a bucket that sat full bank up to one period and hand out an extra token early).
3. `synchronized getToken()` refills; if still empty it waits until the next token is due with `TimeUnit.timedWait(this, nanos)` (`wait(ms, ns)`, releasing the monitor), then refills and loops, since another waiter may have won.

**Background refill (`BackgroundRefillBucket`).**

1. A **daemon** thread (§14.2) wakes every `1/rate` s and, under the lock, adds a token if there is room and calls `notify()`.
2. `getToken()`: `while (tokens == 0) wait(); tokens--;`. `notify()` is justified: every waiter wants the same thing and one token serves exactly one.
3. The refiller sleeps to an absolute deadline (`next += nanosPerToken`), so work time doesn't make the schedule drift.
4. **Factory, not constructor**: the constructor creates the `Thread`; a static `start(capacity, rate)` starts it after construction completes; `close()` interrupts and joins it.

Why not `start()` in the constructor? `Thread.start()` publishes everything written *before* it, but fields assigned *after* it, and subclass fields (subclass constructors run later), can be seen unset; the object is reachable before it is built (**`this` escape**, JCIP §3.2.1); and there is no way to create one without a running thread (tests). The factory makes the order impossible to get wrong.

| | **Lazy refill** | **Background thread** |
|---|---|---|
| Threads | none | one per bucket |
| Idle cost | zero | wakes `rate` times a second forever |
| Per-client buckets (thousands to millions) | fine | impossible (a thread each) |
| Precision | exact, computed from time | depends on sleep accuracy |
| Lifecycle | nothing to stop | must `close()`, or the thread lingers |

Lazy refill is the better default (also the LLD §13.11 design and Guava's `RateLimiter` approach); the background version tests thread-lifecycle skills.

### Code

`LazyRefillBucket.java` (core):

```java
private void refill() {                                  // caller holds the lock
    long now = System.nanoTime();
    if (tokens == capacity) { lastRefillNanos = now; return; }   // a full bucket banks no time
    long earned = (now - lastRefillNanos) / nanosPerToken;
    if (earned > 0) {
        tokens = Math.min(capacity, tokens + earned);
        lastRefillNanos += earned * nanosPerToken;       // keep the fractional remainder
        if (tokens == capacity) lastRefillNanos = now;
    }
}

public synchronized void getToken() throws InterruptedException {
    refill();
    while (tokens == 0) {
        long waitNanos = nanosPerToken - (System.nanoTime() - lastRefillNanos);
        if (waitNanos > 0) TimeUnit.NANOSECONDS.timedWait(this, waitNanos);  // releases the monitor
        refill();                                        // re-check: another waiter may have won
    }
    tokens--;
}
```

`BackgroundRefillBucket.java` (lifecycle and refill):

```java
private BackgroundRefillBucket(int capacity, int tokensPerSecond) {
    ...
    this.refiller = new Thread(this::refillLoop, "token-refiller");  // created, NOT started
    this.refiller.setDaemon(true);
}
public static BackgroundRefillBucket start(int capacity, int tokensPerSecond) {
    BackgroundRefillBucket bucket = new BackgroundRefillBucket(capacity, tokensPerSecond);
    bucket.refiller.start();               // only after the constructor has completed
    return bucket;
}
private void refillLoop() {
    long next = System.nanoTime() + nanosPerToken;
    try {
        while (!Thread.currentThread().isInterrupted()) {
            long sleep = next - System.nanoTime();
            if (sleep > 0) TimeUnit.NANOSECONDS.sleep(sleep);   // deadline-based: no drift
            next += nanosPerToken;
            synchronized (this) {
                if (tokens < capacity) { tokens++; notify(); }  // one token -> one waiter
            }
        }
    } catch (InterruptedException e) { /* close() asked us to stop */ }
}
public synchronized void getToken() throws InterruptedException {
    while (tokens == 0) wait();
    tokens--;
}
```

`Main.java`: capacity 3, 10 tokens/s (one per 100 ms); eight threads released together by a `CountDownLatch` record when they got a token.

### Output

```
LazyRefillBucket (no extra thread)
  grant times, rounded to 100 ms: [0, 0, 0, 100, 200, 300, 400, 500]
  first 3 within 50 ms (burst):    true
  8th token not before ~500 ms:    true
  tryGetToken() right after:       false
BackgroundRefillBucket (daemon refiller)
  grant times, rounded to 100 ms: [0, 0, 0, 100, 200, 300, 400, 500]
  first 3 within 50 ms (burst):    true
  8th token not before ~500 ms:    true
  tryGetToken() right after:       false
  after 350 ms idle, tokens available: 3 (capped at capacity 3)
  refiller alive after close(): false
```

Rounding hides a few ms of jitter: a burst of 3, then one grant per 100 ms. The background refiller ticks on its own schedule from `start()`, so on a loaded machine its first refill can land sooner after the burst; the lazy bucket's timing is always relative to the last grant.

### Pitfalls and follow-ups

- **`Thread.sleep()` inside `synchronized`** keeps the monitor: every other caller, and the background refiller, is stuck — a deadlock. Use `wait`/`await`.
- **Fairness**: a monitor has no ordering. For FIFO use `ReentrantLock(true)` + `Condition`, or a fair `Semaphore` the refiller `release()`s (the cap still needs a check under a lock).
- **Missed signal?** No: the refiller changes `tokens` and notifies under the lock, and waiters check `tokens` under the same lock before `wait()`.
- **Shutdown**: after `close()`, a thread in `getToken()` waits forever; add a `closed` flag that makes `getToken()` throw, plus `notifyAll()` in `close()`. A timed `getToken(timeout)` is `wait(remaining)` in a deadline loop.
- **Follow-ups**: `scheduleAtFixedRate` instead of a raw thread handles drift and lifecycle (§14.11); lock-free = an immutable record in an `AtomicReference` updated by CAS (LLD §13.11), which fits `tryGetToken` since blocking still needs park/wait.

### TL;DR

Lazy refill: compute earned tokens from elapsed `nanoTime` in a `synchronized` method and timed-`wait` until the next is due — no extra thread, the default. Background: a daemon refiller plus `notify()`, created in the constructor but started from a static factory so `this` never escapes, with `close()`.

**Sources:** JDK 21 javadoc for `Object.wait(long, int)`, `TimeUnit.timedWait`, `Thread.start`/`setDaemon`, `System.nanoTime`; JLS §17.4.5 (start rule); JLS §17.5 (final-field semantics); B. Goetz et al., *Java Concurrency in Practice* §3.2.1 "Safe construction practices".

---

## 22.9 Unisex bathroom

### The problem

An office has one bathroom shared by men and women, with two rules:

1. **Exclusion:** men and women are never inside at the same time.
2. **Capacity:** at most `N` people are inside at once (3 in the demo).

Each person is a thread that calls `enter(gender)`, uses the bathroom for a while, then calls `leave(gender)`. Write `enter` and `leave`. Then make sure neither gender can be kept out forever.

### What the interviewer is testing

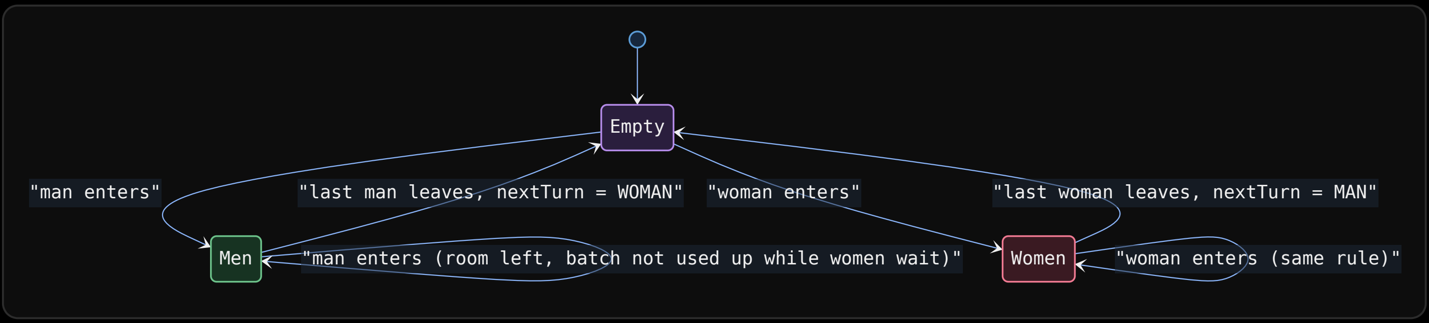
- Recognising a generalised **readers-writers** problem (§22.3): two groups that share with their own kind and exclude the other, plus a capacity.
- Merging the conditions into **one predicate under one lock** instead of stacking primitives (a capacity `Semaphore` plus a gender lock) that check in the wrong order or deadlock.

- **Starvation**: a steady stream of one gender locks the other out. Fix it without losing throughput.

### Approach

1. **State, all guarded by one monitor:** `inside` (how many people are in the room), `insideGender`, `waiting[gender]` (how many of each gender are blocked in `enter`), and two fairness fields described below.
2. **Basic rule**: gender `g` may enter if `inside < capacity` and (`inside == 0` or `insideGender == g`). `leave` decrements `inside` and calls `notifyAll()` — required, because waiters wait for different things (space for a man, an empty room for a woman).
3. **Starvation**: if men keep arriving before the last man leaves, `inside` never reaches 0 and a waiting woman never gets in (almost a full second in the demo).
4. **Fix: batching**, like writer preference in §22.3:
    - Count `admittedThisTurn`, the entries since the room was last empty.
    - Once the *other* gender is waiting, the current gender may admit people only until `capacity` have entered this turn; then newcomers wait, the room drains and the waiting gender goes next.
    - When the room empties with both waiting, `nextTurn` (the gender that did not just finish) breaks the tie.

    Nobody waits behind more than one batch, and a group still fills the room. *Why not stop admitting at once?* Under steady traffic from both sides that degraded to **one person at a time** (`max inside=1` in our first attempt): correct, but no parallelism.
5. **Interrupts**: `waiting[g]--` is in a `finally`, so an interrupted waiter leaves no stale count that would block the other gender forever.


Empty
"man enters"
"last man leaves, nextTurn = WOMAN"
"woman enters"
"last woman leaves, nextTurn = MAN"
Men
"man enters (room left, batch not used up while women wait)"
Women
"woman enters (same rule)"

### Code

`code/ch22/unisexbathroom/Bathroom.java` (core):

```java
private boolean mayEnter(Gender g) {                       // caller holds the monitor
    if (inside == capacity) return false;
    boolean otherWaiting = waiting[g.other().ordinal()] > 0;
    if (inside == 0) return !fair || !otherWaiting || nextTurn == g;
    if (insideGender != g) return false;
    return !fair || !otherWaiting || admittedThisTurn < capacity;  // finish the current batch only
}

public synchronized void enter(Gender g) throws InterruptedException {
    waiting[g.ordinal()]++;
    try {
        while (!mayEnter(g)) wait();
    } finally {
        waiting[g.ordinal()]--;                // also undone on interrupt
        notifyAll();                           // our waiting count changed: others may re-decide
    }
    if (inside == 0) admittedThisTurn = 0;    // a new turn starts
    admittedThisTurn++;
    insideGender = g;
    inside++;
}

public synchronized void leave(Gender g) {
    inside--;
    if (inside == 0) nextTurn = g.other();    // the side that just finished goes to the back
    notifyAll();
}
```

`Main.java`: a **stress** test (6 men, 6 women, 5 visits each, capacity 3; max occupancy and mixed-gender moments) and a **starvation** test (5 men staggered 10 ms apart, 40 ms visits for 1 s; one woman arrives at 100 ms).

### Output

```
stress (fair=false): visits=60, max inside=3, mixed-gender moments=0
stress (fair=true): visits=60, max inside=3, mixed-gender moments=0
starvation test, men stream for 1000 ms, woman arrives at 100 ms:
  unfair: woman waited until the men stopped (>= 800 ms): true
  fair:   woman got in after one batch (<= 100 ms):      true
  [nondeterministic] waits: unfair=952 ms, fair=30 ms
```

### Pitfalls and follow-ups

- **Stacking primitives.** "Take a `Semaphore(3)` permit, then check gender": a man holding a permit while blocked on the gender check keeps women from filling the room — capacity leaks and progress stalls. One predicate avoids it; if you use a semaphore, acquire it *after* the gender check passes.
- **`notify()` is wrong**: waiters wait for different conditions, so it can wake a man when only a woman can proceed — a hang.
- **Fairness within a gender**: the monitor has no arrival order; for strict FIFO use tickets, or `ReentrantLock(true)` with one `Condition` per gender (`menCanEnter`, `womenCanEnter`), which also avoids useless wakeups.
- **Follow-up: K groups**: `insideGroup`, per-group waiting counts and a round-robin `nextTurn`; the predicate keeps its shape.

### TL;DR

One monitor, one predicate (room left, and empty or your gender), `notifyAll` because waiters have different conditions. Batching stops starvation without dropping to one person at a time: once the other side waits, admit at most `capacity` more, drain, hand over the turn.

**Sources:** JDK 21 javadoc for `Object.wait`/`notifyAll`, `ReentrantLock` (fairness parameter), `Condition`; *Java Concurrency in Practice* §14.2 (condition predicates) and §14.2.4 (single vs all notification).

## 22.10 Uber ride (riders of two parties, safe car combinations)

### The problem

Riders of two rival parties (BLUE and RED here) are threads. A car seats exactly **4**: **4 BLUE**, **4 RED** or **2 + 2**, never **3 + 1** (the lone rider would be outnumbered). Every rider calls `seated()`; exactly **one** rider per car, the "captain", calls `drive()` after all four are seated; cars must not mix.

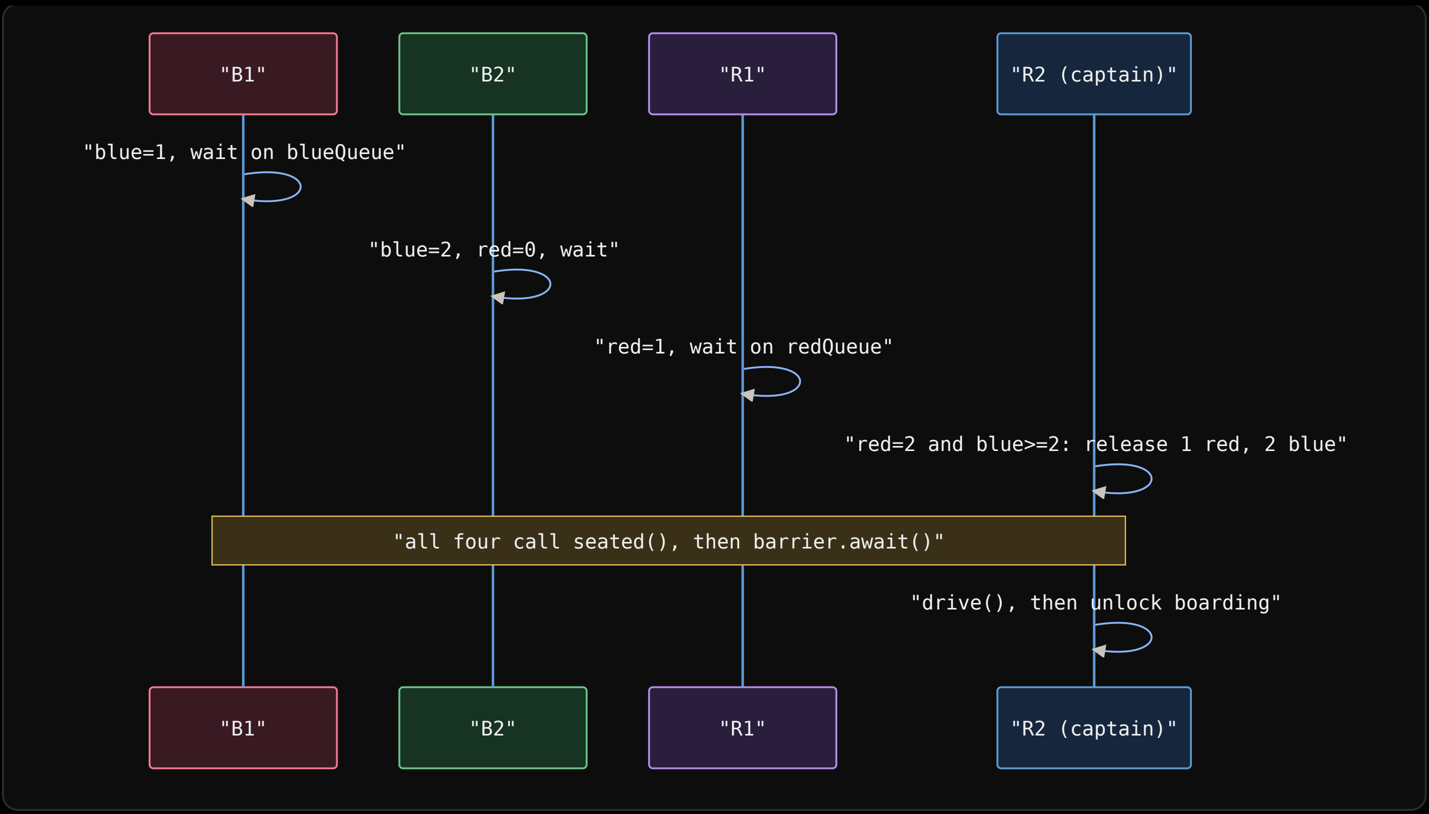
### What the interviewer is testing

- **Grouping**: deciding atomically which waiting threads form a car; **hand-off**: releasing exactly those (per-party semaphores).
- **Rendezvous**: all four seated before anyone drives (a `CyclicBarrier`); exactly one driver; consecutive cars kept apart.

### Approach

1. **One `ReentrantLock` named `boarding`** protects `blueWaiting` and `redWaiting`, the counts of riders who have arrived but have not yet been placed in a car (§14.9).
2. **On arrival**, a rider increments its party's count and checks two rules, in this order:
   - Its party now has 4 waiting: this rider becomes the **captain** and releases 3 of its own party.
   - Its party has exactly 2 waiting and the other party has at least 2: this rider becomes captain and releases 1 of its own party and 2 of the other.

   In either case the captain subtracts the chosen riders from the counts *while still holding the lock*, so no one else can pick them.
3. **Everyone else blocks on their party's `Semaphore(0)`**, which works as a waiting room. The captain's `release(k)` lets exactly *k* of them through. Fair semaphores (`new Semaphore(0, true)`) let riders out in the order they started waiting.
4. **No 3 + 1 car**: riders leave the waiting room only through the two rules, which release 4 + 0 or 2 + 2. A state such as 3 BLUE + 1 RED exists only as *waiting* riders until more arrive.
5. **`CyclicBarrier(4, action)`** (§14.13): all four call `seated()` before anyone moves on; it resets for the next car, and its action (once per car) records the car in the demo.
6. **The captain holds `boarding` until `drive()` returns**, so no new car forms and its riders can't reach the barrier meanwhile — that keeps cars apart. Holding a lock across a blocking call is usually a smell; here it is deliberate and short, and the riders being waited on never need that lock.


"B1"
"B2"
"R1"
"R2 (captain)"
"blue=1, wait on blueQueue"
"blue=2, red=0, wait"
"red=1, wait on redQueue"
"red=2 and blue>=2: release 1 red, 2 blue"
"all four call seated(), then barrier.await()"
"drive(), then unlock boarding"
"B1"
"B2"
"R1"
"R2 (captain)"

### Code

`code/ch22/uberride/UberSeating.java` (core):

```java
public void ride(Party party, String rider) throws InterruptedException, BrokenBarrierException {
    boolean captain = false;
    boarding.lock();
    try {
        if (party == Party.BLUE) blueWaiting++; else redWaiting++;
        int mine = party == Party.BLUE ? blueWaiting : redWaiting;
        int theirs = party == Party.BLUE ? redWaiting : blueWaiting;
        Semaphore myQueue = party == Party.BLUE ? blueQueue : redQueue;
        Semaphore theirQueue = party == Party.BLUE ? redQueue : blueQueue;
        if (mine == 4) {                       // 4 of my party
            myQueue.release(3); take(party, 4, 0); captain = true;
        } else if (mine == 2 && theirs >= 2) { // 2 + 2
            myQueue.release(1); theirQueue.release(2); take(party, 2, 2); captain = true;
        }
    } finally {
        if (!captain) boarding.unlock();      // the captain keeps the lock until it has driven
    }
    try {
        if (!captain) (party == Party.BLUE ? blueQueue : redQueue).acquire();
        car.seated(party, rider);
        allSeated.await();                    // all 4 seated before anyone drives
        if (captain) car.drive(rider);
    } finally {
        if (captain) boarding.unlock();
    }
}
```

`Main.java` runs a scripted arrival order (40 ms apart, so cars are predictable), then 20 BLUE + 20 RED riders in shuffled order with no pauses, checking each car in the barrier action.

### Output

```
arrival order: B1 B2 B3 R1 R2 R3 R4 R5 B4 R6 R7 R8
car 1 seated: [B1, B2, R1, R2]
      driven by R2
car 2 seated: [B3, B4, R3, R4]
      driven by B4
car 3 seated: [R5, R6, R7, R8]
      driven by R8
stress: 40 riders -> cars=10, drive() calls=10, illegal 3+1 cars=0
```

Car 2: when R5 arrives, RED has 3 waiting and BLUE 1 (B3) — a forbidden 3 + 1, so they wait. B4 makes 2 BLUE + 3 RED, a 2 + 2 car forms, and R5 waits for the next car.

### Pitfalls and follow-ups

- **Leftover riders hang** when the totals can't be split into legal cars (e.g. 3 BLUE + 1 RED at the end) — correct for the puzzle; a real system adds `tryAcquire(timeout)` and a policy.
- **Updating counts after unlocking**: a newcomer could count already-chosen riders again and release them twice. Decide and subtract under one lock.
- **Unlocking right after choosing**: the next car's riders could reach `allSeated.await()` first and trip a mixed car. Hold the lock, or use a fresh barrier per car.
- **Interrupts.** A rider interrupted at the barrier breaks it (`BrokenBarrierException` for the other three); the captain's `finally` still unlocks, but the car is lost (`reset()` and retry). Worse, a *chosen* rider interrupted in `acquire()` never reaches the barrier, so the captain waits forever holding `boarding`: production code needs `allSeated.await(timeout, unit)`.
- **Who drives** doesn't matter as long as exactly one does; the rider who formed the car is simplest (the barrier action runs in whichever thread arrives last).

### TL;DR

A lock decides each car (4 + 0 or 2 + 2) and updates the counts atomically; per-party `Semaphore(0)`s release exactly the chosen riders; a `CyclicBarrier(4)` seats all four before the single captain drives, holding the lock until `drive()` returns so cars never mix.

**Sources:** JDK 21 javadoc for `Semaphore` (fairness, `release(int)`), `CyclicBarrier` (barrier action, broken state), `ReentrantLock`.

## 22.11 Sleeping barber

### The problem

One barber, one barber chair, `N` waiting chairs (3 in the demo); customers and the barber are threads (the problem is usually attributed to Dijkstra). With no customers the barber sleeps. A customer who finds every waiting chair taken leaves; otherwise they sit, wake the barber if needed, and wait for **their own** haircut. Service is in arrival order.

### What the interviewer is testing

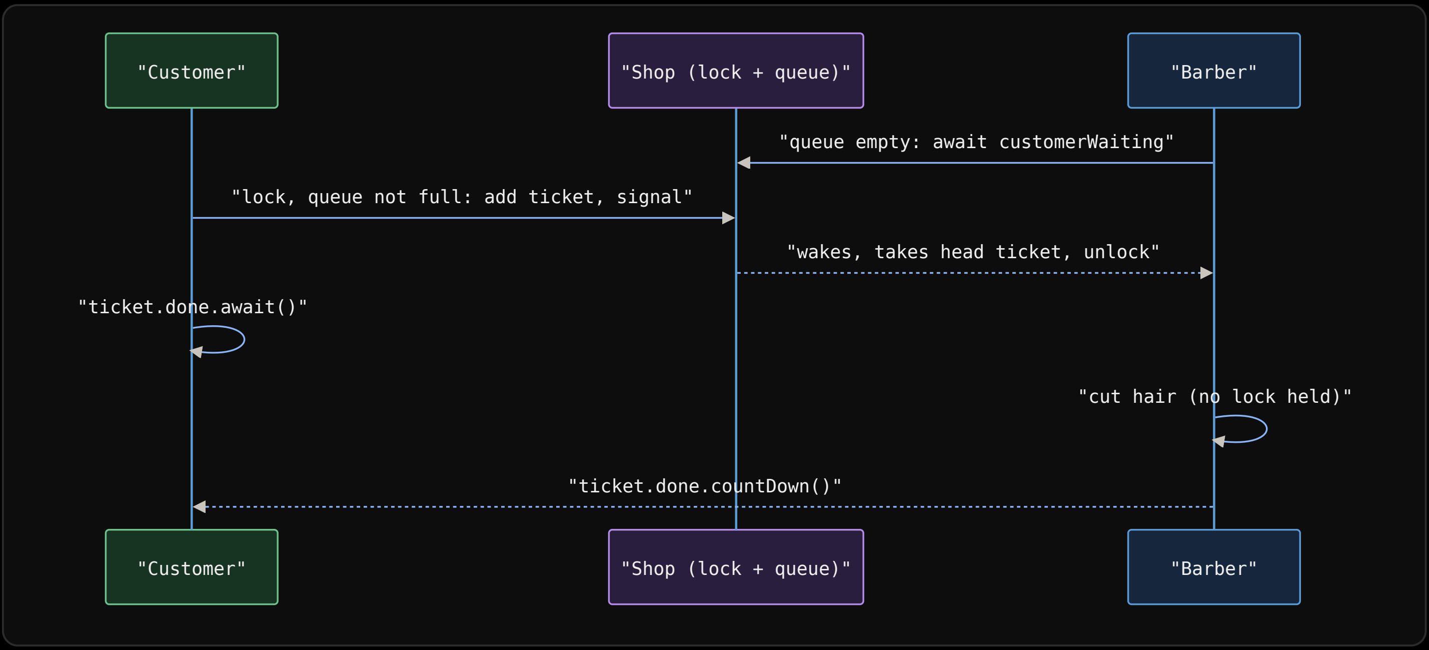
- **Producer-consumer** (§22.1) with a bounded queue that *rejects* when full instead of blocking.
- **No missed wakeup** when a customer arrives just as the barber decides to sleep.
- **Per-customer completion** (*their* haircut, not "some haircut"), slow work **outside** the lock, clean shutdown.

### Approach

1. **Waiting chairs = an `ArrayDeque<Ticket>`** (FIFO) guarded by one `ReentrantLock`.
2. **The barber sleeps on a `Condition`**, `while (chairs.isEmpty() && !closed) customerWaiting.await()`. The check and the sleep happen under the lock a customer needs to enqueue, so nobody slips in between "queue looks empty" and "barber asleep" — the classic lost-wakeup race.

3. **A customer**, under the lock: queue full or shop closed → leave (`false`); else append a `Ticket` and `signal()` (only the barber waits there). Then, without the lock, wait on the ticket's own **`CountDownLatch(1)`**.
4. **The barber** removes the head under the lock (freeing a chair at once), unlocks, cuts, and counts down that ticket's latch, so customers can sit or be turned away during a haircut.
5. **Shutdown**: `close()` sets `closed` and signals; the barber serves those still waiting, then returns.

*The classic semaphore formulation* (a `customers` semaphore the barber sleeps on, a `barberReady` one customers wait on, a mutex for the free-seat count) is equivalent but can't tell *which* customer was served: permits carry no names. Hence tickets with one latch each.


"Customer"
"Shop (lock + queue)"
"Barber"
"queue empty: await customerWaiting"
"lock, queue not full: add ticket, signal"
"wakes, takes head ticket, unlock"
"ticket.done.await()"
"cut hair (no lock held)"
"ticket.done.countDown()"
"Customer"
"Shop (lock + queue)"
"Barber"

### Code

`code/ch22/barbershop/BarberShop.java` (core):

```java
public boolean visit(String customer) throws InterruptedException {
    Ticket ticket = new Ticket(customer, new CountDownLatch(1));
    lock.lock();
    try {
        if (closed || chairs.size() == waitingChairs) return false;   // full: leave at once
        chairs.addLast(ticket);
        customerWaiting.signal();             // only the barber waits on this condition
    } finally { lock.unlock(); }
    ticket.done().await();                    // wait OUTSIDE the lock for my haircut
    return true;
}

public void barberLoop(Consumer<String> cutHair, Runnable onSleep) throws InterruptedException {
    while (true) {
        Ticket next;
        lock.lock();
        try {
            if (chairs.isEmpty() && !closed) onSleep.run();
            while (chairs.isEmpty() && !closed) customerWaiting.await();  // sleep, re-check on wakeup
            if (chairs.isEmpty()) return;     // closed and nobody left
            next = chairs.removeFirst();      // frees a waiting chair now
        } finally { lock.unlock(); }
        cutHair.accept(next.customer());      // slow part, no lock held
        next.done().countDown();
    }
}
```

`Main.java`: haircuts take 230 ms; C1–C8 arrive every 50 ms, C9 at 1.4 s when the barber sleeps again. Events are ≥ 20 ms apart, so on an idle machine the order is the same in every run (under heavy CPU load the margins can slip and C6/C7 may swap fates).

### Output

```
barber: no customers, sleeping
C1 arrives
barber: cutting C1
C2 arrives
C3 arrives
C4 arrives
C5 arrives
C5 leaves: all chairs taken
barber: cutting C2
C6 arrives
C7 arrives
C7 leaves: all chairs taken
C8 arrives
C8 leaves: all chairs taken
barber: cutting C3
barber: cutting C4
barber: cutting C6
barber: no customers, sleeping
C9 arrives
barber: cutting C9
barber: no customers, sleeping
served=6, turned away=3, barber thread ended: true
```

C2 to C4 fill the three chairs while C1 is being served, so C5 is turned away. The barber takes C2 at 230 ms, freeing a chair just in time for C6 at 250 ms.

### Pitfalls and follow-ups

- **Checking a `sleeping` flag without the lock**: a customer sees `false` and doesn't signal just as the barber calls `await()` — he sleeps with a customer in a chair. Check and sleep under one lock, in a `while` (spurious wakeups apply to `Condition` too).
- **Cutting under the lock**: nobody could even be turned away meanwhile.
- **"Done" through a shared semaphore**: any waiting customer may take the permit and leave early; a per-customer latch or `CompletableFuture` ties completion to identity.
- **Production shortcut**: `ArrayBlockingQueue(N)` with `offer()` (fails fast when full) and `take()` (§14.12) — still with the per-customer latch.
- **Follow-ups**: K barbers run the same loop with `signal()` (one arrival wakes one barber; `signalAll()` would be a thundering herd); a customer who gives up calls `done.await(timeout)` and removes its ticket under the lock if still queued.

### TL;DR

A bounded FIFO of tickets under one lock: full → leave, else enqueue and `signal()`. The barber `await()`s in `while (empty && !closed)` under the same lock (no lost wakeup), takes a ticket, cuts without the lock, and counts down that customer's latch.

**Sources:** E. W. Dijkstra, "Cooperating Sequential Processes" (EWD123, 1965), the usual citation for the problem; JDK 21 javadoc for `Condition` (spurious wakeups, `signal`), `ReentrantLock`, `CountDownLatch`, `ArrayBlockingQueue.offer`.

---

## 22.12 Multithreaded merge sort with plain threads (and why Fork/Join is better)

### The problem

Sort a large `int[]` with **merge sort**, using several cores, with plain `Thread` objects. Then explain the problems with that approach and how Fork/Join solves them. (The Fork/Join version on its own is in LLD §13.10 and §15.3. This section is about the raw-thread version and the comparison.)

### What the interviewer is testing

- Why divide and conquer parallelises: the halves touch disjoint ranges, so no locking.
- `join()` as both "wait until done" and the **visibility** guarantee for the child's writes.
- Seeing that a thread per call creates *n − 1* threads, fixing it with a **depth cutoff**, and what Fork/Join adds (fixed pool, **work stealing**, propagated exceptions).

### Approach

1. **Merge step** with one shared scratch buffer: merging `a[lo, mid)` and `a[mid, hi)` copies `a[lo, hi)` into `buf[lo, hi)` and merges back. Each call stays in its own range, so concurrent calls never share cells; `<=` keeps the sort **stable**.
2. **Naive version**: per split, start a thread for the left half, sort the right half in the current thread, `join()`, merge. `join()` makes the child's writes visible (JLS §17.4.5) without `volatile` or locks.
3. **Count the threads**: a binary tree with *n* leaves has *n − 1* internal nodes, so *n − 1* threads — 1,999 OS threads (each with ~1 MB of reserved stack) to sort 2,000 ints that sequential code sorts in microseconds; millions of elements end in `OutOfMemoryError: unable to create native thread`.
4. **Depth cutoff**: threads only for the top `d = ceil(log2(cores))` levels, `Arrays.sort` below: `2^d − 1` threads (3 on 4 cores), all cores busy if the halves cost the same.
5. **Exceptions** in a raw `Thread` go to its uncaught-exception handler; the parent's `join()` returns normally on an unsorted range. The cutoff version captures the `Throwable` and rethrows after `join()`.
6. **Fork/Join** (§15.3): a `RecursiveAction` splits down to 8,192 elements with `invokeAll(left, right)` on a pool of ~#cores workers; idle workers *steal* from busy workers' deques, a waiting worker runs other tasks, and subtask exceptions are rethrown.

| | **Naive thread per split** | **Depth cutoff** | **Fork/Join** |
|---|---|---|---|
| Threads created | *n − 1* | `2^d − 1` | 0 new (shared pool) |
| Uneven work | n/a (too slow anyway) | idle cores while the slowest half finishes | work stealing rebalances |
| Waiting parent | blocked OS thread | blocked OS thread | runs other tasks while waiting |
| Exceptions | lost unless you capture them | must capture and rethrow by hand | propagated by `join` / `invokeAll` |
| Tuning knob | none | `d` (depends on core count) | sequential threshold only |

### Code

`code/ch22/mergesort/ThreadMergeSort.java` (the two raw-thread versions):

```
private static void naive(int[] a, int[] buf, int lo, int hi) throws InterruptedException {
    if (hi - lo < 2) return;
    int mid = (lo + hi) >>> 1;                       // >>> 1: no int overflow for huge lo + hi
    Thread left = new Thread(() -> {
        try { naive(a, buf, lo, mid); } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    });
    left.start();
    naive(a, buf, mid, hi);                          // current thread sorts the right half
    left.join();                                     // the left half's writes are now visible
    merge(a, buf, lo, mid, hi);
}

private static void withCutoff(int[] a, int[] buf, int lo, int hi, int depth) throws InterruptedException {
    if (depth == 0 || hi - lo < 2) { Arrays.sort(a, lo, hi); return; }   // sequential below the cutoff
    int mid = (lo + hi) >>> 1;
    Throwable[] failure = new Throwable[1];
    Thread left = new Thread(() -> {
        try { withCutoff(a, buf, lo, mid, depth - 1); }
        catch (Throwable t) { failure[0] = t; }      // otherwise lost in the child thread
    });
    left.start();
    withCutoff(a, buf, mid, hi, depth - 1);
    left.join();                                     // also publishes failure[0]
    if (failure[0] != null) throw new IllegalStateException("left half failed", failure[0]);
    merge(a, buf, lo, mid, hi);
}
```

The Fork/Join task is in the same file. `Main.java` checks each result against `Arrays.sort`, counts threads created, and reports the best of 5 runs (JIT warm-up).

### Output

```
[machine-dependent] cores=4, cutoff depth=2
2,000 ints:
  naive thread per split   correct=true  threads created=1999   [machine-dependent] 345 ms
  depth cutoff             correct=true  threads created=3      [machine-dependent] 2 ms
4,000,000 ints (best of 5 runs each):
  sequential Arrays.sort   correct=true  threads created=0      [machine-dependent] 272 ms
  depth cutoff             correct=true  threads created=3      [machine-dependent] 130 ms
  Fork/Join                correct=true  threads created=0      [machine-dependent] 140 ms
```

What to notice:

- The naive version is over 100× slower than the cutoff on a tiny input: almost all its time is thread creation and scheduling.
- On a quiet 4-core machine with even halves, the cutoff and Fork/Join are close (100–160 ms across runs), both ~2× sequential because the final merge is single-threaded. Fork/Join's real advantages are no core-count tuning, rebalancing when halves or load are uneven, no blocked OS threads, and propagated exceptions.

### Pitfalls and follow-ups

- **`run()` instead of `start()`** (§14.2): correct but entirely sequential. **Reading before `join()`**: no happens-before edge, so the parent may merge an unsorted or not-yet-visible half.
- **A new buffer per level** works but feeds the GC; one shared buffer is safe because ranges are disjoint.
- **`(lo + hi) / 2` overflows** beyond about a billion elements; `(lo + hi) >>> 1` is always correct.
- **Follow-up: parallel merge.** The final *O(n)* merge is the serial part that caps speed-up (Amdahl, §14.1.3). Split at the median of the larger half, binary-search it in the other, merge both sides recursively; `Arrays.parallelSort` is the production answer.
- **Follow-up: virtual threads?** Cheap to create, but merge sort is CPU-bound and they add no CPU (§15.4); Fork/Join stays the tool.

### TL;DR

A thread per call creates *n* − *1* OS threads: slower than sequential and out of memory on big inputs. A depth cutoff (threads for the top `log2(cores)` levels, `join()` for visibility, exceptions captured by hand) fixes it; Fork/Join adds a fixed pool, work stealing and propagated exceptions; `Arrays.parallelSort` is the one-liner.

**Sources:** JDK 21 javadoc for `Thread.join`, `ForkJoinPool` (work stealing), `RecursiveAction`, `Arrays.parallelSort`; JLS §17.4.5 (join rule); D. Lea, "A Java Fork/Join Framework" (ACM Java Grande 2000).

---

## 22.13 Problems covered elsewhere in this book and in the LLD book

Classic problems with a full write-up and runnable code elsewhere, listed so the set is complete.

| Problem | What it is about | Where it is solved |
|---|---|---|
| **"Superman" problem: thread-safe singleton** | exactly one instance, created lazily, safe under concurrent first calls | Six implementations (eager, synchronized, double-checked locking, holder idiom, enum…): LLD §8.8 Singleton. Why double-checked locking needs `volatile`: §15.8 Java Memory Model |
| **Non-blocking stack** | Treiber stack: `push`/`pop` by compare-and-set on an `AtomicReference` head, the ABA problem | LLD §12.15 CAS and atomics (`LockFreeStack`) |
| **Dining philosophers** | deadlock from circular waiting; fixes by global lock ordering, a limited waiter (`Semaphore(n−1)`), or `tryLock` | LLD §13.5 |
| **Print FooBar alternately** | two threads alternate *n* times (reusable signalling, unlike the one-time gates of §22.7) | LLD §13.1 |
| **Zero, even, odd** | three threads print `0102030…` | LLD §13.2 |
| **FizzBuzz multithreaded** | four threads, each handling one class of numbers, in strict order | LLD §13.3 |
| **Build H2O** | group threads into molecules of 2 H + 1 O, the same grouping idea as §22.10 | LLD §13.4 |